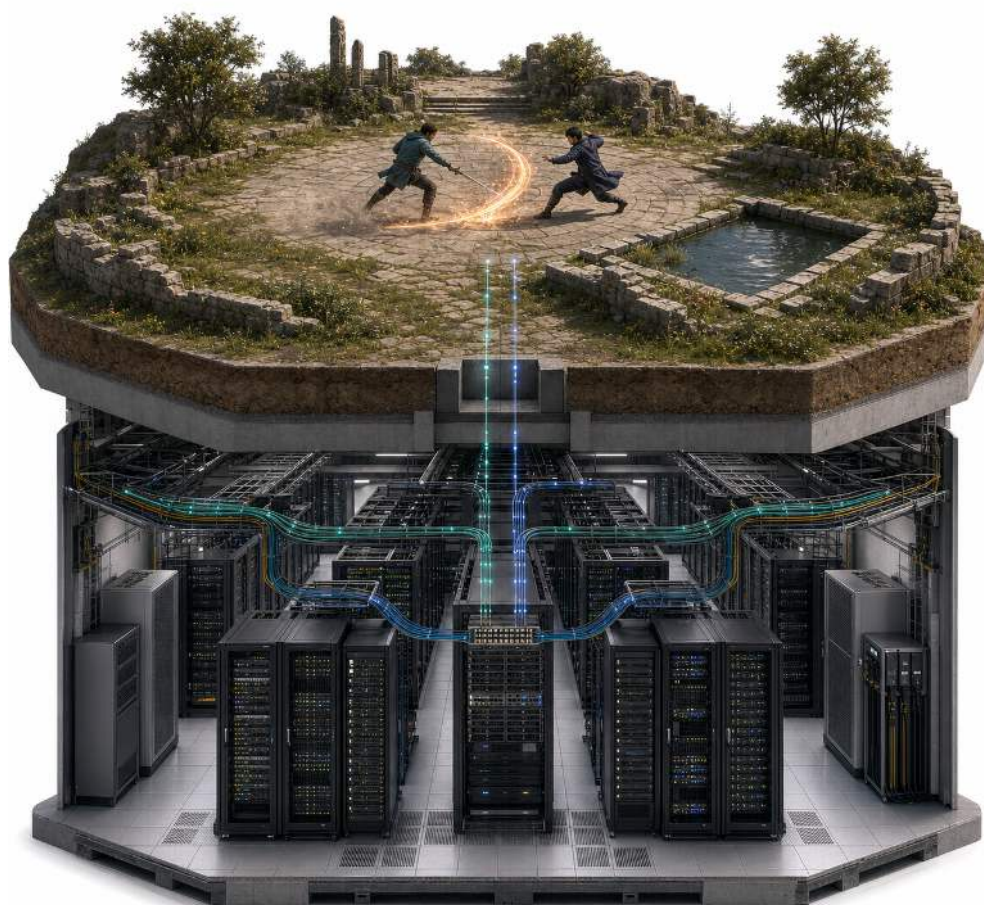




云计算原理与实践：以在线游戏为载体



目录

前言	7
1 第一章 谋定全局：在线系统与云计算	11
1.1 云计算的重要性	11
1.2 在线系统的通用架构	12
1.3 架构设计的评价目标	16
1.3.1 性能：控制延迟并提高有效吞吐	17
1.3.2 状态正确性：明确状态权威与恢复条件	17
1.3.3 可扩展性：让容量随压力变化	18
1.3.4 可用性：限定故障影响并恢复服务	19
1.3.5 成本：约束目标的实现方式	19
1.4 从双人对战到云平台：架构演进的四个阶段	20
1.4.1 阶段一：建立最小在线闭环	20
1.4.2 阶段二：提高单机并发能力	21
1.4.3 阶段三：拆分到多台机器	22
1.4.4 阶段四：交给平台持续治理	23
1.5 从逻辑架构到云基础设施	24
1.6 小结	26
1.7 思考题	27
2 第二章 双雄对战：网络通信	28
2.1 从单机循环到网络通信	29
2.2 网络通信基础：建立传输通道与消息边界	31
2.2.1 建连：从地址端口到传输通道	31
2.2.2 消息边界：如何在字节流中恢复完整消息	33
2.3 P2P 直连原型：输入同步与状态一致	36
2.3.1 锁步输入同步：机制与成立条件	36
2.3.2 P2P 的局限：等待、信任与连接规模	39
2.4 CS 架构与权威服务器设计	39
2.4.1 从 P2P 到权威服务器	39
2.4.2 连接管理：会话生命周期维护与读写调度	42
2.4.3 状态仲裁：一致性防线与权威裁决	45
2.4.4 状态分发与副本收敛	48
2.4.5 三模块协同：贯通服务器处理路径	51
2.5 从可靠字节流到可确认的业务结果	51
2.5.1 小消息何时发出：Nagle 算法与时延取舍	52

2.5.2	写入成功能确认什么：从本机缓冲到业务生效	53
2.5.3	读取应等待多久：截止时间与超时处理	53
2.5.4	连接是否仍然可用：半开连接与健康检测	55
2.5.5	业务是否生效：应用层确认与幂等处理	57
2.5.6	通信逻辑如何复用：连接通道封装	59
2.5.7	前序数据丢失会怎样：队头阻塞	61
2.6	从按序等待到独立交付：TCP 与 UDP 的取舍	61
2.7	最小在线交互的完整流程	63
2.7.1	相遇与会话建立：从可见交互到权威确认	63
2.7.2	结果裁决与结算：从规则校验到一致性写入	64
2.7.3	闭环全景与跨场景迁移	65
2.8	小结：网络通信与共享状态的最小闭环	66
2.9	思考题	67
3	第三章 英雄集结：单机并发	68
3.1	从顺序处理到并发组织	68
3.1.1	顺序处理如何放大延迟	69
3.1.2	操作系统的抽象：进程与线程	72
3.1.3	用 goroutine 组织等待型任务	74
3.1.4	事件驱动与 I/O 多路复用：管理大量连接与等待	77
3.1.5	请求处理链：串联进程、线程、协程与异步通信	83
3.2	并发正确性：保护共享状态	86
3.2.1	共享资源竞争：临界区与竞态条件	88
3.2.2	互斥锁：把临界区收敛成唯一顺序	90
3.2.3	等待与通知：从忙等到条件变量	95
3.2.4	锁的粒度：性能与复杂度的折中设计	99
3.3	单机资源优化与性能验证	102
3.3.1	外部资源连接池：从短连接到连接复用	102
3.3.2	对象池：复用短命对象以缓解 GC 压力	105
3.3.3	缓存分层架构：热数据与冷数据的协同治理	106
3.3.4	性能验证：用可观测指标量化优化效果	110
3.4	小结：单机并发的机制与限制	112
3.5	思考题	113
4	第四章 切分世界：分布式系统	115
4.1	多机扩展的两个拆分维度：职责与对象	116
4.2	均衡：分片与动态调整	117
4.2.1	分片键与基本映射策略	117

4.2.2	一致性哈希：限制节点变化的影响范围	121
4.2.3	分片路由	126
4.2.4	负载监测与入口调度	129
4.2.5	动态调整：扩缩容与热迁移	133
4.2.6	存储分片：从单库瓶颈到跨分片访问	138
4.2.7	缓存层的分布式演进：从单机 Redis 到集群	148
4.3	协同：跨服交互与事务	151
4.3.1	跨服交互的三类场景与约束	151
4.3.2	通知类：事件如何传播	152
4.3.3	查询类：分散状态如何汇成一致视图	157
4.3.4	事务类：跨节点事务如何保证一致	159
4.4	容错：冗余、复制与一致性	161
4.4.1	主从复制与故障切换	163
4.4.2	备份与恢复演练	167
4.4.3	从全写到多数派：Raft 共识	169
4.4.4	CAP 定理与一致性取舍	176
4.5	小结：均衡、协同与容错	178
4.6	思考题	179
5	第五章 飞升入定：云原生部署	181
5.1	容器：上云的虚拟化基础	181
5.1.1	为什么需要容器：环境一致性	181
5.1.2	容器的基本原理与使用方法	183
5.1.3	构建、分发与运行的容器化部署闭环	187
5.2	编排：云上多服务互连与调度的基础	192
5.2.1	编排是什么：从手工命令到声明式拓扑	192
5.2.2	单机多容器编排：Docker Compose	192
5.2.3	从单机到多机：跨机网络、服务发现与集中控制	195
5.2.4	Kubernetes 对象、调度与控制面	196
5.2.5	资源调度：放置决策与资源约束	202
5.2.6	Kubernetes 的控制面与协作链路	204
5.2.7	声明式管理与调谐循环	206
5.2.8	调谐循环的运行时行为	207
5.3	弹性运行：伸缩与自愈	209
5.3.1	伸缩调度：固定容量与波动负载的矛盾	210
5.3.2	应用层自动扩缩：HPA 与 VPA	211
5.3.3	安全缩容：为什么撤掉容量更危险	213

5.3.4	基础设施层自动扩缩：Cluster Autoscaler	214
5.3.5	函数粒度的伸缩调度：Serverless	214
5.3.6	故障自愈：检测、修复与约束	221
5.4	小结：从标准化交付到弹性运行	223
5.5	思考题	224
6	第六章 穷理尽微：云原生核心原理	226
6.1	容器的本质：操作系统级虚拟化的三块基石	226
6.1.1	Namespaces：让进程拥有“独立的系统视图”	226
6.1.2	Cgroups：把资源使用变成“可计量、可限额、可统计”的对象	229
6.1.3	rootfs 与 OverlayFS：镜像分层与容器写入的机制	231
6.1.4	容器的安全边界：共享内核带来的隔离限度	233
6.2	网络虚拟化原理：从隔离的网络命名空间到互通的容器网络	234
6.2.1	容器的第一块网络接口：veth 对与命名空间互联	234
6.2.2	同宿主机容器互联与 bridge 组网	235
6.2.3	容器访问外网：默认网关、SNAT 与连接跟踪	237
6.2.4	外部访问容器：端口映射与 DNAT	238
6.2.5	跨宿主机的 Pod 互通：Overlay 网络与 VXLAN 封装	240
6.2.6	低开销的容器网络：Overlay 封装的代价与前沿优化	242
6.2.7	小结：网络虚拟化的层次结构	244
6.3	弹性伸缩的应用基础：无状态与有状态	245
6.3.1	无状态与可替换实例：安全水平伸缩的六项条件	245
6.3.2	有状态服务：状态归属决定伸缩方式	247
6.3.3	案例：游戏服务端的状态架构与弹性伸缩	248
6.3.4	应用与平台的职责划分	250
6.4	弹性调度的内部机制：从反馈控制到按需执行	251
6.4.1	调度对象与分层	251
6.4.2	反馈闭环的控制要素	251
6.4.3	HPA 在做什么：闭环的工程落地	252
6.4.4	简化 HPA 控制器：用代码表达反馈闭环	254
6.4.5	HPA 的边界：副本控制器不能解决所有问题	255
6.4.6	分层控制：从副本决策到节点放置	256
6.4.7	多目标调度	257
6.4.8	Serverless 引擎：把副本管理继续下沉	258
6.4.9	降低冷启动延迟：容器快速启动的两条研究路线	262
6.5	面向 Agent 的高并发 Sandbox：从一次执行到经验生产	263
6.5.1	总体架构：先分清任务状态、资源状态和轨迹状态	265

6.5.2	控制面：维护全局秩序	268
6.5.3	执行面：环境保真度与资源密度的取舍	272
6.5.4	交互面：分离控制路径与高频数据路径	274
6.5.5	状态面：把回滚和分叉放进训练循环	277
6.5.6	验证面：让轨迹能被复现、筛选和归因	280
6.5.7	小结：Agent sandbox 的五个面向	284
6.6	小结：从基石机制到系统工程方法	284
6.6.1	未来展望：云原生的下一站	285
6.6.2	写给读者的寄语：不畏浮云遮望眼	286
6.7	思考题	286

附录

A 章节重难点知识点索引	288
B 配套实验与开源代码	303
B.1 实验一：双人对战的网络闭环	303
B.2 实验二：多人世界中的并发控制	304
B.3 实验三：多地图与多节点协同	304
B.4 实验四：Kubernetes 上的部署与弹性运行	305

前言

这些年在课堂上，学生常问我：“云计算究竟要学到什么程度，才算真正入门？”产生这种困惑并不奇怪。云计算涉及的技术层次多，Docker、Kubernetes、Raft、Serverless、eBPF 等概念又不断出现。面对越来越长的学习清单，学生往往难以判断哪些内容必须掌握，也不知道自己是否真正理解了云计算。

我认为学习云计算的关键，不是继续积累技术名词，而是建立一条由真实问题驱动的学习线索。沿着这条线索，就可以理解一项技术为什么出现、受到哪些约束、解决了什么问题，又付出了什么代价。本书希望帮助读者建立的，正是这样一条从问题走向机制、再回到工程取舍的线索。

为什么写这本书

要建立这条线索，只掌握概念或只熟悉操作都不够。读者常见的云计算学习材料大致从两个方向展开。一类侧重概念体系，帮助读者理解 IaaS、PaaS、SaaS、CAP 等基本概念；另一类侧重平台操作，通过控制台、配置文件和部署步骤帮助读者快速上手。前者便于建立知识框架，后者便于熟悉具体工具，但从理解概念到独立分析和构建一个云计算系统，中间仍有一段路需要补足。

云计算的难点通常不是某一个知识点格外陌生，而是许多已经学过的知识会同时作用于一个系统。学过 TCP 三次握手，不等于知道大量长连接到来后为什么会排队；学过进程、线程和锁，不等于知道并发为什么既能提高吞吐，也可能破坏共享状态；了解一致性与可用性的概念，也不等于能够判断一次跨节点操作应当怎样确认结果。网络、操作系统、数据结构和分布式系统原本分散在不同课程中，进入真实在线系统后却必须共同工作。

本书位于这些基础课程与复杂系统实践之间。它从系统中已经出现的问题出发，分析问题受到哪些约束，引入相应的机制，再通过实现和运行结果检验设计是否成立。读者需要掌握的不只是某种工具的用法，还包括机制的适用条件，以及性能、状态正确性、可扩展性、可用性和成本之间的取舍。

要把这些内容讲清楚，需要一个能够观察、实现和验证的教学载体。这些年，我先后尝试过选课系统和大模型服务系统。选课系统贴近生活，但问题主要集中在数据访问和名额分配，难以自然覆盖跨节点协作、故障恢复和弹性伸缩；大模型服务涉及队列调度、资源管理和横向扩展，但依赖 GPU，推理过程也具有较强的黑盒属性，不适合所有学生完整实现。

经过比较，我们最终选择了在线游戏。游戏服务既包含长连接、并发处理、共享状态和跨节点协作，也会遇到部署、故障和弹性伸缩问题；更重要的是，它的运行过程足够透明。一次操作从客户端发出，经过网络传输、服务器校验、状态更新和结果同步，每一步都可以通过代码实现，也可以通过实验观察。学生可以运行配套系统，并直接感受延迟、状态冲突和故障对服务的影响。

从选课系统到大模型服务，再到在线游戏，这条教学路线经过了近十年的调整和打磨。本书是这段课堂实践的阶段性总结。

这本书有什么不一样

第一，以持续演进的在线系统引出技术。

全书先建立在线系统的整体视图，再从最小的双人在线交互出发，推动案例随着规模和压力逐步演进：建立网络闭环后，处理单机并发；单机容量接近上限后，将系统拆分到多台机器；服务和实例难以人工管理后，再引入容器、编排和弹性机制；最后进入基础设施内部，解释容器隔离、网络虚拟化和弹性调度如何实现。

每一阶段都承接前一阶段已经暴露的问题。读者由此看到的不是一组并列的技术名词，而是问题、约束、机制、取舍和新问题之间的因果关系。

第二，把分散的知识组织成完整的系统判断。

设计消息协议时，TCP 和 UDP 不再只是网络课程中的定义；组织并发时，进程、线程、goroutine、锁和 I/O 多路复用需要落实到真实代码；拆分系统时，哈希、分片、复制、事务和共识会共同决定请求与状态应当由谁处理和保存。

在线游戏是本书的主要教学案例，但不限定知识范围。正文还会引入在线协作、电商平台和大模型服务等系统，说明传输、并发、一致性、容器和调度机制在不同场景中的作用与适用条件。业务形式可以变化，规模、状态、故障和资源问题具有共通性。

第三，使原理、实现和验证相互对应。

正文解释机制及其适用条件，配套案例提供可运行的代码，四组实验则围绕同一个游戏系统逐步扩大规模。读者需要观察吞吐、延迟、资源占用和错误现象，修改其中一个机制，再用运行结果判断修改是否有效。

最后一章还将这些方法延伸到需要大量隔离执行环境的 Agent Sandbox。当平台管理的不再只是服务副本，还包括需要暂停、恢复、回滚和复现的执行过程时，云计算中的隔离、网络、调度、状态和验证机制会以新的方式组合。这个问题既是对全书知识的综合应用，也为希望继续深入的读者提供了进入系统研究的起点。

本书可以用于云计算和系统工程类课程，也可以作为课程设计、综合实践以及进一步学习云原生系统的基础。

这本书如何学

阅读本书不要求预先掌握云计算或分布式系统，但需要具备基本的编程能力，并了解常见的数据结构。如果已经学过计算机网络和操作系统，阅读会更加顺畅；即使部分知识已经遗忘，也不必先把所有先修课程重新学习一遍，可以跟随案例，在真正需要时重新理解它们。

全书六章构成一条连续的学习路线：

- **第一章“谋定全局：在线系统与云计算”** 建立在线系统的整体视图，并从性能、状态正确性、可扩展性、可用性和成本五个维度评价架构；
- **第二章“双雄对战：网络通信”** 建立最小网络闭环，讨论通信、消息组织、状态同步和结果确认；

- 第三章“英雄集结：单机并发”进入单机并发，处理执行组织、共享状态保护和资源复用；
- 第四章“切分世界：分布式系统”将系统扩展到多台机器，围绕均衡、协同和容错展开；
- 第五章“飞升入定：云原生部署”将系统交给集群平台，讨论容器化交付、编排、弹性和自愈；
- 第六章“穷理尽微：云原生核心原理”继续下沉到容器、网络和弹性调度的内部机制，并探索 Agent Sandbox。

学习本书时，有三点需要注意。

第一，动手。配套程序应当真正运行起来。只有观察到连接排队、状态冲突、尾延迟上升或扩容滞后，书中的机制才会从文字变成可以分析的系统行为。

第二，追问。读到一个解决方案时，可以先思考自己会怎样处理当前问题，再把自己的方案与书中的实现进行比较。两者之间的差异，往往能够揭示此前忽略的条件和代价。

第三，善用 AI，但不依赖 AI。AI 可以帮助解释概念、检查代码和讨论替代方案，但不能代替对代码、运行现象和实验结果的判断。能够辨别一个答案是否符合系统的实际约束，本身就是本书希望培养的能力。

对于采用本书授课的教师，本书可以按一学期组织教学。四组实验分别对应网络闭环、单机并发、多节点协作，以及 Kubernetes 上的部署与弹性运行。课时有限时，可以将第六章的拓展内容压缩为导览，但应为前面的系统演进和动手实验保留足够时间。

配套代码、实验材料、授课视频和 PPT 将在 Gitee 仓库持续更新 (<https://gitee.com/hnu-cloudcomputing>)。实践时应使用相互对应的书稿和代码版本，避免接口或环境差异影响实验结果。

云计算的水面之上，新技术来了又去；水面之下，进程、网络、状态、调度和故障这些基本问题长期存在。理解这些问题，才能在工具变化时保持判断。愿读者既能深入机制，也能回到系统全局。

致谢

本书由陈果、徐方林负责编写。各章初稿撰写分工如下：胡文举（第 1 章）、庞海鑫（第 2 章）、谢先衍（第 3 章）、贺臻（第 4 章）、张道平（第 5 章）、徐方林（第 6 章）。胡文举承担了课程实验内容的验证与复核，并参与指导了书中配图的视觉修订工作；终稿由徐方林统一核定。

实验案例的设计与实现离不开历届学生的参与。陈俊杰、王煌等同学在游戏案例代码、实验文档和测试用例方面做了大量工作。课程视频的制作同样有赖于学生团队的协助，刘筱芊等同学参与了授课视频的录制、剪辑与后期整理。

出版社的编辑老师们为本书的出版同样付出了大量心血，在此一并感谢。

受知识水平和时间精力所限，本书难免存在错误和疏漏。欢迎读者将发现的问题反馈至邮箱 guochen@hnu.edu.cn，我们将持续改进本书的质量。

陈果

2026 年 7 月于长沙

1 第一章 谋定全局：在线系统与云计算

长期运行、通过网络为多个用户提供服务的软件系统，称为在线系统（Online System）。与本地程序不同，在线系统不会在处理完一次输入后就退出，它需要在多次请求之间维持共享状态，并协调不同用户的操作结果。

网页浏览、网上交易、在线协作和网络游戏都属于在线系统，但它们面对的首要困难并不相同，复杂度也逐步加深。浏览网页时，用户最关心服务能否随时访问、内容能否及时返回，这首先是可用与性能的问题。完成网上交易时，除了及时返回，还要保证订单和支付结果准确、不重复、不丢失，这进一步提出了状态正确性的要求。多人共同编辑文档时，几个人可能同时修改同一份内容，这些同时发生的修改称为并发；系统需要协调它们并把结果同步给所有参与者，由此引出并发控制问题。在网络游戏中，服务端不仅要每个玩家的操作作出实时裁决，还要持续维护所有玩家共享的世界状态，对状态、实时性和并发规模同时提出了高要求。

尽管场景不同，这些系统面临的根本挑战是一致的。首先，系统要持续运行，不能因为一次任务结束就退出。其次，系统要同时服务多个用户，而用户数量和请求量随时可能变化。再者，系统要在多用户之间维护共享状态，确保修改有序、结果可靠。此外，随着规模扩大，机器和网络出现局部故障会成为常态，系统需要在故障下继续提供服务。

这些挑战带来了一组相互关联的问题：请求从哪里进入、由谁处理；共享状态由谁确认、保存在哪里；资源不足时怎样扩展；组件失效后怎样恢复。把这些职责分配给不同的组件，并规定它们之间如何配合，就是在线系统的架构设计。

本章沿着资源基础、逻辑结构、评价标准、架构演进和物理落地的路线展开。首先说明云计算为什么是在线系统的资源基础，没有可按需调整的计算、存储和网络资源，架构设计就无从谈起。在此之上，我们建立在线系统的通用逻辑结构，说明一次请求通常经过哪些环节、共享状态如何维护。职责明确后，再从性能、状态正确性、可扩展性、可用性和成本五个维度建立评价标准。随后可以看到，当规模、负载和运维条件变化时，架构会沿着一条可辨识的路径逐步演进，从最小的单机闭环到单机并发，再到多机协作和平台治理。最后，我们把这些逻辑职责落回到数据中心的真实物理资源上，看看它们如何由具体的硬件和设施支撑。

学完本章，读者将获得一套可以迁移到不同在线系统的分析方法：沿一次请求的处理过程识别系统中的各项职责及其相互关系，判断一次架构调整解决了什么问题、依赖哪些条件、付出了什么代价，并用性能、状态正确性、可扩展性、可用性和成本五个维度评价设计的优劣。

1.1 云计算的重要性

在线系统一旦投入运行，就要长期面对两类不断升级的压力。

第一类压力来自请求与共享状态的交互，这是任何在线系统从开始搭建就会遇到的基本问题。大量用户同时访问时，请求不会按顺序排队到达，也不会以同样的速度完成；如果逐一来处理，快请求会被慢请求拖住，系统响应就会整体变慢。更棘手的是多个请求同时读取或修改同一份数据，比如两个用户同时下单同一件商品、两个玩家同时捡起同一个道具。若不协调它们的先后顺序，两个操作可能都读到修改前的数据而同时成功，造成超卖或道具被重复拾取。

第二类压力来自规模与运行条件的变化，它会让第一类问题进一步放大和复杂化。在线服务的用户数量和请求强度通常不是恒定的，白天高、深夜低，活动期间还可能出现突发高峰。长期按峰值准备机器会造成大量闲置，只按平均水平配置又难以承受突发流量，系统需要根据实际需求分配和回收容量，同时保留足够的余量。而当机器数量增长到一定程度，单台机器、单条网络链路甚至单个程序进程的失效都会成为常态，部署过程也可能因环境差异和操作顺序出错。系统拆到多台机器后，同一份数据还可能存在多个副本，本来就棘手的状态正确性问题，现在要跨节点解决。这时再依靠人工逐台处理故障、逐次完成发布，就会越来越难以及时响应。系统需要把交付、监测、故障实例替换和服务恢复组织成可自动重复执行的流程，用自动化替代人工判断和手动操作，才能在规模扩大后保持响应速度和操作一致性。

如果应用固定运行在少数几台机器上，部署、扩容和故障恢复又主要依赖人工，就很难同时应对这些压力。要在持续变化的条件下保持服务质量，系统需要一种新的资源组织方式。云计算的核心作用，是把计算、存储和网络资源组织成可申请、可度量、可调度和可回收的资源池。应用通过虚拟机或容器获得计算资源，通过存储服务和虚拟网络使用存储与网络资源，不必绑定某一台固定机器。平台根据需求安排资源的位置和数量，并在负载或运行状态发生变化时动态调整分配。例如，活动高峰到来时，平台可以为登录服务额外调集若干台机器承接突发流量；高峰过去后，这些机器又被回收到资源池，留给其他服务复用。

不过，资源能够由平台调度，并不表示应用能够随之迁移或扩展。如果程序、配置和状态仍然依赖某一台固定机器的本地环境，平台即使拥有空闲资源，也难以安全地迁移或扩展服务。这就要求应用本身也要**按照可复制、可调度和可恢复的方式组织**，也就是云原生的思路。服务被封装为标准化交付单元，配置和状态按照明确规则管理，平台据此持续完成部署、扩缩容、故障恢复和运行观测。简单说，在线系统描述的是需要长期提供的服务，云原生描述的是这些服务如何利用云平台进行组织和治理。

归根结底，**云计算提供的是可调整的资源 and 可自动执行的治理能力**，但它不决定系统内部的请求路径和状态维护方式。要把可调度的资源转化为可持续提供的服务，还需要明确一次请求经过哪些环节，各项职责由谁承担，以及系统怎样判断和调整运行状态。下一节据此建立在线系统的通用架构。

1.2 在线系统的通用架构

以在线协作文档为例，用户在编辑器里输入一段文字后，编辑器把这次修改连同它在文档中的位置打包成一条请求，发送给服务端；服务端校验并应用这次修改，再把更新后的内容同步给所有正在编辑的人。网页下单、游戏中发出操作等其他交互也都遵循同样的过程：客户端把用户的动作变成请求发给服务端，再等待处理结果。要解释这次交互如何完成，首先需要明确客户端与服务端分别承担什么职责。客户端负责采集用户输入并呈现结果，服务端负责校验请求、执行规则并维护共享状态。客户端可以通过缓存或预测改善交互体验，但不能绕过服务端直接决定其他参与者都要接受的结果。这种职责划分构成客户端/服务器（Client/Server，C/S）架构的基本关系。服务端内部还要继续划分，请求处理、状态维护和运行治理分别由不同部分承担。

只看服务端处理一次业务请求的过程，在线系统似乎只需要接收请求、执行计算并保存状态。要让这条处理路径持续可靠运行，系统还需要部署和调整服务实例，在故障发生时恢复服务，并持续判断各组件的运行状态。这些运行治理工作与业务处理一样，都需要计算、存储和网络资源提供支撑。要同时看清请求处理、运行治理和基础设施三部分之间的关系，用一张分层结构图会更清楚。

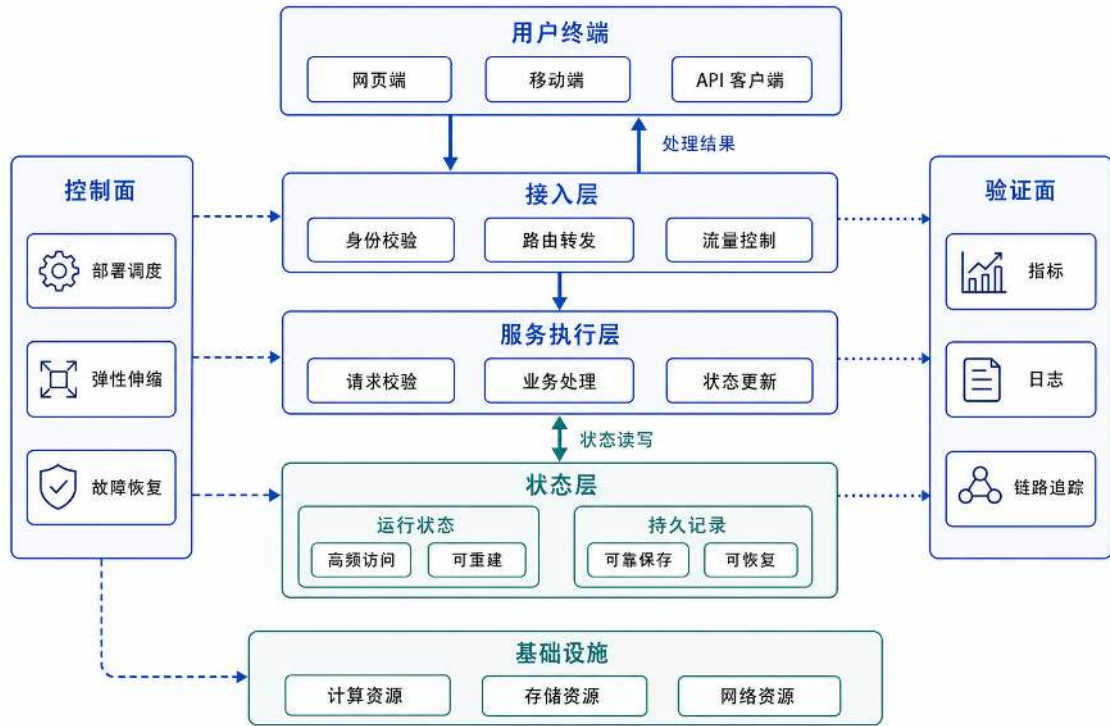


图 1: 在线系统的业务处理与运行治理：主要请求路径完成处理和状态更新，控制面与验证面维持系统运行，基础设施提供所需资源。

图 1 把系统职责组织为三个层面：沿请求路径依次排列的接入层、服务执行层和状态层，负责完成一次请求的处理和状态更新；位于两侧的控制面和验证面，分别负责调整运行方式和记录运行证据；底部的基础设施则为所有活动提供计算、存储和网络资源。下面先沿请求处理路径说明接入、执行和状态三类职责，再讨论控制面与验证面如何在负载变化和组件故障时维持这条路径。这两部分合在一起，构成在线系统通用架构的基本框架。

接入层：建立稳定且受控的系统入口 外部请求到达系统时，面对的往往是多个并行业务实例，而不是单个处理进程。以游戏为例，玩家的一次操作可能由多个战斗服务器中的某一个处理。如果客户端绕过统一入口、自己直连某个实例，这个实例就难以确认玩家身份和会话，系统也无法汇总各实例的剩余容量。更麻烦的是，突发请求可能集中涌向同一个实例，使其迅速过载。因此，系统需要在请求与业务实例之间设置一个统一入口，先判断请求是否可以进入、应该交给哪个实例、当前容量能否继续接收请求，这些职责共同构成接入层。

具体来说，接入层负责接收连接和请求，识别访问者，把请求路由到合适的服务实例，并在

系统过载或请求异常时实施限流、超时和拒绝策略。网络游戏、即时通讯属于长连接系统，客户端与服务端长时间保持连接并持续收发数据，接入层还要维护连接的生命周期；网页搜索、电商下单属于短请求系统，请求到达后很快返回，接入层则更关注请求转发、负载分配和连接复用。

接入层负责的是用户请求如何进入系统，而不是完成最终的业务处理。如果让每个业务服务各自实现身份识别、路由和限流，既重复又容易不一致；把这些职责集中到接入层统一处理，业务服务就能专注完成各自的业务逻辑。

经过接入层，外部请求被转换为身份已经识别、资源使用受到约束且去向明确的内部请求。请求能够进入系统以后，下一步才是判断它应当触发什么计算和状态变化。

服务执行层：校验请求并产生状态变化 请求进入内部后，由服务执行层完成规则校验、业务计算和结果生成。以玩家发动一次攻击为例，服务先校验请求是否合理：玩家是否拥有该技能、冷却是否结束、法力值是否足够；通过后再计算伤害和命中，并据此扣减法力值与目标生命值。如果校验和计算交给客户端自己完成，恶意玩家就能绕过限制，在法力不足时强行发动技能或篡改伤害数值。因此，只有经过服务端校验的请求才能改变共享状态，客户端提交的只是操作意图，最终结果由服务端确认。

服务执行层的设计重点是确定这些计算职责如何组合。频繁访问同一份状态的操作适合放在一起，例如伤害计算和法力扣减都要读写角色状态，合并可以避免反复跨服务取数据。资源消耗、扩展速度或故障影响差异较大的职责则可以拆开，例如战斗和聊天：前者对 CPU 要求高且需要低延迟，后者占用带宽但能容忍一定延迟，扩容节奏也不同，拆分后能各自独立扩展、故障互不牵连。不过拆分也有代价，原本一次函数调用就能完成的事，拆开就要跨网络通信并协调不同服务上的状态，因此是否拆分取决于职责差异是否大到值得付出这些成本，而不是为了形式上多分几层。

服务执行层由此把已经接入的请求转化为经过校验的处理结果和状态更新。接下来还要确定这些状态保存在哪里、保留多久，以及发生故障后如何恢复。

状态层：区分运行状态与持久记录 状态层负责保存服务执行产生的数据，但不同数据对访问速度和故障后保留的要求并不相同。会话上下文、临时计算结果和高频更新数据需要频繁读写，通常保存在内存或缓存中。账号、订单、资产和审计记录一旦丢失就难以重建，需要写入能够在进程或机器故障后保留数据的持久化存储。

如果所有更新都等待持久化存储确认，请求处理可能因写入延迟而变慢。如果所有数据都只保存在内存中，进程重启或退出后关键记录又会丢失。因此，设计状态层时需要依次回答三个问题：哪些数据丢失后可以重新计算，哪些更新必须在返回成功前持久保存，多个副本之间允许多大的状态差异。明确这些要求后，才能选择内存、缓存或数据库，并确定数据复制和故障恢复方式。经过状态层，服务执行产生的数据被分流到不同存储位置：临时状态保留在高速存储中以保证响应速度，关键记录写入持久化存储以保证故障后可恢复。

经过接入层、服务执行层和状态层，一次请求已经能够从进入系统走到结果返回和状态保存。但这只是请求路径本身通了，要让这条路径长期稳定运行，还需要另外两组能力持续维护。

控制面与验证面：让系统能够长期运行 能够完成一次请求，并不表示系统能够长期稳定运行。当实例数量、负载或运行状态发生变化时，还需要持续维护这套处理过程。这项工作由控制面和验证面共同承担：控制面回答系统应当怎样调整，验证面回答系统当前是否运行正常。

控制面负责决定服务部署到哪里、运行多少实例、哪些实例可以接收流量，以及实例异常后如何替换。规模较小时，这些决定可以由人工完成；规模扩大后，需要由控制器持续观察实际状态，并把它调整到预期状态。例如，系统声明需要运行三个服务实例，而某个实例故障后实际只剩两个，控制器就会创建替代实例，直到实际数量重新达到三个。调度、健康检查、扩缩容和生命周期管理都属于控制面的职责范围。

但是，控制器要做出这些调整，首先需要知道系统当前的运行状态，哪些实例故障了、延迟有没有升高、请求成功率是多少，这些都不能靠猜测。验证面通过指标、日志和链路记录保存运行证据，承担校验与追溯职责。系统通过这些外部信号判断内部运行状态的能力，称为可观测性 (Observability)。系统需要知道请求是否成功、延迟发生在哪一段，以及状态为何发生变化，都要从验证面获取信息。没有这些信息，故障恢复只能确认实例是否重新启动，却无法确认服务质量是否恢复。验证面既服务于排障，也为容量规划、发布检查和责任追踪提供依据。

因此，控制面和验证面是相互配合的：控制面依据验证面提供的运行证据调整实例、容量和故障恢复动作，验证面则持续记录运行状态，为调整和结果校验提供依据。两者共同依赖底层的计算、存储和网络资源，任一部分受到限制，都会影响系统的承载能力或恢复能力。

案例：一次在线会话中的职责协作 前面分别介绍了各层的职责，现在用一次完整的在线游戏会话，看各层之间如何配合。分开讨论时看到的是每层各自做什么，放在一起更关键的是层与层之间的交接：请求在什么位置从一层进入下一层、状态在谁手里被修改、运行信息又如何反馈给控制机制。

玩家进入游戏时，接入层首先建立连接、校验身份，并根据当前各实例的负载情况，把玩家会话分配到合适的服务实例。这里接入层和服务执行层之间的交接点是“已经确认身份、绑定到具体实例的会话”，后续操作请求都沿着这条已建立的通道转发。

玩家在游戏中发出移动或技能操作时，请求沿接入通道到达服务执行层。服务执行层从状态层读取当前世界状态，经过规则校验后计算新状态，再写回状态层并把权威结果返回给玩家。这一步的配合重点是，服务执行层负责规则判断，状态层负责数据存取，两者职责分离但必须协同工作——执行层不能直接绕过状态层修改数据，状态层也不会主动判断操作是否合法。

玩家退出游戏时，接入层关闭连接，服务执行层通知状态层回收会话等临时状态，同时把不能丢失的结算结果写入持久化存储。临时状态和持久记录的分流，是状态层内部两类存储之间的配合。

上述步骤构成一次在线会话的主要请求路径。在这条路径背后，控制面根据在线人数和实例状态持续调整服务容量、替换故障实例；验证面则通过指标、日志和链路记录追踪每一步的结果，为扩缩容决策和故障定位提供依据。玩家经历的会话步骤与系统内部各层的职责对应关系，如图 2 所示。

玩家感知到的接入、行动、交互和退出四个阶段，每一步背后都有接入层、执行层和状态层



图 2: 一次在线会话中的职责协作：接入层建立可信通道，服务执行层校验操作并产生结果，状态层更新共享状态并保存关键记录。

的协同：玩家看到的是操作结果，系统内部则完成了身份校验、规则裁决、状态更新和持久保存等一系列工作。在线游戏只是说明上述职责如何协作的一个具体例子。电商平台、在线协作系统和模型服务虽然处理的对象不同，同样需要通过接入层、服务执行层和状态层完成请求处理，并通过控制面与验证面维持系统运行。这组职责及其判断标准可以迁移到不同业务。

明确了在线系统需要承担的职责，接下来的问题是怎样判断一种组织方式好不好。具体架构设计需要在多种实现方式之间选择，而选择之前首先要明确评价标准。

1.3 架构设计的评价目标

能够完成一次请求，只能说明系统中的各项职责可以配合工作。在用户较少、组件正常运行时，把接入、计算和状态访问集中在一个服务中，或者分别部署为多个服务，都可能及时返回正确结果。请求增多、状态被并发修改或组件失效以后，两种组织方式的差异才会显现。评价架构不能只看正常条件下的单次请求，还要逐级加压，检查运行条件变化后系统能否保持预期行为。

第一道压力来自请求数量。当并发请求增多时，系统还能不能在可接受的时间内完成足够多的工作，这是性能要回答的问题。第二道压力来自并发修改。多个操作同时读写同一份状态时，结果是否仍然符合规则，这涉及状态正确性。第三道压力来自负载的持续变化。用户量上涨时系统能不能扩容、退潮时能不能缩容，这考验可扩展性。第四道压力来自组件故障。当机器、进程或链路突然失效时，服务还能不能继续提供，以及多久能够恢复，这取决于可用性。最后，以上所有能力的实现都需要消耗资源和运维投入，因此成本也必须纳入评估。

由此，架构设计可以从性能、状态正确性、可扩展性、可用性和成本五个维度评价。前四个维度衡量系统在不同压力下能做到什么，成本则衡量做到这些需要付出多少。下面以在线游戏在高负载、并发状态更新、容量变化和组件故障下的表现为例，说明这些维度如何约束架构选择。

1.3.1 性能：控制延迟并提高有效吞吐

性能是用户对在线系统最直接的体感：页面能否及时打开、点击能否及时响应、数据能否及时加载，都取决于它。当更多请求同时到达时，系统还能否在可接受的时间内完成足够多的工作，是性能要回答的核心问题。

例如，大型游戏活动开始后，大量玩家可能同时登录、匹配和领取奖励。系统在日常负载下响应很快，到了活动高峰，单位时间内完成的请求却不再增加，部分玩家还会长时间停留在登录或匹配页面。要解释这些现象，不能只用“快”或“慢”描述系统。

因此，性能通常从延迟和吞吐两个方面衡量。延迟表示一次请求从进入系统到得到结果所需的时间，吞吐表示单位时间内能够完成多少请求。平均延迟反映整体情况，但可能掩盖少量等待时间很长的请求。比如 100 个请求里有 95 个都在 100 毫秒内返回，但有 5 个等了 1 秒以上；平均延迟看起来还可以，但已有 5% 的请求明显偏慢。因此还需要观察 P95、P99 等尾延迟，也就是把所有请求按耗时从短到长排序后，排在第 95%、第 99% 位置的请求需要等多久。

请求较少时，CPU、网络或存储可能处于等待状态，适当增加并发可以让更多请求同时推进。随着请求到达速度接近系统的处理能力，CPU、内存、连接池或下游存储会逐渐成为瓶颈。来不及完成的请求进入队列，吞吐增长开始放缓，尾延迟和超时持续增加。

系统内的请求数量、到达速度和平均停留时间之间的关系，可以用排队论中的 **Little 定律** 更准确地描述： $L = \lambda W$ ，其中 L 是系统内平均请求数， λ 是请求到达率， W 是请求的平均停留时间。回到游戏活动的场景来看，如果每秒进来 500 个登录请求，每个请求平均停留 0.5 秒，那么系统内平均有 $500 \times 0.5 = 250$ 个请求正在处理或排队。到达速度加快或处理变慢时，平均停留时间增加，系统内的请求数也会随之增加；若到达速度持续超过处理能力，系统便不再处于稳定状态，队列会不断增长。

读者可能会以为，并发数越高系统整体越快。但当系统接近处理能力上限时，继续增加并发并不能让整体更快，反而会因为队列变长而拖慢所有请求。性能优化的目标不是尽可能提高并发数，而是在延迟和吞吐都可接受的前提下，找到合适的并发水平。

要比较不同架构的性能优劣，需要有明确的测试方法。性能测试需要明确请求到达速度、并发连接数、请求数据量、持续时间和资源配置。测试还要同时记录吞吐、尾延迟、队列长度和资源利用率，不能只测量单次请求耗时。只有测试条件一致，不同架构的性能结果才具有可比性。

性能评价不仅要看目标负载内的表现，还要看超过目标负载后的行为。当负载超过系统的处理能力时，继续接收所有请求会使队列不断扩大，并拖慢原本可以完成的请求。系统需要限制队列长度，并通过限流、超时或拒绝部分请求控制进入系统的工作量，避免局部过载演变为持续拥塞。

1.3.2 状态正确性：明确状态权威与恢复条件

性能衡量请求能否及时完成，以及单位时间内能完成多少。但响应及时并不表示状态变化一定符合业务规则。一旦请求可能并发、重试或中途失败，状态正确性就需要单独检验。客户端在一局对战结束后请求领取奖励。服务器已经发放奖励并返回成功，但响应消息在网络中丢失，客

户端随后再次发送同一请求。如果服务器把它当作新的请求处理，同一份奖励就会发放两次。如果服务器在奖励记录可靠保存之前就返回成功，进程或机器随后发生故障，玩家可能已经看到奖励，对应记录却没有保存。

除了重复请求和响应丢失，共享状态还可能被多个请求同时修改。例如，两个购买请求几乎同时读取同一份余额，都判断余额充足并完成扣款，最终结果就可能违反账户不能透支的规则。这类问题在正常的单请求测试中不会出现，却会在并发执行、网络重试和服务故障时暴露。

这些现象共同指向状态正确性问题。系统需要明确谁能够提交最终状态，以及一次更新在什么时刻算作成功。对于重复请求，还要使同一操作即使执行多次也只产生一次有效结果，这种性质称为幂等性 (Idempotency)；对于故障恢复，则要确定系统能够恢复到哪个位置。单机系统主要处理并发操作对共享数据的竞争；系统扩展到多台机器后，还要处理副本同步、跨节点更新和部分节点失效。

不同状态不必采用相同的确认方式。角色位置、在线状态等临时信息可以允许短暂滞后，资产、订单和结算记录则需要更严格的确认过程。确认过程越严格，通常意味着需要的通信和等待越多，因此状态正确性还要与延迟和可用性共同考虑。

因此，评价状态正确性不能只验证正常流程，还要主动制造并发写入、重复请求、响应丢失和进程重启等情况。验证时需要检查三类关键规则：同一结果不能重复提交，账户数据不能无故增减，已经确认的记录不能在故障后消失。

1.3.3 可扩展性：让容量随压力变化

当负载超过现有处理能力时，最直接的办法是增加机器或服务实例。但新增资源能否真正发挥作用，取决于原来的工作能否交给它处理。新赛季开放后，大量登录请求可能在短时间内集中到达，一台登录服务很快达到处理上限。此时启动第二个登录服务，并把后续请求分给它，两个实例就可以同时处理不同玩家的登录请求。

一场已经开始的对局则不同。玩家的位置、生命值和操作结果仍保存在原来的战斗服务中，新实例并不知道对局进行到了哪一步，因而不能直接接手。如果状态尚不能恢复或迁移，新实例只能承接新的对局。要让它分担已有对局，还需要提供状态恢复、会话接管和安全迁移机制。

两种场景的差异说明，新增资源能否有效分担压力，取决于工作和状态能否被新实例承接。可扩展性包含两个相互关联的方面。系统通过增加资源承载更多工作，这种能力称为扩展性。系统在负载升高时及时增加资源、在负载下降时及时回收资源，这种能力称为弹性。两者能否发挥作用，都取决于工作和状态能否被合理分配。设计时首先要明确哪部分工作需要更多资源，以及这部分工作能否交给新的实例。

给现有实例增加 CPU、内存或网络能力称为纵向扩展。这种方式实施较简单，但最终受单台机器规格限制。增加实例或节点数量称为横向扩展，可以继续扩大总资源，却要求请求和状态能够被分配到不同实例。对于维护房间、会话等状态的服务，横向扩展还要解决新请求怎样分流、已有状态是否迁移以及旧实例何时退出等问题。

增加资源也不一定带来相同比例的有效容量。如果所有实例最终都要访问同一个数据库热点、全局锁或中心节点，共享部分仍会限制整体吞吐，新增实例还可能增加连接和竞争。因此，

扩展设计需要验证原有瓶颈是否真正得到分摊，而不能只统计实例数量。

弹性还受到时间影响。扩容需要准备资源、拉取包含应用程序及其运行环境的镜像、启动服务并预热状态；缩容需要停止接收新流量、处理存量请求并保存必要状态。新增资源如果在流量高峰结束后才能投入使用，或者缩容时直接终止仍在处理请求的实例，都不能达到预期效果。

因此，评价可扩展性不能只检查实例数量是否发生变化。还要验证新增资源是否分担了实际瓶颈、是否及时投入工作，以及缩容是否在不丢失请求和状态的情况下完成。只有这些条件同时成立，容量调整才能真正改善系统在负载变化下的运行能力。

1.3.4 可用性：限定故障影响并恢复服务

可扩展性关注的是负载变化时怎样调整容量；当已有组件突然失效时，系统能否继续服务，则取决于故障隔离与恢复能力。承载对局的服务实例突然失效时，系统需要停止把新玩家送往该实例，并判断已有对局能否由其他实例接管。如果房间状态只保存在故障进程的内存中，即使立即启动新实例，也无法恢复正在进行的对局。如果系统持续保存可恢复状态并准备备用容量，恢复会更快，但数据复制和空闲资源也会增加成本。

可用性关注系统在组件失效、网络波动或版本更新期间能否继续提供服务，以及中断后需要多长时间恢复。它通常用正常服务时间占总运行时间的比例来衡量，例如 99.9% 的可用性意味着每年累计停机时间不超过约 8.76 小时，99.99% 则将年停机时间压缩到约 52 分钟。完整的恢复过程包括发现故障、停止转发新请求、切换到可用实例并恢复必要状态，缺少任何一步都可能延长服务中断时间。

副本数量不能单独证明系统可用。多个副本如果运行在同一台机器、依赖同一个网络入口或共享同一份易失状态，仍可能被一次故障同时影响。备份如果从未执行恢复验证，也不能证明关键数据能够取回。评价可用性时，需要明确准备应对哪类故障、故障会影响多少请求、多久能够发现并恢复，以及恢复后可能丢失多少状态。

1.3.5 成本：约束目标的实现方式

如果说前四个维度衡量的是系统能做到什么，成本衡量的则是做到这些需要付出多少。提高性能、状态正确性、弹性和可用性都需要付出资源与工程代价。性能优化可能增加计算和网络资源，严格确认会增加存储与通信开销，弹性需要保留可调度容量，可用性则需要副本、备份和故障演练。成本不仅包括服务器和存储费用，还包括跨节点通信、监控告警以及维护复杂系统所需的人力。

评价成本时，需要在相同服务目标下比较方案。减少冗余可以降低日常开销，却会压缩应对突发流量和故障的余量；增加冗余能够缩短恢复时间，也会提高长期成本。架构选择需要先明确哪些业务后果不能接受，再为相应目标配置能够承担的资源与运维投入。

从目标到可检验指标 以上五个维度给出了评价架构的方向，但“稳定”“快速”等描述仍然过于笼统，无法直接检验。要让评价真正可操作，需要把目标转化为可测量的指标。请求成功率、P99 延迟和恢复时间等可观测量称为服务等级指标（Service Level Indicator, SLI）。这些指标希

望长期达到的水平称为服务等级目标 (Service Level Objective, SLO)。当目标进一步形成面向用户的服务承诺时,则构成服务等级协议 (Service Level Agreement, SLA)。例如,对外提供的企业 API 可以把实际测得的请求成功率和 P99 延迟作为 SLI,把“每月请求成功率不低于 99.9%”和“P99 延迟不超过 200 毫秒”规定为 SLO;当这些目标及未达成时的补偿责任写入客户合同时,才形成 SLA。从 SLI 到 SLO 再到 SLA,架构目标被逐步转化为可以持续检查的条件。

同时必须看到,这些目标及其成本约束彼此牵制。更严格的状态确认可能增加延迟,更多副本会提高可用性但增加成本,更高的资源利用率会降低容量余量。架构设计的任务不是把某一单项指标推到最高,而是在明确业务后果后选择可以验证的平衡点。平衡点也不是一成不变的:当负载规模、状态复杂度或运维条件发生变化时,原来可行的方案可能不再满足目标,架构就需要随之调整。下一节就沿着这条思路,讨论运行条件变化如何驱动架构逐步演进。

1.4 从双人对战到云平台:架构演进的四个阶段

前两节分别说明了在线系统需要承担哪些职责,以及应当用哪些目标评价架构设计。能够满足当前负载的架构,在请求数量、状态规模或服务实例增加后,未必还能达到原有目标。因此,分析架构演进时,需要判断原有方案在什么条件下开始失效,以及什么机制能够解决随之出现的新问题。

系统通常不会在项目初期就采用复杂的分布式架构。它往往从集中式结构起步,先建立最小的在线闭环;当并发请求成为瓶颈后,再提高单机的并发处理能力;当单机资源达到上限后,再拆分到多台机器协作;当服务实例数量继续增长、人工运维难以支撑时,再把部署和运行管理交给平台持续治理。每一次演进都对应上一阶段已经暴露的容量、状态协作或运维问题。

下面以在线游戏服务端为例展开这一演进过程。系统从只支持两名玩家的双人对战起步,随着玩家数量和系统规模增长,依次面对并发连接排队、单机性能耗尽、多节点状态协作以及实例运维等问题。每往前推进一步,都会遇到新的瓶颈,也就要引入新的机制。

这四个阶段前后衔接,构成了全书后续章节的基本脉络。后续各章将分别深入每一个阶段的具体问题与实现方式,在线游戏服务端也将作为主要案例贯穿其中。当然,这些演进规律并不只适用于游戏,在线协作、电商平台和大模型服务遇到规模压力时,同样会经历类似的调整。

1.4.1 阶段一:建立最小在线闭环

最初阶段要验证请求能否完整地进入、处理、返回和保存。客户端能够建立通信通道,服务能够识别消息并执行规则,结果能够返回相关客户端,关键状态能够在进程退出后恢复。此时把接入、计算和状态访问放在一个服务中,能够减少跨进程通信和部署复杂度。

这一阶段的重点是明确协议规则和状态归属,而不是提前拆分服务。以在线游戏为例,一个简单的双人对战足以检验操作意图能否送达、服务器能否给出唯一结果、双方能否收到同步结果,以及结算记录能否可靠保存。

客户端只提交操作意图,单体服务集中完成消息解析、规则裁决和状态更新,再把统一结果返回相关客户端。需要跨会话保留的关键结果则写入进程之外的持久化存储。从请求进入、规则

裁决到状态更新和结果返回，这些环节首尾相接，构成一个最小的在线闭环，如图 3 所示。

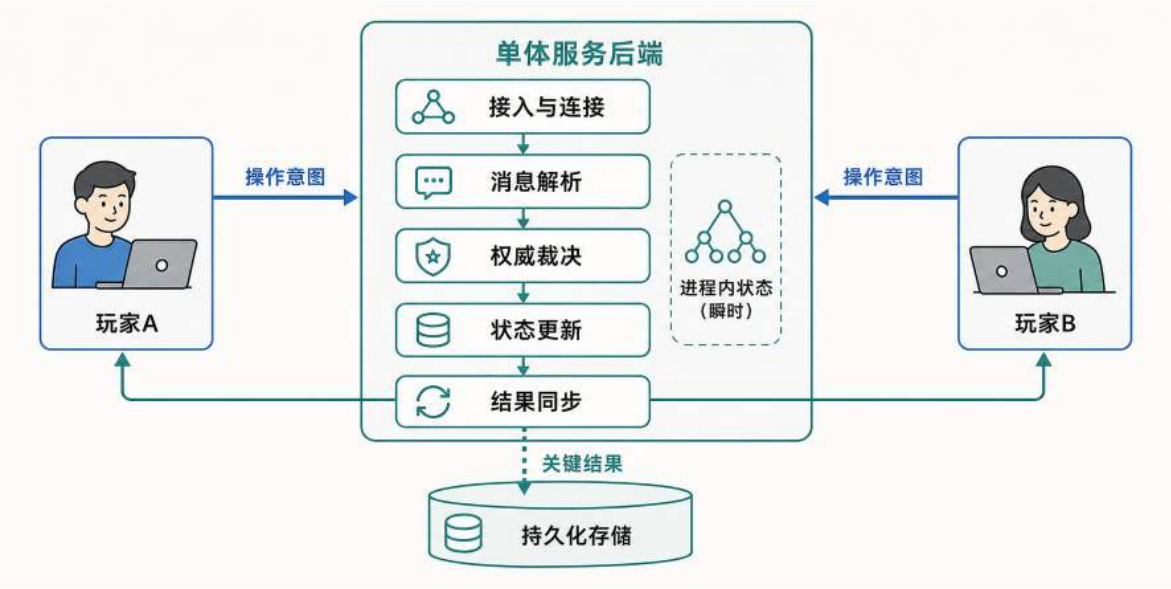


图 3: 最小在线闭环集中完成请求接入、权威裁决、状态更新和结果同步，并把关键结果保存到进程之外。

这一结构最核心的特征是集中：接入、裁决、状态更新和结果同步都在同一个服务进程内完成，只有需要跨会话保留的关键结果才落到进程之外的持久化存储中。它能够以较低复杂度验证核心流程，但关注的仍是一次请求如何完成，尚未说明多个请求同时到达时怎样共享有限的处理资源。

1.4.2 阶段二：提高单机并发能力

最小在线闭环完成后，系统开始承载更多用户，多个请求可能在相近时间到达。当前一个请求仍在等待网络、访问存储或执行计算时，新的请求已经进入系统。如果服务同一时刻只能处理一项请求，后续请求就只能在入口等待。当请求到达速度持续高于处理速度时，等待队列会不断增长，响应延迟也随之上升。

提高单机资源利用率的基本思路，是在一个请求等待外部资源时调度其他请求继续推进。多个请求同时推进后，系统还需要保护共享状态，避免并发修改产生数据竞争。连接池、对象池和缓存等资源复用机制可以进一步减少重复创建和远程访问开销。

这一阶段仍然使用单机资源，目标是在拆分系统之前识别真实瓶颈。请求到达后先进入队列，再由多个处理任务分别执行。这些同时推进的处理任务可以统称为并发执行体，它们访问共享状态时需要进行同步保护，缓存和对象复用则用于减少高频路径中的重复开销。由此形成单机并发处理过程，如图 4 所示。

并发处理的关键环节包括请求排队、多个执行体同时推进、共享状态处的同步保护，以及缓存和对象复用等资源优化手段；排队长度和尾延迟则是衡量系统是否接近饱和的重要信号。并发优化提高的是现有资源的利用效率，并没有增加机器本身的 CPU、内存、网络 and 连接容量。当

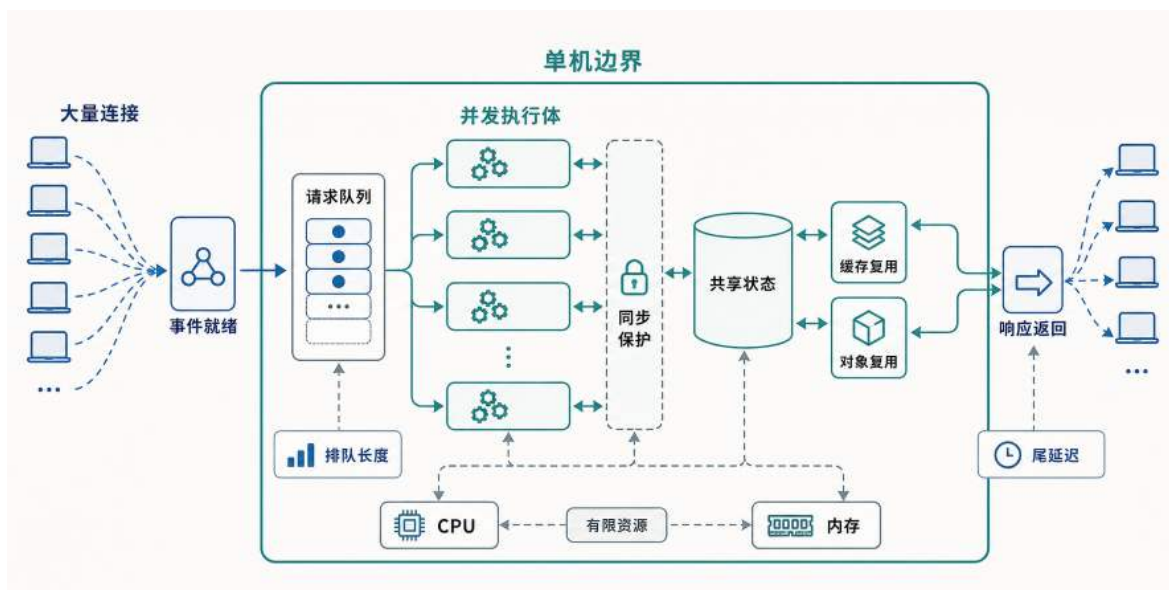


图 4: 单机并发通过请求排队、并发执行、共享状态保护和资源复用扩大处理能力,但仍受单机物理资源约束。

单机容量、故障影响范围或地域部署已经不能满足目标时,系统才需要引入多机拆分;资源接近上限只是其中最常见的触发条件。

1.4.3 阶段三：拆分到多台机器

单台机器的物理资源存在上限。系统扩展到多台机器后,请求、计算和数据必须按照某种规则分配到不同节点。拆分可以增加总容量并缩小单个节点的故障影响,但也会引入路由、状态迁移、副本同步和跨节点事务问题。

在游戏场景中,这种拆分通常按地图或玩家划分,跨地图移动对应状态迁移,跨服交互对应跨节点协调。对应的分片、复制、事务和共识机制,同样适用于订单、文档、任务和其他分布式状态。

服务入口按照状态归属把请求路由到不同计算节点,每个节点内部仍然依靠并发执行处理请求,只是状态被拆分成了不同分片。状态改变归属时,需要把相应数据从一个节点迁移到另一个节点。一次操作涉及多个分片时,节点之间需要通过网络同步必要信息。为了在节点失效时仍能继续服务,需要在备用节点上维持状态副本,由副本同步持续保持一致,故障发生后再由备用节点接管相应请求和状态责任。与此同时,原来在单机内部可以直接完成的调用变成了跨节点网络通信,网络延迟、消息丢失和部分失败成为必须面对的新代价。这些新增的路径、协作关系和代价共同构成多机协作结构,如图 5 所示。

多机拆分引入了三组新的机制和一组新的代价。三组机制分别是:按状态归属的分片路由决定请求去往哪个节点,状态迁移和状态同步处理归属变化与跨分片操作,副本同步与备用节点则在故障时接管相应责任。一组代价则来自跨节点通信:网络延迟使响应变慢,消息丢失需要重传来补救,部分失败则让操作结果不再像单机内部那样确定。横向扩展增加了可用资源,也缩小了单个节点的故障影响范围;代价是原来的进程内操作变成跨网络操作,状态正确性需要在延迟、

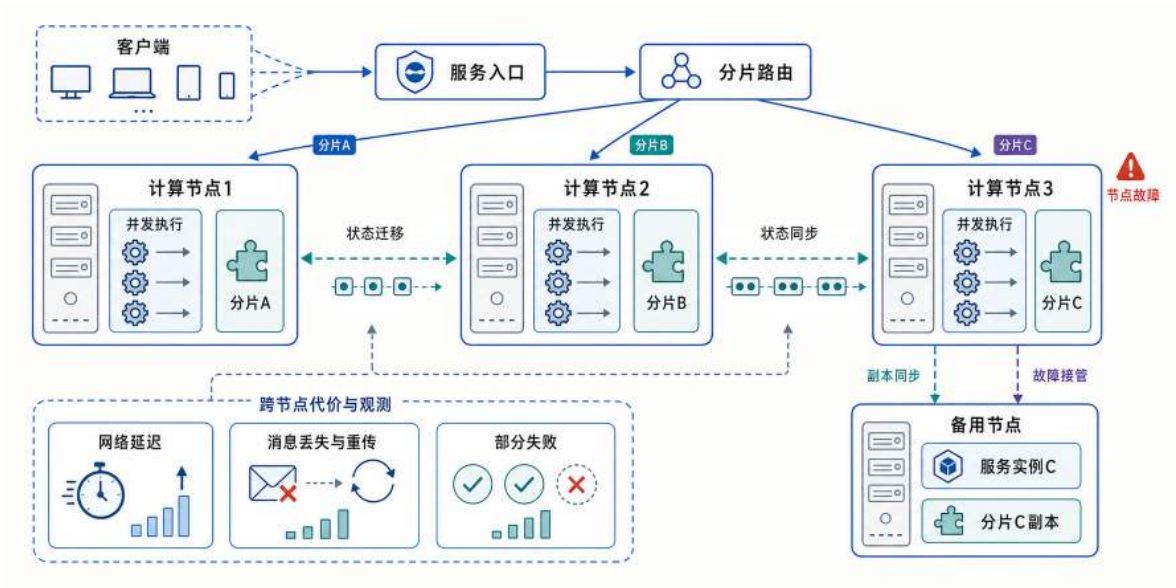


图 5: 多机拆分通过分片路由把请求分配到不同计算节点, 每个节点内部仍并发执行; 节点之间通过状态迁移和状态同步处理归属变化与跨分片操作, 副本同步为故障接管提供前提; 跨节点通信同时带来网络延迟、消息丢失与部分失败等新代价。

丢包和部分失败的情况下重新保证。

1.4.4 阶段四：交给平台持续治理

服务和节点数量增加后，部署与运维本身会成为系统瓶颈。人工安装依赖、启动进程和替换故障实例难以保证一致性，也无法及时响应频繁的负载变化。系统需要把运行环境封装为标准交付单元，再由平台统一管理实例位置、数量、健康状态和访问入口。

平台依据声明的期望状态完成部署、调度、更新、故障恢复和弹性伸缩。这些自动化能力建立在容器隔离、虚拟网络和伸缩控制回路等机制之上。

应用声明版本、副本数量和资源需求后，控制面持续比较期望状态与实际状态，并通过调度创建、替换或回收运行实例。业务请求仍然经服务入口进入应用实例，平台控制路径则在后台维持实例数量和健康状态，由此形成持续调和过程，如图 6 所示。

平台治理的核心是一个持续调和的控制回路：控制面不断比较应用声明的期望状态与集群的实际状态，通过调度执行缩小差异，并在故障重建和弹性伸缩中反复这一过程。平台治理没有消除故障风险和容量限制，而是把运行单元标准化，并把原本依赖人工完成的部署、检查和恢复动作转化为可持续执行的控制过程。

四个阶段串起来看，架构演进的本质不是结构越拆越复杂，而是随着压力增加，把原来集中在一起的职责逐步拆开，用更多组件换取更大的容量、更好的隔离和更稳定的运行。每一次拆分都有明确的代价：并发引入了状态竞争的风险，多机拆分引入了网络延迟和部分失败，平台治理则增加了系统的抽象层数和运维复杂度。没有哪一种架构天然更好，选择的依据永远是当前最突出的瓶颈是什么，以及愿意为新机制付出多少代价。

到这里为止，我们讨论的接入、执行、状态、控制面和验证面，以及它们的演进过程，都还

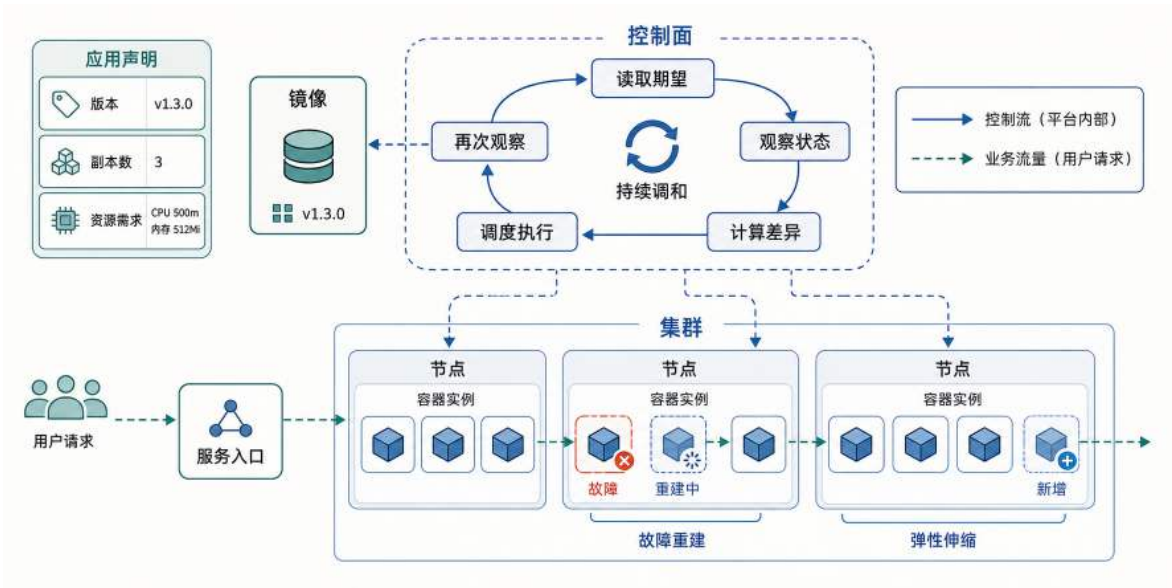


图 6: 平台通过持续调和维持应用声明的版本、容量和资源需求，并在实例故障或负载变化时执行重建与扩缩容。

停留在逻辑职责的层面。但这些职责最终都要运行在真实的计算、存储和网络设备上。逻辑上能成立的方案，落到物理资源上可能受限于带宽、延迟或硬件容量。下一节就把逻辑架构落到真实的基础设施上，看看物理条件如何约束架构的实际能力。

1.5 从逻辑架构到云基础设施

逻辑架构最终要运行在真实的计算、存储和网络资源之上。进程需要 CPU 时间和内存空间，状态需要磁盘或远程存储，服务之间的每次调用都要经过网络链路。逻辑结构能够独立扩展到什么程度，受到这些物理资源及其连接方式的约束。

要理解逻辑职责如何落在真实资源上，可以沿着一次请求的路径，看看它从终端到服务端经过了哪些物理环节。以游戏请求为例，操作指令从玩家设备出发，经过本地接入网络和骨干网络到达数据中心入口，再由内部网络转发到具体服务器和存储设备，这一完整路径如图 7 所示。其他在线系统的请求虽然内容不同，但也要经过相同类型的终端、网络入口、计算节点和状态存储。

数据中心内部由服务器、存储设备和网络设备承载业务处理，并依靠供电、制冷和监控系统维持运行。云平台进一步把具体硬件抽象成虚拟机、容器、存储卷、虚拟网络和托管服务，使应用能够按统一接口申请资源，而不必直接管理每台物理机器。

资源抽象带来三层逐步深化的能力。最基础的是资源池化，多项业务共享同一批硬件，并按需求获得不同配额。在此之上，运行环境可以依据相同镜像和配置在不同机器上重新创建；对状态服务而言，新实例能否继续工作还取决于状态是否已经外置或能够恢复。更进一步，实际运行状态可以被持续监测和调整，控制器能够根据故障和负载变化执行替换或扩缩容——这正是前面讨论的控制面与验证面在资源层的实现基础。

这些能力仍然受物理条件限制。扩容需要找到有剩余 CPU 和内存的节点，镜像和数据需要经过网络传输，状态服务还要满足容量和恢复要求。云平台降低了资源管理的复杂度，但不会消

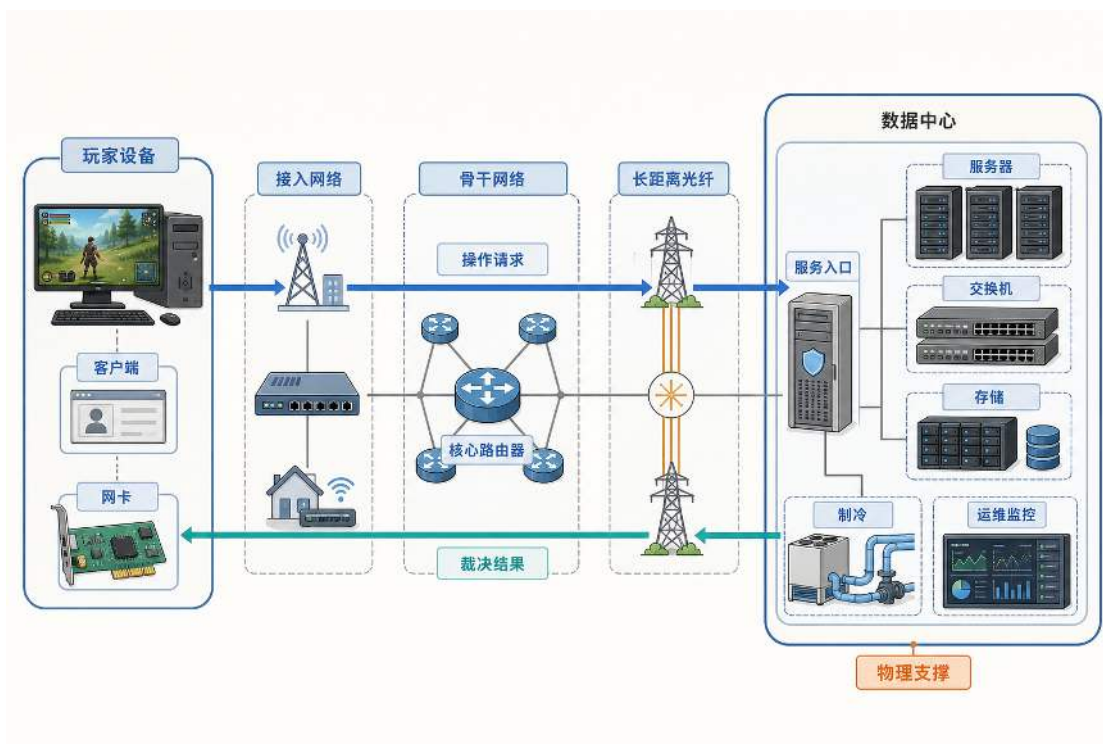


图 7: 在线请求在游戏案例中的基础设施路径：终端请求经过接入网络和骨干网络进入数据中心，再由计算、网络和存储资源共同完成处理。

除带宽、启动时间、存储延迟和故障恢复成本。理解这些限制，是正确使用云服务的前提。

逻辑职责在具体技术中的对应 资源抽象能力最终体现在，应用可以灵活选择承载各项逻辑职责的具体技术组件。以上讨论的接入、执行、状态、控制面和验证面都是逻辑职责，在实际系统中需要由具体技术来承载。不同业务场景可以选择不同的技术组合，但职责划分和权衡原则是相通的。

在线游戏案例中，逻辑职责与技术组件之间存在清晰的对应关系。沿请求路径从左到右，长连接接入由 WebSocket 协议承载，服务执行由 Go 编写的应用服务完成，服务运行环境则打包为 Docker 镜像以保证可复制。高频运行状态保存在内存缓存 Redis 中，需要持久保留的记录则写入 PostgreSQL 等关系型数据库。控制面由 Kubernetes 承担，完成部署调度和故障替换等治理动作。验证面则通过运行指标、事件日志和链路记录三种方式，从各层收集运行证据。这些对应关系如图 8 所示。

这张图把前面讨论的五层逻辑职责落到了具体的技术组件上：沿请求路径，接入层由 WebSocket 承载、服务执行层运行 Go 服务、状态层用 Redis 和 PostgreSQL 分别保存运行状态与持久记录；控制面由 Kubernetes 完成部署调度与故障替换；验证面通过指标、日志和链路记录收集各层运行证据。读者可以顺着请求路径对照正文中学到的每一层职责，找到对应的技术实现。

需要强调的是，技术选择是可变的，但是职责划分是稳定的。接入协议可以从 WebSocket 换成 HTTP 或 gRPC，执行服务可以用 Java、Python 或其他语言编写，状态存储可以换成 MySQL、MongoDB 或云厂商的托管数据库，控制面可以用 Kubernetes 也可以用其他容器编排系统，验

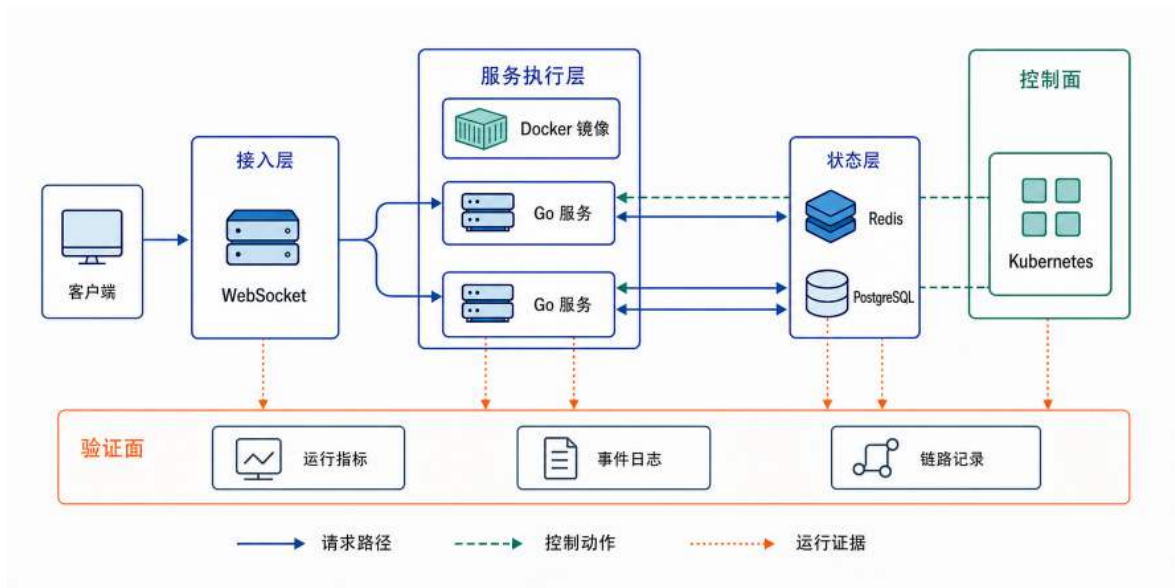


图 8: 逻辑职责在技术栈中的承载方式：接入层、服务执行层和状态层构成主要请求路径，控制面负责交付与治理，验证面从各层采集指标、日志和链路记录。

证面的工具也可以从不同方案中选择。

说到底，图里的每一种技术都只是某个职责的具体载体。在线系统真正稳定的东西，不是用了什么框架、什么数据库，而是请求处理、状态维护、运行治理和可验证性这几组恒常的问题。技术随着时间会一直向前迭代，但问题和职责不会轻易改变。

1.6 小结

在线系统持续运行时，需要同时应对四方面的挑战：规模增长带来的并发压力、共享状态的正确性要求、负载变化下的资源分配，以及故障与运维的自动化需求。接入层、服务执行层和状态层构成主要请求路径，控制面负责部署、调度和故障恢复，验证面通过指标、日志和链路记录判断系统是否正常运行。主要请求路径完成一次业务处理，控制面和验证面让这条路径在负载变化与组件故障下持续运行，所有职责最终都依赖计算、存储和网络资源支撑。

性能、状态正确性、可扩展性、可用性和成本，是评价架构设计的五个基本维度。它们不是孤立的目标，而是彼此牵制：更严格的状态确认可能增加延迟，更多副本会提高可用性但增加成本，更高的资源利用率会压缩应对故障的余量。这些目标共同推动系统从最小闭环走向单机并发、多机协作和平台治理。每一次演进都应当对应已经暴露的容量、状态、故障或运维限制，新增机制的作用也要用具体指标和故障场景来验证。

本章给出的是一套分析在线系统的基本方法：先沿着请求路径拆分职责，再用一组评价维度判断设计的好坏，最后顺着压力增长的方向看架构如何演进。这套方法并不依赖某一种业务或技术栈，换一个场景，职责划分和判断逻辑仍然成立。

后续章节就沿着这条思路，以在线游戏服务端为主要案例，按照压力出现的顺序逐层展开。第 2 章“双雄对战”建立最小网络闭环，讨论通信通道、消息边界和状态裁决。第 3 章“英雄集结”处理单机并发、共享状态保护和资源复用。第 4 章“切分世界”进一步将计算和状态拆到多

台机器，讨论多机分片、跨节点协作、一致性与容错。第 5 章“飞升入定”把服务封装并交给集群平台管理，讨论容器化交付、编排与弹性运行。第 6 章“穷理尽微”则下沉到平台内部，分析容器、网络和弹性控制的底层实现机制，并把这些能力迁移到高并发 Sandbox 场景。

1.7 思考题

1. 选择在线协作、电商或大模型服务中的一种系统，划分其客户端、接入层、服务执行层、状态层、控制面与验证面职责，并说明关键状态由谁最终确认。
2. 登录请求通常可以分配给新启动的服务实例，已经开始的对局或协作会话却不能直接交给任意实例。分别说明两类工作横向扩展所需的状态条件，并给出验证扩展是否有效的指标。
3. 假设一个服务实例突然失效，设计从故障发现到服务恢复的完整过程。指出哪些状态可能丢失，以及缩短恢复时间会增加哪些成本。
4. 比较“始终按峰值准备容量”和“根据负载弹性调整容量”两种方案。分析它们在性能、弹性和成本上的差异，并说明启动时间与排队长度会怎样影响选择。
5. 比较在线游戏与大模型服务在连接持续时间、状态归属、计算资源和扩容信号上的差异。哪些架构判断可以直接迁移，哪些机制需要根据负载特征调整？

2 第二章 双雄对战：网络通信

第一章从整体视角勾勒了在线系统的层次结构。要把这样的整体构想逐步实现为成熟系统，通常先构建一个只保留核心交互和关键约束的最小可行系统（Minimum Viable Product, MVP），用简单请求验证通信路径、状态处理和异常处置是否成立，再逐步扩展功能与规模。

很多人都有过这样的体验：和朋友一起编辑同一份在线文档时，自己这边已经输入了一段话，对方的界面上却要过一会儿才出现；或者在联机游戏里，自己明明已经躲到掩体后面了，却还是被对方击中。

之所以会出现这种“对不上”，是因为在线服务不是在一台设备上跑一个程序，而是在每个用户的设备上都有一份独立运行的客户端，各自保存着一份当前的状态。用户的操作要先变成一条消息，经过网络传到对方那边，对方的状态才会更新。消息传递需要时间，在这段时间里，两边看到的状态自然就不一样。

状态暂时不一样还只是表面问题。更麻烦的是：如果两边都按自己看到的状态来判断结果，同一件事就可能得出两个不同的结论。比如文档编辑时同一句话被两个人同时改掉，交易时一边以为付了钱另一边以为没收到，游戏里一边觉得打中了另一边觉得躲过去了。结果不一致，是所有在线系统都必须解决的核心问题。

要避免这种分歧持续扩大，系统需要打通一次操作从发出到确认的完整链路：先把消息送到正确的程序，再协调各端状态的推进，最后形成双方都认可的结果。因此，本章依次回答三个问题。第一，两个分布在不同设备上的程序如何找到对方，并让接收方从传输的数据中恢复出一条完整消息。第二，各端状态暂时不一致时，怎样使它们继续围绕同一结果推进，而不是越差越远。第三，一次操作是否已经生效，应以谁的判断为准，又怎样让双方确认这一结果。它们分别对应建立连接、同步状态和确认结果，构成本章的主线。

本章以双人在线游戏为贯穿案例。游戏中的每个客户端都根据最近收到的状态版本绘制画面，网络传输所需的时间会使各端在同一时刻掌握不同版本。连续画面把原本隐藏在消息传输和状态更新中的变化直接呈现为角色位置或操作结果的差异，使通信、同步和裁决机制所处理的问题都能对应到具体的状态变化。如图 9 所示，服务器和客户端 B 已经进入状态 v42，客户端 A 仍停留在状态 v41；同一角色在 A 端画面中暴露在掩体外，在 B 端和服务器中却已隐蔽到掩体后方。

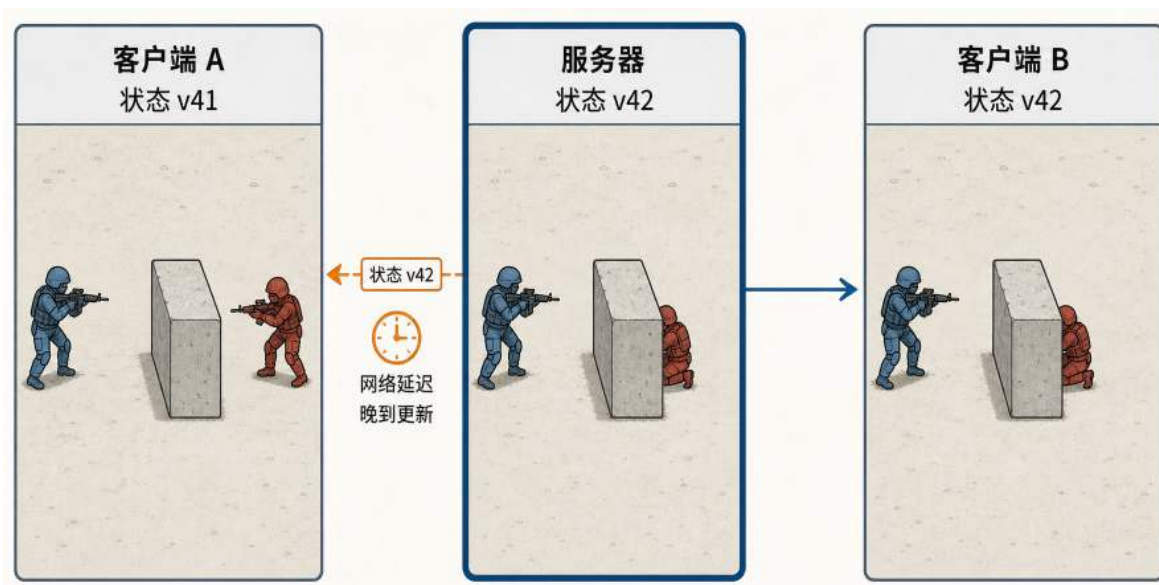


图 9: 网络延迟造成的状态版本差异: 客户端 A 仍显示 v41, 服务器与客户端 B 已进入 v42。

在图示时刻, 客户端 A 仍按照状态 v41 绘制画面, 因此它可能认为目标尚未躲到掩体后方, 并据此发出攻击请求。客户端 A 只能向服务器上报这次攻击, 不能根据本地画面直接认定攻击已经命中。

服务器收到攻击请求后, 以自己保存的状态 v42 和游戏规则为依据进行校验, 并记录最终结果。随后, 服务器把确认后的最新状态和攻击结果发送给两个客户端。客户端 A 收到这些信息后, 校正本地显示的角色位置和攻击结果, 各端由此按照服务器确认的状态继续交互。

协作文档、交易系统和即时通信也需要经过类似的处理: 先由系统统一确认操作结果, 再把确认后的状态同步给各参与方。因此, 游戏案例中的通信、状态同步和结果确认方法, 也可以用于处理其他共享状态系统中的类似问题。

本章以最小原型逐步演进: 先建立双端通信, 再由 P2P 暴露等待、信任和连接规模问题, 引出权威服务器; 随后处理超时、断连和重复请求, 并比较不同传输协议的取舍, 最后用输入、提交、同步、校正四个环节回收全章。

2.1 从单机循环到网络通信

理解多个程序如何通过网络通信协同, 需要先看清单台计算机内部, 一个程序如何接收输入、更新状态并呈现结果。在最简单的单机程序中, 这些操作由同一个进程依次完成, 不需要等待网络消息, 也没有其他独立程序参与。它的运行逻辑可以用一个循环概括:

单进程状态循环（概念示例）

```
1 for {
2     input := readInput()      // 读取本地输入
3     updateState(world, input) // 更新唯一一份本地状态
4     render(world)             // 把当前状态呈现出来
5 }
```

循环包含三个基本环节：`readInput` 读取用户输入，`updateState` 根据输入更新本地状态，`render` 把当前状态呈现到屏幕上。三者共同构成程序的主循环。

在单进程环境中，状态处理比较简单：程序只维护一份当前状态，所有逻辑都读写同一块内存；事件顺序由主循环自然确定，按到达的先后依次处理。由于状态只有一个副本，输入处理的结果也只在本地生效，不会影响其他独立运行的程序。

第二个独立程序实例加入后，两者的操作需要共同影响同一份共享状态。实例 A 的输入和画面运行在设备 A 上，实例 B 的输入和画面运行在设备 B 上；两个实例各自保存一份本地状态，无法再像单进程那样直接读写同一块内存。A 的操作要反映在 B 的画面中，必须先通过通信通道从 A 发送到 B；消息到达之前，B 只能按照旧版本继续呈现。由于网络传输需要时间，两个实例在同一时刻掌握的版本可能不同：A 可能依据较早的版本判断一次操作已经生效，B 却依据较新的版本认为状态已经改变。如果两端继续从各自保存的状态独立计算，同一操作就可能产生不同结果。要避免这种分歧继续扩大，两个实例必须交换操作或状态，并依据共同的信息更新各自保存的本地副本。

同一台机器上也可以启动两个彼此独立的程序进程，用来模拟分布在两台设备上的交互场景。两个进程分别维护自己的输入和状态，不能直接读取对方的数据，必须通过通信通道交换操作。每个进程仍然运行自己的主循环，但状态更新不再只依赖本地输入，还要纳入对端输入。双端的基本循环如下：

双进程交互循环（概念示例）

```
1 for {
2     local := readInput()      // 读取本地输入
3     send(conn, local)         // 把本地输入发给对端
4     remote := receive(conn)   // 等待对端的输入
5     merged := merge(local, remote) // 合并双方输入
6     updateState(world, merged) // 更新本地状态副本
7     render(world)             // 画面呈现
8 }
```

与前面的单进程循环相比，`readInput`、`updateState` 和 `render` 三个基本环节没有改变，双端循环新增了 `send`、`receive` 和 `merge` 三个步骤。`send` 把本地输入发送给对端，`receive` 获取对端输入，`merge` 将双方输入组织为当前一轮的共同输入，随后才能更新本地状态。

这些新增步骤使两个独立程序实例能够处理同一轮操作，也改变了主循环继续执行的条件。

`receive` 一直在等待对端输入而无法返回，本轮状态更新因而停滞；两端收到并处理输入的时刻不同，状态推进的进度可能暂时不一致；如果对端发送的数据超出预期范围，还可能把不可信输入带入后续计算。双端原型由此暴露出三个具体困难：等待消息会阻塞推进，两份状态可能产生分歧，对端提交的数据不能直接作为共同结果。

等待、状态分歧和输入可信性将在后续原型演进中逐步处理。在讨论这些问题之前，两个程序首先要能够找到对方，并从网络传来的字节中恢复出完整消息。下一节先建立这一通信前提；随后，点对点原型讨论两端如何同步推进，集中式服务器架构再解决由谁确认结果的问题。

2.2 网络通信基础：建立传输通道与消息边界

两个独立程序要交换第一条完整消息，需要依次完成三件事：确定对端程序的通信位置，建立一条可以读写字节的传输通道，再从连续字节中恢复出应用程序能够处理的消息。前两件事涉及 IP、端口、socket 和 TCP 连接，第三件事由序列化与消息分帧完成。完成这些工作，只说明消息已经可以在两个程序之间传递；状态如何同步、结果由谁确认，仍要在后续原型中解决。

2.2.1 建连：从地址端口到传输通道

一个程序不能直接读取另一台设备上进程的内存。要把数据送到对端程序，发送方先要确定目标主机和服务入口，再通过操作系统提供的网络接口建立连接。

IP 地址用于标识主机的网络接口。数据进入网络后，沿途路由器根据目标 IP 地址选择下一跳，最终把数据包送到目标主机。一台主机可以同时运行网页服务、聊天程序和游戏程序，仅有 IP 地址还不能确定数据应当交给哪个程序。

端口号用于区分主机上的通信端点。在常见的传输层协议中，端口号是一个 16 位编号，取值范围为 0-65535。服务器通常在约定端口等待连接，客户端的源端口则由操作系统临时分配。IP 地址和端口通常写成“IP: 端口”：IP 确定目标主机，端口进一步确定主机上的服务入口。

确定通信端点后，应用程序还需要一个使用网络能力的接口。操作系统为此提供了套接字（socket）抽象。程序通过 socket 创建通信端点、配置地址、收发数据和关闭连接，不必直接操作数据包或网卡。

应用程序使用的语言库会把这些操作转换为 socket 系统调用。操作系统同时返回一个标识 socket 的句柄；在类 Unix 系统中，这个句柄通常是文件描述符（file descriptor, fd）。内核中的 socket 是一个具体的通信对象，记录所用协议、本地与远端地址、连接状态、发送缓冲区和接收缓冲区等信息。因此，socket 是应用程序进入内核网络功能的入口，不是协议栈中的一个层次。

socket 可以使用不同的传输层协议。当前通信原型采用 TCP，由传输层处理丢失和乱序，使这一阶段可以集中验证建连和消息解析。这只是当前阶段的选择。有些数据必须按序到达，有些数据过期后便没有继续等待的价值；后一类场景可以考虑使用以独立数据报为交付单位的 UDP。第 2.6 节将完整比较两种协议的适用条件。

确定使用 TCP 后，一条连接还需要同时记录协议和两端地址。对本节讨论的普通 TCP 连接，内核用五元组加以区分：< 协议，源 IP，源端口，目的 IP，目的端口 >。IP 和端口标识

通信端点，五元组则把本地端点、远端端点和传输协议组合起来，标识一条具体连接。在连接两端，源地址与目的地址的角色互换，描述的仍是同一对通信端点。服务器同时接入多个客户端时，也依靠这些不同的连接记录把字节交给相应 socket；这一过程将在第 2.4.2 节展开。

端点和协议确定后，双方才能建立 TCP 连接。监听方先在本机地址和端口上创建监听 socket；发起方连接该地址；监听方接受连接后，得到一个专门用于本次通信的已连接 socket。监听 socket 负责接入新连接，已连接 socket 才负责后续读写。

在 Go 中，`net.Listen` 创建监听端点，`Accept` 和 `DialTimeout` 建立连接，并返回实现了 `net.Conn` 接口的连接对象。下面的概念代码把建连、原始字节往返和资源关闭放在同一个过程里。

两个程序实例的 TCP 建连与原始字节往返（概念示例）

```
1 // 主机 A: 监听并等待另一端接入
2 ln, err := net.Listen("tcp", ":8080")
3 if err != nil { /* handle */ }
4 defer ln.Close()
5
6 conn, err := ln.Accept()
7 if err != nil { /* handle */ }
8 defer conn.Close()
9
10 buf := make([]byte, 1024)
11 n, err := conn.Read(buf) // 读取当前可用的一段字节
12 if err != nil { /* handle */ }
13 if _, err := conn.Write(buf[:n]); err != nil { /* handle */ }
14
15 // 主机 B: 连接主机 A，并完成一次原始字节往返
16 conn, err := net.DialTimeout("tcp", "hostIP:8080", 3*time.Second)
17 if err != nil { /* handle */ }
18 defer conn.Close()
19
20 if _, err := conn.Write([]byte("probe")); err != nil { /* handle */ }
21 reply := make([]byte, 1024)
22 n, err := conn.Read(reply)
23 if err != nil { /* handle */ }
24 _ = reply[:n] // 交给后续处理逻辑
```

主机 A 在 8080 端口监听，`Accept` 为接入方返回已连接的 `net.Conn`；主机 B 使用主机 A 的地址和端口发起连接。连接建立后，两端通过 `Read` 和 `Write` 收发字节。示例把收到的内容原样返回，只用于验证通道是否可用，尚未解释这些字节对应什么业务操作。

应用程序调用 `Write` 后，字节经系统调用进入 socket 的发送缓冲区，再由 TCP、IP、链路

层和网卡送入网络。对端按相反方向处理数据，将字节放入接收缓冲区，接收程序再通过 Read 取出。TCP 的两个方向各自维护字节序列，因此两端可以同时发送和接收数据，这种通信方式称为全双工。两个方向仍会共享主机和网络资源。

Write 返回描述的是本次系统调用的结果，不能据此认定对端程序已经处理了数据。第 2.5.2 节将进一步区分本机写入、对端接收和业务生效。

一次字节传输需要经过应用程序、系统调用、socket 缓冲区、TCP/IP 协议栈和网卡。图 10 将两端的数据路径与应用层、内核之间的职责分工放在同一结构中。

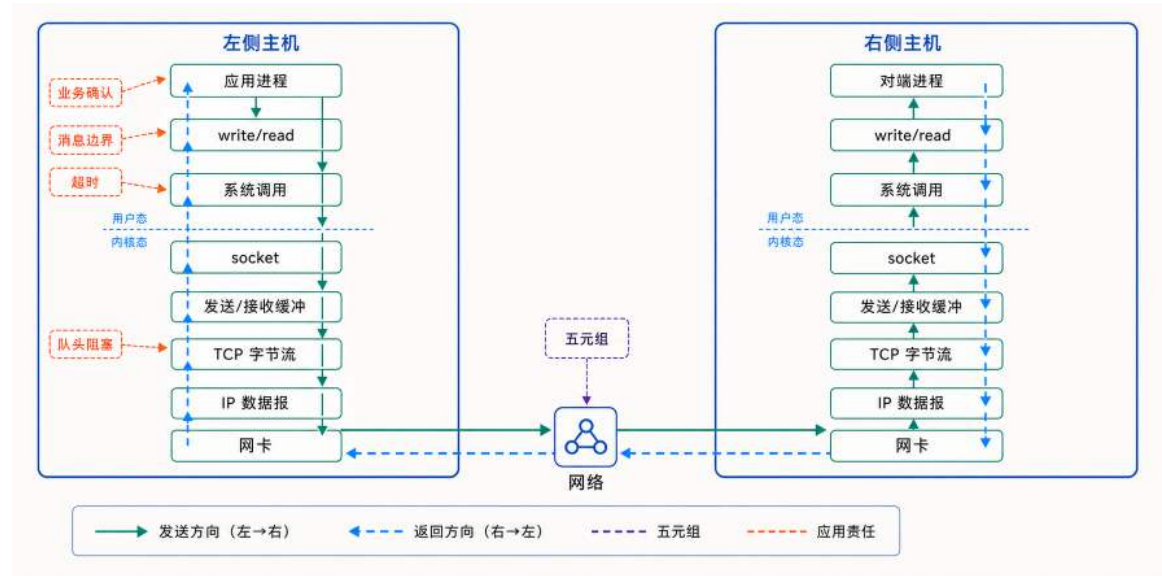


图 10: socket 与协议栈的职责分工：内核沿 TCP/IP 路径在两个通信端点之间传递字节，应用协议负责定义消息结构和业务完成条件。

TCP 在这里负责按序交付字节。下一小节先解决字节流中的消息边界；读取应等待多久、业务何时生效，分别在第 2.5.3 节与第 2.5.5 节展开。按序交付还会使后续数据等待缺失的前序数据，这一队头阻塞问题将在第 2.6 节讨论。

2.2.2 消息边界：如何在字节流中恢复完整消息

在线系统的业务逻辑通常以一条完整的请求或事件为处理单位，例如文档编辑操作、交易指令、即时消息和游戏输入。要把这些对象送入 TCP 通道，应用层协议需要完成两项工作：规定字段如何编码，并规定一条消息从哪里开始、在哪里结束。前者由序列化解解决，后者由分帧解决。

发送前，程序先把包含类型、标识和参数的结构化对象编码成字节序列；接收端取得完整字节后，再把它还原成对象。这两个过程分别称为**序列化**（serialization）和**反序列化**（deserialization）。JSON、Protocol Buffers 和自定义二进制格式都可以完成这种转换。

序列化只决定对象怎样表示，不能保留 TCP 中的消息边界。设发送端连续发送 M1 (attack) 和 M2 (move)。应用程序虽然分别调用两次 Write，TCP 传输的却是连续字节 attackmove。接

收端第一次 Read 可能取得 attackmo，第二次只取得 ve。如果程序把每次读取都当作一条消息，就会得到不存在的指令和无法解析的残片。

图 11 将无边界的读取结果与长度前缀分帧放在同一传输过程中，说明消息边界需要由应用层协议恢复。

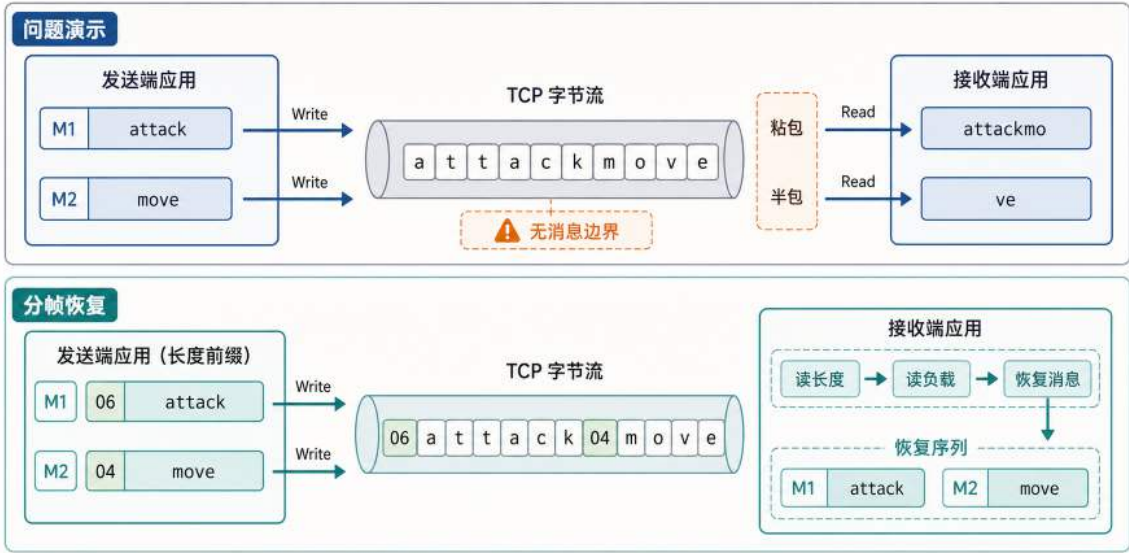


图 11: TCP 字节流与长度前缀分帧：发送端的写入次数与接收端的读取次数没有对应关系；长度前缀使接收端能够从连续字节中恢复消息。

多条消息的字节可能在一次读取中合并，通常称为**粘包**；一条消息也可能分成多次读取，通常称为**半包**。它们不是 TCP 故障，而是字节流不保留应用层分组的直接结果。Read 只返回接收缓冲区中当前可读的字节，读取长度与发送端调用 Write 的次数没有一一对应关系。应用层不能用一次 Read 的返回位置判断消息在哪里结束。

一种常用的分帧方法是在每条消息前放置固定宽度的长度字段，记录后续负载的字节数。教学示例可以把 M1 写成 06 attack、把 M2 写成 04 move；实际实现通常使用固定宽度的二进制整数。接收端先读长度，再持续读取指定数量的负载字节，取得完整负载后才交给业务逻辑。无论一次读取包含多条消息，还是一条消息需要多次读取，接收端都可以按长度字段恢复边界。

长度前缀的最小实现还要处理三个细节。长度字段需要使用双方约定的字节序；头部和负载都可能分多次到达，因此要读满指定长度；在申请负载缓冲区之前，还要检查长度上限，避免异常长度造成过大的内存分配。下一节的锁步输入同步也不能无限等待对端消息，因此示例为读取设置了截止时间。截止时间只用于限制等待，超时后的连接状态将在第 2.5.3 节详细讨论。

长度前缀分帧与限时读取（概念示例）

```
1  const maxFrameSize uint32 = 1 << 20 // 示例上限: 1 MiB
2
3  func writeFull(conn net.Conn, data []byte) error {
4      for len(data) > 0 {
5          n, err := conn.Write(data)
6          if err != nil { return err }
7          if n == 0 { return io.ErrShortWrite }
8          data = data[n:]
9      }
10     return nil
11 }
12
13 func sendFrame(conn net.Conn, payload []byte) error {
14     if uint64(len(payload)) > uint64(maxFrameSize) {
15         return ErrFrameTooLarge
16     }
17     var header [4]byte
18     binary.BigEndian.PutUint32(header[:], uint32(len(payload)))
19     if err := writeFull(conn, header[:]); err != nil { return err }
20     return writeFull(conn, payload)
21 }
22
23 func recvFrame(conn net.Conn, timeout time.Duration) ([]byte, error) {
24     _ = conn.SetReadDeadline(time.Now().Add(timeout))
25
26     var header [4]byte
27     read, err := io.ReadFull(conn, header[:])
28     if err != nil {
29         if read > 0 { return nil, ErrBrokenFrame }
30         return nil, err
31     }
32
33     n := binary.BigEndian.Uint32(header[:])
34     if n > maxFrameSize { return nil, ErrFrameTooLarge }
35
36     payload := make([]byte, int(n))
37     if _, err := io.ReadFull(conn, payload); err != nil {
38         return nil, ErrBrokenFrame
39     }
40     return payload, nil
41 }
```


`binary.BigEndian` 使两端按同一种顺序解释四字节长度字段。`io.ReadFull` 会持续读取，直到填满头部或负载；`writeFull` 则处理一次写入未完成全部字节的情况。`maxFrameSize` 是用于说明检查位置的示例上限，实际值应由应用协议允许的最大消息确定。同一连接上的完整帧还应由单一写流程串行发送，避免不同消息的头部和负载交错。

`sendFrame` 接收的 `payload` 是序列化后的字节，`recvFrame` 返回完整负载后，调用方再执行反序列化。长度字段只负责恢复边界。实际协议还可以在消息头中加入版本、消息类型和序列号，用于兼容协议升级、分派处理逻辑和标识消息所属轮次。

如果读取因超时返回，并且程序尚未取得任何头部字节，调用方可以重新设置截止时间后再读。一旦已经消费部分头部，或者头部完整但负载没有读完，解码位置便停在一条消息中间，不能把剩余字节当作下一条消息的头部。示例返回 `ErrBrokenFrame`，调用方应关闭并重建连接；具体的错误分类和重连处理将在第 2.5.3 节展开。

至此，两个程序已经能够建立 TCP 连接，并交换一条边界清楚的结构化消息。这并不保证两端状态一致，也没有回答发生分歧时由谁确认结果。下一节把通信通道接入状态更新过程，从最简单的 P2P 直连开始讨论这些问题。

2.3 P2P 直连原型：输入同步与状态一致

点对点直连 (Peer-to-Peer, P2P) 允许两个程序实例直接交换数据，不经过中间节点。少量参与者时只需维护少量连接，前文介绍的消息序列化和分帧机制均可直接复用。不过，消息能够到达，并不等于两端已经得到一致的运行状态；接收方还需要解释消息，并将其纳入本地状态更新。

一种做法是直接发送完整状态或状态增量，由接收方覆盖相应数据。另一种做法是只发送引起状态变化的输入，让两端分别计算新状态。本节采用后一种方法：网络负责同步输入，本地程序负责更新状态。由此，**输入同步是通信机制，状态一致是该机制期望得到的结果**。

2.3.1 锁步输入同步：机制与成立条件

如果两端不等齐同一帧的输入就各自推进，该帧使用的输入集合就会不同，状态更新随之分叉。为了避免这种分歧，可以让两端以完全相同的输入集合同步推进，这种方法称为锁步输入同步 (Lockstep)。

锁步把系统运行过程划分为连续的逻辑帧。逻辑帧采用固定的模拟步长，独立于各自的本地呈现频率。在第 $n+1$ 帧中，参与端 A 和参与端 B 分别产生输入 A_{n+1} 与 B_{n+1} ，并将带有帧号的输入发送给对方。两端收齐同一帧的输入后，再从相同的状态 S_n 出发执行同一个更新函数：

$$S_{n+1} = \text{Update}(S_n, A_{n+1}, B_{n+1}).$$

图 12 展示了一个逻辑帧内的数据流。跨越网络的是两条输入消息，状态更新则分别发生在两个程序实例内部。因此，锁步并不是把一份新状态从一端复制到另一端，而是让两端依据相同的输入独立计算下一帧状态。

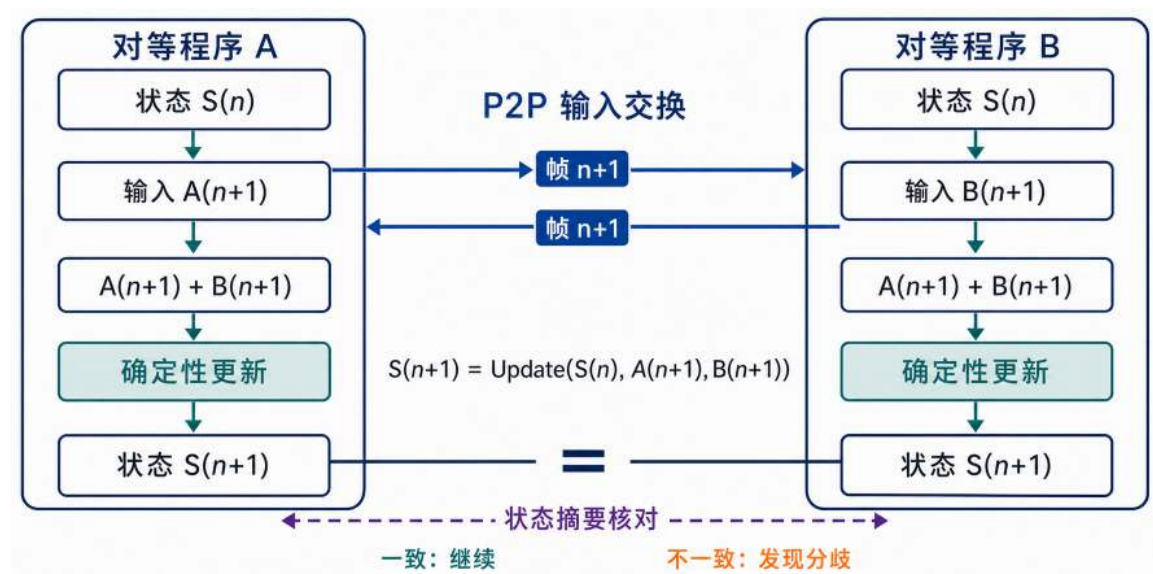


图 12: P2P 锁步输入同步：两个对等程序实例交换带帧号的输入，并在相同状态上执行同一更新函数，分别得到下一帧状态；状态摘要用于发现两份状态是否产生分歧。

上述过程能够维持状态一致，需要满足三个条件。第一，两端必须从相同的初始状态开始。第二，每条输入都必须携带逻辑帧号；某一帧的输入尚未收齐时，该帧不能进入状态更新。第三，更新过程必须具有确定性，即相同状态与相同输入在两端产生相同结果。随机数种子、运算顺序、时间步长以及可能产生平台差异的数值计算，都需要纳入确定性约束。

即使两端从相同状态出发，并使用相同输入和更新规则，程序缺陷、随机数使用不一致或数值计算差异仍可能使结果逐渐分叉。直接交换完整状态进行核对会产生较大的通信开销，因此，两端可以定期为当前状态计算一个较短的摘要，例如 checksum 或 hash，只交换摘要进行比较。摘要相同，表示本轮核对没有发现差异；摘要不同，表示两端状态已经分叉。摘要只能发现问题，不能判断哪一端正确，也不能修复状态。后续恢复仍需要重新加载双方认可的状态快照，或者终止当前会话。

将锁步机制写入程序时，每一帧需要完成四个动作：采样本地输入、发送本地输入、接收对端输入，再用双方输入更新状态。本地输入在一帧内只能采样一次；只要对端输入尚未到达，本帧就不能更新状态，也不能进入下一帧。

发送和接收的顺序同样重要。如果两端都先等待接收，而对方也尚未发送，两个程序就会相互等待。因此，两端都应先发送本地输入，再接收对端输入。下面的 `exchangeInput` 明确展开了这一顺序，其中 `encodeInput` 和 `decodeInput` 表示输入消息的序列化与反序列化，`validateFrame` 用于检查收到的输入是否属于当前帧。

锁步输入同步 (概念示例)

```
1 type InputMsg struct {
2     Frame int    // 输入所属的逻辑帧
3     Action Input // 本帧的具体操作
4 }
5
6 func exchangeInput(
7     conn net.Conn,
8     local InputMsg,
9     timeout time.Duration,
10 ) (InputMsg, error) {
11     // 先把本地输入序列化、分帧并发送
12     payload, err := encodeInput(local)
13     if err != nil {
14         return InputMsg{}, err
15     }
16     if err := sendFrame(conn, payload); err != nil {
17         return InputMsg{}, err
18     }
19
20     // 发送完成后, 再等待对端同一帧的输入
21     payload, err = recvFrame(conn, timeout)
22     if err != nil {
23         return InputMsg{}, err
24     }
25     return decodeInput(payload)
26 }
27
28 state := initialState()
29 frame := 0
30 for {
31     // 本帧只采样一次; 重试时仍发送同一份输入
32     local := InputMsg{
33         Frame: frame,
34         Action: sampleLocalInput(),
35     }
36
37     var remote InputMsg
38     for {
39         var err error
40         remote, err = exchangeInput(
41             conn, local, 50*time.Millisecond,
42             )
```

正常情况下, `InputMsg` 先被序列化并封装成长度前缀帧, 再由 `sendFrame` 发往对端。对端使用 `recvFrame` 取出完整消息并解码。两端都取得当前帧的本地输入和对端输入后, 才调用 `updateState` 更新状态。这条路径把本章前面介绍的 socket、字节流、序列化和分帧连接到了实际的状态更新。

如果交换过程超时, 本帧已经采样的 `local` 必须保持不变, 重试时仍然发送同一份输入, 不能为相同帧生成新的操作。如果超时发生在一条帧只读取了一部分之后, 当前连接已经无法正确识别下一条消息的起点, 需要先重建连接再重发本帧。只要当前帧的双方输入没有收齐, 状态和帧号都不能继续推进。

2.3.2 P2P 的局限: 等待、信任与连接规模

锁步输入同步降低了传输完整状态所需的带宽, 但也使系统推进依赖所有参与者。只要一端的输入延迟到达, 另一端就必须等待; 网络抖动或短暂离线会直接停住共同的逻辑帧。因此, 锁步系统的推进速度受最慢参与者限制。

第二个问题是结果的可信性。两个参与端都在本地计算状态, 其中任何一端都可能修改输入或更新逻辑。状态摘要可以证明两份状态不同, 却不能证明哪一份状态正确。对于竞争结果、账务变更等需要多方共同认可的关键状态, P2P 结构缺少一个独立于参与者的确认位置。

第三个问题是连接规模。双人 P2P 只需要一条连接; 若 N 个参与者两两直连, 则系统需要维护 $N(N-1)/2$ 条连接, 每个参与者需要与其余 $N-1$ 个参与者通信。参与者增加时, 连接管理、输入分发和等待关系都会迅速复杂化。

因此, P2P 适合参与者较少、彼此可信且网络环境相对可控的场景。随着参与者增加, 或者运行结果需要得到共同认可, 系统必须进一步解决两个问题: 如何降低参与者之间的连接复杂度, 以及如何建立独立于参与者的状态确认机制。

2.4 CS 架构与权威服务器设计

P2P 原型暴露的等待、信任和连接规模问题, 使系统需要引入独立服务器。下面先说明客户端与服务器的职责如何变化, 再按照连接管理、状态仲裁和状态分发的顺序展开服务器内部处理。

2.4.1 从 P2P 到权威服务器

CS (Client-Server) 架构通过引入独立服务器同时回应上述需求。客户端不再彼此建立连接, 而是将输入意图提交给服务器; 服务器检查输入是否合法, 依据自身维护的状态推进系统, 再将确认后的结果返回客户端。在这一模型中, 服务器承担**唯一真相源** (single source of truth, SSOT) 的角色, 影响共同结果的状态变更以服务器提交的版本为准。

图 13 从通信路径和状态确认两个维度呈现了这一转变。通信由参与者之间的多对多交换改为客户端与服务器之间的连接, 降低了参与者增加带来的连接复杂度; 状态则由各端分别计算和声明, 改为由服务器统一校验与提交, 使不同客户端能够依据同一份确认结果继续运行。

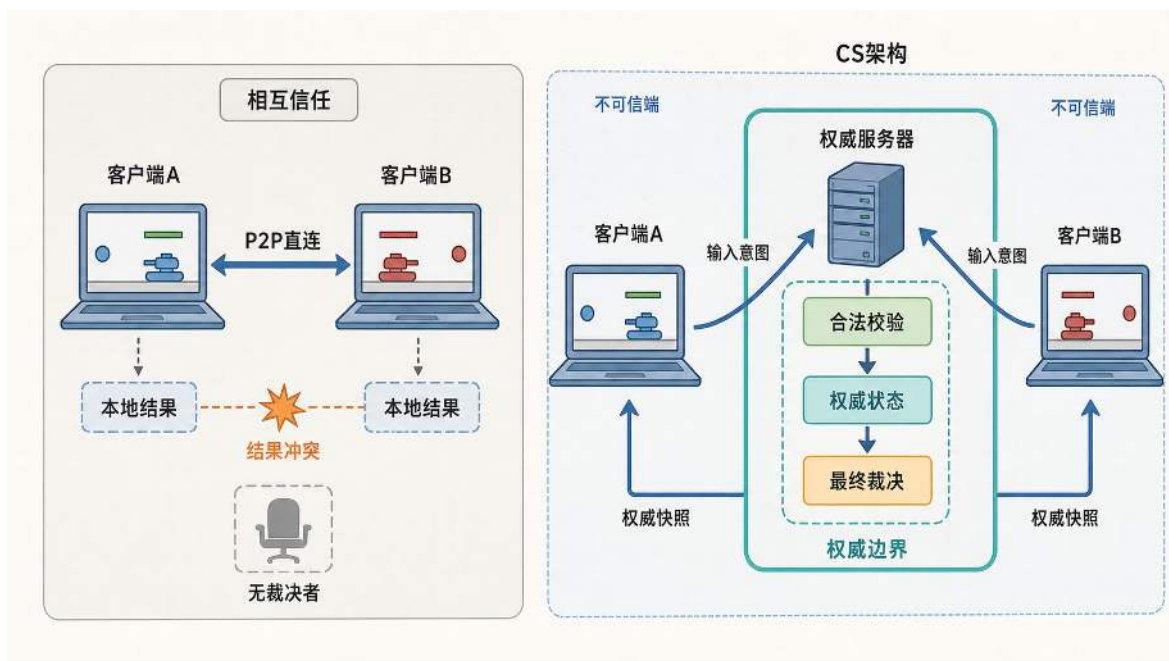


图 13: 从 P2P 到 CS 的通信路径与信任边界变化：P2P 让客户端直接交换并各自得出本地结果，CS 则把输入上报、规则校验和权威状态收拢到服务器。

这一架构变化重新界定了客户端提交信息的语义。在 CS 架构中，客户端提交的是请求或操作意图，服务器依据自身维护的状态完成合法性校验、状态变更和结果确认。电商系统接收下单请求，协作系统接收编辑操作，在线游戏接收移动或攻击指令。这些场景处理的业务对象虽然不同，但均可采用相同的 CS 处理方式：客户端提交操作请求，服务器依据统一状态完成合法性校验，并提交最终的状态变更。

在线游戏中的双人交互场景可以用来说明这一处理方式。首先明确该场景：地图上只有两名玩家，服务器维护二者的坐标、生命值、当前状态和对战会话。玩家可以移动；两人靠近后，客户端显示“可以发起对战”；真正的请求仍要发到服务器，由服务器重新检查距离、状态和会话占用，再决定是否进入战斗。战斗本身先简化成单回合数值裁决：服务器读取双方属性，计算胜负，更新生命值和战绩。

在这个场景中，客户端负责采集输入和显示画面，并上报移动、攻击等操作意图；服务器依据权威坐标与规则判断移动能否执行、攻击是否命中，并提交最终状态。由此，不同客户端显示的局部状态可以暂时存在差异，业务结果仍以服务器的计算为准。

实时游戏通常还会进一步优化显示效果，用客户端预测（client-side prediction）减少本地操作的等待感，用服务器和解（server reconciliation）把预测误差拉回权威状态，用实体插值（entity interpolation）让远端角色的运动更平滑。¹ 这些技术用于改善客户端画面的连续性，本节不深

¹Valve 对 Source 引擎多人网络模型中的 prediction、interpolation 与 lag compensation 有系统说明，见 Valve Developer Community, Source Multiplayer Networking, https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking。关于网络游戏延迟补偿的经典教学文章，也可参见 Glenn Fiedler, What Every Programmer Needs To Know About Game Networking, https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/。

入讨论其具体算法，感兴趣的读者可以结合脚注文献进一步了解。当前讨论继续聚焦 CS 原型的基本职责：关键结果由服务器裁决，客户端维护用于显示和交互的本地副本。

服务器按照固定 tick 消费客户端输入并推进权威状态，客户端则发送输入并接收服务器返回的状态。下面的伪代码给出两端主循环的基本结构：

CS 架构伪代码结构

```
1 for { // 服务器端主循环 (tick 驱动)
2     // 从各连接收集截至当前 tick 的输入 (可能为空)
3     inputs := drainInputQueues()
4     // 推进一帧权威世界
5     updateGameState(inputs)
6     // 广播权威快照/增量
7     broadcastAuthoritativeState()
8 }
9
10 for { // 客户端主循环
11     // 采集本地输入并上报意图
12     localInput := getLocalInput()
13     sendInputToServer(localInput)
14     // 接收并渲染权威状态 (通常带超时/断线处理)
15     gameState, ok := receiveGameStateWithTimeout()
16     if ok { renderGameState(gameState) }
17 }
```

与锁步不同，CS 服务器不以收齐所有客户端的输入作为状态推进条件。网络读取过程将已经到达的输入写入队列，服务器在每个固定 tick 中处理当前可用的输入并更新权威状态，再按照独立的频率向客户端下发状态快照。因此，客户端输入采样、服务器状态更新和快照下发不必一一对应：一个 tick 可以处理多条输入，服务器也可以经过若干个 tick 再下发一次快照。这种解耦使系统能够根据交互延迟、服务器负载和网络带宽分别调整三个周期，同时由权威快照校正客户端暂时存在的状态差异。

前面的主循环说明了客户端与服务器如何交换输入和权威状态。要将这一循环实现为可运行的服务器，还需要把输入接收、规则校验、状态提交和结果返回落实到具体模块。为此，服务器内部划分为连接管理、状态仲裁和状态分发三项职责。

多个客户端的输入会以不同时间和顺序到达服务器，其中还可能夹杂重复、过期或不合法的操作。网络读取过程如果直接修改共享状态，连接抖动和异常输入就会进入状态更新路径，服务器也难以说明某次变更经过了哪些检查。因此，输入到达服务器后，需要先与权威状态的修改过程分开。

图 14 将这项分工放回一次持续运行的服务器处理过程中。连接管理先确认消息来自哪个连接和会话，再把解析后的操作意图写入事件队列。队列使网络接收可以继续进行，服务器主循环

则按照自己的 tick 消费当前可用事件；进入队列只表示服务器已经接收某项操作，并不表示该操作已经生效。

事件随后进入状态仲裁。状态仲裁依据当前权威世界检查移动、攻击等操作是否合法，并对同一 tick 内的事件进行排序和去重；只有通过这些处理的事件才能更新权威状态。这样，客户端提交的操作意图与服务器已经确认的业务结果被明确区分，多个客户端也会围绕同一份已提交状态继续交互。

状态分发读取已经提交的权威状态，根据可见范围和下发节奏生成快照，再发送给相关客户端。客户端用快照校正本地副本，并从校正后的状态产生下一轮输入。状态分发由此把一次服务器更新接回后续交互，使客户端暂时存在的画面差异不会演变为彼此独立的结果。

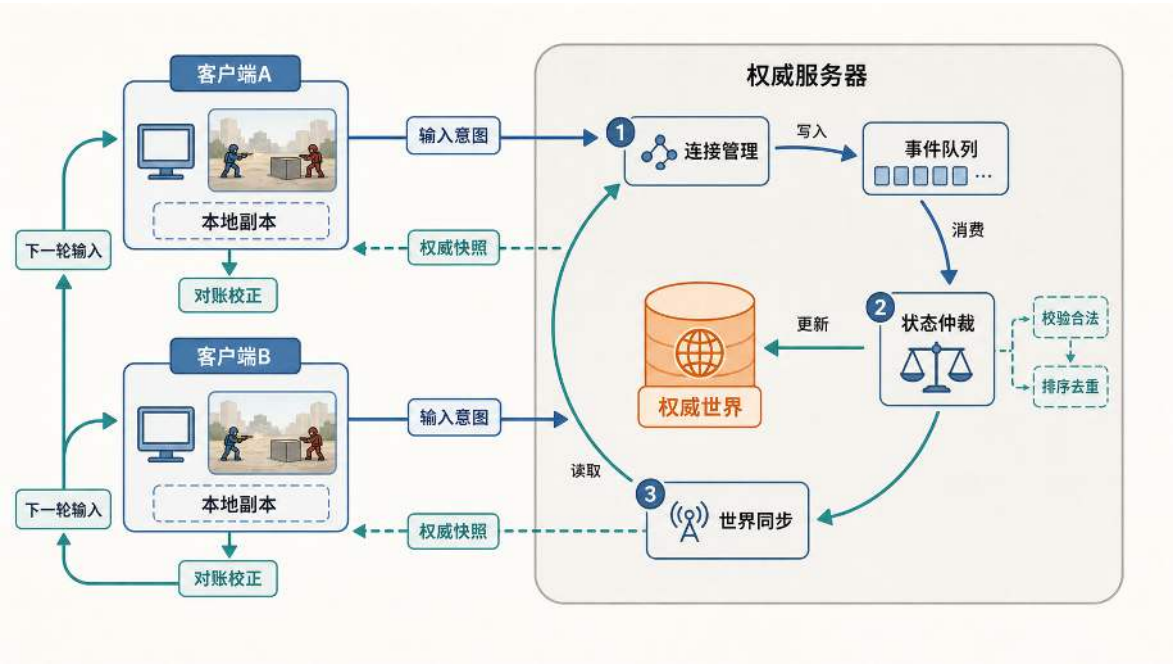


图 14: 权威服务器的持续处理路径：连接管理将输入写入事件队列，状态仲裁校验并提交状态变更，状态分发再用权威快照校正客户端副本并接入下一轮输入。

连接管理、状态仲裁和状态分发会在连续的 tick 中处理不同批次的输入，而非等待一项请求走完整条路径后再接收下一项请求。下文按照连接管理、状态仲裁和状态分发的顺序，分别说明每项职责面对的问题和实现要点。

2.4.2 连接管理：会话生命周期维护与读写调度

连接管理负责判断连接是否有效、连接归属哪个会话，以及网络读写应进入哪条有界队列。它不参与状态变更的合法性判定等业务规则校验；这些工作由状态仲裁完成。连接管理的核心任务是把网络输入转换为可处理的事件、把待发送状态交给独立写流程，并维护连接与会话从建立、运行到释放的完整生命周期。

接入循环与阻塞风险 一种常见错误是将连接接入与单个客户端的消息处理组织为顺序执行流程。服务器通过 `Accept` 建立一条连接后，立即进入该连接的消息读取与处理过程，只有处理过程结束，才会再次调用 `Accept`。如伪代码中展示的，如果该客户端长时间没有发送数据，读取操作将持续等待，服务器也无法接收后续连接；其他客户端的连接请求只能停留在接入队列中。

阻塞式连接处理的反例

```
1 for {
2     conn, err := listener.Accept()
3     if err != nil { continue }
4     handleClient(conn) // 若此处持续等待，后续连接将无法被接入
5 }
```

改进的关键是分离连接接入与消息读取。接入循环只负责建立连接和登记会话，完成后立即再次调用 `Accept`；已经建立的连接则进入独立的读取流程，读取流程将网络消息解析为输入事件并写入队列，服务器主循环在固定 tick 中消费这些事件。多个读取流程如何同时推进，将在下一章讨论。

内核连接与应用会话 客户端接入后，服务器需要同时维护两类状态。**内核态连接状态**随 socket 存在于内核，包括发送与接收缓冲区、TCP 序列号和拥塞窗口等；**应用会话状态**由服务器程序维护，包括玩家身份、最近一次收到消息的时间、应用层序列号以及重连窗口内需要保留的最小快照等。

两类状态的生命周期并不相同。网络中断会使原有 TCP 连接失效，玩家身份和对局进度却可能仍需保留，因此服务器不能只用 socket 表示一个客户端。连接管理需要为每个客户端建立一份可追踪的会话记录：会话 ID (Session ID) 标识这次应用会话，会话上下文 (Session Context) 保存会话运行期间需要持续维护的状态。

持有连接对象不能证明底层连接仍然有效，重新建立 TCP 连接也不会自动恢复应用会话。客户端重连后，服务器需要依据会话 ID 找到原有会话上下文，再把新连接与尚未结束的会话重新关联。

读写分离：网络等待不能阻塞世界推进 如果把 socket 读写直接放进世界主循环，一次网络等待就可能延迟整个共享状态的状态更新。最小服务器因此将网络读写与世界推进分开：连接读取流程持续从 socket 取出数据，把解析后的输入事件写入有界输入队列；世界主循环只在固定 tick 到来时消费队列并推进权威状态，再把待发送状态写入每个连接的有界发送队列；独立写流程负责真正写入 socket。输入队列或发送队列满时，系统必须明确选择等待或丢弃。对于可以被新版本覆盖的实时状态，系统可以淘汰过期的中间状态；不能丢弃的关键操作，则应让新的入队操作等待，或者返回明确的失败结果，使产生数据的一方随下游处理能力减速。这种把压力从下游队列传回上游的机制称为**反压** (backpressure)。若单个连接长期无法消费数据，服务器还应在超时后断开连接，避免它持续占用资源。本章先建立这项职责分工；当连接数量和消息速率上升后，如何同时推进多个读写流程、协调事件入队与消费并限制积压，将成为下一章讨论的主

要问题。

封装连接生命周期 工程上通常会把“连接句柄”和“会话上下文”打包成一个对象。这样服务器面对的就不再是零散的 `socket` 与全局变量，而是一份可追踪、可回收的会话状态。下面的 `clientConn` 把底层连接、会话与客户端标识、输入处理进度、连接响应时间以及收发队列放在一起；其中 `lastSeq` 表示会话上下文可以携带最近处理进度，真正决定某条输入是否生效的逻辑仍然留给状态仲裁模块。

连接管理的结构体定义

```
1 type clientConn struct {
2     id          int          // 唯一会话标识
3     clientID    int          // 会话关联的客户端
4     conn        net.Conn      // 当前底层连接
5     lastSeq     uint32      // 最近提交的输入序列号
6     lastReceivedAt time.Time  // 最近收到合法消息的时间
7     lastPongAt  time.Time  // 最近收到 Pong 的时间
8     lastSent    uint64      // 最近成功写入本机发送路径的状态版本
9     lastAacked  uint64      // 客户端已经确认处理的状态版本
10    eventQueue  EventQueue // 等待仲裁的有界输入队列
11    sendQueue   SnapshotQueue // 等待写入的有界状态队列
12 }
13 // 实例化客户端连接对象
14 var clientA clientConn
15 var clientB clientConn
```

完成这一层封装后，连接管理围绕 `clientConn` 执行两个步骤。`handleInput` 在收到合法的完整输入时更新最近接收时间，并将输入写入事件队列；`handleTick` 在每个服务器 `tick` 检查连接是否长时间没有合法消息，再由状态仲裁统一消费事件队列，随后推进权威状态并把待发送快照放入发送队列。对应的伪代码如下：

连接管理伪代码示例

```
1  const timeoutDuration = 30 * time.Second
2
3  // 收到并解析出一条完整输入后调用
4  func handleInput(c *clientConn, input InputEvent) {
5      c.lastReceivedAt = time.Now()
6      if !c.eventQueue.TryPush(input) {
7          applyInputBackpressure(c) // 不让输入无限积压
8      }
9  }
10
11 // 每个服务器 tick 调用一次
12 func handleTick(clients []*clientConn, frame uint64) {
13     now := time.Now()
14     active := make([]*clientConn, 0, len(clients))
15     for _, c := range clients {
16         if now.Sub(c.lastReceivedAt) > timeoutDuration {
17             handleDisconnect(c)
18             continue
19         }
20         active = append(active, c)
21     }
22     arbitrateFrame(active, frame) // eventQueue 只在仲裁阶段消费一次
23     advanceWorld(frame)           // 推进物理、计时器等服务器状态
24     queueGameStateForClients(active)
25 }
```

此时，连接管理模块已经能够维护每个客户端的会话状态，并把“读取是否超时”“连接是否健康”等问题显式化为可处理的事件。输入收集与超时检测分开后，服务器可以分别处理消息迟到、读取等待和连接超时，而不会让一次网络等待直接阻塞世界推进。

连接管理让双人原型从“一条裸 socket”推进到“可追踪的会话对象”：服务器不仅知道字节来自哪条连接，也知道这条连接属于哪个参与者、多久没有回应、输入应该进入哪个队列。但连接管理能维护的只是会话层面的状态；连接本身是否健康、写出去的字节是否真的被对端处理，这些更深层的可靠性问题，会在第 2.5 节集中面对。

2.4.3 状态仲裁：一致性防线与权威裁决

连接管理把网络读写整理成有序的输入事件后，状态仲裁依据业务规则校验这些事件，并决定相应的状态变更能否提交到服务器的权威状态。客户端上报的是意图，不是结果：位置变更上

报方向或坐标增量，而不是最终位置；关键操作上报请求，而不是计算结果。服务器只接受自己能验证的输入，并把通过校验的输入提交到权威状态。本章所说的权威状态，指的就是由服务器维护并作为所有客户端共同依据的那份状态。客户端可以持有显示用副本，但不能把副本上的推测结果直接变成事实。

状态仲裁还需要明确一条规则：同一 tick 内，服务器必须知道哪些输入参与本帧裁决。一个最小规则可以这样定：输入消息携带 `frame` 和 `seq`；服务器只接受当前裁决窗口内、序列号更新的输入；缺失输入按空操作或上一帧输入处理；迟到输入丢弃或转入下一帧，不能回头改已经广播的历史；同一 tick 内若有冲突，按固定玩家 ID 或服务器队列顺序处理。规则可以按业务调整，但必须在实现中固定下来，否则权威裁决就没有可执行的依据。

服务器将输入视为驱动权威状态机的命令，并根据 `tick` 与 `seq` 确定命令顺序。某个状态版本一旦提交并广播，后续到达的迟到消息就不能再改写这段历史。这些约束保证了三个基本不变量：一次会话只能创建一次，一次关键变更只能生效一次，已提交版本不能倒退。

以位置更新为例，服务器根据上一帧权威位置、速度上限与时间步长计算可达范围，并在必要时裁剪或拒绝超出范围的请求；以竞争结果裁决为例，服务器依据自身记录的权威状态与判定规则生成唯一结果。这样做的目的，是把决定公平性的计算收拢到服务器侧可记录、可复查的裁决路径中。

把状态仲裁放回下面代码中的状态更新逻辑，可以先分成“收集输入”和“提交裁决”两步：

状态仲裁伪代码示例

```
1 func collectForFrame(clients []*clientConn, frame uint64) map[int]InputEvent
2 {
3     inputs := map[int]InputEvent{}
4     for _, c := range clients {
5         input, ok := popLatestInput(c.eventQueue, frame)
6         if !ok || input.Seq <= c.lastSeq {
7             continue // 缺失或重复输入由服务器生成空操作
8         }
9         inputs[c.id] = input
10    }
11    return inputs
12
13 func arbitrateFrame(clients []*clientConn, frame uint64) {
14     inputs := collectForFrame(clients, frame)
15     for _, c := range sortedClientsByID(clients) { // 固定顺序
16         input, ok := inputs[c.id]
17         if !ok {
18             applyInput(&world, emptyInput(c.id, frame))
19             continue
20         }
21         if isValidInput(input, world.Player(c.playerID)) {
22             applyInput(&world, input)
23             c.lastSeq = input.Seq // 只在真实的新输入提交后推进
24         }
25     }
26 }
```

意图校验、序列号去重与权威提交都放在服务器一侧后，仲裁模块就能把“不可信输入”转成“可验证的状态变更”，信任边界也收紧到可控制的范围内。对双人原型而言，这一步意味着客户端不再提交“命中判定”“数值扣减”“胜负结果”这些计算结果，而是提交可校验的输入；真正改变共享状态的动作只发生在服务器的权威时间线上。

持久化：关键状态的保存与恢复 仲裁提交之后，状态还会遇到另一个落点问题：哪些只留在内存，哪些必须持久化。实时交互中的临时坐标、tick 内的临时变量可以只存在内存；而用户标识、配置、累计值等跨会话信息如果不落盘，一次服务重启就会使这些信息丢失，这在真实在线世界里不可接受。持久化同样服从信任边界：把存档交给客户端等同于放弃可审计性；关键资产应由服务器端保存。

存储层的职责是持久保存已经提交的关键状态，使服务重启或用户重连后能够恢复。为了识别同一业务请求的重复提交，请求通常携带稳定的唯一编号。请求编号与幂等处理的完整机制将在第 2.5.5 节展开。下面的存储接口只负责保存和加载服务器已经提交的结果，不判断请求是否重复。至于底层是 SQLite、PostgreSQL 还是云数据库，都不影响下面这层统一化接口。

持久化存储接口

```
1 type StoredEntity struct {
2     ID      string
3     Status  int
4     Data    []byte
5 }
6
7 type Store interface {
8     Load(id string) (*StoredEntity, error)
9     Save(e StoredEntity) error
10 }
```

当状态仲裁模块提交一次重要状态变更时，服务器调用 `Save` 写入持久化存储；客户端重连或重新登录时，再调用 `Load` 恢复必要字段。重复请求的识别以及是否复用已有结果，由状态仲裁层处理，不属于 `Store` 接口的职责。持久化的频率与方式本身是一种取舍：同步写入最直观，但会引入 I/O 延迟；更大规模系统通常会采用批量写回、异步写回与缓存层，第 3.3.3 节会继续讨论热数据、冷数据与缓存分层。

状态仲裁解决了输入“能否生效”的问题，持久化解决了关键结果“能否保留”的问题。但仅靠服务器内部提交和落盘还不够：裁决结果最终要回到客户端画面上，用户才能看到自己的操作产生了什么效果。

2.4.4 状态分发与副本收敛

状态仲裁把输入事件提交成权威状态之后，这些结果还需要到达客户端才能呈现在用户界面上。状态分发负责把服务器已经确认的结果以合适的范围、节奏和版本方式送回客户端。服务器算得再正确，如果客户端迟迟收不到，用户看到的仍然是旧版本。副本收敛要做的，就是让客户端本地保存的状态副本不断向服务器的权威状态靠拢。

状态分发首先要分清三种节奏。服务器的权威状态推进频率决定多久产生一份新的权威版本；状态发送频率决定多久向客户端推送一次状态更新；客户端的界面刷新频率决定用户多久看到一帧新的画面。这三个频率可以独立设置。例如，服务器以每秒 20 次的节奏推进权威状态、每秒发送 10 次状态更新，而客户端以每秒 60 帧的速度刷新界面；一次更新可以汇总多次状态变化，客户端则在相邻两次更新之间绘制多帧过渡画面。

在这个基础上，状态分发需要依次解决四个问题：发给谁、发什么、怎么衔接版本、客户端怎么显示。

发给谁，取决于客户端当前需要知道哪些实体的状态。服务器不会向所有客户端发送同一份全局状态，而是为每个客户端构造一份可见范围内的状态快照或增量。这样做有两个好处：一是减少不必要的网络传输，二是限制客户端获得与当前交互无关的信息。在在线游戏等空间化场景中，这种按可见范围筛选的机制常称为感兴趣区域（Area of Interest, AOI）过滤；在协作文档中，则可能体现为按光标位置或章节发送相关内容。

发什么，需要在全量快照和增量更新之间选择。全量快照包含一个实体或一片区域的完整状态，适合初次同步、重连恢复或纠错时使用；增量更新只发送自上一个已确认版本以来变化过的字段，适合稳定追踪时持续推送。为了让客户端知道增量应该接在哪个版本之后，每次更新通常会携带服务器 tick、版本号或序列号。客户端如果发现中间版本缺失，就不能盲目套用增量，而应请求全量快照或等待下一份可独立解释的完整状态。

版本衔接保证的是客户端收到的更新能够正确拼接到已有副本上。使用已确认版本作为增量的起点，会重复发送尚未确认的变化，但不会让客户端跨过自己尚未收到的版本。发送频率、队列容量和全量快照的成本，需要根据系统的实际负载和带宽条件来确定。

客户端侧的显示策略，则是在“及时反馈”和“最终一致”之间取得平衡。本地用户的操作可以立即在界面上呈现，以获得及时的操作感；但这种呈现只是预测，权威结果仍需等待服务器确认后才能确定。远端对象的状态更新到达后，客户端通常不会直接跳到新位置，而是在新旧两个权威版本之间做平滑过渡，以减少画面跳变。服务器的权威状态到达后，客户端需要用权威版本校正本地预测出的偏差，使本地副本最终与服务器保持一致。

服务器执行状态分发时，需要为每个客户端分别选择可见实体，并依据该客户端已经确认处理的状态版本生成快照或增量。状态分发主循环只负责生成更新并放入有界发送队列，真正的网络写入由每个连接的独立写流程完成。以下以游戏场景为例，给出状态分发的伪代码示意：

状态分发伪代码示例（游戏场景）

```
1 func queueGameStateForClients(clients []*clientConn) {
2     for _, c := range clients {
3         visible := queryVisibleEntities(world, c.playerID)
4         snapshot := buildDeltaForClient(c.playerID, visible, c.lastAked)
5         snapshot.Tick = world.CurrentTick
6         snapshot.Session = currentBattleSession()
7         snapshot.Events = collectPublicEvents(c.playerID)
8
9         if !c.sendQueue.TryPush(snapshot) {
10             // 队列已满：丢弃过期增量，改放一份可独立解释的全量快照
11             c.sendQueue.ReplaceWith(buildFullSnapshotForClient(c))
12         }
13     }
14 }
15
16 func runStateWriter(c *clientConn) {
17     for snapshot := range c.sendQueue.Items() {
18         _ = c.conn.SetWriteDeadline(time.Now().Add(writeTimeout))
19         if err := sendGameStateToClient(c.conn, snapshot); err != nil {
20             markConnectionAbnormal(c)
21             return
22         }
23         c.lastSent = snapshot.Version
24     }
25 }
26
27 func handleSnapshotAck(c *clientConn, version uint64) {
28     if version > c.lastAked && version <= c.lastSent {
29         c.lastAked = version
30     }
31 }
```

其中，`visible` 保存该客户端当前可见的实体，`buildDeltaForClient` 根据客户端已经确认的 `lastAked` 生成状态变化；随后写入服务器 tick、当前会话和公共事件，再把结果放入发送队列。

`runStateWriter` 为每个连接串行发送完整更新，并通过写截止时间限制慢连接占用写流程的时间；只有本地写入成功后才推进 `lastSent`。客户端成功应用状态后返回版本确认，服务器再由 `handleSnapshotAck` 推进 `lastAked`；确认版本不得超过服务器实际发送过的 `lastSent`，

避免客户端伪造未来版本。如果发送队列已满，示例用新的全量快照替换过期增量，使后续状态不依赖已经丢弃的中间版本。第 2.5 节将继续说明写入成功与应用层确认的区别。

实时交互场景的细化策略 在对延迟和画面连续性要求较高的实时交互场景中，上述基本策略还会进一步细化。感兴趣区域过滤需要精确到每个实体的可见范围，客户端预测需要区分本地操作和远端观测，实体插值需要设置合理的缓冲时长来吸收网络抖动。对操作时序敏感的场景还会引入延迟补偿 (lag compensation)：服务器保存最近一小段时间的权威历史状态，收到操作请求后在受限窗口内回看当时的状态，再判断操作是否在有效条件内发生。这样做可以改善用户的操作感受，但必须受到两项限制：回看所依据的状态必须来自服务器保存的权威历史，回看时长也不得超过预设上限，以防止客户端伪造历史状态或利用高延迟获得优势。

这些机制都依赖同一条时间线。为了减小不同机器之间的时钟偏差，真实系统可以采用网络时间协议 (Network Time Protocol, NTP) 进行时间同步，但权威裁决仍不能直接相信客户端本地时钟。更稳妥的做法，是以服务器 tick 和服务器接收窗口为准，客户端时间只作为辅助线索。状态分发与副本收敛真正要保证的，是所有关键事件最终落到服务器认可的逻辑时间线上；物理时钟对齐只能降低误差，不能替代服务器裁决。

状态分发与副本收敛补全了服务器到客户端的返回路径：服务器不只提交结果，还要把结果以合适的范围、版本和节奏送回客户端。客户端本地副本因此可以持续向权威状态靠拢，而不是停留在各自预测出的局部画面里。在实时游戏场景中，这组机制通常被称为“世界同步”，承担的也是同样的职责。

2.4.5 三模块协同：贯通服务器处理路径

至此，连接管理、状态仲裁和状态分发共同构成了服务器的完整处理流程：连接管理将网络输入转换为队列中的事件，状态仲裁将事件提交为权威状态，状态分发再将权威状态发送给客户端。它们不能消除延迟和作弊企图，但能让问题有明确归属：连接是否还存在，输入是否可生效，客户端副本是否已经追上权威状态。双人原型也由此从“能互相发送消息”推进到“能由服务器完成一轮权威处理，并让客户端依据确认后的状态继续交互”。

不过，上述职责划分只回答了“由谁接收输入、提交结果和同步状态”，暂时假设消息能够及时到达，并且发送方能够知道对方是否已经完成处理。真实网络并不提供这样的保证：一次写入成功可能只表示字节进入本机缓冲区，连接中断也未必能被立即发现，超时重试还可能让同一请求重复执行。因此，确定权威结果由谁提交之后，还要继续解决请求和结果怎样被业务双方可靠确认。

2.5 从可靠字节流到可确认的业务结果

本节继续追踪一条消息从应用程序写入到业务处理完成的路径，回答四个问题：连续的小量写入何时进入网络，发送操作成功能够证明字节到达了哪里，连接长时间没有响应时怎样发现异常，超时重试时怎样避免同一业务操作重复生效。这四个问题来自同一个根源：网络通信不能沿

用单机函数调用的假设。工程实践通常把“网络是可靠的”“延迟为零”“带宽无限”等错误假设概括为**分布式计算的八大谬误** (Fallacies of Distributed Computing)²。本节不展开完整列表，而是通过等待、断连、带宽开销和重复请求，检验其中与当前通信路径直接相关的假设。回答这些问题，需要重新检查前面用于 P2P 锁步同步和 CS 权威服务器的 TCP 通道。TCP 向应用程序提供**可靠、有序的字节流**，并通过**流量控制**避免接收端缓冲区被持续写满，通过**拥塞控制**在共享网络出现排队或丢包时调整发送速率。可靠性意味着丢失的字节会被重传，有序性意味着接收端只能按原始顺序读取字节，不能跳过中间缺口直接取得后续数据。

这些机制为当前 CS 原型提供了稳定的字节传输，长度前缀又使客户端和服务端能够从字节流中识别完整消息。到这里，系统仍然无法仅凭 TCP 判断连接是否继续可用、业务消息是否已经处理，以及重复发送是否会造成重复修改。在检查端到端的处理结果之前，还要先看消息是否已经离开发送端：应用程序完成一次写入后，操作系统仍要决定何时把缓冲区中的字节组成 TCP 数据段并发出。

2.5.1 小消息何时发出：Nagle 算法与时延取舍

应用程序调用 Write 后，写入的字节先进入操作系统的 TCP 发送缓冲区，再由 TCP 组成数据段并发送到网络。如果程序连续写入许多很短的消息，并且每次写入都立即产生一个 TCP 数据段，协议首部、确认报文和内核处理带来的开销可能远大于有效数据本身。发送端因而在“尽快发出当前数据”和“减少小数据段数量”之间作出取舍。

许多 TCP 实现提供 Nagle 算法来处理连续的小量写入。该算法由 John Nagle 在 RFC 896 中提出，其基本规则是：如果连接上没有已经发出但尚未收到对端 TCP 确认报文 (ACK) 的数据，新数据可以立即发送；如果发送缓冲区中仍保留着已发送但未确认的数据，后续的小量写入就先暂存在缓冲区中，直到发送端收到相应的 ACK，或者缓冲的数据已经足以组成一个较大的数据段。³

这里的 ACK 由对端 TCP 协议栈返回，只确认相应字节已经到达对端传输层，并不表示对端应用程序已经处理了这些字节。Nagle 算法可以减少连续小写入产生的数据段数量，但等待 TCP ACK 也可能增加后续小消息的发送时延。

位置更新、状态同步和交互指令等消息往往需要及时到达；当连接频繁产生这类小写入时，等待前一批数据确认可能使反馈变慢。这种等待不仅来自 Nagle 算法本身。TCP 的延迟确认机制 (Delayed ACK) 可以短暂等待后续数据或可捎带的反向数据再回复 ACK，实际等待时间由操作系统实现和连接状态决定，并不是固定的 40 ms⁴。Nagle 算法等待 ACK 才发新数据，延迟确认又可能等待更多数据才回复 ACK，两者叠加会放大小消息的等待时间。因此，对低时延优先的连接，

²对经典八项错误假设及其工程影响的列表，可参见 Microsoft Learn, *Smart Client—Building Distributed Apps with NHibernate and Rhino Service Bus*, 2010, <https://learn.microsoft.com/en-us/archive/msdn-magazine/2010/july/smart-client-building-distributed-apps-with-nhibernate-and-rhino-service-bus>。

³John Nagle, *Congestion Control in IP/TCP Internetworks*, RFC 896, 1984。该文提出了后来被称为 Nagle 算法的小数据段发送方法。

⁴TCP 对延迟确认的约束见 RFC 9293, *Transmission Control Protocol*, 第 3.8.6.3 节, <https://www.rfc-editor.org/rfc/rfc9293>。

通常选择关闭 Nagle 算法，使小量写入不必等待先前数据的确认。在 Go 中，`SetNoDelay(true)` 用于关闭这项等待；Go 创建的 `TCPConn` 默认已采用该设置，显式调用可以表明这里选择了低时延优先的发送策略：

降低小消息等待的 TCP 设置（概念示例）

```
1 if tcpConn, ok := conn.(*net.TCPConn); ok {  
2     _ = tcpConn.SetNoDelay(true)  
3 }
```

因此，是否关闭 Nagle 算法取决于通信目标：交互消息优先缩短等待时间，批量数据则优先减少数据段数量和处理开销。无论如何选择，它只改变发送端组织小数据段的方式，既不改变 TCP 的字节流语义，也不能证明消息已经到达并被对端处理。

2.5.2 写入成功能确认什么：从本机缓冲到业务生效

判断一条消息是否已经完成传递，需要区分本机写入、对端 TCP 接收和对端应用处理三个阶段。这三个阶段由不同组件完成，前一阶段成功不能证明后一阶段也已完成。

应用程序调用 `Write(data)` 时，首先把字节交给本机操作系统，并得到已写入字节数 `n` 和错误 `err`。只有当 `n == len(data)` 且 `err == nil` 时，这批数据才算完整写入本机 TCP 发送路径；如果 `n` 小于数据长度，则只有前 `n` 个字节被接收，程序还需要处理返回的错误和未写入的数据。即使完整写入成功，这些字节也可能仍在发送缓冲区中，尚未通过网络到达对端。

字节经过网络到达对端后，对端 TCP 会对已经接收的字节返回确认，即 TCP ACK。这个确认表示相应字节已到达对端 TCP 接收端，但不能说明应用程序已经读取这些字节。字节可能仍停留在接收缓冲区中，等待接收端程序调用 `Read` 读取；该程序随后还要解析消息、检查请求并执行相应操作，业务结果才会形成。

以“创建会话”请求为例，即使对端 TCP 已经确认收到请求字节，对端应用程序仍可能在读取或建立会话之前异常退出。此时传输层已经完成确认，业务层却没有产生有效结果。只有对端应用程序能够判断请求是否处理完成，这体现了**端到端论证**（end-to-end argument）⁵的逻辑：传输层可以保证其职责范围内的字节可靠、有序到达，但不能代替通信端点验证最终结果。

因此，写入返回成功只能表示数据已经完整交给本机 TCP（写入端成功）；它不能证明数据已到达对端，更不能证明业务处理已经完成，最终结果仍需由对端应用程序确认。

2.5.3 读取应等待多久：截止时间与超时处理

对 socket 的 `Read` 默认采用阻塞方式；如果暂时没有数据可读，调用它的执行流程就会继续等待。当网络读取与状态更新位于同一条顺序执行路径中时，这种等待还会延迟该路径上的后续处理。因此，应用程序需要为一次读取规定最长等待时间，使网络等待能够在确定的时刻返回给上层处理。

⁵端到端论证由 Saltzer、Reed 和 Clark 在 1984 年提出，见 *End-to-End Arguments in System Design*, *ACM Transactions on Computer Systems*, 2(4):277–288。

达到等待上限只说明程序没有在规定时间内读到数据，不能证明对端已经失效。相同的读取超时既可能由连接中断引起，也可能由排队、丢包重传或临时拥塞造成。应用程序应当把超时视为一种需要分类处理的运行状态，再结合消息类型、连续超时次数和最近一次有效响应决定后续动作，而不能在第一次超时后直接判定对端不可达。

Go 通过读取截止时间限制阻塞读取的持续时间。程序在调用 `Read` 前使用 `conn.SetReadDeadline` 设置绝对截止时刻；如果截止时刻到达时仍无数据可读，`Read` 返回超时错误，控制权重新交给应用程序。前文的 `recvFrame` 已经在读取消息头和负载前设置了截止时间。连接读取流程调用该函数后，还需要区分超时错误与其他读写错误，再把不同结果交给上层处理：

读取超时的识别与上报（概念示例）

```
1 func receiveOnce(conn net.Conn) error {
2     frame, err := recvFrame(conn, readTimeout)
3     if err == nil {
4         enqueueFrame(frame)
5         return nil
6     }
7     if errors.Is(err, ErrBrokenFrame) {
8         return ErrReconnectRequired
9     }
10    if errors.Is(err, ErrFrameTooLarge) {
11        return ErrProtocolViolation
12    }
13    if netErr, ok := err.(net.Error); ok && netErr.Timeout() {
14        return ErrReadTimeout
15    }
16    return err
17 }
```

`enqueueFrame` 只接收已经完整读取的消息。只有尚未消费任何帧字节的超时才作为 `ErrReadTimeout` 报告给上层，此时可以设置新的截止时间后继续等待；若 `recvFrame` 返回 `ErrBrokenFrame`，说明当前无状态解码函数已经丢失半条帧的读取进度，调用方必须进入重连流程，不能在原连接上重新从消息头开始读取；超过长度上限则属于 `ErrProtocolViolation`，服务器应拒绝消息并关闭连接。调用方再结合消息类型和连接健康状态决定继续等待、重试业务请求或重建连接。截止时间在触发后仍然有效，因此后续读取需要设置新的截止时刻，或者在允许长期等待时清除原有设置。

读取截止时间与 TCP 的重传超时（Retransmission Timeout, RTO）解决不同问题。RTO 位于传输层，用于决定发送端在未收到 TCP ACK 时何时重传数据段；读取截止时间位于应用层，用于限制当前读取操作最多等待多久。即使 TCP 仍在重传，应用层的读取也可能先达到截止时间。对实时状态更新，超时后可以暂时沿用上一份有效状态，等待下一次更新；对登录、结

算等关键操作，则可以进入有限次数的重试或重连流程。这样，系统既不会因一次读取无限等待，也不会把一次短暂延迟直接当作连接失效。

2.5.4 连接是否仍然可用：半开连接与健康检测

TCP 通常通过 FIN 或 RST 报文感知对端关闭连接。如果对端主机突然断电，或者中间网络中断使这些报文无法到达，本端在探测到故障之前仍可能保留原有连接状态。此时 socket 句柄仍然存在，Write 也可能先把数据写入本机发送缓冲区，但对端已经无法继续通信。这种两端连接状态不一致的情况称为**半开连接**。

半开连接会使服务端继续等待输入或发送状态，并保留对应的会话记录、发送缓冲区和资源配额。连接对象仍然存在或最近一次 Write 调用成功，都不能证明对端仍能接收并处理数据；系统还需要主动检查连接能否继续完成双向通信。

操作系统提供的 TCP keepalive 可以在连接空闲一段时间后发送探测报文，用于判断对端 TCP 是否仍可响应。它工作在传输层，探测时机由操作系统参数控制，也不能判断对端应用程序是否仍在正常处理请求。因此，仅依靠 TCP 的机制不能满足所有在线系统的连接健康判断要求。

当系统需要按照自身的交互周期判断对端状态时，可以让两端定期交换轻量级探测消息，并基于超时机制记录最近一次收到有效响应的时间，这就是**应用层心跳机制**。服务器定期发送 Ping，客户端收到后立即返回 Pong；服务器只有在收到 Pong 后才更新该连接的最后响应时间。如果超过设定时长仍未收到响应，就将连接标记为异常。以下代码给出主动检查的基本过程：

应用层心跳与连接健康检查（概念示例）

```
1  const (
2      heartbeatInterval = 5 * time.Second
3      heartbeatTimeout  = 15 * time.Second
4  )
5
6  func sendControlMessage(conn net.Conn, kind string) error {
7      return sendFrame(conn, []byte(kind))
8  }
9
10 // 建立连接后启动独立的周期检查流程
11 func runHealthCheck(c *clientConn) {
12     c.lastPongAt = time.Now()
13     ticker := time.NewTicker(heartbeatInterval)
14     defer ticker.Stop()
15
16     for now := range ticker.C {
17         if now.Sub(c.lastPongAt) > heartbeatTimeout {
18             markConnectionAbnormal(c)
19             return
20         }
21         if err := sendControlMessage(c.conn, "ping"); err != nil {
22             markConnectionAbnormal(c)
23             return
24         }
25     }
26 }
27
28 // 客户端收到 Ping 后返回 Pong
29 func handlePing(conn net.Conn) error {
30     return sendControlMessage(conn, "pong")
31 }
32
33 // 服务端收到 Pong 后更新最后响应时间
34 func handlePong(c *clientConn) {
35     now := time.Now()
36     c.lastPongAt = now
37     c.lastReceivedAt = now
38 }
```

`sendControlMessage` 沿用前文的消息分帧方法，在现有 TCP 连接上发送心跳控制消息。概念代码省略了入队动作；在实际连接管理中，心跳、状态快照和业务确认都应进入该连接的同一条写入流程，避免多个执行流程同时写入而破坏帧边界。`heartbeatInterval` 决定服务器主动发起检查的频率，`heartbeatTimeout` 规定允许连续收不到响应的最长时间。发送 Ping 失败，或者最后一次 Pong 已经超过该时间，都会使连接进入异常状态。这里的 `lastPongAt` 只记录主动探测的响应，`lastReceivedAt` 则记录任意合法入站消息；分开维护可以避免把“最近收到过业务消息”和“最近一次心跳探测已经成功”混为同一个判断。异常状态只表示系统在规定时间内无法确认连接健康；服务器可以先停止接受新的关键操作并保留重连时间，超过重连时间后再释放会话和相关资源。

因此，连接健康的判断依据不是“socket 是否存在”，而是“是否在规定时间内收到符合协议的有效响应”。超时机制提供了等待时间的上限，心跳和健康检测则在此基础上给出连接是否仍然可用的判断。

2.5.5 业务是否生效：应用层确认与幂等处理

前文区分了本机写入、对端 TCP 接收和对端应用处理三个阶段。这一区分带来一个直接问题：发送方如何判断请求已经产生了预期的业务结果。坐标、状态值等周期性信息可以由后续更新覆盖；会话创建、结算确认和关键状态变更等操作则必须得到明确结果，否则客户端无法决定是否继续后续流程。

业务处理完成后，由接收方返回包含请求编号和处理结果的确认消息，这种机制称为**应用层确认**（application-level ACK）。发送方发出请求后将其标记为待确认，只有收到编号匹配的确认消息，才将该请求标记为完成。TCP ACK 由接收端内核产生，应用层确认则由业务程序在处理请求后产生，二者分别证明字节到达和业务处理这两个不同阶段。

如果发送方在规定时间内没有收到应用层确认，它无法仅凭超时判断请求停在哪一步：请求可能尚未到达，可能仍在处理中，也可能已经处理完成但确认消息丢失。为了从前两种情况中恢复，发送方通常需要重发请求；然而在第三种情况下，重发会使已经完成的操作再次到达服务器。例如，结算请求已经执行，但确认消息在返回途中丢失；若服务器把重发请求当作新操作再次执行，就会产生重复处理。

为避免重复副作用，发送方应为每项关键操作分配唯一请求编号，并在重试时保持编号不变。接收方保存已经处理的请求编号及其结果；相同编号再次到达时，接收方直接返回原结果，不再执行状态变更。这种使同一请求执行一次或多次都产生相同业务效果的机制称为**幂等处理**（idempotency）。

以客户端请求创建会话为例，这套机制需要客户端和服务端遵循两项相互配合的规则：客户端超时后重发同一编号的请求，服务器则依据该编号判断请求是首次到达还是重复到达。以下伪代码给出客户端等待确认和服务端返回既有结果的基本过程。

端到端确认与重复请求处理（概念示例）

```
1 // 客户端：重试时保持请求编号不变
2 func createSession() error {
3     req := newCreateSessionRequest() // req.ID 在每次新请求中唯一
4
5     for retry := 0; retry < maxRetries; retry++ {
6         if err := send(req); err != nil {
7             continue
8         }
9         ack, err := waitAck(req.ID, timeout)
10        if err == nil {
11            if ack.Status == "accepted" {
12                return nil
13            }
14            return ErrRejected
15        }
16    }
17    return ErrUnconfirmed
18 }
19
20 // 服务器：重复请求返回首次处理时保存的结果
21 func handleCreateSession(req Request) {
22     if result, ok := findHandledResult(req.ID); ok {
23         sendAck(req.ID, result)
24         return
25     }
26
27     result := processCreateSession(req)
28     saveHandledResult(req.ID, result)
29     sendAck(req.ID, result)
30 }
```

在这条路径中，`send` 成功仍然只表示字节已交给本机内核。客户端收到与请求编号匹配的 `accepted` 结果后，才能确认服务器已经建立会话；收到拒绝结果时则停止重试，并把业务失败返回给上层。`findHandledResult` 使服务器能够识别重复请求，`saveHandledResult` 则保存首次处理的结果，保证重发请求获得相同响应。伪代码省略了业务状态变更与请求结果记录之间的原子提交（两项操作要么同时生效，要么都不生效），以及服务器故障后的记录恢复；这些状态管理问题将在后续章节继续讨论。

在可能丢失消息并触发重试的网络中，仅靠发送和确认不能保证业务操作恰好执行一次。常

见做法是允许请求至少到达一次，再通过稳定的请求编号、结果记录和幂等处理，使重复到达不会重复产生业务效果。

2.5.6 通信逻辑如何复用：连接通道封装

分帧、读写超时和心跳检测并不只服务于某一种业务消息。如果会话请求、状态同步和结算确认的业务代码分别实现这些逻辑，不同处理路径可能采用不同的消息格式、等待时间和连接状态判断，增加维护和排查故障的难度。因此，可以在 TCP 字节流与业务处理之间增加连接通道，集中完成消息编解码、分帧收发、超时控制和响应时间记录。

连接通道只处理消息如何传递，不判断消息产生了什么业务结果。请求编号和确认消息可以通过连接通道传输，但等待哪一项确认、重复请求是否再次执行，仍由业务层决定。下面的概念代码用统一消息结构承载消息类型、请求编号和业务负载，并复用前文定义的 `sendFrame` 与 `recvFrame` 完成收发。

连接通道封装（概念示例）

```
1  type Message struct {
2      Type      byte
3      RequestID uint64
4      Payload   []byte
5  }
6
7  type ReliableConn struct {
8      net.Conn
9      lastPong time.Time
10 }
11
12 func (c *ReliableConn) Send(msg Message) error {
13     data, err := json.Marshal(msg)
14     if err != nil { return err }
15     return sendFrame(c.Conn, data)
16 }
17
18 func (c *ReliableConn) Recv(timeout time.Duration) (*Message, error) {
19     data, err := recvFrame(c.Conn, timeout)
20     if err != nil { return nil, err }
21
22     var msg Message
23     if err := json.Unmarshal(data, &msg); err != nil {
24         return nil, err
25     }
26     return &msg, nil
27 }
28
29 func (c *ReliableConn) RecordPong() {
30     c.lastPong = time.Now()
31 }
32
33 func (c *ReliableConn) Healthy(interval time.Duration) bool {
34     return time.Since(c.lastPong) < interval
35 }
```

`Send` 先把完整消息序列化，再交给 `sendFrame` 添加长度前缀；`Recv` 通过 `recvFrame` 限时读取完整帧，再将字节还原为消息结构。收到 `Pong` 消息后，连接管理逻辑调用 `RecordPong` 更

新时间；Healthy 只判断最近一次有效响应是否仍在允许时间内，不能保证连接之后一定可用。该封装只展示连接通道的职责，并不自动保证并发调用安全：每条连接仍应由单一写入流程调用 Send；若多个执行流程共享心跳状态，还需要使用下一章介绍的同步机制保护 lastPong。

RequestID 由连接通道原样传递，其业务含义仍由上层解释：客户端用它匹配确认消息，服务器用它查找已经处理的请求结果。这样，连接通道负责“怎样传递一条完整消息”，业务层负责“消息是否已经处理并生效”。

由此可以区分三层职责：TCP 在连接有效期间向应用程序提供有序字节流；连接通道恢复消息边界、限制等待时间并记录连接响应；业务层通过请求编号、应用层确认和幂等处理判断关键操作是否已经生效。这种分层没有改变 TCP 的按序交付语义。当较早的字节丢失时，后续字节仍需等待缺口恢复；下一小节将分析这种等待对不同消息的影响。

2.5.7 前序数据丢失会怎样：队头阻塞

TCP 按照字节在数据流中的顺序向应用程序交付数据。当后续内容依赖前序内容时，这种语义可以避免应用程序读到顺序颠倒的数据；相应的代价是，只要前面存在尚未恢复的字节缺口，后面已经到达的字节也不能提前交给应用程序。这段等待是否可以接受，取决于数据过期后是否仍有价值。会话创建、购买和结算等关键操作不能因为等待时间较长而跳过前序结果；坐标和状态值等高频信息则会持续更新，新值到达后，旧值通常已经失去使用价值。

设服务器依次发送状态更新 A、B、C，分别表示第 1、2、3 个版本的状态。应用程序把三条更新写入同一条 TCP 字节流后，TCP 实际传输的是承载这些更新的连续字节，而不是三份彼此独立的消息。如果承载 A 的部分字节在传输中丢失，接收端 TCP 可以暂存随后到达的 B、C 对应字节，但不能越过序号缺口把它们交给应用程序。发送端重传缺失的数据段并填补缺口后，接收端才能继续交付连续字节，连接通道随后才能解析出 A、B、C。这种由前序数据缺失而阻塞后续数据交付的现象称为**队头阻塞 (Head-of-Line Blocking, HOL)**。

在缺口恢复之前，客户端无法从这条连接取得 B、C 中的新权威状态，只能继续使用上一份已交付状态，或者显示本地预测和插值结果。对于必须完整处理的关键操作，等待重传是维持正确性的必要代价；对于能够被新状态覆盖的高频更新，等待过期数据会扩大一次丢失造成的延迟。要让后续更新不受前序缺口阻塞，需要采用彼此独立的交付单位。下一节将对比 TCP 的按序字节流与 UDP 的独立数据报，并给出两种交付方式的结构示意。

2.6 从按序等待到独立交付：TCP 与 UDP 的取舍

沿用上一节 A、B、C 的状态更新场景，TCP 必须先恢复承载 A 的字节缺口，应用程序才能继续取得 B、C。如果应用更关心最新状态，而不需要等待已经过期的旧状态，就需要让每次更新成为可以独立交付的单位。UDP 提供的正是这种**数据报**语义：应用程序每调用一次发送操作，就产生一份独立数据报；接收端也以数据报为单位取得数据，不需要先把它们拼成一条连续字节流。

在 UDP 路径中，A、B、C 分别放入三份数据报。若 A 数据报丢失，已经到达的 B 和 C 仍

可独立交给应用程序，客户端因而可以直接采用较新的 C 更新状态。这种及时性并非没有代价：UDP 不会自动补发 A，也不保证 B、C 按发送顺序到达。接收端需要依据序列号识别新旧状态，丢弃晚到的旧数据；确实不能丢失的消息则需要由应用层另行确认和重传。

两种交付方式最终改变的是应用程序取得新状态的时机：TCP 必须先恢复连续字节，UDP 则可以先处理已经到达的后续数据报。应用层再依据消息是否允许丢失，决定用新状态覆盖旧状态，或者为关键事件增加确认与重传；图 15 概括了这条从传输语义到应用决策的关系。

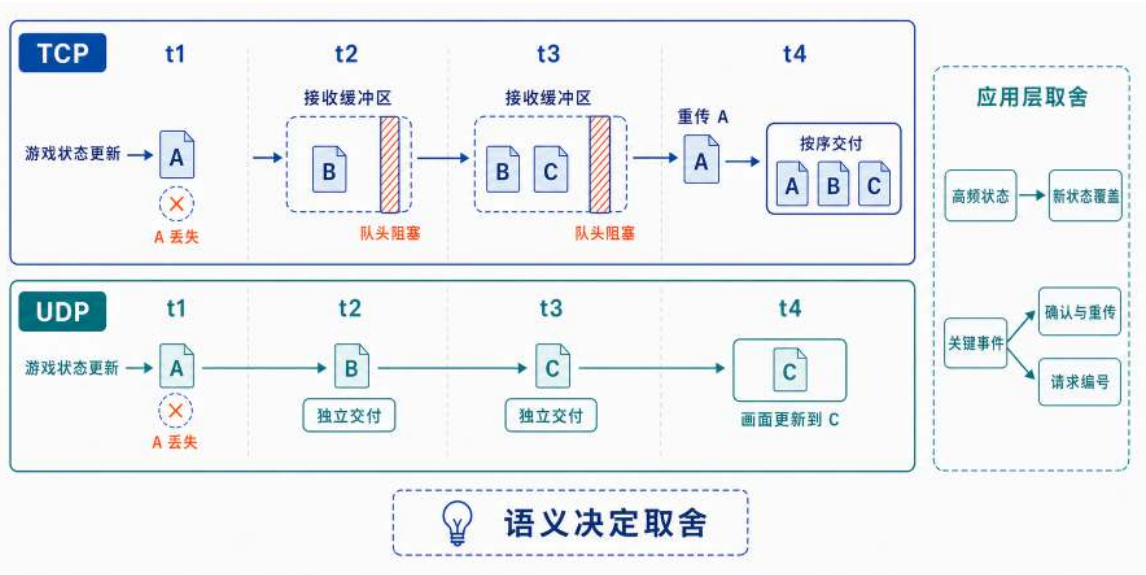


图 15: 前序数据缺失后的两种交付方式：A、B、C 表示应用层状态更新；TCP 填补承载 A 的字节缺口后才能继续按序交付，UDP 则可将已经到达的 B、C 数据报独立交给应用程序。

TCP 与 UDP 的差别并不是简单的理解为“可靠”与“不可靠”。TCP 在传输层统一实现丢失重传和按序交付，应用程序得到的是连续字节流；UDP 保留数据报之间的独立性，把是否重传、如何排序以及哪些消息可以丢失的决定更多交给应用层。选择传输方式前，应先判断一条消息过期后是否仍有价值，以及它的丢失是否会破坏业务规则。

坐标、状态值等高频信息通常符合“越新越有用”的特点。这类消息可以使用 UDP 或基于 UDP 的协议，并携带序列号；接收端保留较新的状态，忽略已经过期的旧状态。会话创建、关键操作和结算确认等关键事件则必须得到明确结果。它们可以通过 TCP 传输，也可以在 UDP 之上增加应用层确认、有限重传、请求编号和幂等处理；关键不在于协议名称，而在于最终交付语义是否满足业务要求。

独立交付也要求应用程序承担更多控制责任。第一，UDP 数据报可能丢失、重复或乱序，接收端必须使用序列号识别重复和过期消息。第二，过大的数据报可能触发 IP 分片，其中任一分片丢失都会使整份数据报无法交付，因此高频状态更新应控制单个数据报的大小。第三，UDP 不自带拥塞控制；发送方仍需限制发送速率，并在排队和丢包增加时主动减速。因此，UDP 避免了单条字节流中的队头阻塞，却没有消除丢包、拥塞和应用层可靠性问题。

选读：浏览器接人与多流传输 基于浏览器的应用通常使用 WebSocket 建立持续双向通道。WebSocket 在一条 TCP 连接上提供消息化的双向通信，WSS 再使用 TLS 保护传输过程。⁶由于底层仍是 TCP，前序字节缺失造成的等待不会因此消失；TLS 也只能保护传输内容，不能替服务器判断客户端上报的操作是否合法。

HTTP/2 在一条 TCP 连接上复用多条应用流，但底层字节缺口仍可能影响这些流；QUIC 则以 UDP 为承载，在其上实现加密连接、可靠传输、拥塞控制和多条独立流，使一个流的缺失数据通常不阻塞其他流。⁷这些协议改变了可靠性与顺序性的控制粒度，但仍不能替应用层决定哪些业务消息可以丢失、哪些必须确认。

本章的最小双人系统继续使用一条 TCP 通道承载全部消息，并在其上加入分帧、超时、心跳和端到端确认。UDP 在这里提供后续优化的判断标准：高频状态更重视及时性时，可以迁移到独立数据报；关键事件无论采用哪种传输方式，都必须保留可确认、可重试和可去重的端到端语义。

2.7 最小在线交互的完整流程

前几节已经依次建立通信通道、P2P 试验原型和 CS 权威服务器，并补充了超时、心跳、确认与幂等处理。这些机制不是零散的知识点，而是围绕同一条处理路径组织起来的。从用户发起操作到双方看到一致的结果，整个过程可以归纳为四个环节：输入、提交、同步和校正。

输入环节，用户的操作意图被封装为结构化消息，经过分帧、编号后通过网络进入服务器的处理队列。提交环节，服务器在自身维护的权威状态上校验操作的合法性，按既定顺序执行状态变更，并将结果写入持久化存储。只有提交完成的结果才算真正生效。同步环节，服务器将已提交的状态以快照或增量的方式，按可见范围和发送节奏分发给各个客户端。校正环节，客户端用收到的权威状态更新本地副本，修正此前预测产生的偏差，使两端最终收敛到一致的状态。

这四个环节共同构成了所有带客户端副本的共享状态系统的基本闭环。无论是协作文档、交易系统还是在线游戏，核心流程都可以对应到这四个环节上，区别只在于业务规则和状态对象不同。

双人在线游戏中的一次完整交互，可以把前面讲过的机制串联成一条可追踪的完整路径。流程从两名参与者相遇开始，到发起交互、服务器裁决、结果写回和会话释放结束，依次覆盖输入、提交、同步、校正四个环节。

2.7.1 相遇与会话建立：从可见交互到权威确认

第 2.4.4 节已经说明，状态分发需要根据每个客户端的可见范围选择同步对象。双人场景可以采用最简单的距离阈值：当两名参与者之间的距离小于设定值时，服务器将对方加入各自的可

⁶WebSocket 的握手、消息分帧和双向通信机制见 RFC 6455: The WebSocket Protocol, <https://www.rfc-editor.org/rfc/rfc6455>; TLS 1.3 的安全目标与握手机制见 RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3, <https://www.rfc-editor.org/rfc/rfc8446>。

⁷HTTP/2 的多路复用机制见 RFC 9113: HTTP/2, <https://www.rfc-editor.org/rfc/rfc9113>; QUIC 的流与多路复用机制见 RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport, <https://www.rfc-editor.org/rfc/rfc9000>。

见列表，并在下一次状态同步中发送“新增可见实体”。这是 AOI 过滤在双人场景中的最小应用。

客户端收到新增实体的状态后，才在本地副本中创建对应的远端实体。网络通道负责传递状态，可见范围过滤负责选择同步对象，可见关系则以服务器保存的权威状态为准。因此，网络延迟或客户端本地数据变化只会暂时影响界面呈现，不会改变服务器认定的可见关系。

进入彼此视野只说明双方可以获取对方的状态，还不足以启动一次交互。两名参与者继续靠近并满足发起条件后，服务器可将双方标记为“允许发起交互”，再把这一状态通知客户端。客户端据此显示操作入口，但该提示只表示当前具备发起条件，不能作为交互已经建立的依据。

客户端发起交互时，只发送请求意图。服务器收到请求后，必须依据当前权威状态重新检查双方距离、参与者状态和会话占用情况；检查通过后，服务器才能创建交互会话（Session），并将双方从“自由活动”切换为“交互锁定”。这种再次检查是必要的，因为客户端显示操作入口之后，距离和会话状态都可能已经发生变化。

资格检查、会话槽占用、会话创建和参与者状态切换应当构成一次不可分割的状态迁移。否则，两个并发请求可能同时占用同一参与者。进入交互的通知不属于这次原子状态迁移：服务器应先提交会话状态，再向客户端发送通知；通知丢失时可以重发或查询，但尚未提交的会话不得提前显示为已经建立。第 3.2.1 节将进一步讨论这类需要把检查与修改合并处理的并发问题。

为处理响应丢失造成的重试，请求还需要携带唯一的 `request id`。服务器对相同标识返回同一会话结果，而不重复创建会话；客户端在收到已提交状态后再进入正式交互界面，超时则重试请求或查询会话状态。由此，“交互是否已经建立”取决于服务器提交的会话状态，而不取决于某一次通知是否成功到达。这正是输入环节中“消息可靠送达”和提交环节中“操作只生效一次”两条原则的具体体现。

2.7.2 结果裁决与结算：从规则校验到一致性写入

交互会话建立后，服务器依据会话中的权威状态处理双方输入并计算结果。参与裁决的数据包括各方当前状态、属性值和已经生效的临时效果。若规则包含随机因素，伪随机数也应由服务器生成并记录其种子或结果，使同一会话的裁决能够复查；客户端不得提供或修改决定结果的随机值。

对于带有时间条件的操作，服务器还要确定输入属于哪个裁决窗口。操作请求可以携带输入序号和客户端显示时间，但客户端时间只能作为辅助信息。服务器应以接收时间、逻辑帧和受限的历史窗口为依据：过旧请求按迟到规则处理，窗口内请求则可以使用服务器保存的历史状态进行延迟补偿。这样既能补偿正常网络延迟，又能防止客户端用自行声明的时间改写已经提交的历史。

裁决完成后，服务器更新状态值、累计值和会话状态，并生成待同步的结果事件。客户端可以预测操作反馈，却不能提交“命中”“生效”或“回避”等结果。将规则校验和结果提交集中在服务器，才能使共同状态具有唯一、可记录和可复查的裁决依据。这是提交环节的核心职责。

结算写入与会话释放 结算阶段必须先明确业务结果的提交位置。在双人单服原型中，服务器可以在一次事件处理中完成状态值、累计值和会话状态的迁移；需要在服务重启后保留的结果，

还应先写入可恢复的持久化记录。只有这些状态成功提交后，服务器才生成带有会话标识和结算版本的下行消息。客户端依据该消息展示结算结果，不再自行修改状态字段。

客户端返回 ACK，只能说明某个结算版本已经送达并被处理，不能决定结算是否成立。若 ACK 或下行消息丢失，服务器可以重发同一版本；客户端断线重连后，也应查询服务器最近提交的会话结果，而不是沿用断线前的本地画面。业务事实由服务器侧可恢复的提交记录确定，消息确认只用于判断是否需要继续发送。

服务器确认结算状态已经提交后，才能释放交互会话，将双方从”交互锁定”切回”自由活动”，并重新纳入常规的状态分发。随后的快照把新状态发送给双方及其可见范围内的其他参与者；客户端收到快照后清理已确认输入，并将本地副本校正到结算版本。若结算需要同时修改多个服务或多个存储分片，单个服务器内的状态迁移便不足以保证一致性，需要采用第 4.4.4 节讨论的分布式一致性机制。

2.7.3 闭环全景与跨场景迁移

走完整条流程可以看到，一次交互从发起到完成，始终围绕输入、提交、同步、校正四个环节展开。客户端产生输入事件，服务器对事件进行接收、排序和校验，再提交新的状态版本；服务器通过状态分发把已提交结果送回客户端，客户端再用权威版本校正本地副本的偏差。这四个环节分别回答了事件如何进入系统、结果在哪里生效、状态如何传播以及副本出现偏差后如何恢复。

同样的闭环也存在于其他共享状态系统中。以协作文档为例，用户的编辑操作是输入，服务端的合并与版本提交是提交，文档内容向所有协作者推送是同步，本地草稿与服务端版本的对齐则是校正。以交易系统为例，下单请求是输入，服务端的库存校验与扣款是提交，订单状态通知是同步，客户端页面与服务端订单状态的对齐是校正。业务对象和规则不同，但处理闭环的结构是一致的。

流程全景图 完整流程需要区分本地显示与权威提交。客户端发出操作请求后可以先显示预测反馈，但结果只有在服务器完成资格校验、会话锁定、结果裁决和状态写回后才正式生效。状态分发随后发送已提交的状态版本，客户端通过对账校正本地副本。图 16 按照最终 CS 原型展示输入意图、服务器提交和客户端收敛的先后关系；前文的 P2P 仅用于暴露双端裁决的问题，因此不进入该流程。

图中的关键区分是”消息送达”与”业务提交”。队列、重传和 ACK 可以提高消息送达的可靠性，却不能代替服务器提交业务结果；客户端预测可以缩短操作反馈时间，却必须允许后续快照纠正；持久化记录则使断线客户端能够恢复已经提交的状态。由此，输入顺序由服务器 tick 和裁决窗口确定，业务结果由服务器提交，客户端副本由状态分发和对账机制持续校正。

双人流程只包含少量连接和共享状态。扩展到多人同空间后，连接管理需要处理更多并发连接和 I/O 等待，状态仲裁需要处理同一 tick 内更密集的输入与状态访问，状态分发则需要承担更高的事件速率和可见范围计算成本。这些变化构成了第 3 章讨论并发问题的起点。

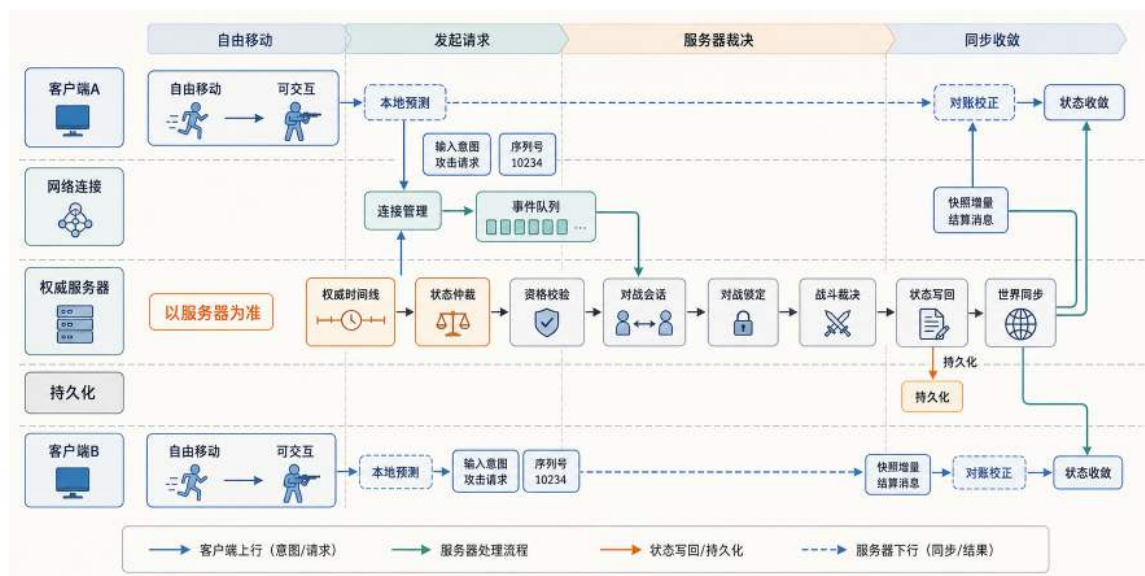


图 16: 双人交互案例的提交与收敛路径：客户端预测只改变本地副本；服务器完成裁决和状态写回后，结果才正式生效；世界同步与客户端对账使两端状态收敛到同一权威版本。

2.8 小结：网络通信与共享状态的最小闭环

本章围绕跨设备共享状态系统的三个基本问题展开：独立程序之间如何传递结构化消息，多份状态副本如何保持一致，以及操作结果如何被双方可靠确认。双人在线游戏作为贯穿案例，为这些问题提供了具体的观察场景，但讨论的机制和判断标准同样适用于协作文档、交易系统和即时通信等其他在线服务。

网络通信基础解决的是第一个问题。IP 地址和端口确定了通信端点，socket 提供了使用网络的编程接口，TCP 提供了可靠有序的字节流传输，序列化和长度前缀分帧则使应用程序能够从字节流中恢复出完整的结构化消息。在此之上，两个独立程序实例才有可能交换操作意图和状态信息。

P2P 直连和 CS 架构回答的是第二个问题。P2P 锁步输入同步展示了两端如何通过相同的输入和确定性更新获得一致的状态，同时也暴露了三个根本局限：推进速度受最慢端限制、缺少独立的状态确认位置、连接数随参与者平方增长。CS 架构通过引入权威服务器同时回应了这些约束：服务器作为唯一真相源统一校验输入和提交结果，客户端只维护用于显示和交互的本地副本。服务器内部进一步分解为连接管理、状态仲裁和状态分发三项职责，分别处理输入接收、结果提交和状态同步，客户端再依据权威版本完成校正，形成输入、提交、同步和校正四个环节。

TCP 可靠性与端到端语义解决的是第三个问题。TCP 的可靠是字节流层面的可靠，写入成功不代表消息已经到达对端应用，更不代表业务操作已经生效。半开连接、心跳检测、读取超时、应用层 ACK、request id 和幂等处理，共同把传输层的字节可靠交付提升为业务层的操作可确认、可重试、只生效一次。TCP 与 UDP 的取舍则取决于消息语义：关键操作和状态变更必须最终生效，高频状态更新则更重视及时性。

双人在线游戏的案例把这些机制串成了一条可以追踪的完整路径。沿着输入、提交、同步、

校正四个环节走一遍，读者可以看到通信机制、状态同步和端到端确认如何在一个具体场景中协同工作。下一章会把同一套闭环放到更多连接和更高消息速率下，讨论并发压力如何改变服务器的处理方式。

2.9 思考题

1. 为什么“TCP 可靠”仍然可能一次读到两条指令（或半条指令）？应用层应该补哪一层机制？
2. 锁步输入同步为什么会被最慢的一端拖慢？它适合哪些场景，不适合哪些场景？
3. 在权威服务器模式下，客户端仍然需要做哪些本地计算来降低感知延迟？哪些计算必须由服务器执行才能保证结果一致？
4. 为什么仅靠“本地检测到 close”不足以判断一条关键消息是否真的到达并生效？应该如何补齐端到端语义？
5. 在一个在线协作文档系统中，哪些状态必须持久化，哪些状态可以只存在内存？请说明理由。
6. 以电商下单为例，购物车更新、提交订单和支付确认三类消息分别更重视及时性还是最终生效？如果引入 UDP，应怎样分配传输与应用层可靠性？
7. 两名用户同时编辑同一段文档，用户 A 删除一个句子，用户 B 在同一位置插入一个段落，且 B 的操作消息晚到 120ms。请沿“分帧—连接管理—状态仲裁—状态分发—客户端校正”的路径说明服务器和客户端分别会看到什么、处理什么。
8. 在线游戏中的“世界同步”与本章讨论的“状态分发与副本收敛”是什么关系？请举出另一个领域中的同类机制。

3 第三章 英雄集结：单机并发

第二章中，我们观察了一次在线请求的完整处理过程：客户端通过网络提交输入，服务器完成校验与状态裁决，再把结果返回相关客户端并保存关键记录。这一阶段关注的是少量请求能否正确完成。当客户端和请求继续增加、系统仍然运行在一台服务器上时，即使网络读写已经与世界主循环分开，服务器仍需在有限时间内逐个完成事件校验、状态更新和结果生成，内部处理路径因而开始承受压力。

更多事件进入同一条内部处理路径后，数据库访问等慢操作和复杂规则计算会延长单个事件的处理时间，后续事件随之在队列中等待，尾延迟不断上升。为了让网络读写、外部访问和彼此独立的计算任务能够同时推进，系统需要引入并发执行；多个执行流又可能同时读写共享状态，打乱原本确定的检查与修改顺序。连接和临时对象的频繁创建、慢速存储的反复访问，还会持续消耗 CPU、内存和网络资源，使单机承载能力提前触顶。这些问题同样存在于即时通信、在线协作、电商交易和模型服务中。

本章要解决的问题是：在不增加机器数量的前提下，如何组织并发执行、保护共享状态并提高资源利用率，使在线系统能够承载更多请求，同时判断单机优化何时接近上限。首先区分等待型任务与计算型任务，说明进程、线程、协程、事件驱动与通道（channel）如何组织执行；随后讨论多个执行流访问共享状态时的竞态、互斥和等待通知；最后说明连接池、对象池和缓存分层如何减少资源开销，并用性能观测验证优化效果。本章以并发隔离等待、同步保护共享状态、资源复用降低开销为主线。在线游戏中的多人同图、非玩家角色（Non-Player Character, NPC）奖励和状态同步是贯穿全章的教学案例，但这些机制同样适用于即时通信、电商交易、协作文档和模型服务等在线系统。

3.1 从顺序处理到并发组织

低负载下，在线系统可以沿一条顺序路径依次完成接入、处理、状态访问和结果返回，代码简单且容易验证。请求数量增加后，多个连接和任务会在相近时间进入系统，顺序路径首先暴露两类不同性质的问题。第一类来自等待：某个客户端网络延迟、一次数据库查询或一次缓存访问，都可能让后续请求停在队列中。第二类来自计算：内容编码、规则计算、数据过滤等工作会持续占用 CPU，单核算力不足时也会形成积压。多人在线场景可以具体呈现这两类压力：连接读写和外部查询带来等待，碰撞检测、AOI（感兴趣区域）过滤和批量序列化消耗算力。同样的两类压力也出现在电商大促的下单峰值（订单查询与库存校验等待，反作弊规则与价格计算消耗算力）、实时协作的消息同步（消息分发等待，权限校验与渲染任务消耗算力）等场景中。等待问题需要让不同任务在时间上交错推进，计算问题则需要把可拆分的计算段分摊到多个 CPU 核上。用更概括的说法，**并发用于隔离等待传播**，让一个任务等待 I/O 时其他任务仍可推进；**并行用于分摊计算负载**，让可拆分的计算段同时使用多个 CPU 核。本节先从顺序主循环中的等待传播讲起，再区分并发与并行，并把这组分工落实到进程、线程和 goroutine 这些执行单元上。

3.1.1 顺序处理如何放大延迟

低负载下，在线系统可以用一条顺序循环依次处理请求：按到达顺序读取连接输入，完成校验与状态更新后再返回结果。由于连接数少、事件密度低，这种顺序实现通常不会立刻造成可感知的延迟。请求数量增加后，顺序结构就会把最慢的环节放大成全局问题。以游戏服务器为例，四人同图时若有一名玩家网络延迟达到 500ms，顺序阻塞读取会让整轮推进被拖慢，原本流畅的其他玩家也会被迫等待。单个慢连接的等待被主循环放大为全局停顿，这就是等待传播的排队效应。

这个例子中的瓶颈来自等待被串进同一条主处理路径。系统需要先把慢连接、缓存查询这类 I/O 等待从主循环中拆开，再把内容编码、规则计算等可拆分的计算段推到多个 CPU 核上。前一段已经把并发与并行的分工概括为“隔离等待、摊开计算”；接下来要把这条分工落实到进程、线程和 goroutine 上。对在线系统而言，慢连接、数据库访问和缓存访问属于可隔离的等待，数据过滤、规则计算和批量序列化属于可摊开的计算段，游戏服务器中的碰撞检测与第二章介绍的 AOI 过滤也属于后者。图17把这组差异放到同一时间轴上。左侧只有一条单核执行带，同一时刻只运行一个任务；某个任务进入 I/O 等待后，CPU 才切换到其他可运行任务。右侧的三个 CPU 核则在同一时间区间分别执行计算段，因此总计算可以早于单核顺序基线完成。

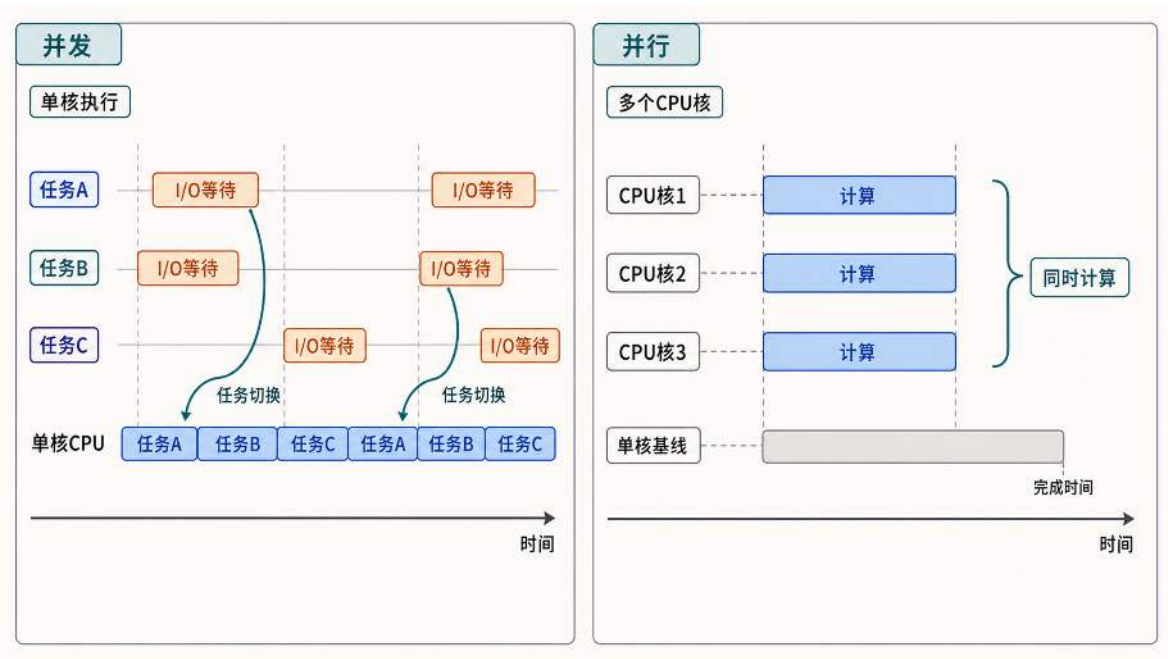


图 17: 并发用交错执行隐藏 I/O 等待，并行用多核同时计算缩短总计算时间。

当连接规模从少量请求扩展到多人同时在线时，顺序执行模型会让多种压力汇入同一条主处理队列。图18对比了请求数量少与连接规模扩大后的两种状态：双人原型下顺序处理仍能稳定推进；当规模扩大到多人同图后，连接数、事件密度、I/O 等待与热点状态访问同时增加，共同推高主处理队列的长度。队列持续变长后，后续请求会在同一条主处理路径上等待更久，尾延迟急剧上升，慢请求还会把等待传播到后续所有请求。

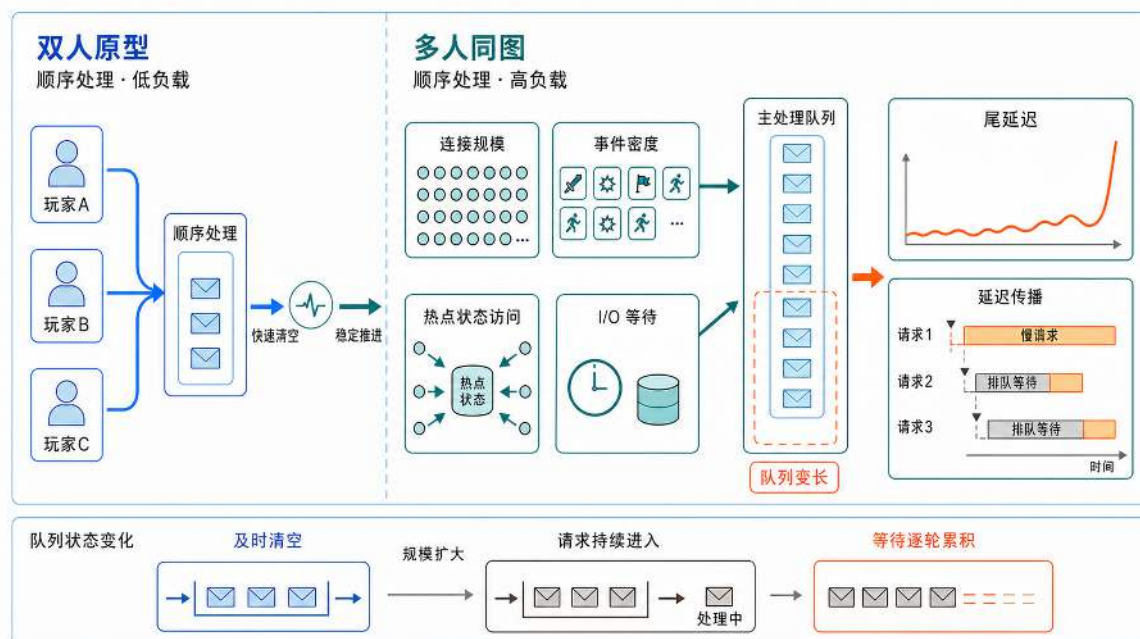


图 18: 从双人原型到多人同图，连接规模、事件密度、I/O 等待和热点状态访问共同汇入主处理队列；队列变长后尾延迟急剧上升，慢请求还会把等待传播到后续请求。

图18中的四类压力进入主处理路径后，会分别在连接维护、事件处理、外部等待和热点状态修改上形成瓶颈。需要注意的是，前三类压力是顺序模型本身就会暴露的；真正的共享状态并发冲突，要等到下一节引入并发执行流后才会成为独立问题。

第一个瓶颈是连接规模的增长。先以 50 个连接为起点：系统需要维护的长连接数（例如 WebSocket 或 TCP）从 2 个增长到 50 个，而每个连接都需要独立的读写缓冲区、心跳检测机制与超时管理逻辑。如果采用传统的一连接一线程模型，每个线程通常需要预留 MB 量级的栈虚拟地址空间（具体随操作系统默认值与配置不同），50 个连接对应的预留量就可能达到数十到数百 MB。预留地址空间不等于已经占用同等规模的物理内存，线程栈会按实际使用逐步提交，但线程数量继续增长后，地址空间、已提交栈内存和调度成本都会累积。这个数字只是教学起点；当连接数继续增长到数百、数千，甚至用来讨论上万连接的资源上限时，线程栈、创建销毁与上下文切换的成本会更快暴露出来。操作系统线程的创建和销毁涉及系统调用，上下文切换需要保存和恢复完整的 CPU 寄存器状态，这些开销在高频场景下会成为明显的性能拖累。

第二个瓶颈是事件密度的快速放大。输入事件数量大致随在线人数线性增长，但输出侧的状态推送会被广播进一步放大。假设一个实时场景中每名用户平均每秒产生 5 次操作，100 名并发用户就需要服务器处理每秒 500 次的输入事件。输出侧的压力更为严峻：每次状态变更都需要推送给相关用户，若平均每人的可见或订阅对象有 10 个，则每秒需要发送 5000 次状态同步消息。如果所有用户都关注同一份全局状态，广播开销会接近平方级；若通过兴趣过滤把推送范围稳定限制住，它更接近“人数乘以平均关注数”的线性放大。在多人游戏中，玩家移动、攻击等操作触发对视野内其他玩家的状态广播，同样面临这一放大效应。在顺序执行模型中，处理这些事件的总时间是所有单次操作时间的简单累加，当事件密度超过处理能力上限时，消息队列会迅速积

压，响应延迟从毫秒级恶化到秒级。

第三个瓶颈是 I/O 等待时间的全局传播。在单机服务器中，许多操作都涉及 I/O 等待：读取用户会话可能要等待一次缓存服务的网络往返，查询订单信息可能要等待数据库或磁盘路径，向客户端发送响应也可能要等待 TCP 发送缓冲区就绪。在顺序执行模型中，主线程在等待某条连接的外部查询响应时会完全阻塞，无法处理其他连接的请求。这意味着一个慢 I/O 会传播到所有后续请求，造成全局性的响应延迟增长。即使系统配备了高性能 CPU 和大容量内存，这些计算资源也会因为 I/O 等待而长时间处于空闲状态，资源利用率极低。

第四个瓶颈会在引入并发后出现：共享资源的并发访问冲突。当多个用户几乎同时操作同一资源（如抢购限量商品、编辑同一份文档的同一行、向同一账户并发转账）时，系统必须确保操作的原子性和结果的确定性。在顺序主循环中，所有操作按到达顺序依次执行，并发冲突不会出现。但当系统为了解决前三个瓶颈而引入并发处理后，如果没有配套的同步机制，就可能出现数据竞争：两条路径同时通过检查条件并执行修改，导致重复扣减或重复发放。以游戏为例，两名玩家同时拾取地图上唯一的宝箱，若无同步保护，就可能都认为自己成功，破坏规则公平性。这类问题的根源在于检查和修改操作之间存在未受保护的间隔，多个执行流可能同时通过检查条件并执行修改操作。

从更整体的视角看，延迟传播背后通常是排队效应。请求到来的速度一旦接近系统的处理速度，哪怕只是少量抖动，也会被队列放大成更长的等待。此时，平均延迟可能还维持在表面可接受的范围里，尾延迟已经被少数慢请求拉高：后续请求被挡在队列里，用户感知到的卡顿或超时更集中地落在尾部分请求上。回到并发与并行的分工：前者把等待拆开，后者把算力摊开，目的都是在有限资源内缩短这条排队链条。

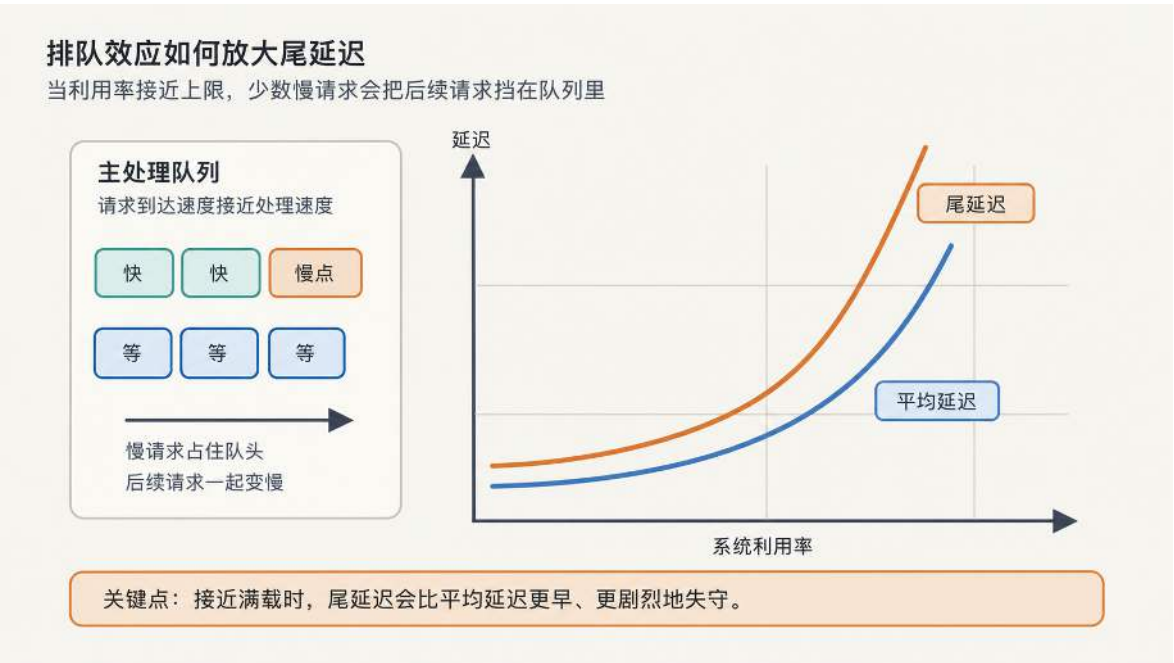


图 19: 利用率接近上限后，少量到达率抖动会被队列放大为尾延迟的非线性恶化。

图19强调的是接近满载时的非线性变化。只要请求到达速度接近服务器处理速度，一个慢请

求占住队头，后续请求就会一起等待。平均延迟可能只是逐步上升，但尾延迟会更早、更剧烈地拉开。这类积压关系可以用 Little 定律 $L = \lambda W$ 描述： L 表示系统中平均积压的请求数， λ 表示请求到达率， W 表示请求从进入系统到处理完成的平均停留时间。当慢请求把 W 拉长时，在相同到达率下，队列中的积压请求数 L 也会增加；积压越多，锁竞争、缓存抖动和批量输出就越容易继续放大延迟。

这些瓶颈也会出现在其他在线服务中。即时通讯、金融交易、协同办公这类在线服务，只要进入大量用户同时在线的阶段，都会遇到类似的资源组织问题：多路连接怎么管理，高频事件怎么消化，I/O 等待怎么隔离，共享状态怎么保护。并发编程的目标，是在给定硬件条件下控制排队长度，让吞吐和尾延迟都留在可接受的范围里。

3.1.2 操作系统的抽象：进程与线程

上一节把顺序主循环如何放大延迟拆解清楚了：等待要拆开，队列要压住。要实现这些，首先需要操作系统提供的执行组织方式。以采用固定节拍推进状态的实时服务为例，一个进程通常同时维护多类工作：连接读写持续接收客户端输入，主事件循环推进业务状态，输出路径把状态变化推送给相关用户，持久化任务把交易结果或日志写入外部存储。这些工作最终都要交给操作系统调度执行。在操作系统提供的抽象中，进程负责组织资源和隔离故障，线程负责承载可被 CPU 调度的执行流。

进程可以理解为一个被操作系统托管的服务实例。在线服务进程拥有自己的地址空间、文件句柄、网络连接和权限信息；网关进程、日志进程和其他业务进程也各自拥有独立的运行环境。这种隔离让故障影响范围变得可控：一个业务进程崩溃时，该实例服务的用户会全部受影响，网关、日志服务和其他业务进程仍可继续运行。

线程位于进程内部，负责执行具体代码。操作系统调度器真正放到 CPU 核上运行的是线程。每个线程有自己的寄存器上下文和调用栈，用来记录当前执行到哪一步；同一进程内的多个线程共享堆内存、文件句柄和网络连接。共享内存让线程之间传递数据更方便，例如连接线程可以把解析出的客户端输入交给业务逻辑线程；这种共享也要求程序写清同步规则，否则多个线程同时读写共享订单、会话状态或推送队列时，结果就会变得不可预期。

这组区别会影响在线服务的组织方式。对在线服务而言，进程主要划定服务实例和故障影响范围，线程则承载连接读写、输入处理和状态推送等可被 CPU 调度的工作。

表 1: 进程与线程在在线服务中的职责差异

对比维度	进程 (Process)	线程 (Thread)
在在线服务中的角色	承载业务服务、网关或日志服务等进程实例	在同一服务实例内执行连接读写、输入处理、状态推送等工作
资源归属	拥有独立地址空间、文件句柄、网络连接和权限信息	共享所属进程的堆内存、文件句柄和网络连接，保留独立栈与寄存器上下文
故障影响	单个业务进程崩溃时，影响主要集中在该实例服务的用户范围	线程出错可能破坏同一进程内的共享状态，严重时导致整个进程退出
并发代价	隔离更强，但跨进程通信和实例切换成本更高	共享数据更方便，但需要同步规则保护共享状态
适用场景	适合划分服务实例和故障影响范围	适合承载实际执行工作，数量增长会带来栈、切换和同步成本

表1说明，进程和线程解决的是两类不同问题：进程更适合划分业务服务、网关和日志服务等进程实例，并把单个实例崩溃后的影响限制在相对明确的范围内；线程更适合承载连接读写、输入处理和状态推送等会被 CPU 调度执行的工作。在一连接一线程的模型下，连接数量增加时线程数量也随之增长，会带来三类成本。第一是栈空间成本。每条线程都有独立栈，连接规模上升后，服务器除了保存会话状态和消息缓冲区，还要为大量线程保留栈空间。第二是切换成本，线程切换需要保存与恢复寄存器、更新调度结构，并可能破坏 CPU 缓存局部性。第三是同步成本，线程共享堆内存和对象引用，越多执行流并行读写同一份状态，越需要明确临界区和锁顺序。

线程数量继续增长时，成本会从“执行计算”扩展到“维持执行流”本身。在线服务在推进业务状态之外，还要长期维持连接读写、存档落库和状态推送等路径。连接读路径会停在客户端下一条输入到达之前，存档路径会停在数据库或缓存返回之前，推送路径会停在客户端连接重新可写之前。这些路径停住时，CPU 未必在执行业务计算；如果每条路径都占用一条操作系统线程，线程栈和调度实体仍然会被保留。连接数和外部访问次数继续上升后，服务器会积累大量停在返回点之前的执行流，线程模型的资源成本也会随之放大。

前面讨论进程和线程时，在线服务已经被拆成两层职责：进程划出服务实例和故障影响范围，线程承载会被 CPU 调度执行的工作。但连接读写、外部查询和状态推送这类路径经常停在等待点上，直接用线程承载会把等待成本也放大成线程成本。协程通常指由用户态运行时管理的轻量执行抽象，但不同语言对调度方式的规定并不相同。Go 中对应的执行单元称为 goroutine；它与经典的协作式协程并非完全同义，因为 Go 运行时会把 goroutine 复用到多条操作系统线程

上，并支持运行时抢占。⁸ 图20把这三层关系放在一起：进程保存资源并隔离故障，线程承接内核调度，以 goroutine 为代表的轻量执行单元把大量等待型工作复用到较少线程上。

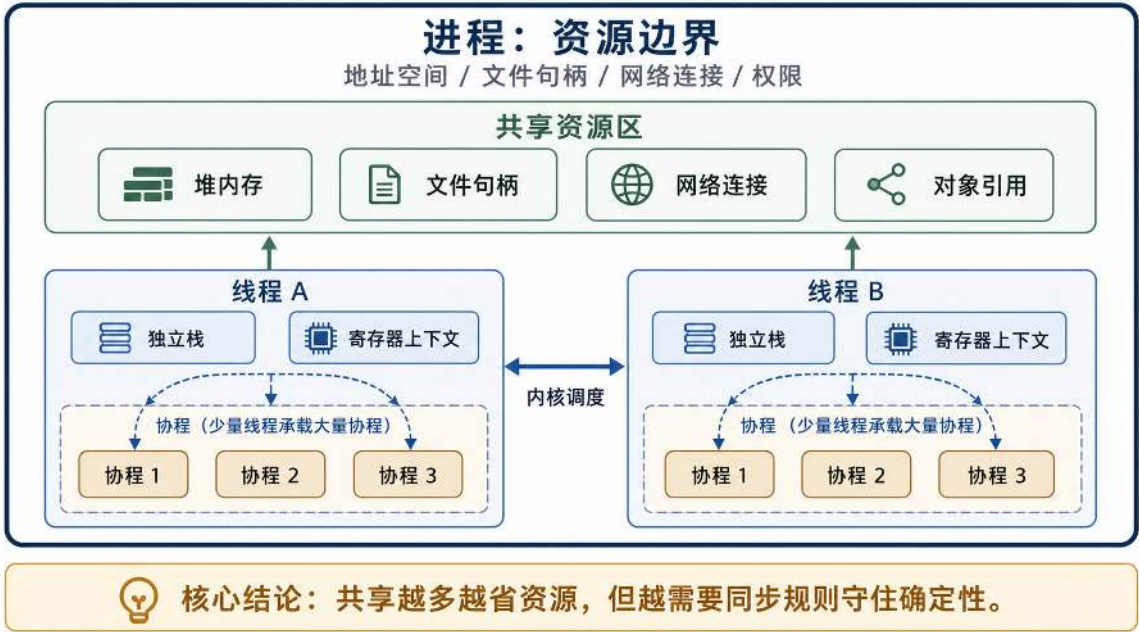


图 20: 进程、线程与轻量执行单元的三层关系：进程保存资源并隔离故障，线程承接内核调度，以 goroutine 为代表的轻量执行单元把大量等待型工作复用到较少线程上。

3.1.3 用 goroutine 组织等待型任务

在 Go 中，goroutine 是由运行时管理的轻量执行单元。它适合承载那些职责清楚、但经常在网络读写、缓存查询或数据库访问时进入等待的执行流程。放到在线服务中，连接读可以写成一个 goroutine，持续等待客户端发来的下一条输入，并把解析后的消息投递到输入队列；连接写可以写成另一个 goroutine，从发送队列取出推送结果，再等待连接可写后发回客户端；用户查询、订单结算落库这类外部 I/O 也可以放入后台 goroutine，等待缓存或数据库返回结果。

这样拆分后，代码仍然按照业务路径组织：连接读写负责网络消息进出，外部 I/O 负责访问数据库或缓存，后台推送负责处理发送缓冲区。但一个新问题随之出现：goroutine 数量远超 OS 线程时，谁来决定哪个 goroutine 占用哪个 OS 线程？

Go 运行时的调度器承担这件事。数量较多的 goroutine 先进入可运行队列，调度器再把其中若干个映射到较少的 OS 线程上执行。当 goroutine 在 channel 或运行时可以管理的网络 I/O 上等待时，它会被挂起，原来承载它的 OS 线程可以继续执行其他已就绪的 goroutine。若 goroutine 进入真正阻塞的系统调用或 cgo 调用，原 OS 线程仍可能被占用；运行时安排其他线程承载可运行的 goroutine。这种把 M 个 goroutine 复用到 N 个 OS 线程（N 通常远小于 M）的策略常称为 **M:N 调度**。它降低了等待型工作与操作系统线程的一一绑定：连接规模扩大时，OS 线程

⁸Go 官方文档对 goroutine 与操作系统线程复用关系的说明见 https://go.dev/doc/effective_go；Go 1.14 引入异步抢占的说明见 <https://go.dev/doc/go1.14>。

数量通常不必随连接数按比例增长，等待期间也不必为每条路径长期保留一条独立线程。

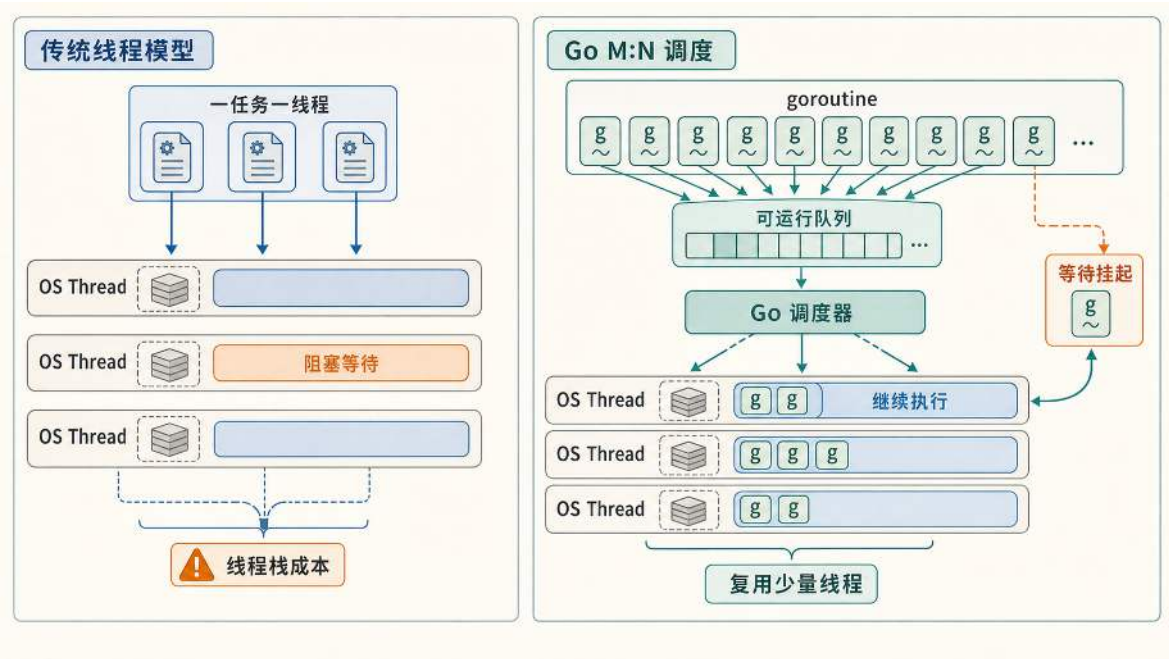


图 21: 传统线程模型让每个任务对应一条操作系统线程；Go 的 M:N 调度把大量 goroutine 复用到较少线程上，使等待型路径减少对操作系统线程的长期占用。

图21把这一资源收益对照成两种形态：在传统线程模型中，每个等待都背上一个完整的线程栈，连接规模扩大时栈成本线性累积；在 Go 的 M:N 调度中，大量 goroutine 由调度器分配到 OS 线程上执行，goroutine 挂起后，原线程可以继续承载其他可运行任务，使线程数量不再与等待任务数一一对应。

goroutine 也不会自动消除所有并发成本。复杂规则计算、大批量数据过滤或一次过大的消息编码属于计算型任务，它们会持续占用 CPU。如果任务彼此独立、计算量足够大，可以根据 CPU 核数安排有界并行；如果任务不可拆分、粒度过细，或者 goroutine 数量远超可用核心，继续拆分主要会增加调度和内存管理压力。因此，在线服务使用 goroutine 的判断标准应回到职责分工：等待型路径适合并发拆分，计算型路径需要控制并行度，权威状态提交则需要明确的串行化规则。对固定节拍的实时状态服务，这个规则可以是单写者主事件循环。所谓单写者（single writer），是指同一份权威状态只由一条固定执行路径提交修改，其他 goroutine 只能向这条路径投递消息；对普通请求服务，也可以使用按对象分区的队列、互斥锁或数据库事务规定提交顺序。后台 goroutine 完成工作后，应把结果交回相应的状态提交路径，而不是绕过既定同步规则直接改写共享状态。

两个代码块比较等待时间的叠加方式。假设服务器需要分别为 3 名玩家读取背包数据，主要耗时来自数据库或缓存返回结果。第一个版本按顺序读取三名玩家的背包，后来的玩家必须等待前一个查询完成；第二个版本用 goroutine 同时发起三次读取，让三段等待在时间上重叠。

顺序读取背包数据：等待串行叠加

```
1 func handlePlayer(playerID string) {
2     fmt.Printf("处理玩家 %s\n", playerID)
3     time.Sleep(2 * time.Second) // 模拟读取背包数据等 I/O 操作
4 }
5
6 func main() {
7     handlePlayer("A")
8     handlePlayer("B")
9     handlePlayer("C")
10    // 三次等待串行叠加，总耗时接近三次等待之和
11 }
```

通过 go 启动 goroutine 后，三个背包读取任务可以并发发起，等待时间更接近一次外部查询。这种写法改变的是等待的排列方式：单次数据库或缓存访问的耗时没有缩短，后来的玩家却不必排在前一个玩家的 I/O 等待之后。

Goroutine 并发读取背包数据：等待相互重叠

```
1 func main() {
2     var wg sync.WaitGroup
3     players := []string{"A", "B", "C"}
4
5     for _, player := range players {
6         wg.Add(1)
7         go func(id string) {
8             defer wg.Done()
9             handlePlayer(id)
10        }(player)
11    }
12
13    wg.Wait()
14    // 三次等待并发重叠，总耗时更接近一次等待时间
15 }
```

这个对比强调的是等待叠加方式的变化：顺序执行会把三次 I/O 等待串起来，并发执行则让这些等待在时间上重叠。对在线服务来说，这意味着某次数据库查询不必挡住其他连接的读写或消息推送。并发减少的是等待传播对整体吞吐的拖累，同时也会带来调度、内存和生命周期管理成本。

对于本节采用固定节拍和单写者模型的服务，goroutine 让连接读写、外部 I/O 和推送可以离开主事件循环单独等待，主循环仍然集中提交用户会话、订单状态和资源配置。这种分工把慢

I/O 和慢连接从权威状态路径中隔离出来，也让服务能够同时维持更多连接。执行流被拆开以后，问题不再只是“能不能并发”，而是这些等待如何被稳定地发现、分派和收束：连接什么时候可读，推送什么时候可写，后台查询什么时候返回，连接什么时候超时或关闭，都不能靠各条路径盲目轮询。这些等待点可以统一看作事件。服务器在事件发生时推进对应路径，在没有事件时让执行流退出等待或挂起等待，避免把整条主循环拖在同一个阻塞点上。

3.1.4 事件驱动与 I/O 多路复用：管理大量连接与等待

I/O 多路复用：把等待集中到内核 在线服务的事件驱动模型，可以从一条客户端连接开始理解。一条连接在大多数时间里没有新数据；只有当客户端发来请求、命令或心跳消息使连接变得可读，发送缓冲区允许继续写入，连接关闭，或者底层出现错误时，服务器才需要处理它。如果服务器反复检查每一条连接，空闲连接越多，无效检查越多；如果每条连接长期占用一条操作系统线程，线程栈和调度成本又会随着连接数增长。事件驱动把这个问题改写为一条稳定的程序组织方式：先等待哪些事情已经发生，再把事件分发给对应处理逻辑，处理完后继续等待下一批事件。客户端连接可读时，服务器读取并解析输入；连接可写时，服务器发送排队的推送结果；连接关闭或出错时，服务器清理会话、缓冲区和用户在线状态。

在操作系统看来，一条网络连接对应一个可被程序引用的句柄。服务器既不能为每条空闲连接长期保留一条线程，也不能反复扫描所有连接；前一种方式会放大线程栈和调度成本，后一种方式会把时间浪费在大量尚未就绪的连接上。

I/O 多路复用机制把等待集中到内核：应用登记一组关心的连接，只处理已经出现可读、可写、关闭或错误事件的句柄。Linux 中常见的 `select` 和 `epoll` 都解决“哪些连接已经就绪”这一问题；`epoll` 通过预先登记关注对象并返回就绪事件，更适合大量连接中只有少数连接活跃的场景。⁹

内核发现事件后，程序还要把不同事件交给相应处理逻辑。事件循环反复执行“等待事件、取出结果、分派处理”的结构，通常称为 Reactor 模式。¹⁰在 Go 服务中，这部分底层工作主要由运行时的 `netpoller` 承担：连接可读时，等待网络输入的 `goroutine` 被重新放回可运行队列，随后完成读取、分帧和解码。业务代码无需自行实现完整事件循环，但仍要把解析出的输入交给后续处理路径，而不能直接在网络读取路径中修改世界状态。

在本节采用的单写者实时服务中，这条客户端输入消息还不能直接修改业务状态。这些输入最终仍要回到按固定节拍推进的主事件循环，由它统一更新用户会话、资源配额和配置状态。如果读取路径绕过既定的状态提交规则，多个连接读路径就可能同时写入同一份状态，把前面已经隔离开的等待问题重新变成竞态问题。因此，读取路径和消费路径之间需要一个明确的数据交接点：读取路径负责产出客户端输入消息，消费 `goroutine` 或主循环负责按顺序取出消息并推进后续处理。

⁹Linux `epoll` 的接口语义见 Linux man-pages: <https://man7.org/linux/man-pages/man7/epoll.7.html>。

¹⁰Reactor 模式的经典描述见 Douglas C. Schmidt, “Reactor: An Object Behavioral Pattern for Demultiplexing and Dispatching Handles for Synchronous Events”, <https://www.dre.vanderbilt.edu/~schmidt/PDF/reactor-siemens.pdf>。

channel：读取路径与消费路径的交接点 最直接的实现方式是让读取路径把消息写入共享队列，再由消费路径从队列取出。但共享队列不仅要保存消息，还要处理并发访问、队列为空时如何等待、队列有新消息时如何唤醒消费者、队列满时如何让发送方感知压力。Go 语言里的 `channel` 把这几件事封装成语言级通道：发送方把消息放入通道，接收方按自己的节奏取出消息；如果一侧暂时没有准备好，运行时记录等待关系，并在条件满足时唤醒对应 `goroutine`。

在在线服务中，`channel` 提供的是一个明确的消息交接点。读取路径把客户端输入放入通道，消费路径再按顺序取出并处理；业务状态的修改仍集中在后续消费路径中完成。这种组织方式体现了 Go 常见的并发思路：让数据以消息形式在 `goroutine` 之间移动，业务状态的读写则集中在少数明确的处理路径中。

`channel` 分为无缓冲与有缓冲两类。无缓冲通道更像一次同步握手，发送方和接收方必须同时就绪，消息才能完成交接；有缓冲通道则带有一段队列空间，发送方可以在缓冲未满时先把消息放进去，让接收方按自己的节奏消费。对在线服务而言，有缓冲通道常被用作连接或服务级输入队列，用来暂存客户端输入的短时突发；队列容量有限，持续过载最终会让发送方停在投递位置。

从并发语义上看，有缓冲的 `channel` 可以抽象为一段有限容量的缓冲队列，以及发送方和接收方的等待关系。当发送方投递消息时，若缓冲未满则可以入队；若缓冲已满且没有接收方完成交接，发送方会等待，直到通道重新具备发送条件。对称地，接收方在缓冲为空且没有发送方完成交接时也会等待。具体运行时可以使用锁、等待队列等数据结构实现这些行为，但这些内部结构不是 Go 语言向程序提供的接口保证。对应用设计而言，重要的是通道把发送、接收、等待和唤醒组织成一条具有明确阻塞条件的同步路径。

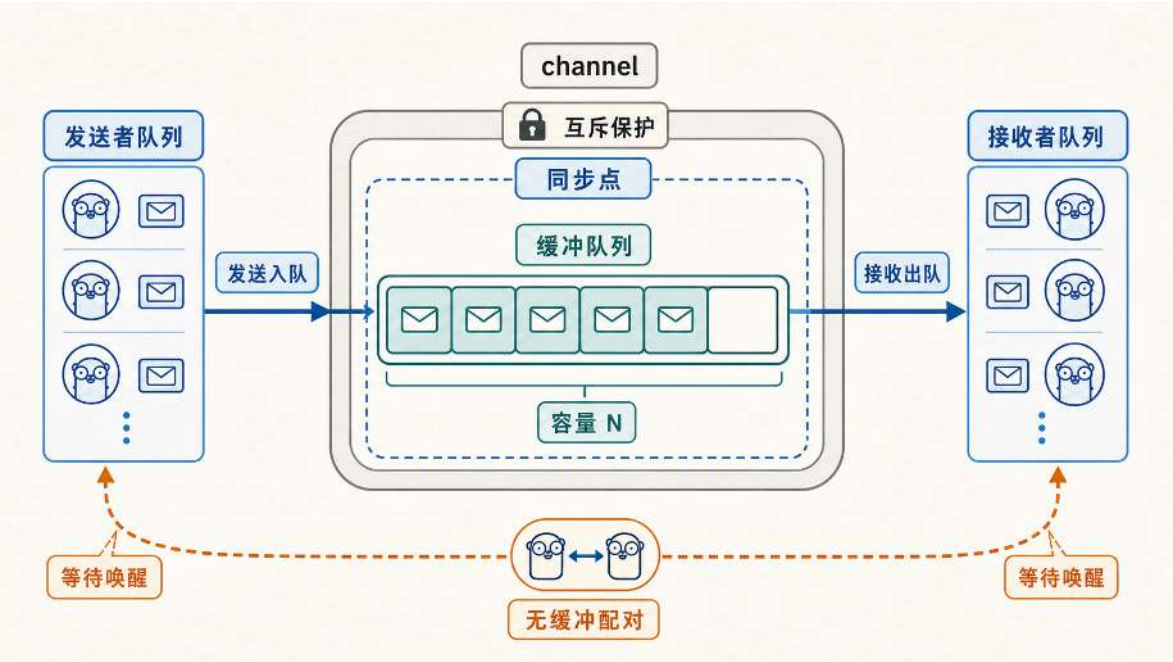


图 22: 从并发语义看，`channel` 用有限缓冲和两侧等待关系，把数据传递组织成具有明确阻塞条件的同步路径。

在一个采用服务级单写者循环的连接处理结构中，每个客户端连接至少会有读取路径和写入路径。读取 goroutine 负责从连接中读取字节流，例如从 WebSocket 连接读取客户端消息，解析成结构化消息后投递到有缓冲的 `channel`；服务级主事件循环再从输入队列中按节奏消费消息并集中裁决。这样，网络读路径只负责把输入变成消息并交出去，业务路径再按自己的节奏消费消息。当某条连接在弱网环境下连续发送消息，或者某次读取短暂变慢时，影响首先停留在这条连接的读取路径和输入队列中；主循环仍然可以按固定节拍消费已经到达的输入，不必因为单条连接的等待而暂停整个服务的推进。

连接读取路径：把消息交给服务级输入队列

```
1 type PlayerMessage struct {
2     PlayerID string
3     Action   string
4     Payload  []byte
5 }
6
7 func handlePlayerConnection(
8     playerID string,
9     svcInput chan<- PlayerMessage,
10 ) {
11     // 网络读循环：只负责读取、解析和投递（示意）
12     for {
13         msg, ok := readFromNetwork(playerID)
14         if !ok {
15             break
16         }
17         svcInput <- msg // 队列满时会阻塞，把压力传回读取路径
18     }
19 }
```

这段代码只展示输入交接路径。每条连接的读取 goroutine 只做 I/O、分帧、解析和轻量校验，再把消息投递到当前服务实例共享的输入队列；服务级主事件循环从同一个队列中按节奏消费消息并集中修改业务状态。单条连接退出时不能关闭这条共享队列，因为同一服务实例中的其他连接仍可能继续投递。

缓冲队列能够吸收短时间的输入突发，但容量有限。当客户端连续发送请求，而业务 worker 又因为状态计算、数据库写入或其他慢 I/O 处理不过来时，消息会先在 `channel` 中排队。如果这种差距只是短暂出现，队列会在后续消费中逐渐恢复；如果输入速度长期高于消费速度，队列就会被填满。

队列满后，读取 goroutine 再向 `channel` 投递消息时会停住。它停住以后，就不会继续从 WebSocket 连接中读取新的客户端输入。这样，下游业务处理能力不足的问题，会沿着 `channel`

反向传回网络读取路径。这种让上游感知下游处理能力不足、从而放慢输入推进的机制，就是背压。

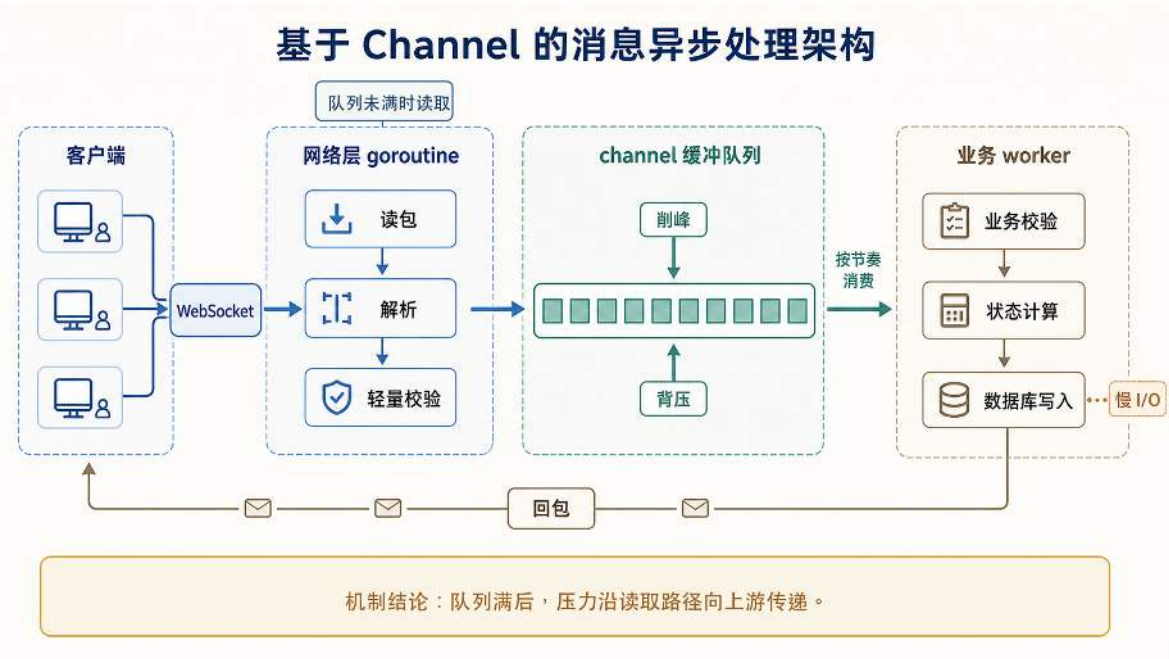


图 23: WebSocket 入口与业务处理通过 channel 解耦，缓冲队列满时压力会回到读取路径。

图23把这条压力传递路径放回在线服务的连接处理流程中。客户端消息先进入 WebSocket 连接，网络层 goroutine 负责读包、解析和轻量校验；解析后的消息进入 channel 缓冲队列；后续消费路径再按自己的节奏完成业务校验、状态计算和必要的外部访问。对采用单写者主循环的实时服务而言，需要修改权威业务状态的计算仍应回到主事件循环，数据库写入等慢 I/O 则应移出关键推进路径。消费路径中的慢 I/O 是一个具体限制：消费速度并不总是稳定的，一次数据库写入或外部访问变慢，就可能让队列开始积压。队列未满时读取路径可以继续投递；队列满后，读取 goroutine 会停在投递位置，压力沿读取路径逐步传回上游。

channel 在连接处理中的职责可以概括为三点：把客户端输入从网络读取路径交给业务处理路径，用有限缓冲吸收短时突发，并在下游处理不过来时把压力暴露回读取路径。

select：把多种等待写成可见分支 但连接处理路径并不只等待客户端消息。它还可能等待发送队列出现推送结果、等待心跳超时、等待连接关闭，或者等待上层取消信号。如果代码只停在某一个通道上等待，其他条件就可能被延迟处理。Go 的 select 用来把这些等待条件放在同一个控制结构中：只要某个分支已经就绪，它就可能被选中执行；如果多个分支同时就绪，select 不保证按源码中的先后顺序选择。因此，不能依靠 case 的书写顺序表达优先级；确实需要优先级时，应通过独立队列或分阶段选择显式实现。这样，等待不再隐藏在某一次阻塞调用里，而是被写成可见的分支：消息到达就处理，通道关闭就退出，超时到达就断开连接，上层取消就回收 goroutine。

超时分支：让等待拥有退出路径

```
1 func handleWithTimeout(playerID string, msgChan chan PlayerMessage) {
2     timer := time.NewTimer(30 * time.Second)
3     defer timer.Stop()
4
5     for {
6         select {
7             case msg, ok := <-msgChan:
8                 if !ok {
9                     return
10                }
11                timer.Reset(30 * time.Second)
12                processMessage(msg)
13            case <-timer.C:
14                fmt.Printf("玩家 %s 超时，断开连接\n", playerID)
15                return
16        }
17    }
18 }
```

这段代码采用 Go 1.23 及以上版本的通道式定时器语义：当主模块的 `go.mod` 声明 `go 1.23` 或更高版本时，`Reset` 返回后不会再收到旧配置产生的过期值，因此可以直接重置定时器。旧版本或显式启用旧定时器语义的程序仍需先停止并排空通道，不能把两种写法混用。¹¹

在这个示例中，`select` 同时等待消息通道和超时定时器。有消息就处理，通道关闭就退出，长期无消息就触发超时逻辑。类似结构常用于心跳检测与连接保活，避免僵尸连接长期占用资源。每一类等待都应当有可解释的退出路径；真实工程中常直接用 `context` 来统一取消链路。

同步与异步、阻塞与非阻塞 操作系统用 `epoll` 等机制等待连接句柄变得可读或可写；Go 运行时用 `netpoller` 把这些底层事件转换为 `goroutine` 的挂起与唤醒；业务代码用 `channel` 交接客户端输入和推送结果，再用 `select` 同时面对消息、关闭、超时和取消。如果不区分这些等待的语义，就容易把“结果怎样返回”和“当前执行流是否停住”混为一谈。

同步与异步描述结果沿什么路径返回：同步调用沿当前调用路径取得结果，异步调用则由稍后的事件、回调或结果句柄交付结果。阻塞与非阻塞描述当前执行流是否停在等待位置：阻塞调用会等待条件满足，非阻塞调用在暂时没有结果时立即返回。图24把同步/异步与阻塞/非阻塞拆成四种组合：异步接口也可能被调用方再次阻塞等待，同步接口也可能用立即返回的方式表达“暂时没有结果”。

¹¹Go 1.23 定时器通道语义说明见 <https://go.dev/wiki/Go123Timer>。

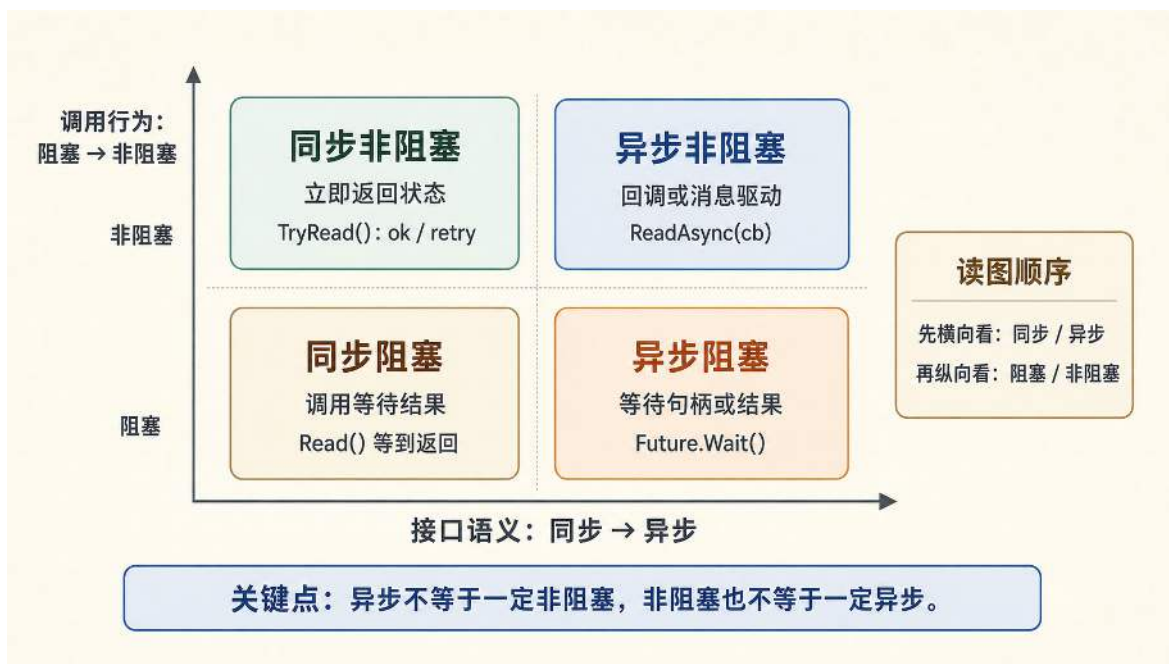


图 24: 同步/异步描述结果如何返回，阻塞/非阻塞描述当前执行流是否停住。

把这四种组合放回在线服务，可以得到更具体的判断。同步阻塞适合连接 goroutine 自己等待输入，但不能放进负责固定节拍状态推进的主事件循环，否则一个慢连接会拖住整个服务。同步非阻塞会在暂时没有数据时立即返回，但程序必须安排下一次检查。异步接口可以先提交数据库写入或跨实例查询；如果调用方随后立即等待结果，它仍然表现为异步发起、阻塞等待。事件驱动的网络路径则把空闲连接交给 netpoller 等机制，连接真正可读时再唤醒对应 goroutine，使服务把执行时间留给已经发生的输入事件和状态推进。

事件驱动并不表示在线服务里没有等待。客户端消息仍要等待网络到达，数据库写入仍可能变慢，空闲连接仍要等待心跳超时。它改变的是等待的组织位置：空闲连接交给运行时或事件循环等待，客户端输入到达后再唤醒读取路径；业务代码用 `select` 把消息、超时、关闭和取消写成明确分支。这样，某条连接暂时没有输入、某次外部 I/O 变慢，或某条连接已经断开，都不必把整条主循环拖在同一个等待点上。

goroutine 生命周期：取消、超时与容量限制 前面已经把连接读写、业务处理、慢 I/O 等工作拆到不同 goroutine 中。这种拆分提高了并发能力，也带来一个新的管理问题：这些 goroutine 什么时候应该结束。客户端断开连接后，读取 goroutine 应该退出；一次数据库写入超过请求期限后，等待它的业务路径不应继续占住资源；服务器准备关闭某个服务实例时，相关的连接处理、后台落库和推送任务也应尽快停止。如果没有统一的退出信号，后台 goroutine 可能继续等待数据库、计时器或外部接口，最终变成难以定位的 goroutine 泄漏。

在 Go 服务端中，这类退出信号通常由 `context.Context` 承担。一次用户请求、一次数据库访问、一个连接会话，都可以带着同一个 `context` 向下传递。当连接断开、请求超时或服务准备关闭时，上层取消这个 `context`，下游 goroutine 就能通过 `<-ctx.Done()` 收到退出信号。把

context 放进 select 分支后，业务代码就能把“有消息就处理、超时就放弃、父任务取消就退出”写成同一套控制结构。

除了管理已经启动的 goroutine，在线服务还要限制同时进入某条路径的任务数量。例如同一个时刻不能让过多订单结算同时写数据库，也不能让所有用户请求一起挤进某个昂贵的匹配或存档路径。如果只负责启动 goroutine，却不给这些路径设置容量上限，高压下队列会继续膨胀，超时也会被拖到更晚才暴露。

带缓冲的 channel 可以用来表达这种容量上限。它可以看作一个容量为 N 的信号量：往通道里放入一个空结构体表示占用一个名额，取出一个空结构体表示归还名额；当前元素数是已占用名额，剩余容量才是还能进入的并发数。当获取名额的动作也写进 select 时，程序就能同时表达“拿到名额就执行”和“等太久就放弃或降级”，避免高压下无休止排队。

容量令牌：把并发上限和超时写进同一条路径

```
1 var sem = make(chan struct{}, 100) // 容量=100，最多允许100个并发
2
3 func withLimit(timeout time.Duration, fn func()) bool {
4     select {
5         case sem <- struct{}{}: // 获取令牌
6             defer func() { <-sem }() // 归还令牌
7             fn()
8             return true
9         case <-time.After(timeout):
10            return false
11     }
12 }
```

这些机制共同服务于同一个目标：让连接等待、业务等待和资源等待都有明确位置，而不把所有不确定性都压回同一条关键处理路径。

3.1.5 请求处理链：串联进程、线程、协程与异步通信

前面已经分别讨论了进程与线程、goroutine、channel、select、context 和容量令牌。这些机制需要放进一条完整的请求处理链，才能看清各自承担的职责：连接由谁等待，输入如何交给业务处理，慢 I/O 如何移出主路径，关键状态又由谁按确定顺序更新。在线服务的一轮事件处理可以把这些职责串成一条完整路径。

以固定节拍推进业务状态的服务为例，假设每 33ms 做一次状态更新与结果推送。与此同时，客户端的输入是连续到来的，有的来自光纤宽带，有的来自移动网络，延迟与抖动差异很大。系统的关键矛盾在于避免把整轮推进节奏绑在最慢的那段等待上，而不只是把某个函数算得更快。理解这一点后，进程、线程、协程与异步通信就不再是抽象名词，而是用来切断等待传播、控制故障半径与管理并发规模的工具。

进程隔离：让故障半径可控 把一类业务放在一个独立进程里，是一种常见的工程选择。它的意义首先是隔离：业务进程崩溃时，影响被限制在该实例范围内，其他服务仍可继续运行；其次是资源治理：每个进程都有独立的内存空间和文件句柄表，更容易做部署、监控与自动拉起。它的代价也很明确：跨实例交互需要跨进程通信，一旦用户要跨实例移动，状态交接就不再是一次函数调用，而会变成一次协议与一致性问题。这类跨进程协作会在第 4.3.4 节继续展开，本章先把视角收敛在单机进程内的并发组织方式上。

线程与多核：计算段最终要落在 CPU 核上 无论上层抽象多么轻量，真正执行指令的仍是操作系统线程。在线服务的计算段，例如数据过滤、规则计算、批量推送的序列化，最终都会被调度到若干操作系统线程上，由多个 CPU 核执行。并行在这里发挥作用：当瓶颈来自算力不足时，利用多核可以压缩单轮计算时间。与此同时，线程也暴露出一个具体限制：阻塞不会消失，只会换一种位置出现。若把网络读写或磁盘 I/O 直接放在少量关键线程上，阻塞仍可能把推进节奏拖进队列。

协程与连接：让等待只阻塞当前工作线 这就引出了协程抽象的价值。对在线服务而言，最自然的拆分方式是以连接为单位拆执行流。每条连接的网络读写可能长时间等待，但这种等待应只影响这条连接，而不是把整个服务拖慢。goroutine 这样的轻量执行单元，使得每条连接都可以有独立的读写工作线：慢连接在自己的 goroutine 里等待，不会阻塞其他连接的处理；连接断开时也能自然回收与退出。与此同时，协程的轻量并不意味着可以无限制创建，系统仍需要在 goroutine 数量、队列长度与资源占用之间保持可观测与可控。

异步通信：把数据所有权与节奏写进程序 当连接级 goroutine 与主事件循环的执行流被拆开后，还需要一种方式把输入汇入主循环，又把主循环的结果分发回各个连接。channel 在这里扮演的是受同步保护的队列，它让数据以消息的形式流动，并把背压点显式化。输入通道的长度、输出通道的长度、以及在高压下是否允许丢弃或降级，都会直接决定系统在尾延迟上的表现。select 则把超时、取消与退路写进控制流，使等待从隐式变成可推理的结构。

在线服务的一种并发骨架（示意）

```
1  type Event struct {
2      ConnID int
3      Msg     Message
4  }
5
6  type Client struct {
7      conn net.Conn
8      out  chan Diff // 每条连接自己的有界发送队列
9  }
10
11 func runServer() {
12     in := make(chan Event, 8192)          // 输入事件队列：吸收网络抖动
13     persist := make(chan Persist, 1024) // 异步写回队列：把慢 I/O 移出主路径
14     clients := newClientRegistry()
15
16     go persistLoop(persist)
17     go mainLoop(in, persist, clients) // 集中更新业务状态
18
19     for { // 接入循环（示意）
20         c := acceptConn()
21         client := &Client{conn: c, out: make(chan Diff, 256)}
22         clients.Add(client)
23         go readLoop(c, in) // 阻塞读只影响该连接
24         go writeLoop(c, client.out) // 慢写停留在该连接
25     }
26 }
27
28 func mainLoop(
29     in <-chan Event,
30     persist chan<- Persist,
31     clients *ClientRegistry,
32 ) {
33     ticker := time.NewTicker(33 * time.Millisecond)
34     defer ticker.Stop()
35
36     const maxEventsPerTick = 2048
37     for range ticker.C {
38         // 每轮只在预算内消费输入，避免事件洪峰拖垮推进节奏
39         drainInput:
40         for i := 0; i < maxEventsPerTick; i++ {
41             select {
42                 case e, ok := <-in:
```


这段骨架把前面的概念对应到在线服务的具体运行路径上：进程划出服务实例的故障影响范围，线程承载实际执行，goroutine 分别负责连接读写、持久化和主循环，channel 把客户端输入与异步写回任务交给后续路径，select 让主循环在每轮预算内消费已经到达的事件。这里的主循环让业务状态主要由同一条循环集中修改：输入事件先进入队列，再按每轮预算被消费，最后统一推进业务状态并推送增量。代码中的 `queueDiffs()` 体现了状态同步的两个关键设计：主循环每轮只按订阅关系生成发生变化的部分，并把结果放入各连接的有界发送队列；独立写流程再完成可能阻塞的 socket 写入，不让慢客户端停住主循环。每条连接的 out 队列容量为 256，队列满时的处理策略沿用第 2.4.2 节“读写分离”中已经建立的分类原则：可覆盖的实时状态（如位置、朝向）合并为最新版本，避免同一对象反复排队；关键消息（如交易结果、奖励发放）不得静默丢弃，必要时通过超时、重同步或断开异常连接来暴露故障；主循环本身不能阻塞在发送队列上，否则整轮推进都会被最慢的客户端拖住。这种增量同步策略直接决定了推送的数据量：如果 100 名客户端同时在线，但本轮只有 3 个状态发生变化，增量同步只需要把这 3 个变化按可见关系发送给相关客户端，而不是把 100 个完整状态发送给所有人。

不过，把业务状态主要集中到 mainLoop 并不能覆盖所有共享数据。真实服务里，连接状态、用户会话、缓存视图、跨对象交易、异步持久化结果，都可能被不同 goroutine 接触；不受约束的实现还可能绕过输入队列，直接读写同一份对象。

问题会出现在“判断”和“修改”之间。以电商库存扣减为例，两个订单几乎同时扣减同一件商品时，两个执行流都可能先读到“库存仍然大于零”，随后分别扣减并确认订单。从每条执行流内部看，代码都像是按顺序执行的；放到并发环境里，库存却可能变成负数。这类错误不会表现为程序立即崩溃，而是表现为业务规则被破坏：同一份库存可能被超卖，或者不同客户端看到互相矛盾的订单状态。以游戏为例，两名玩家同时拾取地图上唯一的宝箱，也会出现同类问题。

这就把并发问题从“怎样让更多任务同时推进”推进到另一个层面：当多个执行流同时接触关键状态时，服务器怎样保证规则仍然只有一个确定结果。

3.2 并发正确性：保护共享状态

上一节把等待路径拆成了多个并发执行流，使更多请求能够同时推进。当这些执行流同时读取或修改共享状态时，系统还必须保证订单库存、账户余额、协作文档版本或游戏奖励仍然只有符合规则的结果。吞吐提升不能以破坏状态正确性为代价，因此并发结构必须配套明确的同步规则。

以电商库存扣减为例，两条处理路径几乎同时扣减同一件商品。路径 A 检查到“库存仍大于零”，还没来得及写回扣减后的数量；路径 B 也读到旧库存，于是两件商品被同时卖出，总库存变成了负数。以游戏 NPC 奖励为例，两条路径几乎同时触发同一只 NPC 的奖励发放，也可能出现同一份奖励被发出两次的情况。这类竞态暴露了并发环境下规则确定的三层隐患。第一，原子性：检查与修改必须不可被插队，否则先到的规则失效。第二，可见性：在多核环境下，路径 A 的写入可能停留在本地缓存中，尚未以其他核心可见的方式发布出去；路径 B 即使在时间上稍后读取，也未必能看到已更新的值。第三，有序性：编译器和 CPU 可能在不改变单线程

结果的前提下重排读写，使得代码中的先后顺序在并发下不再可靠。

并发程序需要用同步机制把关键动作之间的先后关系显式地写进程序：互斥锁让“检查库存”和“写回扣减后数量”在同一段受保护的代码里完成；channel 让发送方交出数据后接收方再继续处理；条件变量让等待者在条件变化后被通知并重新检查条件。这些机制形式不同，但都在回答同一个问题：哪个动作已经完成，哪个动作可以基于它继续。在并发语义中，这类可依赖的先后关系称为 *happens-before*（先发生于）。¹²

把这个场景落到代码层面。goroutine A 将共享计数从正数改成扣减后的值，表示库存已被占用。如果这次写入没有通过锁释放、channel 发送等同步事件传递给另一条处理路径，goroutine B 仍可能按旧状态继续执行。问题在于共享状态没有被放进受保护的使用过程里：检查条件是否满足、写回更新后的状态、基于结果继续处理，这几步必须由同一种同步关系串起来。以互斥锁为例，某条处理路径持有锁时，其他处理路径只能等待；等它完成检查和写回并释放锁之后，下一条处理路径才能进入同一段代码，并基于已经更新后的状态继续判断。并发程序里许多看似偶发的错误，本质上都是可见性与有序性没有被写进程序。要让程序获得一个可依赖的顺序（即前一个动作已经完成，后一个动作可以基于它继续），必须由某条同步事件在两个动作之间建立关系。

图25对照了互斥锁与 channel 两种建立这种关系的常见路径。

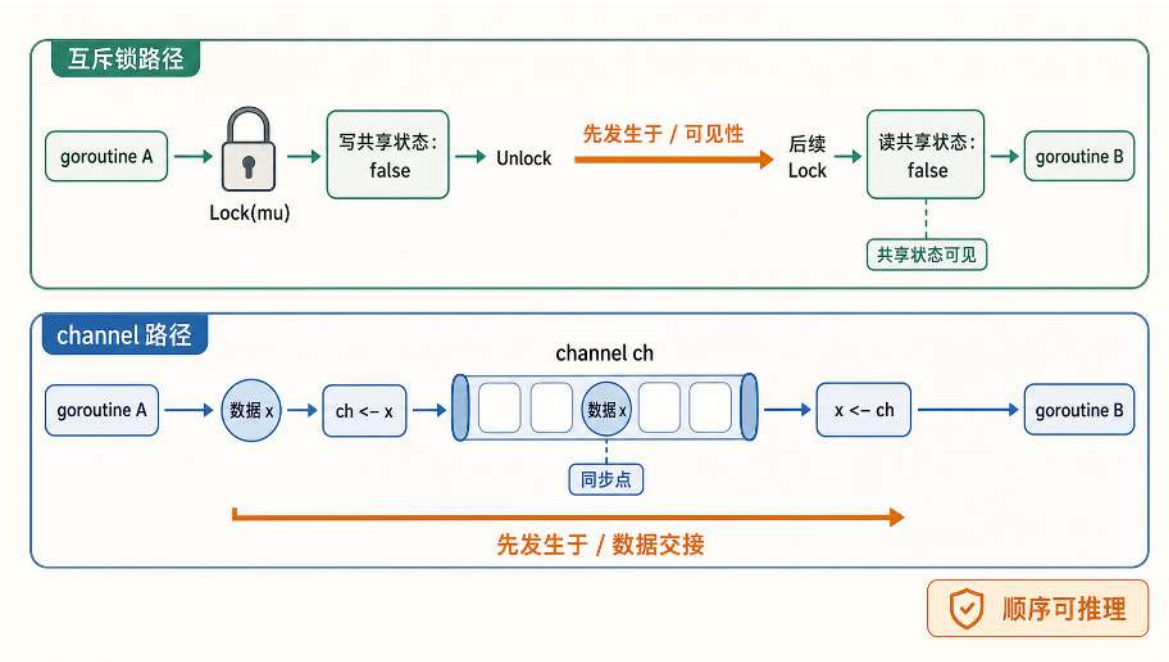


图 25: 互斥锁通过 Unlock 与后续 Lock 建立可见性，channel 通过发送与接收的交接点建立同步关系，两者都在 goroutine 之间形成可推理的先后顺序。

在互斥锁路径中，A 在持有锁期间写共享状态并执行 Unlock，B 后续成功 Lock 之后再读取共享状态，Unlock 与后续 Lock 之间建立了可见性；在 channel 路径中，A 把数据发送到

¹²Go 语言对 happens-before、同步事件和内存可见性的正式定义见官方文档 The Go Memory Model: <https://go.dev/ref/mem>。

channel, B 从同一个 channel 接收后才继续执行, 发送与接收的交接点就是同步点。两条路径形式不同, 但都让前一个动作的结果对后一个动作可见。

围绕 NPC 奖励归属, 后文按三步展开这一组同步问题。第一步是从规则出发识别不变量与临界区, 并用最小反例看见竞态条件如何生成。第二步用互斥锁把临界区收敛成唯一顺序, 同时补上活性视角, 说明死锁与饥饿为什么会让系统走不动。第三步从忙等过渡到条件变量, 把等待-通知机制与 for 复检条件讲清楚, 让正确性不再依赖侥幸的调度时机。

3.2.1 共享资源竞争：临界区与竞态条件

并发程序最常见的约束之一, 是确保共享资源在同一时刻只被一个执行流修改。以在线游戏的 NPC 奖励为例: 地图上随机刷新一只 NPC, 玩家走近触发任务后, NPC 送出宝物并立刻消失, 同一个 NPC 只能被成功触发一次。

这条规则之所以值得单独拿出来, 是因为它代表了服务端最常见的一类约束: 对共享资源的唯一归属。库存扣减、余额转账、限量抢购、文档协同编辑的冲突解决, 本质上都在回答同一个问题: 当多个请求几乎同时到达时, 系统如何保证一份资源只被分配一次, 并且在所有用户看来是公平可解释的。如果这一点守不住, 问题不会停留在一次显示错误上, 它会迅速演化成可被利用的漏洞, 破坏经济系统或业务规则的可信度。

把这条规则写成状态转换会更清楚。NPC 是否还能送出宝物、库存是否仍大于零、账户余额是否足够, 都可以抽象为一个布尔状态或数值状态, 从满足条件到不满足条件只允许发生确定次数。顺序程序里, 这个转换天然被执行顺序保护; 并发程序里, 执行顺序变成了交错顺序, 必须显式地规定: 检查条件与写回状态是一件不可分割的事。竞态条件的出现, 正是因为这段状态转换的检查与写回没有被同步机制保护成不可分割的整体。

在顺序程序里, 这条规则几乎不需要额外说明; 但在并发程序里, 它会立刻变成一个同步问题。两个请求可能同时到达, 两个 goroutine 也可能几乎同时执行到同一段代码。如果这段代码同时读取并修改了同一份状态, 例如 NPC 是否已被触发、库存是否仍大于零, 那么它就是临界区。临界区缺乏保护时, 就会出现竞态条件: 两个执行流都通过了检查, 都认为自己有权修改状态, 最终产生两个成功的结果。

竞态条件的根源, 通常在于检查与修改之间存在时间窗口。把这段窗口收敛到不可分割的原子区, 是守住不变量的唯一可靠做法: 让“读取是否还能送、决定是否送出、写回已送出”这三步在同一段临界区内顺序完成, 期间不允许其他执行流插入。图26对照了未保护和受保护两种执行顺序: 未保护时两个 goroutine 先后读到旧状态、都通过检查、都进入写回前的间隙, 最终重复发放; 用互斥锁保护后, 检查与写回被收紧在同一段临界区内, 后到的请求只能看到前者更新后的状态, 因此只有一条路径能通过。

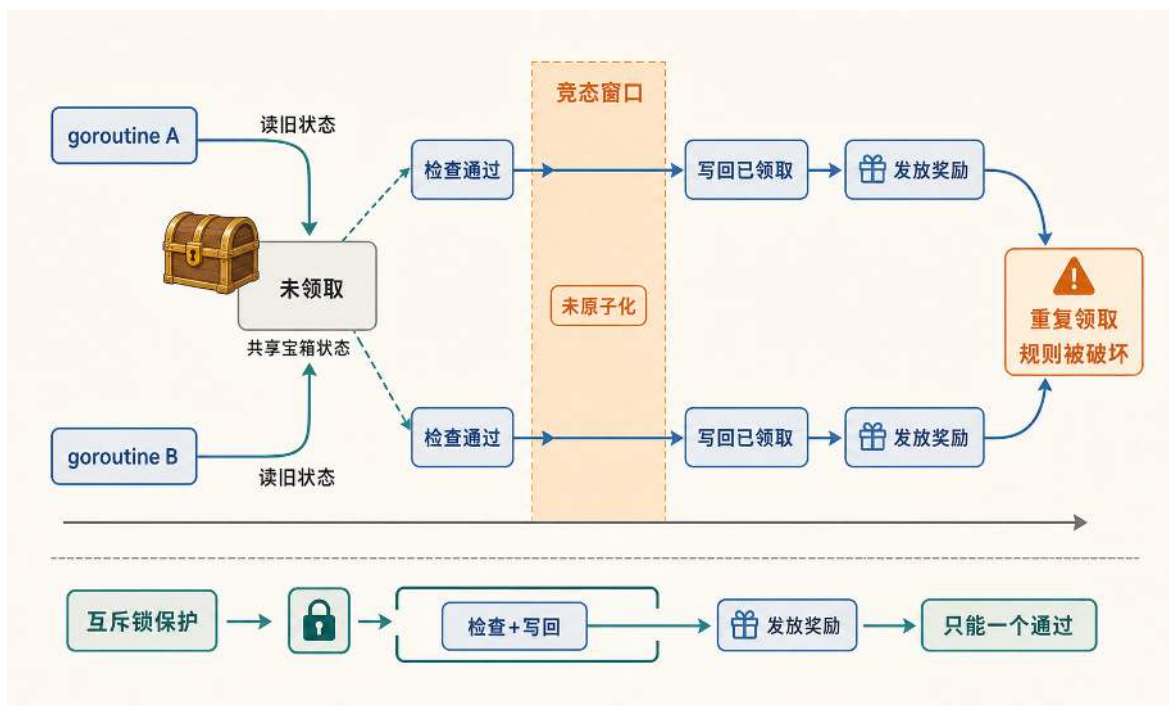


图 26: 竞态条件的时序分析: 缺乏同步保护导致两个 goroutine 同时通过检查并修改共享状态。

从设计视角看, 这段需要原子化的组合操作很像一笔小事务。它要么完整发生, 要么不发生, 中间状态不能被外界观察。互斥锁是一种实现这种小事务的方式, 事件队列和单写者也是。这里先用互斥锁建立基本概念, 因为它能把不可分割的要求准确翻译成运行时约束。

临界区通常不只是单独的那行赋值, 而是维持某个不变量的一段组合操作。对 NPC 奖励归属而言, 不变量是宝物只能送出一次。要维持这个不变量, 程序必须把读取是否还能送、决定是否送出、以及写回已送出这几步视为一个整体。只要其中任何一步可能被别的 goroutine 插入, 规则就可能被破坏。

更准确地说, 竞态并不要求两段代码真的在同一纳秒执行, 它只需要检查与修改之间存在未受保护的间隔。这个间隔可能来自网络延迟、磁盘 I/O、协程调度, 甚至只是一次看似无害的函数调用。并发程序最容易出错的地方, 正是把源代码里挨在一起的两步操作误当成执行时必然连在一起。

一段极简代码可以重现这个竞态窗口, 说明临界区缺乏保护时会发生什么。全局变量 `npcHasTreasure` 表示 NPC 是否仍然可以送出宝物, `greetNPC` 模拟玩家打招呼触发任务的流程: 先检查, 再做一点延迟, 然后修改状态并返回结果。

危险代码：缺乏同步保护的并发访问

```
1 var npcHasTreasure = true // NPC 是否还可以送出宝物（共享状态）
2
3 func greetNPC(playerID string) {
4     if npcHasTreasure { // 步骤1: 检查是否还能触发
5         time.Sleep(time.Millisecond) // 步骤2: 模拟延迟
6         npcHasTreasure = false // 步骤3: 标记已触发
7         fmt.Printf("玩家 %s 获得宝物\n", playerID)
8     } else {
9         fmt.Printf("玩家 %s 触发失败, NPC 已消失\n", playerID)
10    }
11 }
12
13 func main() {
14     var wg sync.WaitGroup
15     wg.Add(2)
16     go func() { defer wg.Done(); greetNPC("A") }()
17     go func() { defer wg.Done(); greetNPC("B") }()
18     wg.Wait()
19 }
```

多次运行这段代码，通常会出现两名玩家都获得宝物的结果。在真实系统里，类似的超卖或重复发放也会以同样方式发生。问题的关键在于检查与修改之间存在明显的时间窗口：执行流 A 检查到条件满足后进入短暂延迟，执行流 B 在这个窗口内也完成了检查，同样进入成功分支。即使延迟只有 1 毫秒，在高并发场景下这个窗口仍然足以触发错误。

这里的 `time.Sleep` 仅用于放大窗口以便观察。真实系统中不需要显式睡眠也会出现同类窗口：一次 JSON 反序列化、一次日志写入、一次缓存访问，都可能把执行流切走。并发规模越大，可能的交错排列越多，原本看似偶发的问题就会越来越频繁，直到在压力场景下近乎必现。

这暴露了一个根本矛盾：并发让多个请求可以同时推进，但某些规则天生要求串行化。系统既要保持整体吞吐，也要让关键的共享状态修改在同一时刻只能由一个执行流完成。要让这条规则在并发下仍然成立，需要互斥锁把临界区收敛成唯一顺序。

3.2.2 互斥锁：把临界区收敛成唯一顺序

临界区收敛与公平性 要解决竞态，核心是把检查与修改变成不可分割的原子区，在同一时刻只允许一个 goroutine 进入临界区执行，其他 goroutine 必须等待。互斥锁（Mutex）就是最常见的同步原语：进入临界区前加锁，离开时解锁。

从抽象上看，互斥锁做了两件事。第一，它把临界区变成一条单车道，同一时刻只允许一个执行流通过，从而保证检查与修改不会被插队。第二，它在通过的两端建立内存同步，让临界区

内的写入对随后进入的执行流可见，临界区内讨论顺序时也就有了可依赖的因果关系。这正是先发生于关系在工程里的落点。

在 Go 中，标准库提供了 `sync.Mutex`。它只有两个核心方法：`Lock()` 获取锁（不可用则等待），`Unlock()` 释放锁。为了避免因为提前返回而忘记解锁，工程上通常会用 `defer` 保证临界区结束时一定释放锁。

互斥锁可以直接修复前面的竞态。函数在检查与修改共享状态之前加锁，并用 `defer` 确保退出时一定解锁。这样，这段临界区在同一时刻只会被一个 goroutine 进入。

安全代码：使用 Mutex 保护共享资源

```
1 var npcHasTreasure = true
2 var npcLock sync.Mutex // 互斥锁
3
4 func greetNPCSafe(playerID string) {
5     npcLock.Lock() // 获取锁，进入临界区
6     defer npcLock.Unlock() // 延迟释放锁
7
8     if npcHasTreasure {
9         npcHasTreasure = false
10        fmt.Printf("玩家 %s 获得宝物\n", playerID)
11    } else {
12        fmt.Printf("玩家 %s 触发失败，NPC 已消失\n", playerID)
13    }
14 }
15
16 func main() {
17     var wg sync.WaitGroup
18     wg.Add(2)
19     go func() { defer wg.Done(); greetNPCSafe("A") }()
20     go func() { defer wg.Done(); greetNPCSafe("B") }()
21     wg.Wait()
22 }
```

此时运行结果稳定为只有一名玩家成功、另一名玩家失败。互斥锁把原本可能并发执行的临界区串行化，让共享状态的修改有了唯一顺序。这里使用 `defer`，是为了让正常返回和提前返回都执行 `Unlock()`，避免因遗漏解锁使其他 goroutine 永久等待。安全版同时删去了人为延迟，因为它不再承担复现竞态窗口的作用；即使保留该延迟，互斥锁仍能保证唯一结果，但真实代码不应在持锁期间执行慢操作。

不过，互斥锁保证的是同一时刻只有一个执行流进入临界区，它并不自动保证先到先得的顺序。谁先拿到锁，取决于调度与竞争时机。在很多规则里，只需要保证唯一性就已经足够；但如

果规则必须按到达顺序或时间戳裁决，就必须把顺序写进协议，例如把请求入队交由一个仲裁者按序处理，或者在消息上携带可验证的序列号。公平不是锁的副产品，它必须作为规则的一部分被明确设计。

活性陷阱：死锁与饥饿 使用互斥锁时有三点常见陷阱。第一，不要复制 `sync.Mutex`；通常把它作为结构体字段，或以指针共享同一把锁。第二，临界区要尽量小，只包住共享状态的读写，把纯计算放在锁外。第三，尽量避免在持有锁时再去获取另一把锁；如果确实需要，必须规定统一的加锁顺序，否则很容易死锁。第三点看似只是编码规范，背后指向的是活性：系统不仅要算对，还要能持续向前推进。活性失守时，用户看到的现象更直接：服务突然停住，或者某一类操作总是超时。死锁和饥饿是两类典型的活性陷阱。

死锁是最极端的一类活性问题。它并不要求代码里有明显的错误语句，只要两个执行流以不同顺序获取多把锁，就可能形成环等待。在最小死锁示例中，A 先拿 `lockA` 再拿 `lockB`，B 反过来。两条执行流各自拿到第一把锁后，再同时尝试获取对方持有的锁，便会形成确定的环等待。

从结构上看，死锁往往同时具备四个特征：资源互斥、占有并等待、不可抢占、环等待。互斥是锁的本性；占有并等待出现在拿着一把锁去等另一把锁；不可抢占意味着别人无法强制把锁收回；最后的环等待则由 A 等 B、B 等 A 构成循环。工程上最有效的切入点通常是破坏环等待，也就是统一加锁顺序。

死锁：锁顺序不一致导致环等待（示意）

```
1  var lockA sync.Mutex
2  var lockB sync.Mutex
3  var aLocked = make(chan struct{})
4  var bLocked = make(chan struct{})
5
6  func goroutineA() {
7      lockA.Lock()
8      defer lockA.Unlock()
9      close(aLocked) // 通知 B: A 已持有 lockA
10     <-bLocked      // 等待 B 持有 lockB
11
12     lockB.Lock()
13     defer lockB.Unlock()
14 }
15
16 func goroutineB() {
17     lockB.Lock()
18     defer lockB.Unlock()
19     close(bLocked) // 通知 A: B 已持有 lockB
20     <-aLocked      // 等待 A 持有 lockA
21
22     lockA.Lock()
23     defer lockA.Unlock()
24 }
25
26 func main() {
27     var wg sync.WaitGroup
28     wg.Add(2)
29
30     go func() {
31         defer wg.Done()
32         goroutineA()
33     }()
34     go func() {
35         defer wg.Done()
36         goroutineB()
37     }()
38
39     wg.Wait() // 两条路径互相等待，程序无法继续推进
40 }
```

示例中的 `aLocked` 和 `bLocked` 只用来确保两条路径都已持有第一把锁，从而稳定复现环等待；它们不是解决死锁的机制。

在在线服务里，死锁最常见的形态之一是双对象操作。比如转账需要同时修改双方账户余额，交易需要同时修改买卖双方库存与资金。如果两条路径各自按自己的顺序加锁，在并发对称触发时就会形成环等待。很多死锁来自对称结构在高并发下更容易被触发的锁顺序冲突，而不一定来自复杂业务逻辑。

运行最小死锁示例时，主 goroutine 等待 `WaitGroup`，而 A、B 又互相等待锁，Go runtime 通常会报出 `fatal error: all goroutines are asleep - deadlock!`。¹³这一现象只在最小示例中成立。真实服务里往往还有网络轮询、定时器或其他活跃 goroutine，局部路径的死锁通常不会被 runtime 直接识别为全局死锁，而是表现为部分请求永久挂起或持续超时。死锁的发现因此不能只依赖 runtime 的全局检测，更要靠锁顺序规约、超时机制和可观测指标来暴露。

工程上避免死锁的核心在于把资源获取变成可推理的协议。最常用的做法是规定统一的加锁顺序，例如总是按对象 ID 从小到大获取锁，任何跨对象操作都遵守同一顺序；其次是尽量减少同时持有多把锁的场景，把跨对象逻辑改写为消息串行化或分阶段提交，让临界区更窄、更短。

饥饿比死锁更隐蔽，也更贴近在线服务的真实体验。饥饿的含义是系统整体仍在运行，但某个执行流或某一类请求长期得不到资源，等待时间没有可依赖的上界。

在实时服务里，饥饿常表现为某类输入事件一直得不到处理。战斗事件持续推进，但移动事件的延迟越来越大，玩家会感觉操作像被丢弃。造成饥饿的原因有时并非算力不足，而是调度策略长期把某类请求放在队尾。

饥饿常见于两种原因：一种是锁或调度策略缺乏公平性，某些任务总被插队；另一种是人为引入了优先级，却忘了给低优先级留下最低保障。以下事件处理示例说明后一种情况：如果总是优先处理战斗事件，而战斗事件又持续不断，那么移动事件就可能一直排不到。

¹³Go runtime 的全局死锁检查逻辑可核对官方源码：<https://go.dev/src/runtime/proc.go>。

饥饿：优先级策略让某类事件长期得不到处理（示意）

```
1 type Event struct{ /* ... */ }
2
3 func loop(combat <-chan Event, movement <-chan Event) {
4     for {
5         select {
6             case e := <-combat:
7                 handleCombat(e)
8                 continue
9             default:
10            }
11            select {
12                case e := <-combat:
13                    handleCombat(e)
14                case e := <-movement:
15                    handleMovement(e)
16            }
17        }
18    }
```

要避免饥饿，通常需要为系统设置最低保障：例如采用配额策略，每处理 N 个高优先级事件就必须处理 1 个低优先级事件；或者把队列变成有界队列，并在拥塞时对高优先级也施加背压，让系统能在极端负载下保持可控。对实时服务而言，活性只保证系统能继续推进往往还不够，还必须把等待收敛到可接受的尾延迟范围内，否则用户感知到的就是尾延迟失控形式的不公平。

3.2.3 等待与通知：从忙等到条件变量

互斥锁解决了临界区的唯一顺序问题，但当条件不满足时，执行流仍需要一种高效等待的方式，而不是不断抢锁。以一个容量有限的任务队列为例：消费者可以从中取走任务，生产者也可以向其中放入新任务。如果队列空了，消费者可以选择等待最多一段时间；如果队列满了，生产者也可能希望等待到有空间再继续。

只用互斥锁很容易写出轮询：周期性加锁检查条件，不满足就解锁，等待一小段时间后再试；如果连这段等待也去掉，就会退化为持续占用 CPU 的忙等。两种写法都没有建立等待与状态变化之间的通知关系，只是轮询的 CPU 消耗通常低于持续忙等。

频繁轮询或忙等在高并发下会制造更坏的反馈：等待者反复抢锁，导致真正持锁的人也更难推进，锁竞争变重，尾延迟被进一步放大。对延迟敏感的在线服务来说，这种把等待变成负载的方式往往是最先需要剔除的。

互斥锁 + 定期轮询（反例示意）

```
1 type TaskQueue struct {
2     Items int
3     mu     sync.Mutex
4 }
5
6 func takeByPolling(q *TaskQueue, timeout time.Duration) bool {
7     deadline := time.Now().Add(timeout)
8     for time.Now().Before(deadline) {
9         q.mu.Lock()
10        if q.Items > 0 {
11            q.Items--
12            q.mu.Unlock()
13            return true
14        }
15        q.mu.Unlock()
16        time.Sleep(10 * time.Millisecond)
17    }
18    return false
19 }
```

用更长的 `Sleep` 或指数退避降低轮询频率，本质上仍是在用经验参数猜测等待时间：等待时间短了会频繁抢锁，时间长了又会错过本该及时响应的机会。轮询没有把等待和唤醒的因果关系写进程序；去掉 `Sleep` 后形成的忙等还会在等待期间持续占用 CPU。

要避免忙等，通常需要条件变量（Condition Variable）补上等待-通知能力。它通常与一把互斥锁绑定：等待方在条件不满足时释放锁并进入休眠；通知方在改变条件后唤醒等待者；等待者被唤醒后会在重新持有锁的前提下继续执行。Go 中可以用 `sync.Cond` 表达这类关系：代码中的 `Wait()` 对应“释放锁并等待”，`Signal()` 与 `Broadcast()` 对应“条件变化后发出通知”。

使用 sync.Cond 实现等待-通知（示意）

```
1  type TaskQueue struct {
2      Items    int
3      Capacity int
4      mu       sync.Mutex
5      notEmpty *sync.Cond
6      notFull  *sync.Cond
7  }
8
9  func NewTaskQueue(capacity int) *TaskQueue {
10     q := &TaskQueue{Capacity: capacity}
11     q.notEmpty = sync.NewCond(&q.mu)
12     q.notFull = sync.NewCond(&q.mu)
13     return q
14 }
15
16 func (q *TaskQueue) Take() {
17     q.mu.Lock()
18     for q.Items == 0 {
19         q.notEmpty.Wait()
20     }
21     q.Items--
22     q.notFull.Signal()
23     q.mu.Unlock()
24 }
25
26 func (q *TaskQueue) Give() {
27     q.mu.Lock()
28     for q.Items == q.Capacity {
29         q.notFull.Wait()
30     }
31     q.Items++
32     q.notEmpty.Signal()
33     q.mu.Unlock()
34 }
```

使用 `sync.Cond` 时，有三点同步语义需要特别注意。第一，`Wait()` 会在内部以原子方式释放互斥锁并休眠，避免先解锁再休眠之间的竞态窗口。第二，`Signal()` 只唤醒一个等待者，`Broadcast()` 会唤醒所有等待者，但可能造成惊群效应。惊群效应是指许多等待者被同时唤醒后

一起竞争同一把锁或同一个资源，但其中大多数被唤醒后仍然无法继续执行，只是增加了调度和锁竞争成本。第三，改变条件必须在互斥锁保护下完成；发送通知（Signal 或 Broadcast）在 Go 中允许在锁外调用¹⁴。工程上常在持锁修改条件后发出通知，使从状态变化到通知等待者的链路遵守同一套同步协议；但这仍不保证被通知者会先于其他竞争者获得锁，因此等待者必须在返回后复检条件。若需要支持最多等待一段时间，`sync.Cond` 本身不提供超时能力，通常需要配合计时器或用 `channel + select` 来组织。更关键的是，等待通常要写成 `for` 循环复检条件，而不是用 `if` 做一次判断就继续执行。

一种常见误解是认为被唤醒就意味着条件已经成立。实际上，条件变量只负责唤醒等待者，并不保证目标条件在它重新获得锁时仍然成立；等待者必须再次检查条件。

唤醒后的竞争是最容易被忽略的一类情况。仍以任务队列为例，假设当前 `Items==0`，两个消费者的 goroutine 都在 `Wait()` 上休眠。此时某个生产者放入 1 个资源，并调用 `Broadcast()` 唤醒所有等待者。两个消费者都会被唤醒，但它们不会同时继续执行，而是要一个个重新抢到同一把互斥锁。第一个抢到锁的消费者把资源取走，使 `Items` 重新变为 0；第二个抢到锁的消费者如果用 `if`，就会假设唤醒即意味着条件成立，从而在空池上继续执行，轻则读到不存在的数据，重则把计数减成负数，破坏规则正确性。用 `for` 循环复检，才能让第二个消费者看到条件已不成立，然后回到等待状态。图 27 展示了这个最小竞争场景：唤醒只是重新检查条件的开始，不是继续执行的许可。

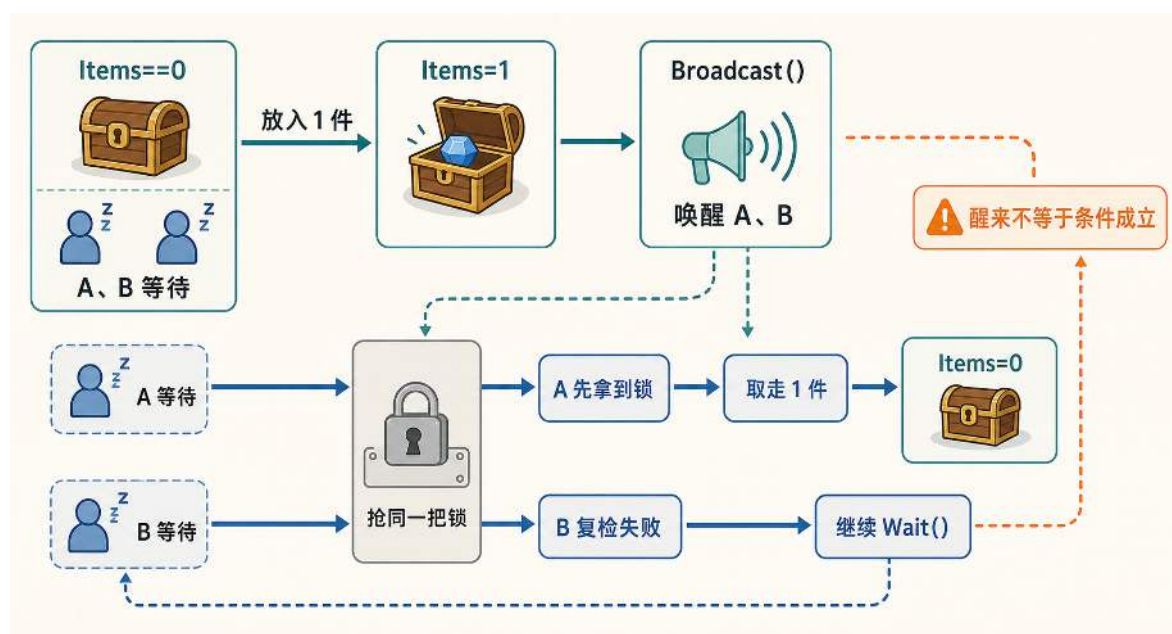


图 27: Broadcast() 唤醒后仍需复检条件。

有两类情况容易混淆。一些条件变量接口的规范允许等待者在没有显式通知时返回，这通常称为虚假唤醒；Go 的 `sync.Cond` 不采用这一语义，官方文档明确说明，`Wait()` 只有在收到

¹⁴Go `sync.Cond` 文档指出 `Signal` 和 `Broadcast` 允许但不要求调用者持有锁，见 <https://pkg.go.dev/sync#Cond.Signal>。

Signal() 或 Broadcast() 后才会返回。¹⁵ Go 仍然要求使用 for 复检条件, 原因不是 Wait() 会无通知返回, 而是通知并不等于为当前等待者预留了资源。从收到通知到重新获得互斥锁之间, 其他 goroutine 可能已经改变状态或消耗了可用资源; 上面的竞争场景已经足以说明, if 并不可靠。

因此, 条件变量的正确用法是每次唤醒后都重新检查条件。拿锁, 检查条件, 不满足就 Wait(); 被唤醒后再次检查, 直到条件成立才继续执行。值得强调的是, Wait() 返回时已经重新持有互斥锁, 因此复检条件与随后操作仍处在同一把锁保护的临界区内。

3.2.4 锁的粒度：性能与复杂度的折中设计

虽然互斥锁解决了并发环境下的正确性问题, 但条件变量的讨论已经暗示了一个趋势: 唤醒后仍要排队抢锁, 等待仍会传播为尾延迟。锁引入的核心性能成本是串行化。被锁保护的代码块在同一时刻只能被一个 goroutine 执行, 其他 goroutine 必须排队等待。这种串行化意味着并发度的下降; 当临界区很大或访问频率很高时, 锁竞争会成为系统瓶颈。因此在实际系统设计中, 锁的粒度选择需要在并发性能与实现复杂度之间权衡。

锁的成本来自两个层面。第一, 即使没有冲突, 获取与释放锁也需要维护锁状态并建立必要的内存同步, 这会让 CPU 付出固定开销。第二, 一旦发生冲突, 等待与唤醒会带来调度开销; 在多核下, 同一把锁对应的缓存行会在不同核之间来回转移, 产生缓存抖动。对延迟敏感的在线服务来说, 这些成本往往最先体现在尾延迟上: 平均耗时可能还看不出问题, 但偶尔一次排队就足以让用户感到卡顿。

因此, 用锁保证正确性只是第一步。更工程化的目标是把共享和竞争控制在可接受的范围内, 让规则确定的代价不会抵消系统的吞吐与稳定性收益。锁粒度的讨论, 就是在回答: 应该在哪里共享, 在哪里分隔, 才能既守住规则又守住性能。

锁粒度需要在实现简单和并发度之间取舍。全局锁把整个服务状态收进同一条队列, 实现最简单, 但无关操作也会互相等待; 区域锁按业务分片或空间分块收缩竞争范围, 使不同区域可以并发推进; 对象锁进一步把锁落到用户会话、订单、资源实例上, 并发度最高, 但跨对象操作会带来更复杂的加锁顺序和规则维护。图28用同一组用户与对象呈现三种策略的差异: 全局锁让所有人共排一条队, 区域锁让分区各自排队, 对象锁让每个实例各自排队。

¹⁵Go sync.Cond.Wait 的返回条件见 <https://pkg.go.dev/sync#Cond.Wait>。

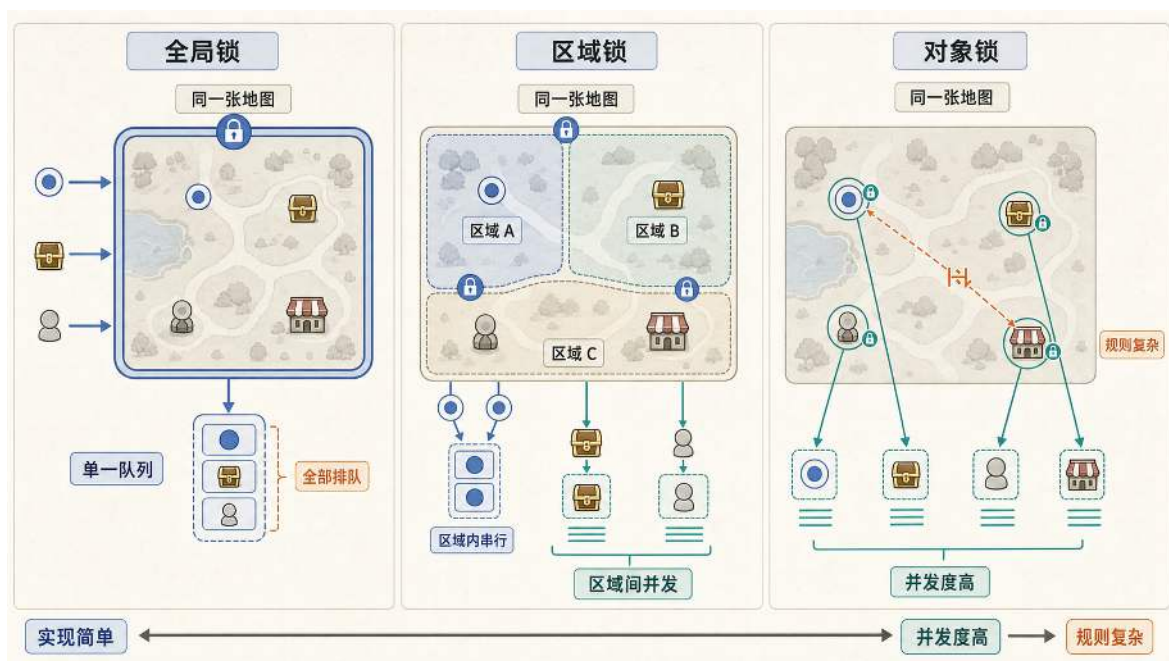


图 28: 锁粒度的三种策略对比：全局锁、区域锁与对象锁在性能和复杂度之间取舍不同。

全局锁是最直接的方案，整个服务器共用一把大锁，所有涉及共享状态的操作都必须获取这把锁才能执行。这种方式的优势是实现简单，死锁风险极低：只有一把锁，不存在多锁之间的获取顺序问题。然而，其代价是所有用户的操作都要排队等待，即使这些操作实际上访问的是完全独立的资源（如用户 A 读取自己的会话状态，用户 B 修改他人的订单记录），也必须依次执行。这会让核心路径接近单线程模型，抵消引入并发带来的收益。因此，全局锁更适合低频的全局配置、启动关闭等路径，在高频业务逻辑里要非常谨慎。

对象锁则走向了另一个极端，为每个独立的资源实例分配独立的锁。在在线服务中，这意味着每个用户的会话状态、每个订单、每个商品库存都由各自的 Mutex 保护。这样一来，用户 A 修改自己的配置时只锁定自己的会话，不必影响用户 B 查询商品列表的操作；两个用户同时操作不同订单时，也互不阻塞。对象锁通常能提供更高的并发度，让无关操作并行推进，但这会带来额外的管理和维护成本。锁的数量会增加，管理和维护成本会上升，也更容易遗漏某些应该保护的共享数据；当一个操作需要同时访问多个资源时（如转账需要同时锁定双方账户），还必须非常小心地设计加锁顺序，否则容易产生死锁。

区域锁在全局锁和细粒度锁之间取得折中，是实践中较为常见的选择。典型的做法是将数据按空间或业务域划分为多个独立区域（如按用户 ID 哈希取模、按地理区域分组），每个区域由独立的 Mutex 保护。区域内的操作需要串行执行，但不同区域之间的操作可以并发进行。当请求能够分散到多个区域，且一次操作通常只访问一个区域时，区域锁可以让不同区域并发推进，因而比全局锁少排队；它使用的锁数量又少于对象锁，生命周期管理也相对简单。相反，如果大多数请求集中在同一区域，或操作经常跨越多个区域，区域锁的并发收益就会明显下降。这种按区域划分负载的思想，与第 4.2 节要讨论的分布式分片存在连续关系：先在进程内把冲突局部化，

当单机承载接近上限时，再把同样的划分思想扩展到多机。

除了互斥锁的粒度选择，还可以根据读写比例选择读写锁。对于读多写少的数据结构，`sync.RWMutex` 允许多个读者并发进入，而写者保持独占，从而降低读路径上的锁竞争。另一类常见需求是限制某类操作的并发度，例如数据库查询或外部服务调用；这更像信号量，工程上常用带缓冲的 `channel` 来表达，容量就是允许的最大并发数。

如果共享状态非常简单，例如一个计数器、一个标志位，或者一次“只有当前值等于旧值时才写入新值”的状态转换，还可以使用原子操作。CAS (Compare-And-Swap, 比较并交换) 就是这类原语的典型形式：它先比较内存中的当前值是否仍等于预期值，若相等就原子地写入新值，否则返回失败，让调用方重试或走降级路径。Go 在 `sync/atomic` 中提供了这些接口。¹⁶原子操作适合保护很小的状态转换；一旦规则涉及多个字段、多个对象或需要维持复杂不变量，互斥锁、单写者事件循环或数据库事务通常更容易被审查和维护。

CAS 失败后通常需要重试，如果竞争很激烈，大量 goroutine 会在“读取旧值、尝试写入、失败再试”的循环里消耗 CPU，这就是自旋成本。另一个经典问题是 ABA：某个值先从 A 变成 B，又变回 A，CAS 只看当前值是否等于 A，可能误以为期间没有发生变化。工程上可以通过版本号、指针标记或更高层的锁与事务来避免误判。对在线服务而言，CAS 更适合计数器、状态位这类很小的局部状态；只要业务规则需要解释“谁先、谁后、为什么成功”，通常就应优先选择更清晰的锁、事件队列或数据库事务。

表2总结了不同锁粒度在在线服务中的典型应用场景与性能特点。在实际工程中，往往会混合使用多种粒度的锁，根据不同资源的访问特征选择合适的保护策略。

表 2: 不同锁粒度的应用场景与性能对比

锁的粒度	应用场景	并发性能
全局锁	服务器配置读取、全局公告	严重阻塞，接近单线程
区域锁	业务分片、地理分组	中等，区域间可并发
对象锁	单个用户会话、单个订单、 单个商品库存	较高，取决于锁顺序 与冲突分布

锁粒度的思路与后续的分布式分片存在连续关系。先在进程内把共享状态按对象或按空间划分成多个相对独立的域，让冲突尽量局部化；当单机的承载接近上限，再把这些域映射到不同的进程与不同的机器上，冲突处理也会从内存里的锁顺序扩展为网络上的状态交接与一致性协议。本章关心的是单机内如何把规则守住并把冲突收敛在局部；第 4.2 节和第 4.2.4 节会把同样的分解思想推到多机协作中。

本节讨论的 `sync.Mutex` 依赖进程内共享内存，只能保护单个进程中的数据。当同一份状态被多个服务器共同访问时，问题会从进程内同步转变为跨节点的状态归属、请求顺序与故障恢复，

¹⁶Go 原子操作接口见官方 `sync/atomic` 文档：<https://pkg.go.dev/sync/atomic>。

不能简单地把本地锁替换成一次远程操作。第 4 章将继续讨论这些机制；在本章的单机阶段，重点仍是通过合理的临界区与锁粒度，在正确性和并发性能之间取得平衡。

除了互斥锁和条件变量，Go 中还有三种常用机制可以补足并发控制：`defer` 保证锁释放和资源回收在函数退出时执行，带缓冲的 `channel` 可以充当信号量限制并发数，`select` 配合 `time.After` 可以为 `channel` 操作设置超时上限。这些机制也会在后面的连接池上限、等待超时和对象归还路径中再次出现。

至此，并发正确性已经从概念落到了具体工具：互斥锁保证临界区的唯一顺序，条件变量建立等待与通知关系，锁粒度决定串行化成本，`defer`、`channel` 信号量和 `select` 超时则补充资源释放、容量限制与退出路径。设计并发结构时，可以按不变量、同步关系、活性、可观测性的顺序检查这些机制是否真正成立。

首先写清必须始终成立的规则，例如同一个 NPC 只能送出一次宝物、同一个用户的关键消息必须按序处理、金币扣减不能丢失或重复。随后找出规则在哪里被检查、在哪里被修改，并确认这些动作是否处在同一套同步协议中。同步协议既可以是互斥锁，也可以是单写者事件循环、`channel` 所有权转移或串行事件队列；关键是让共享状态的每条读写路径都具有可依赖的先后发生于关系。

同步规则还必须保证系统能够持续推进。多把锁是否形成环等待，低优先级请求是否可能长期饥饿，队列是否有容量上限，等待是否能被超时或取消收住，连接退出后是否仍有 `goroutine` 泄漏，都会决定系统在压力下是否仍然可用。最后还要为这些判断提供可观测信号，例如锁等待时间、队列长度、`goroutine` 数和 P95/P99 延迟。规则被守住且运行状态能够被量化之后，后续瓶颈才会更清楚地落在连接、内存与数据访问等资源路径上。

3.3 单机资源优化与性能验证

到这里，单个服务实例内并发处理的两个基础问题已经得到解决：用并发结构隔离连接与 I/O 等待，用同步原语保证共享状态修改具有确定结果。接下来瓶颈会转向资源利用本身，例如连接的创建与销毁、临时对象的分配与回收，以及数据访问路径带来的延迟抖动。本节从连接、内存和数据访问三个方面讨论资源复用，再用压测与观测指标验证优化是否真正提高单个应用实例的承载能力。这里的“单机优化”强调应用实例尚未拆分到多台计算节点；数据库或缓存可以是独立进程或外部服务，正文关注的是当前实例如何控制访问它们的连接、并发和数据路径。

3.3.1 外部资源连接池：从短连接到连接复用

前面讨论的客户端连接，重点是稳定保活、及时读写和断线回收。在线服务访问数据库、缓存服务或其他后端组件时，也需要建立 TCP 连接，这里的连接管理转向服务与外部资源之间的访问路径。这类外部资源连接不应当随用随建，而应该通过连接池复用和限流。

每次建立 TCP 连接都需要完成三次握手过程，这意味着至少一次网络往返，还需要分配内核资源（例如套接字缓冲区、连接控制块等）并在用户态与内核态之间多次切换。以常见环境的数量级估算，局域网下握手往返通常在毫秒级，广域网可能到几十毫秒。如果在每次数据交互时

都重新建立连接，并在交互完成后立即关闭，端到端延迟会被握手与挥手拖长，系统调用与内核资源管理也会成为明显开销。

在高并发场景下，缺乏连接复用会让每次访问都重复支付建连与断连成本。以每秒 10,000 次数据库访问为例，如果每次访问都建立新连接，系统就要反复执行 TCP 握手、系统调用和内核对象的分配释放。单次开销可能不大，但大量请求叠加后会形成更长的排队与尾延迟。

频繁断连还会持续占用临时端口和连接状态。TCP 主动关闭方通常会在一段时间内保留 TIME_WAIT 状态，避免延迟到达的旧数据包干扰后续连接。创建和销毁速度过高时，新连接可能因端口或内核连接资源不足而失败。对本章而言，关键结论是：短连接把固定的协议与内核管理成本重复放大，连接池则把这些成本摊到一组可复用连接上。

连接池（Connection Pool）机制正是为了解决这一问题而设计的资源复用方案。连接池的核心思想是维护一组可复用的长连接：业务逻辑需要访问外部资源时，从池中借出空闲连接；池中如果没有空闲连接且未到上限时按需新建。使用完成后，连接不会被销毁，而是归还到池中等待下次复用。只有当池中所有连接都在使用且达到配置的最大连接数上限时，新请求才会等待，直到有连接被归还，或者在上层超时退出。Go 标准库的 `database/sql` 内置了这类池化能力。示例用“上限、空闲量、生命周期”三个配置说明资源约束如何落到程序中。

连接池上限与生命周期（示意）

```
1 func initDB(ctx context.Context) (*sql.DB, error) {
2     db, err := sql.Open("postgres", dsn)
3     if err != nil {
4         return nil, err
5     }
6
7     db.SetMaxOpenConns(100) // 最大打开连接数
8     db.SetMaxIdleConns(10)  // 最大空闲连接数
9     db.SetConnMaxLifetime(time.Hour) // 连接最大生命周期
10
11     if err := db.PingContext(ctx); err != nil {
12         db.Close()
13         return nil, err
14     }
15
16     return db, nil
17 }
```

示例中的三个配置量分别对应连接池的三个机制约束：总连接上限、空闲连接保留和连接生命周期。`MaxOpenConns` 限制同时打开的最大连接数，防止数据库服务器因连接过多而过载；当并发请求数超过这个限制时，多余请求会进入等待队列。`MaxIdleConns` 控制空闲连接的保留数量，使短时间内的新请求可以复用已有连接而不必重新握手。`ConnMaxLifetime` 为连接设置最大

存活时间，避免长期运行的连接累积状态异常或资源泄漏。通过这样的配置，连接数的膨胀被上限收住：无论瞬时并发有多大，系统最多只会维持 `MaxOpenConns` 条打开连接，其余请求要么排队等待，要么在上层触发超时与降级。握手与端口压力也因此从随负载失控增长，变成可配置、可观测的上限。

连接池上限并不是越大越好。`MaxOpenConns` 本质上把数据库可承受的并发显式化：设得过大，拥塞更容易被推到数据库端，查询排队与抖动会加重；设得过小，等待会被推回应用端，吞吐被过早截断。合理的上限通常需要结合数据库承载、单次查询耗时与尾延迟目标，通过压测与观测来确定。

短连接的问题在于每次请求都重复支付建连、查询、断开和 `TIME_WAIT` 的成本；连接池则把这些成本推到一组可复用连接上。图29对比了短连接模式与连接池模式：随用随建的线性路径每次请求都重复握手与断开；借出、归还与上限控制形成的循环路径则把连接复用起来，将成本收敛为可配置的资源管理。池子的上限不会让数据库容量变大，但它能把连接膨胀收敛为一个可配置、可观测的连接数上限。

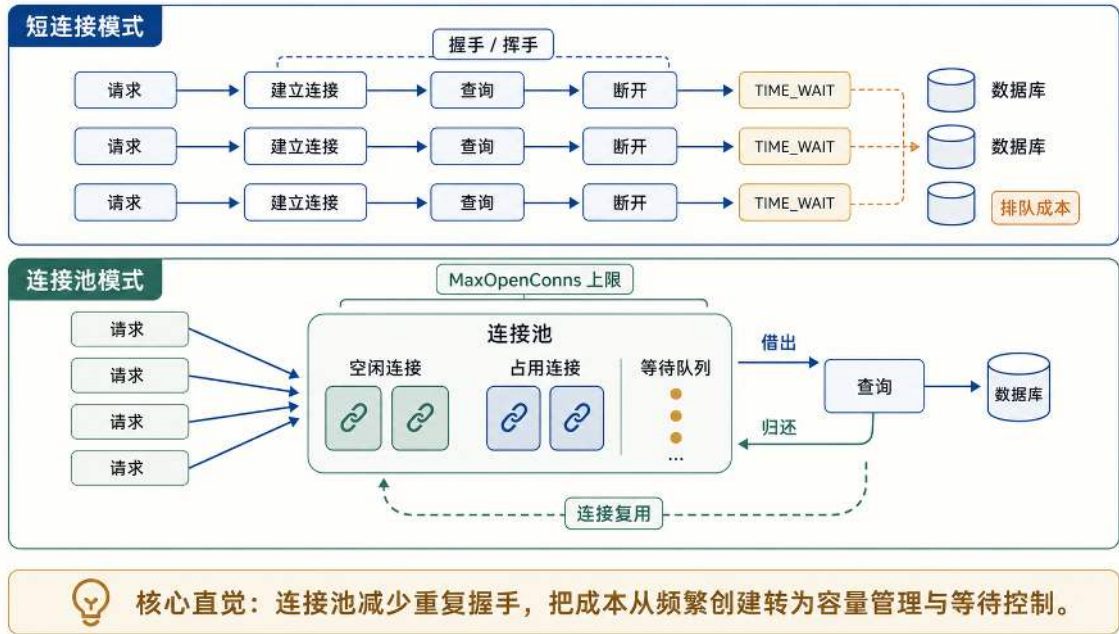


图 29: 连接池通过借出、归还和上限控制，把频繁建连变成可复用的资源循环。

连接池的关键循环很简单：有空闲连接就借出，没有空闲连接且未到上限就新建，达到上限就让请求等待。业务逻辑完成数据库操作后通常会调用 `Close()`（例如关闭查询结果集 `sql.Rows`），但在连接池语义下，这一步往往意味着归还连接，而不是真正关闭 TCP 连接。也正因为上限会带来等待，连接池必须配合超时、等待时间观测和数据库端指标一起使用，不能只把数值调大。

连接池的思路可以概括为：稀缺资源不要随用随造，而是放进一个有上限的池子里，按需借出，用完归还。这类做法可以统一称为池化（Pooling）。池化并不会凭空增加资源总量，它改变的是资源生命周期：把高频创建和销毁，改成可复用、可限额、可观测的循环。云计算里的很多资源管理也沿着这条思路展开，例如虚拟机资源池、容器镜像缓存池、负载均衡器后端池。从本

章的角度看，连接池是在单机内部先练习资源池化：把不可控的创建成本，变成有上限、可观测、可调参的资源管理问题。

3.3.2 对象池：复用短命对象以缓解 GC 压力

除了外部连接资源，程序内部的临时对象分配也可能成为性能瓶颈。高并发服务的网络与序列化路径会频繁创建字节缓冲区（`[]byte`）、请求上下文、编码对象和日志字符串。这些对象的生命周期通常只有几毫秒到几百毫秒，使用完成后很快成为垃圾。

在高并发场景下，这类对象的分配和回收频率可能达到每秒数万次甚至数十万次，对 Go 运行时的垃圾回收器（GC）造成压力。Go 的垃圾回收器通过并发标记等机制缩短全局暂停，但 GC 并非零成本：标记、扫描、写屏障和必要的短暂停都会消耗 CPU，并可能影响缓存局部性。具体影响取决于 Go 版本、堆大小、对象存活率与分配速度，必须通过 GC 暂停、分配率和 P99 延迟等指标实际测量。

对象池把连接池建立的池化思路从外部资源延伸到进程内部：把频繁创建、很快丢弃的临时对象，改成“借出、清理、归还、再次复用”的循环。Go 标准库中的 `sync.Pool` 可以作为这种短命对象缓存的实现。¹⁷当业务代码需要临时对象时，优先从池中取出已有对象；使用完成后，将对象归还到池中而非立即丢弃。通过减少分配次数，对象池可以降低 GC 的触发频率和工作负载。

短命缓冲区的复用路径（示意）

```
1 var bufferPool = sync.Pool{
2     New: func() interface{} {
3         return new(bytes.Buffer)
4     },
5 }
6
7 func handleMessage(data []byte) {
8     buf := bufferPool.Get().(*bytes.Buffer)
9     buf.Reset()
10    defer bufferPool.Put(buf)
11
12    buf.Write(data)
13    processData(buf.Bytes())
14 }
```

这段示例把对象复用拆成三个保证复用正确性的约束。第一，取出的对象在再次使用前必须清理状态，避免受到上一次请求遗留数据的影响。第二，归还对象之前，不能让 `buf.Bytes()` 这样的底层切片继续被外部持有，否则对象回到池里后可能被下一次请求改写。第三，池中的对象

¹⁷`sync.Pool` 的语义限制见 Go 官方文档：<https://pkg.go.dev/sync#Pool>。

不保证一直存在，Go 运行时在 GC 时可能清空池，因此创建函数仍然是必要的兜底，不能假设池中永远有可复用对象。

这种对象池的价值在于把分配与回收的线性生命周期，改造成缓存与复用的循环模式。对于频繁创建且生命周期很短的对象，复用可以显著降低分配率，从而降低垃圾回收（GC）的触发频率与工作量，减少延迟抖动。不过，对象池有其适用边界：它只适合缓存短命对象，不适合作为长期持有的状态存储；需要跨请求保留的状态，仍应使用普通数据结构或专门的缓存系统。

3.3.3 缓存分层架构：热数据与冷数据的协同治理

连接池解决了外部连接的创建与销毁开销，对象池缓解了短命对象的分配与回收压力。至此，单机性能优化还剩最后一个资源维度：数据访问路径本身。在线系统中的数据访问模式存在显著差异：会话状态、热点计数和实时视图会被高频读写，要求较低访问延迟；账号、订单、历史记录和配置等数据访问频率可能较低，但对持久性和一致性要求更高。以游戏为例，玩家位置、在线状态、历史成就和装备配置分别对应上述不同的数据类型，但同样的划分也适用于电商的商品详情与库存、社交的好友动态、模型服务的热点参数等场景。

如果所有热数据都直接打到磁盘数据库，缓存未命中、索引扫描、锁等待或真实磁盘 I/O 都可能把查询拖进队列；即便数据库命中 buffer cache 或操作系统页缓存，高并发下也仍会消耗连接、CPU 与锁资源。反过来，如果所有数据都放在内存中，访问速度虽然快，但内存成本远高于磁盘，断电后也无法满足长期恢复要求。因此，单机阶段同样需要分层存储架构，让不同特征的数据落到合适介质上。

高频热数据：低延迟视图 热数据是指在在线服务运行过程中被高频率读取或修改的数据，这类数据对响应延迟极度敏感。典型场景包括用户会话状态、实时在线人数、热点商品库存、实时排行榜以及限流计数器。这些数据都读写频繁、数据量相对较小，但“能不能丢”的语义并不相同：用户会话短时丢失可能由客户端重新建立，限流计数器短时丢失会影响风控精度，排行榜则往往需要能从权威事件重新构建。

这类热路径通常会放到 Redis 这类内存存储中，使读写避开磁盘路径，优先满足低延迟访问。具体实现中可以通过客户端库访问 Redis，也可以使用批量操作减少往返次数；但这些工具细节服务的仍是同一个机制目标：把高频、低延迟、可重建或可补偿的数据放到更快的介质上。为了降低数据丢失风险，可以配置 Redis 的 AOF（Append Only File）持久化机制，将写操作记录到磁盘日志，在服务器重启后重放日志恢复数据。不过 AOF 也不等于零丢失，具体保障取决于刷盘策略、故障时机与恢复流程。

低频冷数据：权威持久记录 冷数据是指访问频率较低、更新不频繁，但对数据完整性和持久性要求较高的数据。典型场景包括用户注册信息、历史成就记录、基础属性配置、交易历史以及全局配置。这类数据的共同特征是访问频率低、数据量可能很大、必须保证持久性和可恢复性。

这类数据更适合落在 PostgreSQL 这类关系型数据库中，由数据库事务、约束和持久化日志保护权威记录。正文关心的重点不是某个 ORM 如何映射模型，而是哪一层保存最终事实：冷数

据通常可以在 Redis 中保留一份副本用于加速读取；当 Redis 缓存失效或服务器重启时，再从 PostgreSQL 加载权威数据恢复视图。实验代码中可以使用 gorm 这类 ORM 操作数据库，相关表结构和实验入口见附录 B。

Redis 与 PostgreSQL 的分层并不难，难点在于写清两层的职责分工。在两层架构中，Redis 位于低延迟热路径，PostgreSQL 保存权威数据与持久化记录，两层之间的同步关系主要落在三条跨层路径上。路径 A 处理热数据读取：先访问 Redis，未命中时回源 PostgreSQL 并将结果回填缓存。路径 B 处理关键写入：先落到 PostgreSQL 的事务或持久化日志，确保权威记录可恢复。路径 C 处理失效更新：当权威数据变更后，删除旧缓存或更新 Redis 中的派生视图，避免长期返回旧结果。这样，Redis 承担低延迟访问，PostgreSQL 守住持久性和一致性，两层之间的同步规则也不会退化成含糊的“双写”。

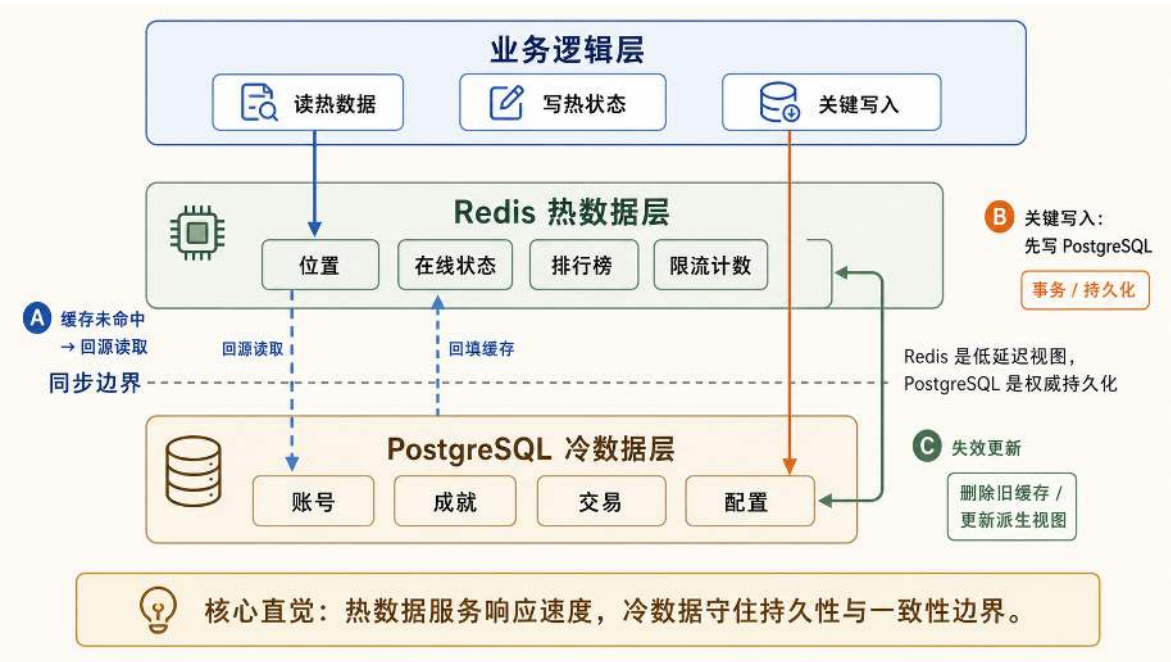


图 30: Redis 承接低延迟热路径，PostgreSQL 保存权威数据；跨层路径要分别处理未命中、关键写入和失效更新。

缓存容量达到上限后，系统必须决定哪些键先被移出，这就是缓存淘汰策略要解决的问题。最常见的两类策略是 LRU（Least Recently Used，最近最少使用）和 LFU（Least Frequently Used，最不经常使用）。LRU 假设最近访问过的数据短期内更可能再次被访问，因此优先淘汰最久没有访问的键。LFU 更看重访问频率，倾向于保留长期高频访问的数据。¹⁸ 在线服务里的会话状态、排行榜与配置缓存访问模式不同，淘汰策略也不应一概而论。临时状态可以设置较短 TTL，让过期机制主动释放内存；全局配置更适合通过版本号或失效通知刷新，而不是完全依赖容量淘汰。

¹⁸ 页面置换与缓存淘汰算法的经典研究可追溯到 Belady 1966，见 L. A. Belady, “A Study of Replacement Algorithms for a Virtual-Storage Computer”, IBM Systems Journal, 1966, <https://doi.org/10.1147/sj.52.0078>。

缓存还会遇到三类经典失效问题。第一类是缓存穿透：请求访问的键本来就不存在，如果每次都绕过缓存访问数据库，恶意请求或异常参数会把压力直接推到后端。常见处理方式是在入口做参数校验，并对“确定不存在”的结果设置短 TTL 的空值缓存。第二类是缓存击穿：某个热点键突然过期，大量请求同时回源，数据库会在短时间内承受集中访问。处理方式可以是互斥回源、提前刷新或逻辑过期。第三类是缓存雪崩：大量键在同一时间失效，导致回源流量集中爆发。处理方式通常是给 TTL 加随机抖动，分散过期时间，并为回源路径设置限流与降级。这三类问题说明，缓存分层不只是把数据放进 Redis；它还要把未命中、回源、淘汰和失效传播做成可控流程。

数据同步策略的选择 在分层存储架构中，Redis 和 PostgreSQL 之间需要明确的数据同步策略，以说明哪一层是权威数据，哪一层只是加速视图。不同的同步模式在一致性保证、性能开销和适用场景上存在差异。

对于关键资产操作，最稳妥的做法是把结果先写入 PostgreSQL 的事务或持久化日志，确保权威记录可恢复，再可靠地更新 Redis 视图。这种“权威写入 + 缓存刷新”模式让最终事实和恢复依据最清楚，但缓存刷新仍可能失败，因此需要可重试的更新或失效机制。该模式的写路径较长，适用于账户转账、订单支付等不容出错的场景。

写穿（Write Through）模式要求每次写操作同时更新 Redis 和 PostgreSQL，写入操作必须等待两个存储系统都确认成功后才返回。它能降低读到旧数据的概率，但 Redis 与 PostgreSQL 的双写本身并不是一个天然的原子事务；如果中间失败，仍然需要重试、幂等键、补偿任务或以数据库事务/事件日志作为权威来源。

写回（Write Back）模式先将数据写入 Redis 并立即返回，随后通过异步任务定期将 Redis 中的数据批量刷入 PostgreSQL。这种模式的写入延迟更低（通常在毫秒级），吞吐量高，但存在数据丢失风险：如果 Redis 在数据尚未刷入数据库时发生故障，未持久化的数据会永久丢失。写回模式适用于允许短时数据丢失的场景，如用户位置同步、临时会话状态、计数器更新等。

失效更新（Cache Aside）模式在写入时只更新 PostgreSQL，并删除 Redis 中的对应缓存。下次读取时，发现 Redis 中没有数据，则从数据库加载并回填到 Redis。这种模式适合读多写少的场景，如静态配置数据（商品属性、系统参数、区域配置等）、历史记录查询等。由于配置数据很少变化，大部分读取都能命中 Redis 缓存，只有在配置更新后的第一次读取会访问数据库。

表3总结了四种数据同步策略的特征与适用场景。在实际工程中，不同类型的数据会采用不同的同步策略。关键资产先保正确性，热路径再谈速度；能丢、能补、能重建的数据，才有空间换取更低延迟。

表 3: 数据同步策略的特征与适用场景

同步策略	一致性/性能	适用场景
权威写入 + 缓存刷新	权威来源清晰, 缓存可能短时陈旧, 写路径较长	账户转账、订单支付等关键资产操作
写穿 (Write Through)	缩短缓存陈旧窗口, 部分失败仍需修复, 写延迟较高	可补偿的同步视图、低风险状态同步
写回 (Write Back)	最终一致, 写延迟低	用户位置、临时状态等允许短时丢失的数据
失效更新 (Cache Aside)	最终一致, 读性能优	商品配置、系统参数等读多写少的静态数据

在实际工程中, 不同操作会根据数据特征选择不同的同步策略。以在线服务为例, 用户位置或会话心跳是最高频的操作, 每秒可能触发数十次更新, 但丢失最近几秒的数据通常影响较小: 服务端可以回滚到最近一次权威快照, 或根据客户端重新上报重新计算。这里要守住服务端权威模型, 客户端上报只能作为输入线索, 必须经过校验规则, 不能直接当作可信状态。因此, 这类高频低敏数据可以采用写回策略, 先写入 Redis 保证低延迟响应, 再由后台任务每隔数秒批量刷入持久存储。

交易与支付事件频率远低于位置更新, 但它们会影响账户余额与订单状态。更稳妥的做法是把交易事件先写入权威事件日志或 PostgreSQL 订单表, 再用 Redis 视图作为可重建的派生缓存, 而不是把“双写成功”当作天然一致。账户余额同理, 作为核心资产, 通常应以数据库事务、账本或事件流作为权威来源, 再可靠地刷新缓存。商品配置、系统参数等静态数据属于典型的读多写少场景, 适合采用失效更新策略: 首次读取时从数据库加载并缓存到 Redis, 后续读取直接命中缓存; 只有在运营人员修改配置时才删除缓存, 触发下次读取重新加载。示例代码中各操作与缓存策略的对应关系, 可结合附录 B 的实验指南阅读。

这种分层存储架构不仅是技术实现的优化, 也是成本与可靠性的工程权衡。Redis 按内存容量计费, 成本较高但延迟更低; PostgreSQL 以磁盘存储为主, 成本更低但延迟更高。通过将热数据与冷数据分离, 可以在满足性能需求的前提下, 把内存规模控制在合理范围。以下数字只用于建立数量级概念, 不是通用工程标准: 以一个 10 万在线用户的实时服务为例, 如果每个用户会话都直接放在 Redis 中, 内存需求可能达到 10GB 量级; 而在分层架构下, Redis 只承载在线状态、当前位置等少量热数据, 单会话只保留 KB 级别即可, 把内存需求压到百 MB 量级, 其余冷数据仍落在 PostgreSQL 上。把性能与成本同时纳入设计目标, 是单机阶段优化能够落地的重要前提。

3.3.4 性能验证：用可观测指标量化优化效果

上述从连接池、对象池到缓存分层的优化设计，都是为了在不拆分应用实例的前提下争取更高的并发承载能力。这些优化最终需要由压测曲线验证：吞吐是否上升，P99 是否下降，CPU、GC、锁等待和连接等待是否转移到新的瓶颈位置。只有建立了可观测的性能基线，才能判断继续优化单机代码是否还有收益，何时继续纵向优化，何时转向第 4 章的横向扩展。到了第 5.3 节，这些指标还会进一步变成弹性伸缩的输入信号。

性能验证的核心在于定义和测量关键指标。对于在线系统，最重要的三类指标是吞吐量、延迟分布和资源占用。

吞吐量 (Throughput) 通常用 QPS (Queries Per Second) 或 TPS (Transactions Per Second) 来衡量，表示系统每秒能够处理的请求数量。对本章的在线服务，可以测量每秒处理的客户端请求数、规则计算数、状态推送数等。吞吐量反映系统的整体承载能力，是评估优化效果最直观的指标。

延迟分布 (Latency Distribution) 关注每个请求从发起到收到响应的耗时。与平均延迟相比，P95 和 P99 延迟（即 95% 和 99% 的请求在多长时间完成）更能反映用户的真实体验。对于实时服务，例如平均延迟只有 10 毫秒级，但 P99 延迟达到 500 毫秒级，意味着每 100 个请求中约有 1 个会遭遇明显卡顿；由于每个用户在一次会话中会发出大量请求，多数活跃用户都可能在某些时刻遇到这种延迟峰值。

资源占用 (Resource Utilization) 包括 CPU 使用率、内存占用、网络带宽消耗以及 GC 统计信息。这些指标用于定位瓶颈所在：如果 CPU 使用率长期接近 90% 但内存充足，说明瓶颈更可能在计算；如果 GC 暂停频繁或分配率过高，说明内存分配模式需要优化；如果网络带宽打满，说明需要优化推送范围、增量消息或序列化路径。

指标能指出哪里不对，但要回答为什么，必须把现象映射到更具体的观测手段。表 4 给出一条常用的定位路径：先看信号，再选工具，最后用细粒度指标归因到代码或结构。

表 4: 从现象到工具：常见瓶颈的定位路径

现象/信号	常见原因	观测手段	重点指标
吞吐低但 CPU 很高	算法慢、热点函数集中	pprof CPU profile	火焰图、Top 函数
延迟抖动且 GC 频繁	分配率高、短命对象多	pprof heap 与分配统计	allocs/op、GC 暂停与频率
并发上来后明显变慢	锁竞争、阻塞等待	pprof mutex / runtime trace	阻塞时间占比、锁等待
goroutine 数持续增长	goroutine 泄漏、退出路径缺失	pprof goroutine	goroutine 数趋势、堆栈归因
请求延迟升高且连接池等待增加	连接池过小、数据库处理能力不足	sql.DBStats + 数据库监控	WaitCount/WaitDuration、活跃与空闲连接
缓存加速收益不稳定	命中率低、集中回源或网络往返升高	缓存指标 + Redis/dis/网络监控	命中率、回源比例、Redis RTT
网络吞吐接近上限	广播放大、序列化路径重	压测 + 网络监控	带宽、重传、发送队列

连接池等待指标用于判断连接复用是否只是把排队转移到了池内，缓存命中率与回源比例用于判断热点读取是否真正离开了数据库。它们应与吞吐和尾延迟一起观察，不能只用单次请求耗时判断优化是否有效。

量化优化效果时，需要设计对比实验。可以先用一个小型压测样本确定指标的大致数量级：这些数字只用于比较趋势，实际结果取决于硬件、网络与实现细节。场景可以设为模拟 100 个并发客户端，每个客户端每秒发送 5 次状态更新请求和 1 次业务计算请求，持续运行 5 分钟，记录吞吐量、P99 延迟以及资源占用。后续要评估数千连接或上万连接的承载上限时，可以沿用同一套指标，只是把并发规模逐级抬高。

对比实验的关键是控制变量：每次只引入一项优化，其余条件尽量保持不变。这样，当吞吐与尾延迟发生变化时，才能把变化可靠地归因到这项优化，而不是把多个变量叠在一起得到一组不可解释的数字。

表 5: 控制变量压测的示意性趋势

实验阶段	吞吐变化	P99 延迟变化	主要观察点
基线版本	偏低	偏高	建连、分配、数据库访问是否排队
加入连接池	可能上升	可能下降	连接等待、数据库端连接数
加入 <code>sync.Pool</code>	小幅变化	可能更稳定	分配率、GC 暂停、堆增长
加入 Redis 热路径	可能上升	可能下降	缓存命中率、回源比例、一致性补偿

在这个基准场景下，如果未引入连接池，频繁建连与断连会把时间花在握手与系统调用上，吞吐容易受限，尾延迟也更容易升高。引入数据库连接池后，复用机制吸收了握手开销，压测时应重点观察连接等待和 P99 延迟是否回落。

进一步引入 `sync.Pool` 减少短对象分配后，GC 触发频率与暂停抖动通常会下降，延迟曲线也可能更稳定。最后，当热路径迁移到 Redis 并采用两级存储策略后，在数据库访问未命中缓存、走磁盘路径时可达十毫秒量级，而 Redis 经网络访问通常在亚毫秒到低毫秒量级。具体差值取决于数据是否命中内存、网络距离、查询复杂度和当前负载，必须由压测结果确认。通过这样的对比实验，不仅能看到每项优化带来的方向性收益，也能识别出当前阶段的主导瓶颈。

更重要的是，性能验证能帮助判断继续优化单机代码是否还有收益。随着并发连接数继续上升，如果看到 CPU 长期接近饱和，而热点函数、锁等待、GC 与数据库访问都被逐项压过，继续微调代码只能带来很小的收益，就说明单机算力可能已接近上限。此时继续堆砌优化技巧的边际收益会下降。相反，把系统拆成多个实例并通过负载均衡分散请求，常常能继续提高承载能力，但提升幅度取决于状态拆分、跨服通信与共享存储瓶颈，并不天然线性。是否转向横向扩展，要比较两条路径的净收益：继续单机优化能否以合理投入满足容量目标，以及拆分带来的容量、弹性与可用性收益能否覆盖状态管理和跨节点协作成本。当继续单机优化的投入增加而容量收益已很有限，且可行的拆分方案能够带来足够收益时，才形成转向第 4 章分布式架构的信号。

3.4 小结：单机并发的机制与限制

本章的第一条主线，是用并发隔离等待传播。进程划分资源与故障影响范围，线程承接操作系统调度，goroutine 把大量等待型任务复用到较少线程上，channel 与 `select` 则把消息交接、超时和背压写进程序。

第二条主线，是用同步保护共享状态。库存扣减、余额转账、协作文档提交、限量抢购、NPC 奖励归属，本质上都要求检查与修改遵守确定顺序。`mutex` 可以保护临界区，条件变量可以补上等待与通知，锁粒度则决定正确性需要付出多少串行化成本。

第三条主线，是通过资源复用与数据分层降低单次操作的固定开销。连接池、`sync.Pool` 和分层存储分别减少反复建连、短命对象分配和慢速访问带来的成本，压测与观测指标负责验证这些优化是否真正改善吞吐、尾延迟和资源占用。

从本书的原型演进来看，本章完成的是单机优化阶段的主体工作：先明确单机内的并发潜力、资源复用方式和数据访问路径，再用压力测试和性能监控建立基线。如果增加单个实例的 CPU、内存或其他资源，才属于纵向扩展（Scale Up）。这样才能量化地回答几个问题：当前硬件能支撑多少并发请求或连接，每项优化带来了多大收益，主要瓶颈还位于哪里。

判断系统是否应转向横向扩展（Scale Out）时，不能只看某个固定阈值，而要综合观察一组信号。第一，CPU、网络带宽、连接数或内存占用长期逼近安全水位，继续加压会直接推高队列长度和 P99 延迟。第二，热点函数、锁等待、GC、连接池和数据库访问已经逐项优化，局部改动只能带来很小收益。第三，性能瓶颈开始集中在单机不可再分的资源上，例如单进程事件循环、单库写入、单一状态分区或单个网络出口。第四，系统已经能够说明按用户、对象、区域或服务职责拆分后的状态归属、拆分键和跨实例交接规则。当多项信号持续出现，且拆分带来的容量、弹性或可用性收益能够支撑额外的状态管理与跨节点协作成本时，第 4 章的分布式架构才成为可得出的工程选择。

第 4 章将从单台服务器进入分布式阶段。此时的核心矛盾会从单机内的并发效率，转向多个实例之间怎样分配请求、同步状态并在故障时继续协作。电商订单、协作文档、即时通讯、模型任务以及在线游戏中的跨服裁决和玩家迁移，都需要处理跨节点的数据同步、全局顺序与故障恢复。再往后，第 5.2.4 节会把这些服务放到集群编排环境中，第 6.1 节会进一步解释容器如何把进程、资源限制与隔离机制工程化。

3.5 思考题

1. 在顺序主循环里，服务器会依次从每个连接读取输入、推进逻辑、再推送状态。假设其中一个连接的读取偶尔阻塞到 500ms，而其余连接都在 10ms 内就绪。请用因果链解释为什么延迟会从一个连接传播到所有连接，并指出把收包、处理与推送拆开后，哪一段等待被隔离了。然后思考：当我们引入并发后，哪些共享状态最容易首先被写穿，从而把问题从性能转移为正确性。
2. 某个实现把每个连接的收包与处理拆成 goroutine，发现平均延迟变化不大，但 P99 明显下降；另一个实现把规则计算改为多核并行，单轮计算时间下降但 CPU 使用率上升。请分别判断这两种改造更接近并发还是并行，并说明它们各自主要缓解的瓶颈是什么。再思考：在多人在线场景里，什么时候并发比并行更优先，什么时候反过来。
3. 为了做数量级估算，假设操作系统线程需要预留 1MB 栈虚拟地址空间，而 goroutine 的初始栈为 2KB。若服务器要承载 20,000 个在线连接，分别用一连接一线程和一连接一 goroutine 组织读写路径，单从栈预留量角度看两者大约相差多少个数量级。这里计算的是理想化的预留量，不代表进程实际驻留内存；然后思考：除了栈空间之外，

还有哪些因素会限制并发规模，为什么这也是系统要可观测的原因。

4. 本章强调同步与异步描述的是接口语义，而阻塞与非阻塞描述的是调用行为。请把下面四种接口放到图24的四个象限中，并说明理由：`Read()` 一直阻塞到结果、`TryRead()` 立即返回 (`data, ok`)、`ReadAsync(cb)` 回调返回结果、`ReadAsync()` 返回句柄并允许 `Wait()` 等待。再思考：在线服务的收包路径和落库路径各适合哪些组合，为什么。
5. `channel` 常被当作连接内的事件队列。请结合多人在线的消息处理链路解释：缓冲区过小会把什么压力暴露到上游，缓冲区过大又可能把什么问题拖到更晚才爆发，从而放大尾延迟。然后思考：如果要在高压下避免无休止排队，超时与降级应该写在什么位置，并用哪些可观测指标验证它确实保护了系统而不是掩盖了问题。
6. 以 NPC 抢宝为例，请写出必须被维护的最小不变量（例如同一份宝物最多只会被领取一次），并指出检查条件与修改状态之间的竞态窗口位于哪里。随后说明互斥锁或单写者模型如何把这段关键区收敛成唯一顺序。在此基础上切换到容量有限的任务队列场景：当队列为空时消费者需要等待，当队列满时生产者也需要等待，为什么 `Wait()` 必须放在 `for` 循环里复检条件，才能让正确性不依赖侥幸的调度时机。
7. 假设 Go 没有提供 `sync.Cond`，请仅使用 `channel` 和 `select` 重新设计一个容量有限的任务队列：队列为空时消费者应如何等待，新任务入队后如何唤醒后续消费者，又如何处理超时或服务取消。请对比这种消息传递方案与“互斥锁 + 条件变量”方案在状态归属、多等待者唤醒、超时取消和实现复杂度方面的取舍。
8. 双对象操作是死锁的高发区。请以转账或交易结算为例，构造一个最小并发时序使两条路径形成环等待，并给出一种统一的加锁顺序规约来消除环。然后思考：如果事件处理引入优先级，为什么仍可能出现饥饿，怎样的最低保障策略能把等待时间收敛到可接受的范围内。
9. 本章引入了连接池、`sync.Pool` 与缓存分层三类优化。请设计一个控制变量的对比实验：每次只引入一项优化，记录吞吐、P95/P99、CPU、内存、GC 与阻塞时间等指标，并说明如何根据观测结果判断瓶颈主要来自 I/O 等待、锁竞争还是 GC 抖动。再思考：当单机优化的边际收益显著下降时，哪些信号最能支持转向分布式扩展的决策。

4 第四章 切分世界：分布式系统

第三章把客户端连接从双人推进到单机多人同图：服务器能够同时维护多条连接，按固定节奏处理请求，并通过同步机制保护共享状态。连接池、对象池、缓存分层和压测指标进一步说明了一台服务器内部的资源使用方式。至此，单机版本已经具备了承载多用户在线负载的基本结构，但所有连接、计算、缓存访问和持久化写入仍然汇聚到同一台机器上。

当用户规模继续扩大，单机优化会逐渐遇到无法绕开的资源上限。在线用户数和请求到达率继续增加时，单台机器的 CPU 要处理更多请求计算和消息广播，网络要发送更多状态同步消息，数据库和缓存也要承受更高频的查询与写入。细化锁粒度、减少对象分配、调整连接池容量等方式具有一定价值，但这些优化只能改善单台机器内部的效率，不能让单台机器拥有更多物理资源。

提高系统容量有两条基本路径：一条是为现有服务器增加处理器、内存或网络带宽，这称为纵向扩展；另一条是增加服务器，让多台机器共同承担负载，这称为横向扩展。本章所说的多机扩展主要指横向扩展：把原来集中在一台机器上的连接、请求计算和数据访问分配给多个节点处理，使新增节点能够承担一部分负载。换一种视角看，单机优化是在做除法，即通过更精细的资源管理压缩每次操作的开销；多机扩展则是在做乘法，即通过增加节点叠加系统可用的资源总量。这种叠加并不保证节点数增加一倍，系统容量就一定增加一倍；跨节点通信、协调开销和负载不均都会降低实际扩展收益。

单机架构中只有一条从客户端到应用服务器再到数据库的固定路径：请求由哪台服务器处理、数据保存在哪里、一次操作由哪个进程裁决，都由这条路径确定。当连接、计算和数据访问分散到多个节点后，这些默认关系随之消失。一条请求进入系统后，入口必须先从多个实例中选择处理者；操作跨越多个节点时，参与节点必须形成一个可确认的结果；已经承担处理任务的节点失效时，系统还要选择接管者，并判断此前哪些操作已经生效。增加节点由此把单机中由固定路径确定的处理关系，转化为请求分配、结果确认和故障接管三类显式决策。图 31 通过单机与多机请求路径的对照，说明这三类决策为什么会随横向扩展一同出现。

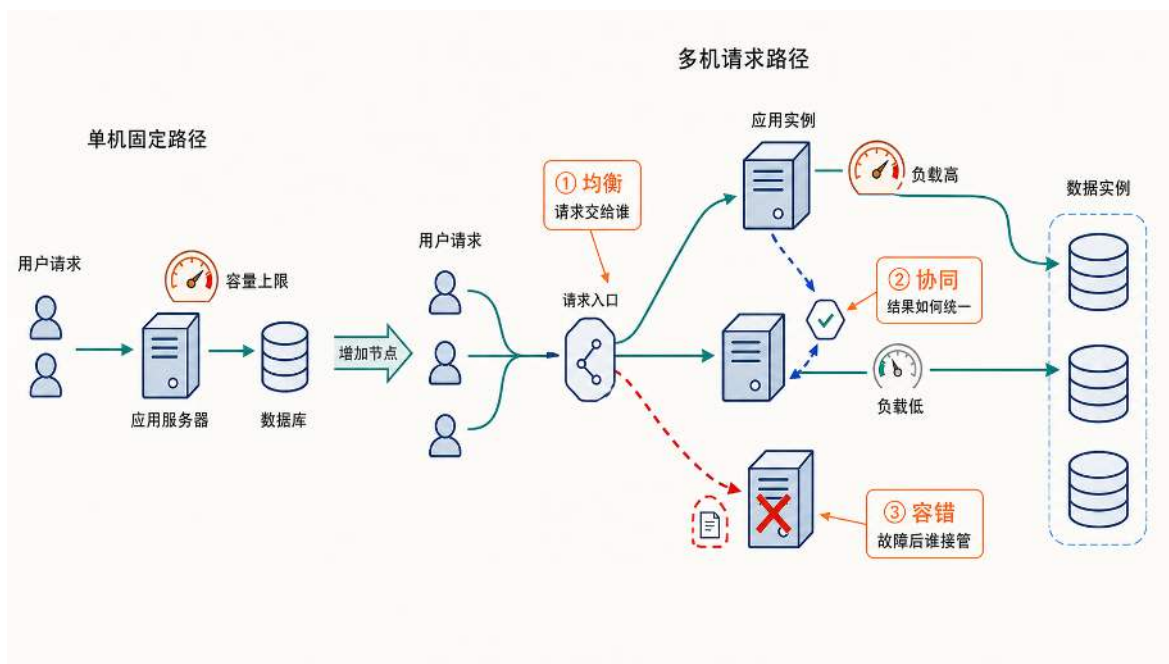


图 31: 横向扩展将单机中的固定处理关系转化为三类显式决策：入口分配请求，节点协同确认结果，故障后重新接管服务。

这三类决策分别形成多机系统的三个设计维度。**均衡**关注如何把请求、数据和计算对象分配到合适的实例，请求分配和数据拆分都属于这类问题。**协同**关注多个实例如何共同完成一次操作，并满足该操作所需的一致性要求，跨节点裁决是其中的典型场景。**容错**关注节点故障后如何恢复服务，并明确哪些操作已经生效。只有建立相应的分配、协同和接管规则，新增节点提供的资源才能被请求实际利用，并在跨节点操作和节点故障发生时维持正确服务。

本章先区分多机扩展中的两种基本拆分方式，再沿均衡、协同和容错三个维度展开相应机制及其取舍。完成本章后，读者应能判断对象应如何分配，多实例应如何共同完成操作，以及系统应如何在节点故障后恢复服务。

4.1 多机扩展的两个拆分维度：职责与对象

从单机进入多机后，新增机器需要承接原有系统中的一部分工作。设计者必须回答两类问题：哪些功能应由不同服务负责，同一类服务中的用户、会话、计算任务或数据记录又应由哪个实例负责。前者是按职责拆分，后者是按对象拆分。它们不是相互替代的两种方案：系统通常先按职责分开功能，再在每类职责内部按对象分摊容量。

按职责拆分先确定每类服务承担什么功能。单机阶段，连接接入、状态推进、消息路由、任务排队和数据读写可以放在同一个进程中完成；规模扩大后，这些功能的资源需求、故障影响和迭代节奏逐渐分化。系统可以据此拆出维护连接和转发消息的网关服务、处理请求和推进状态的业务逻辑服务，以及负责缓存访问和持久化写入的数据服务。职责分开后，各类服务可以独立演进，某一类服务的故障也更容易被限制在相应的职责范围内。

但是，按职责拆分只改变了功能由哪一类服务承担。如果每类服务仍然只有一个实例，网关仍无法维护不断增长的连接，逻辑服务仍无法处理所有房间或任务，数据服务也仍无法保存和访问全部用户记录。为了继续扩大容量，系统还要在每类职责内部部署多个同类实例，并让不同实例分别负责一部分对象，这就是按对象拆分。网关按连接、会话或入口区域做**连接分片**，计算服务按房间 ID、任务 ID 或游戏实例 ID 做**计算分片**，数据库按用户 ID、组织 ID 或区域 ID 做**存储分片**。三类分片共同解决单一服务实例的容量上限。

一次用户请求可以同时经过这两个拆分维度。系统先根据职责把请求交给网关服务，再根据连接归属选择一个网关实例；网关把请求转交给业务逻辑服务后，还要根据房间、任务或游戏实例的归属找到具体的计算实例；读取用户状态时，又要根据用户 ID 找到相应的数据分片。按职责拆分决定请求需要经过哪些服务，按对象拆分决定请求在每类服务中落到哪个实例。

本章后续首先沿按对象拆开展开，因为同类实例增加后，系统必须确定对象归属、实例映射、请求路由和动态调整规则，这些问题构成下一节讨论的均衡机制。与此同时，按职责拆分把单体程序中的函数调用变成了跨服务网络调用，按对象拆分又使一次操作可能跨越多个实例；由此产生的超时、部分失败、结果确认和故障恢复问题，将在后面的协同与容错部分继续讨论。

4.2 均衡：分片与动态调整

多机扩展中的**均衡**，是把请求、数据和计算负载分散到多个实例，避免少数实例过载而其他实例长期空闲，使集群中的资源得到有效利用。均衡并不要求每个实例承受完全相同的压力；当机器容量、对象访问频率和处理成本不同时，更重要的是让各实例的负载保持在可承受范围内，并避免长期热点。

增加实例本身不会自动形成均衡。系统必须建立对象到实例的分配规则，使连接、计算任务和数据能够分散到合适的位置，这种按对象划分负载的方法称为**分片**。分片决定初始归属；当热点、实例数量或节点状态发生变化时，系统还需要根据监测结果动态调整归属，才能在运行过程中继续维持均衡。

在本章讨论的在线系统中，对象分片主要出现在三个位置：网关层分配连接和会话，形成**连接分片**；逻辑服务层分配房间、任务或游戏实例，形成**计算分片**；存储层分配用户或组织数据，形成**存储分片**。图 32 将三类分片各自处理的对象、常用分片键和目标实例放在同一条请求路径中，并给出它们共享的处理机制。

本节先从三类分片出发，讨论分片键以及范围分片、哈希分片和一致性哈希等映射策略；再讨论分片路由、负载监测和动态调整，说明请求如何找到目标实例，以及系统如何应对热点与扩容；最后进入存储层和缓存层，分析跨分片查询、跨分片事务与集群化带来的具体问题。

4.2.1 分片键与基本映射策略

选择分片键：划分对象的依据 把负载分散到多台机器之前，系统要先确定哪些对象应当保持共同归属。同一会话的后续消息需要回到保存会话状态的网关；同一房间的参与者需要由一个逻辑服务实例共同推进状态；同一用户的资料、配置和业务状态也通常需要存放在同一个数据分

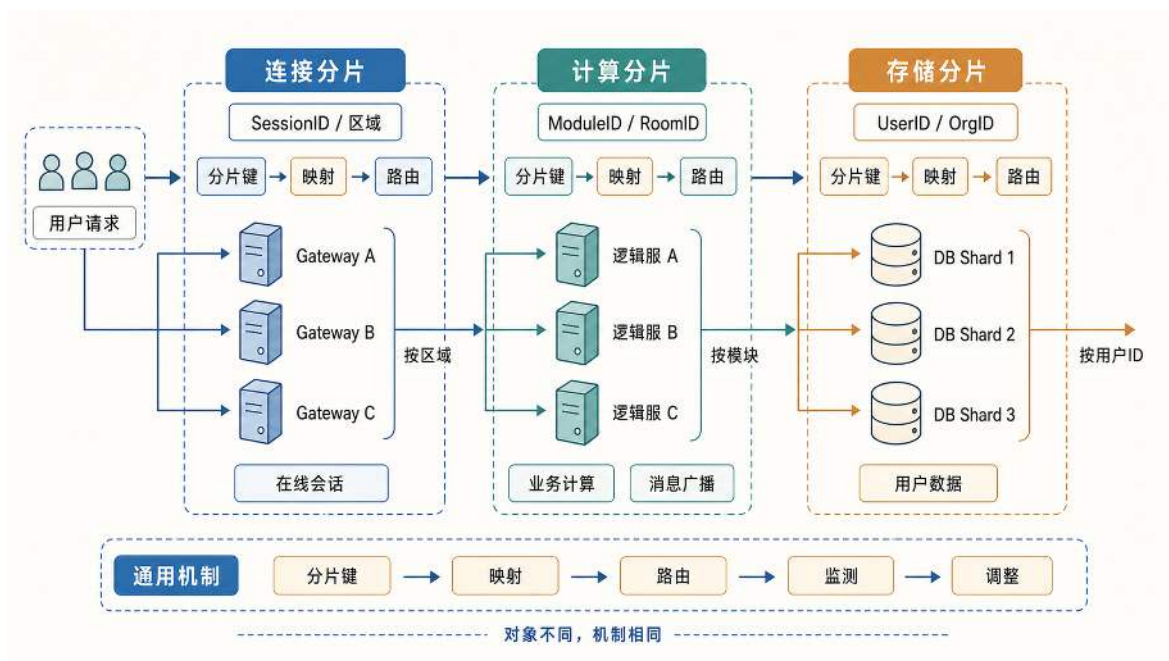


图 32: 连接、计算和存储分片处理不同对象，但都通过分片键、映射和路由建立对象归属，并通过监测和调整维持运行时均衡。

片。系统需要从请求中取得一个稳定标识，把这些相关对象归入同一组。

这个用于表示对象归属的标识称为分片键 (Sharding Key)。连接分片通常使用会话 ID、连接 ID 或接入区域，计算分片通常使用地图 ID、房间 ID 或游戏实例 ID，存储分片通常使用用户 ID 或组织 ID。只要分片键保持不变，不同服务器在不同时间处理同一对象时，就能依据相同标识查找它的归属。

一个合适的分片键首先要避免负载集中。如果交易记录按日期分片，当天产生的新记录会全部写入最新分片；当常见访问围绕用户展开时，改用分布较散的用户 ID，写入更容易落到多个分片。不过，字段值分散并不意味着实际压力一定均匀，少数高频用户或大型组织仍可能形成热点。

分片键要尽量保留访问局部性。用户资料若按物品类型拆分，查询一个用户的全部数据就要访问多个分片；按用户 ID 拆分，则可以把这次查询限制在一个分片。实时计算也有相同要求：以房间 ID 为键，可以让需要共同推进状态的参与者留在同一计算实例，减少跨节点同步。

负载均匀性和访问局部性有时会相互制约。划分过细可以把负载分得更散，却可能拆开需要共同处理的对象；划分过粗可以减少跨分片访问，却容易让大对象形成热点。确定分片键，就是在这两项要求之间选择适合当前访问模式的归属单位。

分片键只确定哪些对象属于同一组，还没有指定这一组由哪个实例负责。下面以用户 ID 和数据库实例为例，比较范围分片和哈希分片两种基本映射策略。

范围分片：按连续区间分配实例 范围分片 (Range-based Sharding) 按照分片键的数值或字典序区间分配实例。假设有三个数据库实例，可以按 ID 数值范围分配：ID 从 1 到 1,000,000 的用户存放在第一个数据库 (分片 0)，ID 从 1,000,001 到 2,000,000 的存放在第二个数据库 (分

片 1)，ID 从 2,000,001 以上的存放在第三个数据库（分片 2）。这种方式的优点在于规则简单明了，容易理解和管理。当需要查询某个用户的数据时，只需要简单地判断 ID 落在哪个区间即可。扩容时也可以把新的 ID 范围交给新分片，未来注册的用户会自然流向新机器。

但范围分片有一个明显隐患：负载热点。如果用户注册在时间上非常集中，比如系统刚上线或举办大型活动期间，短时间内会有大量新用户涌入，他们会被连续分配到相邻的 ID 区间。假设一周内新注册了 50 万用户，ID 都落在 2,000,001 到 2,500,000 这个范围内，全部被分配到分片 2。而这些新用户往往最活跃，登录、操作、做任务都更频繁，分片 2 的负载就会明显高于其他分片。新建一个分片只能承接后续用户，不能自动减轻这个已经变热的区间。要处理已有热区，还是得把旧区间拆开，把一部分存量数据搬出去。

哈希分片：打散连续键的分布 为了避免连续区间形成热点，可以使用哈希分片（Hash-based Sharding）。具体做法是：对每个用户 ID 执行一个固定的哈希运算，将结果对数据库实例数量取模，根据余数来决定数据存放的位置。用伪代码表示如下：

哈希分片的映射计算

```
1 // 假设有4个数据库分片
2 const numShards = 4
3
4 // 根据用户ID计算目标分片
5 func getShardID(userID int) int {
6     // 教学示例：为便于观察取模效果，这里直接用 userID 数值代替其哈希结果；
7     // 真实系统中应先经过 MurmurHash 等确定性哈希函数。
8     hash := userID
9     // 对分片数量取模，得到分片编号（0-3）
10    shardID := hash % numShards
11    return shardID
12 }
13
14 // 示例：不同用户ID被分配到不同分片
15 // 用户1 -> 分片1，用户2 -> 分片2，用户3 -> 分片3
16 // 用户4 -> 分片0，用户5 -> 分片1，用户6 -> 分片2
17 // 即使ID连续，也会被打散到不同分片
```

由于哈希函数对不同输入产生的输出在数值空间上分布较为均匀，即使用户 ID 在时间上连续注册（1, 2, 3, 4...），经过哈希和取模之后也会被打散到不同的数据库实例中。哈希函数在这里的作用，是把用户 ID 转成一个稳定且分布较均匀的整数结果。真实系统通常会把用户 ID 转成固定格式的字节串，再交给一个确定性的哈希函数，例如 MurmurHash、xxHash 或 FNV。只要输入用户 ID 相同，任何一台应用服务器、任何一次进程重启后都应得到相同哈希值。随后系统再用这个哈希值对分片数量取模，得到目标分片。上面的代码为了便于观察，直接用用户 ID 代

替哈希值；真实路由实现中，确定性哈希函数可以减少 ID 生成规则、批量导入或特殊编号带来的分布偏差。

哈希分片把连续注册的用户打散到不同分片，缓解了范围分片中的新用户热点。但这个规则有一个前提：分片数量保持不变。当系统从 4 个分片扩展到 5 个分片时，路由公式会从 $\text{hash}(\text{userID}) \% 4$ 变成 $\text{hash}(\text{userID}) \% 5$ 。取模基数一变，许多用户的余数都会改变；即使用户 ID 和哈希函数都没有变化，目标分片也可能变成另一台数据库。取模基数变化会直接改写对象与实例之间的映射关系。图 33 以用户 ID 为例，对比从 4 个分片扩展到 5 个分片前后的归属结果。

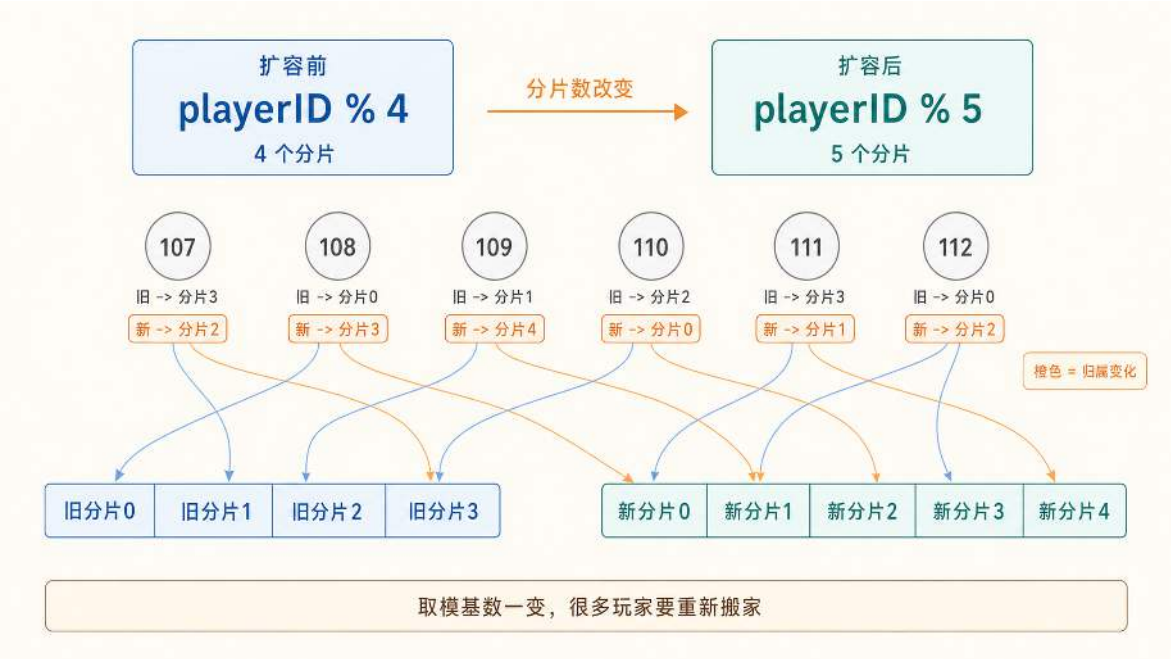


图 33: 以用户 ID 为例，普通哈希分片从 4 个分片扩展到 5 个分片后，取模基数变化导致大量对象重新归属。

图中的用户在扩容后都改变了归属。例如，用户 107 原本在分片 3 ($107 \% 4 = 3$)，扩容后被分到分片 2 ($107 \% 5 = 2$)。这组样本并不表示每个用户都一定迁移，但说明取模基数变化可能使大量用户重新计算目标分片。系统必须重新判断每个用户应该迁到哪里，再搬动大量数据。对于数百万用户的系统，这类迁移耗时长，也容易影响线上服务。

这正是普通哈希分片的主要缺点：对象归属与当前实例数量直接绑定。实例增加、减少或故障退出时，取模基数随之变化，大量对象会被重新映射到其他实例，进而产生大规模数据迁移和服务切换开销。要降低这类开销，映射策略需要把节点变化的影响限制在局部范围，而不是重新计算大部分对象的归属。

4.2.2 一致性哈希：限制节点变化的影响范围

一致性哈希（Consistent Hashing）¹⁹ 的主要优点，是在节点集合变化时保留大部分已有映射关系。新增节点时，只有相邻区间中的部分对象转移到新节点；删除节点时，也只有原来属于该节点的对象需要迁出，其余对象与留存节点之间的归属关系保持不变。因此，一致性哈希能够显著减少扩缩容和节点退出时的迁移范围。

为实现这种局部变化，一致性哈希不再把哈希值直接对节点数量取模，而是把实际数据库节点和用户 ID 映射到同一个首尾相接的哈希空间，再根据对象与节点在空间中的相对位置确定归属。它减少的是节点变化引起的迁移范围，并不自动保证负载均匀；节点在哈希空间中的位置偏斜还需要通过虚拟节点缓解，少量热点键也需要其他机制处理。这里的实际节点也称为物理节点，指真正承担数据读写的数据库实例，并不要求每个实例独占一台物理服务器。最简单的做法是让每个实际节点在哈希空间中只对应一个位置。查询用户数据时，系统计算用户 ID 的哈希值，再沿顺时针方向找到第一个节点位置，并访问该位置对应的数据库实例。但是，少量实际节点经过哈希后未必均匀分布：相邻节点之间的区间可能大小悬殊，负责较大区间的数据库会承载更多数据和请求。

为减小这种偶然偏差，系统为同一个实际节点生成多个分散的代表位置，这些位置称为虚拟节点。例如，db1#0、db1#1 和 db1#2 是实际数据库 db1 在哈希空间中的三个代表位置。用户 ID 命中其中任意一个虚拟节点时，最终访问的仍然是 db1。所有实际节点都拥有多个虚拟节点后，每台数据库负责的区间会被拆成散布在哈希空间中的多段小区间，数据和请求通常能够分布得更均匀。新增实际节点时，只有各新虚拟节点与其逆时针方向前一个节点之间的区间改变归属，其他区间保持不变。

一致性哈希需要同时回答两个问题：正常请求怎样找到负责该对象的节点，节点加入后又有哪些对象必须改变归属。在图 34 中，这两个过程放在同一个哈希环上。数据库节点 A、B、C 先通过多个虚拟节点映射到环上；用户 ID 经过哈希得到环上的数据键位置，再沿顺时针方向找到最近的虚拟节点，并由该虚拟节点确定实际数据库。新增节点 D 时，新的虚拟节点插入原有顺序，只接管各插入位置之前的一小段区间；环上其他区间的顺序和归属保持不变。

¹⁹一致性哈希最早由 Karger 等人在 Web 缓存场景中系统提出，见 David Karger et al., “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web”, STOC 1997, <https://people.csail.mit.edu/karger/Papers/web.pdf>。

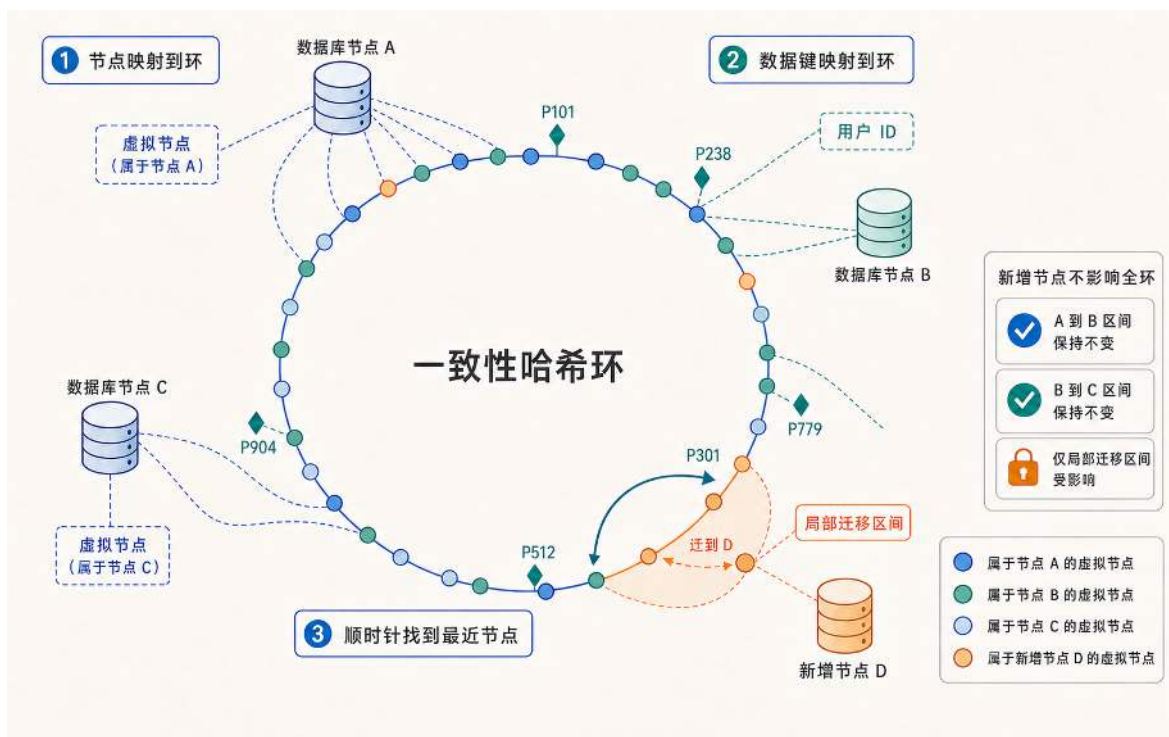


图 34: 物理节点通过多个虚拟节点映射到哈希环，数据键沿顺时针方向找到归属节点；新增节点 D 只接管插入位置相邻的局部区间。

正常查询和扩容迁移都依赖同一个条件：系统必须稳定保存虚拟节点在环上的顺序，以及每个虚拟节点对应的物理节点。圆环只是这种循环顺序的可视化；实际实现通常使用一张按哈希值排序的路由表，而不需要维护圆形数据结构。表中的每一项以虚拟节点在哈希空间中的整数位置为键，以对应的物理节点为值。有了这组有序关系，系统便可以依次完成三个动作：建立哈希环、查询对象归属，以及在节点变化时计算迁移区间。

下面的算法一实现图中的第一步，把每个物理节点展开为多个虚拟节点，并建立可供后续查询的有序路由状态。结构体 `HashRing` 中，`ring` 记录“虚拟节点位置到实际数据库节点”的映射，`sorted` 保存所有虚拟节点位置，并按从小到大排序。`BuildRing` 负责初始化这两组数据，`AddNode` 则把一个实际数据库展开成多个虚拟节点并插入哈希环。

算法一：构建一致性哈希环

```
1 type HashRing struct {
2     ring          map[uint32]string
3     sorted        []uint32
4     virtualNodesPerNode int
5 }
6
7 func BuildRing(nodes []string, virtualNodesPerNode int) *HashRing {
8     r := &HashRing{
9         ring:          make(map[uint32]string),
10        virtualNodesPerNode: virtualNodesPerNode,
11    }
12    for _, node := range nodes {
13        AddNode(r, node)
14    }
15    return r
16 }
17
18 func AddNode(r *HashRing, node string) {
19     for i := 0; i < r.virtualNodesPerNode; i++ {
20         key := hash(fmt.Sprintf("%s#%d", node, i))
21         r.ring[key] = node
22         r.sorted = append(r.sorted, key)
23     }
24     sort.Slice(r.sorted, func(i, j int) bool {
25         return r.sorted[i] < r.sorted[j]
26     })
27 }
```

每个实际节点对应的虚拟节点数 `virtualNodesPerNode` 需要在负载均衡精度和维护成本之间取一个折中。虚拟节点越多，各物理节点在环上的分布越均匀，负载偏差越小；代价是哈希环条目更多，内存占用和二分查找开销略有增加。虚拟节点数没有适用于所有系统的固定推荐值，应根据节点数量、哈希函数、键分布、实际请求偏斜和路由表维护成本进行测量。以 4 个数据库分片、每个分片 150 个虚拟节点为例，哈希环上共有 600 个条目；这个数字适合用来说明路由表规模，但是否足以平滑负载仍要由线上指标验证。

哈希环建立后，算法二实现图中的第二、三步：先把用户 ID 映射为环上的数据键位置，再沿顺时针方向找到负责该位置的虚拟节点。在有序数组 `sorted` 中，这等价于查找第一个不小于数据键位置的虚拟节点；如果查找越过数组末尾，就回到数组开头，以保持环形顺序。最后通过 `ring` 取得该虚拟节点对应的实际数据库。只要节点集合不变，同一个用户 ID 就会得到相同的归

属结果。

算法二：根据哈希环查询对象归属

```
1 func Lookup(r *HashRing, userID string) string {
2     if len(r.ring) == 0 {
3         return ""
4     }
5     h := hash(userID)
6     idx := sort.Search(len(r.sorted), func(i int) bool {
7         return r.sorted[i] >= h
8     })
9     if idx == len(r.sorted) {
10        idx = 0
11    }
12    return r.ring[r.sorted[idx]]
13 }
```

算法二能够正确查询的前提，是数据已经存放在当前哈希环所指定的数据库中。扩容时，如果先调用 `AddNode` 修改正式哈希环，请求就可能被送到尚未接收数据的新节点 `D`，导致查询失败。因此，系统不能先切换归属再搬数据，而要先根据旧环和新节点计算迁移范围。这正对应图中的橙色局部区间，算法三把它转化为迁移计划。

新增物理节点 `D` 会在环上产生若干新虚拟节点。对每一个新虚拟节点 v_{new} ，设它的位置为 p_{new} ，它在新环中的逆时针前驱位置为 p_{prev} ，那么只有原本落在区间 $(p_{prev}, p_{new}]$ 内的数据改由 `D` 负责。算法三的任务不是直接复制数据，而是找出这些区间原来所属的节点，并输出“来源节点、目标节点、区间起点和区间终点”，供后台迁移任务执行。

在哈希分布足够均匀、虚拟节点数量足够多时，整体迁移量的期望值大约是全部数据的 $\frac{1}{N+1}$ ，其中 N 是原有物理节点数。如果原来有 4 个节点，新增第 5 个节点大约只需要迁移 20% 的数据。普通取模哈希在同样场景下会让大量用户重新定位，迁移量通常更高。

为生成这份计划，算法必须同时保留扩容前后的两种节点顺序。旧环 `r.sorted` 用于判断数据当前属于哪个节点；算法复制这组位置得到 `finalSorted`，再把新实际节点对应的虚拟节点位置记录到 `newKeys`，并加入 `finalSorted` 重新排序，由此得到扩容后的节点顺序。对于每个新虚拟节点，它在新环中的前驱与自身位置共同确定需要迁移的区间；同一位置在旧环中对应的节点，则是这段数据的来源节点。算法将来源节点、目标节点和区间端点组合成 `MigrateRange`，供后续迁移任务逐段搬运数据。

算法三：新增节点时计算需要迁移的数据范围

```
1 type MigrateRange struct {
2     From      string
3     To        string
4     StartKey  uint32
5     EndKey    uint32
6 }
7
8 func MigrateRanges(r *HashRing, newNode string) []MigrateRange {
9     // 迁移计划基于旧环计算；finalSorted 只用于推演加入新节点后的区间
10    finalSorted := append([]uint32(nil), r.sorted...)
11    newKeys := make([]uint32, 0, r.virtualNodesPerNode)
12    seenNew := make(map[uint32]bool)
13
14    for i := 0; i < r.virtualNodesPerNode; i++ {
15        newKey := hash(fmt.Sprintf("%s#%d", newNode, i))
16        if _, exists := r.ring[newKey]; exists || seenNew[newKey] {
17            continue // 教学代码中直接跳过哈希碰撞
18        }
19        seenNew[newKey] = true
20        newKeys = append(newKeys, newKey)
21        finalSorted = append(finalSorted, newKey)
22    }
23    sort.Slice(finalSorted, func(i, j int) bool {
24        return finalSorted[i] < finalSorted[j]
25    })
26
27    var ranges []MigrateRange
28    for _, newKey := range newKeys {
29        // 前驱必须在完整新环中查找，避免多个新虚拟节点产生重叠区间
30        finalIdx := sort.Search(len(finalSorted), func(j int) bool {
31            return finalSorted[j] >= newKey
32        })
33        prevIdx := (finalIdx - 1 + len(finalSorted)) % len(finalSorted)
34        prevKey := finalSorted[prevIdx]
35
36        // 数据在扩容前属于旧环中顺时针方向的第一个节点
37        oldIdx := sort.Search(len(r.sorted), func(j int) bool {
38            return r.sorted[j] >= newKey
39        })
40        srcNode := r.ring[r.sorted[oldIdx%len(r.sorted)]]
41        ranges = append(ranges, MigrateRange{
42            From:      srcNode,
```


当多个新虚拟节点落在同一个旧区间内时，它们会按照新环顺序形成互不重叠的小区间，扩容时可以分批迁移。

迁移区间确定后，线上扩容还需要把这些区间转化为可执行的迁移流程。比较稳妥的做法是先计算迁移范围，再由后台任务分批搬迁存量数据；搬迁期间继续记录源分片上的增量写入，等存量搬完后追平增量；随后校验两边数据，再把这一小段哈希区间的路由切到新节点。切换后通常还会保留一段回滚窗口，确认没有异常再清理旧数据。这种流程增加了迁移步骤，但可以把停机时间和故障影响控制在更小范围内。

三个算法由此覆盖哈希环从建立、使用到扩容的完整过程。系统初始化时，算法一建立虚拟节点的有序状态；节点集合稳定时，算法二根据该状态查询每个对象的归属；新增节点时，算法三先基于旧环推演新环并生成迁移计划，系统完成数据搬迁和校验后，再由算法一把新节点加入正式哈希环。正式环更新后，算法二才会把相应对象的后续请求指向新节点。

到这里，分片策略已经形成了一条完整逻辑判断链：分片键决定用哪个业务字段表示对象归属，范围分片、哈希分片和一致性哈希决定这个字段如何映射到节点；节点变化时，一致性哈希还可以限制并计算需要迁移的区间。算法二虽然能够算出目标节点，但它返回的只是节点标识，还没有建立网络连接或转发请求。因此，以上内容回答的是“对象的归属”，下一步还要回答“请求怎样真正到达”。

4.2.3 分片路由

分片策略和一致性哈希确定了对象归属以及节点变化时的迁移范围，但一次请求进入系统后，还要把算法二得到的节点标识转化为实际请求路径。分片路由负责完成这一过程，负载监测则为后续动态调整提供输入。在存储层，这意味着从用户 ID 找到目标数据库连接；在计算层，这意味着从模块 ID 找到目标逻辑服地址；在连接层，这意味着从会话标识找到目标网关入口。三种场景的路由实现细节不同，但结构相似：系统需要维护能够从分片键确定目标实例的路由规则与元数据，并在扩缩容时一致地更新这些信息。它可以是范围表、槽位表或一致性哈希环，并不意味着为每个业务键保存一条独立记录。

无论分片键是用户 ID、模块 ID 还是会话标识，路由过程都包含三个环节：从请求中提取分片键，根据路由元数据确定对象归属，再把请求转发到对应的数据库分片、逻辑服或网关实例（见图 35）。

完成分片键到目标实例的映射后，路由逻辑可以放在应用进程内，也可以集中到代理或协调节点中。两种方式都在维护同一件事，即分片键到目标实例的运行时映射。

下面以存储分片路由为例，说明路由层的两种常见放置方式。第一种是应用内路由：路由逻辑直接放在应用服务器进程中。应用服务器启动时读取分片规则和数据库实例表，建立到各个分片的连接池；处理用户请求时，先根据用户 ID 计算目标分片，再从本地连接池中取出对应数据库连接并发起查询。下面的代码展示的就是这个逻辑：`ShardRouter` 保存分片数量和各分片连接，`getDBConnection` 根据用户 ID 选择连接，`queryUser` 再用这个连接查询用户资料。

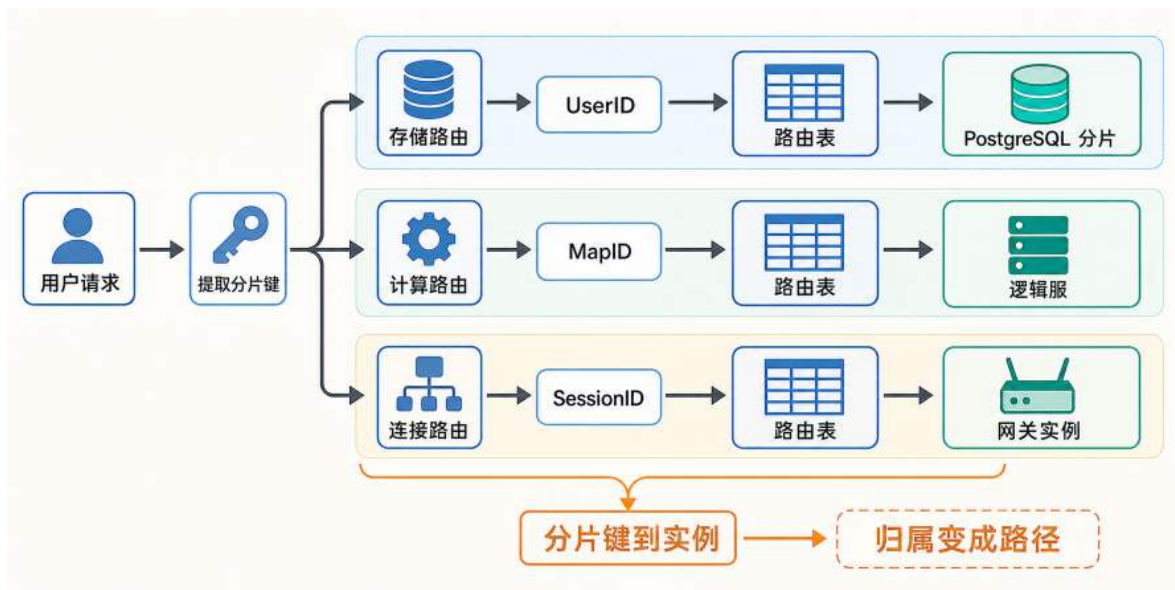


图 35: 分片路由把对象归属转化为请求路径：用户 ID、模块 ID 和会话标识分别经过路由元数据，到达数据库分片、逻辑服或网关实例。

应用层分片路由的实现示例

```

1 // 分片路由器
2 type ShardRouter struct {
3     numShards int
4     dbConnections []*sql.DB // 各分片的数据库连接
5 }
6
7 // 根据用户ID获取目标分片的数据库连接
8 func (r *ShardRouter) getDBConnection(userID int) *sql.DB {
9     shardID := userID % r.numShards
10    return r.dbConnections[shardID]
11 }
12
13 // 查询用户数据
14 func (r *ShardRouter) queryUser(userID int) (*User, error) {
15     // 1. 计算目标分片
16     db := r.getDBConnection(userID)
17
18     // 2. 向目标分片发起查询
19     var user User
20     err := db.QueryRow(
21         "SELECT id, name, power FROM users WHERE id = $1",
22         userID,
23     ).Scan(&user.ID, &user.Name, &user.Power)
24
25     return &user, err
26 }
  
```

应用层路由适合分片规则简单、服务数量不多的场景。应用服务器在本进程内完成分片计算和连接选择，不需要再经过一层代理服务，额外网络开销较少；分片计算本身通常只是一次哈希或取模，CPU 成本也很低。这种方式适合服务数量较少、开发语言统一、分片规则相对稳定的阶段。

它的代价是规则更新会直接影响应用进程。如果新增分片、调整一致性哈希环或切换分片表版本，所有应用服务器都需要及时加载新规则。如果某些进程还停留在旧规则上，同一个用户 ID 就可能被不同服务器路由到不同位置。因此，应用内路由通常还要配合配置中心、版本号和灰度切换机制，保证分片表更新过程可控。

第二种方式是代理中间件路由。当接入数据库的应用进程较多、开发语言不一致或分片规则经常变化时，在每个进程中分别实现路由会增加规则同步和版本管理的成本。代理中间件位于应用服务器与数据库分片之间，把分片映射、连接选择和请求转发集中到统一入口，应用层只需要连接代理。

应用服务器提交 SQL 和参数后，代理从 `player_id` 取得分片键，依据当前分片规则计算唯一的目标分片，再从连接池选择对应连接。若计算结果为分片 2，SQL 只会发送到分片 2，其他分片不参与这次查询；查询结果再经代理返回应用服务器。图 36 将这条请求闭环组织成一条确定路径，说明分片路由先确定数据归属，再沿该归属完成访问，而不是在多个分片之间分配同一条查询。

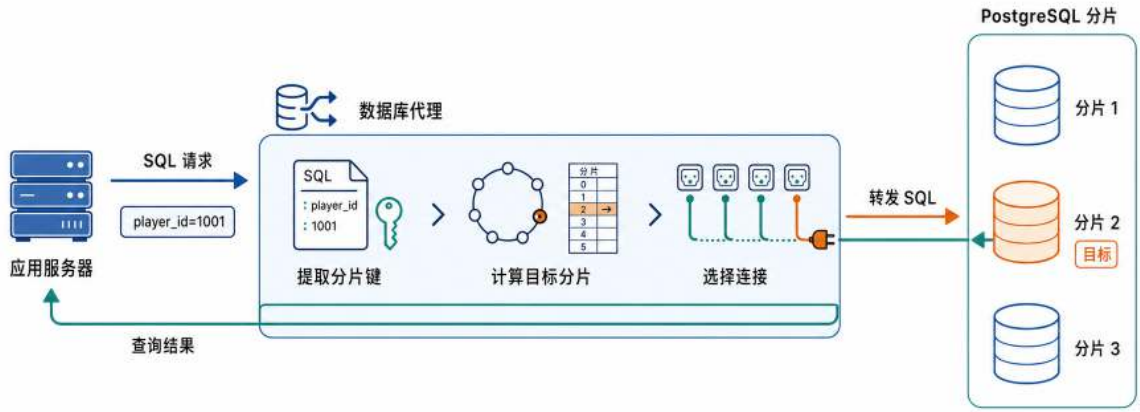


图 36: 数据库代理把分片键计算转化为确定的访问路径：SQL 只进入计算得到的目标 PostgreSQL 分片，查询结果再经代理返回应用服务器。

代理路由由此把应用依赖从多个数据库地址和分片规则收缩为一个稳定入口。分片表更新、节点扩容和连接池管理可以在代理侧集中完成，不必由各应用进程分别实现；在多种语言或多个业务服务共同访问一组分片时，这种集中维护尤其能够减少重复逻辑和规则版本不一致。

它的代价是请求路径多了一跳。每次查询都要先到代理，再由代理转发到目标分片；代理还要解析 SQL、提取分片键、维护连接池，并保证自身高可用。如果代理容量不足，它也可能成为

新的瓶颈。因此，代理中间件适合接入方较多、规则变化频繁、希望集中管理分片规则的系统；应用内路由则更适合路径短、规则稳定、服务规模较小的阶段。

工程中已经有多种成熟的代理中间件、分片网关和协调节点，为应用提供统一的数据库访问入口。它们的具体形态不同，但都把分片元数据和路由判断从业务服务中分离出来，使业务服务不必直接面对一组不断变化的数据库节点。Apache ShardingSphere-Proxy 是透明数据库代理，应用像连接普通数据库一样连接代理，由代理处理分片规则、SQL 路由和结果归并。²⁰ Vitess 面向 MySQL 分片场景，应用连接 VTGate，VTGate 根据 VSchema、表结构和查询条件把请求路由到正确分片。²¹ Citus 面向 PostgreSQL 分布式表，应用连接 coordinator，coordinator 根据分布式表元数据把查询路由到单个 worker 或并行发送到多个 worker。²² 这些系统的差异表明：分片路由不一定只能写在业务代码里，也不一定只是一个简单代理。它可以是透明数据库代理，可以是分片网关，也可以是分布式数据库自己的协调节点。选择哪一种形态，取决于系统希望把分片规则、连接池、SQL 路由和扩容变更交给谁来维护。

前面的数据库代理是分片路由在存储访问中的一种实现。回到整个在线系统，计算层、存储层和路由层承担不同职责：计算层承接玩家连接、会话和业务计算，存储层保存持久化数据，路由层则维护分片键到目标实例的映射，把对象归属转化为实际请求路径。路由层在逻辑上连接计算与存储，使两层无需维持固定的连接关系。

当游戏服务器需要读写持久化数据时，它将玩家 ID 和请求交给分片路由层，由路由层依据当前映射选择数据库分片。同一台游戏服务器因而可以服务数据分布在不同分片上的玩家，同一个数据库分片也可以接收来自多台游戏服务器的请求。两层的访问关系由路由映射建立，不再依赖部署时的一一绑定。

图 37 用两条访问路径说明：游戏服务器发出的请求先经过分片路由层，再到达玩家数据所属的数据库分片。路径 A 和路径 B 从不同的游戏服务器出发，经过共享的分片路由层后，分别进入玩家 ID 所属的数据库分片。请求来自哪台游戏服务器只表示计算入口，玩家 ID 才决定数据的存储位置；两条路径共同说明，计算节点与数据库分片之间不需要保持一一对应。

这种多对多访问关系使两层可以按各自承担的压力独立伸缩：在线用户数量增加时扩展游戏服务器，数据总量或读写压力增加时扩展数据库分片。不过，分片路由只回答请求按照对象归属应当去哪里，不会说明目标实例当前有多忙。系统要识别热点并决定新请求如何分配，还需要持续获得各实例的运行状态。

4.2.4 负载监测与入口调度

分片映射只保证对象归属正确，不保证每个实例上的对象活跃度和运行压力始终相近。对象数量相近的两个分片，可能因为业务活跃度不同而产生完全不同的运行压力。例如，两个计算分片负责的对象数量相同，其中一个分片承载高频交互的计算任务，另一个分片上的对象大多处于

²⁰ Apache ShardingSphere-Proxy 官方文档：<https://shardingsphere.apache.org/document/current/en/quick-start/shardingsphere-proxy-quick-start/>。

²¹ Vitess VTGate 路由说明：<https://vitess.io/docs/faq/advanced-configuration/vtgate/how-does-vtgate-know-which-shard-to-route-a-query/>。

²² Citus 概念文档：https://docs.citusdata.com/en/stable/get_started/concepts.html。

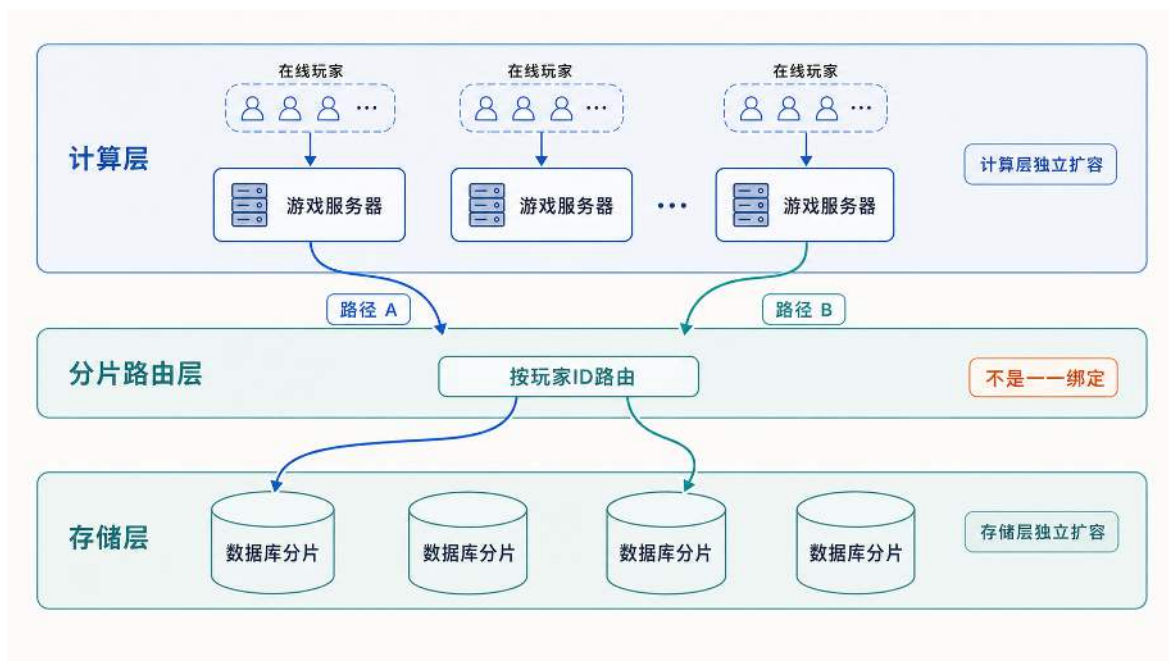


图 37: 计算层与存储层解耦：游戏服务器承接玩家连接，分片路由按玩家 ID 选择数据库分片，使两层形成可独立扩展的多对多访问关系。

空闲等待状态；前者的 CPU 使用率、消息队列长度和响应时间会显著高于后者。存储分片也可能因为少数活跃用户集中而承受更高的查询和写入压力。因此，入口调度和后续动态调整不能只依赖分片映射，还需要通过负载监测持续识别哪些分片过载、哪些分片仍有余量。

负载监测把分片实例的运行状态整理成可比较的数据。最基本的监测是存活状态，即节点是否可达、进程是否正常；进一步可以统计对象规模，例如连接数、在线用户数、模块数或缓存键数量；更细致的压力指标包括 CPU 使用率、内存占用、网络带宽、请求速率、队列长度和响应时间。对象数量描述分片承载了多少任务，压力指标则反映这些任务实际消耗了多少资源。所以监测服务需要同时记录对象规模和运行压力，不能只依赖某一类指标。

状态采集通常采用推拉结合的方式。分片实例通过推送（Push）定期上报心跳和关键指标，监测服务据此维护状态基线；监测服务也可以通过拉取（Pull）周期性探测实例，或在指标异常时发起补充检查。心跳负责持续更新，主动探测用于缩短上报间隔形成的观测盲区。一条通用的分片状态消息可以同时携带对象规模、资源压力和服务质量指标：

分片实例心跳上报的关键数据

```
1 type ShardStatus struct {  
2     ShardID      string  
3     ActiveObjects int    // 连接、用户、模块或键的数量  
4     CPUUsage      float64 // CPU使用率 (0-100)  
5     MemUsage      float64 // 内存使用率 (0-100)  
6     RequestsPerSec float64 // 每秒请求数  
7     P99Latency    int     // P99响应时间 (毫秒)  
8 }  
9 // 每隔固定间隔发送一次心跳到监测服务
```

图 38 把负载监测的维度与感知方式放在同一框架中比较。左侧说明监测信息如何从简单的存活检测逐步丰富到多维资源压力，信息越丰富、调度越准，采集成本也越高；右侧说明服务器主动上报与监测方主动查询如何互补，工程上通常采用推拉结合的方式获取最新状态。

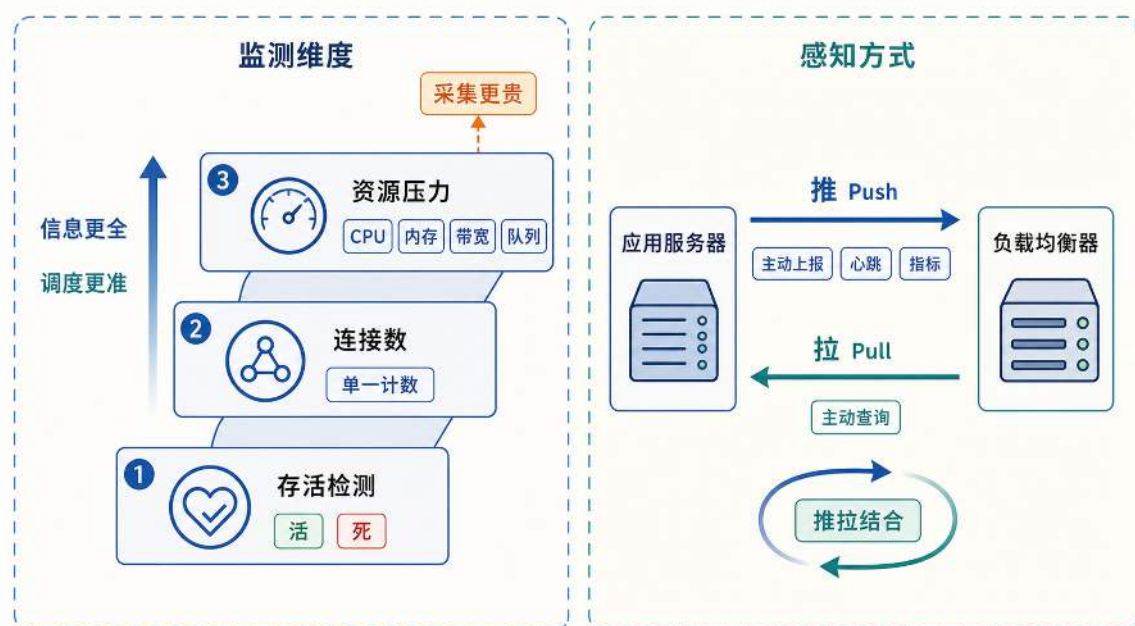


图 38: 负载监测的维度从存活检测到资源压力逐步丰富，感知方式通过推拉结合获取：服务器主动上报心跳和指标，均衡器在需要时主动查询最新状态。

监测服务为每个分片维护最近一次状态快照，并把不同量纲的指标归一化后计算综合负载值。连接分片可以提高连接数和网络带宽的权重，计算分片可以提高 CPU 使用率和队列长度的权重，存储分片则可以提高请求速率和尾延迟的权重。综合负载值用于识别“对象多但压力低”和“对象少但压力高”两类状态，并为过载告警、扩容判断和迁移决策提供输入。

监测结果首先可以用于没有既有会话归属的首次接入。用户第一次进入系统时，尚未与任何应用服务器形成会话归属。负载均衡器可以在健康的应用服务器副本中比较在线人数、CPU 使用率等状态，再选择一个能够承接连接的节点；当这些副本无状态、配置相同且负载接近时，简

单轮询或随机选择也可以成立。

用户接入后，应用服务器可能在内存中保留会话状态。网络连接中断并不意味着这份状态立即消失；客户端重新连接时会携带会话标识，入口层可以据此查找会话归属。如果服务器 C 仍然可用并保存着该会话，重连请求就应优先返回 C，避免在其他服务器上丢失或重新加载状态。这种让后续连接尽量回到原服务器的机制称为**会话粘性**（Session Stickiness）。

上述两个接入场景的节点选择依据并不相同，图 39 将其并列呈现。首次接入依据状态表在可用副本中选择服务器 B；断线重连则依据会话标识找到归属服务器 C，并在 C 仍可用时返回该节点。

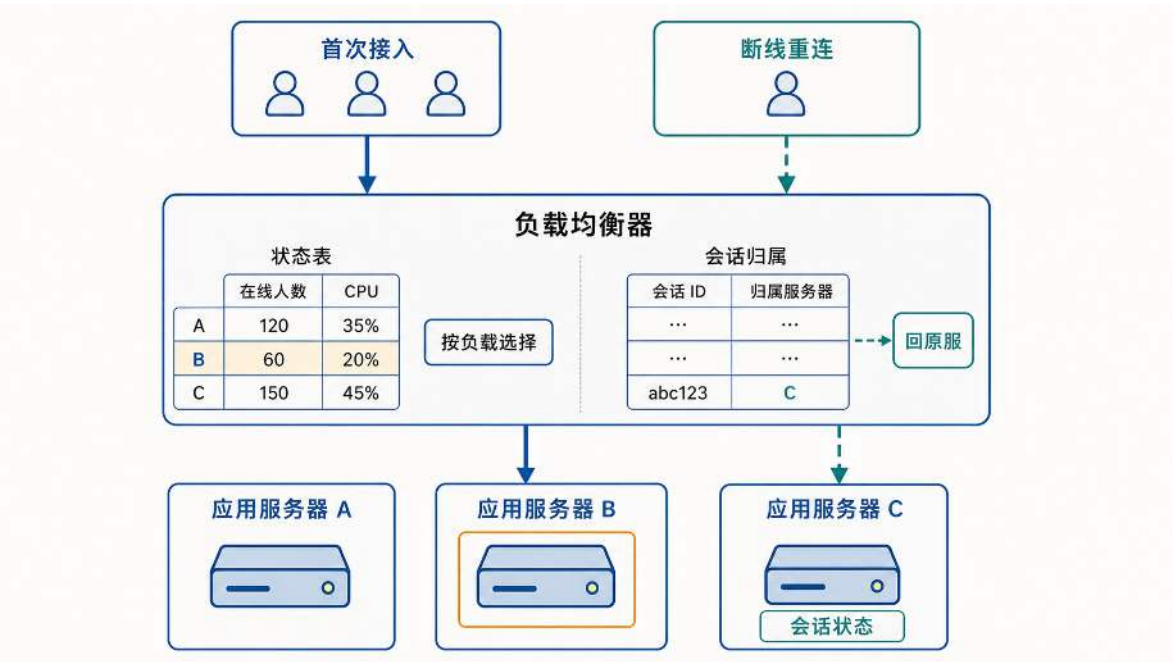


图 39: 同一入口下的两种接入场景：首次接入依据负载状态选择服务器 B，断线重连依据会话归属返回仍保存会话的服务器 C。

会话粘性成立的前提是原服务器仍然可用且会话尚未失效。如果原服务器不可达，入口层需要选择新的承接节点，并从外部状态恢复会话或重新建立会话，不能继续依赖原有归属。

Nginx 是入口负载均衡器的一种实现。对于没有会话归属约束的首次接入，它可以使用加权轮询或最少连接等策略选择上游实例；当业务会话已经具有归属时，入口还需要获得并遵循相应的会话映射。²³

入口调度是在可承接请求的计算节点之间进行选择；存储层仍然按照分片键维持数据归属，不会因为其他数据库分片更空闲而临时改变请求去向。

负载监测本身不会改变对象归属。即使监测服务发现分片 A 过载、分片 B 空闲，路由层仍必须把属于分片 A 的请求送到分片 A；在状态尚未迁移之前，直接改发请求会造成数据或运行状态缺失。因此，发现负载失衡后，系统还必须执行动态调整：整体容量不足时增加实例，容量

²³Nginx Stream 上游模块支持加权轮询、最少连接和哈希等连接分配方式：https://nginx.org/en/docs/stream/nginx_stream_upstream_module.html。

长期过剩时移出实例；总容量足够但对象分布不均时，则重新划分对象归属。无论采用哪种调整，只要分片集合或映射关系发生变化，就需要把受影响的对象状态迁移到新的归属节点。

4.2.5 动态调整：扩缩容与热迁移

动态调整可能由扩缩容、持续的负载不均或实例故障触发，并依次解决两个问题：首先确定哪些对象需要改变归属，其次将这些对象的运行状态转移到新的实例。当调整涉及实例增减时，**一致性哈希**用于限制映射变化的范围，减少需要迁移的对象；确定迁移对象后，**热迁移**负责实际转移状态，并控制迁移过程中的服务中断。前者在上文映射策略中已经给出算法，本节聚焦后者。

热迁移要解决的不只是状态传输，还包括处理权和请求路径的交接。设计算分片 A 因突发流量持续过载，分片 B 仍有余量，路由表当前把会话 u 指向 A。A 的内存中保存着 u 的位置、业务状态、任务进度、计时器、最后处理的请求序号和尚未完成的操作；与此同时，客户端仍在发送会改变这些状态的新请求。如果入口立即把请求改发到 B，B 没有这些状态，无法从 A 停止的位置继续处理；如果 A 在继续处理请求的同时只向 B 发送一次快照，传输完成时这份快照又可能已经落后。因此，迁移必须协调状态复制、处理权转移和路由切换，不能把它们拆成互不相关的操作。

这个例子中的迁移单元是应用层会话，而不是一条正在传输字节的 TCP 连接。系统需要转移的是继续处理会话所需的业务状态、请求进度和连接上下文。如果客户端通过稳定入口访问，入口可以在切换后把后续请求转发给 B；如果客户端直接连接业务服务器，则仍需要与 B 重新建立连接。同样的机制也适用于视频会议的房间状态、实时协作的文档会话，以及其他需要在实例之间改变处理者的有状态对象。

因此，一次正确的迁移至少要满足三个条件。第一，B 接管时必须拥有截至切换点已经生效的状态，以及继续处理未完成请求所需的信息；第二，同一时刻只能有一个实例拥有该会话的写入权，避免 A 和 B 同时修改状态；第三，只有 B 完成状态装载并确认可以服务后，系统才能切换对象归属和请求路由，A 也只能在接管确认后清理旧状态。

围绕这些条件，迁移通常包含一组共同动作：控制器选择迁移对象和目标实例，目标实例完成必要的资源准备；源实例复制状态，并根据方案停止状态修改或记录复制期间的增量；目标实例装载并校验状态；控制器把唯一写入权和请求路由切换到目标实例；目标实例恢复服务，源实例在收到接管确认后清理旧状态。不同方案都要完成这些动作，区别在于何时停止状态修改，以及哪些数据必须在停止服务期间传输。

如果源实例从复制开始就停止修改，状态容易保持一致，但停服窗口会覆盖整个传输过程；如果源实例在复制期间继续运行，系统就必须追踪并补齐其间发生的变化。下面先以停止复制法建立对照基线，再讨论预复制和按优先级分波迁移如何缩短停服窗口。

方案一：停止复制法 (Stop-and-Copy) 最直接的做法是“先停、再搬、再启”，即停止复制法，如图 40 所示。迁移开始时，源服务器立即停止接收该用户的新请求，并暂停会修改会话的计时器和后台任务，将用户的完整会话状态序列化为一个可传输的数据结构，通过网络传输到目标服务器，目标服务器反序列化后恢复用户会话，再切换会话归属和请求路由，由目标服务

器继续提供服务；客户端直接连接业务服务器时，还需要重新连接到目标服务器。这种方案的优点是实现极为简单：在所有状态修改入口都已暂停的前提下，停机期间状态不再变化，序列化的快照可以保持一致，目标服务器也不必追赶复制期间的新变化；系统仍需按顺序完成目标确认、归属切换和路由切换。缺点也同样明显：用户在整个迁移过程中处于完全停服状态，停机时长与需要迁移的状态数据量成正比。对于一个状态复杂的用户（如实时协作或高频交互场景中的大量并发状态），全量序列化和传输可能耗时数秒甚至更长，用户会明显感知到连接中断或请求超时。因此，停止复制法通常只适用于离线维护窗口或用户处于彻底空闲状态的场景，不适合作为生产环境中的主动负载均衡手段。

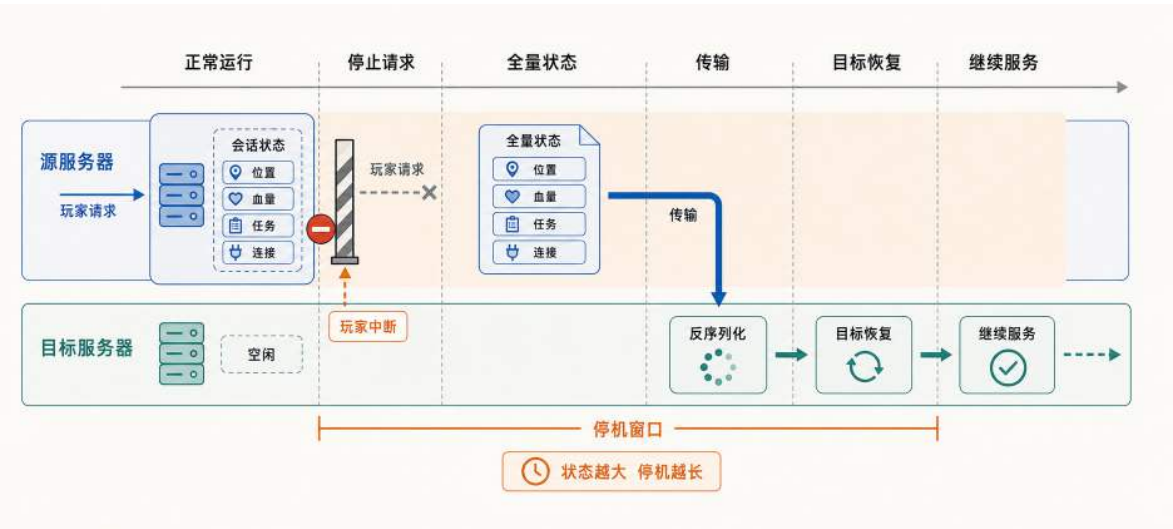


图 40: 停止复制法：源服务器暂停服务后传输全量状态，停机时间随状态体量增长。

停机窗口覆盖了整个状态序列化与传输过程。状态越大、网络越慢，用户等待的时间就越长。这自然引出一个优化方向：能不能把大部分传输提前到停机之前完成？

方案二：预复制法（Pre-Copy） 为了缩短用户感知到的停机时间，可以把大部分数据的传输提前到停机之前完成，这就是预复制法，如图 41 所示。其核心思路是：在源服务器仍然运行的同时，先把一份状态快照传输到目标服务器；每一轮传输结束后，再追踪本轮期间发生变化的状态块，并在下一轮只传输这些增量，直到每轮增量收敛到足够小；最后才短暂停止状态修改，将最后一批增量传输过去，完成最终切换。如果迁移单元是虚拟机或完整进程，变化追踪可以发生在内存页层，修改过的页通常称为脏页（Dirty Pages）；如果迁移单元是应用层会话，更常见的做法是把会话编码为带版本号的逻辑状态块，或者用变更日志记录快照之后的增量。两种实现的共同前提是：状态变化速度低于增量传输速度。绝大多数用户数据（已完成的任务记录、不在使用的配置项、离线关系列表等）在单次传输间隔内根本不会被修改，只有当前正在活跃变化的那部分状态（如实时位置、状态值）才会产生新的增量。随着迭代轮次增加，每轮需要重传的数据逐渐减少，最终只需在较短的停机窗口内传输最后一批增量就能完成切换；实际窗口取决于剩余增量、网络带宽、序列化成本和切换协议，没有固定的毫秒数保证。预复制法的代价是总传输量会超过全量状态大小。部分状态块在多轮迭代中被反复传输，网络带宽的消耗高于停止复制法；

对于那些持续高频修改的“热状态”，无论迭代多少轮都无法在停机前完全收敛，最终仍然要在停机阶段传输，这构成了预复制法停机时间的理论下界。

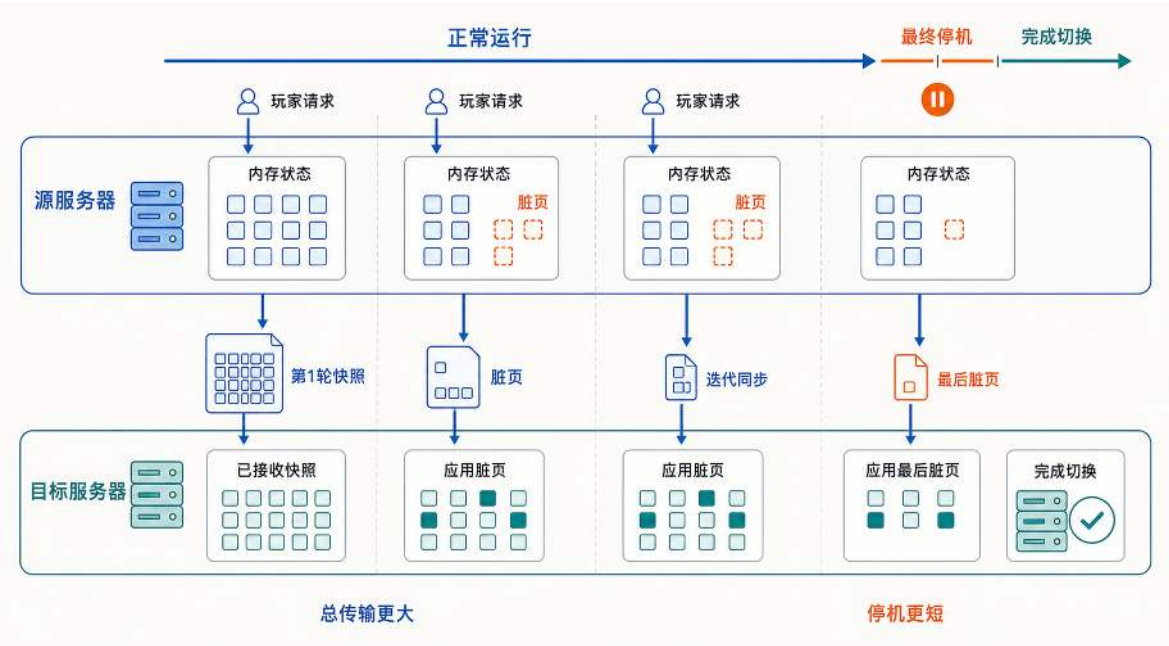


图 41: 预复制法：运行中迭代传输变化的状态块，以总传输量增加为代价，把最终停机窗口压缩到最后一轮。虚拟机可以按脏页追踪变化，应用会话通常按逻辑状态块或增量日志追踪变化。

预复制法把大部分数据传输挪到了源服务器仍在运行的阶段，停机窗口只剩最后未收敛的增量传输。代价是传输期间会持续消耗网络带宽，且状态变化速率必须低于传输速率，否则停机窗口无法缩小。

方案三：按优先级分波迁移（Wave-Based Migration） 预复制法是虚拟机热迁移中的典型思路，停止复制法则提供了便于比较停服时间的基线。在虚拟机场景中，迁移单元是整个虚拟机的内存状态（包括全部内存页和设备寄存器状态）。在线服务的状态迁移有所不同：应用能够识别哪些状态决定后续请求的正确性，哪些只是可重新加载的缓存、派生视图或背景数据。切换服务器时，所有会影响业务判断和后续写入的权威状态都必须进入第一波；只有不参与当前决策、允许重新获取或短暂陈旧的数据，才能延后同步。基于这种分类，可以设计按优先级分波迁移法，如图 42 所示。

分波迁移包含五个连续阶段。第一步是预热：目标服务器提前加载静态资源，把可预先完成的初始化移出切换路径。第二步是交接准备：源服务器停止接收该会话的新写入，排空正在执行的请求，并为本次迁移生成单调递增的版本号或 epoch。第三步是关键状态传输：源服务器发送第一波权威状态，目标服务器装入并校验这些状态后只返回“准备完成”，暂不对外提供服务。第四步是所有权切换：路由表或租约服务以本次 epoch 把唯一写入权转交给目标服务器，旧 epoch 的请求随后被拒绝；只有这一步成功后，目标服务器才能接收新请求。第五步是后台补齐：可重新获取的缓存、派生视图和背景数据继续分批同步。任何会影响业务判定的状态都不能留到这一

阶段。其核心实现逻辑如下：

按优先级分波迁移的核心逻辑

```
1 // 阶段1: 提前预热目标服务器 (后台静默执行)
2 targetServer.PreloadStaticAssets(session.ScopeID)
3
4 // 源端进入交接状态: 停止新写入并排空正在执行的请求
5 epoch := sourceServer.BeginHandoff(session.ID)
6 sourceServer.DrainInFlight(session.ID)
7
8 // 第一波: 所有影响后续业务判断的权威状态
9 wave1 := SessionCriticalState{
10     Position:      session.Position,
11     StateSnapshot: session.StateSnapshot, // 保证状态连续
12     CoreValue:     session.CoreValue,
13     LastCommandSeq: session.LastCommandSeq,
14     PendingActions: session.PendingActions,
15     TimerState:    session.TimerState,
16 }
17 targetServer.Prepare(epoch, wave1) // 此时尚未接收用户请求
18
19 // 路由或租约服务原子转移唯一写入权; 旧 epoch 的请求会被拒绝
20 ownership.Transfer(session.ID, sourceServer, targetServer, epoch)
21 targetServer.Activate(epoch)
22
23 // 后续波次只能包含可重建、可短暂陈旧的非权威状态
24 wave2 := collectDerivedViews(session)
25 targetServer.ApplyWave(2, wave2)
26
27 wave3 := collectReloadableCaches(session)
28 targetServer.ApplyWave(3, wave3)
29
30 // 目标端确认接管后, 源端清理旧 epoch 的状态
31 sourceServer.CleanupSession(session.ID, epoch)
```

分波迁移以会话为单位，转移的是继续处理该会话所需的应用状态及其唯一写入权。它与同一进程中的线程切换有一个共同点：都要保留后续执行所需的状态；区别在于，线程切换时应用状态仍留在原机器的内存中，而跨服务器迁移还要通过网络复制状态，并在两台服务器之间转移唯一写入权。

停止新写入并排空在途请求后，源服务器生成一份不再变化的权威状态快照，并用本次 epoch

标识它的版本。除了位置和业务状态，这份状态还要包含最后处理的请求序号、尚未完成的操作、定时器以及防止重复执行所需的信息，使目标服务器能够从源服务器停止的位置继续处理。目标服务器通过 `Prepare(epoch, wave1)` 装入并校验这些状态，但此时写入权仍属于源服务器，因此不能接收新请求。

关键状态传输完成并不等于服务已经恢复。路由表或租约服务成功转移唯一写入权后，目标服务器执行 `Activate(epoch)`，入口层才能把该会话的后续请求转发到目标服务器。客户端经过稳定入口访问时，通常只会感受到短暂停顿或请求重试；如果客户端直接连接业务服务器，则需要重新建立连接。

静态资源可以在交接前预热，缓存和派生视图也可以在资源允许时提前预复制，但它们不能成为目标服务器接管写入的依据。源服务器仍在运行时，这些非权威状态可能继续变化，提前传输的副本因而可能过期；所有权切换后，目标服务器还要检查版本，并刷新或重新生成这些状态。分波迁移先恢复正确处理请求所需的最小状态，再逐步恢复完整运行状态，从而避免缓存和背景数据延长停写时间。图 42 按这一顺序标出了关键状态传输、所有权切换、恢复服务和后台补齐之间的关系，并补出了目标服务器确认接管后，源服务器才清理旧状态这一收尾动作。

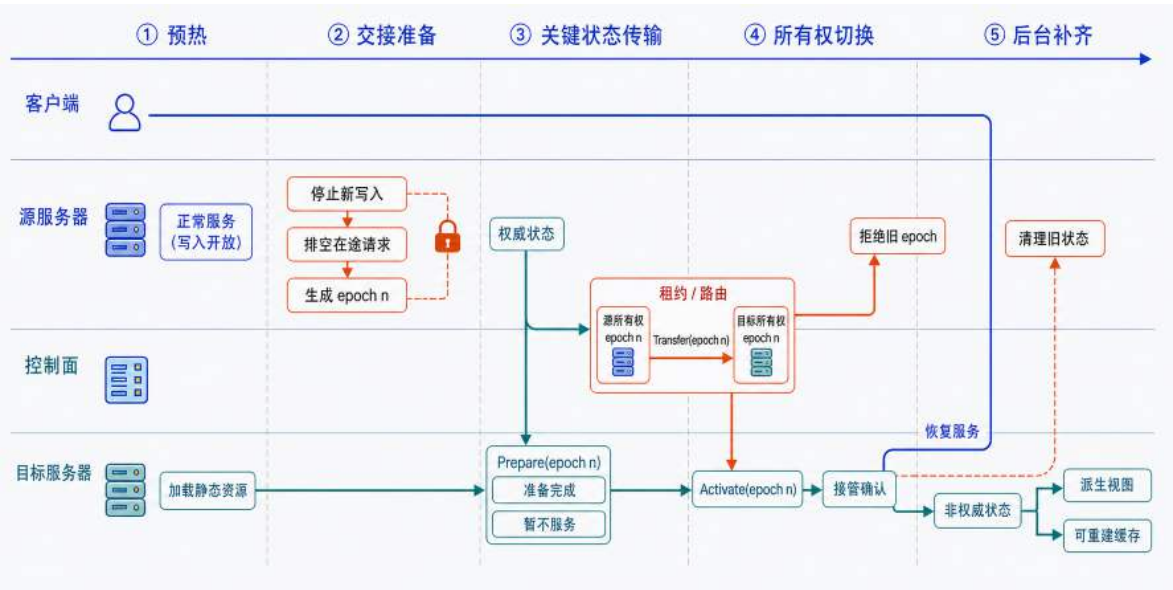


图 42: 分波迁移先传输权威状态并完成唯一写入权交接；目标服务器确认接管后，源服务器清理旧状态，非权威状态则在后台继续补齐。

三种方案的本质区别在于如何权衡停机时间、传输量和实现复杂度。停止复制法以最简单的实现换来最长的停机时间，适合维护窗口或离线场景。预复制法通过迭代传输变化状态压缩停机时间，但总传输量更大，适合通用服务器迁移场景，VMware 的 vMotion 和 Xen 的虚拟机热迁移均采用此思路。按优先级分波迁移法利用状态本身有重要性层次这一特点，把权威状态交接与非权威状态补齐分开，代价是开发者必须完成状态分类、版本控制、写入冻结和路由切换，实现复杂度最高，但在这些条件成立时可以进一步缩短用户感知的中断。

这三种方案比较的是有状态对象迁移中的通用取舍：停止复制提供最简单的正确性基线，预复制通过追踪增量减少停服时间，分波迁移则进一步利用应用能够区分权威状态与非权威状态这

一条件。这些思路不只适用于用户会话，也可以用于视频会议房间、实时协作文档以及其他有状态服务；其中分波迁移只有在非权威状态允许延后恢复时才成立。

运行时迁移并非所有应用服务器都必须具备的能力。生产环境中更常见的做法是以房间或游戏实例为单位等待任务自然结束后再摘流，或者在客户端重连时根据会话令牌从外部存储恢复状态。只有当系统不能等待对象自然结束，又需要尽快释放过载实例时，主动迁移的额外复杂度才可能值得承担。

决定执行迁移后，调度器还要选择合适的对象。高频交互中的会话持续修改位置、状态值和计时器，迁移期间需要追踪的变化多，难以迅速收敛；正在执行交易、资产变更等一致性敏感操作的会话，还需要额外协调事务状态，通常不应作为优先迁移对象。更合适的对象是当前状态变化较少的会话，例如长时间未操作的空闲用户，或者刚结束主要任务、暂时只在查看结果的用户。这些对象需要复制的增量较少，迁移更容易在较短的停机窗口内完成。

无论采用哪种迁移方案，工程实践中都需要配合约束条件，防止迁移本身成为新的负担。

第一项约束是迁移冷却期。迁移过程本身会消耗源服务器和目标服务器的 CPU、内存和网络带宽：源服务器要序列化状态并发送，目标服务器要接收并反序列化恢复。如果允许同一台服务器在短时间内反复触发迁移，迁移操作占用的资源可能反而加重服务器负担，甚至出现“越迁越忙”的恶性循环。因此，同一台服务器在完成一次迁移后，通常需要等待一段冷却时间才能再次触发，让服务器有时间消化迁移带来的瞬时开销并稳定负载观测值。

第二项约束是单次迁移的规模上限。如果一次迁移把大量对象同时送到同一台目标服务器，目标服务器需要在短时间内完成大量会话恢复、缓存加载和连接建立。这种瞬时涌入可能导致目标服务器本身进入高负载状态，把过载从一台服务器搬到了另一台。工程上通常限制单次迁移的对象数量，并将迁移分批执行：每批迁移完成后观察目标服务器的负载变化，确认稳定后再执行下一批。

从更高的角度来看，运行时迁移把分片再均衡从“修改映射”推进到“转移状态”：系统不仅要计算新的对象归属，还要在不中断体验的前提下完成状态交接。第 6.3.2 节讨论有状态服务伸缩时会继续回到这个约束：扩容和缩容都不是把数字改大或改小就结束，还要考虑状态、连接和迁移成本。

4.2.6 存储分片：从单库瓶颈到跨分片访问

前面的分片键、映射、路由、负载监测和动态调整，描述了对对象分片的通用控制链。应用到存储层时，被划分的对象变成用户数据，承载实例变成数据库节点；数据持久化、全局查询和事务提交也随之进入分片需要讨论的问题。

在单库架构中，所有用户数据和读写请求集中在一个数据库实例上，容量上限和故障影响也集中在同一个位置。存储分片可以分散数据容量、请求压力和故障范围，却会把原本由单一数据库完成的查询与事务拆到多个节点。因此，存储分片需要回答两个连续的问题：单一数据库为什么必须拆分，以及拆分之后如何处理跨分片查询和跨分片更新。

单一数据库的瓶颈与风险 第三章中，单机服务器使用 Redis 和 PostgreSQL 做分层存储。Redis 承载在线状态、实时计数、排行榜等热数据，PostgreSQL 保存账号信息、业务记录、配置等冷数据。在单机多人原型中，应用服务器只需要连接一个 Redis 实例和一个 PostgreSQL 实例，就能完成任意用户的缓存读取、状态更新和持久化写入。

当用户规模继续扩大，数据层的压力会集中到这两个实例上。PostgreSQL 负责保存关系型数据，并通过事务、索引、锁和写前日志保证数据可靠落库。一次查询进入 PostgreSQL 后，通常要占用一个数据库连接，经过 SQL 解析、执行计划、索引查找和数据页读取；一次写入还要修改数据页，并把变更记录写入日志。查询和写入持续增加时，连接数、磁盘 I/O、锁等待和日志写入都会限制整体吞吐。Redis 的职责不同，它主要把热数据保存在内存中，用内存访问换取更低延迟。但内存容量同样有限，在线状态、实时位置和排行榜片段持续增长后，缓存淘汰会变多；缓存未命中的请求又会回到 PostgreSQL，进一步加重数据库压力。纵向扩容可以提高单机配置，却不能改变所有用户数据仍集中到同一组存储实例上的事实。

单一数据库的瓶颈首先来自请求排队。在单机架构中，登录、查询和写入等请求最终都会访问同一个 PostgreSQL 实例。每个请求进入数据库后，都要占用连接和执行资源；查询需要查找索引、读取数据页，写入需要修改数据页、记录写前日志，并在必要时等待锁释放。当并发请求超过数据库能够稳定处理的速度时，新请求不会立刻完成，而是在连接池、数据库执行队列、磁盘 I/O 队列或锁等待中排队。排队时间继续累积后，用户看到的就是登录变慢、查询响应延迟、写入请求超时，甚至新连接被拒绝。

提高单机硬件配置可以推迟瓶颈出现，但不能消除单实例结构本身的限制。更多 CPU 核心可以并行处理更多查询，更多内存可以缓存更多数据页，更快的磁盘可以提高日志写入和数据读取速度。但数据库并不是所有环节都能随硬件线性增长：事务提交仍可能等待日志刷盘，热点记录仍可能发生锁竞争，索引更新仍要维护内部结构，网络接口和内存带宽也有上限。因此，单机配置越高，成本通常增长得更快，而吞吐提升会逐渐变慢。

单一数据库还会把故障影响集中到一个位置。所有用户的核心数据都在同一个实例上时，磁盘故障、数据库进程崩溃、操作系统故障或网络中断，都会同时影响登录、存档、结算和排行榜。备份可以降低永久丢失数据的风险，但备份不能直接提供实时服务。从备份恢复到可用状态需要时间，这段时间内用户仍然无法正常读取和写入数据；如果备份间隔较长，还可能丢失最近一段时间内已经发生的更新。因此，单一数据库同时暴露出两个问题：单个实例的容量有限，实例故障又会影响全部用户。这两个问题需要采用不同机制处理。容量上限要求系统把数据和请求分散到多个数据库实例；故障后继续提供服务则需要副本和故障切换机制。本节先解决容量问题：系统按照分片键把数据集合划分给多个数据库实例，并让路由层把请求送到保存目标数据的实例，这种横向拆分称为**数据库分片**。

图 43 使用同一组玩家比较拆分前后的数据归属和请求流向。在左侧的单库结构中，玩家组 A、B、C、D 共享同一个请求队列和数据库，所有请求都会竞争同一组连接、执行和存储资源。在右侧的分片结构中，路由层按照玩家 ID 将四组请求分别送入对应分片，每个数据库实例只保存并处理其中一部分玩家的数据。以玩家组 C 为例，它的请求进入分片 3；分片 3 发生故障时，玩家组 C 暂时无法访问数据，但其他三个分片仍可继续处理各自玩家组的请求。

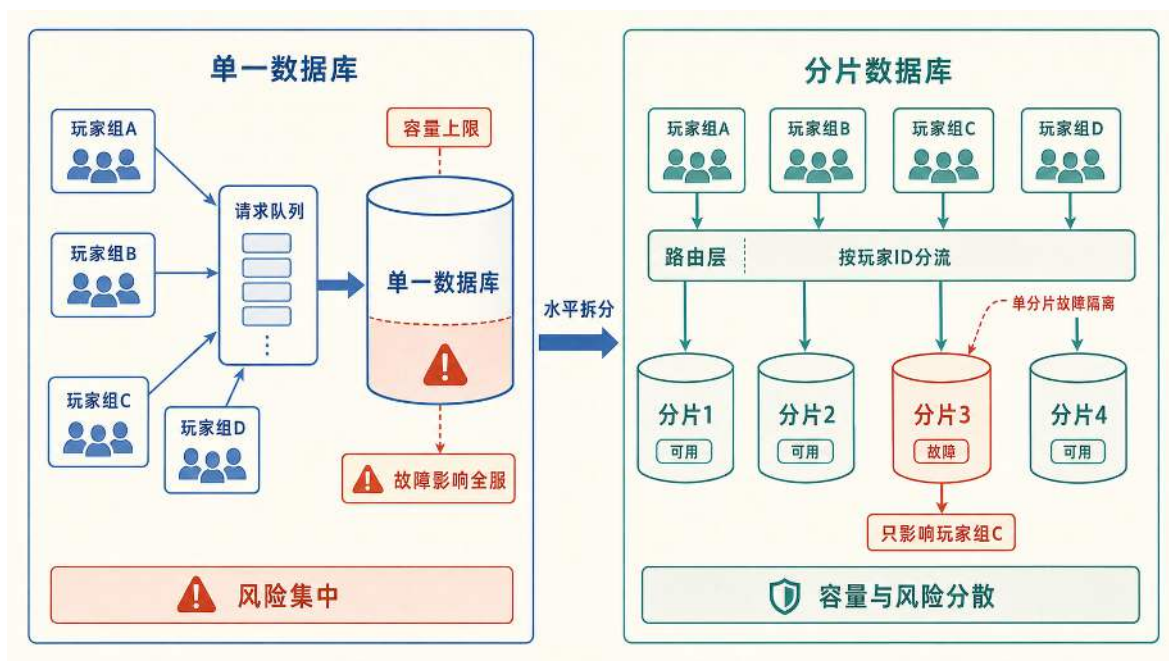


图 43: 水平拆分按玩家 ID 分散请求和数据容量，并把单个数据库故障的影响限制在对应玩家组；分片缩小了故障范围，但不能保证故障分片继续提供服务。

对于单个玩家的局部访问，分片使多个数据库能够并行承接请求，并把单个实例的故障影响限制在其负责的玩家范围内。但分片并不等于高可用：分片 3 故障后，玩家组 C 仍然无法读写数据；要让该分片继续提供服务，还需要在分片内部配置副本和故障切换机制。

上述容量扩展和故障范围缩小都依赖稳定的数据归属。同一玩家的请求持续进入同一分片，资料、配置和状态等局部访问才能在一个数据库内部完成；与此同时，需要覆盖全部玩家或同时修改多个玩家数据的操作，也会因为数据已经分散而跨越多个分片。下面分别讨论跨分片查询和跨分片更新带来的问题。

跨分片查询 当查询条件包含分片键且只涉及一个用户时，路由层可以直接定位目标分片，查询仍由单个数据库完成。例如，用户 1001 的资料、配置和状态位于分片 1，应用服务器只需访问该分片。

当查询范围超过一个分片时，单个数据库只能返回局部结果。比如，全服排行榜需要比较所有用户的战力分数，而每个分片只保存其中一部分用户；查询任意一个分片，都无法得到完整排名。因此需要读取多个分片来汇总结果，这类操作称为**跨分片查询**。

以全服排行榜为例，在单一数据库中，全部用户记录位于同一个查询范围内，可以通过一条排序语句得到全局前一百名：

单一数据库中的排行榜查询

```
1 SELECT id, name, score
2 FROM users
3 ORDER BY score DESC
4 LIMIT 100;
5 -- 在单个数据库内完成全局排序并返回前100名
```

数据库分片后，完整用户集合分散在多个数据库实例中。在任意一个分片上执行上述 SQL，都只能得到该分片的局部 Top100，不能直接代表全服排名。为了得到全局 Top100，系统需要在所有分片上并行执行这条查询，收集各分片的局部 Top100，再合并候选记录、重新排序并截取前一百名。汇总时不必读取各分片的全部用户记录，每个分片只需返回自己的局部 Top100。下面的伪代码给出了并行查询和集中汇总的核心过程：

跨分片排行榜的并行汇总伪代码

```
1 func getGlobalTop100(shards []Shard) ([]User, error) {
2     // 1. 并行查询每个分片的局部Top100
3     localRankings, err := parallelQuery(
4         shards,
5         queryLocalTop100,
6     )
7     if err != nil {
8         return nil, err
9     }
10
11     // 2. 合并候选记录并重新排序
12     candidates := merge(localRankings)
13     sortByScoreDescending(candidates)
14
15     // 3. 截取全服Top100
16     return firstN(candidates, 100), nil
17 }
```

并行查询避免了逐个等待所有分片，但整体延迟仍由最慢的分片决定。任何一个分片超时或失败，都可能使本次排行榜缺少一部分候选记录；应用层还需要处理多次网络请求、各分片的局部排序以及候选结果的二次合并。因此，跨分片查询的主要代价不是 SQL 语句变长，而是一次数据库内部查询变成了跨多个节点的并行查询、等待和汇总过程。

实时跨分片查询的问题在于，每个用户查看排行榜时，系统都要重新向所有分片发起查询并合并结果，而不同用户读取的其实是同一份全服排名。如果每次请求都重复执行相同的分片查询和排序，数据库、网络和应用服务器会反复承担同一项计算。

排行榜通常允许短时间延迟，不要求每次查询都包含刚刚结束的最后一批操作。因此，可以

把全局合并从用户的查询路径移到后台更新路径，这种做法称为异步汇总。排行榜汇总服务定期拉取各分片的局部 Top100，或者持续消费用户分数变化事件，在后台计算全服 Top100，再把结果写入 Redis 缓存。这份提前计算并保存的全局结果可以看作排行榜的物化视图（Materialized View），把查询结果预先计算并存储，之后直接读取，减少重复计算与访问。

Redis 在这里不保存用户分数的权威记录，也不负责实时执行跨分片查询。它保存的是汇总服务已经计算完成的全服榜单：用户 ID 作为成员，分数作为排序值写入有序集合；用户查看排行榜时，只需从 Redis 读取分数最高的一百名。图 44 中的 Redis 榜单位于两条路径的交点，但职责边界并未改变：后台更新路径写入物化结果，用户查询路径只读取全服 Top100，不再实时访问 PostgreSQL 分片。

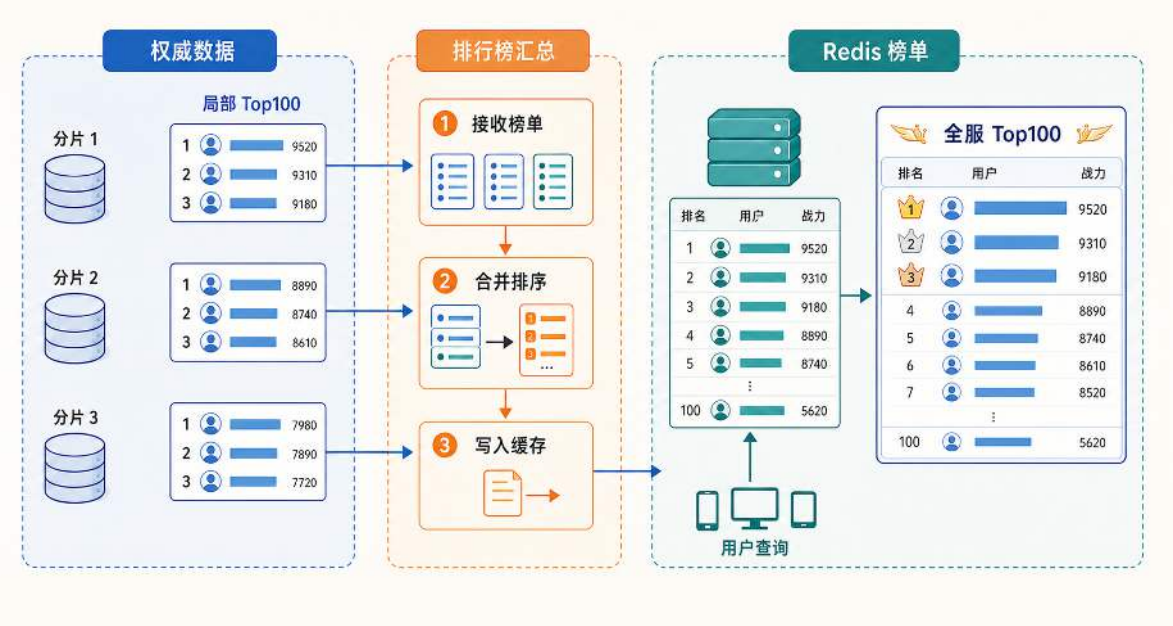


图 44: 排行榜汇总服务在后台合并各 PostgreSQL 分片的局部 Top100，并写入 Redis 物化榜单；用户请求只读取 Redis 中的全服 Top100，不再实时查询所有数据库分片。

异步汇总把一次读取从“访问所有 PostgreSQL 分片并现场合并”缩短为“一次 Redis 查询”，代价是结果具有一定延迟。刷新间隔越短，榜单越接近实时，但分片查询和缓存更新越频繁；刷新间隔越长，系统开销越低，用户看到的排名也越陈旧。汇总服务暂时失败时，系统还可以继续返回 Redis 中的上一版本榜单，待后台恢复后再生成新版本。

跨分片查询可以通过并行查询和集中汇总获得完整结果；对于允许短暂延迟的查询，还可以使用异步汇总降低读取开销。如果一次业务操作需要更新（写入）多个分片，多机系统中的处理会复杂得多。

跨分片更新：从本地事务到两阶段提交 在单个数据库内部，多项更新可以由一个本地事务统一处理，过程相对而言比较直接。以用户 A 向用户 B 转移 10 个金币为例，这次操作包含两项修改：从 A 的账户扣除 10 个金币，同时向 B 的账户增加 10 个金币。当 A 和 B 的账户记录位于同一个数据库时，数据库可以在一个事务中依次执行扣款和入账：两项修改都成功后统一提交，任一修改失败则整体回滚。

如果 A 和 B 的账户记录分别位于分片 1 和分片 2，情况就复杂得多。扣款由分片 1 的本地事务处理，入账由分片 2 的本地事务处理，原来的一次数据库事务被拆成了两个独立提交点。每个分片只能决定自己的事务是否提交，无法保证另一个分片作出相同决定。如果扣款已经提交而入账失败，金币就会消失；如果入账已经提交而扣款失败，系统又会凭空增加金币。

因此，这笔转账要求扣款和入账全部生效或全部不生效，不能留下只完成一部分的结果，这种性质称为原子性（Atomicity）。

在单个数据库中，事务管理器能够同时看到两项修改，并统一决定提交或回滚。分片后，每个数据库只能看到自己的本地事务：分片 1 不知道分片 2 是否完成入账，分片 2 也不知道分片 1 是否完成扣款。网络超时还可能使“操作失败”和“操作已经完成但响应丢失”无法区分，因此任何一个分片都不能仅根据本地结果决定整笔转账提交或回滚。

因此，跨分片更新的难点不是分别执行各项修改，而是让多个独立数据库对整项操作形成一个最终决定。两阶段提交协议（Two-Phase Commit, 2PC）为此引入协调者，收集各分片的执行状态，再向所有分片发布统一的提交或回滚决定。

协议先在准备阶段确认所有参与分片是否具备提交条件，再在决策阶段根据投票结果要求所有分片共同提交或共同回滚。准备阶段从协调者向所有参与分片发送准备请求开始。各分片执行本地事务、记录事务状态并锁定相关数据，但暂不提交；如果能够保证后续完成提交，就回复 Ready，否则回复 Abort。只有全部参与者都回复 Ready，协调者才会发布 Commit，各分片随后提交本地事务并释放锁。任一参与者回复 Abort，或者准备阶段等待超时，协调者都会发布 Rollback，各分片回滚本地事务并释放锁。这样，扣款和入账不能只提交一侧：全局结果只能是两个本地事务共同提交，或者共同回滚。

参与者回复 Ready 后，本地修改虽然尚未提交，却已经承诺能够执行后续提交，因此不能自行结束事务。从接受准备请求到收到最终决定，相关数据行必须保持锁定；图 45 以金币转账为例，分别标出扣款和入账在准备阶段的未提交状态，以及协调者在决策阶段发出的统一命令。

2PC 的原子性来自所有参与者服从同一个全局决定，它的运行代价也来自等待这个决定的过程。首先，协议需要两轮网络往返：准备阶段收集投票，决策阶段广播提交或回滚；每一轮都要等待参与分片响应，端到端延迟明显高于单分片事务。其次，参与者进入就绪状态后会持续持有行锁，其他读写相同数据的事务只能排队，高并发场景下吞吐量会被压低。最后，协调者在收集投票后、发布最终决定前发生故障，参与者可能长期停留在就绪状态，相关锁也无法及时释放。

为使事务能够在故障后恢复，协调者和参与者都要把各阶段的状态写入稳定存储中的事务日志。参与者一旦进入就绪状态，就等于已经承诺“只要全局决定提交，我就能够提交”。如果此时协调者失联，参与者既不能擅自提交，也不能随意回滚；标准 2PC 要求参与者阻塞等待协调者恢复。部分实现允许参与者询问其他节点是否知道全局结果，但这已经超出标准 2PC 的范围。生产环境中，这类故障会同时影响事务恢复、锁释放和请求延迟，排查与恢复成本也较高。

由 2PC 的运行代价可以得到一个分片设计原则：越是要求原子提交的业务事实，越应尽量减少它跨越的提交点。如果一次业务结果不可避免地涉及多个分片，2PC 可以保证这些写入最终共同提交或共同回滚，但也会把锁等待、网络等待和协调者故障带入请求主链路。因此，在线系统通常只在少数必须同步完成的操作中使用 2PC；对于允许短时间不一致的更新，则通过调整

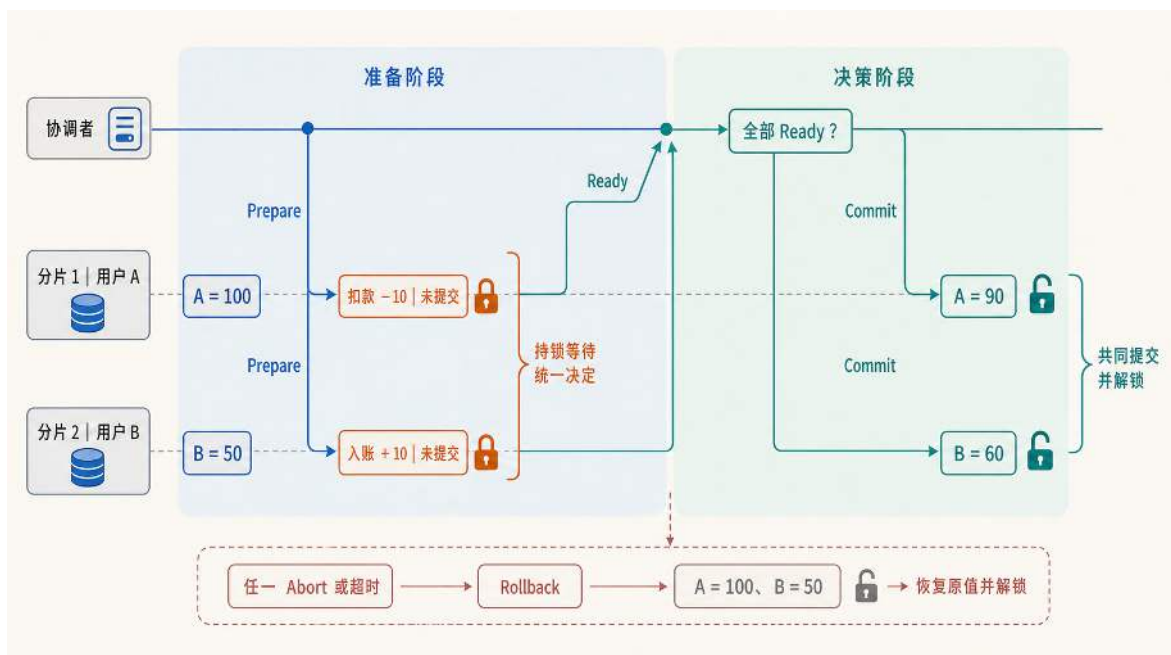


图 45: 2PC 先让两个分片完成本地修改并持锁等待；收齐 Ready 后共同提交，任一参与者中止或超时则共同回滚。

数据归属或异步传播降低跨分片协调成本。下面介绍两种常用做法。

第一种策略是异步消息队列。交互结束后，业务服务器把双方用户 ID、结果信息、时间戳等打包成一条操作结果消息，投递到消息队列。专门的记录更新服务从队列中消费消息，依次向各分片发起更新；若某次更新因分片暂时不可用而失败，消息会留在队列中等待重试。在队列持久化、重试和幂等处理都成立的前提下，丢失风险可以降到很低。这种方案把跨分片同步原子提交改为异步更新。记录可能晚几秒刷新，但在可靠投递与幂等处理成立后会逐步收敛，系统也避开了 2PC 的锁等待和协调成本。消息队列本身必须具备持久化和至少一次投递的保障，否则队列自身的故障反而会成为新的数据丢失入口。

下面的代码展示异步记录更新的核心逻辑。`finishInteraction` 只负责把操作结果写成事件并投递到消息队列；`consumeResult` 由后台消费者执行，分别访问相关用户所在分片。重要的是主链路和后台链路的分离：交互请求不再等待两个分片都更新完成。

异步记录更新的核心逻辑

```
1 type OperationResultEvent struct {
2     UserA    int
3     UserB    int
4     MatchID  string
5 }
6
7 func finishInteraction(result OperationResult) error {
8     event := OperationResultEvent{
9         UserA:    result.UserA,
10        UserB:    result.UserB,
11        MatchID: result.MatchID,
12    }
13    return messageQueue.Publish(event) // 事件持久化后才确认请求已受理
14 }
15
16 func updateOnce(userID int, matchID string) error {
17     db := shardFor(userID)
18     return db.Transaction(func(tx Tx) error {
19         if tx.HasProcessed(matchID, userID) {
20             return nil // 重试时不重复更新
21         }
22         if err := tx.UpdateRecord(userID); err != nil {
23             return err
24         }
25         return tx.MarkProcessed(matchID, userID)
26     })
27 }
28
29 func consumeResult(event OperationResultEvent) error {
30     if err := updateOnce(event.UserA, event.MatchID); err != nil {
31         return err // 保留消息，稍后重试
32     }
33     return updateOnce(event.UserB, event.MatchID)
34 }
```

在异步消息方案中，请求主链路返回前只需要确认操作结果事件已经可靠写入队列。这表示更新任务已经被系统受理，并不表示相关用户分片已经完成更新。消费者随后以 `MatchID` 和用户 ID 作为幂等标识，在每个目标分片的本地事务中完成记录更新和已处理标记。即使用户 A 已

经更新、用户 B 更新失败后触发整条消息重试，用户 A 也不会被重复更新。

如果队列中已经积压较多事件，或者某个分片写入失败后进入重试，用户界面上的记录可能晚于操作结果更新。这种方案避免了交互请求同时等待两个分片完成写入，但允许各分片的记录在短时间内处于不同进度。

本例没有在业务服务器的本地数据库中另存一份正式结果，队列中的结果事件就是后台更新各用户分片的依据，因此主链路只有“写入持久化队列”一个持久化动作。如果系统还必须在本地数据库中保存正式结果，再向其他节点发送消息，就会同时涉及数据库写入和消息投递；这两个动作无法由同一个普通数据库事务直接保护，需要采用第 4.3.4 节介绍的事务性发件箱。

不过，异步消息队列只是把跨分片写入从请求主链路移到后台。记录仍然分散在用户分片中，后台消费者最终还是要分别更新相关用户所在数据库。如果记录写入非常频繁，或者历史记录查询本身成为核心访问，系统还可以进一步改变数据归属：让记录由专门服务统一管理。

第二种策略是重新设计数据模型，从根本上减少跨分片事务出现的机会。如果记录是高频写入的核心数据，可以将记录从用户分片中完全剥离，交由一个独立的记录服务集中管理。这个服务可以使用单一数据库（若数据量可控），也可以使用专门为追加写入优化的时序数据库。所有对记录的读写操作都发生在该服务内部，从结构上消除跨用户分片写入。代价是系统多了一个独立服务和对应的数据库，运维复杂度会略有上升，但记录访问路径会更清楚。尤其是查询历史交互记录这类高频只读场景，单服务内查询通常比跨分片聚合更高效。

在代码中应用服务器在交互结束后调用 `reportResult`，把结果提交给 `RecordService`。`RecordService` 在自己的数据库事务中写入操作结果，并更新双方记录。由于这两个更新发生在记录服务内部，它们不再分别被放置在相关用户所在的用户分片上。

独立记录服务的核心逻辑

```
1 type MatchResult struct {
2     WinnerID int
3     LoserID  int
4     MatchID  string
5 }
6
7 func reportResult(result MatchResult) {
8     recordService.SaveResult(result)
9 }
10
11 func (s *RecordService) SaveResult(result MatchResult) {
12     s.db.Transaction(func(tx Tx) {
13         tx.Insert("match_results", result)
14         tx.Update("user_records", result.WinnerID, "wins + 1")
15         tx.Update("user_records", result.LoserID, "losses + 1")
16     })
17 }
18
19 func queryRecord(userID int) UserRecord {
20     return recordService.GetRecord(userID)
21 }
```

独立记录服务改变的不是提交协议，而是事务的边界。一次操作的记录、双方记录都写入记录服务自己的数据库，因此可以放在同一个本地事务中完成。相关用户的用户资料仍然可以位于不同分片，但记录更新不再经过这两个用户分片，也就不需要使用 2PC 协调多个提交点。代价是系统增加了一个专门服务，记录服务本身也要处理容量、备份和可用性问题。

异步消息队列和独立记录服务采用了不同的处理思路。前者保留原有数据归属，把跨分片写入移到后台执行；后者重新划分数据归属，使相关写入重新落入同一个事务边界。两种方案都降低了请求主链路中的跨分片协调成本，但分别引入了消息延迟和独立服务运维等代价。

PostgreSQL 分片在分散数据容量和请求压力的同时，也改变了数据访问的范围。单用户读取可以由路由层直接定位一个分片；全局读取需要并行查询多个分片并汇总结果，也可以通过物化视图把重复计算移到后台。

当一次写操作涉及多个分片时，多个本地事务还需要形成统一结果。2PC 可以保证它们共同提交或共同回滚，但会增加网络等待、持锁时间和阻塞风险；对于允许延迟生效的更新，异步消息或重新划分数据归属可以降低请求主链路中的协调成本。

前面的讨论针对 PostgreSQL 持久化层。应用服务器还会频繁访问 Redis 中的在线状态、实时位置和排行榜等热数据；如果这些数据仍然集中在单个 Redis 实例中，缓存层就会成为新的容

量与可用性瓶颈。下面继续讨论缓存层如何从单机 Redis 演进为 Redis 集群。

4.2.7 缓存层的分布式演进：从单机 Redis 到集群

Redis 是一种键值数据库：应用用一个字符串形式的键（Key）定位一份值（Value），例如用 `user:1001:pos` 读取用户 1001 的实时位置。在单机阶段，所有键都放在同一个 Redis 实例中，应用只需要连接这一个入口；进入多机阶段后，如果所有热数据仍然集中在这个实例中，缓存层就会成为新的集中点。在线人数增长时，单个 Redis 实例要同时承受更多键、更高网络吞吐和更多命令处理压力；一旦它不可用，原本命中缓存的读取会集中回到 PostgreSQL，前面通过数据库分片分散掉的压力又会在回源请求中被放大。因此，缓存层需要从单机 Redis 演进到 Redis 集群。

PostgreSQL 分片说明了一个基本原则：数据拆到多个节点后，请求只访问与自身查询范围相关的节点。单用户资料可以根据用户 ID 直达一个分片，全服排行榜则必须访问多个分片并合并结果。

缓存层扩展时沿用同一原则。Redis 不再把全部热数据集中在一个实例中，而是把键空间拆分给多个缓存节点，使每个节点只承担一部分 key 的存储和访问。

PostgreSQL 分片和 Redis 缓存分片的目标相同，都是把数据与请求压力分散到多个节点，但二者组织数据的方式不同。PostgreSQL 中的用户资料具有明确的业务归属，一条用户记录可以根据用户 ID 确定所属分片，全局查询则根据查询范围决定是否访问多个分片。Redis 保存的在线状态、实时位置和会话等数据具有不同结构，但每份数据都由唯一的 key 标识，因此缓存路由不需要解析具体业务字段，只需要根据 key 确定负责节点。

如果直接用 key 的哈希值和当前节点数量计算目标节点，节点数量就会成为路由公式的一部分。例如，集群从三个节点扩展到四个节点后，许多 key 的计算结果会随之改变；数据仍在原节点，请求却可能被发送到新节点，系统必须迁移大量数据，否则就会出现大范围缓存未命中。这种不稳定性来自 key 归属与节点拓扑直接绑定：每次增加或移除节点，都可能重新计算大量 key 的位置。

Redis 集群在 key 和物理节点之间增加了一层数量固定的哈希槽。key 先通过固定算法映射到槽位，槽位再通过槽位表归属于某个 Redis 节点。节点扩缩容时，key 对应的槽位编号保持不变，集群只需把部分槽位及其中的数据迁移给新节点，并更新槽位表。这里的“稳定”并不是让一个 key 永远停留在同一台物理节点上，而是让 key 到槽位的计算不受节点数量变化影响，使数据迁移能够按槽位分批进行。哈希槽（Hash Slot）就是 Redis 集群内部预先划好的编号位置。整个集群固定包含 16384 个槽位，这是 Redis 集群协议规定的槽位总数，业务侧不能把它改成其他数量。²⁴ 客户端拿到一个 key 后，先通过哈希计算得到槽位编号；随后根据本地保存的槽位表，查出这个槽位当前由哪个 Redis 主节点负责。槽位表可以理解为一张“编号区间到节点”的映射，例如槽位 0 到 5460 由节点 A 负责，槽位 5461 到 10922 由节点 B 负责，槽位 10923 到 16383 由节点 C 负责。这样，`user:1001:pos` 先被计算到某个槽位，再由槽位表定位到具体

²⁴Redis Cluster 官方规范说明了哈希槽和槽位重定向规则，见 https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/。

Redis 节点。

当系统新增 Redis 节点时，只需要把一部分槽位从旧节点迁移给新节点；迁移完成后，落在这些槽位上的 key 就由新节点负责。业务代码仍然按原来的 key 读写缓存，变化被集群客户端和槽位表吸收。如果客户端本地的槽位表已经过期，请求可能到达旧节点；旧节点会返回重定向信息，客户端更新槽位表后，再访问新的负责节点。

一次缓存请求先计算固定槽位，再依据槽位表选择 Redis 节点；扩容时，变化发生在槽位到节点的映射，而不是 key 到槽位的计算。图 46 上半部分展示 key 如何先计算出固定槽位，再通过槽位表定位到 Redis 节点；下半部分以新增节点 D 为例，展示槽位 13653 到 16383 及其中的数据如何从节点 C 迁移到节点 D，并相应更新槽位表，而 key 到槽位的计算规则保持不变。

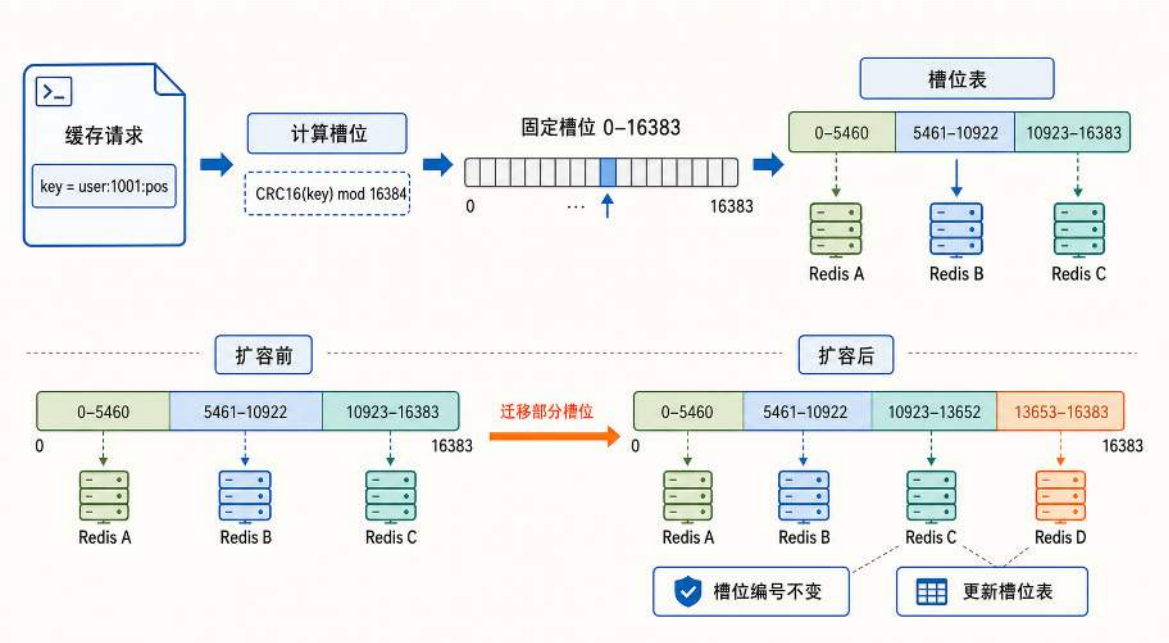


图 46: Redis Cluster 通过固定哈希槽解耦 key 与物理节点：key 先映射到槽位，槽位表再定位节点；扩缩容时只迁移部分槽位及其中的数据。

固定槽位把 key 的计算结果与当前节点数量解耦，使扩缩容可以按槽位分批迁移，而不必重新计算全部 key 的归属。

上面的讨论只处理了单个 key 的定位；当一次业务流程需要同时读写多个相关 key 时，槽位分布还会影响这条流程能否在一个 Redis 节点上完成。单个 key 可以自然定位到一个槽位，但协作状态、房间状态、同一用户多份临时状态这类操作，经常需要在一次流程中同时访问多个 key。例如，逻辑服在处理用户 1001 断线重连时，可能需要同时拿到该用户是否在线以及最后一次上报的位置。如果写成一次缓存读取，请求中会同时出现两个 key：

多 key 读取命令示例

```
1 MGET user:1001:online user:1001:pos
```

这类“一条命令同时操作多个 key”的请求，就是多 key 操作。如果这两个 key 分别落到不同 Redis 节点，单独读取任何一个 key 都没有问题；但合并为一条命令后，Redis Cluster 会直

接拒绝执行并返回 CROSSSLOT 错误，因为它不会跨节点协调多 key 操作。

Redis 集群提供哈希标签 (Hash Tag) 为这类相关 key 提供同槽位约束。²⁵ 哈希标签是一种键名约定：如果 key 中包含大括号，Redis 只取大括号内的部分来计算槽位。例如 `{user:1001}:pos` 和 `{user:1001}:online` 都使用 `user:1001` 计算槽位，因此它们一定落到同一个槽位和同一个 Redis 节点，多 key 操作就可以在该节点上完整执行。图 47 展示了这个对比：左侧使用默认键名，两份相关数据被分到不同槽位和不同节点，多 key 操作无法一次完成；右侧使用哈希标签后，两份数据落到同一节点，可以在一条命令内完整处理。

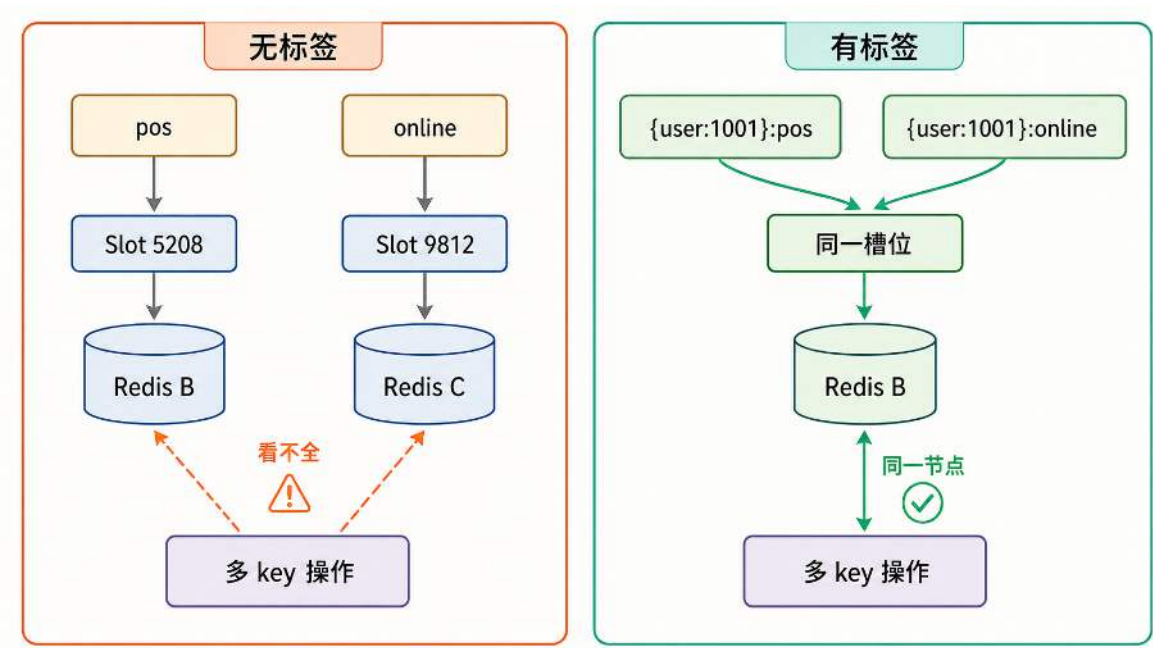


图 47: 哈希标签将相关 key 放到同一槽位：默认键名下，同一用户的在线状态和位置可能分别被放置到不同 Redis 节点；使用 `{user:1001}` 后，相关 key 可以由同一节点完整处理。

这也是缓存键设计需要考虑业务归属的原因：同一用户、同一房间或同一队伍中需要一起读写的热数据，适合使用同一个哈希标签；没有共同操作关系的 key 则不应随意放进同一标签，否则会把压力重新集中到少数槽位上。

缓存分片只决定热数据落在哪个 Redis 节点上，并不要求 Redis 节点和 PostgreSQL 分片固定绑在同一台机器上。两层存储可以分别扩展，但都围绕同一个业务主语组织数据：用户 1001 的权威记录通过 PostgreSQL 分片路由定位，用户 1001 的热数据通过 Redis 槽位定位。应用服务在处理登录、移动、断线重连或排行榜刷新时，仍然以用户 ID 作为入口，只是底层访问路径分别进入持久化层和缓存层。经过这一层拆分，数据层形成了两条可扩展路径：冷数据按分片路由进入 PostgreSQL，热数据按槽位路由进入 Redis；两类集中访问点都被拆到多个节点上，容量、吞吐和故障影响也随之分散。

²⁵Redis Cluster 对哈希标签和多 key 操作限制的说明见官方规范：https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/。

4.3 协同：跨服交互与事务

前面的“均衡”把用户、连接和数据分配到多个实例上，使每台服务器只负责一部分对象。在单机系统中，一个进程能够看到完成操作所需的全部状态，并直接确定处理顺序和最终结果；进入多机系统后，每台服务器只能观察自己的局部状态，一次业务操作却可能同时依赖多个服务器。

当一次业务操作所需的参与者、数据或处理结果分布在不同服务器上，单台服务器无法仅凭本地状态完成整个过程，这类操作称为**跨服交互**。跨服交互需要服务器通过网络交换信息，但信息能够传到其他服务器，并不意味着多台服务器已经形成业务可以接受的结果。

要让多个独立运行的服务器共同完成一项业务，还需要规定哪些状态差异可以暂时存在、由谁确定正式结果，以及结果如何传播并落实到其他服务器。这组使多个节点形成业务可接受结果的跨节点规则，构成多机系统中的**协同**。

按照跨节点处理的对象和目标，跨服交互可以分为三类：通知类传播已经确定的事件，查询类把分散的局部状态组成读取视图，事务类则让多项更新共同生效或使多个服务器接受同一个结果。三类场景能够容忍的状态差异不同，所需的协同约束也随之不同。

4.3.1 跨服交互的三类场景与约束

通知类交互传播的是已经产生的事件。例如世界 Boss 出现通知。世界 Boss 已经在服务器 A 负责的地图中生成，该服用户随即看到活动公告、地图标记和挑战倒计时。此时服务器 B 和服务器 C 可能尚未收到消息，它们的用户仍停留在原来的界面；随着消息传播，各服陆续显示活动提示。活动入口是否开放、倒计时从何时开始等业务状态，应以可持久化的权威记录为准，不能取决于服务器是否收到这条即时通知。短暂的到达时间差不会改变 Boss 已经生成这一事实，但系统仍要先区分通知的重要程度。允许用户偶尔错过的即时提示可以采用尽力而为传播；若遗漏会影响后续业务，事件就必须进入可持久化、可确认和可重放的消息路径，不能只依赖瞬时通知。同样的结构也出现在电商促销启动通知、直播间的全局广播等场景中。

查询类交互需要把多个服务器上的局部数据组成一个读取视图。例如跨服竞技场排名系统。竞技场开放报名后，服务器 A 的用户刚刚加入队伍，服务器 B 的一名用户同时退出报名，而大厅中的参赛列表仍显示上一次汇总结果。用户在刷新前可能暂时看不到新加入者，也可能看到已经退出的旧成员；只要系统在规定时间内重新汇总各服数据，列表就会恢复为最新状态。查询类场景允许读取视图短时间陈旧，但必须限定旧数据持续的最长时间，这一要求称为有界收敛。分布式搜索索引、跨区域库存预览也采用类似的异步汇总机制。

事务类交互会同时改变多份状态，或者要求多个服务器接受同一个决定。以跨服交易为例，用户 A 在服务器 1 确认支付一定资源，用户 B 在服务器 2 等待接收对应资源。如果网络中断后服务器 1 已经扣款，服务器 2 却没有交付资源，那么两名用户面对的就不是短暂延迟，而是一笔结果矛盾的交易。跨服战斗也不能让一台服务器记录甲方获胜，另一台服务器却记录乙方获胜。事务类场景不能接受这类冲突结果，需要让多项更新同时成立或同时撤销，或者由一个权威位置给出各服共同接受的唯一结果。跨账户转账、分布式库存扣减和联名票务预订都属于同一类

事务类约束。

前述三类交互都会在服务器之间传递信息，但“信息已经发出”并不等于业务已经完成。图 48 将三类场景归结为一个共同问题：当不同服务器看到的状态还不相同时，一次跨服操作在什么条件下才算完成？

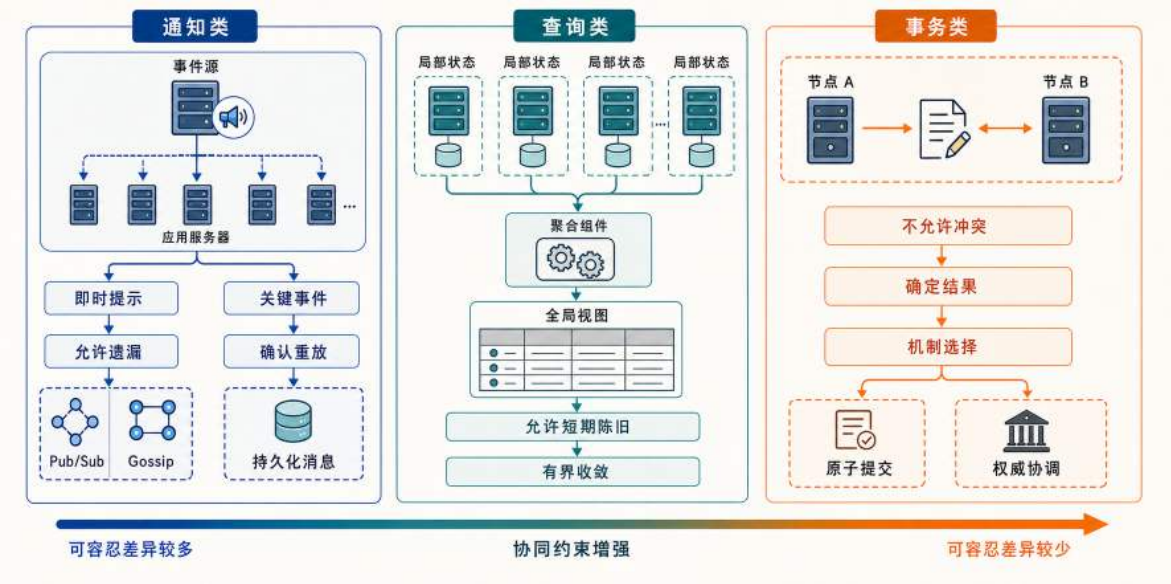


图 48: 跨服交互的典型约束与机制选择：通知类按事件重要程度选择即时传播或可靠消息，查询类通过有界收敛形成全局视图，事务类通过原子提交或权威协调形成确定结果。

对于通知类，事件在源服务器产生后已经成立，其他服务器是否必须收到取决于事件的重要程度；对于查询类，全局视图可以暂时落后，但必须在限定时间内更新；对于事务类，参与节点尚未形成同一个结果时，操作就不能算完成。完成条件的不同，分别引出了事件传播、视图聚合和事务协同三类机制。下面依次讨论这些机制的工作方式、适用条件与实现选择。

4.3.2 通知类：事件如何传播

通知类交互传播的是已经确定的事实，但不同事件需要的送达保证并不相同。允许偶尔遗漏的即时提示重视低延迟和覆盖范围。遗漏会影响后续业务的事件，还要进入可持久化、可确认、可重放的消息路径。两类传播都要控制重复消息和广播开销。

以在线游戏为例，某个地图分片生成世界 Boss 后，可以用即时通知让其他地图分片尽快显示公告。活动入口和倒计时属于业务状态，要从可持久化的权威记录中读取。

即时公告要尽快到达较多服务器，又不能占用过多网络 and 计算资源。同一事件可能从多条路径重复到达，服务器不能重复显示公告；服务器数量增加后，事件源也不宜为每个目标长期维护一条广播连接。

转发责任有两种常见安排。可以让一个专门的中间节点接收事件并负责复制，应用服务器只需要发布和订阅；也可以让已经收到事件的服务器继续转发，使传播压力分散到多个节点。前一

种方式路径短、规则集中，但中心节点会承受可用性和容量压力；后一种方式减少了中心依赖，却必须处理重复消息、传播范围和停止条件。下面分别对这两种责任分配方式详细展开。

集中式转发：从事件源到订阅者 最直接的设计是增加一个消息中间件，让所有应用服务器订阅同一个事件频道。产生世界 Boss 的服务器把事件发布到频道，中间件再把同一条消息转发给当前在线的订阅者，这种通信方式称为发布/订阅（Publish/Subscribe）。

以 Redis Pub/Sub 为例，流程如下：所有应用服务器启动后，都向 Redis 订阅一个名为 `world_events` 的频道。当服务器 A 上刷新了世界 Boss，服务器 A 向 `world_events` 频道发布一条消息，内容包含事件类型、Boss ID、刷新位置和时间戳。Redis 收到消息后，立即将它推送给当前保持订阅连接的服务器。各服务器收到后更新本地公告。活动入口和计时仍以权威活动状态为准。

这个过程可以拆成两个长期运行的组件：产生事件的服务器负责发布消息，其他服务器启动时建立订阅并持续等待新消息。订阅端收到事件后，还要在本地显示公告；Redis 只负责转发消息，不负责修改活动状态。下面的伪代码把发布、订阅和事件处理连成一次消息流转：

Redis Pub/Sub 的发布与订阅流程

```
1  const channel = "global_events"
2
3  // 事件源：全局活动在服务器 A 触发
4  func publishGlobalEvent(redis Redis, entity Entity) {
5      event := GlobalEvent{
6          ID:          newEventID(),
7          EntityID:    entity.ID,
8          Position:    entity.Position,
9          Timestamp:   now(),
10     }
11     redis.Publish(channel, encode(event))
12 }
13
14 // 其他应用服务器启动时订阅频道
15 func subscribeGlobalEvents(redis Redis) {
16     messages := redis.Subscribe(channel)
17     for message := range messages {
18         event := decode(message.Payload)
19         displayNotification(event)
20     }
21 }
```

发布者只需发送一次消息，复制和分发由中间件完成，应用服务器之间不需要彼此维护广播连接。

集中式消息分发的优势在于实现简单、延迟可控。消息从发布者到当前在线的订阅者只经过一个中间节点，转发路径较短。新增一台应用服务器时，只需让它连接到同一个 Redis 实例并订阅相应频道，不需要修改其他服务器的配置。

集中式方案也有明显的局限。中间件不可用时，所有广播都会中断；服务器数量增多后，连接数和消息复制流量也会集中在这个节点上。假设集群有 100 台应用服务器，一条广播消息需要被 Redis 复制并推送 100 次；如果每秒产生多条广播事件，Redis 的网络出口带宽和 CPU 会成为瓶颈。Redis Pub/Sub 也不保存离线期间错过的消息；需要回放的关键事件应进入持久化消息日志，而不是只依赖即时频道。如果希望减轻中心节点的连接和复制压力，可以把转发责任交给已经收到事件的应用服务器，由它们继续向其他节点传播，这就是去中心化扩散。

去中心化扩散：让接收者继续传播 在这种模式下，服务器收到事件后既完成本地处理，也承担下一轮传播任务。事件源不必直接连接集群中的每个节点，传播压力会随着知情节点数量增加而分散。

Gossip 协议（也称流行病传播协议，Epidemic Protocol）²⁶就是这种去中心化广播方法的代表。它不要求事件源一次性联系所有节点，而是让已经收到事件的节点继续向少量随机目标转发。新收到事件的节点在后续轮次中也参与传播，传播能力因而会随着知情节点数量增加而扩大。

随机选点不要求每个节点保存完整的集群成员列表，但节点至少要维护一组可以联系的成员。系统也可以通过另一套 Gossip 过程更新这份成员视图，使节点逐步发现新成员和失效成员。要判断这种方案是否可用，既要考察覆盖范围如何随轮次扩大，也要处理随机转发带来的重复到达和传播停止问题。图 49 以 $\text{fanout}=2$ 的一次扩散为例，连续呈现了同一组节点的状态变化。

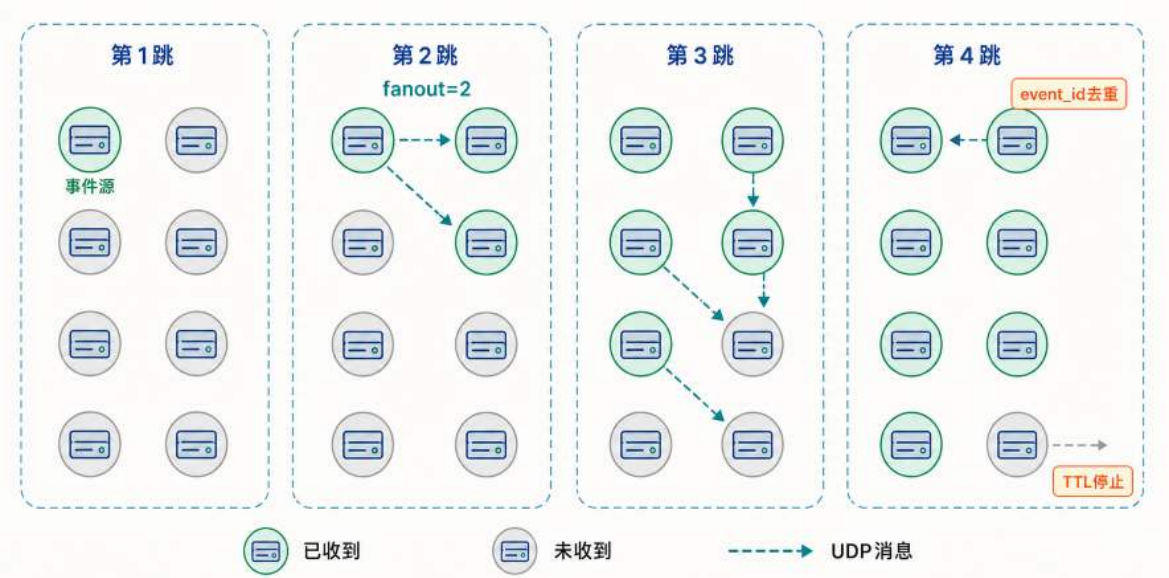


图 49: Gossip 逐跳扩散事件：已收到事件的节点在每轮选择少量随机目标转发，覆盖范围逐步扩大；节点根据事件 ID 避免重复执行业务，并通过 TTL 停止传播过期事件。图中的轮次和覆盖结果仅为示意，不表示固定轮次必然覆盖全部节点。

²⁶Gossip/Epidemic 协议的经典分析见 A. Demers et al., “Epidemic Algorithms for Replicated Database Maintenance”, PODC 1987.

事件产生时，只有事件源处于“已收到”状态。传播扇出量 $\text{fanout}=2$ 表示一个知情节点每轮选择两个随机目标；第一批目标收到事件后，会在后续轮次中继续选择新的目标。转发者由一个增加到多个，覆盖范围随之扩大，而事件源不必直接连接集群中的全部节点。

随机选点并不保证每次发送都命中尚未收到事件的节点。同一事件可能沿不同路径再次到达知情节点，固定轮次结束后也可能仍有节点没有收到消息。事件 ID 使节点在重复收到事件时不再重复执行业务，TTL 则使过期事件停止参加后续传播。二者分别控制重复处理和传播停止，但不能把概率扩散变成严格送达。

fanout 越大，消息扩散越快，但重复消息和网络流量也越多； fanout 越小，带宽开销较低，收敛时间则会变长。在一个 N 台节点的集群中，如果每轮每个已经收到事件的节点向 b 台随机节点转发，消息覆盖大部分节点所需的轮次通常随 $\log N$ 增长。当 $N = 100$ 、 $b = 2$ 、每一跳的转发延迟约为 200 毫秒时，一条消息在理想随机选择和节点持续可达的条件下，可能在 1~2 秒内到达大多数节点。这个数值只用于说明传播量级，不是覆盖全部节点的时延上界。对于公告和全局事件通知来说，这个收敛速度通常是可以接受的。

各节点按自己的周期执行转发，不需要等待全体确认。一次丢包通常不会中断整个传播过程，因为其他知情节点和后续轮次还会提供新的发送机会。这种冗余提高了覆盖概率，但仍不能保证每个节点都收到消息。

实现时，每个节点需要分别维护“该事件是否已经完成本地处理”和“该事件是否仍需继续传播”两类状态。前者由事件 ID 判断，后者由过期时间或剩余跳数判断。下面的伪代码将这两个判断分别放入接收流程和周期转发流程。

Gossip 的转发判断

```
1 func OnEvent(event Event) {
2     if now().After(event.ExpiresAt) {
3         return
4     }
5
6     // 重复到达时不重复执行业务；处理失败则等待下次重试
7     if err := applyOnce(event.ID, event); err != nil {
8         return
9     }
10
11     activeEvents.PutIfAbsent(event.ID, event)
12 }
13
14 func GossipRound() {
15     activeEvents.DeleteExpired(now())
16     for _, event := range activeEvents.All() {
17         for _, peer := range pickRandomPeers(fanout) {
18             go gossipTo(peer, event)
19         }
20     }
21 }
```

`applyOnce(event.ID, event)` 负责本地幂等处理。事件 ID 的记录和业务更新应在同一个可靠步骤中完成；如果只能分步执行，业务更新本身也要允许幂等重试，避免进程在两步之间崩溃后留下错误状态。`activeEvents` 保存仍需传播的事件，`GossipRound` 周期性地为它们选择新目标。这里用 `ExpiresAt` 表示绝对过期时间；如果实现采用剩余跳数，每次转发时还要将跳数减一。待转发状态默认保存在内存中。节点重启后如果还要继续传播，就要持久化这部分状态。即使如此，Gossip 提供的仍是概率覆盖；严格送达还需要持久化消息、确认或重放。

Gossip 并不依赖单一的中心转发节点。任一普通节点宕机后，其他已收到消息的节点仍可以沿剩余路径继续传播，广播不会因为一个中心转发节点失效而整体中断。当集群规模很大（数百台服务器）时，Gossip 会把转发压力分散到多个参与节点，通常比集中式方案更少依赖单一中间件出口；但随机选点和节点能力差异并不保证每个节点承担完全相同的流量。Gossip 的劣势是延迟不如集中式可控，且由于消息沿随机路径扩散，不同节点收到同一组消息的顺序可能不同，因此不适合需要严格有序的场景。

实际系统可以根据事件的送达要求组合使用这两种方式。单个区域内的活动公告适合集中式 Pub/Sub，路径短，实现简单；跨区域节点的存活状态探测会周期性地产生新结果，可以用 Gossip 分散转发压力。

两类事件的差别在于，漏掉一次通知后，后续消息能否纠正当前状态。某个节点没有收到一轮存活探测结果，仍有机会在后续轮次获得最新结果，因此单次漏达通常不会长期影响判断。成员加入或移出、用户权限变更则不同：每次变更都会独立改变系统状态，漏掉其中一次就可能使节点长期保留错误结果，因而不能只依赖 Gossip 传播。

对于这类不能遗漏的事件，系统应先把最终结果写入数据库或持久化事件日志，再把它作为通知传播。接收方处理成功后返回确认；未确认或暂时离线的节点恢复后，可以重放遗漏事件，或者读取数据库中的最新结果完成补齐。

通知类交互让相关服务器尽快知道一件已经发生的事实。当用户打开跨服报名列表时，系统面对的则是持续变化的多份局部状态：各服的报名和退出可能先后发生，用户需要看到的是由这些状态汇总而成的全局视图。这个视图不必在所有服务器上同时更新，但不能长期停留在旧状态，查询类交互讨论的便是这个问题。

4.3.3 查询类：分散状态如何汇成一致视图

查询类交互需要把多个服务器上的局部数据组成一个读取视图。以在线游戏为例，跨服擂台开放报名后，每台应用服务器最先知道的是本服参赛者，用户界面却需要显示全服可挑战对象。如果每次打开列表都实时询问所有服务器，请求延迟会由最慢节点决定，任一服务器超时还会使列表缺少一部分用户。

报名列表通常允许几秒钟的刷新延迟。如果上报、传输和汇总都能在规定期限内继续运行，用户后续刷新时就能看到更新后的名单。这正是本节开头提到的有界收敛：读取视图可以短暂落后，但落后时间必须有明确上限。因此可以沿用前文全服排行榜的异步汇总思路，把“每次打开列表都实时询问所有服务器”换成“后台定期汇总、用户只读缓存”。

各应用服务器定期上报本服的报名快照，或者在用户报名、退出时发送变更事件；汇总服务把这些局部状态合并成一份全服视图写入缓存。这条汇总路径可以拆成两个长期运行的组件：各服负责周期性上报自己的报名者，汇总服务负责周期性合并并刷新缓存。

跨服报名列表的异步汇总流程

```
1 // 各应用服务器：周期性上报本服当前报名者
2 func reportRegistrations(hub Aggregator, server ServerID) {
3     for range ticker(reportInterval) {
4         snapshot := localRegistrations()
5         hub.Report(server, snapshot, now())
6     }
7 }
8
9 // 汇总服务：周期性合并各服快照，全量写入缓存
10 func aggregateLoop(hub Aggregator, cache Cache) {
11     for range ticker(refreshInterval) {
12         var view []Entry
13         for server, report := range hub.Latest() {
14             if expired(report.Time) {
15                 continue // 超过刷新期限未上报，视为过期，丢弃
16             }
17             view = append(view, report.Entries...)
18         }
19         cache.Store("arena_list", view) // 每轮全量覆盖
20     }
21 }
```

代码中的 `hub.Latest()` 取每台服务器最近一次上报，用更新时间区分新旧，让新快照覆盖旧快照，而不是让先后到达的上报互相错乱。`expired(report.Time)` 对应过期标记：某台服务器超过刷新期限没有上报，它的报名者就从视图中移除，已经退出或掉线的失效对手不会长期留在列表里。

关键在最后一行：汇总服务每一轮都重新全量生成视图，而不是在旧视图上增量修补。这样即使某次上报因为网络原因丢失或乱序，也只影响当前这一轮的结果，下一周期就会被重新汇总的数据覆盖。当各服务器按固定周期持续上报、网络延迟存在可接受上限、汇总服务持续可用，并且丢失的上报会在下一周期被新快照替代时，陈旧时间才可以近似限制在“一个上报周期、一次有界传输延迟和一个汇总刷新周期”之内。这些成立条件一旦被节点长期失联或汇总服务故障破坏，系统只能剔除过期数据或进入降级状态，不能继续宣称视图会在原上界内收敛。

刷新周期越短，列表越接近实时，后台汇总和缓存写入也越频繁；周期越长，系统开销越低，用户看到旧名单的时间也越长。跨服报名列表与图 44 中的物化榜单采用了相同思想，区别只在于数据来源从 PostgreSQL 分片变成了多个应用服务器。在擂台系统中，协调服务器接收各服报名变更并维护权威参赛状态，缓存只承担列表浏览的读取视图；报名和退出是否真正生效，仍由协调服务器判断。

4.3.4 事务类：跨节点事务如何保证一致

上一小节中的全局参赛者列表只是供用户浏览的读取视图，允许短暂陈旧。甲从列表中选择乙并发起挑战后，系统开始改变双方的参赛状态、积分和战绩，交互也由查询进入事务处理。此时需要分别解决两个问题：谁来确认双方仍可参战并生成唯一结果；结果确定后，怎样保证服务器 A 和服务器 B 最终都按这份结果更新，并避免故障或重试造成遗漏和重复执行。

为聚焦跨服协调，本例将战斗过程简化为根据报名时上报的状态快照计算胜负。甲由服务器 A 管理，乙由服务器 B 管理。如果两台服务器分别依据本地状态接受挑战，并发请求可能同时把同一名参与者判断为空闲，使其进入多场比赛。因此，系统首先要把参战资格检查和正式结果生成收束到一个裁决位置；结果如何可靠地落实到两台服务器，则在后文继续解决。

唯一裁决点：协调服务器与本服代理 参赛报名时，各应用服务器把参与者标识、所在节点和状态快照上报给协调服务器，由协调服务器维护全局参赛状态。服务器 A 收到甲挑战乙的请求后，不根据本地列表直接判定，而是由本地代理把请求转发给协调服务器。

协调服务器需要把资格检查与状态占用作为一次不可分割的操作：只有甲、乙仍在报名且处于空闲状态时，才同时把双方标记为“交互中”。这样，后到的并发请求会发现甲或乙已被占用，不能再次接受同一参与者；随后协调服务器为本场比赛生成操作编号 M1024，并根据报名时保存的状态快照形成一份正式结果。

协调服务器把这份结果分别发送给服务器 A 和服务器 B，两台应用服务器只更新各自参与者的积分和战绩，不再根据本地状态重新判断胜负。正式结果、通知任务和各服的执行记录都携带同一个 `match_id`，以便识别它们属于同一次挑战。挑战请求从服务器 A 经本地代理上行到协调服务器，唯一结果再分别下发到服务器 A 和服务器 B，由此形成图 50 所示的“单点裁决、多服执行”结构。

这样就可以把裁决权与执行权分开。来自不同区域的挑战都要先经过协调服务器；一个请求占用甲或乙后，后续并发请求会在同一处发现其状态已经改变。应用服务器不再独立裁决，同一场比赛也只接受协调服务器生成的正式结果。

单点裁决解决了“谁生成正式结果”的问题，却没有保证结果一定能够落实到两台服务器。图中的两条结果只表示消息应当发往服务器 A 和服务器 B，并不表示两条消息都已经送达并执行。前文的异步记录更新方案把持久化队列中的结果事件作为后台更新依据，请求主链路不再另外保存一份正式结果。当前的跨服竞技场景则有不同要求：协调服务器既要在本地数据库中保存正式比赛结果，作为后续查询和故障恢复的依据，又要通知服务器 A 和服务器 B 更新各自参与者的积分与战绩。

保存比赛结果属于本地数据库写入，发送通知属于跨节点网络操作。这两个动作由不同系统完成，不能直接放进同一个数据库事务。无论先执行哪一个动作，服务器在两步之间发生故障，都可能只完成其中一项。

事务性发件箱：让裁决结果与待发消息原子提交 事务性发件箱 (Transactional Outbox) 的核心做法，是把“稍后要发送的通知”先写成与比赛结果同库的待发记录。为保存这两类记录，

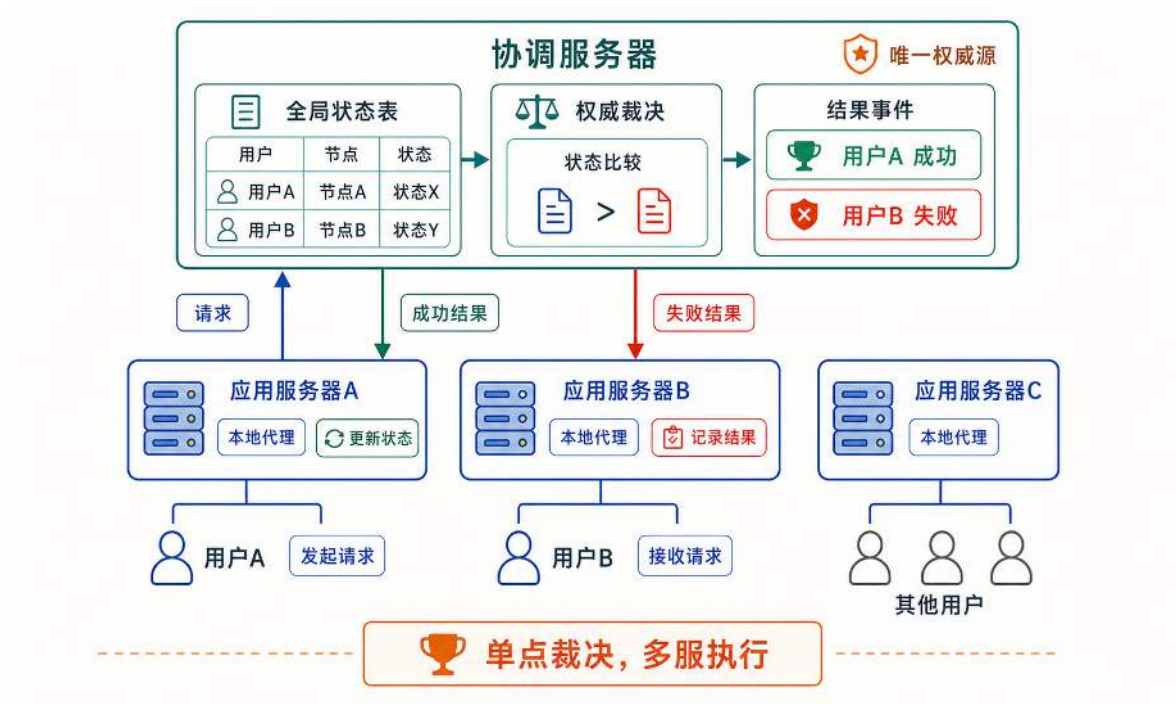


图 50: 挑战请求由本服应用服务器上到协调服务器；协调服务器依据全局参赛状态生成唯一结果，再将结果下发到相关应用服务器执行。

协调服务器会在本地数据库中建立两张表：比赛结果表 `match_results` 保存已经确定的裁决结果，发件箱表 `outbox` 保存尚待发送的通知任务。协调服务器裁决“甲获胜、乙失败”后，在一个本地事务中完成三项写入：向结果表写入比赛结果，再向发件箱表写入通知服务器 A 加分和通知服务器 B 记录败场的两条任务。

只有三项写入都成功，事务才会提交；任一写入失败，三项写入全部撤销。这样，只要正式结果已经保存，后续通知双方的任务就不会缺失。

不过，事务提交只保证结果和待发任务已经保存，并不表示两台服务器已经更新。提交之后，每条任务还要经过发送、处理和确认；未收到确认的任务必须继续保留并重试，已经处理过的任务则不能因重试而重复执行。图 51 将这一过程展开为“一次提交、首次派发和失败重试”三个连续阶段。

图中第一阶段利用原子性保证比赛结果与两条待发任务不会分离；持久性则保证协调服务器重启后仍能从数据库中恢复尚未完成的任务。

随后，发件箱派发程序 (Outbox Dispatcher) 逐条读取待发任务并异步发送。假设服务器 A 完成本地更新并返回处理确认，Dispatcher 就把发往 A 的任务标记为“已发送”；服务器 B 暂时不可达或没有返回确认时，发往 B 的任务仍保持“待发送”，无需撤销服务器 A 已经完成的更新。

Dispatcher 会在后续扫描中重试发往服务器 B 的任务。由于上一次发送可能已经执行、只是确认丢失，同一结果可能多次到达；服务器 B 在更新战绩前检查 `match_id`，已经处理过 M1024 就只返回原有结果，不再重复更新。这种持续重试直到收到确认、同时允许消息重复到达的方式称为至少一次投递 (At-Least-Once Delivery)。

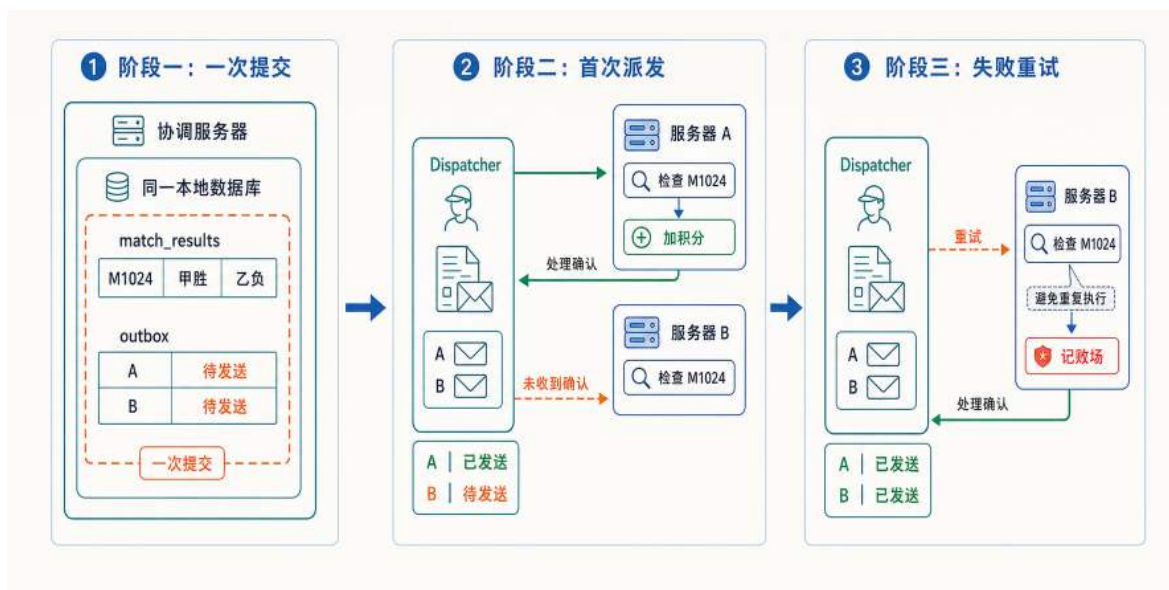


图 51: 事务性发件箱先共同提交比赛结果与两条待发任务，再逐个投递：服务器 A 确认后对应任务变为“已发送”，服务器 B 未确认时仍保持“待发送”，后续重试通过 `match_id` 避免重复执行。

整个过程采用两类正确性保障：本地事务保证结果与通知任务不会分离，处理确认、失败重试和幂等检查共同推动远端状态最终完成更新。事务性发件箱因此保证通知任务不会被遗忘，但不会把两台远端服务器纳入一次同步数据库提交。

这与第 4.2.6 节中的跨分片 2PC 形成两种不同的一致性边界。2PC 适用于同一个业务事实必须在多个数据库提交点上原子成立的场景，但任一参与者迟迟不能完成都会阻塞整条链路。跨服对战先把裁决事实收束到协调服务器的本地数据库，再把积分、奖励等派生状态放到异步传播阶段，从而避免让远端应用服务器共同进入一次同步提交。这种方案依赖一个业务前提：双方服务器可以在短时间内先后收到结果；如果所有远端状态都必须在响应用户之前同步生效，就仍需采用更强的跨节点协调机制。

跨节点事务由此形成两层正确性保证：唯一裁决点保证同一次操作只产生一份权威结果，事务性发件箱保证这份结果能够被持续重试并最终落实到相关服务器。权威结果的原子提交被限制在协调服务器的本地事务内，跨服派生状态则通过可靠消息和幂等重试逐步收敛。这种结构不仅适用于竞技场，也适用于其他需要“单一权威节点生成结果、多个执行节点落实更新”的跨节点事务。

这套设计仍留下一个尚未解决的问题：擂台协调服务器本身是唯一裁决位置。一旦它不可用，所有区域都无法提交报名和挑战；如果直接部署多个能够独立裁决的副本，又会重新出现多份结果。如何让多个副本在节点故障后仍沿着同一份裁决历史继续服务，构成下一节“容错”的起点。

4.4 容错：冗余、复制与一致性

均衡解决了“怎样分配”，协同解决了“怎样互动”，分布式系统还必须面对第三类问题：当节点故障、网络中断或数据损坏发生时，系统如何维持服务并恢复到正确状态？

分片和多机部署提高了系统的总容量，但节点数量增加后，系统中至少出现一台故障机器的概率也会升高。硬件损坏、进程崩溃和网络中断不会因为部署了更多机器而消失。多机容错能够改变的是故障发生后的结果：先避免故障沿共享依赖扩散，再为受影响的服务保留继续运行或恢复的路径。

第一个任务是控制故障影响范围。多台机器怎样组成请求路径，决定了一台机器故障后，影响会扩散到所有请求，还是被限制在部分用户。从故障传播方式看，请求依赖关系可以抽象为两种模型：所有请求共享必经路径的串联依赖，以及不同用户组沿独立分片路径访问的分片隔离。二者并不是互斥的部署方案，真实系统通常会在每条分片路径内部继续串联多个服务节点。

在串联依赖模型中，一次服务依次经过多个节点，每个节点都是所有用户请求的必经路径。任意节点停止工作，整条路径就中断，所有用户的请求因此共享同一个故障传播路径。在各机器故障相互独立的理想化假设下，如果每台机器的可用性为 99.9%，且 100 台机器串联、任意一台不可用都会使请求失败，那么整体可用性只有 $99.9\%^{100} \approx 90.5\%$ ；节点越多，整条路径保持可用的概率越低。

在分片隔离模型中，请求先按用户归属选择目标分片，只依赖负责该用户组的节点。比如当分片 3 的主库发生故障时，分片 1、2、4、5 的请求路径不经过该节点，仍可继续处理各自用户的请求；故障影响被限制在分片 3 负责的用户组内，而不会扩散为全服中断。

图 52 将故障发生后的两个任务放在同一幅图中：上半部分说明串联依赖如何使单点故障扩散为全局中断；下半部分先用分片隔离把影响限制在分片 3，再由该分片内部的备用副本接管服务。

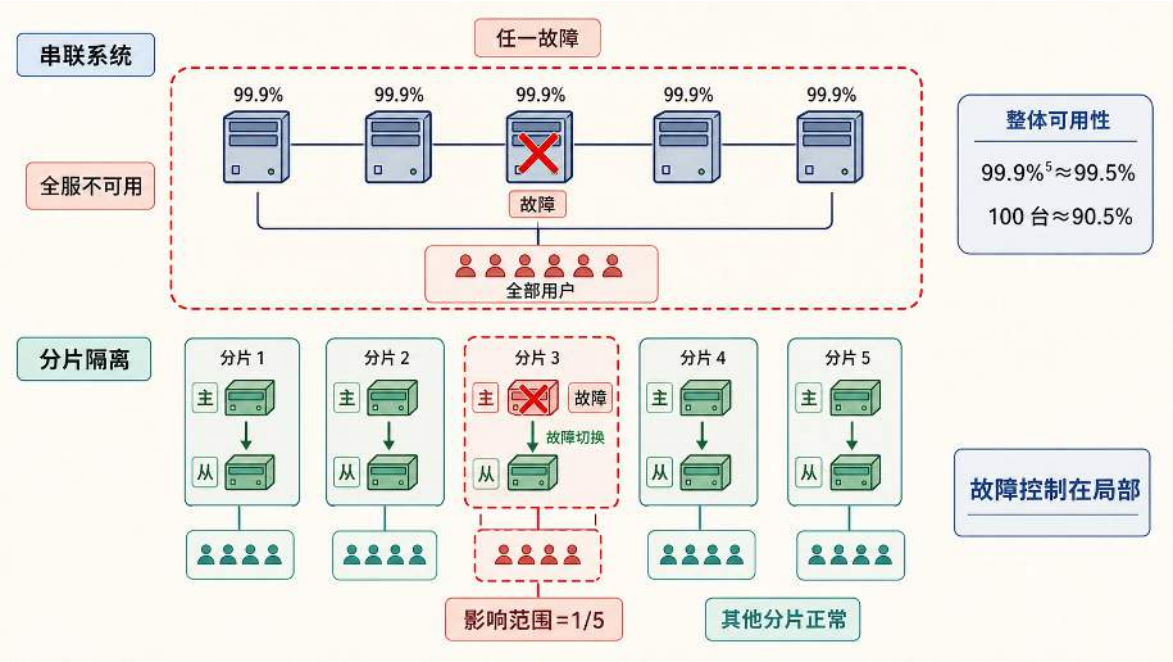


图 52: 无冗余的串联路径会把必经节点故障扩散为全服中断；分片隔离先把影响限制在对应用户组，备用副本再在分片内部接管服务。

分片隔离只完成了第一个任务，并没有恢复发生故障的分片。如果分片 3 只有一个主库且没有备用副本，其他分片虽然仍可运行，分片 3 负责的用户却只能等待主库恢复，无法继续读写数

据。第二个任务是让受影响的服务重新可用，因此系统还要为每个局部单元维护数据足够新、能够接管请求的冗余副本，并在故障发生后完成检测和切换。

冗余能够带来多大收益，也可以先用一个理想化模型估算。假设同一服务有三个故障相互独立的副本，而且任意一个存活副本都能立即、正确地接管服务，那么至少一个副本可用的概率为 $1 - (1 - 99.9\%)^3 = 99.999999\%$ 。即使在这些理想条件下，可用性仍未达到 100%，三个副本仍然可能同时不可用。现实系统中，多个副本可能同时受到机房断电或网络中断的影响；即使仍有副本存活，也可能因复制滞后、故障检测和请求切换而无法立即接管服务，使用户实际感知的可用性进一步低于上述理想值。上述计算因而只能说明并联冗余的数学潜力，并不是三副本状态服务的实际可用性承诺。增加副本只能降低服务完全中断的概率，不能消除故障本身，同时还会带来多倍资源成本，以及副本同步、一致性维护和故障切换等新问题。

多机架构的容错因此不能依赖“故障不会发生”，而要在故障发生后先把影响控制在局部，再为受影响的服务建立继续运行或恢复到正确状态的路径。本节先讨论三类保护机制：主从复制维持可接管的在线副本，备份恢复保留历史时间点用于回退错误操作，共识算法保证协调服务在节点故障后沿同一操作历史继续服务；最后再分析这些机制在一致性、可用性和响应延迟之间需要作出的取舍。

4.4.1 主从复制与故障切换

分片把数据分散到多台机器后，单个分片内部仍可能只有一台数据库提供服务；一旦这台数据库不可用，属于该分片的读写请求仍会被阻塞或失败。为了让同一份数据在原节点故障后仍有副本可以接管，系统可以在一个分片内保存多个持续同步的数据副本，并指定其中一个副本负责产生权威写入。这种结构称为主从复制（Master-Slave Replication）。

主从复制通常包含一个主库（Master）和一个或多个从库（Slave）。主库接收写请求并确定数据变更的先后顺序，从库持续跟随这些变更，形成可供读取或故障接管的数据副本。较新的技术文档也常将二者称为 primary 和 replica，本节代码示例采用后一组命名。从库有两种常见用途：一种是平时不处理业务请求，只在主库故障时接管；另一种是在正常运行时分担一部分读请求。后一种做法把写请求集中到主库，把部分读请求交给从库，称为读写分离。主从复制并不要求一定采用读写分离，系统需要根据读取压力和一致性要求决定是否启用。

无论从库是否承担读请求，主从复制都需要连续解决三个问题：正常请求应当进入哪个副本，主库产生的数据变更怎样传到从库，以及主库失效后怎样把写入入口切换到可接管的从库。图 53 将这三个问题组织为正常请求路径、状态复制路径和故障接管过程，为后面的机制分析提供整体结构。

沿着图中的正常请求路径，写请求统一进入主库，由主库产生该分片的权威数据。如果系统还希望利用从库分担读取压力，可以把登录时读取用户信息、查看配置和刷新排行榜等普通读请求分配给不同从库，而把写请求继续留在主库。这种读写分离可以避免主库同时承担全部查询和写入，应用层或数据库代理可以依据轮询、负载指标或连接池策略选择从库。

请求能够分流的前提，是从库持续获得主库的最新变更，这对应图中的状态复制路径。主库处理写请求时，会把数据修改及其提交顺序写入复制日志；从库接收这些日志，并在本地按

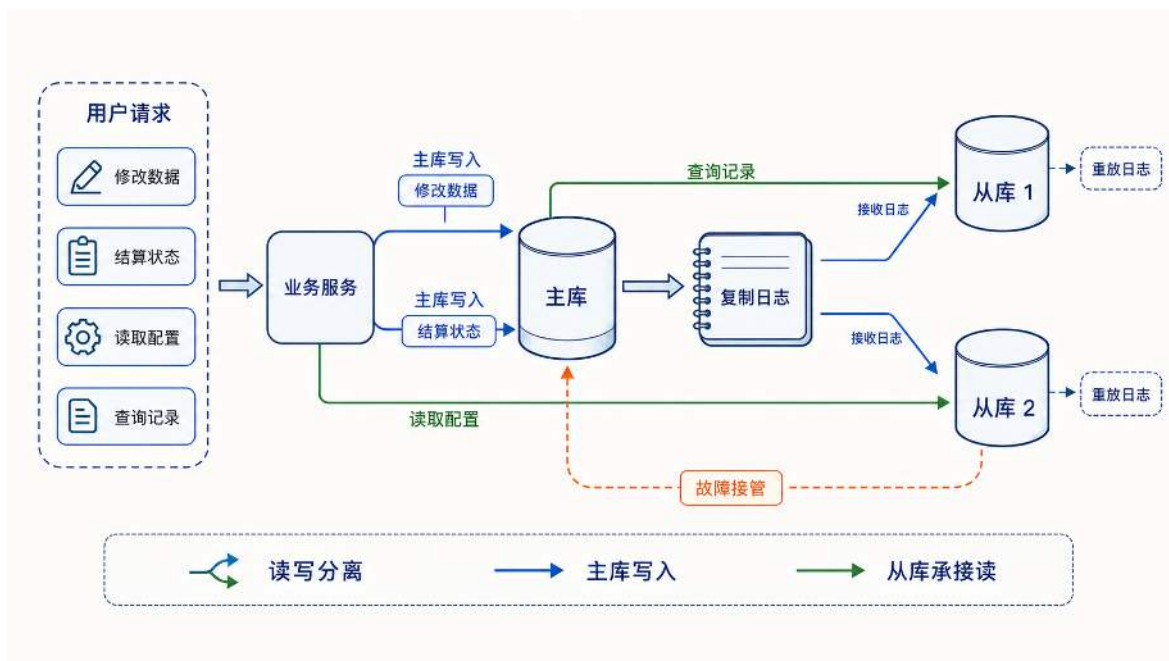


图 53: 主从复制通过请求分流确定主库和从库的正常职责，通过复制日志维持可接管的数据副本，并在主库故障后把写入入口切换到新的主库。

照相同顺序重放，使自身数据逐步追上主库。不同数据库对这类日志有不同命名：MySQL 常用 binary log（二进制日志）记录可复制的数据变更，PostgreSQL 常用 write-ahead log（WAL，预写日志）作为写入恢复和复制的基础。虽然具体格式不同，它们在主从复制中的共同作用都是保留主库的变更顺序，让从库能够重建相同的数据状态。

日志传输和重放都需要时间，因此从库与主库之间通常存在复制延迟。主库已经提交一项写入时，从库可能还没有重放对应日志，暂时保留着写入之前的数据。一旦读请求被分配到这样的从库，读写分离就会进一步带来一致性问题。

以一次写入后立刻读取为例。用户完成写入后，主库已经把更新写入成功；如果界面立刻刷新，而这次读取被分配到尚未重放对应复制日志的从库，用户就可能看到数据仍是旧值。这里的失败点在于：一次写入已经向用户确认成功，随后的读取却没有看到这次写入。这种情况通常称为“读到旧数据”；从一致性的角度看，它破坏的是写后读一致性。

为避免用户在写入成功后立即读到旧数据，系统需要确保随后的读取能够看到这次更新，常见做法有三类。第一，关键读请求直接读主库，例如关键写入后的第一次读取。第二，在一次写入后的短时间内，让同一个用户的相关读请求继续读主库，避免马上读到旧从库。第三，应用层可以关注从库的复制进度：只有当从库已经重放到本次写入之后，才把读请求交给从库。实际业务中，如果不想直接管理复杂的数据库复制进度，也可以把这个判断转换成新旧版本号的比较。例如，用户资料、配置记录或状态都可以带一个随写入递增的版本。可以先把它理解为对象状态的编号：金币余额处于旧状态时是 v41，领取奖励成功后变成 v42。后续读请求如果要求至少读到 v42，从库返回 v41 就说明它还没有重放到这次写入。下面的伪代码展示了这一流程：写入奖励时，主库返回新的资产版本；刷新金币时，读请求先查从库，再用版本号判断是否需要回到主库。

写入后的版本号读写流程

```
1 type Coins struct {
2     Amount int
3     Version int64
4 }
5
6 func ClaimReward(userID int, reward int) int64 {
7     // 主库把金币从 v41 更新到 v42, 并返回新的版本号
8     newVersion := primary.AddCoins(userID, reward)
9     return newVersion
10 }
11
12 func ReadCoins(userID int, minVersion int64) Coins {
13     coins := replica.ReadCoins(userID)
14
15     // 读到 v41, 但本次请求至少需要 v42, 说明从库仍然落后
16     if coins.Version < minVersion {
17         coins = primary.ReadCoins(userID)
18     }
19
20     return coins
21 }
```

这段代码中的 `ClaimReward` 对应写路径:奖励写入主库后,主库返回新的资产版本。`ReadCoins` 对应读路径:它先读从库,因为普通读请求仍然希望由从库承担;随后检查 `coins.Version < minVersion`。这个判断就是写后读一致性的业务表达:如果从库版本低于本次请求要求的最低版本,说明从库仍然落后,读请求需要回到主库。

例如,用户写入前有 100 个单位,资产版本是 `v41`。写入后,主库把数值写成 150,并把版本推进到 `v42`,然后把 `v42` 返回给业务服务。如果用户立刻刷新界面,业务服务会带着“至少读取 `v42`”这个要求访问从库。此时从库如果还停留在 `v41`,返回的可能仍是 100;业务服务发现版本不够,就改读主库,拿到 `v42` 对应的 150。这样,复制延迟仍然存在,但用户不会在写入成功后马上看到旧数据。

上面的伪代码只写了核心逻辑,真实实现还要进一步处理以下两种情况。第一种情况是,改读主库这一步本身也可能失败。当从库版本不够、读请求转而访问主库时,如果主库此刻正好宕机或响应很慢,这次读取就会卡住或报错。因此这一步不能无限期等待,需要设定超时时间并允许重试;如果重试后确认主库已经不可用,这次读取就应当快速失败,向用户返回“稍后再试”之类的提示,而不是让刷新请求一直悬挂在那里。至于主库本身的恢复,则由系统层面的机制接手。第二种情况是,从库的复制延迟可能长时间偏大。前面说过,从库靠重放主库的复制日志跟进数

据，正常情况下这段延迟很短。但如果主库写入太快，或者从库与主库之间的网络出问题，从库重放的进度就会明显落在后面，复制延迟持续偏大。这时几乎每个带版本要求的读都会发现从库版本不够，于是全部转去读主库，从库分担读压力的作用就失效了，主库反而被这些额外的读请求压满。为避免这种情况，实际系统会持续测量各个从库的复制延迟，一旦超过设定阈值就告警并介入处理。对于本来就允许数据短暂陈旧的读取场景，还可以在延迟过高时主动放宽要求，接受从库返回的旧版本数据，用可控的陈旧换取主库的稳定。

前面的读写与复制过程发生在主库正常运行期间；主库不可用后，故障接管过程才会启动。系统需要从已有从库中选择一个提升为新主库，把后续写请求切换到接管节点，并让其他从库改为跟随新的主库。这组动作称为故障转移（Failover），它把持续同步的数据副本转变为新的写入口，使该分片能够恢复读写服务。

故障转移的第一项困难，是无法仅凭“联系不上主库”断定它已经停止工作。主库可能已经宕机，也可能仍在运行，只是与监控组件和从库之间的网络已经中断。后一种节点仍在运行、节点组之间却无法通信的状态，称为网络分区（Network Partition）。

如果此时直接提升从库，而旧主库仍能接收部分客户端的写入，系统中就会同时出现两个主库，并形成两份相互冲突的数据。因此，新主库开始写入之前，必须先确认旧主库已经失去写入资格。系统通常通过连续健康检查判断主库是否长时间无响应，并用带有效期的主库身份限制其写入权限；身份到期后，旧主库必须停止写入。若旧主库仍可访问，还可以通过关闭其写入口或隔离节点来强制执行这一限制，这类措施统称为隔离（Fencing）。

在此基础上，一次完整的接管包含四项动作：确认旧主库不能继续写入，选择数据进度合适的从库，将其提升为新主库，并把数据库代理、服务发现或客户端连接切换到新的写入口。其他从库随后改为跟随新主库；旧主库恢复后，也必须先追赶新主库的数据或重新初始化，不能直接恢复写入。PostgreSQL 的 Patroni、pg_auto_failover 和 MySQL 的 Orchestrator 等高可用组件，可以把这些动作组织成自动执行的故障转移流程。²⁷

接管完成后，还要判断最近的写入是否保留下来。如果主库在本地提交后立即向客户端返回成功，再由从库随后复制，这种方式称为异步复制；主库突然故障时，尚未到达接管节点的写入可能丢失。如果主库必须等至少一个指定从库确认收到写入后才返回成功，这种方式称为同步复制；从已确认写入的副本中选择新主库，可以降低已确认写入丢失的风险，但从库响应变慢或不可用时，主库的写入也会等待甚至暂停。因此，系统既要选择复制进度足够新的接管节点，也要根据业务能够容忍的数据丢失程度选择复制方式。

除了确认最近写入能否保留，系统还要关注故障后服务需要多长时间才能恢复。恢复时间包括发现主库失效、提升从库以及切换请求入口所需的时间。自动化不能消除这些步骤，却能减少人工确认、执行命令和修改连接配置造成的额外等待，使接管过程更快且更稳定。

主从复制由此解决的是“当前主库不能继续工作”时如何利用在线副本接管服务。然而，在线副本保存的仍是数据库的当前状态。如果清理脚本误删了用户数据，或者程序缺陷写入了错误余额，数据库会把它们当作合法写入提交，从库也会照常复制。结果是所有副本都进入同一个错

²⁷ 参见 <https://patroni.readthedocs.io/en/latest/faq.html#automatic-failover>、<https://pg-auto-failover.readthedocs.io/> 和 <https://github.com/openark/orchestrator>。

误状态，提升任何从库都无法找回原来的数据。恢复这类错误，需要保存事故发生前的历史状态。

4.4.2 备份与恢复演练

备份把数据库的历史状态保存在独立存储中，使系统能够从错误写入传播之前重新开始恢复。最基础的做法是全量备份（Full Backup），即定期保存整个数据库的数据与结构。例如，系统可以每天凌晨生成一份完整副本，并存入独立的备份服务器或云存储服务。加载全量备份可以直接回到备份生成时的状态，但备份文件较大，生成、传输和恢复都需要较多时间，因此很难频繁执行。

为了在两次全量备份之间保存更多恢复点，系统还可以只记录发生变化的数据，这称为增量备份（Incremental Backup）。它能够减少单次备份的数据量，但恢复时需要组合全量备份和后续增量备份，而且离散的备份点仍未必足够接近误操作发生的时刻。

更细粒度的做法，是保存一份完整的数据基线，并连续记录此后发生的写入。以 PostgreSQL 为例，完整的数据基线称为基础备份（Base Backup），写入历史记录在 WAL 中并持续归档到独立存储。恢复时，系统先加载基础备份，再按顺序重放 WAL，并在指定时刻之前停止。由此，数据库可以恢复到选定的时间点，这种能力称为时间点恢复（Point-in-Time Recovery, PITR）。

假设某个数据库分片在凌晨 0 点完成基础备份，此后持续归档 WAL；下午 3 点 20 分，一条错误脚本删除了部分用户的配置数据。如果归档日志完整，恢复程序可以加载凌晨的基础备份，再把 WAL 重放到 3 点 19 分 59 秒，使数据库停在误删发生之前。

一次恢复不能只用“是否成功”来判断，还必须得到两个结果：数据能够回到哪个时刻，以及这份数据何时能够重新对外服务。对于前面的误删场景，恢复程序需要先把日志重放停在错误写入之前，再对恢复后的数据进行校验并切换服务。图 54 将这两个结果放在同一次恢复中：数据时间线用于确定日志在哪里停止，恢复流程线用于确定恢复结果何时能够接替线上服务。

沿数据时间线看，基础备份只提供恢复起点，连续归档的 WAL 才能把数据库逐步推进到误删发生之前。当日志重放在误删记录之前停止时，错误操作就不会进入恢复结果；恢复点与误删时刻之间的距离，就是实际数据丢失窗口。业务能够接受的最大数据丢失窗口称为恢复点目标（Recovery Point Objective, RPO），它会约束 WAL 归档的延迟与完整性。

沿恢复流程线看，得到一个可恢复的数据时间点并不意味着服务已经恢复。系统还要选择并校验备份，在隔离环境中加载基础备份、重放 WAL，对结果进行只读校验，最后才能切换服务。从发现错误到重新提供服务的总耗时，应当不超过恢复时间目标（Recovery Time Objective, RTO）。因此，较小的 RPO 只能说明可恢复的数据更接近事故时刻，并不能保证恢复过程同样迅速；RPO 与 RTO 分别约束数据损失和服务中断，不能相互替代。

仅仅在备份存储中看到基础备份和 WAL 文件，还不能证明这些文件真的能够恢复数据库。基础备份可能已经损坏，WAL 归档中也可能缺少一段日志；即使数据库能够启动，恢复后的用户余额、订单记录等关键数据也可能不完整。因此，系统需要定期在隔离环境中实际执行恢复：加载基础备份，把 WAL 重放到选定时刻，以只读方式启动数据库，并检查关键业务数据是否正确。这类恢复演练既不会覆盖线上分片，又能验证数据库能否恢复到指定时刻、整个恢复过程能否在规定时间内完成。下面的伪代码给出了其中的核心检查。

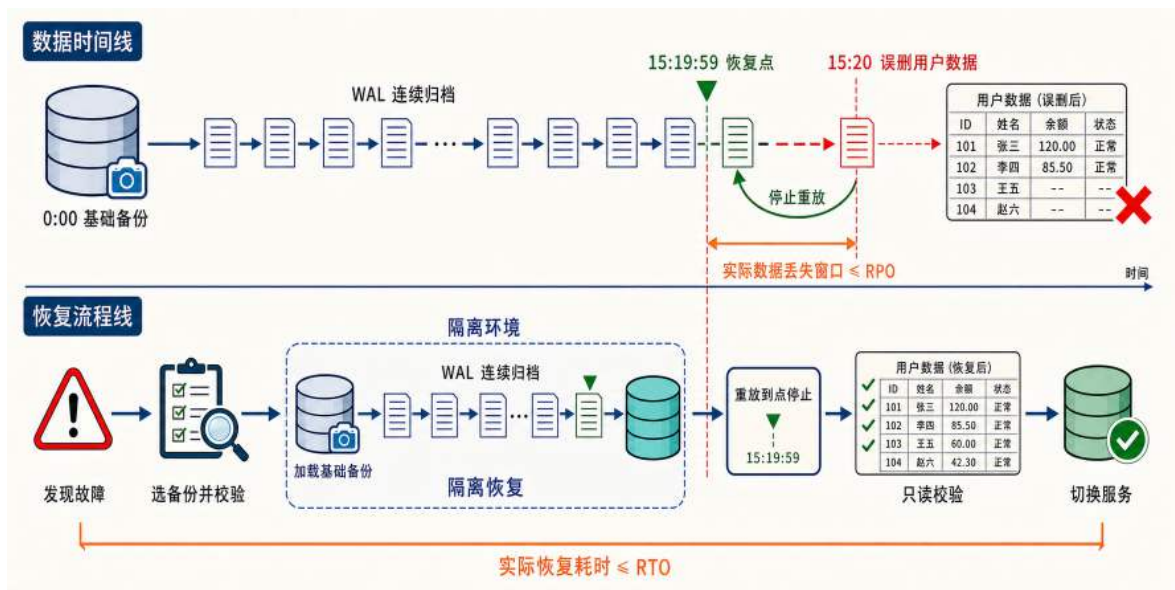


图 54: PITR 将恢复点与恢复过程分开度量：基础备份和连续 WAL 决定数据能够恢复到哪里，备份加载、日志重放、数据校验和服务切换决定数据何时能够重新提供服务。

基于 PITR 的恢复与校验流程

```

1 type RecoveryReport struct {
2     TargetTime time.Time
3     Elapsed    time.Duration
4     Verified   bool
5 }
6
7 func RestoreAndVerify(shardID string, targetTime time.Time) RecoveryReport {
8     startedAt := time.Now()
9
10    // 选择目标时刻之前最近的基础备份
11    base := backupCatalog.LatestBaseBefore(shardID, targetTime)
12
13    // WAL 链存在缺口时，数据库无法连续推进到目标时刻
14    if !walArchive.Continuous(base.EndLSN, targetTime) {
15        return RecoveryReport{TargetTime: targetTime, Verified: false}
16    }
17
18    // 在隔离环境恢复，不覆盖线上分片
19    candidate := recoveryEnv.RestoreBaseBackup(base)
20    candidate.ReplayWALUntil(targetTime)
21    candidate.StartReadOnly()
22
23    // 校验用户资产、订单数量等关键业务约束
24    verified := candidate.VerifyBusinessRules()
25    return RecoveryReport{
26        TargetTime: targetTime,
27        Elapsed:     time.Since(startedAt),
28        Verified:    verified,
29    }
30 }

```

伪代码先根据 `targetTime` 找到目标时刻之前最近的基础备份，再检查从这份备份到目标时刻之间的 WAL 是否完整。只要中间缺少一段日志，数据库就无法连续推进到目标时刻，本次演练应当直接判定失败。日志完整时，系统才会在隔离环境中加载基础备份、重放 WAL，并以只读方式启动恢复实例。最后的业务校验会检查用户资产、订单数量等关键数据；只有数据库能够启动且这些数据正确，`Verified` 才会记录为真。`Elapsed` 记录上述恢复和校验用了多长时间，但它还不包含数据补写和服务切换，因此不是完整的服务中断时间。

演练验证到这里就可以结束，因为这个恢复实例不会接收线上请求。真实事故的目标则是让恢复实例接替原来的数据库。系统通常先暂停受影响分片的新写入，避免恢复期间继续产生变化；随后完成恢复，补回仍应保留的数据并再次校验，最后才把业务连接切换到恢复实例。

恢复操作并不能只撤销那一条错误写入。恢复到误删之前，确实能够去掉错误操作，但恢复点之后发生的正确充值、交易和状态更新也不会出现在恢复结果中。系统需要根据支付流水、订单记录或消息日志找出这些已经生效的业务操作，并把它们重新写入恢复后的数据库，这个过程称为数据补偿。恢复点距离事故越远，需要重新核对和补写的数据通常越多。

因此，缩短数据丢失窗口，需要让 WAL 更及时、完整地归档；缩短服务中断时间，则需要加快备份加载、日志重放、数据校验和连接切换。恢复演练的作用，就是在事故发生之前检查这两项要求能否真正达到，而不是等到线上数据出错后再第一次执行恢复流程。

至此，数据库分片面对三类问题分别有了对应办法：主库故障时由在线从库接管，数据被误删或误改时用备份和 WAL 回到历史状态，恢复流程本身则通过演练提前验证。这些办法保护的只是一个数据库分片的数据和服务，但跨服协调服务还有不同的问题。当协调服务部署多个节点时，每个节点都可能收到新的比赛结果；系统必须保证这些节点对“由谁接收写入”和“哪些结果已经生效”作出相同判断。如果网络中断后两个节点分别把不同结果当成正式结果，后续即使恢复数据库，也无法判断应该保留哪一份操作历史。让多个节点在故障和网络延迟下仍确认唯一能够提交写入的节点，并按同一顺序确认操作，称为分布式共识。Raft 是解决这类问题的经典算法，下面从它为何采用多数派确认开始分析。

4.4.3 从全写到多数派：Raft 共识

假设擂台协调服务部署了三个副本，一场比赛的裁决结果需要被复制到这些副本中，才能在当前节点故障后由其他节点继续处理。最直接的策略是等待三个副本全部写入成功，再向应用服务器确裁决已经成立，这种策略称为“全写”（Write-All）。全写可以使所有副本立即保存相同记录，却把每个副本都变成了写入成功的必要条件：只要一个副本响应缓慢或暂时失联，新的比赛裁决就无法提交。复制本来是为了让部分节点故障时仍能继续服务，全写却使任意单个副本故障都阻塞整个协调服务，因此不能直接作为容错集群的提交规则。

Raft 不再要求全部副本都写入成功，而是把超过半数节点确认作为提交条件，这称为多数派提交。这样，少数节点暂时不可用时，集群仍有继续处理请求的可能；半数及以下的节点则不能独自提交新的结果。

多数派提交只规定一项操作在什么条件下成为正式结果，还不足以构成完整的共识算法。Raft 还需要通过领导者选举确定当前由谁接收写入，通过日志复制让各副本按相同顺序记录操

作，并通过安全性约束保证故障后的新领导者能够继承已经提交的历史。下面依次介绍这三项机制。

Leader 与任期：先确定谁能接收写入 Raft 集群中的节点有三种角色：跟随者 (Follower)、候选者 (Candidate) 和领导者 (Leader)。正常运行时，Leader 负责接收客户端写请求并组织复制；Follower 只响应 Leader 或 Candidate 发来的消息；Candidate 是选举期间的临时角色。对跨服对战来说，挑战请求统一交给当前 Leader，避免多个节点同时决定比赛结果。

Raft 用任期 (Term) 标识一轮选举及其产生的领导关系。任期编号只会递增，可以把它理解为集群内部的逻辑时钟：它不表示现实时间经过了多久，而是用来判断一条消息或一个 Leader 是否已经过期。每条选举和复制消息都会携带发送方的任期；节点发现更高任期后，会更新自己的任期并转为 Follower，收到任期较低的请求则直接拒绝。一个任期可能选出一位 Leader，也可能因选票分散而没有选出 Leader，但同一任期内最多只能有一位节点取得多数票。

Leader 通过周期性心跳表明自己仍在工作，Follower 每次收到有效心跳后都会重新启动选举计时。如果等待时间超过选举超时，Follower 无法判断 Leader 是已经宕机还是网络暂时中断，只能认为当前 Leader 已经无法继续组织集群，并转为 Candidate 发起选举。为避免多个 Follower 在同一时刻发起选举并瓜分选票，每个节点会从一个时间区间内随机选择自己的选举超时；通常最先超时的节点能够在其他节点超时之前取得选票并发送新的心跳。选举超时过短，正常的网络延迟也可能频繁触发选举；选举超时过长，Leader 故障后的接管又会变慢，因此需要根据网络延迟和故障恢复目标进行配置。

一个节点需要在选举超时、收到心跳、收到投票请求和收到投票回复时更新自身状态。以下伪代码把这四类事件放在同一流程中，用于说明节点如何推进任期、发起选举并确认 Leader。其中，`CandidateLogAtLeastAsNew` 只表示候选者的日志新旧检查，具体比较依据将在后面的日志复制与安全性约束中说明。

Raft 任期、心跳与领导者选举流程

```
1 var role = Follower
2 var currentTerm int64 = 0
3 var votedFor = noVote
4 var grantedVotes = NewVoteSet()
5
6 // 收到更高任期时，承认新的选举轮次并退回 Follower
7 func observeHigherTerm(term int64) {
8     if term > currentTerm {
9         currentTerm = term
10        votedFor = noVote
11        role = Follower
12        Persist(currentTerm, votedFor)
13    }
```



```

14 }
15
16 // 非 Leader 节点等待心跳超时后发起一轮新选举
17 func onElectionTimeout() {
18     if role == Leader {
19         return
20     }
21     role = Candidate
22     currentTerm++
23     votedFor = selfID
24     grantedVotes = NewVoteSet(selfID)
25     Persist(currentTerm, votedFor)
26     ResetElectionTimer(RandomTimeout())
27     ParallelRequestVotes(currentTerm, LastLogInfo())
28 }
29
30 // Leader 周期性发送当前任期的心跳
31 func onHeartbeatTick() {
32     if role == Leader {
33         BroadcastHeartbeat(currentTerm)
34     }
35 }
36
37 // 接受有效心跳，并推迟下一次选举
38 func onHeartbeat(term int64) bool {
39     if term < currentTerm {
40         return false
41     }
42     observeHigherTerm(term)
43     role = Follower
44     ResetElectionTimer(RandomTimeout())
45     return true
46 }
47
48 // 同一任期只投一票，并拒绝日志较旧的候选者
49 func onVoteRequest(req VoteRequest) bool {
50     if req.Term < currentTerm {
51         return false

```

```

52     }
53     observeHigherTerm(req.Term)
54
55     canVote := votedFor == noVote ||
56         votedFor == req.CandidateID
57     if canVote && CandidateLogAtLeastAsNew(req) {
58         votedFor = req.CandidateID
59         Persist(currentTerm, votedFor)
60         ResetElectionTimer(RandomTimeout())
61         return true
62     }
63     return false
64 }
65
66 // Candidate 收集本任期的投票，超过半数后成为 Leader
67 func onVoteReply(reply VoteReply) {
68     if reply.Term > currentTerm {
69         observeHigherTerm(reply.Term)
70         return
71     }
72     if role != Candidate || reply.Term != currentTerm {
73         return
74     }
75     if !reply.Granted {
76         return
77     }
78
79     grantedVotes.Add(reply.From)
80     if grantedVotes.Size() > nodeCount/2 {
81         role = Leader
82         BroadcastHeartbeat(currentTerm)
83     }
84 }

```

任期并不是由某个中心节点统一分配的。节点在选举超时后把自己的 `currentTerm` 加一，并把新任期写入投票请求；其他节点收到更高任期后更新本地状态，因此较新的任期会随消息逐步传播到集群中。任期较低的 Leader 或 Candidate 由此会被识别为过期节点。

心跳也不会创建新任期。Candidate 取得多数票后立即发送当前任期的心跳，并在担任 Leader 期间周期性发送；Follower 接受有效心跳后重置随机选举计时器。只要心跳持续到达，Follower

就不会发起新选举；心跳中断并超过选举超时后，新的 Candidate 才会把任期推进一轮。

如果选票分散，没有 Candidate 能够进入成为 Leader 的分支，节点会在下一次超时后以更高任期重试。伪代码中的 `Persist` 还要求节点在投票前保存当前任期和投票对象，使节点重启后仍然遵守“一任期一票”。至此，角色转换、任期、心跳和投票共同组成了一次完整的 Leader 选举。

沿着这段流程，可以进一步分析网络分区时的角色变化。原 Leader 如果与多数节点失去联系，包含多数节点的一侧可以通过新一轮选举产生任期更高的 Leader。旧 Leader 可能暂时仍认为自己负责写入，甚至继续接收客户端请求，但它无法取得多数节点确认，因此这些请求不能提交，也不能作为成功结果返回。网络恢复后，旧 Leader 从消息中看到更高任期便转为 Follower；它在隔离期间产生的未提交记录不会进入集群的正式历史，后续由日志复制过程清理。

日志复制：把写请求变成可恢复的历史 Leader 选出后，外部写请求先写成日志，再由状态机执行。日志才是集群中的操作历史，状态机只是按日志顺序回放之后得到的结果。只要多个节点持有相同日志，并交给确定性的状态机按相同顺序执行，它们就会得到相同状态。这里的确定性是指：初始状态和输入相同，执行结果也相同，不能在执行时临时读取本机时间或生成随机结果。故障恢复只需补齐缺失日志并按顺序回放，不再需要比较两个数据库中哪一份数据更新。

以一次擂台挑战为例，Leader 收到请求后，先把“挑战者、被挑战者、裁决输入和待提交动作”封装为日志条目，追加到本地日志。随后 Leader 将这条日志复制给 Follower。Follower 接收日志时会检查前一条日志的位置和任期是否与自身日志匹配；如果不匹配，就拒绝本次追加，Leader 再回退到更早的位置重试，直到两边日志前缀重新对齐。这项检查用于发现并修复 Follower 与 Leader 之间不一致的日志前缀。不同节点上尚未提交的日志可能暂时冲突，但不能把这些冲突条目同时当作已经提交的操作历史；Leader 会在后续复制中删除或覆盖 Follower 上与权威前缀不一致的未提交条目。

当 Leader 收到多数节点的成功确认后，这条由当前 Leader 在本任期追加的日志被标记为已提交。在擂台场景中，已提交意味着 Leader 可以把裁决结果应用到本地状态机：原子写入 `match_results` 和 `outbox`。日志条目中应带有已经确定的裁决结果和任务标识，不能在各副本执行时重新随机生成。随后 Leader 通过后续心跳把提交位置通知给其他节点，Follower 也按同一顺序应用日志。即使某个 Follower 暂时失联，Leader 仍可在多数节点确认后继续服务；该 Follower 恢复后再通过日志同步补齐缺失记录。各副本都会得到相同的待发任务，但只有当前 Leader 运行 Dispatcher。Leader 切换后，未完成任务可能再次发送，目标服务器仍要用 `match_id` 做幂等检查。

领导者选举和日志复制并不是两条彼此独立的机制，它们会在一次故障切换中连续发生。如图 55 所示，集群由节点 A、B、C 组成，初始 Leader A 失联后，B 参与新一轮选举；日志条目 3 表示新 Leader 接管后收到的一次写操作。节点 B 不能直接接管写入，而要先通过日志新旧检查，并与节点 C 组成投票多数派，成为新任期的 Leader。随后，写请求形成日志条目 3；即使节点 A 仍然延迟，B 和 C 的确认已经构成提交多数派，这条日志仍可提交并应用到状态机。这次三节点故障切换先后形成两个多数派：投票多数派确定新的唯一写入口，提交多数派确认新

的操作已经进入集群历史。二者都要求超过半数节点参与，再结合投票时的日志新旧限制，已经提交的历史才能由故障后的 Leader 继续继承。

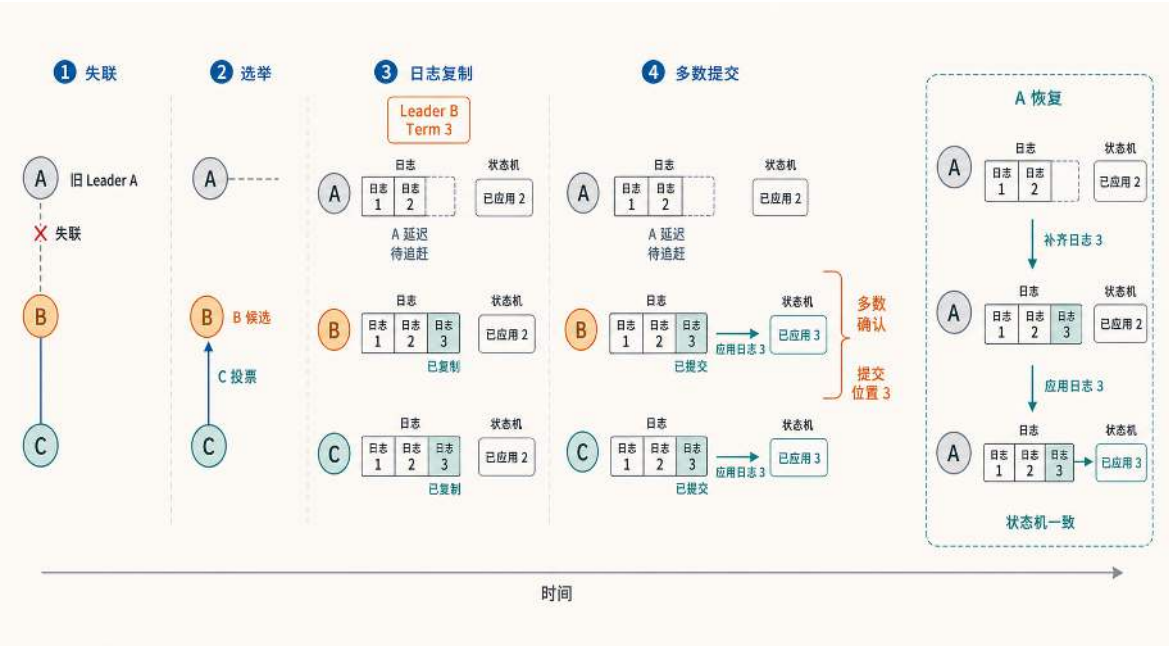


图 55: Raft 中日志复制、提交和应用是三个连续状态：B、C 先复制日志 3，再由多数派确认提交并应用；延迟的节点 A 恢复后才补齐并应用该日志，三个状态机最终一致。

前面借助三节点集群，说明了故障切换会先后经过投票多数派和提交多数派两个环节。下面换一个五节点集群，考察 Leader 崩溃、集群重新选举时，一条已经提交的日志会经历什么。

协调服务由 N1 至 N5 五个 Raft 节点组成，N1 是当前 Leader。甲挑战乙后，N1 生成“甲获胜、乙失败”的裁决，把它写成第 7 条日志，记为 E7，并复制给其他节点。N2 和 N3 回复成功后，N1、N2、N3 三个节点持有 E7，达到五节点集群所需的三节点多数派；N1 因而提交 E7，并把裁决应用到状态机。现在 N1 崩溃，集群要重新选举。问题是：新选出的 Leader 会不会正好是一个没有 E7 的节点，从而让这条已经提交的裁决从集群里消失？

E7 原本保存在 N1、N2、N3 三个节点上，现在 N1 下线，仍然在线并持有 E7 的是 N2 和 N3；还能参与投票的节点是 N2、N3、N4、N5。任何候选者都必须集齐三票才能当选，而从这四个节点里凑出三票，无论如何都会包含 N2 或 N3 中的至少一个——四个节点里选三个，不可能把 N2 和 N3 同时排除在外。换句话说，任何能够当选的新 Leader，它的票源里一定有一个持有 E7 的节点。

这一个节点就足以挡住日志落后的候选者。N2 和 N3 持有 E7，按照投票时的日志新旧规则，它们只会把票投给日志不比自己旧的候选者。N4、N5 没有 E7，日志落在后面，拿不到 N2 或 N3 的票，也就集不齐三票。结合多数派交集和日志新旧检查，缺少 E7 的候选者无法取得足够选票。最终当选的 Leader 必须包含 E7，已经提交的裁决不会因为换了 Leader 而消失。

同一条交集规则也限制了网络分区后的写入范围。五节点集群中，合法 Leader 至少需要三票。网络分区后，只有包含至少三个节点并取得多数确认的节点组能够继续提交；少数派节点组

即使还保留着旧 Leader，也不能提交新的裁决。因此，同一场挑战不会在不同节点组中产生两份合法结果。这种限制用于防止脑裂（split-brain），即不同节点组分别接受写入并形成冲突历史。

把这套机制应用到跨服擂台后，原来的单台协调服务器变成一个复制同一操作历史的协调服务器集群。应用服务器只把挑战请求发送给当前 Leader。裁决操作复制到多数节点并提交后，各副本按顺序应用同一条日志；当前 Leader 的 Dispatcher 再派发 `outbox` 中的任务。Leader 故障时，保存了已提交历史的节点重新选出负责人，新的写请求沿同一条历史继续执行。图 56 只展开当前 Leader 的写入和派发路径，Follower 上相同的本地状态机没有重复画出。右侧给出 Leader 故障后的重新选举。

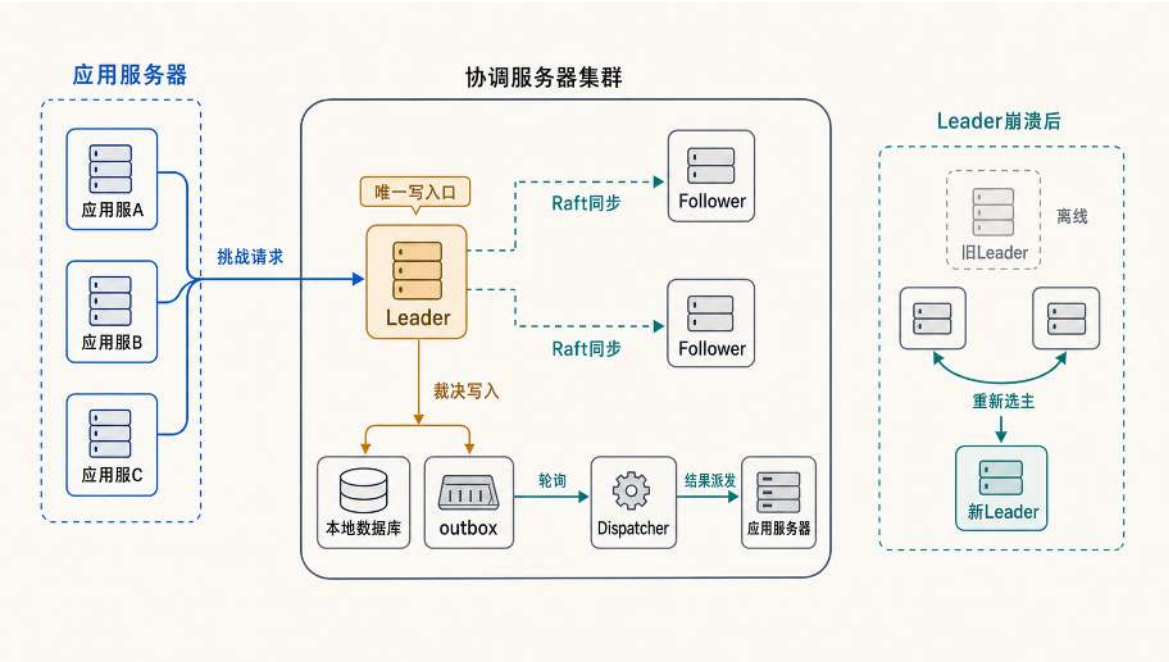


图 56: Raft 将单点裁决扩展为可接管的协调服务器集群：Leader 接收挑战请求并复制裁决日志，提交后派发结果；Follower 保存同一日志，故障后由新 Leader 接续处理。

协调服务器扩展为 Raft 集群后，事务性发件箱仍然从正式裁决提交之后开始工作。Raft 先通过多数派确认哪一条裁决已经进入集群的正式历史。各副本应用这条日志时，各自在本地事务中写入比赛结果和 `outbox`；当前 Leader 的 Dispatcher 再发送待发任务。Leader 切换可能带来重复发送，但不会丢掉已经写入日志的任务，接收端用 `match_id` 去重。

工程取舍：节点数、读路径与恢复成本 N 个节点的 Raft 集群最多可以在 $\lfloor \frac{N-1}{2} \rfloor$ 个节点故障后继续提交日志，前提是剩余节点能够互相通信并连接客户端。从奇数规模增加到相邻的偶数规模时，这个容错上限不会提高，但提交所需的多数派规模会增加。因此，Raft 集群通常采用奇数节点，避免只增加确认范围而没有提高可容忍的故障数。节点数量确定后，还要检查各节点是否位于独立的故障域，以及多数派之间的通信延迟能否满足写入要求。

读请求也需要区分语义。裁决相关查询必须执行线性一致读，因为旧状态可能造成重复裁决或遗漏已完成战斗。仅把请求发给 Leader 还不够，Leader 在返回结果前还要确认自己仍能联系

多数派；实现中常用读索引（ReadIndex）或多数派心跳，租约读则依赖对时钟偏差的限制。排名浏览、历史记录这类对实时性要求较低的查询，可以在明确接受短暂滞后的前提下从 Follower 分流。这种分流能减轻 Leader 压力，但可能放弃最新视图，不能用于需要立即反映裁决结果的路径。

Raft 日志还需要配合持久化和压缩机制。每个节点通常以 WAL 的形式顺序追加日志，崩溃重启后先读取日志恢复状态。日志持续增长后，系统会定期生成状态机快照（Snapshot），删除快照之前的旧日志。重启或新节点加入时，只需加载最近快照，再补齐快照之后的少量日志。这类工程机制不改变 Raft 的安全性原则，但决定了集群故障恢复和新节点追赶的实际成本。

经过这一层区分，容错机制形成三层保护：**主从复制保护数据分片的在线副本，备份保护历史时间点的数据回退，Raft 保护需要强一致操作历史的权威服务**。三者分层协作，不可互替。2PC、事务性发件箱和 Raft 也分别处理不同的跨节点问题：2PC 要求一次业务操作落在多个分片上的不同写入全部提交或全部回滚；事务性发件箱负责在本地事实成立后把结果可靠传播到其他服务器；Raft 则把同一条操作日志复制到多个副本，并确定哪一段历史已经得到多数节点确认。Raft 用多数派换来强一致历史，因此每次写入都要多节点通信。这正是一致性与性能取舍的来源，也是下一小节讨论的重点。

4.4.4 CAP 定理与一致性取舍

前面的故障转移和 Raft 多数派都在处理网络分区造成的后果。网络分区发生后，仍在运行的节点会被分成无法交换信息的节点组；如果不同节点组同时收到针对同一份状态的写请求，系统应当如何响应？图 57 用节点 A、B 与节点 C 之间的通信中断建立具体场景，并给出两种处理路径。

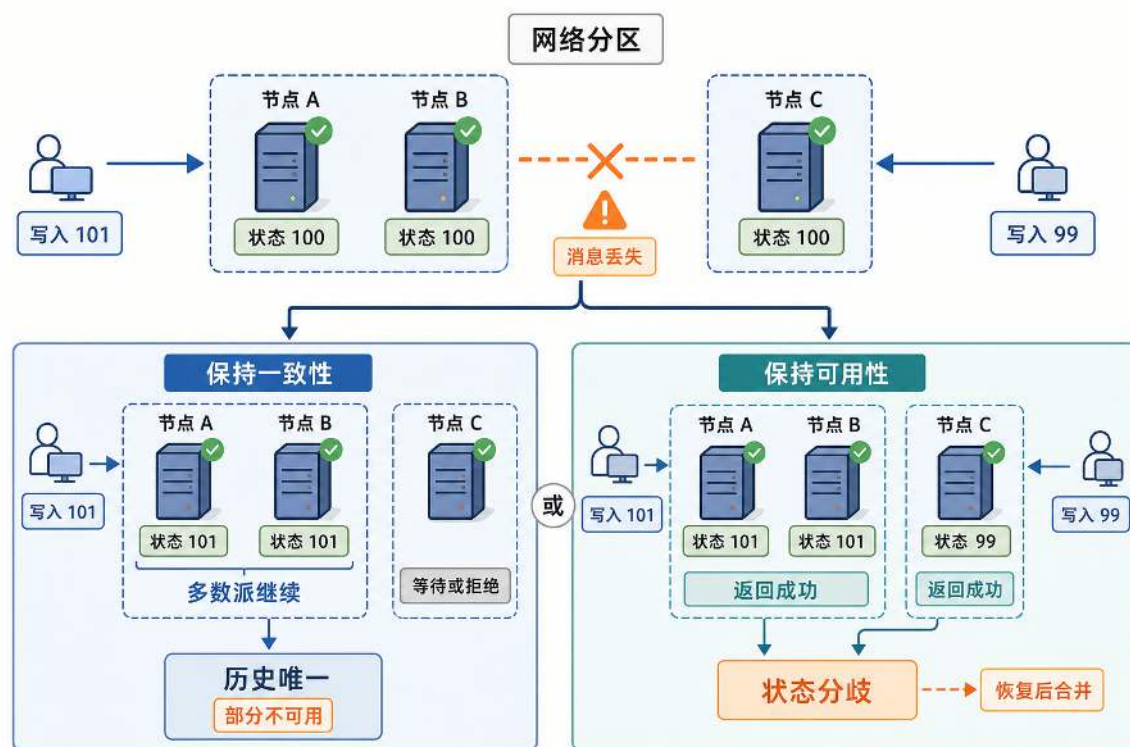


图 57: 网络分区把仍在运行的节点分成无法通信的两组: 保持一致性需要让至少一个节点组等待或拒绝请求; 保持可用性则允许两个节点组继续响应, 但必须接受暂时的状态分歧。

图的上半部分中, 节点 A、B 和 C 在分区前都保存状态 100; 分区后, 节点 A、B 所在的节点组收到写入 101, 节点 C 所在的节点组收到写入 99, 但两个节点组无法交换这两次更新。左下分支选择保持一致性: 节点 A、B 组成的多数派继续处理写入, 节点 C 则等待或拒绝请求, 因而不会把写入 99 确认为另一条成功历史; 代价是节点 C 所在的节点组暂时无法完成该请求。右下分支选择保持可用性: 两个节点组都立即返回成功, 节点 A、B 保存状态 101, 节点 C 保存状态 99; 系统继续响应了请求, 却产生了需要在网络恢复后处理的状态分歧。

这两种处理路径对应 **CAP 定理**在分区期间的核心限制。一致性 (Consistency) 要求各节点对写入顺序和最新状态给出相同的观察结果; 可用性 (Availability) 要求发送到仍在运行节点的请求能够得到非错误响应; 分区容忍性 (Partition Tolerance) 则表示系统的设计允许节点组之间的消息持续丢失或延迟, 并且仍有明确的处理策略。对跨节点保存共享状态的系统来说, 网络分区是已经发生的故障约束, 而不是日常运行时可以任意舍弃的选项。分区期间, 系统若坚持一致性, 就必须牺牲部分可用性; 若要让各节点组都继续响应, 就必须暂时放宽立即一致的要求, 而不是把三个属性任意 “选两个”。

所谓的强一致, 强调一次写入被确认后, 后续操作按照同一全局顺序观察数据, 不会在其他节点继续读到这次写入之前的旧值; 严格描述这一性质时, 通常使用线性一致性 (Linearizability)。弱一致是允许节点在一段时间内观察到不同版本的统称, 并不承诺写入立即对所有读取可见。最

终一致性是其中一种常见保证：如果不再产生新的更新，各节点经过传播后最终会收敛到相同状态。

即使网络没有分区，强一致也意味着更多通信和等待，延迟会因此上升。CAP 的延伸模型 **PACELC** 强调的正是这一点：有分区时看一致性与可用性的取舍，没有分区时仍要看一致性与延迟的取舍。²⁸

回到本章讨论过的几条路径，它们对应的是不同位置上的取舍。2PC 把多个数据库提交点绑定成一个原子提交结果，也把两轮通信、锁等待和协调者故障风险放进请求主链路。它本身解决的是共同提交或共同回滚；事务的隔离级别和客户端观察到的读写顺序，还要由参与数据库及上层协议另外保证。事务性发件箱先把业务事实收束到本地事务，再把跨服传播放到异步阶段，用幂等重试换取最终收敛。Raft 则用于保护协调服务器这类权威服务的操作历史：多数派确认之后，即使 Leader 故障，已提交的裁决记录也不会丢失。这三种机制并非简单的强弱一致性的排序，而是把其成本放在了不同位置：请求主链路、本地事务边界，或权威服务的复制日志。

架构选择的判断标准不是方案本身的一致性强度，而是每条业务链路对延迟、短暂不一致和错误的容忍程度。跨服交易里金币和物品的转移必须形成不可拆分、对外一致的结果，排行榜和一般通知可以允许几秒的延迟收敛，运营公告甚至可以接受分钟级更新。根据业务链路选择一致性强度，而不是把所有链路都拉到最强一致，是分布式系统设计的核心判断标准。

4.5 小结：均衡、协同与容错

本章讨论了应用服务器从单机走向多机后出现的核心变化：原来集中在一台机器上的对象归属、结果提交和故障恢复，被拆到多个实例和多条网络路径中。多机扩展不是简单增加服务器数量，而是要回答三类问题：对象如何分配到合适实例，多个实例怎样共同完成一次操作，节点失败后系统如何恢复到可解释的状态。

均衡解决的是对象归属和负载分摊问题。职责拆分让网关接入、业务处理、消息分发、负载监测和数据访问等功能能够分别演进；对象分片则把用户数据、计算任务和在线连接分散到多个同类实例上。分片键决定对象按什么字段归属，范围分片、哈希分片和一致性哈希决定归属如何映射到实例；分片路由、负载监测、健康实例选择、扩缩容和热迁移，则把对象归属变成运行时可调整的调度机制。

协同解决的是对象分开之后如何共同完成一次业务操作。通知类、查询类和事务类跨服交互，对一致性的要求不同，因此不能使用同一种机制处理所有链路。通知类事件要先区分尽力而为传播与可靠送达：Pub/Sub 适合在线节点的低延迟即时通知，Gossip 适合高概率扩散，需要严格送达时还必须增加持久化、确认或重放；查询类视图由各服上报异步汇总而成，在上报、网络和汇总周期均有上限时允许有界陈旧；事务类操作则或用 2PC 让多个提交点原子成立，或用事务

²⁸CAP 的形式化证明见 Seth Gilbert and Nancy Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”, ACM SIGACT News, 2002; PACELC 见 Daniel J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story”, IEEE Computer, 2012, <https://www.cs.umd.edu/~abadi/papers/abadi-pacelc.pdf>。

性发件箱把本地事务和跨服传播拆开，用幂等重试换取最终收敛。跨服事务案例说明，协调服务器的作用是给一次跨服事件提供统一裁决点，应用服务器只代理请求、执行结果和同步状态。

容错解决的是失败之后如何继续服务。主从复制维持可接管的在线副本；备份与连续归档的 WAL 保留历史状态，借助时间点恢复（PITR）把数据库停在误删、误改或程序缺陷发生之前，恢复演练则检验这份历史能否在 RPO 和 RTO 约束内真正恢复；Raft 则保护协调服务这类强一致组件的操作历史，使 Leader 故障后仍能从多数派确认的日志中恢复已提交结果。CAP 定理和 PACELC 模型提醒我们，一致性、可用性和延迟之间始终存在取舍；架构设计的重点不是把所有链路都拉到最强一致，而是根据业务能否容忍延迟、短暂不一致和错误结果来选择机制。

至此，多机架构在逻辑上已经成型：数据有分片承载，连接有负载调度分配，跨服事件有协调点裁决，关键状态有副本、备份或共识机制保护。下一步的问题不再只是“这些组件应该怎样设计”，而是“这些组件如何被稳定地交付和管理”。当网关、逻辑服、缓存、数据库分片、协调器和副本集群都需要部署、升级、扩缩容和故障替换时，手工操作会成为新的瓶颈。第五章将从这个交付问题出发，讨论容器、镜像和编排平台如何把多组件系统变成可复制、可调度、可恢复的运行单元。

4.6 思考题

1. 某游戏需要同时支持“按用户查询数据”和“全服战力排行榜”两类访问。如果用用户 ID 做哈希分片键，按用户查询会得到什么好处？排行榜查询又会遇到什么困难？请说明跨分片聚合的成本，并设计一种异步汇总排行榜的基本思路。
2. 一个一致性哈希环上原有 4 个物理节点，每个物理节点对应 150 个虚拟节点。现在新增第 5 个物理节点。请估算理想情况下大约有多少比例的键空间需要迁移，并解释虚拟节点数量为什么会影响负载均匀度和迁移波动。
3. 跨服擂台的战斗结算需要同时更新玩家 A 所在分片和玩家 B 所在分片的战绩。请分别用 2PC 和事务性发件箱两种方案描述处理流程，并比较它们在延迟、阻塞风险和一致性语义上的取舍。进一步思考：为什么事务性发件箱方案要求消费端必须做幂等处理？
4. 一个 5 节点 Raft 集群中，Leader 已经把一条日志复制到多数节点并提交，但还没来得及把新的 `commitIndex` 通知给所有 Follower 就崩溃了。请推演后续选举过程，并解释为什么“两个多数派必有交集”能够保护已提交日志不被丢失。
5. 服务器 A 正在进行一场 100 人公会战，CPU 使用率达到 95%；服务器 B 和 C 的 CPU 使用率只有 30%。如果继续使用简单轮询，新玩家仍可能被分配到 A。请设计一种基于心跳上报的动态调度策略，说明需要上报哪些指标，综合负载值如何计算，以及如何避免所有新玩家同时涌向同一台“最空闲”的服务器。
6. 假设网络分区已经发生，给出三个跨服场景：(a) 跨服战斗的命中判定；(b) 全服排

行榜的实时刷新；(c) 玩家跨服迁移时的资产交接。请分别判断它们更偏向 CP 还是 AP，并说明理由。进一步思考：哪些场景可以接受几秒钟的最终一致性，哪些场景必须在请求路径上保证强一致？

7. 一个游戏同时需要存储分片（按 PlayerID 拆数据库）、计算分片（按 MapID 分配战斗服务）和连接分片（按区域分配网关）。请为每种分片设计路由查找的基本流程，并说明三类分片的分片键为什么不同。
8. 主从复制、备份恢复和 Raft 分别保护什么对象？如果主库故障且最新从库的复制进度落后 5 秒，故障切换后这 5 秒内的写入会怎样？与基础备份加 WAL 连续归档的时间点恢复相比，故障切换的优势和局限分别是什么？
9. Raft 通过复制日志，使多个副本按照相同顺序执行相同命令；2PC 则协调多个独立数据库的本地事务共同提交或回滚。为什么跨分片交易不能简单地把操作写进一个 Raft 日志，就认为它已经替代了 2PC？请从参与对象、提交范围和故障后的恢复责任三个方面比较二者。

5 第五章 飞升入定：云原生部署

到第四章为止，游戏后端在逻辑上已经具备多机运行的结构：数据库分片承载存储容量，负载均衡分配玩家连接，协调服务器裁决跨服事件，Raft 集群保护关键状态。但这些组件要真正运行在多台服务器上，还需要回答一个工程问题：如何把它们稳定地交付、部署和维护？

本章所说的“上云”，是将应用部署到由云平台管理的计算、网络和存储资源上，使其能够面向互联网持续提供服务，并在实例增减、节点故障和环境差异下保持可交付、可调度和可恢复。部署对象不再是固定服务器上的若干进程，而是一组可能被复制、替换和重新放置的服务实例。

当系统只有一两台服务器时，运维人员可以逐台登录、手工安装依赖、逐个启动进程。一旦组件数量增多、服务器台数增加，手工操作的可靠性会迅速下降。具体表现为三类困难。第一，环境差异：同一份代码在开发机上能运行，换到另一台服务器后可能因为系统库版本、时区设置或目录结构不同而无法启动。第二，部署复杂度：大厅服、战斗服、Redis、PostgreSQL 和入口层分散在多台机器上，启动顺序、配置注入和网络互联都需要人工对齐。第三，运维脆弱性：扩容和故障恢复依赖人工响应，而高峰时段正是人最容易出错的时候。

本章按照“容器—编排—弹性运行”三大块展开。第一块讨论容器如何把代码、依赖和运行配置固化为可复制的交付物，解决“换一台机器还能否按同样方式运行”的问题。第二块讨论编排如何描述多个服务的启动、互联和副本关系，并把单机部署扩展到多机集群，解决“多个组件由谁统一部署和维护”的问题。第三块讨论系统运行之后，平台如何完成资源调度、自动伸缩、按需执行和故障自愈，解决“负载变化或实例故障时如何维持服务目标”的问题。三块内容依次把关注点从单个运行环境，推进到多服务拓扑，再推进到持续运行中的容量与故障。

5.1 容器：上云的虚拟化基础

容器部分先说明为什么同一份代码换机器后可能无法启动，再介绍镜像、容器、网络和数据卷等机制如何解决这一问题。这些机制共同支撑一条“构建一次、多处运行”的交付链路：镜像在开发环境构建完成，推送到仓库统一分发，再由目标机器拉取后直接运行。本节的重点在于理解容器为什么能把一次部署结果稳定地复制到其他机器，具体的 Docker 命令会在必要时作为示例出现。

5.1.1 为什么需要容器：环境一致性

一份程序能够正常运行，不只取决于源代码本身，还取决于它所处的运行环境。环境包括操作系统版本、动态链接库、内核接口、目录结构、用户权限、时区设置和字符编码等条件。当开发机和部署目标机器的环境不同时，同一份可执行文件可能在一边正常启动，在另一边无法运行。理解环境差异的来源，是理解容器、镜像不可变和运行时配置注入等后续机制的前提。

隐性依赖：代码之外的运行条件 程序的运行依赖可以分为两类。一类是显式依赖：源代码中直接引用的第三方库、框架和工具，通常记录在依赖清单（如 `go.mod`、`requirements.txt`）中。另一类是隐性依赖：程序并未在代码中声明，但运行时仍然需要的系统条件，例如特定版本

的 glibc、特定路径下的证书文件、特定的内核参数或时区数据。隐性依赖不出现在依赖清单中，往往在更换机器后才暴露。

以第四章的游戏后端为例：大厅服在启用 CGO 的场景下会链接宿主机的系统库，战斗服依赖高精度定时器接口，Redis 和 PostgreSQL 各自有版本兼容性要求，入口层的配置文件引用了特定的模块路径。这些依赖大多不会出现在代码的依赖清单中，正是前面所说的隐性依赖。在一台全新的服务器上重现这套环境，运维人员需要逐一安装依赖、统一版本、调整配置，这些步骤需要在每台服务器上重复一遍。当需要紧急扩容十台服务器时，这种逐台手工操作难以在短时间内完成。

隐性依赖导致的典型故障包括：动态库版本不匹配导致进程启动即崩溃、证书或时区文件缺失导致 TLS 握手失败或时间戳错误、目录权限不同导致写入被拒绝。这些问题的共同特点是：在开发机上不会触发，因为开发机的环境恰好满足了这些隐性依赖；换到另一台机器后条件不再成立，故障才会出现。对在线游戏而言，环境问题的影响集中在扩容和故障恢复两个场景，这两个场景正是要求在最短时间内启动新的服务实例。

三种环境交付方案：手工安装、虚拟机与容器 环境一致性问题有三种主要的解决思路，它们在一致性保证、资源开销和扩容效率之间做出不同取舍。

第一种方案是手工安装依赖。运维人员登录每台服务器，按照文档逐步执行 `apt install`、`pip install`、修改配置文件等操作。这种方式直接且无额外运行开销，但同一份文档由不同的人在不同时间执行，结果往往不同。软件源更新、软件包废弃、操作顺序颠倒都会导致环境偏移，而且这种偏移通常是隐性的，直到线上出现异常时才被发现。

第二种方案是虚拟机镜像。将应用连同完整的 Guest OS、内核和系统库一起封装为虚拟机镜像，部署时直接启动镜像实例。这种方式能够固定虚拟机内部的操作系统、系统库和应用文件，显著减少不同服务器之间的环境差异。但每个虚拟机实例都要运行一套完整的操作系统内核，通常比容器占用更多内存，启动路径也更长；实际开销取决于镜像、虚拟化平台和硬件配置。当游戏服务需要在短时间内扩容多个实例时，虚拟机的启动延迟会显著拉长扩容响应时间。

第三种方案是容器。容器只打包应用运行所需的用户态环境（代码、运行时、系统工具、依赖库），不包含完整的操作系统内核，而是直接共享宿主机的 Linux 内核。这种方式同样能够固定用户态环境：在处理器架构一致、宿主机内核接口兼容的前提下，同一份容器镜像可以提供相同的代码、工具和依赖库；同时避免了虚拟机的内核冗余开销。因此，容器通常比完整虚拟机更小、启动更快，但实际体积和启动时间仍取决于镜像内容、运行时、硬件和初始化过程。

	手工安装	虚拟机	容器
环境一致性	差	好	好
资源开销	无额外运行层	较高	较低
启动速度	—	通常较慢	通常较快
可重复性	差	好	好
扩容效率	极低	低	高

表 6: 三种环境交付方式的对比

表 6 从五个维度对比了三种方案。手工安装不增加运行开销，但无法保证跨机器的一致性和可重复性。虚拟机提供了完整的环境隔离，但携带了整套操作系统的资源成本。容器只携带用户态环境，共享宿主机内核，在一致性和轻量之间取得了折中，代价是隔离强度弱于虚拟机。

容器是一个通用概念，多个技术实现都遵循上述思路。其中 Docker 在工程中使用最广泛，它将环境交付的单位从源代码或安装包变为完整的运行环境镜像：同一份镜像在开发机和云端服务器上提供相同的用户态条件。Docker 的核心机制围绕三个问题展开：镜像如何分层构建和复用，容器如何在共享内核上实现隔离，数据如何从容器生命周期中分离。

5.1.2 容器的基本原理与使用方法

Docker 将容器技术的构建、分发和运行三个环节标准化为一套工具链。开发者通过 Dockerfile 描述环境构建过程，构建产物是一份不可变的镜像；镜像可以推送到仓库供任意机器拉取；拉取后在任意支持 Docker 的宿主机上以容器形式运行。下面先建立镜像、容器与隔离机制，再把网络、镜像构建、仓库分发和实例运行连成一条完整的部署链路。

镜像与容器：模板与实例 Docker 中有两个必须区分的核心概念：镜像（Image）与容器（Container）。镜像、容器可写层和数据卷具有不同的生命周期，只有先区分这三类对象，才能判断哪些内容可以随实例删除，哪些状态必须长期保留。图 58 将这种生命周期关系置于同一模型中。

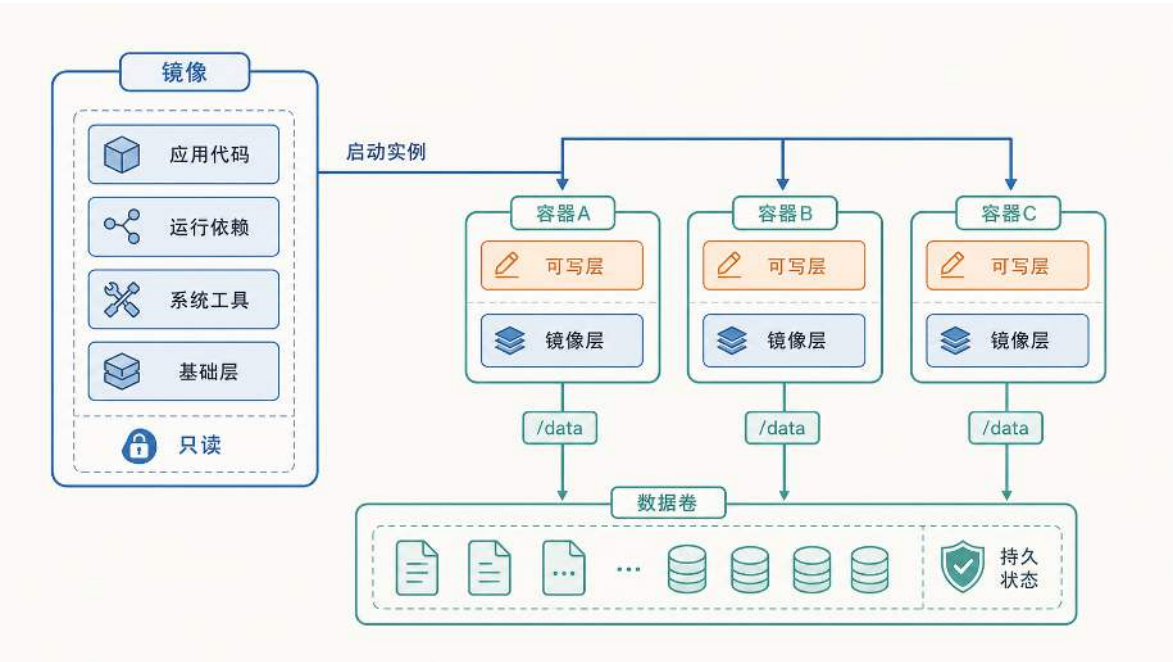


图 58: 镜像、容器与数据卷的生命周期关系：镜像负责复用，容器负责运行，数据卷保存需要长期保留的状态。

镜像由多层只读文件系统叠加而成，从底向上依次是基础层（精简的操作系统用户空间）、系统工具、运行依赖和应用代码。除了文件内容，镜像还包含一组运行元数据：默认启动命令

(ENTRYPOINT/CMD)、工作目录、环境变量默认值和对外声明的端口。镜像一旦构建完成就不再被修改，这一不可变性保证了环境一致性：任意一台机器拉取到同一份镜像后，得到的文件内容和启动方式都是确定的。

当镜像在宿主机上启动时，Docker 在只读镜像层之上挂载一层独立的可写层，形成容器。多个容器可以共享同一份只读镜像模板，各自拥有独立的可写空间、进程空间和网络环境。运行期间程序产生的状态变更（写日志、生成缓存、创建临时文件）都发生在各自的可写层中，镜像层始终保持只读。

可写层的生命周期与容器绑定，容器被删除后，其可写层随之消失，写入其中的数据永久丢失。对于数据库文件、玩家存档等需要长期保留的数据，Docker 提供了数据卷（Volume）机制：数据卷独立于容器生命周期，通过挂载操作映射到容器内的特定路径（如 `/data`）。程序向该路径写入数据时，数据直接落入宿主机磁盘，不经过可写层。这样，计算逻辑（容器）与持久数据（卷）的生命周期被分离：删除容器或升级镜像后，只要挂载同一个数据卷，新容器仍然能接管之前的数据。

镜像不可修改这一性质还带来一个工程问题：同一份镜像要部署到开发、测试、生产等不同环境时，数据库地址、端口号、密钥等配置各不相同，如何处理？做法是将程序本体固化在镜像中，将随环境变化的配置留到运行时注入。注入方式包括环境变量（通过 `-e` 参数或 Compose 的 `environment` 字段传入）、挂载配置文件和挂载数据卷。这样，同一份镜像无需重新构建即可在多种环境中运行，差异完全来自显式声明的运行时配置。这一思路对应 Twelve-Factor App 方法论²⁹中“将配置存储在环境中”这一原则：应用制品本身不包含环境相关信息，环境差异通过运行时注入解决。

至此，本节建立了 Docker 中三个核心对象的关系：镜像提供只读的环境模板，容器在镜像之上添加可写层形成运行实例，数据卷将需要长期保留的状态从容器生命周期中分离出来。镜像不可修改保证了环境一致性，运行时注入解决了跨环境配置差异。在此基础上，一个关键问题尚待回答：容器如何在共享宿主机内核的前提下实现进程和资源的隔离？

隔离原理：Namespace 与 Cgroups 前面提到，容器与虚拟机不同：容器不包含独立的操作系统内核，而是共享宿主机的 Linux 内核。这就带来一个问题：多个容器运行在同一个内核之上，如果没有额外机制，它们就能看到彼此的进程、访问同一套网络接口、争抢同一份 CPU 和内存资源。容器隔离要解决的正是这两件事：让每个容器只能看到属于自己的系统资源视图，以及限制每个容器能消耗的物理资源总量。

虚拟机早已解决了同样的问题，但它的做法是为每个实例配备独立内核，由 Hypervisor 提供硬件级隔离。不同 VM 中的进程运行在完全独立的内核实例上，彼此无法看到对方的进程列表或访问对方的内存空间。这种隔离非常彻底，但每个 VM 都要为 Guest OS 和内核付出数百 MB 到数 GB 的内存开销，传统虚拟机的启动时间通常以分钟计。

容器选择了另一条路径：在共享内核上用软件机制达到类似效果。多个容器各自只包含应用和用户态库，底部共享同一个 Linux 内核。内核通过 Namespace 提供视图隔离，通过 Cgroups

²⁹Twelve-Factor App 的核心主张包括代码与配置分离、依赖显式声明、进程无状态化等。

提供资源配额限制。共享内核降低了运行开销，也使隔离能力依赖内核机制；图 59 将这一取舍与虚拟机的独立内核方案置于相同条件下比较。

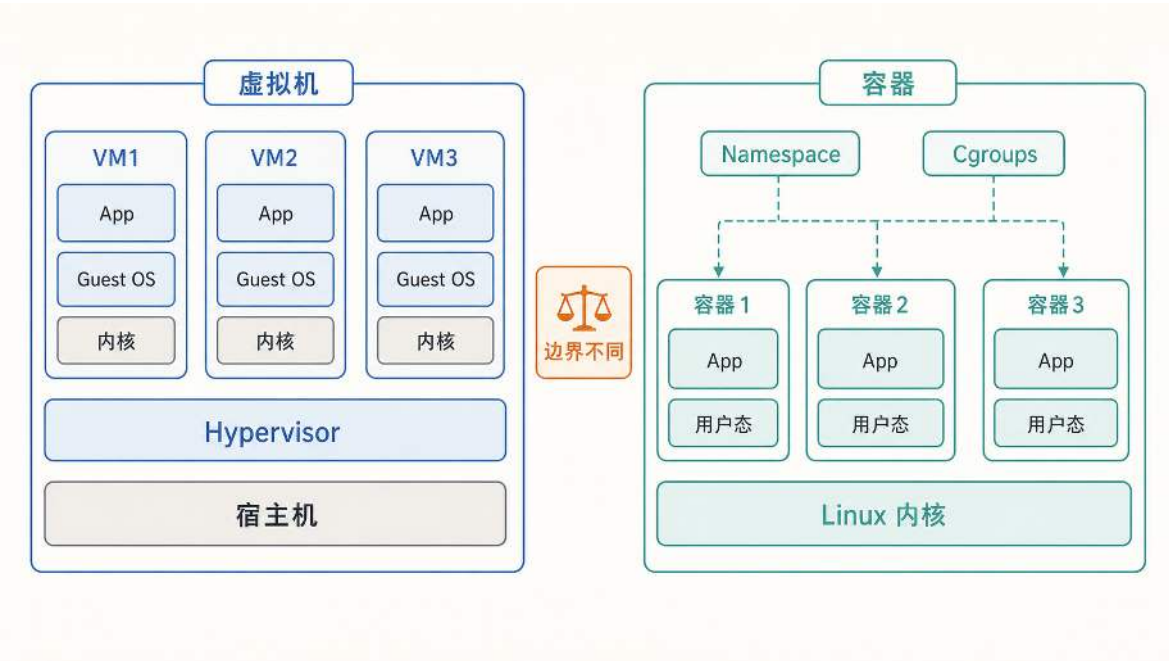


图 59: 虚拟机通过独立 Guest OS 和内核提供强隔离；容器共享 Linux 内核，依靠 Namespace 和 Cgroups 获得更轻量的隔离。

虚拟机用独立内核换取强隔离，容器用共享内核换取轻量和快速启动。

Namespace（命名空间）解决的是“容器能看到什么”的问题。Linux 内核为每个容器创建一组独立的命名空间，使容器只能看到属于自己的系统资源。PID Namespace 让容器内部只能看到自己的进程列表，无法感知宿主机或其他容器中的进程。NET Namespace 让每个容器拥有独立的网络栈（IP 地址、端口空间、路由表），同一个端口号在不同容器中互不冲突。MNT Namespace 让每个容器拥有独立的文件系统挂载视图，容器内的路径结构与宿主机或其他容器不同。通过这些命名空间的组合，容器内的程序“认为”自己运行在一台独立的机器上，而实际上它只是宿主机上一个受限的进程。

Cgroups（控制组）解决的是“容器能用多少”的问题。即使容器通过 Namespace 看不到其他容器的进程，它仍然与其他容器共享同一份物理 CPU 和内存。如果某个容器中的程序出现内存泄漏或死循环，在没有限制的情况下它可能耗尽整台宿主机的资源，导致同一宿主机上的所有其他容器一起受到影响。Cgroups 为每个容器设定 CPU 时间片上限和内存使用上限：容器的资源消耗达到上限后，内核会限制其继续获取 CPU 时间或直接触发 OOM（Out of Memory）终止，而不是让它影响其他容器。

将两个机制合在一起看：Namespace 提供视图隔离（容器看不到别人），Cgroups 提供资源隔离（容器用不了别人的份额）。二者共同使得容器在共享内核的前提下获得了进程、网络、文件系统和资源配额四个维度的独立性。

与虚拟机相比，容器的隔离代价更低（无需额外内核和 Guest OS），但隔离强度也更弱。虚拟机的隔离由独立内核保证，一个 VM 中的内核漏洞不会影响到另一个 VM。容器的隔离由共享

内核中的软件机制保证，如果 Linux 内核本身存在漏洞，理论上一个容器可能突破 Namespace 和 Cgroups 的限制影响其他容器。因此，在对隔离强度要求更高的多租户场景中，工程实践会在容器之外叠加更强的隔离层（如轻量级虚拟机或硬件辅助隔离），在启动速度和安全性之间取得折中。Namespace 的创建方式、Cgroups 控制文件与配额周期等内核实现，将在第 6.1 节进一步展开。

隔离带来了安全性，但也引入了一个新问题：每个容器拥有独立的 Network Namespace，意味着容器之间以及容器与外部客户端之间无法像同一台机器上的进程那样直接通信。第四章中大厅服、战斗服、Redis 和 PostgreSQL 需要互相连接，玩家客户端也需要从外部访问入口层。Docker 通过虚拟网桥和端口映射在网络隔离的前提下解决这两类通信需求。

容器网络：Network Namespace 与端口映射 回顾第四章的部署场景：大厅服需要连接 Redis 读取缓存，需要连接 PostgreSQL 读写玩家数据；入口层需要把玩家请求转发给大厅服；玩家客户端需要从公网连接入口层。在容器化之前，如果这些进程运行在同一台服务器上，大厅服用 127.0.0.1:5432 就能连到本机的 PostgreSQL，用 127.0.0.1:6379 就能连到本机的 Redis。

容器化之后，这种连接方式会失效。上一节介绍的 Network Namespace 为每个容器分配了独立的网络栈：每个容器拥有自己的 IP 地址、网卡和路由表。这意味着容器内的 127.0.0.1 只指向容器自身的网络空间，不再指向宿主机，也不指向同一宿主机上的其他容器。如果大厅服容器中仍然将数据库地址配置为 127.0.0.1:5432，连接目标会变成大厅服容器自己的 5432 端口（那里什么都没有），而不是 PostgreSQL 容器，连接会立即失败。

网络隔离带来了两个需要重新解决的问题：第一，同一宿主机上的多个容器之间如何互相找到对方并建立连接；第二，宿主机外部的玩家客户端如何到达容器内部的服务。Docker 分别用用户自定义网络和端口映射来解决这两个问题。

先看容器间通信。Docker 允许创建用户自定义网络，本质上是宿主机内部的一张虚拟局域网。多个容器接入同一张网络后，每个容器获得一个网络内部 IP（例如 172.18.0.2、172.18.0.3），并通过虚拟网卡在这张局域网内收发数据包。但直接使用 IP 地址连接其他容器并不可靠：容器重启后内部 IP 可能发生变化，把 IP 写死在配置中会导致连接中断。Docker 在每张用户自定义网络中运行一个内置 DNS 服务器，维护“服务名 → 容器 IP”的解析关系。当大厅服容器需要连接 Redis 时，它向 DNS 查询 `redis` 这个服务名，DNS 返回 Redis 容器当前的内部 IP（如 172.18.0.4），大厅服再用这个 IP 建立到 172.18.0.4:6379 的 TCP 连接。即使 Redis 容器因为重启导致 IP 从 172.18.0.4 变为 172.18.0.6，服务名 `redis` 始终有效，调用方的配置不需要改动。因此，容器之间互相访问时，地址统一写成“服务名 + 容器内端口”的形式，例如 `redis:6379`、`postgres:5432`。服务名解析、虚拟网卡之间的数据包转发以及底层的 bridge/NAT 规则属于网络虚拟化的实现细节，将在第 6.2 节展开。

再看外部访问。用户自定义网络中的 IP 和端口只在虚拟局域网内可见，宿主机之外的客户端无法直接访问。玩家的游戏客户端运行在自己的手机或 PC 上，它不知道也无法到达 172.18.0.2 这个内部地址。要让外部流量进入容器，需要在宿主机和容器之间建立端口映射：将宿主机上的某个端口转接到目标容器的端口。例如网关容器在内部监听 8080 端口，启动时配置 `-p 8080:8080`。

这条规则的含义是：当外部客户端连接到云服务器公网 IP:8080 时，Docker 把到达宿主机 8080 端口的流量转接到网关容器的 8080 端口。从玩家视角看，连接的是云服务器的公网地址；从网关容器视角看，有一个连接到达了自己的 8080 端口。后续 Compose 配置中的 `ports`：“8080:8080”以及 Kubernetes 中的 `port/targetPort` 描述的都是这种转接关系。

容器间访问和外部访问处于不同的地址范围，前者依赖服务发现，后者依赖宿主机入口。图 60 将两条路径统一到同一网络模型中。

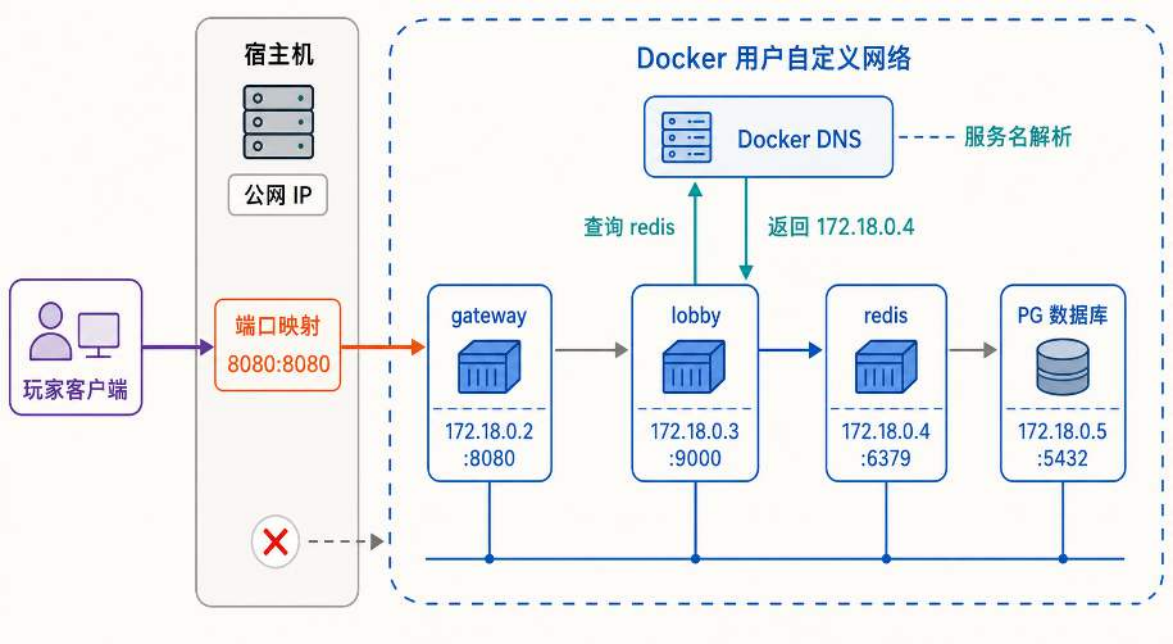


图 60: Docker 用户自定义网络中的两条访问路径：容器间通过服务名和 DNS 在虚拟网络内部互通，外部客户端则必须经过宿主机端口映射才能进入容器。

地址作用域的差异可以进一步归纳为两条配置规则。容器之间互联时，地址写成”服务名:容器内端口”（如 `redis:6379`），由 Docker DNS 负责将服务名解析为容器的内部 IP。外部客户端访问时，地址写成”云服务器公网 IP: 宿主机端口”（如 `203.0.113.10:8080`），由端口映射负责将流量转接到目标容器。两种地址不能混用：容器内部写公网 IP 会绕出虚拟网络再绕回来，外部写服务名则根本无法解析。

至此，Docker 的运行模型已经建立：镜像提供不可变的环境模板，容器在隔离的 Namespace 中运行，数据卷保存持久状态，用户自定义网络和端口映射解决通信问题。但前面的讨论一直假设镜像已经存在，尚未涉及一个基本问题：这份镜像本身是如何构建出来的？

5.1.3 构建、分发与运行的容器化部署闭环

制作镜像：Dockerfile 与分层构建 Docker 通过 Dockerfile 描述镜像的构建过程：开发者在其中逐条写明基础环境、依赖安装和文件复制等步骤，Docker 按顺序执行这些指令，每执行一条就生成镜像中的一个只读层，最终所有层叠加形成完整镜像。

编译型应用如果把工具链和运行环境装入同一镜像，最终制品会携带部署时不再需要的内容。多阶段构建把编译与运行分开，图 61 用这一分离说明镜像为何能够缩小并复用构建缓存。

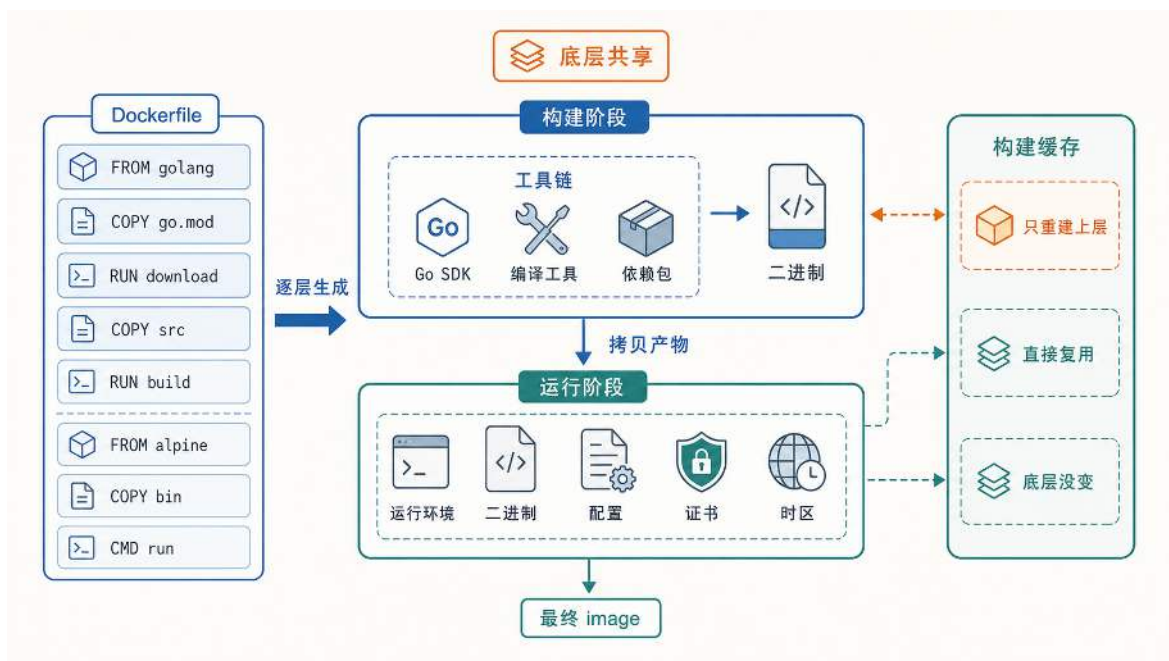


图 61: Dockerfile 的分层构建过程：构建阶段在完整工具链中编译产物，运行阶段只保留最终二进制和最小运行环境，未变化的层直接复用缓存。

构建从 Dockerfile 的第一条指令开始。FROM golang 指定了构建阶段的基础环境，随后 COPY go.mod、RUN download 安装依赖，COPY src、RUN build 编译源码。Go SDK、编译工具和依赖包都存在于这个阶段，最终输出一份二进制文件。接下来 Dockerfile 用第二条 FROM alpine 开启运行阶段，切换到一个仅包含基本用户态工具的极简基础镜像，再把构建阶段产出的二进制、配置文件、证书和时区数据复制进来，形成最终的可分发镜像。

这种两阶段设计的直接效果是镜像体积大幅减小：构建阶段的 Go 工具链（数百 MB）不会出现在最终镜像中，运行阶段只携带实际运行所需的文件（通常十几 MB）。扩容时节点拉取镜像的速度因此加快。

分层构建还带来一个工程上的加速效果：构建缓存。Docker 在构建时逐层检查：如果某一层的指令和输入文件与上次构建完全相同，该层直接复用缓存，不必重新执行。变动频率高的内容（业务源码）放在 Dockerfile 靠后的位置，变动频率低的内容（基础环境和依赖安装）放在靠前的位置。这意味着日常开发中只改动了业务代码时，Docker 只需重建最上面几层，底层全部命中缓存。

当服务数量增加时，为每个服务单独维护一份 Dockerfile 会让构建逻辑的重复随服务数量线性增长。第四章中的游戏后端已经拆分为大厅服与战斗服两个独立模块，后续还可能增加新的服务实例。如果每个模块各有一份几乎相同的 Dockerfile，任何基础环境或依赖安装的改动都需要在多份文件中同步修改，维护成本显著上升。下面这份 Dockerfile 通过构建参数 SERVICE 实现了同一份构建描述适用于多个服务：

通用 Dockerfile：多阶段构建

```
1 FROM golang:1.21-alpine AS builder
2 WORKDIR /app
3 COPY go.mod go.sum ./
4 RUN go mod download
5 COPY . .
6 ARG SERVICE
7 RUN if [ -z "$SERVICE" ]; then echo "ERR: SERVICE not set" ; exit 1; fi
8 RUN CGO_ENABLED=0 GOOS=linux go build -o server_bin ./cmd/${SERVICE}
9 FROM alpine:latest
10 RUN apk --no-cache add ca-certificates tzdata
11 WORKDIR /root/
12 COPY --from=builder /app/server_bin .
13 COPY --from=builder /app/config ./config
14 CMD [ "./server_bin" ]
```

这份 Dockerfile 中有三处值得注意的工程选择，分别对应镜像体积、构建速度和运行时独立性。

第一处是多阶段构建。Dockerfile 中出现了两条 FROM 指令：第一条 FROM golang:1.21-alpine AS builder 引入包含完整 Go 工具链的编译环境，第二条 FROM alpine:latest 切换到极简运行环境。运行阶段通过 COPY --from=builder 只从编译阶段取走最终二进制和配置，Go 编译器本身不进入最终镜像。由此产出的镜像通常只有十几 MB，远小于编译环境的数百 MB。

第二处是依赖下载与源码复制的顺序。Dockerfile 先复制 go.mod 和 go.sum 并执行 go mod download，之后才复制全部源码。由于依赖清单的变动频率远低于业务代码，这种顺序保证了大多数构建中依赖下载层命中缓存，只有源码层需要重建。

第三处是静态编译。构建命令中的 CGO_ENABLED=0 指示 Go 编译器生成不依赖任何动态链接库的静态二进制。这意味着运行阶段的 alpine 镜像中不需要安装额外的 .so 文件，容器可以在同架构的任何 Linux 内核上直接启动，不受宿主机用户态库版本的影响。

分发镜像：Registry 与版本锁定 镜像构建完成后，需要一种机制让多台云服务器都能获取同一份镜像。Docker 的标准做法是引入一个集中式的镜像仓库（Registry）：开发机完成构建后将镜像推送到 Registry，部署时各台云服务器再从 Registry 拉取同一份镜像。由于镜像本身是分层的，拉取时 Docker 只需下载本地尚未存在的层，已有层直接复用，传输量通常远小于镜像的完整体积。分发链路还必须保证制品能够传播到多台机器，并且各机器获取的内容保持一致。图 62 将构建、仓库分发和版本校验连成同一过程，为后文区分 Tag 与 Digest 建立基础。

分发链路解决了“从哪里拉取”的问题，但还有一个更隐蔽的问题：如何保证每台机器拉取到的确实是同一份内容？Docker 的版本标识有两种形式。Tag（如 latest、v1.2）是供人阅读的别名，便于记忆和引用，但它是可变的：同一个 Tag 可以在不同时间被重新指向另一份镜像内

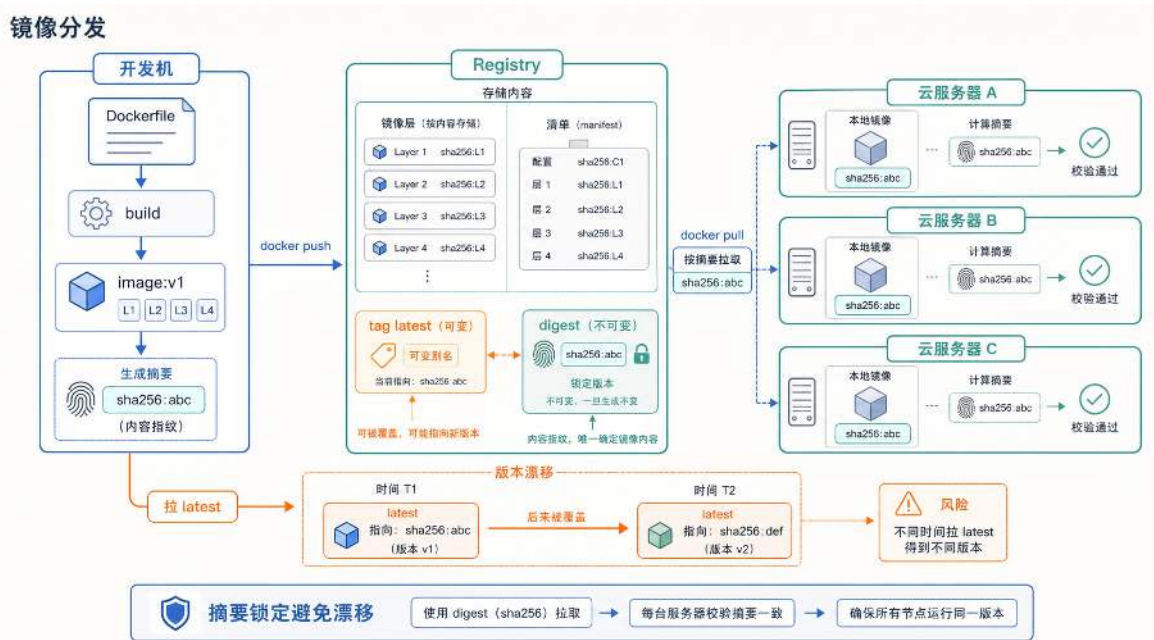


图 62: Docker 镜像的标准化分发链路：开发机推送镜像，云服务器按同一版本摘要拉取，避免线上版本漂移。

容。例如，周一推送的镜像标记为 `v1.2`，周三修复了一个 bug 后重新构建并再次标记为 `v1.2`，此时 Tag 没有变化，但对应的镜像内容已经不同。如果三台云服务器分别在周一、周三和周四拉取 `v1.2`，它们实际运行的镜像可能并不一致，这正是 Tag 漂移问题。

Digest（内容摘要，如 `sha256:a3b7c9...`）是镜像 manifest 的 SHA256 哈希值，在推送到 Registry 时由镜像内容唯一确定。只要两份镜像的 Digest 相同，它们的文件内容就一定相同；内容有任何变化，Digest 必然不同。因此，生产环境中要保证部署的可复现性，部署配置中锁定的应当是 Digest 而非 Tag。写法为 `repo/image@sha256:a3b7c9...`，部署系统按此摘要拉取时，得到的镜像内容是确定的，不受后续重新推送的影响。

构建参数、镜像名和容器服务名这三个标识符名称相近，但分别作用于编译、交付和运行三个不同阶段，初学者容易把它们混为一谈。下面逐一厘清。构建参数（如 `SERVICE=lobby`）对应 Dockerfile 中的 `ARG SERVICE`，决定编译阶段选择哪个入口路径。镜像名与版本（如 `game-lobby:v1`）标识构建产物，是推送到 Registry 和拉取时使用的交付单位。容器服务名（如 `lobby-api`）是容器在 Docker 网络中注册的 DNS 名称，其他容器通过该名称发现并连接它。三者分别回答“编译什么”“交付什么”“在网络中如何被寻址”，各自独立变化：同一份镜像可以在不同环境中以不同服务名运行，同一个服务名也可以随版本升级指向新的镜像。

至此，镜像构建、分层缓存、仓库分发和版本锁定等机制已经逐一介绍完毕。下一步是将它们合在一起使用：把第四章的游戏后端各组件分别装入容器，在一台机器上配置网络、端口和存储，验证整套拓扑能否正常运行。

将游戏后端装入容器 将第四章的游戏后端架构映射到容器：大厅服、战斗服、协调服务器属于计算进程，各自封装为独立镜像；Redis 与 PostgreSQL 属于基础设施进程，同样各自对应一

份镜像；入口层（自研网关或 Nginx 等反向代理）也是一个可容器化的进程。每个容器化组件需要回答两个基本问题：它在容器内监听哪个端口？它要主动连接哪些上游服务的地址和端口？

下面在一台机器上把这些组件以容器形式启动并互相连通，目的是验证前几节介绍的网络、端口、数据卷和配置注入机制在实际部署中如何配合。为简化演示，这里只保留入口层、大厅服、Redis 和 PostgreSQL 四个组件；战斗服与协调服务器的接入方式与大厅服相同，不再重复。

单机部署示例（结构示意）

```
1 docker network create game-net || true
2
3 docker run -d --name postgres --network game-net \
4   -v pg-data:/var/lib/postgresql/data postgres:15
5 docker run -d --name redis --network game-net redis:6
6
7 docker run -d --name lobby-api --network game-net \
8   -e DB_ADDR=postgres:5432 -e REDIS_ADDR=redis:6379 \
9   game-lobby:v1
10 docker run -d --name gateway --network game-net \
11   -p 8080:8080 -e LOBBY_ADDR=lobby-api:8080 \
12   game-gateway:v1
```

这段命令中体现了前几节介绍的五个配置机制。

第一，所有容器通过 `--network game-net` 接入同一张用户自定义网络。接入后，每个容器的 `--name`（如 `postgres`、`redis`、`lobby-api`）自动注册为 Docker 内置 DNS 中的服务名。大厅服连接数据库时使用 `postgres:5432` 而非 IP 地址，正是依赖这一解析机制。

第二，运行时配置通过环境变量注入。大厅服的 `-e DB_ADDR=postgres:5432` 和 `-e REDIS_ADDR=redis:6379` 将上游地址以环境变量形式传入容器，镜像本身不包含任何环境相关的硬编码地址。这是前文介绍的 Twelve-Factor 原则在实际部署中的体现。

第三，端口映射仅出现在需要对外暴露的入口层。只有网关容器使用了 `-p 8080:8080`，将宿主机的 8080 端口转接到容器内部的 8080 端口，供外部玩家客户端连接。其余容器不需要端口映射，因为它们之间的通信完全在 Docker 网络内部通过服务名完成。

第四，PostgreSQL 容器挂载了数据卷 `pg-data`，使数据库文件持久化到宿主机磁盘。即使 PostgreSQL 容器被删除或升级镜像后重新创建，只要挂载同一个数据卷，数据仍然完整保留。

以上五项配置（网络归属、服务名注册、环境变量注入、端口映射和数据卷挂载）是容器启动时需要显式指定的部署参数。但即使只有四个组件，维护这些配置已经暴露出明显的脆弱性：启动顺序完全依赖执行者的记忆（PostgreSQL 和 Redis 必须先于大厅服启动），配置参数散落在各条命令的 `-e` 和 `-p` 标志中无法统一审查，新增一个组件需要手动补齐网络接入、环境变量和启动顺序。当组件数量从四个增长到十几个时，一串 `docker run` 命令将难以维护和复现。

解决这个问题的思路是：把所有组件的镜像版本、网络归属、端口映射、环境变量和数据卷声明集中写入一份文件，用工具统一解析和执行。Docker Compose 正是这种声明式部署方式的

标准实现。

5.2 编排：云上多服务互连与调度的基础

上一部分已经能把单个服务装入容器，但一组在线服务还需要共同启动、互相发现、按约束放置，并在配置变化后自动将实际状态拉回与声明一致。编排解决的就是这组问题：用户描述服务需要达到的状态，平台负责创建实例、连接网络、维护副本，并在现实状态偏离目标时执行纠正。本部分先用 Docker Compose 说明单机多容器编排，再分析跨机器时新增的网络、发现和集中控制问题，最后进入 Kubernetes 的集群级自动化编排。

5.2.1 编排是什么：从手工命令到声明式拓扑

手工管理关注的是“下一条命令执行什么”，编排关注的是“整组服务最终应处于什么状态”。一份编排声明至少需要回答四个问题：运行哪些服务、每个服务需要多少实例、服务之间如何互联、实例失败或配置变化后如何恢复到目标状态。单机工具只能完成其中一部分，集群级平台还要增加跨节点调度、故障补位和滚动更新等能力。

手工管理一组容器时，运维人员需要逐条执行 `docker run` 命令，按记忆维护启动顺序，手动检查每个容器是否存活，出现故障时逐个登录服务器排查。编排则把同样的信息写成一份声明文件：服务需要达到什么状态由文件描述，状态偏离目标时由平台自动纠偏。这种差异不仅体现在工具层面，更体现在管理视角上。手工管理看到的是“这个容器挂了”，编排看到的是“这个服务应当保持 3 个副本，当前只有 2 个，需要补 1 个”。

5.2.2 单机多容器编排：Docker Compose

Compose 文件使用 YAML 格式描述一组容器的期望状态。文件的核心结构是 `services` 块：其中每一个条目定义一类服务，Compose 根据该定义创建一个或多个容器；条目名称同时作为项目网络中的服务名。每个条目内部声明这个容器需要的全部配置：使用什么镜像、注入哪些环境变量、暴露哪些端口、挂载什么数据卷、依赖哪些其他服务。换言之，第 5.1.3 节部署示例中散落在各条 `docker run` 命令里的信息，现在按容器分组写进了同一份文件。

下面这份配置描述了与该示例相同的四个组件（PostgreSQL、Redis、大厅服务和网关）：

Compose 配置示例 (docker-compose.yml)

```
1 services:
2   postgres:
3     image: postgres:15
4     volumes:
5       - pg-data:/var/lib/postgresql/data
6   redis:
7     image: redis:6
8   lobby-api:
9     image: game-lobby:v1
10    depends_on: [postgres, redis]
11    environment:
12      DB_ADDR: postgres:5432
13      REDIS_ADDR: redis:6379
14  gateway:
15    image: game-gateway:v1
16    ports: ["8080:8080"]
17    depends_on: [lobby-api]
18    environment:
19      LOBBY_ADDR: lobby-api:8080
20  volumes:
21    pg-data:
```

执行命令 `docker compose up` 后，Compose 将这份声明转化为实际运行的容器拓扑。同一份声明只有在服务、网络、数据卷、配置和入口规则被一致地转化为运行对象时，才能复现相同的应用拓扑。图 63 将这种从声明到运行状态的映射固定下来。

Compose 的构建过程按以下顺序推进。首先，Compose 创建一张项目专属的用户自定义网络，所有容器将接入这张网络并获得内部 IP。YAML 中不需要显式声明网络名称，Compose 默认整个项目创建一张共享网络。接着，Compose 根据 `depends_on` 确定容器的启动顺序。上例中 `lobby-api` 声明依赖 `postgres` 和 `redis`，`gateway` 又声明依赖 `lobby-api`，因此启动顺序按依赖链推进：先启动数据库与缓存，再启动大厅服，最后启动网关。上例使用的是 `depends_on` 的简写形式，它控制容器进程的启动顺序，但不保证容器内的服务已经就绪。Compose 也支持将依赖条件设置为 `service_healthy`，等待依赖服务通过健康检查后再启动下游；不过健康检查只能覆盖已经声明的检测条件，应用仍应能够处理依赖短暂不可用。例如 PostgreSQL 容器启动后，数据库进程可能仍在执行初始化，尚未开始监听连接；如果 `lobby-api` 此时尝试连接，仍然会失败。因此应用代码中需要包含带退避的连接重试逻辑：每次连接失败后等待一段时间再重试，且等待时间应随失败次数增加而逐渐拉长（这种策略称为退避，backoff）。不能把启动顺序当作依赖在整个运行期间始终可用的保证。每个容器启动时，Compose 将 YAML 中声明的环境

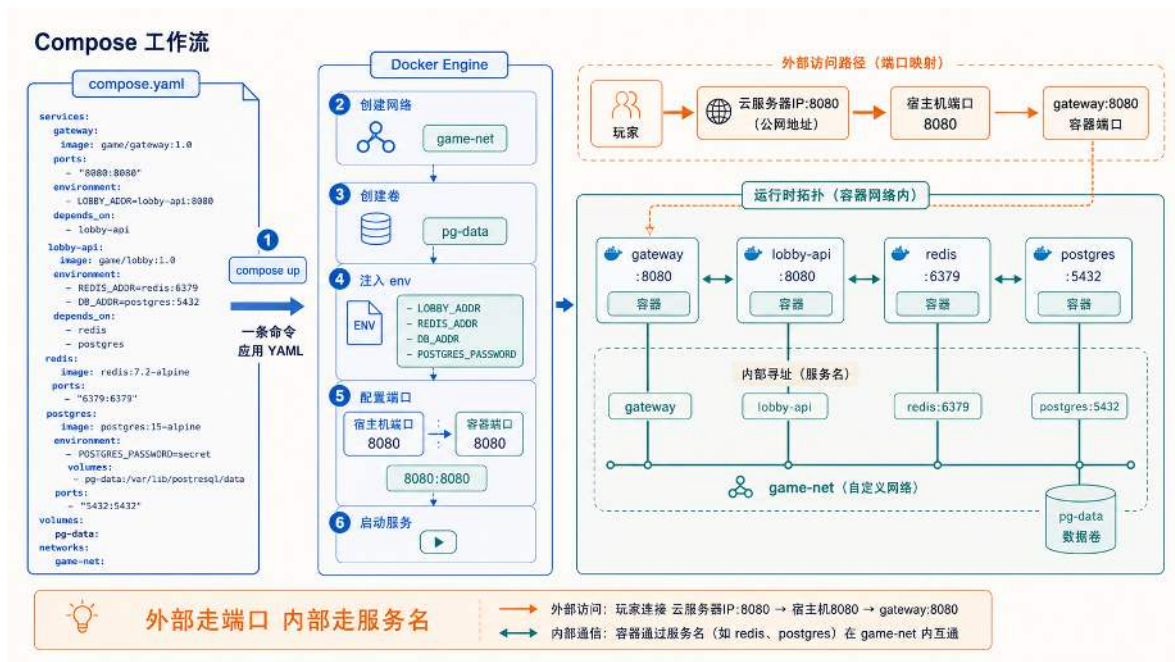


图 63: Compose 将 YAML 声明转化为运行时拓扑：服务、网络、数据卷与环境变量经过构建流程，生成实际运行的容器组及其网络连接。

变量注入容器进程，将 `ports` 字段指定的宿主机端口映射到容器端口，并将 `volumes` 字段指定的数据卷挂载到容器内路径。同时，Compose 将每个服务名注册到网络内置 DNS 中，服务名即为容器间互相访问时使用的 DNS 名称。启动完成后，`lobby-api` 容器访问 `postgres:5432` 时，DNS 将 `postgres` 解析为 PostgreSQL 容器的当前 IP，连接在网络内部直接建立。如果启动过程中出现端口冲突或镜像拉取失败，Compose 会报告具体错误，但已经创建或启动的容器是否保留取决于失败发生的阶段和执行参数，不能把这一过程理解为自动事务回滚。需要恢复到干净状态时，应显式检查项目状态并执行停止或清理操作。

Compose 的另一个重要特性是重复执行时的幂等行为。对同一份 YAML 文件多次执行 `docker compose up`，Compose 会检查当前运行状态与文件描述之间的差异：已存在且配置未变的容器保持运行不做重建，配置发生变化的容器才会被停止并重新创建。这意味着修改 YAML 中某个服务的镜像版本后重新执行，只有该服务会被更新，其余服务不受影响。

Compose 还支持为容器配置重启策略（如 `restart: unless-stopped`）。设置后，容器进程异常退出时 Docker Engine 会自动将其重新拉起，无需人工介入。但这种自动恢复有两个盲区。第一，容器进程可能处于“假死”状态，即进程仍在运行但不再响应请求，`restart` 策略不会触发重启，因为 Docker Engine 只能检测到进程退出，无法判断应用是否还能正常提供服务。要覆盖这类故障，需要应用层面的健康检查机制，这也是集群级编排平台与单机工具的重要差别之一。第二，这种恢复仅限于宿主机本身正常运行的前提下。如果宿主机断电或内核崩溃，运行在其上的所有容器都会一同丢失，Compose 没有能力在另一台机器上补出替代实例。

表 7 从操作模式、管理视角和适用规模等维度比较手工管理与集群级编排。Docker Compose 已具备声明式配置、内置 DNS 服务发现和自动重启等单机能力，但跨机调度、集中配置管理和声明式存储需要集群级平台。编排的核心价值不在于增加几个自动化命令，而在于把管理视角

从”逐个容器”提升到”应用整体”：运维人员描述服务应当达到的状态，平台负责在现实偏离目标时自动纠偏。

表 7: 手工管理与多容器编排的对比

维度	手工管理（如 Shell + docker run）	集群级编排（以 Kubernetes 为例）
操作模式	命令式：按顺序执行命令，每一步依赖前一步成功	声明式：描述期望状态，平台自动收敛现实
管理视角	容器个体：逐个检查每个容器的存活和配置	应用整体：关注服务副本数、互联关系和健康状态
服务发现	需人工配置 IP 和端口，容器重启后重新对齐	内置 DNS 与服务名解析，容器漂移后自动更新
故障处理	人工登录排查、手动重启，响应依赖值班	自动检测失败容器并重新创建，跨机故障时可自动迁移到其他节点
扩缩容	手动执行多次 docker run，逐个修改负载均衡	修改副本数字段即可触发自动调度与扩展
配置管理	环境变量写死在命令行，分散在各条启动脚本中	ConfigMap / Secret 集中管理，与镜像解耦
存储管理	人工指定宿主机路径，迁移时数据跟随困难	声明式卷 (Persistent Volume)，平台负责挂载与迁移
适用规模	单机、开发测试、简单应用	生产环境、大规模集群、微服务架构

5.2.3 从单机到多机：跨机网络、服务发现与集中控制

上一节末尾提到，Compose 配置的重启策略只能在宿主机本身正常的前提下生效。这暴露了 Compose 的根本约束：它的控制对象是单台宿主机上的 Docker Engine。Compose 读取 YAML 后，将配置翻译为一组 API 调用，发送给本机的 Docker Engine；一个 Engine 对应一台机器，因此 Compose 只能将容器编排在一台机器上。

当游戏后端从开发环境走向线上时，单机部署的局限会迅速显现。第一，单机故障等于整体故障：宿主机断电后所有容器丢失，无法在另一台机器上自动补出替代实例。第二，单机资源存在上限：当 CPU 和内存耗尽时，Compose 没有能力将新副本调度到其他机器。第三，跨机器的服务发现和流量入口需要额外维护：Compose 创建的用户自定义网络和 DNS 仅在本机可见，不同机器上的容器无法通过服务名互相访问。

这三项限制分别要求集群提供统一控制面、跨节点网络与服务发现，以及在节点故障后重新放置副本的能力。决定性的变化是期望状态不再绑定到一台 Docker Engine：集群控制面可以在存活节点上恢复缺失副本。图 64 将这一变化置于单机与多机两种管理范围中比较。

统一控制、跨机互联和故障补位构成了多机编排平台的共同问题。下面先用 Docker Swarm 作为过渡实例说明基本机制，再进入功能更完整的 Kubernetes。

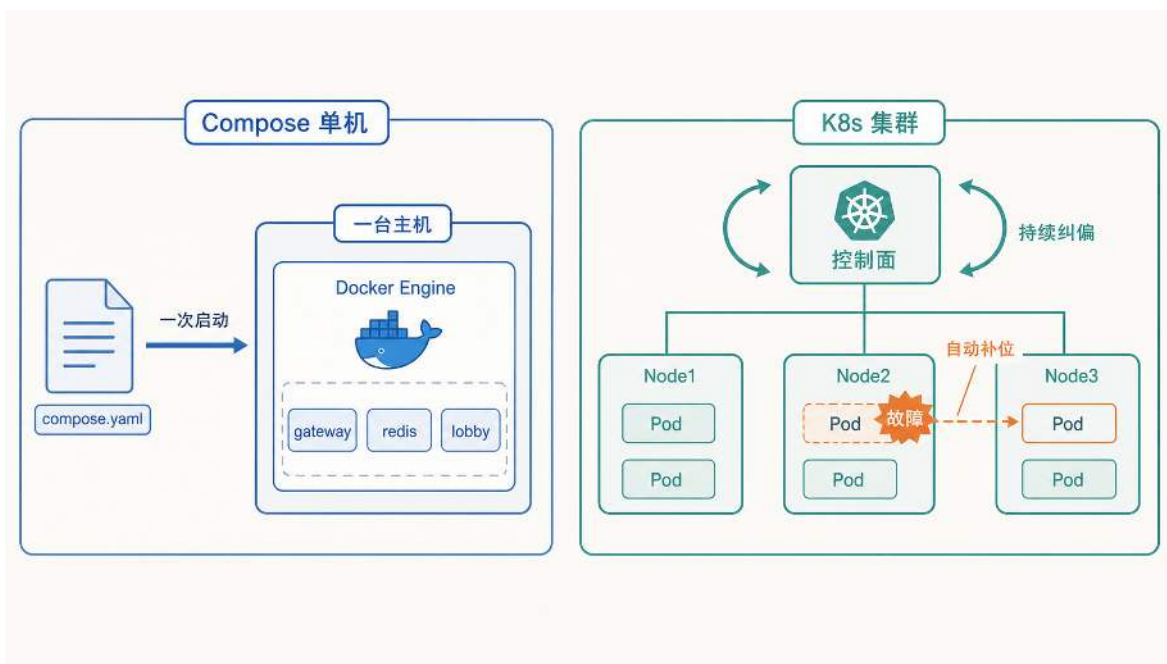


图 64: 单机 Compose 把声明交给一台宿主机；多机集群由控制面持续纠偏，并在节点故障后于存活节点补位。

过渡实例：Docker Swarm Docker 自身为多机场景提供了一套轻量级集群方案 Swarm Mode：将多台机器的 Docker Engine 组织为一个集群，Manager 节点负责控制，Worker 节点负责执行。Swarm 解决了 Compose 无法覆盖的三个关键问题：通过 overlay 网络实现跨节点容器通信，通过集群 DNS 实现跨机服务发现，以及通过 Manager 的统一控制面实现副本分配与故障补位。节点故障后，Manager 依据期望副本数在存活节点上补位，说明统一控制面如何把故障转化为需要纠正的状态偏差。图 65 用正常运行与故障恢复两个阶段验证这一机制。

但 Swarm 的调度规则相对简单，缺少细粒度的亲和性、反亲和性和可扩展控制接口。当集群规模继续增长时，Kubernetes 提供了更完整的机制组合。下面不再比较产品差异，而是直接分析 Kubernetes 如何用同样的”期望状态与实际状态对齐”思路，把控制职责进一步拆分。

5.2.4 Kubernetes 对象、调度与控制面

Kubernetes（简称 K8s）沿用 Swarm 已经建立的”期望状态与实际状态对齐”思路，并进一步把对象定义、放置决策、控制职责和状态调谐拆分为独立机制。下面先建立三个基本对象及其协作关系，再看调度器如何决定副本放在哪台 Node 上，最后分析控制面各组件怎样共同维持集群状态。

Pod、Deployment 与 Service 的协作关系 Pod、Deployment 与 Service 分别承担运行、维护和访问职责，三者不通过固定实例绑定。Deployment 依据标签维持一组可替换的 Pod，Service 通过 Selector（如 `app=lobby`）选择其中已经就绪的实例，因此 Pod 被重建后仍可重新加入同一服务。图 66 建立了这三个对象的协作关系，下面依次说明各对象如何参与部署与恢复。

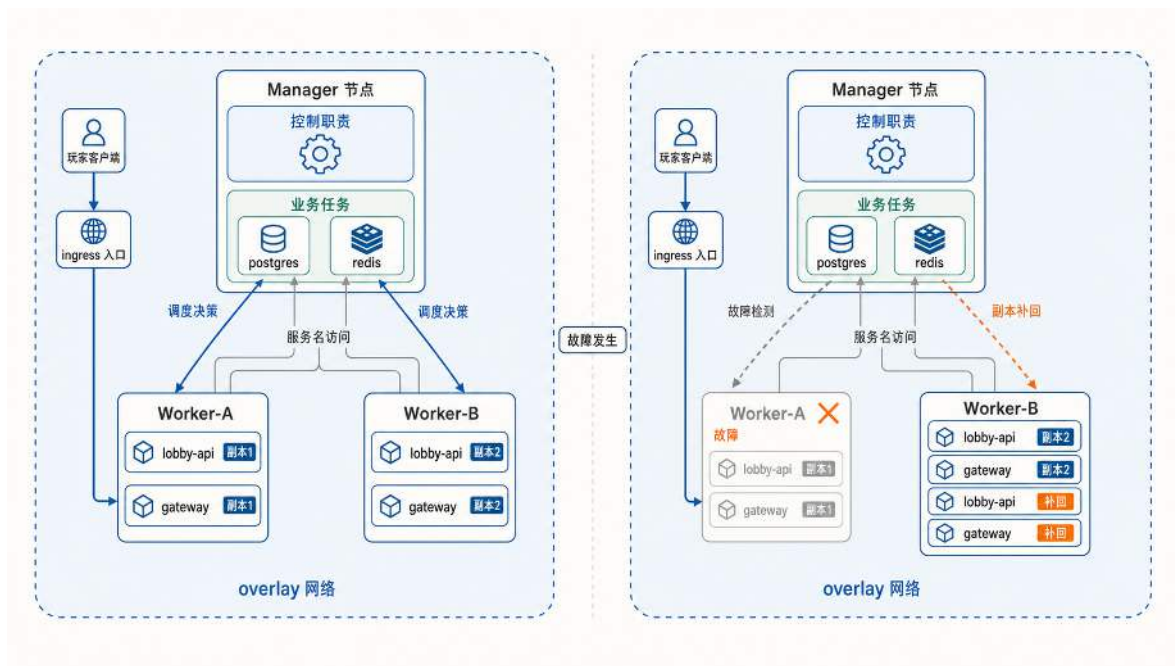


图 65: Docker Swarm 集群的正常运行与故障恢复: Manager 负责控制与调度, Worker 执行业务任务; 节点故障后 Manager 在存活节点上自动补回缺失副本。

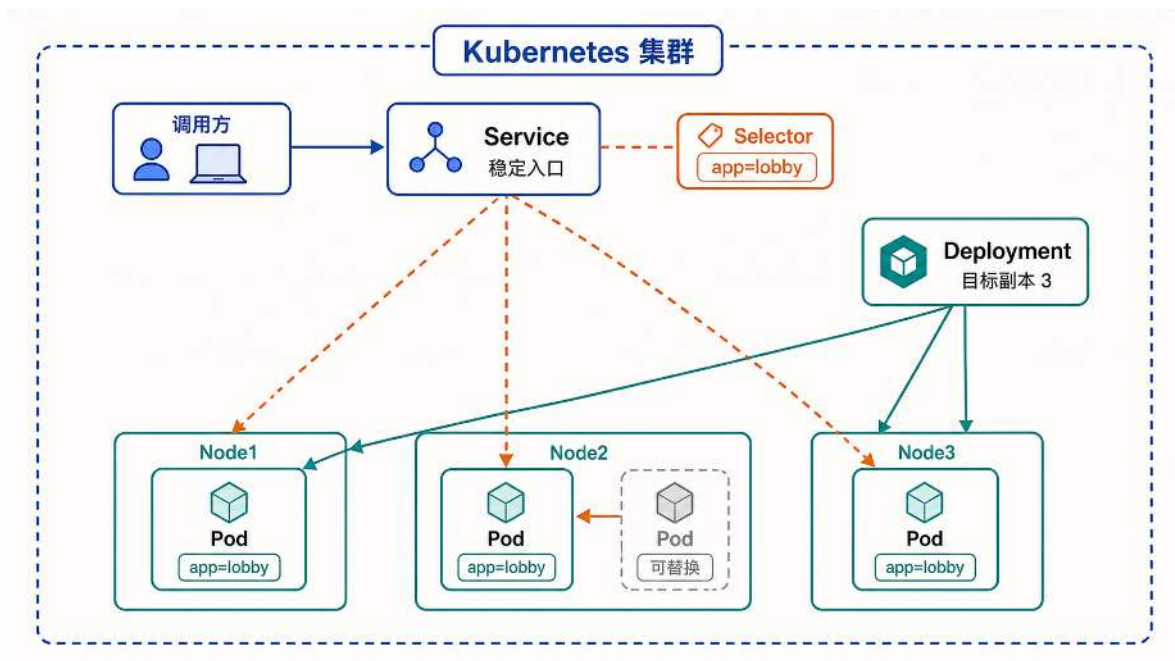


图 66: K8s 核心对象总览: 调用方通过 Service 的稳定入口访问服务, Service 通过 Selector 选择符合标签的 Pod, Deployment 维持 Pod 的副本数目标, Pod 分布在多个 Node 上且可被替换。

Pod：最小的调度单位 在 K8s 的扩容流程中，Scheduler 选择节点后，kubelet 在本地启动的对象是 Pod 而非单个容器。Pod 是 K8s 中最小的调度和管理单位，它包含一个或多个共享网络命名空间和存储卷的容器。

为什么不直接以容器为最小单位？因为有些场景下，两个进程必须紧密协作：它们需要通过 localhost 互相通信，或共享同一份临时文件。例如游戏的战斗服容器旁边可能需要一个日志采集容器（通常称为 sidecar），两者共享同一个日志目录，战斗服写入日志，sidecar 负责将日志转发到集中存储。如果把它们放在不同的调度单位中，就可能出现战斗服已经启动但日志采集还未就绪的不一致状态。

Pod 将这类强耦合关系固化为平台可以理解的内容：同一个 Pod 内的容器被调度到同一个节点，共享网络和存储。容器进程异常时，kubelet 可以在原 Pod 内单独重启容器；Pod 被删除或所在节点失效后，控制器会创建一个新的 Pod，新 Pod 可能被调度到另一节点。这个过程是“创建替代 Pod”，而不是把原 Pod 迁移过去。对于大多数无状态服务（如 lobby-api、gateway），一个 Pod 内通常只有一个业务容器，此时 Pod 可以简单理解为“一个容器实例加上它的网络身份和生命周期”。

Pod 本身是短暂的。它可能因为节点故障被销毁，也可能在滚动更新中被新版本替换。K8s 不试图让某个具体的 Pod 永远存活，而是通过更高层的机制保证“始终有足够数量的同类 Pod 在运行”。这个机制就是 Deployment。

Deployment：声明副本数与更新策略 以一个典型的扩容场景为例：Controller Manager 检测到“期望副本数为 4 而实际为 2”后发起纠正。这里的“期望副本数”就记录在 Deployment 对象中。Deployment 是用户向 K8s 表达以下意图的方式：这个服务应该始终保持多少个 Pod 副本在运行，当需要更新版本时应该按什么策略进行替换。

以游戏的大厅服为例，一份 Deployment 的关键字段如下：

大厅服 Deployment 配置示例

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: lobby-api
5  spec:
6    replicas: 3
7    selector:
8      matchLabels:
9        app: lobby-api
10   strategy:
11     type: RollingUpdate
12     rollingUpdate:
13       maxSurge: 1
14       maxUnavailable: 0
15   template:
16     metadata:
17       labels:
18         app: lobby-api
19     spec:
20       containers:
21       - name: lobby
22         image: game-lobby:v2
23         readinessProbe:
24           httpGet:
25             path: /healthz
26             port: 9000
27           periodSeconds: 5
```

这段配置表达了三层意图。第一层是副本数：**replicas: 3** 告诉平台大厅服应长期保持 3 个目标 Pod。目标数量不等于可用数量：Pod 只有在成功调度、容器启动并通过就绪检查后，才算作可以接收流量的就绪副本。当某个 Pod 因节点故障消失时，Controller Manager 中的 Deployment 控制器会检测到实际数量低于期望值，并创建新的 Pod 来补位。

第二层是更新策略。**strategy** 字段定义了版本升级的节奏：**maxSurge: 1** 表示更新过程中最多比目标多出 1 个 Pod（即先启动新的，再下线旧的），**maxUnavailable: 0** 表示控制器以“不主动减少现有可用副本”为更新约束。只有在集群仍有可调度容量、新 Pod 能成功启动并通过就绪检查时，更新过程中才能维持不少于 3 个就绪副本；否则滚动更新会停滞，而不是凭配置自动获得可用容量。这种逐步替换而非一次性全换的方式就是滚动更新（Rolling Update）。

第三层是就绪判定。**readinessProbe** 告诉 kubelet 如何判断一个新启动的 Pod 是否已经准

备好接收流量。在上面的配置中，kubelet 每 5 秒向 Pod 的 9000 端口发送一次 HTTP 请求，只有收到成功响应后，该 Pod 才会被加入 Service 的后端列表。这解决了一个实际问题：容器进程启动成功不代表服务就绪，大厅服可能还在加载配置、建立数据库连接或预热缓存，如果此时就接入玩家流量，玩家会看到超时或错误。

对于游戏场景中的长连接服务（如 gateway 维护的 WebSocket 连接），滚动更新还涉及”排水”问题：旧 Pod 在下线前需要等待已有连接自然结束或主动迁移，而不是直接切断。Readiness Probe 能让平台识别”这个 Pod 是否适合接入新流量”，但存量连接何时结束取决于业务逻辑。K8s 提供了 preStop 钩子和 terminationGracePeriodSeconds 来给旧 Pod 留出排水时间，具体的优雅关闭策略需要应用代码配合实现。

与 readinessProbe 不同，livenessProbe 用于检测已经运行的 Pod 是否还活着；如果 livenessProbe 连续失败，kubelet 会重启该 Pod 内的容器，从而覆盖”进程假死但仍在运行”的盲区。

Service：稳定的访问入口 前面讨论容器网络时，Docker 的内置 DNS 解决了单机上的服务名解析问题：容器通过服务名找到对方的 IP。在集群环境中，原 Pod 消失后，控制器创建的替代 Pod 可能被调度到另一节点；新 Pod 的 IP 地址也可能变化。如果调用方将某个 Pod 的 IP 写死在配置中，Pod 一旦被替换，连接就会中断。

Service 解决的正是这个问题。它为了一组 Pod 提供一个稳定的访问入口（一个固定的集群内 IP 和 DNS 名称），调用方只需访问 Service 名称（如 lobby-api），平台负责将流量转发到当前存活且就绪的 Pod。

Service 如何知道应该把流量转给哪些 Pod？答案是标签（Label）与选择器（Selector）。在上面的 Deployment 配置中，每个 Pod 都带有 app: lobby-api 标签。Service 的定义则通过选择器指定它关心的标签：

大厅服 Service 配置示例

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: lobby-api
5  spec:
6    selector:
7      app: lobby-api
8    ports:
9      - port: 9000
10      targetPort: 9000
```

这段配置的含义是：凡是带有 app: lobby-api 标签且通过 Readiness 检查的 Pod，都会被自动加入这个 Service 的后端列表。当 Pod 因故障被销毁、新 Pod 被补位创建后，只要新 Pod 也带有相同标签并通过就绪检查，它会自动出现在 Service 的后端列表中，无需人工修改任何配

置。

这三个对象由标签和状态连接起来：调用方访问 Service，Service 通过 Selector 找到符合标签且已经就绪的 Pod，Deployment 持续保证这些 Pod 的数量和版本达标。当某个 Node 故障时，Deployment 在其他 Node 上补出新 Pod，新 Pod 通过就绪检查后被 Service 纳入后端列表。整个恢复过程中，调用方使用的入口地址始终不变。

Service 解决了”调用方访问谁”的问题，但还没有解释”不同 Node 上的 Pod 如何互相信”。K8s 的网络模型要求每个 Pod 拥有一个集群内唯一的 IP，且不同 Node 上的 Pod 能够直接互通。这部分底层机制（地址分配、跨节点路由、VXLAN 封装）由 CNI（Container Network Interface）插件承担，将在第 6.2 节结合网络虚拟化展开。

三个抽象各有分工：Pod 定义运行单元的边界，Deployment 定义副本数量与更新策略，Service 定义稳定的访问入口。对于本书的游戏后端，gateway、lobby-api、coordinator、battle 都可以按这个模式部署：每个组件对应一个 Deployment 和一个 Service，每个副本是一个 Pod，服务间通过 Service 名称互相访问。至于 redis 与 postgres 这类有状态组件，它们同样需要稳定入口，但对”可替换性”有额外限制（数据不能随 Pod 消失），这个取舍会在后续章节讨论。

实际部署中还需要解决跨环境配置差异：同一份镜像要在开发、测试、生产三个环境中运行，数据库地址和密码各不相同。K8s 用 ConfigMap 保存非敏感配置（服务地址、日志级别、超时时间），用 Secret 保存敏感配置（数据库口令、证书私钥），二者以环境变量或文件挂载的方式注入 Pod。这样镜像本身不必为不同环境重复构建，环境差异完全由外部配置对象承载。

上述对象已经覆盖了服务部署的核心需求：多节点分布、副本数量保障和稳定入口。ConfigMap 和 Secret 负责将环境配置与镜像解耦，可以独立创建后再注入 Pod。将这些对象组织成一条部署链路，可以进一步说明从集群资源准备到服务具备访问能力，各类对象依次承担什么职责。

部署链路：从空集群到可访问的服务 第一步，准备集群资源。控制面负责保存和协调集群状态，工作节点负责承载 Pod。教学环境可以使用单个控制面节点搭建最小集群；生产环境通常采用高可用控制面，或使用云平台托管的控制面，避免控制节点成为单点故障。工作节点注册后，平台可以通过统一的状态视图管理整组计算资源。

第二步，声明 Deployment。运维人员编写一份 YAML 文件，指明服务需要运行几个 Pod 副本、使用哪个镜像、每个副本请求多少 CPU 和内存。将这份文件提交给 API Server 后，Controller Manager 会检测到实际副本数与期望值的差距，并推动创建新的 Pod。

第三步，声明 Service。由于 Pod 的 IP 可能随重建而变化，直接访问 Pod 不可靠。运维人员再编写一份 Service YAML，通过标签选择器把一组 Pod 绑定到一个固定的集群内 DNS 名称和虚拟 IP 上。集群内部的其他服务通过这个 DNS 名称互相访问，不受 Pod 重建的影响。

第四步，提供受控的外部入口。集群内的 Service 默认只在内部网络可达。生产环境通常在集群外设置统一入口，由它承接公网地址、TLS 终止和请求路由，再把流量导向集群内的 Service；常见实现包括云负载均衡、Ingress 或 Gateway。NodePort 直接在节点上暴露端口，更适合教学实验或作为外部入口的底层接入方式，不应让客户端长期依赖某个工作节点的公网地址。无论采

用哪种实现，外部流量都应先经过稳定入口和 Service，再到达当前就绪的 Pod。

这四步完成后，服务已经具备基本的可访问性。但发布到一半突然出错、节点宕机、镜像拉取失败时，系统到底处于什么状态，又靠什么机制回到正常？这正是声明式模型与调谐循环要解决的问题。

5.2.5 资源调度：放置决策与资源约束

前面讨论的副本故障补位、版本滚动更新和启动就绪检查三种场景，都涉及控制器创建新 Pod。在前面的分析中，这些新 Pod 默认被放在“可用 Node 上”启动。但集群通常有多台 Node，资源余量各不相同，已有的负载分布也不同。如果放置不加控制，两类问题会传导到玩家侧：一是控制器已经决定补副本，却没有任何一台 Node 的剩余资源能承载这个 Pod，Pod 停留在待调度状态，玩家看到容量始终不恢复；二是 Pod 被放到已经承载了大量同类副本的 Node 上，该 Node 一旦故障，对局大面积中断。

K8s 通过调度器 (Scheduler) 解决这一决策。控制器负责决定“做什么” (补副本、替换版本)，调度器负责决定“放在哪台 Node 上”。每个新创建的 Pod 先进入调度队列，调度器根据 Pod 声明的资源需求和集群各 Node 的当前状态，选定一台目标 Node 并完成绑定。调度器的决策质量取决于两个前提：Pod 是否准确声明了自己的资源需求，以及调度器能否获取各 Node 的实时可分配资源。下面先讨论第一个前提。

资源声明：requests 与 limits 调度器判断节点是否能够承载一个 Pod，主要依据 Pod 对资源需求的显式声明。如果 Pod 没有声明资源需求，调度器就无法按照其真实消耗预留容量，可能把过多工作负载放到同一节点。K8s 用两个字段承载这一声明：**requests** 和 **limits**。

requests 表示 Pod 在调度时申请预留的资源量 (CPU 和内存)。调度器在选择 Node 时，会计算该 Node 上所有已有 Pod 的 **requests** 总和，只有剩余可分配量大于等于当前 Pod 的 **requests**，该 Node 才进入候选。**limits** 表示 Pod 运行时允许使用的资源上限。它不参与调度决策，而是在运行时由 Cgroups 执行：CPU 超出 **limits** 时进程被限速 (throttle)，内存超出 **limits** 时进程被终止 (OOM Kill)。

两个字段的分工可以这样理解：**requests** 决定 Pod 能否被放上某台 Node (调度阶段)，**limits** 决定 Pod 放上去之后能消耗多少资源 (运行阶段)。如果 **requests** 设置得过低，调度器会低估 Pod 的真实消耗，多个 Pod 被叠加到同一台 Node 上，运行时出现资源争用；如果 **limits** 设置得过紧，Pod 在负载升高时频繁被限速或终止。前面隔离一节介绍的 Cgroups 正是 **limits** 的底层执行机制：**limits** 字段的值最终转化为 Cgroups 的 CPU 时间片上限和内存上限配置。

过滤与打分：调度器的两步决策 当一个新 Pod 被创建并进入调度队列后，调度器按两步完成放置。

第一步是过滤 (Filtering)。调度器逐一检查集群中的每台 Node，排除不满足条件的 Node。过滤条件包括：该 Node 的剩余可分配资源是否大于等于 Pod 的 **requests**；该 Node 是否被标记为不可调度 (例如正在维护)；Pod 是否声明了特殊约束 (例如必须运行在配备 GPU 的 Node

上)。过滤结束后，剩余的 Node 构成候选集合。如果候选集合为空，Pod 停留在 Pending 状态，直到有 Node 释放出足够资源。

第二步是打分 (Scoring)。调度器对候选集合中的每台 Node 评分，选择得分最高的一台作为目标。得分由启用的评分插件共同决定，可以考虑资源利用与均衡、亲和或反亲和规则、拓扑分布等因素，并不存在对所有集群都成立的单一“剩余资源越多越优”规则。

`battle` 服务的一次补位调度可以把上述过程具体化。待调度 Pod 声明了 `requests: {cpu: 2, memory: 4Gi}`，集群中有三台 Node；调度器先过滤掉资源不足或不满足约束的 Node。本例假设评分策略同时偏好保留资源余量和分散同类副本，因此 Node-C 得分最高，Pod 最终绑定到 Node-C。这次放置结果说明，调度由硬约束过滤和软目标排序共同产生；图 67 用一次完整决策验证这一点。

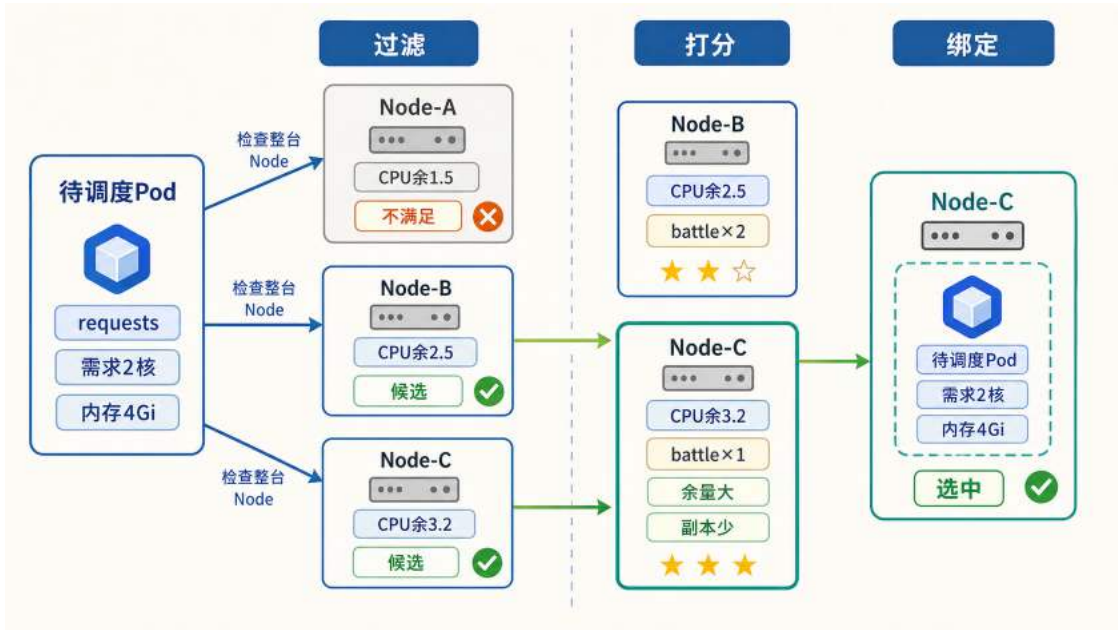


图 67: 调度器的过滤、打分与绑定过程: Node-A 无法容纳完整 Pod 而被排除, Node-B 和 Node-C 进入打分, Node-C 因资源余量更大且已有 `battle` 副本更少而被选中。

这个例子揭示了一个重要约束：调度器比较的不是集群的资源总量，而是每台 Node 能否独立容纳整个 Pod。Node-B 和 Node-C 都通过了资源门槛，但 Node-C 放置后仍有更大余量，且已有同类副本更少，因此更符合分散放置的目标。这些约束并不意味着默认调度器能够找到全局性能最优的放置结果。调度器依据工作负载已经声明的资源需求、拓扑要求和评分规则，从当前可行节点中选择目标；端到端延迟、数据本地性或专用硬件利用率等目标，可能还需要更精确的指标、专用调度策略或自定义调度器。资源声明与拓扑约束提供的是可执行的放置条件，而不是对运行性能的完整预测。

资源碎片 上述约束带来了一种常见的调度失败场景：集群中所有 Node 的剩余资源总和足够承载一个新 Pod，但没有任何一台 Node 单独满足 Pod 的 `requests`。例如 `battle` 的每个副本需要 2 核 CPU 和 4GiB 内存，但集群中三台 Node 分别只剩余 1.5 核、1.2 核和 1.8 核 CPU。三

台 Node 的余量之和为 4.5 核，远超单个副本的需求，却没有任何一台达到 2 核的门槛。此时控制器已经发出补充副本的指令，但 Pod 停留在 Pending 状态无法运行。从集群监控面板看整体利用率并不高，但玩家侧已经感知到匹配队列变长、开局延迟上升。

这种现象称为资源碎片。其根本原因在于 Pod 必须完整地运行在一台 Node 上，不能跨 Node 拆分资源。集群的有效可调度容量不等于各 Node 剩余资源的简单加总。

分散放置与拓扑约束 资源约束决定 Pod 能否被放置，但放置决策还需要考虑故障隔离。如果同一个 Deployment 的多个副本被集中放置在同一台 Node 上，该 Node 宕机时所有副本同时消失。以 `battle` 为例：6 个副本中若有 4 个运行在 Node-B 上，Node-B 故障会导致 4 个副本同时不可用，正在进行的对局大面积中断。如果副本均匀分散到 3 台 Node 上，同样的单机故障只影响 2 个副本，冲击范围更可控。

K8s 提供两类相关机制。Pod 反亲和（Pod Anti-Affinity）描述哪些工作负载不应集中在同一故障范围内，可以把同类副本分散到不同节点。拓扑分布约束（Topology Spread Constraints）进一步描述副本在节点或可用区之间应当保持多大程度的均匀，例如限制不同可用区的副本数差值不能超过给定范围。前者回答“哪些 Pod 应当彼此分开”，后者回答“副本应当在多个故障范围之间分布得多均匀”。二者共同降低单个节点或单个可用区故障时同时丢失大量副本的风险。

调度作为控制面的前置环节 控制器决定“做什么”（补副本、替换版本），调度器决定“放在哪台 Node 上”。但调度器本身并不直接操作节点，它只把调度结果写回 API Server；真正在目标节点上创建容器的是 kubelet。那么，API Server、etcd、Controller Manager、Scheduler 和 kubelet 之间是如何协作的？这正是 Kubernetes 控制面要回答的问题。

5.2.6 Kubernetes 的控制面与协作链路

Swarm 的 Manager 将 API 接收、状态存储、调度和健康检查四项职责集中在同一类角色中。K8s 将这四项职责拆分为独立的组件，每个组件只承担一项任务，组件之间通过统一的枢纽协调，而不是在同一进程内直接调用。

K8s 的核心控制面通常由四类组件构成。**API Server** 是集群中所有操作的唯一入口：部署、扩容、查询、删除等一切请求都必须经过它的接收和验证，其他组件不直接相互通信，而是统一通过 API Server 读写集群状态。**etcd** 是一个独立部署的分布式键值存储，作为 API Server 的后端存储，保存集群中的资源对象和状态信息。将存储独立出来意味着即使 API Server 或其他组件重启，集群的配置信息和运行记录不会丢失。**Scheduler** 只负责放置决策：当一个新的工作负载需要运行时，它根据各节点的 CPU 和内存余量、亲和性规则和约束条件，为其选择一个合适的目标节点。选完之后 Scheduler 的工作就结束了，它不参与后续的容器启动过程。**Controller Manager** 负责“期望状态与实际状态持续对齐”这一控制循环。前面在 Swarm 中已经接触过这种思路，K8s 将其进一步细化为多个独立控制器，每个控制器负责一类资源的对齐：Deployment 控制器持续检查副本数是否达标，Node 控制器持续检查各节点是否存活。控制器发现偏差时不直接操作节点，而是向 API Server 提交纠正请求，由后续组件完成执行。

节点执行面的职责由 **kubelet** 承担。每个工作节点上运行一个 kubelet 进程，它持续从 API Server 获取分配给本节点的容器列表，调用容器运行时完成创建或销毁，并向 API Server 汇报节点和容器的健康状态。kubelet 是控制面决策的最终执行者。

Swarm 与 Kubernetes 都把控制职责和节点执行职责分开，但 Kubernetes 进一步拆分了状态入口、持久化、调度和状态调谐。图 68 用两种控制结构说明这种职责粒度的差异，而非比较产品优劣。

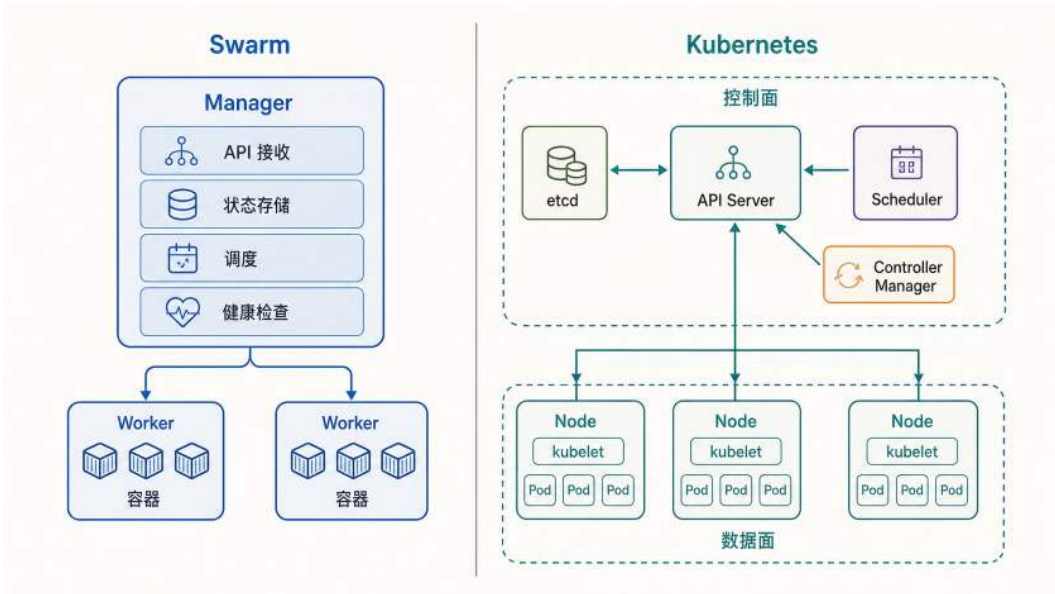


图 68: Swarm 与 Kubernetes 控制结构对比: Swarm 的 Manager 角色集中承担主要控制职责, Kubernetes 将其拆分为 API Server、etcd、Scheduler 和 Controller Manager 等独立组件, 工作节点由 kubelet 执行控制面形成的放置结果。

以上是各组件的独立职责。为了看清它们如何协作，以一个具体场景为例：运维团队决定将战斗服从 2 个副本扩容到 4 个副本。

在 Swarm 中，运维人员执行 `docker service scale battle=4`，请求进入 Manager 后，由 Manager 角色内的编排逻辑统一完成验证、状态更新、节点选择和任务下发。

在 K8s 中，同一个扩容意图经过的路径截然不同。运维人员执行 `kubectl scale deployment battle --replicas=4`，请求首先到达 API Server。API Server 验证请求合法后，将“期望副本数 = 4”写入 etcd。此时集群状态已变更，但还没有任何实际动作发生。

接下来由 Controller Manager 接手。它内部的 Deployment 控制器通过 API Server 观察 Deployment 的状态，检测到期望副本数为 4 而当前副本不足，于是创建新的 Pod 对象来补位。这些 Pod 处于“待调度”状态，尚未绑定到具体节点。

Scheduler 随即检测到存在待调度的 Pod，为每个 Pod 选择目标节点，并将调度结果写回 API Server。至此，控制面的决策环节全部完成，剩下的是执行。

目标节点上的 kubelet 从 API Server 获取到本节点新增的 Pod 信息，拉取战斗服镜像、创建容器并启动，随后持续向 API Server 汇报运行状态。从运维人员发出命令到新副本上线，整个过程由多个组件接力完成，每个组件只处理自己职责范围内的一步。

这个过程中有一个关键的设计原则：每个组件只在自己的职责范围内行动，通过修改 API Server 中的状态来触发下一个组件的响应。Controller Manager 不向 Scheduler 直接发出调度指令，Scheduler 也不向 kubelet 直接发出启动容器的命令。所有协作都通过状态变更驱动，而非直接的命令调用。这种松耦合使得单个组件可以独立重启或升级，短暂的不可用不会导致其他组件连锁故障：重启完成后，组件从 etcd 中读取当前状态即可恢复工作。

对于游戏后端运维而言，这种架构打开了 Swarm 无法提供的可能性。Scheduler 可以替换为自定义实现：例如将同一房间内的战斗服副本调度到网络延迟最低的节点组合上，这在 Swarm 的固定调度逻辑中做不到。etcd 独立部署为高可用集群后，状态存储不再绑在某一个控制角色内部，而是成为可单独维护和备份的基础组件。控制面的可拆分设计还意味着团队可以根据业务需要替换或扩展其中的某个组件，而无需修改平台本身的代码。

至此，对象定义、协作关系和放置决策已经建立。但上述协作只说明了“一次扩容”的执行路径。Deployment 的副本数声明为什么能被持续维持？Pod 故障后为什么能自动补位？下一节将分析声明式管理与调谐循环。

5.2.7 声明式管理与调谐循环

多机环境下，发布与运维最棘手的问题不是“如何执行操作”，而是**操作执行到一半失败后，系统到底处于什么状态**。

考虑一个典型场景：lobby-api 在高峰时段出现间歇性超时，运维人员决定紧急发布热修版本。发布脚本按顺序执行：拉取新镜像、逐台重启进程、更新后端列表。替换进行到第三台时，一台 Node 短暂不可达，两个实例拉镜像失败，另外三个新实例启动后未能通过健康检查。此时玩家侧的表现是：部分用户能正常登录，部分用户持续等待，部分用户进入房间后偶发失败。故障难以归因，因为系统既不是完全可用，也不是完全不可用，而是处于一种无法描述的中间状态。

这类故障的根源在于命令式发布的基本假设：脚本描述的是步骤序列，每一步依赖前一步的成功。一旦中途失败，系统无法回答“哪些步骤已生效”“哪些步骤被跳过”“当前整体是否可用”。为了应对中途失败，运维流程不得不叠加探测、重试、回滚和补偿逻辑。节点越多，失败点越多，这层补偿逻辑的复杂度增长不可控。

解决这一问题的思路是改变描述维度：不再追踪“上一步执行到了哪里”，而是持续回答三个问题。第一，**目标是什么**（例如大厅服期望保持 3 个可用副本）；第二，**现状是什么**（当前有几个副本可用、入口指向了谁）；第三，**差距是什么**（离目标还差几个、哪些需要补建）。一旦以目标与现状的差距作为驱动，系统在任意时刻都只需要缩小当前差距，而不需要追溯历史步骤或判断中断发生在哪一环。

这就是**声明式管理** (Declarative Management) 的出发点：用户提交**期望状态** (Desired State)，平台负责将现实状态持续拉向目标。**调谐循环** (Reconciliation Loop) 是实现这一目标的执行机制：控制器在后台不断重复“观察现状 → 对比差距 → 执行纠偏 → 再次观察”的过程。它将“失败”重新定义为“与目标之间的偏差”，不再依赖人工判断应该补执行哪一步。只要纠偏动作满足幂等性（对同一目标重复执行与执行一次的效果相同），系统就能在多次尝试后收敛到目标状态。

声明式管理成立的关键是目标状态能够被持久化，纠偏动作通过统一状态接口接力，实例是否接收流量则由就绪状态决定。故障发生后，同一组组件重新进入循环，而不需要恢复某条命令执行到一半的位置。图 69 将控制面决策与节点执行连成这一持续纠偏闭环。

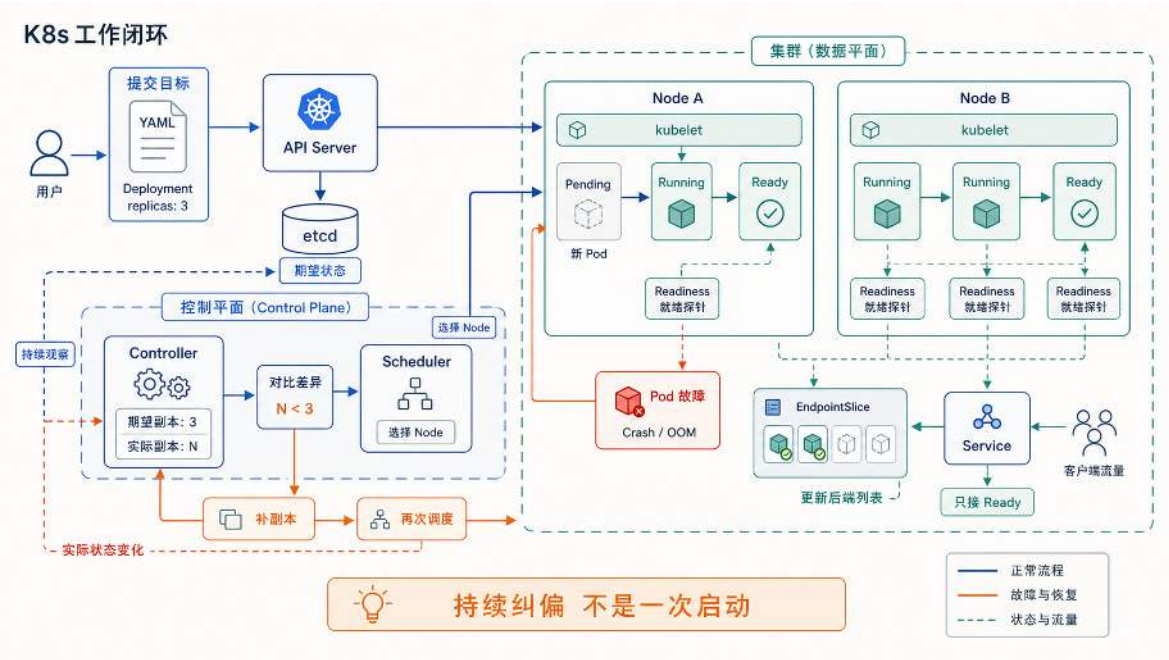


图 69: K8s 工作闭环：用户提交目标后，控制面持续观察、对比差异、调度补位，数据面执行就绪检查并更新后端列表，形成完整的持续纠偏回路。

控制闭环建立后，命令式与声明式管理的差别集中在恢复责任上。图 70 将两种模式置于同一中途失败场景，比较由谁判断现状并继续恢复。

命令式流程一旦中途失败，运维人员必须人工判断应从哪一步恢复；声明式流程则将目标状态持久化在控制面中，由控制器围绕“观察现状”“对比差距”“执行纠偏”形成持续回路，发现现实状态偏离目标时自动纠偏。这种持续纠偏的行为类似恒温器的工作方式：用户设定目标温度，系统根据当前温度决定加热或停止，具体操作几次、每次持续多久由控制回路自行决定。对应到 K8s 中，目标温度就是 Deployment 的期望副本数，当前温度就是实际运行的 Pod 数量，加热或停止就是创建或删除 Pod 的纠偏动作。

在这个模型下，平台的自愈能力有了清晰的因果链：期望状态被持久化在控制面（etcd）中，控制器持续对比现实与期望，偏差触发纠偏动作。Deployment 负责将副本数量和版本拉回目标，Service 负责提供稳定入口并维护后端 Pod 列表。两条因果关系构成自愈的基础：目标被显式记录，以及平台持续将现实拉向目标。

5.2.8 调谐循环的运行时行为

本节以 lobby-api（大厅服）为例，分析声明式约束在运行时的具体行为。假设初始状态如下：Deployment 声明 replicas: 3，三个 Pod 分布在不同 Node 上，Service 通过标签选择器定义访问范围，Service 将匹配且就绪的 Pod 纳入其后端列表，调用方通过 Service 名称 lobby-api

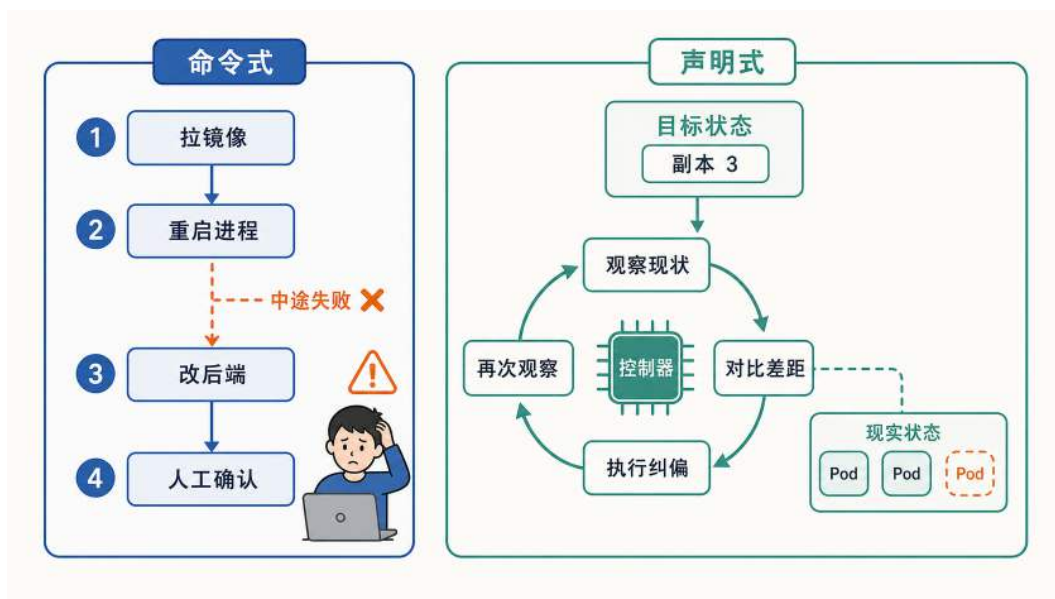


图 70: 声明式管理与命令式运维的对比：前者描述目标并持续纠偏，后者依赖按步骤执行和人工补偿。

访问大厅服。从这一稳态出发，下面依次考察副本故障、版本升级和启动就绪三个场景。

场景一：副本异常退出 假设其中一个 Pod 因内存溢出 (OOM) 被操作系统终止。此时集群中还剩两个正常运行的 Pod，第三个已经消失。Deployment 控制器在下次对比中发现实际副本数 (2) 低于期望值 (3)，随即创建一个新 Pod，调度器为其选择目标 Node 后由 kubelet 启动。新 Pod 启动并通过 Readiness Probe 后，其就绪状态会反映到 Service 的后端列表中；与此同时，已终止的 Pod 会从该列表中移除。对调用方而言，整个过程中访问的入口始终是 Service 名称 lobby-api，后端成员的变化不影响调用方的配置和代码。

这个过程揭示了副本数约束的关键性质：它不是部署时执行一次的指令，而是控制器持续维持的不变式。只要实际值偏离期望值，补位动作就会被触发，无论偏离的原因是节点宕机、进程崩溃还是人工误删。如果 Pod 并未完全终止，而是处于进程假死状态（进程仍在但不再响应请求），livenessProbe 的连续失败会触发 kubelet 重启容器，随后 Deployment 控制器检查副本数是否恢复达标。无论触发原因是 OOM 终止还是 livenessProbe 重启，调谐循环的最终效果都是将实际状态拉回期望状态。

场景二：滚动升级 假设大厅服需要从 v1 升级到 v2。运维人员将 Deployment 中的镜像字段从 game-lobby:v1 改为 game-lobby:v2，控制器随即开始执行升级。如果一次性终止全部 3 个旧副本再启动新副本，升级期间没有可用实例，玩家侧表现为集中断线和登录失败。滚动更新策略避免了这种中断：控制器将升级拆分为多轮，每轮只替换一部分副本，在任意时刻都保持足够数量的可用实例承载流量。替换节奏由两个参数控制：maxSurge 规定过程中允许临时超出目标数量的上限，maxUnavailable 规定允许暂时不可用的上限。在前面 YAML 示例的配置下 (maxSurge=1, maxUnavailable=0)，控制器先启动一个新版本 Pod，待其就绪后再终止一个旧

版本 Pod，如此循环直到 3 个副本全部替换完毕。maxUnavailable=0 保证在整个过程中始终有不少于 3 个就绪副本承载流量。

滚动更新将升级从瞬时切换变为可控过程：Deployment 管理的不只是最终的目标版本，还包括到达目标版本的过程中可用性的下限。

场景三：启动但未就绪 场景一和场景二都涉及新 Pod 的创建。一个容易被忽略的问题是：Pod 的进程启动成功，不代表它已经能够正常处理请求。大厅服启动后通常还需要加载配置、建立数据库连接池、预热本地缓存，这些初始化步骤可能耗时数秒。如果平台在进程刚启动时就把该 Pod 标为 Service 的可用端点，调用方的请求会被转发到一个尚未准备好的实例上，导致超时或错误。

Readiness Probe 解决的正是这个问题。它在 Pod 与 Service 之间充当闸门：kubelet 按配置的频率（前面 YAML 中为每 5 秒）向 Pod 发送探测请求，只有探测成功后，该 Pod 才会在 EndpointSlice 中成为可接收流量的端点。未通过就绪检查的 Pod 不会接收 Service 流量，启动阶段的不确定性不会传导到调用方。

Readiness Probe 与 Liveness Probe 是两个独立的判断：Liveness Probe 判断进程是否仍能正常运行、是否需要被重启；Readiness Probe 判断进程是否可以接收流量、加入服务。一个 Pod 可能处于“存活但未就绪”的状态（正在初始化），也可能已经通过就绪检查，但随后因死锁、内存泄漏或探测失败被 Liveness Probe 重启。两个探针各自独立工作，分别触发不同的平台动作。

三个场景共同展示了声明式约束在运行时的行为模式：副本数约束驱动控制器持续补位，滚动更新约束控制替换节奏，就绪探针将未准备好的实例隔离在入口之外。系统是否已收敛到期望状态，可以通过两个可观测指标判断：副本数是否达标，以及 Service 后端列表中可用端点的数量和状态是否符合预期。

将同样的分析框架应用到 **battle**（对局服务）时，约束之间的关系会更复杂。副本不足时匹配队列增长、开局延迟上升；滚动替换时如果没有排水机制，正在进行的对局可能被中断。平台能维持副本数量和控制替换节奏，但对局是否允许被中断、中断后状态如何恢复，必须由业务逻辑定义。平台提供基础设施层面的保障，业务语义层面的约束需要应用代码配合实现。

上述三个场景都以已经声明的副本数和版本为目标，调谐循环负责在状态偏离后恢复这一目标。负载持续变化时，系统还需要动态判断目标副本数和底层节点容量应当如何调整，管理对象由“维持既定状态”进一步扩展为“根据运行信号修正目标状态”。

5.3 弹性运行：伸缩与自愈

编排已经解决了多服务在集群中的启动、互联、副本维护和放置问题。但线上系统还要面对两类持续变化：负载升降会改变所需容量，进程或节点故障会破坏现有实例。平台需要在这些变化中维持可度量的服务质量，同时控制资源成本。平台可以为延迟、错误率、可用性或队列等待时间等指标设定目标值，这类目标称为服务级别目标（Service Level Objective, SLO），并为容量调整和故障恢复提供判断标准。弹性运行由两条相互关联的链路组成：自动伸缩根据负载调整

实例与节点数量，故障自愈根据健康状态修复失效实例。前者处理容量偏差，后者处理健康状态偏差，共同服务于既定的 SLO。

5.3.1 伸缩调度：固定容量与波动负载的矛盾

在线系统的负载通常具有时间结构：活动开场、促销开始、协作高峰或版本更新上线都可能在短时间内使请求量成倍上升，而低谷时段又可能长时间处于低利用率。固定容量以空闲成本换取即时承载能力，弹性容量减少低谷浪费，却引入了检测和扩容时延。图 71 将这一取舍置于同一负载时间轴上；当负载上升速度超过扩容速度时，请求积压和尾部延迟仍可能被放大。

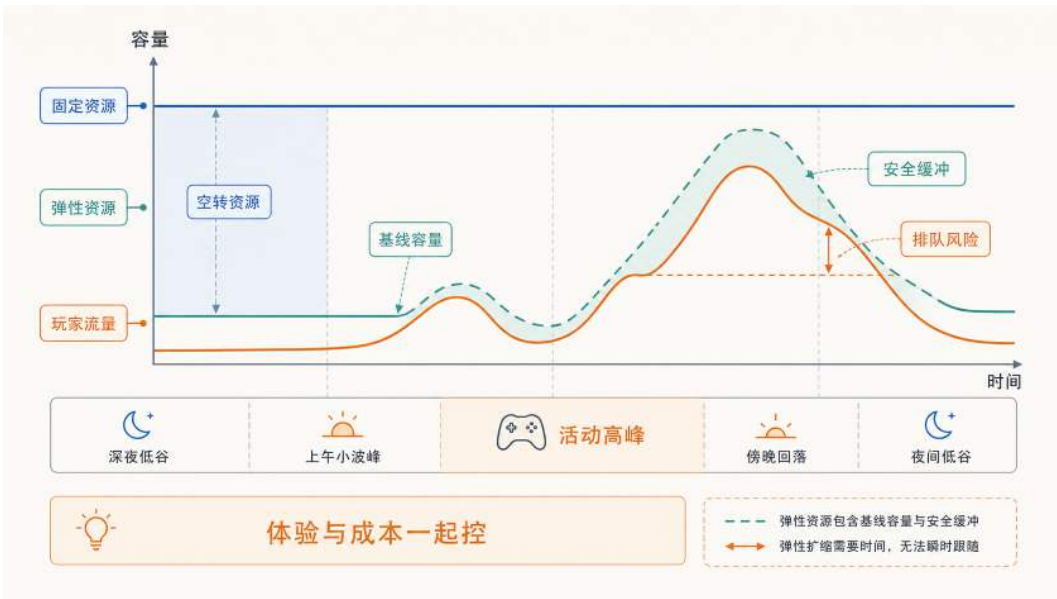


图 71: 固定容量与弹性容量的资源利用对比：固定容量按峰值预留时会在低谷期产生空转浪费；弹性容量根据实际负载动态调整，并通过安全缓冲吸收高峰波动，但扩缩容存在时间滞后，突发变化仍可能带来排队风险。

以一个具体场景说明这种矛盾的量级。假设活动高峰期同时在线 5 万人，需要 50 台服务器支撑；而工作日上午在线人数降至 5000，5 台即可满足。如果始终维持 50 台，90% 的时间中 90% 的资源处于空转状态。如果按平均负载配置（例如 15 台），高峰期的请求到达速率将远超处理能力，客户端出现连接超时和响应失败。

这一矛盾不仅来自周期性的峰谷波动，还来自不可预测的突发事件。一次外部推荐或运营活动可能在数分钟内使请求量增长数十倍，远超任何基于历史均值的静态预估。突发流量对玩家体验的冲击比持续性高负载更严重：大量用户在同一时间窗口内集中遭遇登录失败或操作超时，对服务可用性的感知迅速恶化。

非线性放大：排队论视角 容量不足的后果并非线性增长。当请求到达速率逼近服务处理能力的上限时，队列长度和尾部延迟可能迅速放大。其原因是每个请求的处理时间存在波动：当系统缺少空闲处理能力时，偶发的慢请求无法被及时消化，积压逐步累积，后续请求的等待时间随队列增长而延长。

排队论中的一个经典结论是：当系统利用率接近 1 时（到达速率接近处理速率），平均等待时间会随利用率急剧上升。利用率越高，可用于吸收服务时间波动的空闲容量越少，偶发的处理延迟越容易转化为持续积压。

因此，扩缩容不能只在错误率升高后才启动，还应关注排队长度和等待时间等更早出现的过载信号。第 6.4.2 节将结合 Little 定律说明这些信号如何进入弹性控制闭环。

体验恶化的时间顺序 将上述机制投射到一次活动开场的时间线上，可以观察到体验恶化的典型顺序：首先是请求到达速率和活跃连接数上升；当到达速率超过处理能力后，请求开始排队；排队累积导致响应时间逐步拉长，P95/P99 等尾部延迟指标率先恶化；当积压持续加深，部分请求的等待时间超过客户端设定的超时阈值，错误率开始上升。

这一顺序意味着：当客户端开始出现响应缓慢时，系统往往已经进入排队累积阶段，但尚未触发错误率告警。弹性伸缩的目标是在排队累积的早期阶段介入，在尾部延迟恶化传导到错误率之前补充处理能力。

但负载的变化是持续且不可预测的：高峰可能在几分钟内到来，也可能在活动结束后迅速回落。一次性的手工扩容只能应对已经发生的过载，无法跟随后续的波动。要在变化的负载下持续维持容量与需求的匹配，系统需要一种能够反复执行“观察当前状态 → 判断是否偏离目标 → 调整容量 → 再次观察”的机制。这种持续修正偏差的结构在控制论中称为反馈闭环（feedback loop）。Kubernetes 的弹性伸缩正是基于这一机制构建的。

5.3.2 应用层自动扩缩：HPA 与 VPA

在手工运维模式下，扩容是一组顺序操作：启动新的服务器实例，部署同版本的服务，将新实例注册到负载均衡的后端列表；缩容则需先将目标实例从后端列表中移除，等待其上的存量请求或会话结束，最后终止进程。整个过程由运维人员判断时机、执行动作、观察效果，本质上是一个人工驱动的控制过程。

Kubernetes 通过 Horizontal Pod Autoscaler (HPA) 将这一过程自动化。运维人员不再逐步操作，而是声明一个目标状态（例如“将 lobby-api 的平均 CPU 利用率维持在 60% 附近”），HPA 持续将实际状态向目标靠拢。从使用者角度看，HPA 需要声明目标指标、目标值以及最小和最大副本数；平台随后反复完成指标采集、容量计算、实例创建或回收和结果再观察。控制器怎样处理容忍区间、稳定窗口和执行滞后，将在第 6.4 节从内部机制展开。

计算逻辑 HPA 的核心计算逻辑是按比例换算：当前指标高于目标时增加副本，低于目标时减少副本，偏差越大，期望调整幅度通常也越大。具体计算并不等同于容量立即生效，新副本仍需经过调度、启动和就绪检查。

信号选择 HPA 的决策质量取决于输入信号与玩家体验之间的相关性。CPU 利用率是最常用的默认指标，优点是获取成本低且无需应用改造，但它与用户可感知的延迟之间并非线性对应。对入口服务，更直接的信号可能是活跃连接数、请求到达速率（QPS）或 P95/P99 延迟；对匹

配服务，等待队列深度比 CPU 更能反映用户等待时间；对战斗服务，活跃房间数和逻辑帧耗时与玩家体验的关联更紧密。

信号选择的本质是确定闭环保护的对象：选择 CPU 意味着闭环保护的是计算资源余量，选择请求延迟意味着闭环直接保护用户响应时间。当 CPU 尚有余量但延迟已经升高时（瓶颈在网络 I/O 或下游依赖），以 CPU 为信号的 HPA 不会触发扩容，用户体验持续恶化。因此，信号的选择决定了闭环能覆盖哪些过载场景。

闭环的两类约束 HPA 的工程实现面临两类固有约束。第一类是**信号噪声**：负载指标存在短时波动，如果控制器对每次波动都响应，系统会频繁扩缩产生振荡。第二类是**执行时延**：从计算期望副本数到新 Pod 实际承接流量，中间还要经历调度、镜像拉取、进程启动和就绪检查，耗时可能从数秒到数十秒。两类约束共同决定了灵敏度设置的核心矛盾：过高则振荡，过低则滞后。控制器如何通过容忍区间、稳定窗口和冷却时间在这之间取得平衡，属于平台内部的实现细节，将在第 6.4 节展开。

扩容的边界 HPA 通常配置最小和最大副本数：最小值确保低负载时段仍保留基本冗余，最大值受集群资源和成本预算约束。即使 Pod 数量增加到最大值，如果性能瓶颈位于下游依赖（数据库连接池耗尽、缓存命中率下降、某个分区成为热点），增加入口层副本无法缓解这些瓶颈。此时需要配合限流、降级或故障隔离等手段保护下游依赖，而非继续扩容入口层。

另一条路径：垂直扩缩 (VPA) HPA 沿水平方向调节容量：负载升高就增加副本，负载回落就减少副本。但有一类容量问题不是副本数不够，而是单个副本的资源规格本身不合适。如果一个 Pod 的内存 `requests` 估得过低，它会在负载升高时频繁触发 OOM 被重启；估得过高，则每个副本长期占用超过实际需要的资源，压低整台 Node 的部署密度。这类问题靠增减副本数无法解决，需要调整的是单副本的 `requests` 和 `limits`。承担这一职责的机制是垂直扩缩 (Vertical Pod Autoscaler, VPA)：它观察 Pod 的历史资源使用情况，自动调整其资源请求与上限，使规格贴近真实消耗。VPA 与 HPA 的分工可以这样区分：HPA 回答“需要多少个副本”，适合可水平扩展的无状态服务；VPA 回答“每个副本该配多大”，适合副本数不宜频繁变动、但规格需要校准的服务（如前面提到的 Redis、PostgreSQL 这类有状态组件）。需要注意的是，VPA 自动改变规格通常要重建 Pod（新规格只能在新 Pod 上生效），对在线服务意味着一次重启。因此在延迟敏感的实时链路上，VPA 更常配置为只给出资源建议、由人工在发布窗口采纳，或仅用于可以承受重启的非实时服务；真正“边运行边改规格”的原地调整能力仍受运行时和平台支持程度限制。

均值掩盖局部过载 HPA 的默认决策基于所有副本的指标均值。当负载分布不均时（例如少数副本因会话粘性或热点用户承受了远高于均值的负载），均值可能仍在目标范围内，HPA 不会触发扩容。结果是多数用户体验正常，少数用户持续遭遇高延迟。要覆盖这类局部过载场景，需要使用更精细的指标（如分位数指标或单副本最大值）或结合负载均衡策略调整流量分配。

HPA 在扩容方向的逻辑相对直观：负载高就加副本。缩容方向的问题更为复杂：扩容不足的后果是响应变慢，而缩容失误的后果可能是用户连接直接中断。

5.3.3 安全缩容：为什么撤掉容量更危险

扩容不足的后果主要是响应变慢：玩家等待时间增加，已建立的连接通常不会立即中断。缩容失误的后果则严重得多：如果一个正在处理请求或承载会话的 Pod 被直接终止，用户看到的是连接断开、对局中断或数据丢失。因此，缩容的工程复杂度不在于计算“应该减少多少副本”，而在于如何安全地将一个正在服务的副本从系统中移除。

安全缩容不是直接删除实例，而是先改变流量状态，再排空存量工作，最后回收容量。图 72 建立了这一不可颠倒的顺序。

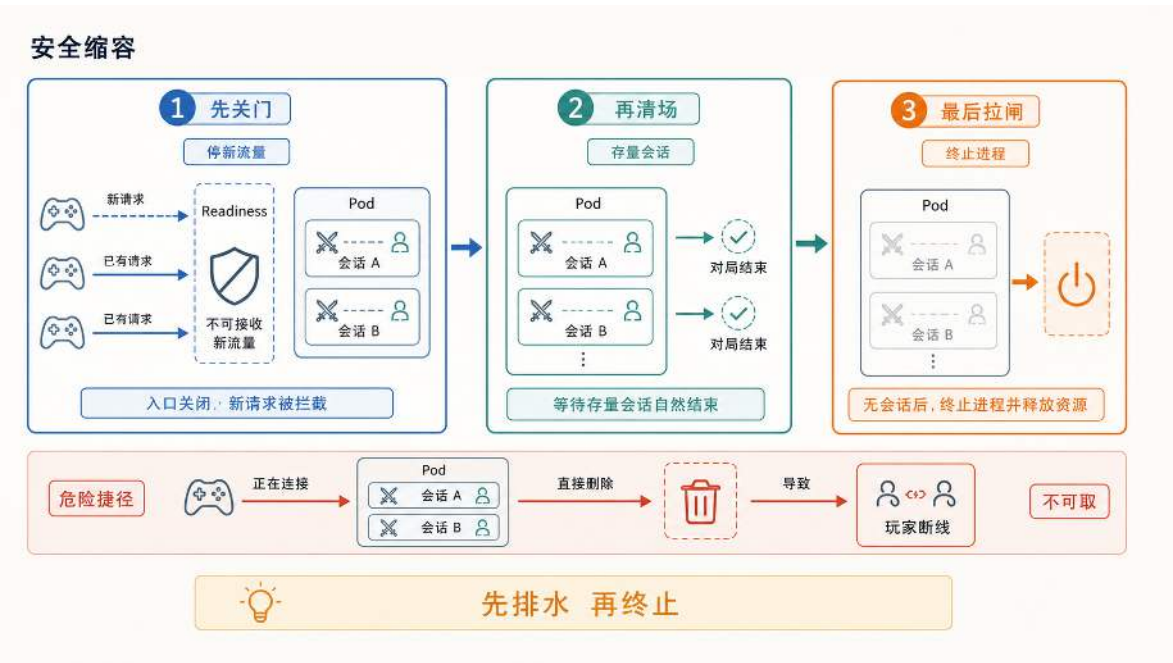


图 72: 安全缩容的基本顺序：先停止接收新流量，再等待存量请求或会话结束，最后终止进程。

三个阶段依次是：停止接收新流量，将实例从服务入口中移除；等待存量请求或会话自然完成；最后终止进程并回收资源。对短请求服务，排空可能只需毫秒级；对有状态或长连接服务（如承载对局的战斗服），则需要等待对局结束或配合会话迁移，耗时远更长。平台提供优雅终止的时序保证，但“实例上的工作何时可以安全中断”属于业务语义，需要应用层定义退出条件。平台机制与应用退出逻辑如何配合，将在第 6.3 节从应用设计角度展开。

HPA 在负载回落时会计算出更小的期望副本数，触发上述缩容流程。但 HPA 能够扩容的前提是集群中存在足够的可用资源。当所有 Node 的剩余资源都无法满足新 Pod 的 requests 时，Pod 会进入 Pending 状态，扩容在 Pod 层面无法继续推进。解决这一问题需要将扩缩从 Pod 层面延伸到 Node 层面。

5.3.4 基础设施层自动扩缩：Cluster Autoscaler

HPA 决定服务需要多少 Pod 副本，但副本能否实际运行取决于集群是否存在满足资源、标签和拓扑约束的 Node。Pod 无法调度时，Cluster Autoscaler 会评估预先配置的节点组；只有新增某类 Node 确实能够使 Pod 可调度，才会向云平台申请机器。Node 创建和加入集群通常比 Pod 启动更慢，缩容还要确认工作负载能够迁走，因此工程上仍需保留一定节点余量。HPA、调度器与节点扩容器怎样分层协作，将在第 6.4 节展开。

容量规划：基线与按需的两段式策略 弹性机制解决了容量如何跟随负载变化的问题，但同时引入了一个成本决策：常驻多少容量、按需扩展多少容量。

按峰值一次性准备服务器时，只要日常负载显著低于峰值，就会产生长期闲置。云平台提供按使用量付费的计费模型，可以减少为波动负载长期保留资源的成本。但前文已经说明，Pod 和 Node 的扩容都存在启动与就绪时延。如果将常驻容量压到极低，每次负载上升都必须等待扩容完成，用户在等待期间会经历排队和延迟升高。

因此，工程上通常采用两段式容量规划：用一部分常驻容量覆盖可预期的日常负载（基线），用弹性扩缩覆盖超出基线的波动部分。基线的作用是确保日常负载波动不触发扩容延迟，使 HPA 只需要应对超出日常范围的突增。以 `lobby-api` 为例，如果日常峰值稳定在 8 个 Pod，基线可以设为 6 到 8 个副本（HPA 的 `minReplicas`），活动期间的额外负载由 HPA 扩容至 20 到 30 个副本承接。

基线设置过低，系统在每次日常波动时都会触发扩容和缩容循环，带来不必要的延迟抖动和资源振荡。基线设置过高，低谷时段大量副本处于空闲状态，资源利用率下降。从反馈闭环的角度看，基线相当于将系统的稳态工作点设在目标水位附近，减少控制器需要追赶的偏差幅度。

云资源采购模型 两段式策略的成本优化依赖于常驻容量与按需容量的配比。常驻容量适合覆盖可预期的日常负载，单位成本较低但灵活性差；按需容量适合应对波动，单位成本较高但可按需释放。具体计费模型因云平台而异，核心取舍是：常驻比例越高，扩容延迟越小，但低谷空转成本越大；常驻比例越低，成本越灵活，但高峰扩容等待时间越长。

对游戏场景，典型的配比思路是：将长期在线的基线并发（入口层、匹配服务、数据库代理）放在成本较低的常驻实例上，将活动期间的临时扩容放在灵活的按需实例上，将非实时的运维任务（日志归档、离线统计）放在价格更低但可能被回收的竞价实例上。

5.3.5 函数粒度的伸缩调度：Serverless

Serverless（无服务器计算）是一种与常驻服务不同的执行模型：开发者将业务逻辑以函数或事件处理器的形式交付给云平台，平台负责准备执行环境、运行代码、返回结果，并在空闲时回收资源。”无服务器”并不是没有服务器，而是开发者不再直接管理服务器实例、操作系统和扩缩容流程。计费通常按代码实际执行的时间、调用次数以及配置的预热容量计算，而非按长期租用服务器计费。在没有事件触发且未配置预置并发或最小实例数的普通按需模式下，代码不运行，费用接近零。

与前文讨论的 Kubernetes 弹性伸缩相比，Serverless 在控制粒度上有本质区别。HPA 的控制对象是常驻 Pod 的副本数：观察到负载上升后增加副本，观察到负载下降后减少副本，但最小副本数通常不为零。Serverless 的控制对象是事件触发的执行实例：请求或事件到达时，平台优先复用已经准备好的热实例；当没有可用实例或并发容量不足时，再创建新的执行环境。处理完成后，实例可能被保留一段时间等待复用，也可能在空闲窗口后被回收。在允许 scale-to-zero 且未配置最小实例数的模式下，长时间无请求时实例数可以降为零。这种模型将触发信号从“资源水位偏差”前移到“事件到达”本身，减少了对周期性指标采样的依赖；但实例创建、并发扩容和平台配额仍会带来延迟。

代价是平台必须在极短时间内完成执行环境的创建和初始化。当请求到达时如果没有可用的预热实例，平台需要从零启动一个新的执行环境（冷启动），这一过程引入了额外延迟。冷启动的时延和 Serverless 的适用边界是本节讨论的核心问题。

从常驻服务到按需函数 部署模型的选择本质上是在常驻成本与首次请求延迟之间取舍。图 73 将 Kubernetes 的常驻服务模式与 Serverless 的按需执行模式置于同一资源占用条件下比较。

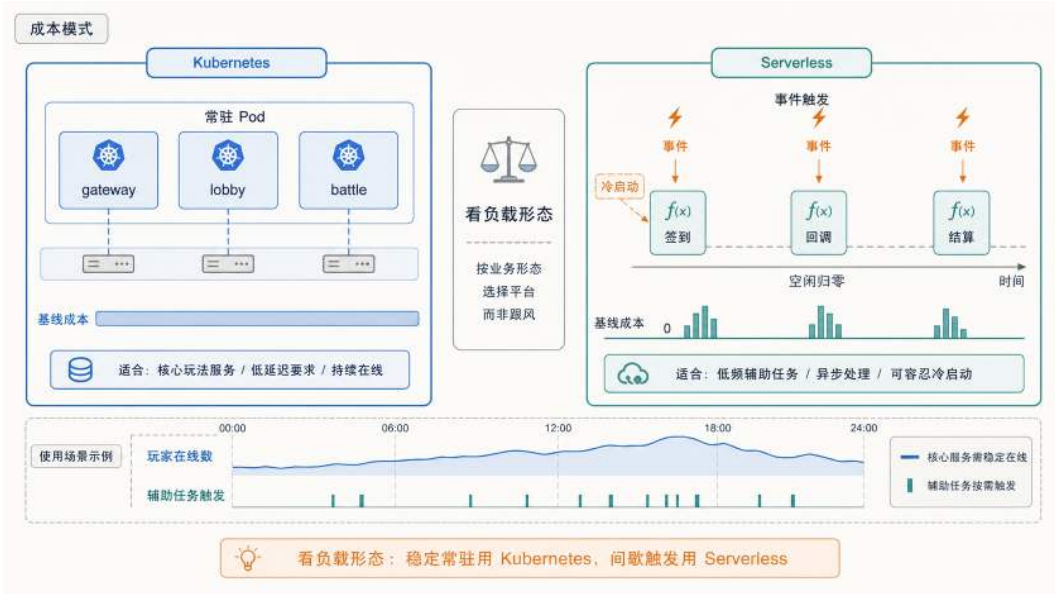


图 73: Kubernetes 与 Serverless 的成本模式对比：核心服务常驻以保证低延迟，低频任务可按事件触发；在允许归零且未配置预热容量时，空闲期可以释放运行实例。

在 Kubernetes 常驻模式下，服务以 Pod 形式持续运行，即使没有请求也占用 CPU 和内存配额；运维团队需要持续关注集群健康、Node 规格、镜像更新和安全补丁。对核心链路（入口层、匹配服务、战斗服），这种常驻投入是保证低延迟的必要成本。但对低频辅助功能（签到奖励发放、支付回调、排行榜定时结算），绝大多数时间 Pod 处于空闲状态，运维开销与实际计算量严重不匹配。

在 Serverless 按需模式下，开发者将业务逻辑封装为函数并上传到平台。平台在事件到达时调用函数，优先使用可复用的热实例；当并发请求超过热实例容量时，平台再创建新的执行实例。

处理完成后实例可能被保留一段时间以复用，空闲超过平台设定的回收窗口后被释放。若平台允许归零且没有配置预置并发或最小实例数，没有事件触发时通常不存在运行中的业务实例；若为了降低冷启动配置了预热容量，则空闲期仍会保留实例并产生相应成本。

以每日签到为例。玩家的签到行为集中在早晨和晚间高峰，其余时间调用量接近零。如果实现为常驻服务，需要配置 HPA 最小副本数、维护 Deployment 和监控，低谷时段副本空转。如果实现为 Serverless 函数，平台在玩家点击签到时复用热实例或创建新实例，计算并发放奖励，返回结果后实例进入空闲等待复用。高峰期平台自动创建更多并发实例，高峰过后实例逐步回收。运维团队不再直接维护这组空闲副本，但仍需要关注并发配额、下游容量、错误重试和成本上限。

下面用伪代码展示这个签到函数的完整结构。与常驻服务的关键区别在于：整个文件没有启动 HTTP 服务器的代码，没有路由注册，没有监听端口。开发者只需要导出一个处理函数，平台负责在事件到达时调用它。

签到奖励的 Serverless 函数实现

```
1 package checkin
2
3 // Handler 是平台在每次事件到达时调用的入口函数
4 func Handler(event Event, ctx Context) Response {
5     playerID := event.Body["player_id"]
6     today := ctx.CurrentDate // "2026-06-27"
7
8     // 幂等写入：同一玩家同一天只能签到一次
9     key := fmt.Sprintf("checkin:%s:%s", playerID, today)
10    db := GetConnection()
11
12    // InsertCheckin 使用唯一约束或条件写入，成功时才允许发奖
13    tx := db.BeginTx()
14    reward := CalculateReward(playerID, today)
15    ok := tx.InsertCheckin(key, playerID, today, reward)
16    if !ok {
17        tx.Rollback()
18        return Response{Code: 200, Msg: "already checked in"}
19    }
20
21    tx.AddToInventory(playerID, reward)
22    tx.Commit()
23
24    return Response{Code: 200, Reward: reward}
25 }
```

函数接收两个参数：`event` 包含触发事件的具体内容（HTTP 请求体、消息队列消息等），

ctx 包含平台注入的运行时信息（请求 ID、剩余执行时间、当前日期等）。函数返回一个结构化响应，平台将其转发给调用方。示例中，玩家 ID 和日期组成业务唯一键，InsertCheckin 依赖数据库唯一约束或条件写入完成原子占位；只有占位成功的请求才继续发奖并提交事务。这里不能写成简单的”先 Exists 再 Set”：两个重复请求可能同时通过查询，再分别写入奖励，反而破坏幂等性。

与函数代码配套的是一份部署配置，声明函数如何被触发、分配多少资源、超时多久：

签到函数的部署配置

```
1 function:
2   name: daily-checkin
3   runtime: go1.21
4   handler: checkin.Handler      # 包名.函数名
5   memory: 128MB
6   timeout: 5s                  # 单次执行最长时间
7
8 trigger:
9   type: http
10  method: POST
11  path: /api/checkin
12
13 environment:
14  DB_HOST: ${postgres_endpoint}
15  DB_NAME: game_rewards
```

这份配置体现了平台与开发者之间的职责边界。开发者声明运行时版本、入口函数、内存上限和超时阈值；平台根据这些声明决定使用何种沙箱、分配多少资源、何时回收实例。触发器部分将一个 HTTP POST 请求绑定到签到函数：玩家客户端发送 POST /api/checkin 时，平台自动路由到 daily-checkin 函数的一个执行实例。如果将 trigger.type 改为 schedule（如 cron: "0 4 * * *"），同一个函数就变成了定时任务，每天凌晨 4 点由平台主动触发执行。

对比前面用 Kubernetes 部署常驻服务的方式，两者的差异集中在生命周期管理上。Kubernetes 的 Deployment 声明”始终保持 N 个副本运行”，平台持续维护这些副本的存活状态；Serverless 的部署配置声明”事件到达时如何调用函数、资源和超时如何约束”，平台负责执行实例的创建、复用、回收和并发调度。前者需要运维团队关注副本健康、资源利用率和滚动更新；后者把实例生命周期交给平台，但开发者仍需管理触发器、权限、依赖版本、监控告警、幂等逻辑和下游容量。

应用约束 Serverless 平台可能随时回收执行实例，也可能在故障后重新投递事件。因此，函数不能把业务正确性建立在本地内存或本地文件上，带有外部副作用的操作还必须能够识别重复请求。状态外置和幂等性决定了一个功能能否安全地按需创建与重试；具体设计条件将在第 6.3 节

统一说明。

状态外置和幂等使平台能够按需创建和销毁实例，但“按需创建”本身带有延迟代价。当一个函数长时间无调用后，平台会回收所有空闲实例以节省资源；下一次请求到达时，平台必须重新分配执行环境，这个从无到有的准备过程称为冷启动（cold start）。这里先关注它对选型判断的影响；沙箱精简、快照恢复和运行时优化等内部机制将在第 6.4.8 节展开。

签到是典型的适合场景。签到请求执行时间短（查询记录、计算奖励、写入数据库，整条链路通常很短），所有状态存在外部 Redis 和 PostgreSQL 中，调用量集中在早晚高峰且其余时段接近零。早晨第一批玩家可能触发冷启动，但只要首次响应仍在业务可接受范围内，这一额外延迟通常不会破坏签到流程。高峰过后实例逐步回收；在未配置预热容量的按需模式下，空闲期不必维持业务实例常驻。

支付回调也适合。第三方支付平台在玩家完成付款后，向游戏服务器发送一个 HTTP 回调通知。这类请求完全由外部事件驱动、到达时间不可预测、单次执行只需验证签名并写入一条订单记录。回调对响应延迟的要求宽松（支付平台通常允许数秒内返回确认），且自带重试机制（未收到确认会重新投递）。用 Serverless 处理回调，不需要维护常驻服务来等待这些偶发事件，同时重试机制与函数的幂等设计天然配合。

实时对战则不适合。战斗服需要维持与每位玩家的 WebSocket 长连接，持续接收操作指令并广播帧同步数据。一场对局会持续较长时间，期间连接不能中断、状态不能丢失。典型函数平台通常带有单次执行时长、请求网关超时和实例回收等限制，执行环境也不适合承载长期驻留的连接状态。即使某些容器型 Serverless 平台支持较长连接，冷启动、扩容抖动、实例回收和连接迁移仍然会给强实时链路带来额外风险。帧同步对延迟极其敏感，任何无法稳定控制的首次启动时延都可能表现为明显卡顿。战斗服更适合以常驻 Pod 形式运行，由 Kubernetes 保证副本持续存活，并在应用层处理排水和会话恢复。

前面章节讨论的匹配服务同样不适合直接改成按请求启动的函数。这里的匹配服务在进程内存中维护一个连续变化的匹配队列，持续扫描等待中的玩家、计算评分差异并组合对局。如果每次调用都从外部存储加载完整队列、执行匹配、再写回，不仅延迟增加，多个实例还可能并发修改同一份队列。业务可以把队列状态外置并把匹配拆成可协调的短步骤，也可以按分区维护多个常驻队列；但无论采用哪种方式，都必须先解决队列归属和并发更新问题，不能只靠 HPA 增加副本。

从这四个场景可以归纳出判断标准：函数执行时间短、无需维持连接、状态可外置、对首次延迟不敏感的功能适合 Serverless；需要长连接驻留、依赖进程内状态、或对延迟有严格要求的功能应保持常驻。表 8 将这些判断按业务特征归类，供快速查询参考。

表中“突发流量”一行需要补充一个前提：Serverless 的弹性能否兑现，取决于下游是否承接得住并发。平台可以在短时间内拉起大量函数实例，但如果下游数据库的连接池、缓存的吞吐上限或第三方接口的速率限制先被打满，玩家看到的仍然是超时。云厂商通常还会设置单函数并发配额和区域容量边界，这些约束需要在线前通过压测确认，不能仅凭“自动扩缩”的描述做容量假设。

实际项目通常按组件的连接时长、状态位置、调用频率和延迟目标选择部署模型，而不是为

表 8: Serverless 与 Kubernetes 的场景化决策参考

业务特征	推荐方案	主要原因
长连接实时交互	Kubernetes	WebSocket/TCP 长连接需要稳定连接驻留，容器常驻模型更匹配
有状态会话管理	Kubernetes	会话粘性、进程内状态与低抖动路由更容易在 K8s 中实现
低频触发式任务	Serverless	普通按需模式按调用计费，不必长期为空闲实例付费
不可预测突发流量	视任务而定	无状态、可并行、可重试的短任务适合 Serverless；长连接或强状态链路仍应常驻
定时批处理	Serverless	与事件调度天然契合，任务短平快且资源利用率高
延迟极敏感	Kubernetes	冷启动尾延迟风险较高，常驻实例更容易保障严格时延目标

整个系统采用同一种方案。长期有效的划分原则是：延迟敏感或需要维持连接和进程内状态的核心链路保持常驻，短时、无状态、事件驱动的周边任务按需执行。图 74 用游戏后端验证这一划分方法。

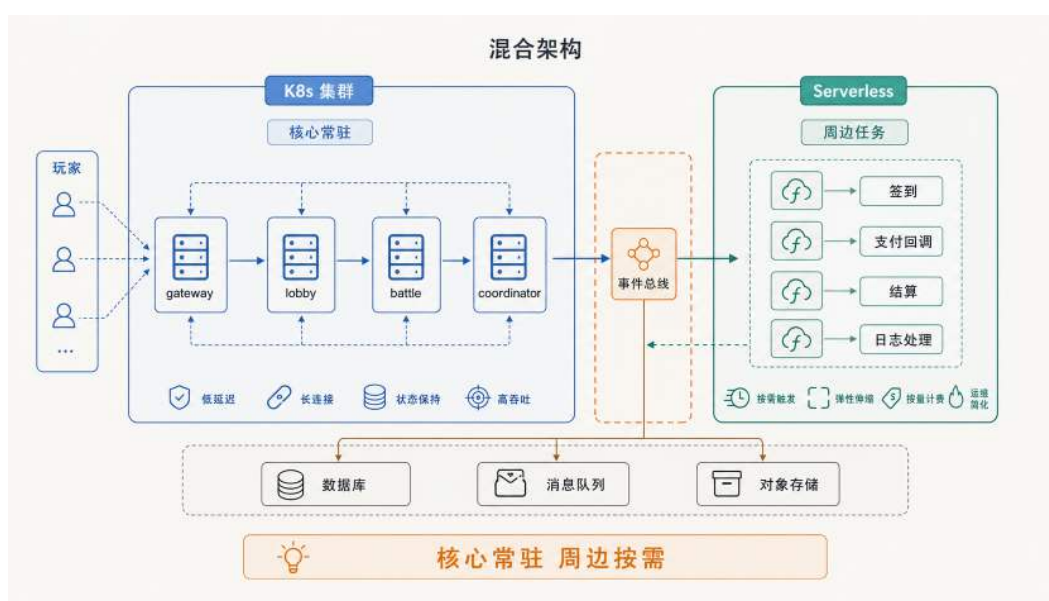


图 74: K8s + Serverless 混合架构：实时连接和对局常驻在 K8s，签到、回调、结算等周边功能按需执行。

常驻部分由 K8s 集群承载：gateway 负责接入玩家长连接，lobby 处理大厅逻辑，battle 运行对局，coordinator 裁决跨服事件。玩家客户端通过长连接接入 gateway，再由 gateway 将请求路由到内部核心服务；延迟和连接稳定性由常驻 Pod 保障。按需部分由 Serverless 平台承载：签到、支付回调、排行榜结算和日志处理均以事件驱动方式执行。两侧通过中间的事件总线连接：当核心服务产生需要异步处理的事件（如对局结束后触发结算、玩家操作触发日志归档），事件总线将其投递到对应的 Serverless 函数。底部的数据库、消息队列和对象存储是两侧共享的基础

设施：核心服务和 Serverless 函数都通过这些外部存储读写状态，函数本身不持有任何本地状态，这正是前面强调无状态约束的落点。

这种混合架构已经回答了” 什么功能放哪里” 的问题，但还留下一个工程细节：Serverless 函数在长时间无调用后，原有执行实例可能被平台回收；如果下一次请求到达时没有可复用的热实例，平台就需要重新创建执行环境。这个重建过程的延迟特征、影响因素和优化思路，是 Serverless 落地时必须量化评估的成本。

冷启动：用首次延迟换常驻成本 冷启动延迟由执行环境、语言运行时和应用初始化等阶段共同累积；暖启动通过复用已有实例跳过其中一部分准备过程。图 75 将两条调用路径置于同一时间顺序中，说明首次延迟与常驻资源成本之间的交换关系。

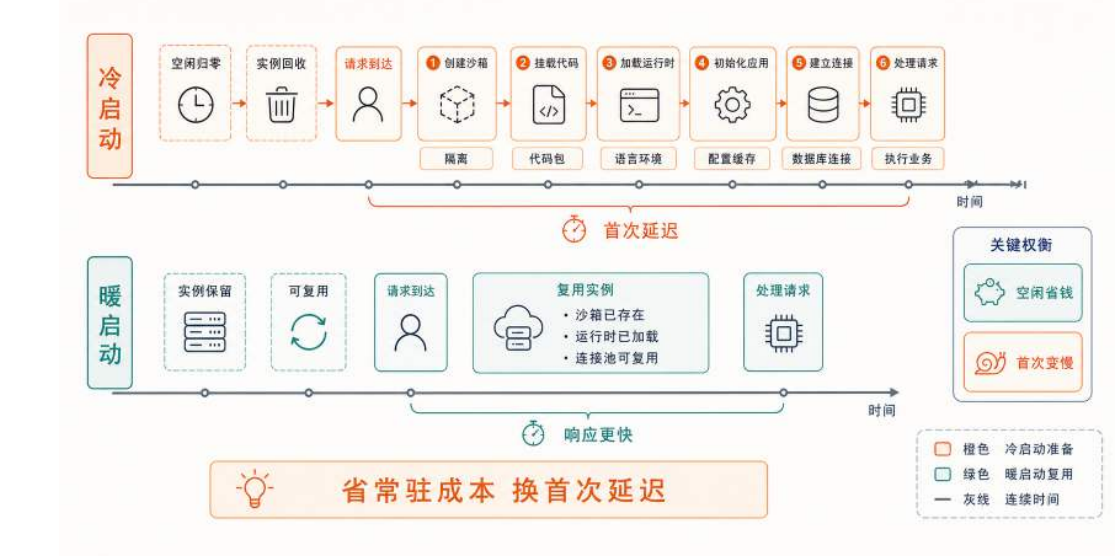


图 75: Serverless 冷启动与暖启动的调用路径对比：冷启动需要重新准备执行环境，暖启动则复用已有实例。

Serverless 最需要评估的问题是冷启动延迟。当平台已有可复用的实例时，请求直接交给现成的执行环境处理，延迟较低，这称为**暖启动**。当函数长期空闲后，平台可能回收原有实例以节省资源，下一次请求到达时如果没有可复用实例，平台就需要重新分配隔离沙箱并启动运行时，这称为**冷启动**。冷启动不是偶然异常，而是 Serverless 用**首次延迟**交换**常驻成本**的自然结果。落到玩家视角，冷启动通常表现为” 第一次特别慢”：例如早晨第一批玩家打开游戏点签到，按钮转圈明显比平时久；又或者某个低频活动入口长时间无人访问，第一次进入时页面加载突兀地卡一下。

除了首次延迟，函数平台通常还会限制执行时间，并可能在故障后重试调用；本地磁盘也不能作为长期状态依赖。这些条件要求业务逻辑能够拆成可独立重试的步骤，并把长期状态写入外部存储。冷启动由哪些阶段组成、MicroVM、快照和 Wasm 分别压缩哪一段成本，将在第 6.4.8 节从引擎视角展开。

5.3.6 故障自愈：检测、修复与约束

前面几节围绕容量变化展开：弹性伸缩让服务在负载起伏时增减副本，Serverless 让低频功能在无请求时归零。这些机制都以实例能够正常工作为前提，调整对象是实例数量。线上环境还会发生进程死锁、内存溢出、节点宕机和网络分区等故障。此时改变的是实例健康状态，单纯增减副本数不能直接修复已经失效的实例。扩缩容解决的是容量随负载变化，故障恢复解决的是实例在运行中失效；两者都用于维持服务目标，但触发信号和处理动作不同。平台自动完成故障检测、修复和恢复验证的过程，通常称为故障自愈（self-healing）。

自动恢复只有在修复动作能够被验证并再次触发检测时，才构成可以持续运行的控制过程。平台需要把故障信号转换为故障层级，在可用性、依赖状态和排水条件允许的范围内选择修复粒度，并根据恢复验证决定本轮处理是否结束。图 76 将这一控制关系概括为故障自愈闭环。



图 76: 故障自愈把检测、分级、约束检查、修复与恢复验证连成闭环；修复未能恢复服务目标时，系统重新进入检测与决策过程。

检测信号的准确性决定后续修复是否会被正确触发，因此首先需要明确平台如何判断实例或节点已经失效。

检测：平台如何发现故障 自愈的第一步是发现故障。平台无法直接“知道”一个实例是否还能正常工作，只能通过外部信号推断，这些信号大致分为三类。第一类是主动探测：平台按固定周期向实例发送探测请求（如 K8s 的 Liveness/Readiness Probe、负载均衡的健康检查），根据是否在超时内正确响应判断实例状态。前面场景三中介绍的就绪探针就是这类信号的一种。第二类是被动上报：实例主动向注册中心发送心跳，或在出错时上报异常；一旦心跳超时，注册中心将其从可用列表中剔除。这类信号依赖实例自己“报平安”，适合服务注册发现体系。第三类是指标异常：监控系统持续采集错误率、尾部延迟等指标，当指标越过阈值（如错误率超过 5%、P99 延迟超过 2 秒）时间接推断出下游可能已经故障。这类信号发现的往往不是单个实例的死

活，而是整体服务质量的劣化。三类信号各有盲区：检查内容过浅的主动探测可能只确认进程存在，却没有覆盖关键业务路径；被动上报依赖实例自身仍能通信，指标异常则可能滞后于故障发生。生产系统通常组合使用，而不是只依赖其中一种。

修复：改变执行面上的哪一层对象 检测到故障后，平台的修复动作按影响范围分为由轻到重的三级，每一级改变的是执行面上不同粒度的对象。第一级是进程重启：在原节点上重启容器，适合内存泄漏、死锁这类进程内故障。中断粒度最小（秒级），Pod 的 IP 和调度位置都不变，对应 Liveness Probe 失败后 kubelet 的动作。第二级是实例重建：删除故障 Pod、由控制器在合适的 Node 上重新创建一个，适合节点磁盘写满、局部网络不通这类靠重启进程无法解决的故障。这正是前面调谐循环中“副本数低于期望值就补位”的机制，代价是新 Pod 的 IP 会变化，因此调用方必须通过 Service 而非固定 IP 访问。第三级是节点替换：下线故障节点、申请新机器加入集群，适合物理机宕机、磁盘损坏这类整机故障。这个动作通常由云平台的节点组、实例组自动修复或专门的节点修复机制完成；Cluster Autoscaler 主要根据不可调度 Pod 和节点利用情况调整节点数量，并不等同于故障节点修复器。节点替换耗时最长，且要求节点上的状态已外置，否则数据可能随节点一起丢失。这三级动作并不是孤立的工具，而是平台在不同故障粒度上采用的修复方式：进程级故障就地重启，Pod 失效时由工作负载控制器创建替代实例，节点级故障则由集群与云平台基础设施协同替换机器。它们能够自动进行的共同前提是实例可以被安全替换，也就是前面反复强调的无状态或状态外置。一个把状态留在本地内存或本地磁盘的实例，删掉重建就等于丢数据，平台的自动修复反而会造成损失。

自愈策略的边界与约束 上面三级动作容易给人一种印象：出故障就重启或重建即可。但生产级自愈不是“进程死了就立刻拉起”这么简单，否则可能引发新的问题。最典型的是重启风暴：如果某个依赖（如数据库）临时不可用，导致一批实例同时探测失败，平台若不加约束地同时重启全部实例，重启后它们又会同时涌向尚未恢复的依赖，再次失败、再次重启，系统在“崩溃—重启—再崩溃”之间振荡，始终无法恢复。因此，真实的自愈是一个带约束的决策，而不是无条件的重启。这些约束包括：其一，可用性约束。计划驱逐一个 Pod 前要确认它不是该服务最后的可用副本，否则维护操作会直接导致服务完全中断。K8s 用 PodDisruptionBudget (PDB) 表达“自愿中断期间至少保留多少个可用副本”，主要约束节点排空、集群升级等可计划的驱逐操作。PDB 不能阻止节点突然宕机、进程崩溃等非自愿故障，因此不能替代副本冗余和故障恢复机制。其二，依赖与预热约束。有些实例重启后需要重新加载大体积缓存或建立连接池才能提供服务，这段预热时间内不应被计入可用容量，否则流量会打到尚未就绪的实例上，这正是 Readiness Probe 与 Service 后端列表要守住的边界。其三，下线时序约束。对长连接和有状态服务，修复动作在终止旧实例前需要先排水：停止接收新流量、等待存量请求或会话结束，必要时迁移状态。这与前面安全缩容所采用的“停止新流量、处理存量、再终止实例”是同一顺序。把这些约束合起来看，自愈的本质是在“尽快恢复”和“不因恢复动作制造新故障”之间取平衡。

回到游戏后端：不同服务的自愈代价 同一套检测与修复机制，落到游戏后端的不同服务上，代价差别很大，判断依据仍然是状态特征。对 lobby-api 这类短请求、无状态服务，自愈最

简单：探测失败后删除重建，新 Pod 就绪即自动加入 Service 后端，个别失败请求由客户端重试吸收，玩家几乎无感。对 gateway 和 battle 这类长连接、有状态服务，自愈不能简单地删了重建：网关上承载着大量 WebSocket 长连接，战斗服上运行着正在进行的对局，直接重启会让相关玩家同时断线或掉出对局。这类服务的修复必须配合排水、连接重连或对局状态恢复，平台提供下线时序保证，具体何时可以安全中断由业务逻辑定义。对 redis、postgres 这类有状态存储，自愈的边界最严：它们的数据不能随实例消失，因此不能靠“删除重建”修复，而要依赖数据外置、副本复制和主从切换等机制，平台的自动修复至多是重新调度任务，不能替代数据层面的高可用设计。前面章节介绍的 Deployment（维持副本数）、Service（稳定入口）、Readiness/Liveness Probe（就绪与存活判定）、PDB（约束自愿中断）、preStop 与优雅终止（排水时序）、反亲和（分散故障影响）、Cluster Autoscaler（容量不足时扩展节点），分别在容量调整、计划维护和故障恢复中承担不同职责。明确这些职责后，可以把它们统一到本章的总原则：把失败作为常态纳入系统设计。更高阶的故障场景——有状态服务的一致性恢复、依赖链上的级联故障，以及用熔断（circuit breaker）和自适应限流阻断故障扩散——已经超出本章“如何部署与运维”的范围。

5.4 小结：从标准化交付到弹性运行

本章围绕在线系统进入云环境后面临的环境差异、服务拓扑、负载变化和实例故障，按照“容器—编排—弹性运行”三块建立部署与运维链路，并以第四章的分布式游戏后端验证这些机制。三块内容分别处理运行环境、服务拓扑和持续运行状态，最终目标都是让系统在实例会重启、被替换、重新放置和伸缩的条件下仍能维持服务目标。

第一块是容器。Docker 通过分层构建与摘要锁定，把一次偶然成功的部署变成可复制的制品；镜像、容器、数据卷、隔离机制和容器网络共同保证同一服务可以在不同机器上按相同方式启动。这个阶段解决的是“交付什么，以及换机器后为什么还能一致运行”。

第二块是编排。Compose 把单机上的多容器拓扑写成声明，集群级平台进一步处理跨节点网络、服务发现、放置和副本维护。Kubernetes 用 Pod、Deployment、Service 等对象描述目标状态，再由控制面和调谐循环持续把现实状态拉回目标。这个阶段解决的是“多个服务如何统一部署、互联和长期维护”。

第三块是弹性运行。资源调度决定副本放在哪里，HPA、VPA 与 Cluster Autoscaler 调整应用和基础设施容量，Serverless 为低频短任务提供按需执行方式，故障自愈则在实例或节点损坏后完成检测、修复和恢复验证。这些机制都受 SLO、启动时延、状态归属、下游容量和安全下线条件约束，不能被简化为“自动加机器”或“出错就重启”。

三块内容最终汇聚到云原生系统中的一个基本原则：**为失败而设计**（Design for Failure）。系统设计阶段就要承认机器会宕机、实例会重启、网络会抖动、流量会突增、请求会重复到达。副本冗余、Readiness 闸门、滚动更新、反亲和分散、缩容排水、幂等写入和可观测性，都是把失败纳入平台流程之后形成的工程机制。

读完这一章，至少应该留下几条能带走的判断：镜像要锁内容摘要，入口要和实例解耦，副本数要由平台长期维持，缩容要先排水，核心链路不能轻易归零。第 6 章会把这些判断放到底层

机制里解释：容器隔离、网络虚拟化、伸缩闭环与 Serverless 引擎，都会回到“变化中如何维持稳定”这条主线。

5.5 思考题

1. 团队用 `latest` 标签部署了三台大厅服。某天有人推送了新的同名镜像，导致一台服务器重启后运行新版本，另外两台仍然运行旧版本。请解释这种“版本漂移”是怎样发生的，并说明用镜像摘要（digest）锁定版本为什么能避免这类问题。
2. 周五晚高峰，运维人员用脚本逐台重启进程做热修。脚本执行到一半时网络抖动，导致部分实例已更新、部分实例仍是旧版本。请解释这种中途失败会怎样表现为“有些玩家能登录、有些玩家不能登录”，并说明声明式管理如何把“脚本执行到一半”转化为“现实状态偏离目标状态”来处理。
3. 新部署的 `lobby-api` Pod 进程已经启动，但配置尚未加载完，Redis 连接也没有建立。如果此时没有 Readiness Probe，玩家侧可能看到什么现象？如果 Liveness Probe 设置得过于激进，例如 1 秒内没有响应就重启，又会带来什么问题？请分别说明 Readiness 和 Liveness 守住的是哪两类边界。
4. `lobby-api` 的 CPU 使用率只有 40%，但 P99 延迟已经升到 800ms。如果 HPA 只看 CPU，它会做出什么判断？请列出至少两类“CPU 不高但延迟很高”的原因，例如下游服务变慢、连接池耗尽、锁竞争或队列积压，并提出一个更贴近玩家体验的自定义指标作为扩缩容输入。
5. 战斗服正在缩容，某个 Pod 上还有 5 场正在进行的对局。如果平台直接杀掉进程，玩家侧会看到什么现象？请设计一个“先关门、再清场、最后拉闸”的排水流程，并说明 Readiness Probe、Service 后端摘除和 `preStop` hook 分别在哪一步发挥作用。
6. 给出三种游戏功能：(a) 每日签到奖励发放；(b) 实时对战的帧同步；(c) 赛季结束时的排行榜结算。请分别判断它们是否适合 Serverless，并从连接驻留、状态粘性、冷启动延迟和可重试性四个维度说明理由。
7. 某活动预计在前 10 分钟内把登录 QPS 从 2,000 提升到 8,000。已知一个 `lobby-api` 副本在 P99 延迟可接受的前提下大约能稳定处理 600 QPS，新 Pod 从创建到 Ready 约需 45 秒。请设计一个活动前后的容量策略：基线副本数应如何设置，高峰前是否需要预热，HPA 的扩容速度应如何限制，活动结束后缩容应如何排水。进一步思考：如果只按平均 QPS 配置容量，玩家会在哪些时刻最先感受到排队？
8. 数据库 `postgres` 临时不可用，导致一批 `lobby-api` 实例的探针在几秒内同时失败。如果平台不加约束地立即重启全部失败实例，会出现什么现象（提示：重启后它们仍会同时连向尚未恢复的数据库）？请说明这种“重启风暴”为什么无法靠单纯重启收

敛，为什么 PodDisruptionBudget 不能阻止 Liveness Probe 触发的容器重启，以及区分 Liveness 与 Readiness、加入退避和依赖检查分别能在哪一环降低振荡风险。再进一步：为什么说“能否自愈由架构决定，而不是由平台决定”？

9. 某在线协作系统包含三类组件：处理文档列表查询的无状态 API、维护 WebSocket 会话的实时协作网关，以及每天凌晨执行一次的文档归档任务。请分别选择常驻容器或 Serverless 作为部署方式，并说明选择依据。若实时协作网关需要缩容，还应使用什么指标触发伸缩，怎样完成连接排水？
10. 某模型推理服务的每个 Pod 启动后需要加载模型并预热 90 秒。高峰到来时，CPU 利用率仍然不高，但 GPU 请求队列和 P99 延迟持续上升。请为该服务确定一个可量化的 SLO，选择比 CPU 更合适的伸缩信号，并说明 Readiness Probe、预热容量、HPA 与 Cluster Autoscaler 应如何配合。若只在队列已经积压后才申请新节点，系统会面临什么时间差？

6 第六章 穷理尽微：云原生核心原理

第五章以游戏服务的部署与运维为主线，说明游戏服务为什么需要容器、编排、弹性伸缩与 Serverless，以及开发者如何通过 Docker、Kubernetes、HPA 等平台接口声明和使用这些能力。通过第五章，读者主要掌握相关技术的应用场景、需求来源、操作对象与配置方法。

本章进一步将关注点下沉到平台内部，分析这些能力的实现机制与工程代价：隔离边界如何建立，数据与状态如何流动，控制决策如何形成，时间、资源与管理开销产生于哪些环节，以及各项机制具有怎样的适用条件与能力边界。仅会配置和调用平台接口，并不足以支撑复杂系统的诊断与优化。当系统出现卡顿、排队、冷启动或资源泄漏时，开发者需要沿平台内部的数据路径和控制路径定位问题。例如，战斗服延迟升高，既可能源于容器资源限额，也可能与网络虚拟化路径有关；HPA 扩容不及时，既可能由监控指标滞后引起，也可能受制于 Pod 的创建与启动过程；Serverless 首次响应较慢，则需要进一步拆解实例分配、运行环境初始化、代码加载与应用启动等冷启动环节。

因此，第五章重点回答“这些能力适用于什么场景，以及如何声明和使用”，本章则重点回答“平台如何实现这些能力，以及实现过程中存在哪些限制与代价”。全章依次分析容器隔离、容器网络、弹性控制与快速启动机制，最后以面向 Agent 的高并发 Sandbox 为综合案例，考察这些云计算机制如何在高并发、强隔离、短生命周期和动态负载等新型工作负载下重新组合。

6.1 容器的本质：操作系统级虚拟化的三块基石

本节讨论的“容器”，指的是 Linux 上的**操作系统级虚拟化**。虚拟机通常在硬件之上再“虚拟出一台机器”，每台虚拟机有自己的客户机内核；容器采用另一条路径：多个应用进程**共享同一个宿主机内核**，同时让每一组进程拥有独立的系统视图。从结果看，容器也能做到“互不干扰”，它依靠的是内核提供的三类机制：**Namespaces** 用来隔离“看见什么”，**Cgroups** 用来限制“能用多少”，**rootfs 与镜像存储机制**用来构造“看到的文件系统长什么样”。在常见的 Linux 容器实现中，OverlayFS 经常用于将共享的只读镜像层与容器私有可写层合并为统一视图。容器运行时的职责，就是在启动应用进程之前，把这三类内核机制按同一份配置组合好：准备文件系统视图、创建命名空间、再把进程加入对应控制组。这样，一个普通进程才会在进入用户态后看到一套受限、独立、可复制的运行环境。

6.1.1 Namespaces：让进程拥有“独立的系统视图”

Namespace 是容器操作系统级虚拟化的第一类基石机制。读者第一次接触容器时，最容易观察到的差异往往来自进程视图：在宿主机上执行 `ps aux` 能看到很多进程，在容器里执行同样的命令，却通常只看到容器自己的那几个进程。这个现象直接指向 Namespace 要解决的问题：同一个内核怎样向不同进程组呈现不同的系统视图。图 77 展示了同一内核如何向不同进程组呈现不同视图：容器里并没有多出一个 Linux 内核，只是不同进程组看到的系统视图不同。

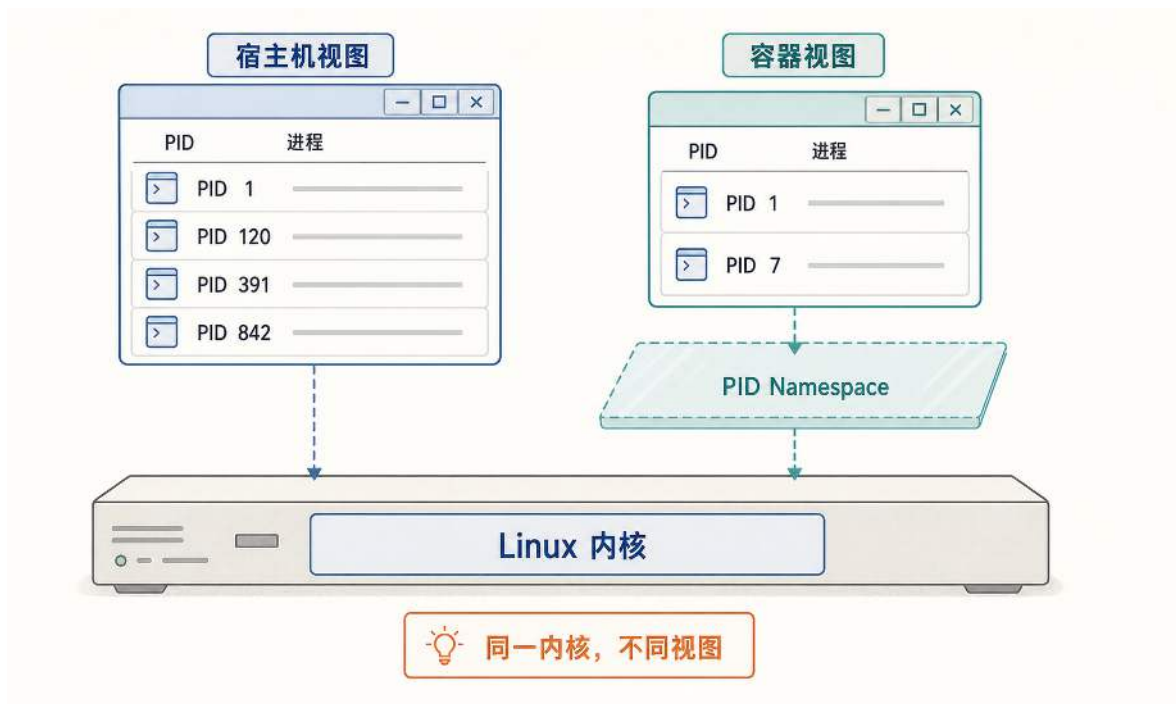


图 77: Namespace 改变进程看到的系统视图：宿主机能看到全局进程，容器只看到本命名空间内的进程。

更准确地说，宿主机处在默认的 root PID Namespace 中，容器处在运行时为它创建的 PID Namespace 中。宿主机视图中的 PID 1 是系统的 init 进程（通常是 systemd），负责启动和管理所有系统服务；容器视图中的 PID 1 则是容器的入口进程，例如游戏服务的主程序。两个 PID 1 是完全不同的进程，只是各自在所属的编号空间中被分配了相同的编号。但编号相同也意味着一些运行时责任相似：在 Linux 中，PID 1 需要承担回收孤儿子进程等职责，并且具有特殊的信号处理语义。宿主机的 PID 1 由 systemd 担任；容器内如果入口进程本身不能正确回收子进程、处理终止信号并转发给子进程，容器运行时通常会引入一个轻量的 init 进程（例如 `tini`）来承担这些工作，避免容器内子进程退出后无法被正确回收。图中宿主机看到四个进程（PID 1、120、391、842），容器只看到两个（PID 1、PID 7），因为内核为每个命名空间独立维护进程编号，容器内的程序通过 `getpid()` 得到的是命名空间内的编号，通过 `/proc` 看到的也只有同一命名空间中的进程。

由此可以得到 Namespace 的第一条边界：它解决的是“进程能看到什么”；“进程能使用多少资源”要交给后面的 Cgroups 处理。

在没有隔离机制时，Linux 系统中的许多关键资源在逻辑上是“全局共享”的。例如，系统只有一套进程号的编号规则，只有一棵统一的挂载目录树，只有一套网络协议栈，也只有一个主机名。进程之所以能“看见”这些资源，是因为它通过系统调用向内核查询或操作这些全局对象：例如 `getpid()` 返回进程号，`hostname` 查询主机名，`mount` 查询挂载信息，`ip addr` 查询网络接口等。由于这些对象在默认情况下是全局的，所以任何进程看到的结果基本一致。

Namespace（命名空间） 是 Linux 内核提供了一种隔离机制，它的思想可以用一句通俗的

话概括：同一类系统资源可以从“全局唯一的一份”，划分成多份；每个进程只属于其中一份，因此它只能看见那一份资源。Namespace 的作用可以表述为：把“全局对象”变成“按组划分的对象”，从而让不同进程组拥有不同的系统视图。

容器常用的几类 Namespace 分别隔离不同维度的视图。PID Namespace 隔离进程号空间；Mount Namespace 隔离挂载树；Network Namespace 隔离网络栈；UTS Namespace 隔离主机名。它们叠加在一起，就形成了容器最直观的效果：容器内运行 `ps` 时只看到容器自己的进程；容器内查看网络接口时，只看到属于容器的接口（最初往往只有回环接口 `lo`）；容器内的主机名可以与宿主机不同；容器内的目录结构也可以与宿主机不同。

需要特别强调的一点是：Namespace 生效在内核层，用户态工具只是读取内核提供的视图。例如，容器内执行 `ps` 看不到宿主机进程，不是“内核自动把 `/proc` 改了”，而是容器运行时通常会在新的 Mount Namespace 下挂载新的 `/proc` 并配合 PID Namespace 只暴露该命名空间中的进程。同理，`ip addr` 显示的网络接口来自内核网络子系统；Network Namespace 生效后，该命名空间拥有独立的网络接口集合，命令读到的也就只剩下容器自己的那一份。这个例子揭示了容器隔离的层次：隔离发生在内核层，用户态工具只是如实读取内核提供的视图。

从“如何创建”角度看，Namespace 是内核为进程提供的一个属性。容器运行时在启动子进程时，通过 `clone` 系统调用的标志位告诉内核需要哪些独立的命名空间。下面的片段保留了这一过程的核心结构：

用 `clone` 标志位为子进程创建独立命名空间（伪代码）

```
1 cmd := exec.Command("/bin/sh")
2 cmd.SysProcAttr = &syscall.SysProcAttr{
3     Cloneflags: syscall.CLONE_NEWPID | // 独立的进程编号空间
4             syscall.CLONE_NEWNS | // 独立的挂载树
5             syscall.CLONE_NEWNET | // 独立的网络栈
6             syscall.CLONE_NEWUTS, // 独立的主机名
7 }
8 cmd.Start()
9 defer cmd.Wait()
```

每个 `CLONE_NEW*` 标志对应前面介绍的一类 Namespace。当内核执行这段代码时，子进程 `/bin/sh` 进入四个新的命名空间：它新的 PID Namespace 中成为 PID 1；新的 UTS Namespace 初始继承父命名空间的主机名，但之后可以独立修改；新的 Network Namespace 最初只有回环接口 `lo`；新的 Mount Namespace 以父进程挂载列表的副本作为初始视图。除了在创建子进程时指定标志位，进程也可以通过 `unshare` 系统调用为自己或后续子进程创建新的命名空间；如果要想让进程加入一个已经存在的命名空间，则通常使用 `setns` 系统调用。PID Namespace 是一个例外：调用进程执行 `unshare(CLONE_NEWPID)` 后，新的 PID Namespace 主要对后续创建的子进程生效，调用进程本身不会立刻变成新命名空间里的 PID 1。

但仅靠这几个标志位，子进程得到的还只是尚未完成容器化配置的系统视图：网络栈没有外部连接，文件系统以父进程挂载列表的副本为起点，资源使用也没有任何限制。Mount Namespace

隔离的是挂载列表；后续挂载事件是否会跨命名空间传播，还取决于挂载点被配置为共享、私有还是从属（shared/private/slave），容器运行时通常会进一步调整传播关系。容器运行时在创建命名空间之后，还要完成三件事才能把一个普通进程变成可用的容器：用 Cgroups 限制它能使用多少 CPU 和内存，用 rootfs 为它准备独立的文件系统，再配置网络使它能与外部通信。前两件事分别是接下来两节的主题。

综合以上机制，Namespace 的作用可以概括为：**Namespace 把原本全局共享的系统资源按组切分，并为进程提供一套“只属于它那一组”的系统视图**。有了这层视图隔离，容器才具备“看起来像一台独立机器”的第一步。

6.1.2 Cgroups：把资源使用变成“可计量、可限额、可统计”的对象

如果说 Namespace 解决的是“容器看见什么”，那么 Cgroups 解决的就是“容器最多能用多少”。仅靠视图隔离并不能避免资源争用：某个容器即使看不见其他容器的进程，也仍然可以把 CPU 长时间跑满，或者把内存申请到系统紧张，最终拖慢整台机器上的所有服务。因此，一个真正可用的容器体系必须提供**资源控制**，让每个容器的资源使用在可预期范围内。

Linux 的 **Cgroups (Control Groups, 控制组)** 的基本思想是：先把一批相关进程归为一个控制组，再由内核对该组的资源使用进行限制和统计。上一节的代码为子进程创建了命名空间，但没有限制它能使用多少资源。接下来在那段代码的基础上，为同一个子进程加上 Cgroups 限额：

为子进程创建控制组并设置 CPU 和内存限额（伪代码）

```
1 // 在 /sys/fs/cgroup 下创建一个控制组目录
2 cgroupPath := "/sys/fs/cgroup/game-battle"
3 os.MkdirAll(cgroupPath, 0755)
4
5 // CPU 限额：示例写法为每 1s 周期内最多使用 500ms CPU 时间
6 os.WriteFile(cgroupPath+"/cpu.max", []byte("500000 1000000"), 0644)
7
8 // 内存上限：最多使用 512MB
9 os.WriteFile(cgroupPath+"/memory.max", []byte("536870912"), 0644)
10
11 // 将子进程加入该控制组
12 pid := strconv.Itoa(cmd.Process.Pid)
13 os.WriteFile(cgroupPath+"/cgroup.procs", []byte(pid), 0644)
```

这段代码揭示了 Cgroups 的实现本质：控制组在文件系统中表现为 `/sys/fs/cgroup` 下的一个目录，资源限额通过写入该目录中的控制文件来设置，将进程加入控制组也是一次文件写入。Kubernetes 中的 CPU limit 和 memory limit 最终也会映射为这类控制文件中的数值。真实容器运行时还需要处理 cgroup v2 的挂载方式、写入权限、controller 是否已启用，以及进程在执行应用代码前完成归组等细节；这里的伪代码只保留资源限额的核心路径。

图 78 将 Cgroups 的资源控制链路按执行顺序展开：进程先被归入控制组，控制组再获得 CPU 与内存限额，内核随后在运行时统计用量并执行限制。理解这条链路之后，`cpu.max` 与 `memory.max` 就不只是两个配置项，而是控制组约束资源使用的入口。

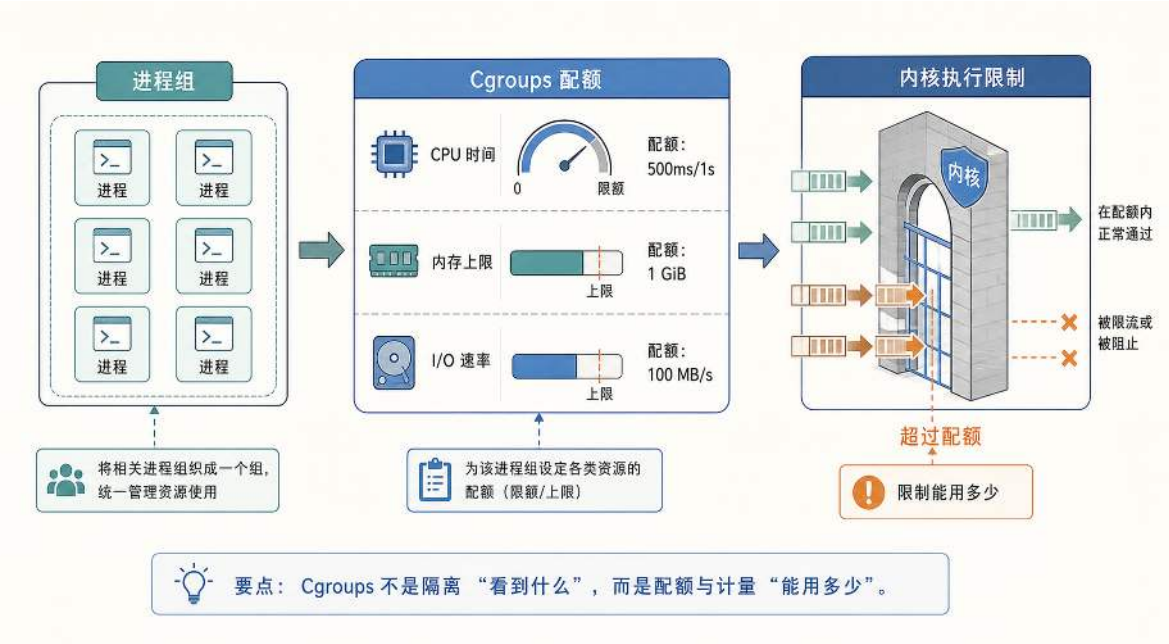


图 78: Cgroups 将进程组织成控制组，并对 CPU、内存和 I/O 等资源进行统计与限额。

代码中 `cpu.max` 写入的 500000 1000000 直接对应 CPU 限制的核心机制。这里将周期写成 1 秒，主要是教学演示；在许多实现里也常见 100 毫秒量级的配额周期（100000 微秒）。其含义是：内核每个周期为该控制组发放 500000 微秒（500ms）的 CPU 运行预算。组内所有线程在运行时共同消耗这份预算；一旦 500ms 用完，控制组内的所有线程会被暂停（throttled），直到下一个 1s 周期开始才重新获得预算。由此可见，CPU 限制关注的是“一段时间里累计可运行的总时长”，而不是固定分配某个核心。

这个机制会带来一个初学者容易误解的现象：**同样的配额，在单线程与多线程场景下表现完全不同**。预算按控制组整体计算，而不是按线程分别计算。如果战斗服容器内同时有 4 个线程在不同核心上并行运行，500ms 预算会以约 4 倍速消耗，大约 125ms 墙钟时间后整个控制组就会被暂停。宿主机还有空闲 CPU 核心，但容器内所有线程都停下来等待下一个周期。第 3.3.4 节用 CPU 占用和 P99 延迟来判断单机瓶颈；到了容器环境，这些指标还要和 Cgroups 配额一起解读，才能判断应该优化代码、放宽配额还是增加副本。

同样的思想也适用于内存控制。代码中 `memory.max` 设置的 512MB 是该控制组的硬上限。`memory.high` 是内存压力阈值：使用量超过该值后，组内进程会被节流并承受更强的页面回收压力，但不会仅因超过该阈值触发 OOM。当使用量达到 `memory.max`，且无法通过回收将其降低时，才进入该控制组的 OOM 处理。是否杀掉整个控制组相关进程，取决于 `memory.oom.group` 的配置；否则内核通常只在组内选择牺牲进程。无论哪种策略，内存限制目标是把局部失效限制

在可控单元内，避免一个容器的内存泄漏扩散为整台机器故障。

从 Namespace 到 Cgroups，容器运行时完成了两件事：隔离视图和限制资源。但容器内的程序仍然需要一套独立的文件系统才能运行：rootfs 决定进程把哪棵目录树作为根目录，镜像存储机制则负责组织和复用其中的文件内容。

6.1.3 rootfs 与 OverlayFS：镜像分层与容器写入的机制

容器内的程序需要可执行文件、动态库和配置文件才能启动，这些都存放在文件系统中。如果没有文件系统层面的隔离，容器内的进程会直接读写宿主机的目录树，无法形成独立的运行环境。这就引出文件系统层面需要解决的三个问题：**容器里看到的文件系统从哪里来？多个容器能否共享同一份基础环境？容器运行时产生的文件存到哪里？**

Linux 进程运行时，需要读取可执行文件、动态库和配置文件，这些文件都通过路径访问，而路径是相对于“根目录 /”展开的。例如，进程加载 `/bin/sh` 时，内核从当前进程看到的根目录开始查找 `bin/sh`。如果容器进程与宿主机共享同一个根目录，它读到的 `/bin`、`/lib`、`/etc` 就是宿主机的目录内容，无法构成独立环境。解决办法是：为容器进程指定一个不同的目录作为它的根目录 `/`。这个被指定为容器根目录的目录树，就是 **rootfs (root filesystem)**。当 rootfs 包含一套完整的目录结构时，容器内的程序通过 `/bin` 找到可执行文件，通过 `/lib` 找到动态库，通过 `/etc` 找到配置，rootfs 覆盖的这些普通文件路径就与宿主机目录树隔离开来。不过，`/proc`、`/sys` 等伪文件系统仍由宿主机内核提供，容器也可以通过 `bind mount` 或外部卷显式挂入宿主机路径。容器镜像的核心作用，就是为 rootfs 提供内容：一份可以被用作根目录的文件系统快照。

如果每个容器都需要一份完整的 rootfs，那么在同一台机器上启动多个容器时，磁盘空间会被大量重复文件占满。实际上，同一批容器往往共享相同的基础环境：同一个 Linux 发行版、同一套运行时库、同一套依赖包。更理想的做法是：**基础环境只保存一份并且只读共享，每个容器只保存自己的改动**。下文以采用 **OverlayFS (叠加文件系统)** 的常见 Linux 容器实现为例，说明镜像层共享和容器可写层的实现机制。

OverlayFS 的做法是让基础环境只读、共享，各容器的改动私有。图 79 展示了这一分层结构：底部若干层是只读镜像层，自底向上叠出基础目录、运行时库和应用文件，多个容器共用同一批而不各自复制；顶部是每个容器私有的可写层，未挂载外部卷的路径一旦写入，改动就落到这里。容器进程看到的不是某一层，而是各层合并后的**合并视图**：同一路径只出现一次，内核自上而下查找，优先返回可写层里的版本。这样，共享的只读层省下了磁盘，私有的可写层又守住了隔离。

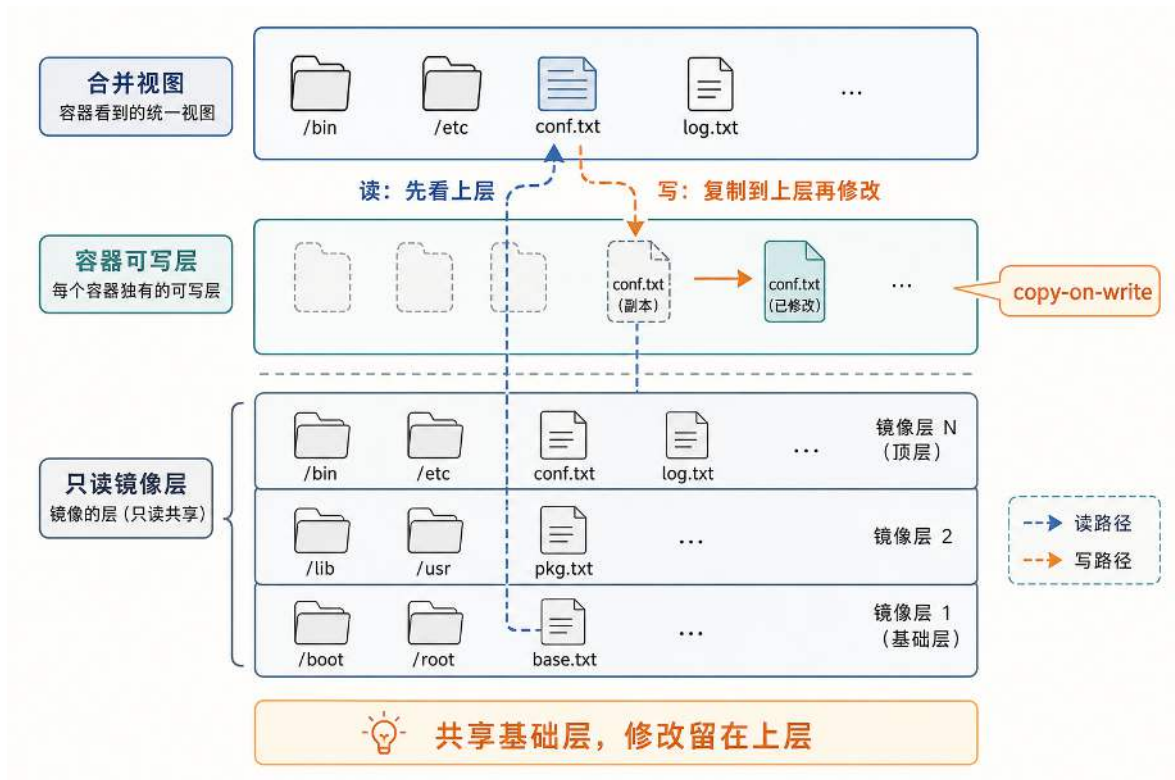


图 79: OverlayFS 将多层只读镜像层与容器可写层合并为统一视图, 写入时通过 copy-on-write 把文件复制到上层再修改。

写入路径上还藏着一个关键细节。当容器需要修改一个只存在于只读层的文件(如 `conf.txt`)时, 内核不会直接改动只读层, 而是先将该文件复制到可写层, 再在可写层中修改副本。这就是 **copy-on-write (写时复制)**: 只有实际发生写入时才触发复制, 复制粒度是整个文件 (而非块存储系统中常见的块级复制); 平时读取只读层中的文件不会触发 copy-up, 只承担合并视图查找带来的少量元数据开销。此后, 合并视图中该路径指向的就是可写层中的新版本, 只读层中的原始文件仍然完好。如果容器删除一个只读层中的文件, OverlayFS 不会修改只读层, 而是在可写层中创建一个特殊的 **whiteout** 文件, 标记该路径已被删除; 合并视图在遍历到 whiteout 标记时会跳过对应的只读层文件, 使其从容器视角消失。

至此, 前面提出的三个问题都有了明确的答案。容器看到的文件系统来自 **rootfs**: 一个被指定为根目录的目录树。多个容器通过共享只读镜像层来复用基础环境, OverlayFS 保证共享的同时互不干扰。在未挂载外部卷、bind mount 或 tmpfs 的路径上, 容器运行时产生的文件, 无论是新创建的还是对已有文件的修改副本, 都存储在该容器独有的可写层中。可写层的生命周期与容器绑定: 容器被删除时, 可写层随之清除, 只读镜像层不受影响。

这一机制也解释了一个常见现象: “删除容器后数据丢失”。运行时的改动保存在可写层中, 容器被删除时可写层一并清理, 镜像层提供的环境仍然保留, 但可写层中的运行数据已经不存在。如果希望数据不随容器消失, 就需要把数据放到不属于可写层的地方 (例如挂载宿主机目录或使用独立卷), 这就是容器持久化存储的动机。

综合来看, **rootfs** 解决的是 “容器把哪一份目录树当作 /”; **OverlayFS** 解决的是 “这份目

录树如何既能共享基础环境，又能允许每个容器独立写入”。理解了这两点，“镜像分层”“容器可写”“删除容器数据消失”等现象就可以从机制层面统一解释。

从工程视角看，这也正是容器解决“依赖环境复杂”的关键。游戏服务往往同时依赖特定版本的运行时与系统库：例如某个组件需要特定版本的 `glibc/libstdc++`，另一个组件依赖某套语言运行时与第三方包；如果这些依赖直接安装在宿主机上，就容易出现版本漂移与相互污染，导致同一份代码在不同机器上表现不一致。镜像提供的 `rootfs` 本质上是可复制的用户态依赖环境：可执行文件、动态库、配置目录都放进同一套目录树，容器启动时再通过挂载变成进程看到的根目录 `/`，从而让 `/bin`、`/lib`、`/usr` 的视图与宿主机解耦。

镜像分层与 OverlayFS 进一步把依赖环境的复用与隔离同时做到：基础层可以被多个服务共享（同一发行版与通用运行时只保存一份），服务层只叠加该服务独有的依赖与二进制；运行时写入则落在容器自己的可写层里，不会回写到基础层，也不会污染其他容器。这样，只要节点安装了容器运行时并能拉取同一份镜像，调度器把服务放到哪台机器上启动，依赖环境都是一致的。

6.1.4 容器的安全边界：共享内核带来的隔离限度

理解了 Namespaces、Cgroups、rootfs 与常见的 OverlayFS 实现之后，容器的大部分表面现象已经可以解释：它看起来像一台独立机器，有自己的进程列表、网络接口与目录结构，并且资源使用还能被限制。但在安全边界上，容器与虚拟机仍然存在本质差异。虚拟机的隔离来自**硬件虚拟化与 Hypervisor**：每个虚拟机通常运行自己的内核，内核之间由虚拟化层隔开。容器则不同：**所有容器共享同一个宿主机内核**。因此，容器的隔离更准确地说是“同一内核中的隔离”，而不是“不同内核之间的隔离”。

这一区别带来的直接后果是：Namespace 与 Cgroups 虽然能让容器“像是独立的”，但它们并不自动提供与虚拟机同等级别的安全边界。Namespace 主要改变的是**进程看到的系统视图**，例如看见哪些进程、哪些网络接口、哪些挂载点；Cgroups 主要控制的是**资源使用额度**，例如最多能用多少 CPU 时间、最多能占用多少内存。它们都不改变一个事实：容器内的进程执行系统调用时，最终仍然进入**同一个宿主机内核**。因此，一旦内核存在可被利用的漏洞，攻击者就可能借助漏洞突破隔离边界，从容器影响宿主机，这就是所谓“容器逃逸”的根源。

由于容器共享内核，工程实践通常会进一步收紧容器权限，以降低风险。例如，容器运行时会尽量减少进程拥有的特权能力（capabilities），并通过系统调用过滤（seccomp）限制可调用的内核接口；在需要更强策略控制时，还会配合 AppArmor/SELinux 等安全模块。若业务必须运行不可信代码（例如用户提交的脚本或插件），常见做法是引入额外隔离层：在容器与内核之间加入“系统调用拦截层”（如用户态内核方案），或者干脆把每个容器放入轻量虚拟机中运行，以换取更强的安全边界。选择哪种方案，本质上是在性能、隔离强度与运维复杂度之间做权衡。

本节的核心结论是：**容器之所以像独立系统，是因为 Namespace 改变了进程看到的系统视图，Cgroups 限制了进程组能使用的资源额度，rootfs 构造了容器看到的文件系统；OverlayFS 则是实现镜像层共享和容器独立写入的一种常见机制。而容器之所以不等于虚拟机，是因为它们仍然共享同一个内核，这决定了它们的安全边界与风险模型不同。**

到这里,单个容器已经具备隔离视图、受限资源和独立文件系统,但它的 Network Namespace 中只有一个回环接口 `lo`,既无法与同机的其他容器通信,也无法访问外部网络。这引出容器网络虚拟化的核心问题:如何在同一台宿主机上为多个隔离的网络命名空间搭建出一张互通的虚拟网络。

6.2 网络虚拟化原理:从隔离的网络命名空间到互通的容器网络

上一节说明, Network Namespace 为每个容器创建了一套独立的网络协议栈,包含独立的接口集合、路由表和端口空间。新创建的网络命名空间中,内核只提供一个回环接口 `lo`,没有外部链路,也没有可用的 IP 地址。容器要与其他容器或外部网络通信,必须由容器运行时在命名空间之间建立连接。

容器网络可以按两个维度分类:一是通信是否跨越宿主机,二是跨越宿主机边界时由哪一侧主动发起连接。本节以一条“典型 Linux 实现”为主线来讲解(veth、bridge、SNAT/DNAT 与 VXLAN),因为它把每一层要解决的问题都显式暴露了出来,便于逐步拆解。需要提醒的是,这只是众多实现中的一种:实际的 CNI 还可能采用三层路由、eBPF、云厂商 VPC 原生网络或硬件卸载,其数据路径不一定包含 bridge、iptables 或 VXLAN。读者应把下面的主线理解为“一种把问题讲清楚的典型路径”,而非唯一路径。本节围绕四个递进的问题展开。第一,容器如何获得一块可以收发数据包的网络接口和一个 IP 地址?第二,同一台宿主机上的多个容器如何相互通信?第三,容器主动访问外网与外部主动访问容器时,宿主机分别如何转发数据包?第四,不同宿主机上的 Pod 如何互通?这四个问题分别对应 veth、bridge、SNAT/DNAT 和跨主机路由或 Overlay 网络,不能把它们混为同一组候选方案。

6.2.1 容器的第一块网络接口: veth 对与命名空间互联

在讨论容器网络之前,需要先明确一个基础概念。Linux 中通常所说的“网卡”,更准确的名称是**网络接口 (network interface)**。网络接口不一定对应真实的硬件电路板,也可以是内核创建的软件对象。无论是物理接口(例如连接网线的以太网口)还是虚拟接口,它们对上层协议栈的作用相同:为内核协议栈提供一个收发网络数据的出入口。对于以太网接口和 veth 这类设备来说,这个出入口在链路层表现为二层以太网帧的收发。

上一节提到,新创建的 Network Namespace 中只有一个回环接口 `lo`。`lo` 是 Linux 的**回环接口 (loopback interface)**,从它发出的数据包不会到达任何外部链路,而是直接回到本机协议栈的接收路径。回环接口的作用是让同一命名空间内的进程通过 TCP/IP 互相通信(例如访问 `127.0.0.1`)。需要注意,`localhost` 并不等于 `lo`: `localhost` 是主机名解析层面的约定(通常在 `/etc/hosts` 中映射到 `127.0.0.1`),而 `lo` 是内核层的网络接口对象。程序访问 `127.0.0.1` 时,内核将回环地址的流量交给 `lo` 处理。然而,`lo` 无法承载与宿主机或其他容器之间的流量。

要让容器与外部通信,必须为它的网络命名空间添加一个可以连接到宿主机网络栈的接口。Linux 提供的基础机制是 **veth (virtual ethernet) 对**。veth 总是成对创建,形成两个网络接口对象,它们之间满足一条稳定性质:从一端发出的二层以太网帧,会出现在另一端的接收方向;

反过来亦然。veth 对在逻辑上等价于一条点对点的二层链路，常用于连接两个不同的网络命名空间。

在容器网络中，veth 对的典型用法是：把一端放入容器的 Network Namespace 并命名为 `eth0`，另一端留在宿主机的网络命名空间。`eth0` 历史上表示“第一块以太网接口”；现代 Linux 发行版的物理网卡可能采用可预测命名（如 `ens33`、`enp0s3`），但容器运行时通常仍将容器内的主接口命名为 `eth0`，与宿主机上的物理网卡无关。配置完成后，容器通过 `eth0` 发出的数据包会经 veth 对到达宿主机网络栈；宿主机发往该容器的数据包也会通过 veth 对进入容器的 `eth0`。至此，容器拥有了一个可以连接外部的网络出入口。

veth 对为容器提供了一条点对点的二层链路，但单独一对 veth 只能在直连的两端之间搬运以太网帧。要让容器能够向任意 IP 地址发送数据包，还需要两项三层配置：一个 IP 地址作为容器在网络中的身份标识，以及一条默认路由告诉内核将非本地网段的流量交给哪一个下一跳。容器运行时通常会完成这些配置：为容器分配一个私网 IP，并设置默认路由指向宿主机侧的网关地址（这个网关地址位于容器网段内，由容器运行时预先配置）。配置完成后，单个容器具备了发送和接收 IP 数据包的完整条件：有接口、有地址、有路由。但如果宿主机上运行了多个容器，它们各自的 veth 宿主机端仍然是彼此独立的接口，数据包并不会自动在这些接口之间转发。换言之，每个容器都已具备一条独立的二层链路，但这些链路的宿主机端彼此隔离，尚未接入同一台交换设备。

6.2.2 同宿主机容器互联与 bridge 组网

在实际部署中，同一台宿主机上往往运行多个容器，例如战斗服容器、匹配服务容器和配置中心容器。这些容器之间经常需要通过网络交换数据：战斗服启动时要从配置中心拉取房间参数，匹配完成后要通知战斗服创建房间。上一节已经为每个容器建立了通往宿主机的 veth 链路，但这些链路的宿主机端彼此独立，容器 A 发出的帧不会自动到达容器 B。问题因此变为：如何把多条独立的 veth 链路汇聚起来，使同宿主机上的容器像处于同一个局域网中一样互通。

解决思路与物理网络相同：多台主机各自有网线，要让它们互通，需要把网线插入同一台交换机。在容器场景中，Linux 内核提供的 **bridge（网桥）** 承担了这个角色。bridge 是一台纯软件实现的**二层交换机**，容器运行时将每个容器的 veth 宿主机端绑定到 bridge 的一个端口上，所有容器就处于同一个二层广播域中。图 80 展示了同宿主机容器通过 bridge 互联的拓扑。图中以 Docker 默认创建的 `docker0` 网桥为例：两个容器各自通过 `eth0` 和 veth 对连接到 bridge，形成二层局域网。

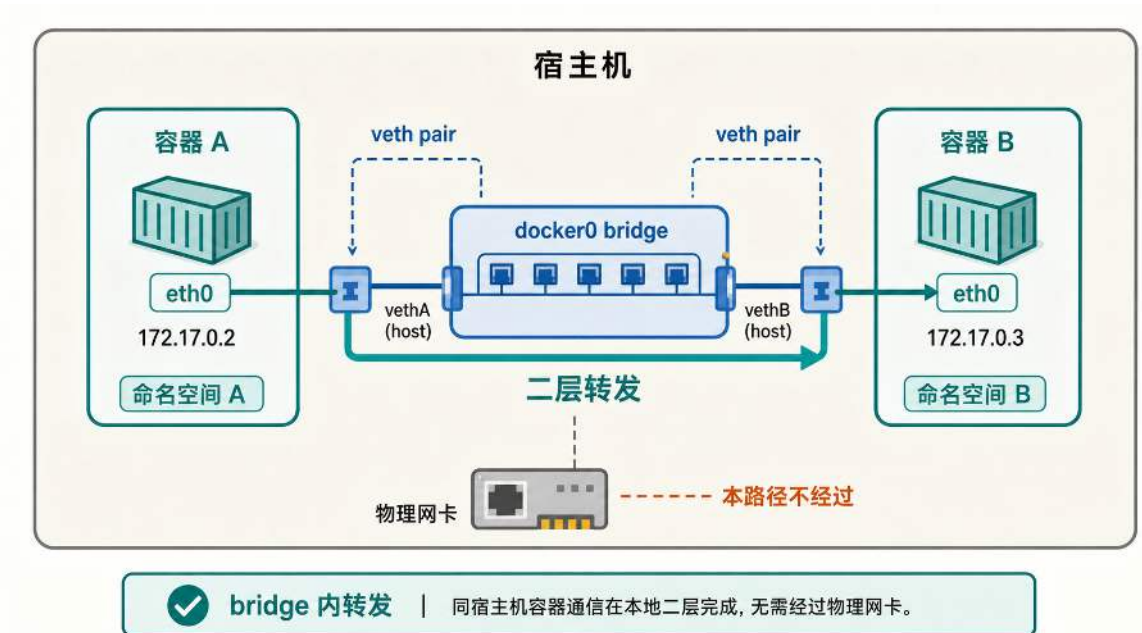


图 80: 同宿主机容器通过 veth 接入同一个 bridge，形成二层局域网。bridge 在端口之间转发以太网帧，使容器间可以按 IP 地址互相访问。

bridge 的转发机制决定了容器间通信的具体路径。bridge 内部维护一张 MAC 地址表，记录每个 MAC 地址最近一次从哪个端口出现。当某个端口收到以太网帧时，bridge 根据帧头中的目的 MAC 地址查表：如果目的 MAC 已有记录，帧只被转发到对应端口；如果尚未记录，bridge 将帧广播到除入端口外的所有端口。

以图中的容器间通信为例。容器 A (172.17.0.2) 要向同子网的容器 B (172.17.0.3) 发送 IP 数据包。IP 层判断目的地址 172.17.0.3 在本地网段内，不需要经过网关，但发送以太网帧需要知道目的 IP 对应的 MAC 地址。容器 A 通过 **ARP (Address Resolution Protocol, 地址解析协议)** 获取这一映射：A 发出一个广播帧，请求目的 IP 的持有者回复其 MAC 地址。该广播帧从 A 的 eth0 经 veth 进入 bridge，bridge 将其广播到除入端口外的所有端口，容器 B 收到后回复自己的 MAC 地址。A 获得 B 的 MAC 地址后，将数据帧的目的 MAC 填入帧头，从 eth0 发出。帧经 veth 到达 bridge，bridge 查表找到 B 对应的端口并转发；帧随后通过 B 的 veth 进入容器 B 的 eth0，交由 B 的协议栈处理。整个过程都在宿主机内核中完成，数据包直接在 bridge 内部完成二层转发，无需经过宿主机的物理网卡。

至此，同宿主机上的容器互联问题已经解决：veth 对为每个容器提供一条通往宿主机的二层链路，bridge 将这些链路汇聚为一张局域网并负责帧转发。在 veth 对已接入同一 bridge 且容器的 IP 地址配置在同一网段的前提下，它们即可通过二层转发直接互通。但容器通常还需要访问宿主机之外的服务，这就引出下一个问题：容器的数据包如何离开这张局域网，到达外部网络。

6.2.3 容器访问外网：默认网关、SNAT 与连接跟踪

上一节解决了同宿主机容器之间的互通，但容器通常还需要访问宿主机之外的服务，例如拉取依赖包、调用第三方 API 或连接外部数据库。此时容器面临的问题是：它的 IP 数据包如何离开由 bridge 组成的局域网，到达外部网络。

上一节已经说明，容器的默认路由会将非本地网段的流量指向宿主机侧的网关地址。现在需要回答的问题是：数据包到达网关后，如何继续走向外部网络，并让返回包最终回到容器？

在容器网络中，这个网关通常是 bridge 接口在容器网段中持有的 IP 地址。容器向外发送数据包时，先通过 ARP 获取网关的 MAC 地址，然后将帧从 eth0 经 veth 送入 bridge，进入宿主机的网络栈。宿主机随后根据自己的路由表，将数据包从 bridge 接口转发到通往外部网络的物理网卡接口（这要求宿主机内核已启用 IP 转发功能）。至此，数据包离开了容器局域网。

但数据包能够发出，并不意味着通信能够完成。容器使用的是私网地址（例如 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16），这些地址在公网或未配置容器网段路由的外部网络中不可路由：外部网络不会为它们维护路由条目，返回包无法按原路送回容器。因此，仅靠路由转发不足以完成外网访问，还需要在数据包离开宿主机时进行地址改写。

这一步由 **SNAT (Source Network Address Translation, 源地址转换)** 完成。宿主机在出口处将数据包的源 IP 从容器的私网地址改写为宿主机出口接口在外部网络中可达的地址。外部服务收到的请求看起来来自宿主机，因此返回包会先回到宿主机。Linux 提供了一种常用的 SNAT 规则形式：如果出口地址可能由接口动态决定，就不必在规则里写死一个源地址，而是让内核在转发时使用出口接口当前的地址完成改写，这种规则称为 **MASQUERADE**。在 Docker bridge 这类默认容器网络中，容器出网通常就是通过 iptables 或 nftables 中的 **MASQUERADE** 规则完成的。这条规则要回答三个问题：哪些包需要改写，改写成什么地址，以及改写后如何让返回包找到原来的容器。**MASQUERADE** 的核心动作如下：

MASQUERADE 的核心动作

```
1 func MasqueradeOutbound(pkt *Packet, out Iface) {
2     if !ContainerCIDR.Contains(pkt.SrcIP) {
3         return
4     }
5     if out.Name != HostExternalIface.Name {
6         return
7     }
8
9     original := pkt.FiveTuple()
10    pkt.SrcIP = out.CurrentIP()
11    pkt.SrcPort = NAT.AllocatePortIfNeeded(pkt.SrcPort)
12    rewritten := pkt.FiveTuple()
13
14    Conntrack.Remember(original, rewritten)
15 }
```

这段伪代码中的前两个判断说明，只有来自容器网段、并且即将从宿主机外部接口离开的包才需要被改写；中间两行改写源地址和必要的源端口，使外部服务看到的是宿主机出口接口；最后一行记录改写前后的五元组。这条记录正是返回路径成立的前提：宿主机收到返回包后，需要将目的地址从宿主机 IP 还原为发起请求的容器 IP，并通过 bridge 和 veth 将包送回正确的容器。这一还原过程依赖 **连接跟踪（connection tracking, conntrack）** 机制。内核在每次出向连接建立时记录一条映射：原始的容器 IP 与端口、改写后的宿主机 IP 与端口、以及对端的 IP 与端口。返回包到达宿主机时，内核在连接跟踪表中查找匹配条目，将目的地址还原后转发给对应的容器。SNAT 在改写源地址时将映射写入连接跟踪表，返回包到达宿主机时依据同一条目完成地址还原，二者共同完成双向通信。

这一机制也解释了容器网络中的一个常见现象：容器可以主动访问外网，但外部网络默认无法主动访问容器。出向 SNAT 只为容器发起的连接建立映射，外部网络不知道容器的私网地址；即使外部流量到达宿主机，宿主机内核的连接跟踪表中也没有对应条目可以匹配，这些数据包会被丢弃。如果需要从外部访问容器内的服务，必须通过额外的机制（例如端口映射或显式路由）为外部流量提供入口。

概括来说，容器访问外网依赖两层机制的组合：**路由转发**将数据包从容器局域网送往宿主机的出口接口，**SNAT 与连接跟踪**在出口处完成地址改写与还原，使外部网络能够将返回包送回宿主机，再由宿主机交还给正确的容器。

6.2.4 外部访问容器：端口映射与 DNAT

SNAT 只为容器主动发起的连接建立映射，尚未解决相反方向的问题：外部用户如何主动访问容器内的服务。外部流量要到达容器，面临两个障碍：容器使用的私网 IP 在外部网络中不可

路由，外部网络即使能到达宿主机也无法确定应将流量送给哪个容器。端口映射通过在宿主机入口处建立明确的转发规则来解决这两个问题。

端口映射的机制可以分为三步。第一步是**入口定位**：外部用户向宿主机的某个对外端口发起连接，例如访问 宿主机 IP:8080。宿主机的对外 IP 是公网可路由的，因此外部网络能够将数据包送达宿主机。第二步是**目的地址转换 (DNAT, Destination NAT)**：宿主机收到入向数据包后，在网络栈入口处将目的地址从 宿主机 IP:8080 改写为 容器 IP:80（假设容器服务监听 80 端口）。第三步是**转发到容器**：改写后的数据包经 bridge 与 veth 到达容器的 eth0，交给容器内的服务进程处理。

SNAT 与 DNAT 都发生在宿主机中，也都依赖连接跟踪完成往返，但二者不能混为同一条路径。图 81 把它们放在同一坐标系中比较：上半部分由容器主动发起连接，宿主机改写源地址；下半部分由外部用户主动发起连接，宿主机改写目的地址。两条返回路径都依据连接跟踪中保存的映射还原地址。

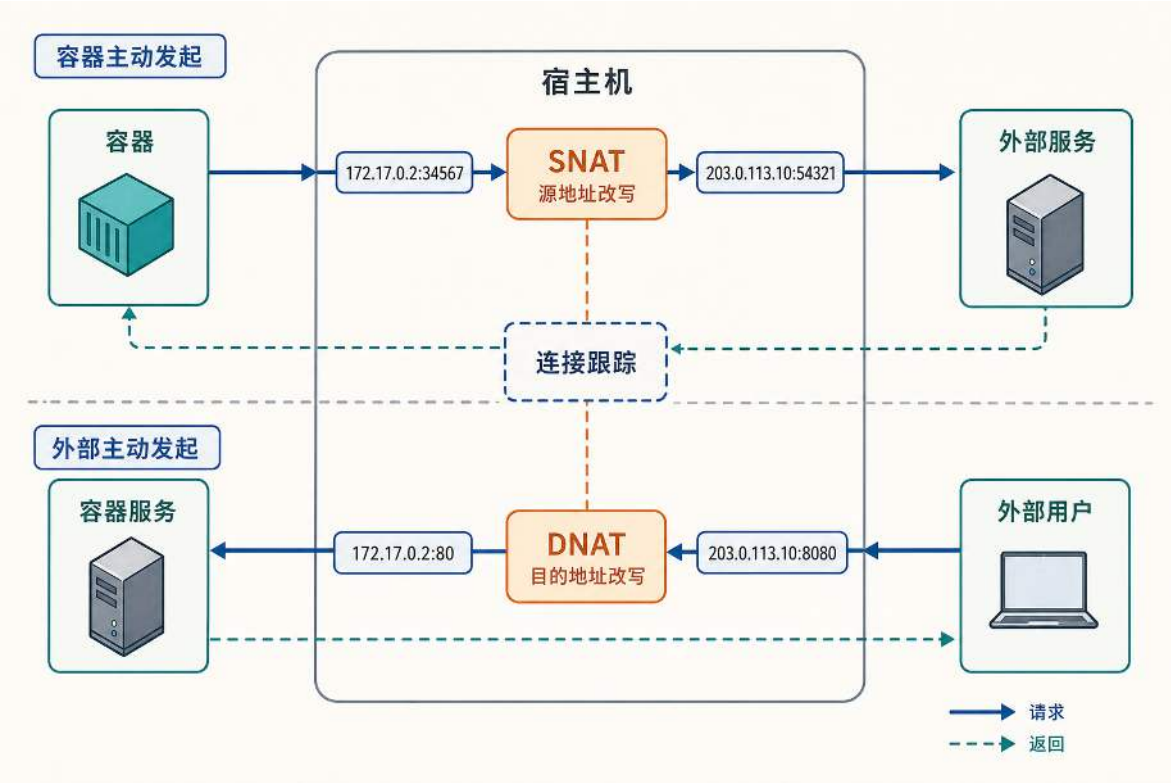


图 81: SNAT 与 DNAT 的方向对照：容器主动出网时改写源地址，外部主动入网时改写目的地址；两类连接都由宿主机记录映射，并在返回方向依据连接跟踪还原地址。

端口映射是双向的会话级转发，而不只是入向的一次改写。DNAT 建立后，宿主机通过连接跟踪记录这条连接的映射关系。返回数据包离开宿主机时，源地址需要从 容器 IP:80 还原为 宿主机 IP:8080，否则外部客户端会收到来自未知地址的响应，导致连接失败。因此，端口映射的完整语义是：**入向改写目的地址并送入容器，返回改写源地址并送回外部。**

这一机制也解释了几个常见问题。端口映射必须在宿主机上完成，因为外部流量首先到达宿

主机的对外 IP，容器的私网 IP 对外不可达，容器本身无法接收外部的入向数据包。映射以端口为单位，因为 TCP/UDP 连接通过四元组（源 IP、源端口、目的 IP、目的端口）标识，宿主机正是通过不同的对外端口将流量分流到不同容器。在简单的一对一端口映射中，同一个宿主机端口不能同时映射给多个容器，因为相同的入口地址对应不同的转发目标时，宿主机无法确定数据包应送往哪个容器（若需将同一入口端口对应多个后端，则需要额外引入负载均衡或服务代理层）。

概括来说，端口映射的作用是：**容器仍处于私网中，外部网络只能到达宿主机；宿主机通过 DNAT 与连接跟踪，将指定端口上的入向连接转发给目标容器，并将响应以宿主机身份送回外部。**

第五章介绍的 Kubernetes Service 位于更高的抽象层：它为一组会变化的 Pod 提供稳定地址，并按照后端列表选择目标实例。默认的 ClusterIP 主要服务集群内部访问；需要从集群外接入时，还要通过 NodePort、云负载均衡器、Ingress 或 Gateway 提供外部入口。流量进入节点后，可以由地址转换、路由、代理或其他数据面实现送往目标 Pod。因此，Service 解决的是“调用方通过哪个稳定入口找到一组后端”，DNAT 解释的则是其中一种底层地址转换机制，二者不能直接等同。

到这里，单台宿主机的两种边界通信已经完整：容器主动出网由 SNAT 建立返回路径，外部主动入网由 DNAT 建立明确入口。接下来讨论另一类问题：通信双方都位于集群内部，但 Pod 分布在不同宿主机上，此时需要解决 Pod 地址如何跨越物理网络。

6.2.5 跨宿主机的 Pod 互通：Overlay 网络与 VXLAN 封装

前几节解决了同宿主机容器互通以及容器网络的出入方向。但在 Kubernetes 集群中，服务通常分布在多台宿主机上，不同宿主机上的 Pod 之间同样需要按 IP 地址直接通信。此时的核心问题是：**不同宿主机上的 Pod 地址如何彼此可达。**

单机内部的 bridge 和 ARP 广播域仅限于本机。一旦跨出宿主机，数据包进入机房的三层物理网络，由路由器按宿主机 IP 转发。物理网络通常不会为每个 Pod 的地址维护路由条目，也不会将 Pod 网段的二层广播跨宿主机扩散。因此，Node1 上的 PodA 发往 Node2 上 PodB 的数据包，仅靠本机的 veth 和 bridge 无法送达对端。

一种直接的思路是将每个节点的 Pod 子网作为路由前缀发布到机房物理网络中，使路由器直接转发 Pod 地址。这在原理上可行，但在通用集群中会带来工程代价：如果按节点级 Pod 子网发布路由，节点加入、退出或 Pod 子网重新分配时，底层网络需要同步更新；如果进一步细化到每个 Pod 或更小粒度的路由，Pod 迁移和扩缩容又会把更高频的虚拟端点变化暴露给物理网络。物理设备的转发表容量、路由收敛速度和运维边界都不适合无限承接这类变化。另一种常见的工程取舍是：**物理网络只负责转发宿主机之间的三层流量，Pod 的互通由节点上的软件在边界处完成。**

这一取舍对应云原生网络中的两个概念。在本节采用的 VXLAN Overlay 模型中，**Underlay** 是机房的底层网络，只需要转发节点或 VTEP 地址，无需维护 Pod 地址的转发状态。**Overlay** 则是在 Underlay 之上构造的一张逻辑网络，使 Pod 的地址在集群范围内可达。VXLAN 是常见的 Overlay 实现之一。

VXLAN 的核心机制是封装与解封装，它将两层网络的职责分开：**物理网络中只出现节点到节点的流量，通信两端仍保持 Pod 到 Pod 的地址语义**。每个节点上都会配置一个 VXLAN 隧道端点，称为 **VTEP (VXLAN Tunnel Endpoint)**；它可以由内核 VXLAN 设备、虚拟交换机或网络插件配置出来，不一定是一个独立运行的用户态进程。当本机某个 Pod 发出的数据包的目的地址不在本机时，数据包不会直接交给物理网卡，而是被路由到本机的 VTEP。VTEP 把 Pod 之间完整的**内层以太网帧**整体作为载荷，外面再套上一层“节点 IP + UDP + VXLAN”报头（其中外层 IP 以节点地址为源和目的），使物理网络能够按宿主机路由转发。对端节点的 VTEP 收到后执行解封装，剥掉外层报头，还原出原始的 Pod 间以太网帧，再交给本机的 bridge 和 veth 送入目标 Pod。

图 82 把这一分工具体化：外层用节点地址供物理网络转发，内层保留 Pod 地址在两端不变。数据包因此在物理网络里以节点身份穿行，到两端再还原成 Pod 到 Pod 的通信，物理网络自始至终看不到 Pod 地址。

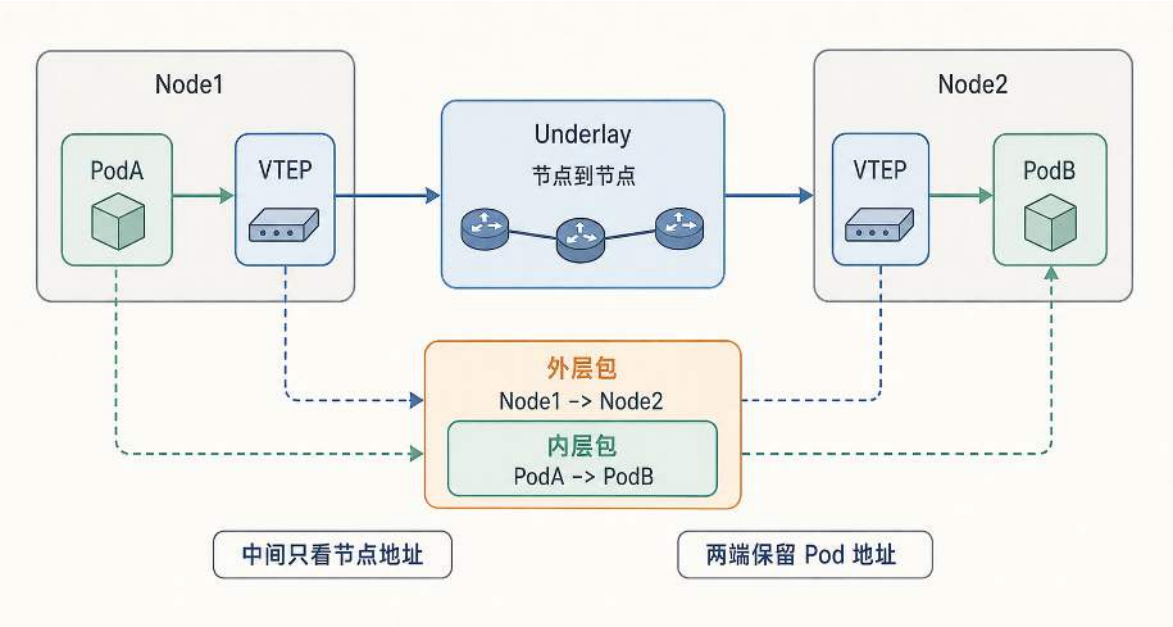


图 82: VXLAN 封装结构：内层保留 Pod 到 Pod 的地址语义，外层使用节点到节点的地址，使物理网络无需感知 Pod 地址。

以一次具体的跨节点通信为例。PodA 位于 Node1，PodB 位于 Node2。PodA 发往 PodB 的数据包从 PodA 的 `eth0` 经 veth 进入 Node1 的网络栈。Node1 的路由判断目的 Pod 不在本机子网内，将数据包交给 VTEP。VTEP 不改变内层以太网帧所承载的 PodA 到 PodB 的 IP 包，而是在整个内层以太网帧外添加以 Node1 为源、Node2 为目的的外层报头。封装完成后，物理网络将其作为一个普通的节点间报文转发到 Node2。Node2 的 VTEP 执行解封装，去掉外层报头，还原出原始的内层以太网帧。此后回到单机路径：帧进入 Node2 的 bridge，经 veth 送入 PodB 的 `eth0`，再由 PodB 的协议栈处理其中的 IP 包。从端到端看，PodA 与 PodB 始终按 Pod 地址通信；从物理网络看，跨宿主机这一跳只是节点到节点的常规转发。**主机内互通由**

veth 和 bridge 完成，跨主机这一跳由 VXLAN 的封装与解封装补齐。

VTEP 在封装时需要知道目的 Pod 所在的节点地址。这依赖一张映射表：在 Kubernetes CNI 的教材化理解中，可以把它看成“Pod 地址段到节点/VTEP 地址”的对应关系；在更底层的 VXLAN 语义中，它也会涉及内层二层标识、VNI 与远端 VTEP 地址的对应。这里最关键的是：远端地址不是公网意义上的“外部 IP”，而是该节点在 Underlay 网络中可达的节点地址或 VTEP 地址。集群的控制面（例如 Kubernetes 的网络插件）负责维护和分发这类映射，VTEP 按表完成封装。**控制面维护并分发映射表，数据面按表完成封装、转发与解封装。**

封装还会带来一个附加影响：同样的业务数据在外层报头的包裹下，在链路上占用更多字节。链路一次能够承载的最大报文长度称为 MTU (Maximum Transmission Unit, 最大传输单元)。如果封装后的报文长度超过物理链路的 MTU，系统可能需要将报文拆成更小的片段转发，或者在某些配置下直接丢弃。因此，使用 Overlay 网络时通常需要适当调低容器侧的 MTU，为外层报头预留空间。

至此，容器网络的主机内互通、出向访问、入向访问和跨主机互通已经分别建立。它们解决的是不同通信范围和发起方向的问题，不能把 SNAT、DNAT 与 VXLAN 当作同一目的下的替代方案。

6.2.6 低开销的容器网络：Overlay 封装的代价与前沿优化

前述 Overlay 网络用 VXLAN 把每一个跨宿主机的数据包重新封装一次，让分布在多台机器上的容器像在同一张二层网络里一样互通。这层封装换来了可移植性，但会带来额外开销：每个数据包在发送端要被封装、在接收端要被解封装，封装后的包还要在宿主机内核的网络协议栈里多走一遍。当容器间通信频繁、吞吐要求高时，这部分固定开销会直接体现在延迟、吞吐和 CPU 占用上。

具体而言，许多典型 Overlay 实现会在内核里走**逐包转换**：对每个包做连接跟踪、包过滤、路由查表，再套上外层 VXLAN 头。结果是同一个包要在发送端和接收端各自的内核网络栈里穿行两次——一次处理容器看到的内层地址，一次处理宿主机之间传输用的外层地址。对一条已经建立的连接来说，这些处理每个包都要重复执行，即使结果变化很小。针对这种固定开销，研究者给出了两条思路，都可以落回本书关心的高频交互面。

第一条思路是把虚拟化从“每个包”提升到“每条连接”。Slim³⁰的做法是：只在连接建立时改写一次连接级的元数据，之后这条连接上的数据包就直接走宿主机的网络栈，不再逐包封装，每个包只穿越内核网络栈一次而不是两次。论文报告，在内存键值存储场景下，这一改动把吞吐提升约 66%、延迟降低约 42%、CPU 占用降低约 54%。代价是它工作在连接级，需要专门的组件介入 socket 建连过程，改变了标准 socket 的部分行为。

连接级改写还引入了一个更直接的新代价：建立连接时，收发双方要额外做一轮往返，交换宿主机 socket 与容器 socket 的映射信息，连接建立因此变慢。对需要高速处理大量短连接的应

³⁰Danyang Zhuo 等, *Slim: OS Kernel Support for a Low-Overhead Container Overlay Network*, NSDI 2019, pp. 331–344. <https://www.usenix.org/conference/nsdi19/presentation/zhuo>

用（如代理、缓存），这笔建连开销会盖过数据传输上省下的收益。SlimFast³¹正是针对这一点，在保留 Slim 低数据开销的同时优化了建连流程，让短连接也能受益；论文报告，相较 Slim，在短连接实验中，SlimFast 将 Nginx 代理吞吐提高约 89%，并将 Memcached 响应率提高约 219%。可以看到，连接级这条路线内部也在推进：先用 Slim 消除逐包封装，再用 SlimFast 补上它带来的建连开销。

第二条思路是保留标准 Overlay 的数据路径，但把其中“每个包都重算、结果却不变”的部分缓存下来。ONCache³²观察到：一条连接存活期间，连接跟踪结果、包过滤结果、主机内路由决策和外层封装头这几项在正常情况下都是不变的。于是它用 eBPF 在收发路径两端各挂一段程序，第一个包正常走完整路径并把这些结果缓存起来，后续包命中缓存后直接套用缓存的外层头、按缓存的路由转发，跳过重复处理。整套快速路径只用了约 524 行 eBPF 代码，可以作为 Antrea、Flannel 等标准 Overlay 的插件叠加上去；一旦缓存不可用或网络发生变化（容器删除、迁移、过滤规则变更），流量自动回退到标准路径，保证正确性。论文报告 TCP 吞吐和请求-响应事务率分别提升约 12% 和 36%，同时显著降低每包 CPU 开销。

图 83 将三种路径并列起来：标准 Overlay 把封装、路由和过滤留在逐包路径上，Slim/SlimFast 将改写提升到连接建立阶段，ONCache 则让首包建立流级缓存并让后续包复用结果。三者的比较重点在于重复处理被移出逐包数据路径的方式：连接级改写更深地介入内核与 socket 路径，缓存旁路更接近对标准 Overlay 的加速插件，二者对应不同的兼容性和适用场景。

³¹Fusheng Lin 等, *Slim and Fast: Low-Overhead Container Overlay Network With Fast Connection Setup*, IEEE Transactions on Cloud Computing, Vol. 12, 2024, pp. 1–12, DOI:10.1109/TCC.2023.3282238. <https://ieeexplore.ieee.org/document/10143220>

³²Shengkai Lin 等, *ONCache: A Cache-Based Low-Overhead Container Overlay Network*, NSDI 2025, arXiv:2305.05455. <https://arxiv.org/abs/2305.05455>

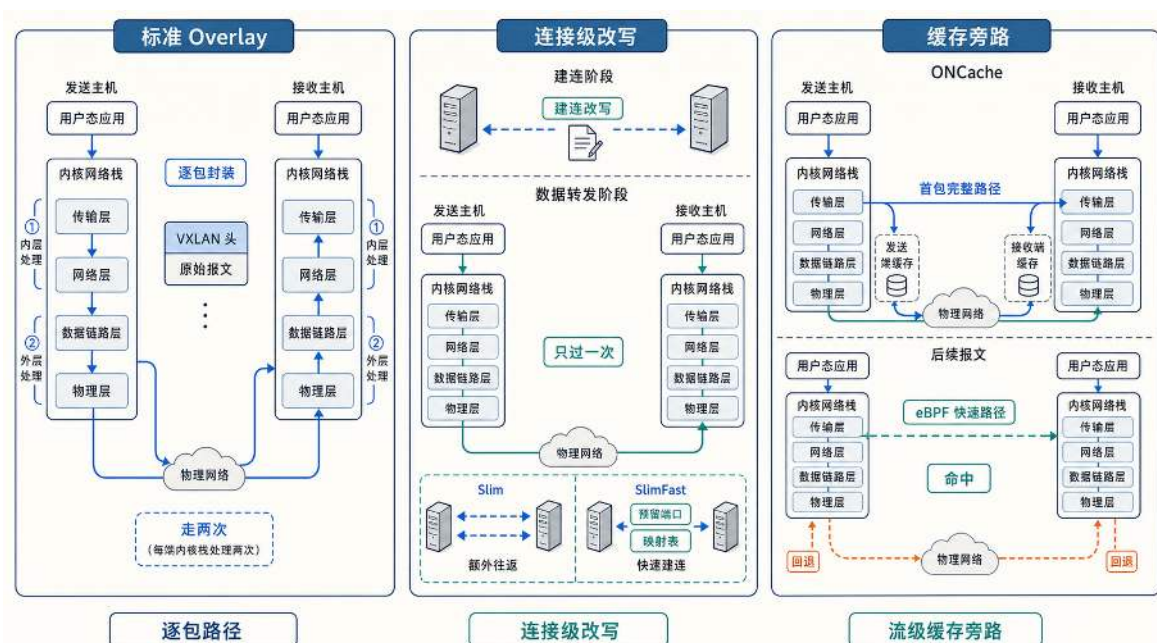


图 83: 三种容器 Overlay 网络处理路径的对比：标准 Overlay 将封装、路由和过滤留在逐包路径上，Slim 与 SlimFast 将虚拟化改写提升到连接建立阶段，使数据转发阶段减少重复的内核栈处理，ONCache 则缓存每流不变量，让后续包复用首包得到的处理结果。图揭示的是两类降低固定开销的思路：连接级改写与缓存旁路都在把重复处理从逐包数据路径中移出，但二者在侵入性、兼容性和适用场景上不同。

这两条优化思路都可以回收到本章的交互面：当大量隔离环境持续交换动作、观察和同步消息时，跨主机通信中的固定开销会被交互频率放大。Overlay 的通用封装换来了可移植性；连接级改写和缓存旁路则把部分通用处理移出高频路径，代价是更强的内核依赖和兼容性约束。后文讨论 Agent sandbox 时仍会遇到同一类取舍：交互越密集，越需要判断哪些通用层次可以保留，哪些固定开销必须下压。

6.2.7 小结：网络虚拟化的层次结构

容器网络需要依次解决四个递进问题：隔离各自的网络栈、建立主机内互通、打通外网出入口、实现跨宿主主机通信。对应这四个问题，网络虚拟化可以归纳为四个层次。

第一层是**分栈**：Network Namespace 将网络协议栈切分为多份，同一 Network Namespace 内的进程共享一组独立的接口集合、地址、路由与端口空间。分栈之后，每个网络栈最初只有回环接口 lo，隔离并不会自动产生互联。

第二层是**连线与组网**：veth 提供点对点的二层连接，将不同命名空间的接口端点接回宿主机；bridge 将多条二层连接汇聚为一张虚拟局域网，使同局域网内的容器通过 ARP 与 MAC 地址表完成直接互通。这一层完成的是主机内部的局域网虚拟化。

第三层是**出入口控制**：宿主机在虚拟局域网与外部网络的边界处承担路由转发角色，通过

SNAT 与连接跟踪使容器能以宿主机身份访问外网，通过 DNAT 与端口映射为外部流量提供入口。

第四层是**跨主机互通**：当容器分布在多台宿主机上时，Overlay 网络（如 VXLAN）通过封装与解封装，在物理网络之上构造一张逻辑网络，使不同宿主机上的容器地址（在 Kubernetes 中对应 Pod 地址）在集群范围内可达。

在此基础上，针对 Overlay 封装带来的固定开销，研究者还提出了连接级改写与缓存旁路等优化思路，它们不增加新的网络层次，而是把重复处理移出高频数据路径。

网络虚拟化通过隔离网络栈、构造二层连接、提供二层汇聚、在边界做三层转发与地址转换、在集群范围内做隧道封装，让同一台或多台物理主机上出现多个逻辑独立且可控互联的网络环境。

6.3 弹性伸缩的应用基础：无状态与有状态

上一节的网络虚拟化解决了实例之间如何互通，但实例能够互通，并不意味着应用能够安全地增减实例。第五章已经从业务场景说明了为什么需要弹性伸缩，并介绍了 HPA、Cluster Autoscaler 和 Serverless 等使用方式。本节进一步讨论应用侧的前提：平台增加一个副本后，这个副本怎样真正承担业务负载；平台回收或替换副本时，应用又怎样保持业务正确性。

副本数增加并不等于有效容量增加。平台创建和回收的是执行实例，而应用实际分配和保护的是队列分片、房间、会话等状态单元；只有当状态单元的处理权与新实例对应起来时，新增副本才真正分担负载。一个新实例要走到这一步，至少要通过三道门。第一道是**资源与实例就绪**：平台已经为实例分配计算资源，应用也完成了配置加载、依赖连接和缓存预热。第二道是**流量接入**：服务发现和请求路由已经把新实例纳入可用路径。第三道是**状态接管**：如果请求绑定队列分片、房间或其他状态单元，新实例还要获得相应的处理权。实例只通过第一道门，只能说明进程已经运行；通过前两道门，才可能收到请求；三道门全部通过，有状态服务的新增副本才会形成有效容量。

这三道门主要回答扩容时新实例如何进入服务。安全伸缩还要处理实例退出的过程：计划缩容时有排空机会，故障时则可能突然中断。此外，一次请求在结果不明确时还可能被重复投递，应用必须保证重试不会破坏业务正确性。因此，弹性伸缩不仅要让新实例顺利接入，还要定义旧实例怎样安全退出、故障后怎样恢复、以及重复请求怎样保持幂等。下面先从不需要交接业务状态的可替换实例开始，再讨论有状态服务怎样跨过第三道门。

6.3.1 无状态与可替换实例：安全水平伸缩的六项条件

到达的请求如果可以交给任意一个可用实例处理，不依赖某个实例独占的本地业务状态，这类服务通常称为**无状态服务**。无状态并不意味着服务完全不使用状态，而是指需要保留的状态不绑定在特定实例的本地内存或文件系统中。实例因此可以被替换，不需要先把一份业务状态的修改权交给新实例。对无状态服务而言，第三道门被简化了，扩容主要取决于新实例能否完成初始化并进入流量路径。

让无状态服务能够安全地水平伸缩，需要满足六项设计条件。需要先厘清的是，真正定义“无状态”的只有状态外置一项：它决定实例是否可以被任意替换。其余五项并不属于无状态的定义，而是让水平伸缩能够安全进行的配套条件。为便于理解，可以把这六项按解决的问题分成三组。

第一组是**状态与实例**，它决定服务本身是否真正无状态。

状态外置。所有需要在实例重启或迁移后仍然保留的数据，必须写入外部存储（数据库、缓存或对象存储），不能仅存于进程内存。例如玩家签到记录、订单信息和持久化配置都应写入 PostgreSQL 或 Redis；实例本地内存中只允许保留可重建的缓存或临时计算结果。只有做到这一点，任意实例才能接手任意请求，实例才成为可替换的单元。

有些系统会让负载均衡器把同一用户长期送往同一实例，以便继续使用本地会话。这种做法称为**会话粘性**。它可以减少会话读取开销，却没有消除状态与实例的绑定：实例故障后，本地状态仍会丢失；实例负载不均时，请求也不能自由转移。会话粘性可以作为性能优化，但不能替代状态外置或状态恢复协议。状态外置同样不是没有代价，它会把一部分本地访问变成远程访问，并使数据库或缓存成为共享依赖；设计者还要为外部存储准备容量、故障恢复和一致性机制。

第二组是**流量接入**，它决定新增或替换的实例能否被正确地接入流量。

稳定入口。客户端始终连接一个固定地址（域名、虚拟 IP 或网关），由入口层根据后端实例列表动态路由。实例增减时，客户端不需要更换目标地址。Kubernetes Service 或云负载均衡器承担的就是这一角色。

服务发现。入口层需要及时获得当前可用的后端实例集合。成员信息既可以由实例主动注册，也可以由平台控制器根据实例状态维护；在 Kubernetes 中，平台根据 Pod 的标签与就绪状态更新 Service 的后端集合。没有服务发现，新扩容的实例无法承接流量，已经失效或尚未就绪的实例也可能继续收到请求。

就绪检查。实例启动后并非立刻可用：可能需要加载配置、预热缓存、建立数据库连接。平台通过就绪探针确认实例真正具备处理能力后，才将其加入服务发现列表。跳过就绪检查会导致请求被发给尚未准备好的实例，引发超时或错误。

第三组是**重试与生命周期**，它保证在重复投递和实例退出时业务仍然正确。

幂等性。同一请求被多次处理时，结果应当与处理一次相同。平台在故障恢复或网络重试时可能重复投递请求；如果服务不具备幂等保护，重复请求就会变成重复发奖或重复扣款。典型的幂等实现是为每次业务操作生成唯一键，并用数据库唯一约束或条件写入把“检查是否已处理”和“记录已处理”合并为原子动作。

优雅下线。实例终止前需要先停止接收新请求，把已接收的请求处理到可安全中断的位置，再退出进程。对短请求服务，这意味着完成存量请求；对长连接服务，这意味着等待连接自然关闭或完成会话迁移。平台提供终止信号和超时窗口，但何时能够安全退出由应用定义。

这六项条件并不是同一层面的属性，而是分别作用于实例生命周期的不同阶段。状态外置使实例成为可替换单元；就绪检查保证实例通过第一道门，稳定入口与服务发现使它通过第二道门；幂等性处理结果不明确后的重复请求；优雅下线则保证实例退出时先处理存量工作。一个没有为伸缩设计过的应用，即使能够被平台复制，也不一定能形成有效容量；即使状态已经外置，缺少

幂等保护或优雅下线，重试和缩容仍可能破坏业务正确性。

6.3.2 有状态服务：状态归属决定伸缩方式

并非所有服务都能做成无状态。游戏战斗服维护的对局状态、匹配队列中的等待关系、实时协作系统正在编辑的文档，都要求系统持续保存并协调其变化。这类服务称为**有状态服务**。平台可以为它们创建新实例，但新实例能否形成容量，取决于应用能否把状态单元及其处理权交给新实例。

分析有状态服务时，不应先根据服务名称判断它“属于哪一类”，而应围绕权威来源、修改权、状态粒度、生命周期和变化处理五个问题，逐步确定状态怎样随实例交接。这五个问题并非彼此孤立，可以沿着“正确性依据、分配方式、变化处理”的顺序展开。

首先要确定系统依据什么维持正确性。发生分歧或故障恢复时，哪一份记录是权威来源；系统正常运行时，又由一个实例独占修改权，还是允许多个实例并发修改。前者决定恢复时以什么为准，后者决定并发更新和所有权转移需要采用什么机制。若某一时刻只能有一个所有者，转移修改权时还必须阻止旧所有者继续写入。

确定正确性依据后，还要说明状态怎样分配给实例。系统分配和迁移的是一名用户、一场对局、一个队列分片，还是整个数据库分片；粒度越小，负载越容易均衡，但需要维护的归属记录和路由规则也越多。状态的生命周期又决定了这种分配能否及时调整：短请求产生的临时状态可能很快释放，一场对局或一次协作会话则会持续较长时间，实例退出时未必能够等待它自然结束。

最后才能选择实例变化时的处理方式。扩容时，新实例是重新计算状态，还是接管已有状态；计划缩容时，存量状态是等待排空，还是在线迁移；实例故障时，又要从哪一份持久记录恢复。不同选择会改变服务中断时间、实现复杂度和资源开销，也决定新增实例能否真正形成容量、退出实例会不会破坏业务正确性。

根据这些问题的主要回答，工程上可以归纳出四种常见的状态特征。它们帮助设计者识别主要风险，但并不是互斥的服务分类；同一份状态可能同时具有其中多种特征。

可重建缓存：例如从数据库加载的玩家基础信息缓存、热点商品列表。这类状态丢失后可以从持久存储重新加载，只是会短暂增加延迟。伸缩时不需要专门迁移，新实例启动后自行重建即可。

连接与会话状态：例如 WebSocket 连接与玩家会话的对应关系、正在进行的对局上下文。这类状态如果丢失，相关连接或会话会被直接中断。伸缩时需要决定：等待存量会话自然结束，还是把会话状态迁移到新实例；前者实现简单但下线较慢，后者切换更快但需要恢复连接和状态的协议。

共享协调状态：例如匹配队列、协作文档和房间归属表。它们会被多个用户或实例共同读写，主要问题是规定更新顺序、处理并发冲突和转移修改权。单纯把数据存入外部数据库并不能自动解决这些问题，应用仍需选择锁、事务、版本号或共识等协调机制。

权威持久状态：例如账户余额、已完成订单和角色资产。这类状态是后续业务判断的依据，主要问题是已经确认的结果不能丢失，恢复后也不能退回更早的值。它通常由数据库或分布式存储系统保管；系统是否允许副本暂时出现差异、读操作能看到多新的值，则由所选一致性模型决

定。

共享协调与权威持久描述的是两个不同维度。前者关注“并发修改时怎样形成唯一有效的顺序或所有者”，后者关注“系统恢复后以什么记录作为事实”。匹配队列在运行时需要协调多个玩家和处理实例，它记录的“玩家是否仍在等待”也可能是匹配阶段的权威事实；账户余额必须持久保存，同时也需要事务或锁来协调并发扣减。因此，四种特征不能代替前面的五个问题。

有状态服务的伸缩还要区分两个对象。**伸缩单元**是平台创建和回收的实例，**状态单元**是应用分配和保护的话、房间、任务或分片。增加一台战斗实例，只是增加了一个伸缩单元；只有把新的对局或可迁移的既有对局分配给它，系统才增加了实际对局容量。状态单元的粒度、所有权和生命周期，决定了伸缩单元怎样通过第三道门。

当状态量大到单台机器无法承载时，系统需要**分片**（sharding）：把状态按某个键（如玩家ID、房间号、地理区域）划分到不同实例。分片使每个实例只负责一部分状态，从而让总容量随实例数增长。但分片也带来新的约束：请求必须按分片键路由到正确实例；扩容时需要**数据重平衡**，把部分分片从旧实例迁移到新实例；缩容时则需要把被回收实例上的分片迁走。

如果状态数据保存在原实例本地，数据重平衡通常要经过四个步骤。第一步复制一份**基础快照**，让目标实例获得某个时刻的完整起点；如果没有这一步，目标实例就只能从空状态开始。第二步追平复制快照期间产生的**增量更新**，把新旧实例之间的差距缩小到可以切换的范围。第三步切换**状态归属与请求路由**，使后续请求到达新所有者。这一步对应的正是状态分析五问中的第二问——修改权的转移：为防止旧所有者在网络延迟或暂停恢复后继续写入，可以为每次归属分配一个单调递增的版本号，存储端只接受当前版本的写入；这种用新版本阻挡旧所有者的编号称为**隔离令牌**（fencing token）。第四步验证新实例能够稳定处理读写，再清理旧副本；过早清理会使切换失败后失去回退路径。

并非每次状态接管都要复制大量数据。如果匹配玩家已经保存在共享队列中，扩容时可以只转移队列分片的处理权、更新隔离令牌和路由；如果数据只存在原实例本地，才需要走完快照、增量追平和切换过程。只有当迁移同时改变某个共识组的副本集合时，才需要使用第四章介绍的Raft 成员变更机制。有状态伸缩的共同点不是采用同一种迁移算法，而是让状态数据、修改权和请求路径完成一致的交接。

6.3.3 案例：游戏服务端的状态架构与弹性伸缩

前两小节分别建立了有效容量的三道门和状态分析五问。下面沿“登录与大厅、匹配、战斗、结算”这条业务路径，观察同一个游戏服务端怎样处理扩容、缩容和故障替换。

先观察扩缩容为何失效 活动开始后，在线人数快速增加，匹配队列长度和等待时间持续上升。平台根据负载指标增加了大厅、匹配和战斗实例。新增大厅实例完成初始化并进入服务发现列表后，能够分担登录和查询请求；新增匹配与战斗实例虽然已经运行，匹配吞吐却没有同步提高，新实例的资源利用率也一直很低。原因是应用仍使用部署时写定的匹配分片归属表和房间分配规则：队列分片仍由原有匹配实例处理，新对局也仍被分配给原有战斗实例。新增副本通过了资源与实例就绪这道门，却没有完成状态接管，因而没有形成有效容量。

另一次流量回落后的缩容又产生了另一类故障。平台选择一台战斗实例并终止进程，但应用没有先停止分配新对局，也没有等待正在进行的对局结束。该实例上的玩家连接随进程一起断开，对局上下文又只存在本地内存，替代实例无法恢复。这说明缩容不是扩容的逆操作：扩容关注新实例如何接管工作，缩容必须先使旧实例失去新工作的入口，再处理它已经拥有的状态单元。

用状态分析五问拆解业务路径 玩家资料的权威来源是数据库，大厅实例只保存可重新加载的缓存，缓存的生命周期可以与实例一致。登录连接由当前网关持有，连接断开后可以由客户端重新建立，但用户会话和目标战斗服地址要保存在外部存储，不能只存在网关内存中。

匹配阶段的状态单元是队列分片。共享队列记录哪些玩家仍在等待，分片归属记录当前哪个匹配实例拥有处理权；它们既涉及并发协调，也是匹配阶段作出后续判断的依据。战斗阶段的状态单元则是一场对局。对局从房间创建开始，到结算结果确认后结束；在这段时间内，某个战斗实例负责推进状态。结算结果和角色资产由数据库作为权威来源，并以“对局标识与玩家标识”等唯一业务键识别重复提交。

经过这一步，系统不再按“大厅服无状态、战斗服有状态”给组件贴标签，而是明确了每份状态的权威来源、当前所有者、分配粒度、生命周期和实例变化时的处理方式。

让新增实例获得匹配分片 系统按照游戏模式和玩家所在区域等稳定字段划分匹配队列，把待匹配玩家写入按分片组织的共享队列，并用一份共享的归属记录说明每个分片当前由哪个实例处理。归属调整不能只看实例数量，还要观察各分片的等待人数、请求到达速率和处理速率，再选择需要转移的分片，使新旧实例承担的工作回到各自可处理的范围内。

增加匹配实例时，新实例先完成初始化并声明就绪，请求入口随后才能把它纳入可用成员集合。负责调整归属的控制逻辑再把部分分片交给新实例，并为新的归属生成更高版本的隔离令牌。旧实例停止处理这些分片，共享队列或它前面的写入接口拒绝携带旧令牌的请求；路由更新后，新请求才到达新所有者。至此，新实例依次通过资源与实例就绪、流量接入和状态接管三道门，获得了可以实际处理的队列分片。

让新增战斗实例承接新对局 战斗服务不必为普通扩容迁移正在进行的对局。创建房间时，匹配服务从已经就绪且未进入排空状态的战斗实例中选择目标，把房间与实例的对应关系写入共享的房间归属记录。扩容后，新实例进入候选集合，房间分配逻辑根据各实例尚可承载的对局数，把更多新对局送往负载较低的新增实例；已经开始的对局继续由原实例推进，直到完成结算。

这种方案把一场对局作为不可拆分的状态单元，优点是扩容不必暂停现有对局，也不必在高峰期额外传输大量状态。代价是容量生效速度受新对局产生速度影响：如果少数对局持续时间很长，旧实例可能长期保持较高负载。只有当对局持续时间超过可接受的排空期限，或者必须在限定时间内维护某台机器时，系统才需要考虑主动迁移。主动迁移要选择一致状态点生成快照、追平增量操作、切换房间所有权和路由，并恢复连接、定时器与未完成消息；这些开销可能进一步加重高峰压力，因此它是带条件的进阶方案，而不是常规扩容的默认步骤。

分别处理安全缩容与故障替换 计划缩容时，平台开始把实例移出新流量路径，同时发出终止通知并开始计算终止宽限期。应用立即把目标实例标记为排空状态；即使路由更新尚未传播完成，也不再接受新的状态单元。匹配控制逻辑停止向它分配新分片，房间分配逻辑也不再创建新对局；已有匹配分片完成所有权交接，正在进行的对局则等待自然结束。状态单元数量降为零后，应用主动退出；若直到宽限期结束仍未排空，平台会强制终止实例。因此，设计阶段必须根据状态单元的持续时间设置宽限期，并明确无法在期限内排空时，是迁移剩余状态还是中断超时任务。安全退出点由业务状态决定，宽限期则给这一过程设置了平台能够接受的时间上限。

故障替换没有这样的排空阶段。平台通过健康探测、节点状态或心跳发现实例失效并创建替代实例，但平台并不知道一场对局进行到了哪一步。状态控制逻辑必须先使旧实例的租约失效，或提升隔离令牌版本，防止暂时失联的旧实例恢复后继续写入。如果业务只承诺连接可以重建，不承诺对局恢复，那么故障会中断受影响的对局；如果业务要求继续对局，就必须事先保存快照或操作日志，由新实例恢复状态、取得更高版本的所有权，再更新房间路由并允许玩家重连。平台重新创建实例只能恢复一个可运行的执行环境，能否恢复业务取决于应用事先保留了什么状态和恢复协议。

用运行结果验证状态架构 改造完成后，不能只检查副本数是否达到期望值。扩容验证还要确认新实例是否通过就绪检查、是否收到请求、获得了多少队列分片和新对局，以及队列长度和等待时间是否随有效容量增加而下降。缩容验证要观察待回收实例上的分片数和进行中对局数是否持续降为零，并记录排空所需时间。结算链路还要检查重试后是否出现重复资产变更。这些是应用侧的业务验证指标，关注的是状态架构是否真正配合了扩缩动作，与第 5.3 节讨论的平台侧伸缩触发指标（如 CPU 利用率、请求速率）处在不同层面。只有业务状态和运行指标同时符合预期，才能说明新增副本转化成了有效容量，退出实例也没有破坏正确性。

这个案例得到的结论可以迁移到其他在线系统：**弹性伸缩不要求所有组件都无状态，关键是让状态单元的粒度、所有权和生命周期与实例的创建、接入、退出和替换过程相互配合。**

6.3.4 应用与平台的职责划分

上述案例说明，应用与平台的职责不能只按组件静态划分，而要在实例生命周期中相互衔接。

扩容阶段，平台根据容量决策创建实例、分配资源并提供服务发现机制；应用完成配置加载、依赖连接和缓存预热，通过就绪条件说明自己已经可以处理请求。平台把就绪实例加入通用流量入口后，应用或专用状态控制器还要分配队列分片、房间等状态单元，并更新相应的业务路由。平台能够决定“需要多少个实例”和“实例放在哪里”，应用则必须说明“新实例可以处理什么状态”。

缩容阶段，平台选择待回收实例，启动流量摘除并发出终止通知；应用随即进入排空状态，在路由更新传播期间也不再接收新的状态单元，转移可迁移的所有权，并等待其余存量工作到达安全退出点。平台负责提供生命周期通知、终止宽限期和最终回收资源的机制，应用负责用业务状态判断何时可以主动退出，并保证无法在期限内排空时有明确的处理策略。固定的等待时间只能提供上限，不能代替这个判断。

故障替换阶段，平台负责检测实例失效、隔离故障位置并创建替代实例；应用或专用控制器负责废止旧所有权、从权威来源恢复状态、执行幂等重试并更新业务路由。数据库、存储系统或 Operator 可以自动执行复制和重平衡，但它们仍要遵守应用定义的一致性要求、状态粒度和切换条件。

因此，平台提供的是实例生命周期和通用资源治理能力，应用提供的是状态语义和安全转换条件。任何一侧缺失，副本变化都可能停留在“进程数量改变”这一层，无法变成安全、有效的业务伸缩。**弹性伸缩必须进入应用架构设计本身：从设计之初就把实例的增加、退出和失败视为正常变化，并为这些变化准备状态接管、排空与恢复流程。**

接下来，在应用已经满足上述条件的前提下，平台内部如何把负载变化转换为调度决策，是下一节的主题。

6.4 弹性调度的内部机制：从反馈控制到按需执行

上一节说明应用需要满足哪些条件才能被安全伸缩。本节进入平台内部，分析控制器如何把负载变化转换为调度决策：观测哪些指标、如何抑制噪声、怎样分层执行、以及 Serverless 如何把同一套容量管理继续下沉到请求级执行环境。

6.4.1 调度对象与分层

弹性调度需要调节的对象不止一种。HPA 调节的是同类服务实例的数量；VPA 调节的是单个实例的资源规格；Cluster Autoscaler 调节的是集群节点数量；有状态系统的分片控制器调节的是分片或副本的分布；Serverless 调节的则是函数执行环境的创建与回收。不同对象对应不同层级的资源，也对应不同类型的延迟和约束。

这些调节器通常按分层结构组织。上层控制器（如 HPA）根据业务指标决定“需要多少容量”；中层调度器（如 Kubernetes Scheduler）根据资源约束决定“把新实例放到哪台机器上”；下层基础设施控制器（如 Cluster Autoscaler）根据节点资源余量决定“是否需要申请新机器”。上层决策向下层传导，下层的执行结果又通过指标反馈回上层，形成完整的控制链路。

分层的意义在于隔离决策频率和时延。HPA 的同步周期通常是十几秒，因为它只需要读取指标并计算一个数字；调度器在每次创建 Pod 时运行一次，决策时间通常是毫秒到秒级；Node 级扩容可能需要数分钟。如果把所有决策压在同一层，慢路径会拖快路径，快路径的频繁抖动又会干扰慢路径的稳定性。

6.4.2 反馈闭环的控制要素

无论调节对象是副本数、节点数还是函数实例，反馈闭环都包含四个共同要素：指标、目标、动作和校验。

指标是闭环的输入。它必须能够反映负载变化，但又不能太敏感。常见的做法是用滑动窗口或指数移动平均（EMA）对原始指标做平滑，避免单次尖峰触发不必要的扩缩。指标的选择也决定了闭环保护的对象：CPU 利用率保护的是计算资源余量，请求队列深度和等待时间保护的

才是用户体验。稳态下，Little 定律 $L = \lambda W$ 把平均积压量 L 、请求到达率 λ 和平均停留时间 W 联系起来；当到达率大致稳定而积压量持续上升时，等待时间也在变长，队列指标因此可以比 CPU 更早暴露容量不足。

目标是闭环希望维持的水位。例如“每个 Pod 的平均 CPU 维持在 60%”或“请求等待时间不超过 200 毫秒”。目标值不是越低越好：设得太低，资源利用率不足；设得太高，缓冲余量太小，偶发波动就容易突破容量上限。

动作是控制器根据偏差计算出的调节量。动作必须足够克制：设置最小规模防止低峰缩得过多，设置最大规模控制预算，限制单次增减幅度并设置冷却时间，避免在新实例尚未接管流量前连续过冲。

校验是闭环收敛的保障。控制器执行动作后，必须等待一定时间让动作生效，再读取新指标判断偏差是否缩小。若扩容后关键指标仍无改善，说明瓶颈可能不在容量扩展层。此时不应直接继续盲目扩容，而应把异常传递给上层排查链路。HPA 的职责是把容量决策标准化，而不是替代完整的根因归因；工程上通常配合限流、降级、队列治理等机制，避免误把非容量问题当成“再加更多副本”来处理。

这四个要素的共同挑战是**延迟**：指标恶化到被察觉需要时间，计算和决策需要时间，动作生效（镜像拉取、进程启动、预热）也需要时间。延迟越大，闭环越滞后；延迟不确定，控制器就越难设置合适的冷却时间。工程上通常通过容忍区间过滤噪声、通过稳定窗口抑制振荡、通过速率限制防止过冲，本质都是在“延迟不可避免”的前提下，让闭环尽可能稳健。

6.4.3 HPA 在做什么：闭环的工程落地

HPA (Horizontal Pod Autoscaler) 是 Kubernetes 中实现弹性伸缩闭环的标准控制器。它的职责可以归结为一个周期性动作：**读取当前指标，计算期望副本数，将结果写回工作负载的副本数配置**。写入完成后，Kubernetes 负责创建或回收对应的 Pod。

以一个大厅接口服务 `lobby-api` 为例（负责玩家上线后拉取公告、房间列表、好友状态等），这类服务请求短、状态少，适合水平伸缩。为它设定一个目标：每个 Pod 的平均 CPU 利用率维持在 60%，既不因过低而浪费资源，也不因过高而产生排队。这里的利用率是相对于容器 **CPU request** 而言的，而不是相对于宿主机总 CPU 或某种绝对占用；因此若容器没有配置 CPU request，这个百分比无法定义，HPA 也就无法依据该指标伸缩。副本数范围设为 2 到 10：下限保证低峰可用性，上限对应预算与集群资源约束。

图 84 把 HPA 控制器的职责边界具体化：它真正负责的只是读取指标、计算期望副本数、写回 Deployment 这一小段，写回之后的 Pod 调度、创建与回收都交给了 Kubernetes 控制平面。正是这条边界，决定了扩容不及预期时该往指标滞后、HPA 计算还是 Pod 调度上去查。

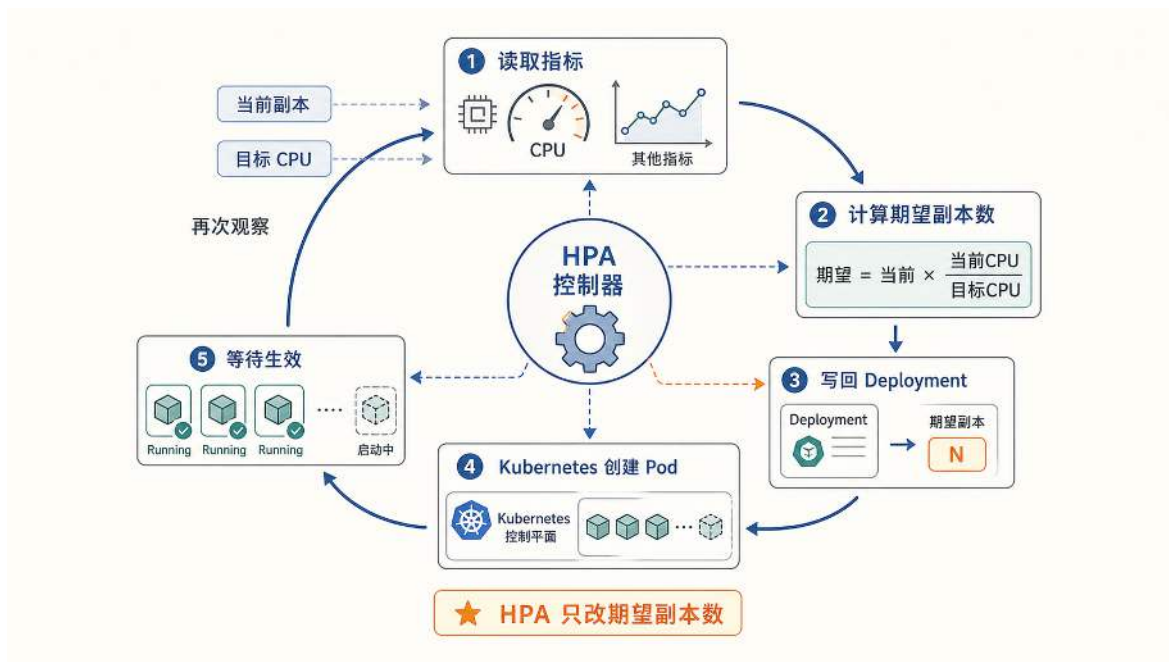


图 84: HPA 控制器周期性读取指标并计算期望副本数，随后把结果写回 Deployment，由 Kubernetes 完成 Pod 创建与回收。

因此，HPA 的输出是一个新的期望副本数；真正多出来的 Pod 还要等待 Kubernetes 调度、创建并通过就绪检查。

以晚间活动开始为例：玩家集中涌入后，最先变化的是**指标**而非副本数。`lobby-api` 的请求速率上升，单个 Pod 的 CPU 利用率从 30% 升至 90%。HPA 不把“达到 90%”作为一个单独的触发条件，而是在每个同步周期（默认 15 秒）计算当前指标与目标指标的比值，再据此得到期望副本数。控制器为这个指标比值设置了一个容忍区间（默认 10%）：如果比值仍落在目标值上下 10% 的范围内，控制器不会动作。扩容与缩容在响应速度上存在不对称：扩容默认没有稳定窗口，只要一次同步周期中的指标比值超出容忍区间，且计算出的期望副本数大于当前值，扩容就可以执行；而缩容则有默认 5 分钟的稳定窗口，只有较小的期望副本数在窗口内保持稳定，才会真正执行。这种不对称设计的逻辑是：扩容需要尽快响应以缓解排队，而缩容需要更多确认以避免震荡。

当一次同步得到的指标比值超出容忍区间后，HPA 进入计算期望容量阶段：将当前负载与目标负载做比例换算，得出期望副本数。其核心公式为

$$n_{\text{des}} \approx \left\lceil n \cdot \frac{\text{当前指标}}{\text{目标指标}} \right\rceil.$$

例如当前有 $n = 2$ 个 Pod，平均 CPU 为 90%，目标为 60%，则期望副本数为

$$n_{\text{des}} = \left\lceil 2 \cdot \frac{0.90}{0.60} \right\rceil = 3.$$

需要区分决策结果和执行结果：**HPA 的决策结果只是一个期望副本数**。它将 `lobby-api` 的副本数从 2 改写为 3，但不负责启动容器。随后 Kubernetes 才真正创建新 Pod、调度到节点、拉起进程并等待就绪检查通过。这就是前文提到的执行时延：从 HPA 写入期望副本数到新 Pod 实际承接流量，中间要经历镜像拉取、进程启动和预热等步骤，耗时可能从数秒到数十秒不等。

实际运行中，HPA 往往需要多轮迭代才能收敛。新 Pod 刚启动时，旧 Pod 仍然承受着全部负载，CPU 利用率可能仍然偏高。下一轮 HPA 再次读取指标，如果 3 个 Pod 时平均 CPU 仍在 80%，就会将期望副本数更新为 4；再过一轮，如果指标回落到 60% 附近，扩容停止。这种逐步逼近而非一次到位的行为，正是反馈闭环的特征：每一轮动作的效果成为下一轮决策的输入。

当活动结束后、流量回落时，指标反向变化，HPA 开始计算更小的期望副本数。对 `lobby-api` 这类短请求无状态服务，减少副本的风险较低；但对长连接或强状态服务（例如战斗服、带会话的网关），直接回收 Pod 意味着正在进行的连接被强制中断。因此缩容策略通常比扩容更保守：HPA 默认使用 5 分钟的缩容稳定窗口，在窗口内保留近期计算出的期望副本数，并选择其中较保守的结果，避免短时低谷触发快速缩容；如果需要进一步限制单次缩减幅度，还可以通过 `scaleDown.policies` 显式配置缩容速率。同时，服务自身需要提供安全退出能力，例如收到终止信号后停止接入新会话，并为存量连接预留完成时间。两者配合，HPA 触发的副本减少才不会转化为用户可感知的中断。

HPA 并未改变弹性伸缩的本质，而是将反馈闭环封装为一个标准控制器，并把 Pod 的创建、回收和流量接入等执行细节交给 Kubernetes。第 6.4.2 节提出的指标、目标、动作和校验，在 HPA 中分别落实为指标采集、期望副本数、规模写回和下一轮观测；第 6.3 节提出的应用条件则决定缩容能否安全完成。

下一小节用一段 Go 伪代码实现一个最小化的 HPA 控制器，通过 Kubernetes API 执行一轮完整的 Reconciliation Loop：读取真实状态、与目标状态比较、将差异写回系统对象。

6.4.4 简化 HPA 控制器：用代码表达反馈闭环

上一节从外部视角描述了 HPA 的行为：周期性读取指标、计算期望副本数、写回工作负载对象。本节用一段 Go 伪代码将这个控制器的内部逻辑展开，使闭环的每一步对应到具体的 API 调用。

这个迷你控制器只针对一种场景：对某个 `Deployment` 自动调整副本数，使平均 CPU 利用率接近目标值（例如 60%）。这类目标适用于请求短、状态轻、多副本之间可自然分流的服务，如大厅接口或匹配入口。对于战斗服这种强状态组件，增加副本并不能分摊已有会话的负载，比例公式的前提不成立。这是第 6.3 节讨论的边界：弹性伸缩只对可分摊的负载有效。

控制器每一轮循环由三个步骤按因果顺序构成：先读取当前状态，才能计算差距；计算出期望副本数，才能写回系统。

读取状态。控制器需要两个输入：当前副本数（来自 `Deployment` 的 `/scale` 子资源）和当前负载水平（来自 `metrics-server` 提供的 `Metrics API`）。本例使用所有 Pod 的平均 CPU 利用率作为负载信号：平均 CPU 高于目标说明单个 Pod 处理能力不足，需要扩容；低于目标说明存在资源冗余，可以缩容。

计算期望副本数。沿用上一节的比例公式：

$$\text{des} = \left\lceil \text{cur} \cdot \frac{\text{avgCPU}}{\text{targetCPU}} \right\rceil.$$

计算结果需要经过边界约束：下限 `minReplicas` 保证低峰可用性，上限 `maxReplicas` 防止资源耗尽。

写回副本数。控制器将期望副本数写入 `Deployment` 的副本数字段，由 Kubernetes 负责实际的 Pod 创建或回收。写入后效果不会立即体现：新 Pod 需要经历调度、镜像拉取、进程启动和就绪检查。正因为存在这段执行时延，控制器在写入后必须等待一个周期再进入下一轮，避免在动作尚未生效时重复计算导致过冲。

以下伪代码保留了闭环的骨架，省略了稳态窗口、缩容排空和异常处理等工程细节：

迷你 HPA 控制器伪代码

```
1  for {
2      avgCPU := MetricsAvgCPU(ns, deploy)           // 读平均 CPU
3      cur    := GetDeploymentReplicas(ns, deploy)    // 读副本数
4
5      des := ceil(float(cur) * float(avgCPU) /
6                  float(targetCPU))                 // 比例计算
7      des = clamp(des, minReplicas,
8                  maxReplicas)                       // 边界约束
9
10     if des != cur {
11         PatchDeploymentScale(ns, deploy, des) // PATCH .../deployments/{name}
12         }/scale
13     }
14     sleep(15 * second) // 按控制周期采样更新指标与状态
15 }
```

这段伪代码展示了 HPA 的核心结构：一个持续运行的控制环。工业级实现会在此骨架上补充容忍度与稳定窗口（避免对短暂波动做出反应）和异常处理（API 调用失败时的退避策略）。需要区分的是，HPA 控制器本身只负责计算并写入期望副本数；当 Kubernetes 因此回收 Pod 时，连接排空和存量会话完成属于 Pod 终止阶段的职责，由 `preStop` 钩子、优雅终止窗口和应用自身的退出逻辑来保障。两者配合，缩容动作才不会引发用户可感知的中断。

6.4.5 HPA 的边界：副本控制器不能解决所有问题

HPA 将副本数的增减自动化，但它的能力止步于此。一个常见的误解是：只要在 Kubernetes 上配置了 HPA，高并发和资源成本问题就自动解决。以下几个方向上的局限说明，自动化副本数只是弹性系统的一环，而非全部。

第一个局限来自**最小副本约束**。从成本角度看，凌晨无人时副本收缩是自然目标，但“直接收缩到 0”并非对所有服务都安全。若服务入口没有请求缓冲、预热机制或快速就绪路径，副本数为 0 会让首批请求等待更长或被拒绝。即使平台支持从零启动，新实例也要经历镜像拉取、进程启动、配置加载和数据库连接初始化，这些动作通常需要可观时间。对长连接或状态依赖强的组件，频繁从 0 拉起会放大尾延迟。工程实践通常会给这类服务保留非零底仓；当平台明确支持事件驱动、请求排队、快速预热和冷启动保护时，HPAScaleToZero 或类似策略才有机会安全落地。

第二个局限来自**指标滞后**。HPA 最常用的触发信号是 CPU 或内存利用率，它们获取方便但属于粗粒度的资源指标。在游戏匹配系统中，当大量玩家同时点击匹配时，最先恶化的是排队长度和等待时间；CPU 利用率要等到队列持续积压后才明显升高。**CPU 的上升常常滞后于玩家体验的恶化**。如果等 CPU 达到阈值才触发扩容，加上新 Pod 的启动时间，玩家已经经历了一段明显的等待。更极端的情况是，服务因数据库锁冲突而阻塞，此时 CPU 利用率很低，但服务实际上已不可用。解决方向是引入排队长度、业务并发数、响应时间等自定义指标，但这要求业务侧提供对应的埋点和监控接口。

第三个局限来自**容量参数与退出流程**。即使伸缩动作自动化了，开发者仍然需要回答一系列容量问题：每个 Pod 申请多少 CPU 和内存（CPU 配额过低会触发 throttling 导致延迟抖动，内存配额过低会触发 OOM 终止容器，过高则浪费配额）、目标利用率设为多少、最大副本数是否会超出下游连接池容量。对于包含有状态组件的游戏系统，问题更进一步：匹配入口这类无状态服务适合由 HPA 管理，但战斗服这类承载真实对局的强状态单元不能随意终止进程，需要专门的优雅退出、排空和状态迁移逻辑。HPA 并未消除基础设施的设计负担，而是将其从手工扩缩操作转化为资源配额、指标选择、伸缩边界和退出流程的设计问题。

第四个局限来自**不可水平扩展的瓶颈**。如果入口服务可以增加副本，而所有请求仍要经过同一个写锁、单分片数据库或串行协调器，那么新增副本只会更快地把压力送到这个单点。Amdahl 定律说明，系统中不能并行扩展的比例为 s 时，即使可扩展部分增加无限多副本，整体加速上限也只有 $1/s$ 。因此，弹性控制器只能调节已经具备并行扩展条件的部分，不能消除架构中的串行瓶颈。

这些局限的共同根源在于：**HPA 仍然以副本为中心**。开发者必须关心运行多少实例、每个实例分配多少资源、低峰至少保留几个、高峰最多允许多少。如果平台能够在请求到来时自动准备运行环境，在请求结束后回收环境，并将路由、资源分配和冷启动优化收敛到基础设施层，开发者就不再直接管理副本数。这正是 Serverless 要推进的方向：在 HPA 的基础上进一步下沉副本管理，同时需要评估它在游戏这类延迟敏感业务中的适用条件。

6.4.6 分层控制：从副本决策到节点放置

HPA 决定需要多少 Pod，但新 Pod 能否实际运行还取决于调度器和节点容量。完整的弹性控制链路可以概括为三层。

第一层是**副本决策**，由 HPA 完成：根据指标偏差计算期望副本数，写回 Deployment。这一层只关心“需要多少实例”，不关心实例放在哪里。

第二层是**放置决策**，由调度器完成。当 Deployment 的期望副本数增加后，Kubernetes 调度器为新 Pod 选择一台 Node。调度器的决策分两步：先按硬约束过滤掉不满足条件的 Node（资源不足、标签不匹配、存在亲和性冲突），再对候选 Node 按软目标打分（资源均衡、分散同类副本、数据本地性），选择得分最高的一台完成绑定。如果所有 Node 都无法通过过滤，Pod 会进入 Pending 状态，等待约束变化或由节点扩容器继续评估。

第三层是**基础设施决策**，由 Cluster Autoscaler 完成。它根据 Pending Pod 的资源与调度约束，判断预先配置的某个节点组新增机器后能否使 Pod 可调度；只有判断成立时，才向云平台申请新的虚拟机实例。资源不足通常可以通过增加合适规格的 Node 缓解，但不存在匹配标签、污点无法容忍或存储拓扑无法满足等约束，不会因为简单增加任意节点而消失。Node 级扩容还要经历虚拟机创建、系统初始化和节点注册，通常比 Pod 级扩容慢得多。

这三层控制器各自独立运行，通过对象状态间接协作：HPA 修改副本数字段，调度器读取 Pod 规格并写入 Node 绑定，Cluster Autoscaler 读取 Pending Pod 并修改节点组规模。任何一层的瓶颈都会传导到上层：调度器找不到合适 Node，HPA 的计算结果就无法落地；Node 扩容慢，调度器就只能让 Pod 持续 Pending。

图 85 把三层控制器的交接关系放在同一条链路中：上层先提出新的容量需求，中层尝试利用现有节点完成放置，下层只在新增节点能够解除调度约束时扩充基础设施。

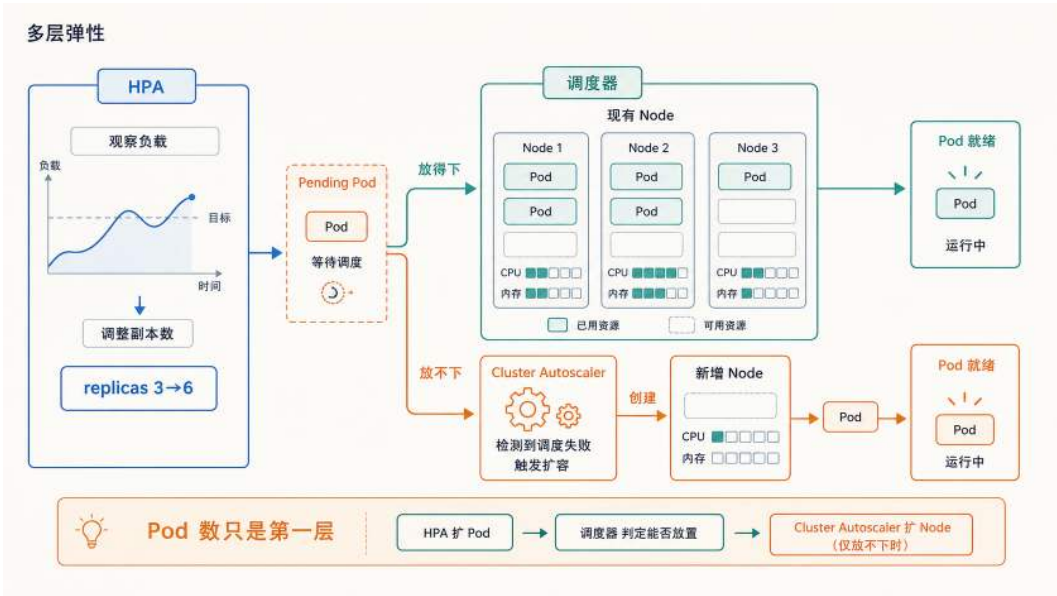


图 85: 分层弹性控制链路：HPA 产生新的 Pod 容量需求，调度器先在现有 Node 中寻找可行位置，节点扩容器再评估新增 Node 能否解除资源约束。

这张图也说明，HPA 已经写入期望副本数并不等于容量已经生效；只有放置、节点供给和就绪检查依次完成，新副本才真正进入服务路径。

6.4.7 多目标调度

第五章已经通过一次 battle 补位展示了“先过滤、再打分”的两步决策。本节进一步把打分环节形式化，以说明放置决策为何本质上是一个多目标优化问题：调度器不能只看“哪台机器

还有空闲 CPU”，还要同时考虑延迟、成本、故障隔离和数据位置等多个目标。

硬约束（必须满足）包括：Node 剩余资源是否大于 Pod 声明的 requests；Node 是否被标记为不可调度；Pod 是否声明了特殊的硬件需求（如 GPU）。调度器依次执行过滤规则，得到可行节点集合 $\mathcal{N}_{\text{feasible}}$ ；如果集合为空，本轮调度失败，Pod 返回队列等待后续重试。

软目标（综合权衡）包括：资源利用均衡（避免某些 Node 满载而其他 Node 空闲）、副本分散（同一服务的多个副本尽量放在不同 Node 或不同机架，降低单点故障影响范围）、数据本地性（把计算任务放到已缓存所需数据的 Node 附近，减少网络传输）、预热状态（优先选择已经缓存了对应镜像或预热了运行时环境的 Node，缩短启动时间）。

进入可行集合后，各项软目标先被转换到可比较的评分范围，再按权重合并。一个简化的评分函数可以写成

$$S(n) = w_r R(n) + w_f F(n) + w_d D(n) + w_p P(n), \quad n \in \mathcal{N}_{\text{feasible}},$$

其中 R 表示资源匹配程度， F 表示故障域分散程度， D 表示数据本地性， P 表示镜像或运行时预热程度， w_r, w_f, w_d, w_p 是各目标的权重。调度器选择得分最高的节点；若业务更重视故障隔离，就提高 w_f ，若任务需要读取大量本地数据，就提高 w_d 。

这类打分是在当前集群状态下对一个待调度 Pod 作出的在线决策，并不保证未来所有 Pod 的整体放置达到全局最优。不同目标的权重也因集群和业务而异。调度器的工程价值在于先用硬约束保证结果可执行，再用可配置的评分规则表达软目标，并把每一次选择限制在可解释、可复查的决策范围内。

6.4.8 Serverless 引擎：把副本管理继续下沉

HPA 将弹性粒度定在服务副本数：开发者仍然要设置最小副本数、最大副本数和指标阈值。Serverless 则把容量管理细化为按请求/事件驱动的执行环境分配：请求到来时优先复用空闲环境；没有可用实例时再按需创建；请求处理完后实例可回收或暂时保留。开发者交付的是一段业务逻辑代码（Function），平台负责为它分配执行环境并管理生命周期。这里的无服务器并非物理上不存在服务器，而是指开发者不再直接管理服务器和副本数。

粒度变化带来的第一个机制差异是**触发方式**。HPA 依赖周期性轮询（Polling）：控制器每隔固定时间读取指标，再计算期望副本数。Serverless 的入口更接近事件驱动（Event-Driven）：HTTP 请求、消息队列消息、对象存储更新等事件到达时，事件网关（Event Gateway）直接感知并分配给可用的函数实例。以游戏场景为例，玩家发起匹配请求、上传截图到对象存储、或消息队列中出现一封待发送的全服补偿邮件，都可以作为触发事件。

当事件瞬间爆发时，网关不需要等待 CPU 平均值升高再触发扩容。它直接判断当前是否有空闲函数实例：有则分配，无则在入口处短暂缓冲请求，同时通知调度器准备新的执行环境。环境就绪后，网关将请求转交执行。请求数量上升时，平台并行准备更多实例；请求处理完毕后，实例标记为空闲，空闲超时后回收。与 HPA 相比，容量分配从服务副本层向请求/事件层前移，响应路径也更靠近入口事件。

但事件驱动带来一个直接矛盾：容器从零启动需要时间（镜像拉取、文件系统准备、运行时初始化等），而 Serverless 又要求在请求到来时才准备执行环境。如果每次请求都从头启动，用户仍然会经历明显等待。因此 Serverless 引擎必须解决一个具体的底层问题：**如何将启动一个隔离执行环境的时间从秒级压缩到毫秒级。**

第 6.1 节拆解过容器的启动路径。冷启动慢的原因在于多个步骤串联在同一条请求路径上：镜像获取与解压、文件系统准备、网络配置、隔离环境创建、语言运行时加载、应用依赖初始化。将这些步骤按层次归类，冷启动时间可以分解为三部分：

$$T_{\text{cold}} \approx T_{\text{sandbox}} + T_{\text{runtime}} + T_{\text{app}}.$$

其中 T_{sandbox} 是环境准备与隔离创建的成本（镜像获取、文件系统、网络、沙箱权限边界等）； T_{runtime} 是语言运行时与框架初始化的成本（解释器/虚拟机启动、依赖装载、JIT 预编译等）； T_{app} 是应用层自带的启动工作（读配置、建连接池、拉取热数据、预热缓存等）。三项之和是**环境初始化成本**，它是冷启动优化的主要对象；但它并不等于用户感知到的完整首次响应时间——后者还可能叠加请求排队、准入判断、路由分发以及首个请求自身的执行时间。要将冷启动压到毫秒级，需要逐段压缩这里的初始化常数项，或将其中的大头从请求关键路径移到后台路径。

针对 T_{sandbox} ，工程上的代表方案是**极简微虚拟机 (MicroVM)**。容器依赖 Namespaces 与 Cgroups 实现视图隔离和资源限制，借助 OverlayFS 实现镜像分层与文件系统共享，进程共享宿主机内核，启动路径短；但共享内核意味着隔离边界依赖宿主机内核的正确性，多租户场景下故障影响范围较大。传统虚拟机每个实例有独立内核与地址空间，隔离边界更强；代价是完整的设备模型和引导流程将 T_{sandbox} 拉回秒级。MicroVM 在两者之间取舍：保留虚拟机的隔离边界（独立内核、硬件虚拟化），同时将虚拟机中不服务于云端工作负载的部分压到最少。以 Firecracker 为例，它使用轻量虚拟机管理器（VMM）与精简设备模型，只模拟网络和块存储等 Serverless 真正需要的虚拟硬件，不模拟键盘、声卡、显示器等无关外设。这样，启动一个强隔离执行环境可以被压到百毫秒量级。但 MicroVM 只解决沙箱层的启动速度， T_{runtime} 与 T_{app} 仍然需要其他手段。

针对 T_{runtime} 与 T_{app} ，代表方案是**内存快照与恢复 (Snapshot & Restore)**。即使沙箱启动压到百毫秒，语言运行时和应用初始化仍可能耗时几百毫秒到几秒（解释器启动、依赖加载、JIT 预热、连接池建立等）。快照的思路是：先在后台将执行环境启动到可接收请求的状态，暂停并保存当时的内存映像。下次请求到来时，系统将快照恢复到内存中，函数直接进入可用状态，跳过完整的初始化路径。原本昂贵的初始化被摊平到后台预热阶段，冷启动路径上只剩恢复与少量补齐的成本。

制作快照时，平台需要先把一个执行环境初始化到可接收请求的状态，再将其内存和运行状态保存下来；快照写入存储后，生成它的实例不必继续常驻。请求到来时，平台仍要支付快照读取、内存恢复和状态补齐的成本；数据库连接、网络会话、当前时间戳和随机数种子等外部或时效性状态也需要重新建立或校验。如果平台还要进一步压低恢复时间，可以另行维护预恢复的暖池，但这是附加的容量预留策略，并不是快照恢复本身的必要条件。

另一个方向是 **WebAssembly (Wasm)**，它将隔离边界下沉到指令集与运行时层面。Wasm 模块运行在受限的线性内存中，可用的系统交互由宿主能力接口（如 WASI）决定。由于不需要完整内核、用户态环境和设备模型，实例化一个 Wasm 模块的路径更短，很多场景下可以将启动开销压到毫秒级。Wasm 适合承接短小、轻量、对系统调用依赖少的计算单元，例如边缘侧逻辑、数据清洗或规则判定。其边界同样清晰：如果函数强依赖完整操作系统语义、需要大量原生扩展或长期维护复杂网络连接，迁移到 Wasm 需要重写运行时边界与依赖交付方式。工程上更常见的做法是将 Wasm 作为更轻的沙箱层：适合的部分用它换取更快启动与更小常驻开销，不适合的部分仍落在容器或 MicroVM 上。

这三类手段的共同目标，是降低请求关键路径上的初始化延迟，但采用的思路并不相同，可以归纳为三条：**减少必须完成的工作**——MicroVM 精简设备模型、Wasm 省去完整内核与用户态环境，让需要执行的初始化本身变少；**复用已经完成的工作**——快照恢复、Zygote fork、分层缓存都是把一次昂贵的初始化结果重复利用；**预留已经就绪的容量**——暖池、预创建实例和预恢复环境提前把工作做在后台，请求到来时直接取用。只有第三条才是严格意义上的“用常驻成本换首次延迟”：它必须持续占用内存和计算来维持就绪实例。前两条不一定伴随常驻暖实例，例如快照可以保存在存储中按需恢复，是否再叠加一个预恢复池是另一层独立的资源与延迟取舍。以游戏中低频的签到、运营活动入口为例：放到 Serverless 上可以显著降低低峰成本，但第一批请求更容易遇到短暂等待。引入预热池、快照恢复或 Wasm 执行层后，这个等待可以被压到用户几乎感知不到的范围；代价则依手段而异，可能是更多常驻资源，也可能是恢复时的补齐成本或运行时兼容性约束。

图 86 将完整冷启动与三类优化并排放在同一条时间尺度上：MicroVM 和 Wasm 通过精简执行环境减少必须完成的工作，Snapshot 通过恢复已保存的状态复用初始化结果，暖池则直接预留已经就绪的容量。这样才能区分某项技术究竟缩短了工作、复用了工作，还是提前支付了工作。

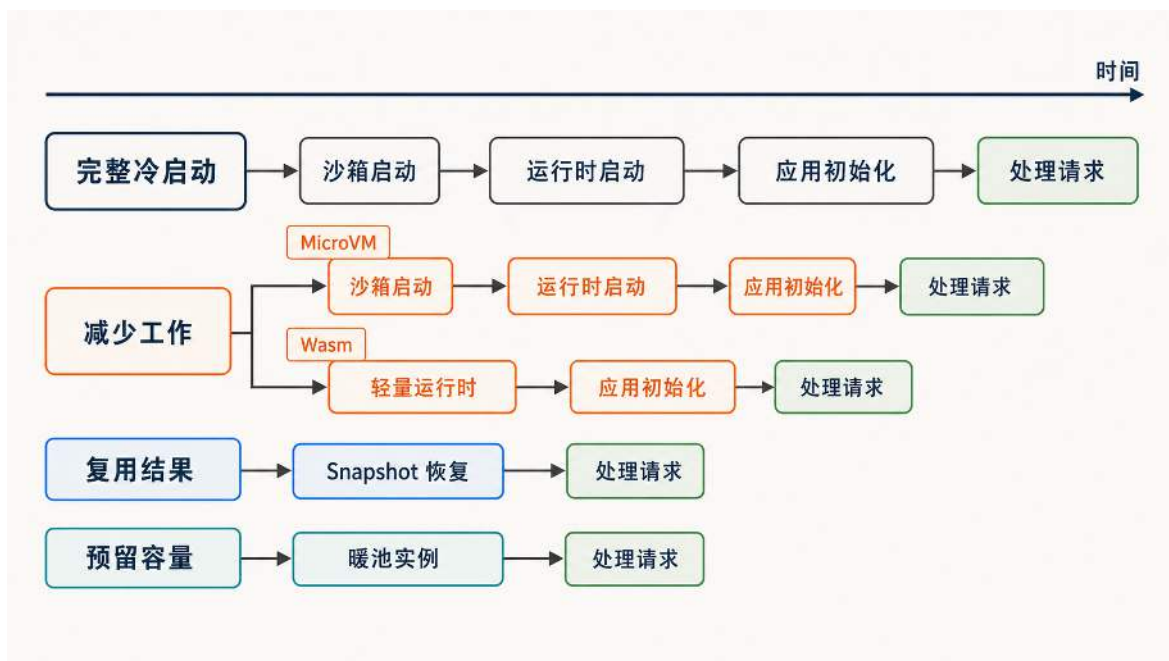


图 86: Serverless 冷启动的三类优化机制：MicroVM 和 Wasm 减少初始化工作，Snapshot 复用已经完成的初始化，暖池预留可直接接收请求的容量。

Serverless 的目标并非让启动成本凭空消失，而是通过更轻的沙箱、更快的恢复和更充分的后台预热，将用户能感知到的那部分成本压低。

然而，即使冷启动问题得到缓解，以函数为交付单位的典型 Serverless 形态（FaaS，Function as a Service）仍然不适合所有游戏服务端。FaaS 通常要求函数按无状态方式设计，并且不承诺某个执行环境会在整局对战期间持续存在、保持固定连接和会话归属；即使环境可能在多次调用之间被复用，应用也不能把这种复用当作生命周期保证。

从网络模型看，无论是 MOBA 还是 FPS，客户端与战斗服之间需要维持长达数十分钟的稳定 TCP 或 UDP 连接。典型 FaaS 不提供某个执行环境持续承载一局对战的生命周期保证，因而无法直接承载长驻对局的连接模型。从执行模型看，战斗服通常有一个持续运行的循环（Game Loop），以每秒 30 到 60 次的频率推进玩家位置、弹道计算和物理碰撞。典型 FaaS 以事件调用为基本执行单位，调用结束后环境可能被冻结、复用或回收，这种生命周期契约与持续运行的主循环不兼容。如果将每一帧同步拆成独立的函数调用，产生的请求量、网关压力和外部状态访问反而破坏实时性。从状态模型看，典型 FaaS 通常要求计算单元无状态，所有持久数据落到外部存储。对于大厅查询、签到奖励或回合制游戏的单次结算，这是可接受的。但在状态同步或帧同步的动作游戏中，每次技能判定都要读写外部 Redis，单次访问虽然只有毫秒级延迟，但在每秒数十帧的循环中累积起来会严重影响体验。

上述生命周期约束解释了第五章表 8 的场景判断：典型 FaaS 更适合大厅查询、商城请求、签到奖励、运营活动入口和异步任务处理等短请求或低频任务；核心战斗服这类强状态、长连接、高频循环的组件，仍然更适合运行在常驻进程或专门的房间服架构中。Serverless 的价值在于将适合事件驱动的部分从常驻资源中拆出来，而非替代所有服务端形态。

从 HPA 到 Serverless，底层基础设施变得更复杂，但暴露给开发者的接口在变简单。复杂性没有消失，而是被平台收敛到调度器、网关、镜像系统、运行时、快照机制和预热池中。两者共享同一套基础问题：隔离环境如何创建、网络入口如何转发、请求如何排队、实例如何预热、状态如何恢复、失败如何被观察。差异仅在于这些问题由谁负责回答：HPA 模式下由开发者配置参数，Serverless 模式下由平台自动决策。

6.4.9 降低冷启动延迟：容器快速启动的两条研究路线

前述 MicroVM、内存快照和 Wasm 是工业界压缩冷启动的通用方案，它们分别针对 T_{sandbox} 、 T_{runtime} 和 T_{app} 下手。围绕同一个 T_{cold} 拆解，研究者还从两个更细的角度做了优化：一是追问容器启动本身为什么慢，二是追问预热成本能不能在更多函数之间摊薄。这两条路线对下一节要讨论的 Agent sandbox 尤其重要：那里需要在很短时间内批量创建大量隔离环境，启动路径上的每一毫秒都会被实例数量放大。

第一条路线是把容器启动路径本身做精简。SOCK³³ 先测量了通用容器的启动开销，发现瓶颈有两处：一是 Linux 的存储与网络命名空间等原语在创建和销毁时本身不可扩展，二是像 Python 这样的运行时在导入大量库时会额外增加约 100 毫秒。针对第一处，它构造**精简容器 (lean container)**：为 Serverless 函数量身裁剪文件系统、缓存 cgroup、跳过创建代价高的命名空间，只保留隔离真正需要的部分，仅此一项就比 Docker 快约 18 倍。针对第二处，它用**通用化 Zygote**：预先维护一批已经启动、且已导入常用库的模板进程，新函数实例直接从模板进程 fork 出来，跳过运行时初始化和库导入，再叠加约 3 倍加速。换句话说，SOCK 同时压低了 T_{sandbox} 和 T_{runtime} 。

第二条路线承认预热无法避免，转而让预热成本被更多函数复用。全量缓存整个容器最简单，但每个空闲容器都常驻一份完整内存，代价高；只缓存一部分或让多个函数共享容器又各有难处。RainbowCake³⁴ 的做法是按初始化阶段把容器拆成几层：Bare 层对应基础环境和工具，Lang 层对应语言运行时，User 层对应用户代码和依赖。User 容器只适合同一函数复用，Lang 容器可以在同语言函数之间复用，Bare 容器则可以服务任意函数。空闲容器超时后先清理用户代码并降级为 Lang，再释放运行时并降级为 Bare，于是平台可以保留更通用的下层、只在命中时补齐上层。它把用常驻成本换首次延迟这个取舍做得更细：常驻的不再是整只容器，而是更容易共享的初始化层。

图 87 对比了两条路线降低冷启动代价的不同方式：SOCK 缩短单次启动路径中的沙箱准备和运行时初始化，RainbowCake 则把容器状态拆成可复用的层，让命中的下层继续保留，只补齐请求缺失的上层。前者降低一次启动要付出的时间，后者降低预热状态长期占用的成本，二者可以叠加。

³³Edward Oakes 等, *SOCK: Rapid Task Provisioning with Serverless-Optimized Containers*, USENIX ATC 2018, pp. 57–70. <https://www.usenix.org/conference/atc18/presentation/oakes>

³⁴Hanfei Yu 等, *RainbowCake: Mitigating Cold-starts in Serverless with Layer-wise Container Caching and Sharing*, ASPLOS 2024, pp. 335–350, DOI:10.1145/3617232.3624871. <https://dl.acm.org/doi/10.1145/3617232.3624871>

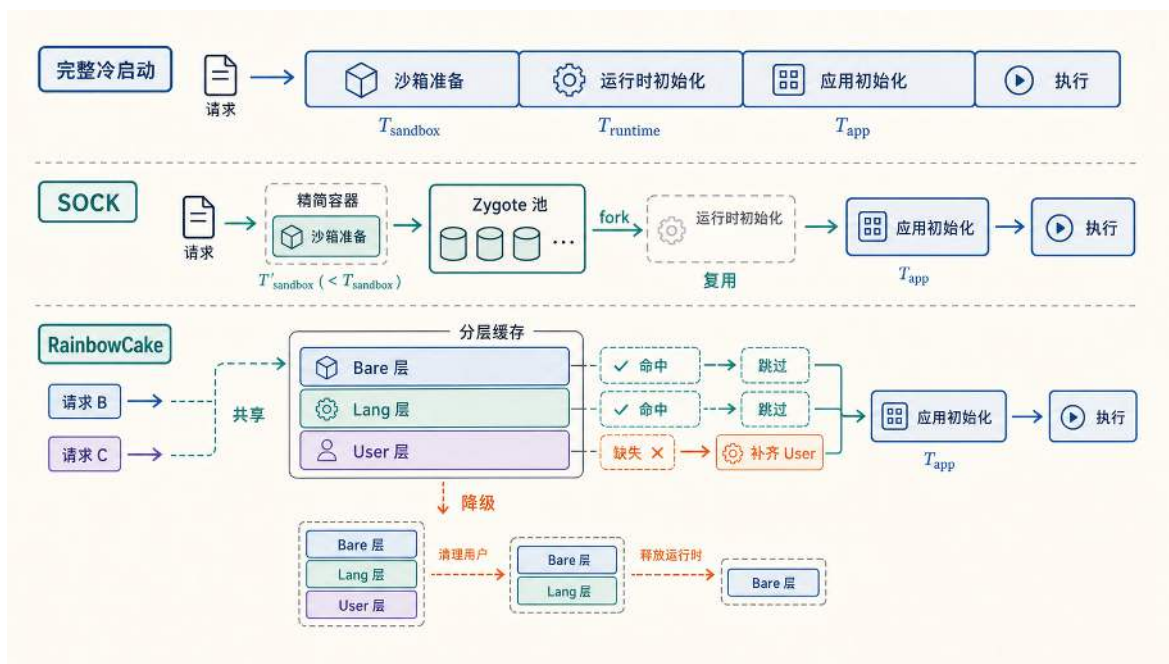


图 87: 容器快速启动的两条研究路线：SOCK 通过精简容器和通用化 Zygote 压低单次启动的 T_{sandbox} 与 T_{runtime} ，RainbowCake 将容器状态拆成 Bare、Lang、User 三层，使空闲容器可以逐层降级，并在后续请求命中时只补齐缺失的上层。图揭示了两条路线并不冲突：一条压低单次启动成本，一条摊薄常驻预热成本。

这两项工作和上面的工业方案指向同一个判断标准：冷启动不可能凭空消失，工程上能做的是把它的常数项压低，或把预热的常驻成本摊薄，问题只在于谁来付、以什么形式付。到了 Agent 场景，这个问题会以更极端的形式出现：需要在很短时间内创建大量可交互的隔离环境，快速启动不再是锦上添花，而是能否支撑训练与评测的前提。

6.5 面向 Agent 的高并发 Sandbox：从一次执行到经验生产

前面各节分别讨论了容器隔离、网络虚拟化、弹性调度和快速启动。Agent 训练与评测平台把这些能力组合到一个新的工作负载上：在短时间内批量创建大量可交互的隔离环境，让每个环境在模型多步动作之间持续存在，并把执行过程变成可验证、可复现的训练数据。

Agent 在本节中指的是一套闭环执行系统：模型观察环境状态，选择动作；动作通过工具作用到环境上（运行命令、修改文件、点击页面、安装依赖等），环境状态发生变化后将新的观察返回给模型。这个循环持续进行，直到任务完成、失败或达到步数上限。以修复代码仓库中的测试失败为例，模型需要运行测试、阅读报错、修改代码、再次运行测试，每一步动作都会改变后续环境状态。Agent 任务因此是一条连续展开的执行轨迹，而非单次函数调用。

训练数据的生产过程本身就是计算密集的：每个任务实例需要一个可交互的隔离环境，模型在其中执行多步动作；同一任务可能有多次尝试和多条路径；训练和评测随模型版本迭代反复执行。平台要承载的规模来自任务实例数、尝试次数、轨迹长度和交互频率的共同放大。EvoCUA

的技术报告和官方文章³⁵都把基础设施压力落在大规模 sandbox 承载、高频交互请求和分钟级批量创建能力上。这里不把不同发布口径中的请求吞吐写成单一固定数字，而用它说明同一个云计算问题：平台必须持续生产大量可验证的环境交互轨迹。DeepSeek-V4-Pro 的后训练框架³⁶同样将 Agentic Coding 和工具使用纳入能力培养，其基础设施需求也指向这一方向。

与游戏服务相比，Agent sandbox 的压力特征有所不同：游戏服务要求平台稳定承载玩家请求，Agent sandbox 要求平台批量承载会被模型不断修改的执行环境。两者都在考验隔离、调度、状态恢复和可观测性，但 Agent 场景额外要求环境可复位、可分叉、可批量创建。

图 88 把这些问题归到五个面向上，后面几节就沿这五个面向逐一展开：控制面负责任务准入门与资源租约，执行面提供可隔离、可复位的运行环境，交互面承接模型动作与环境观察，状态面支持 checkpoint 与 rollback，验证面把执行结果变成可复查的数据。

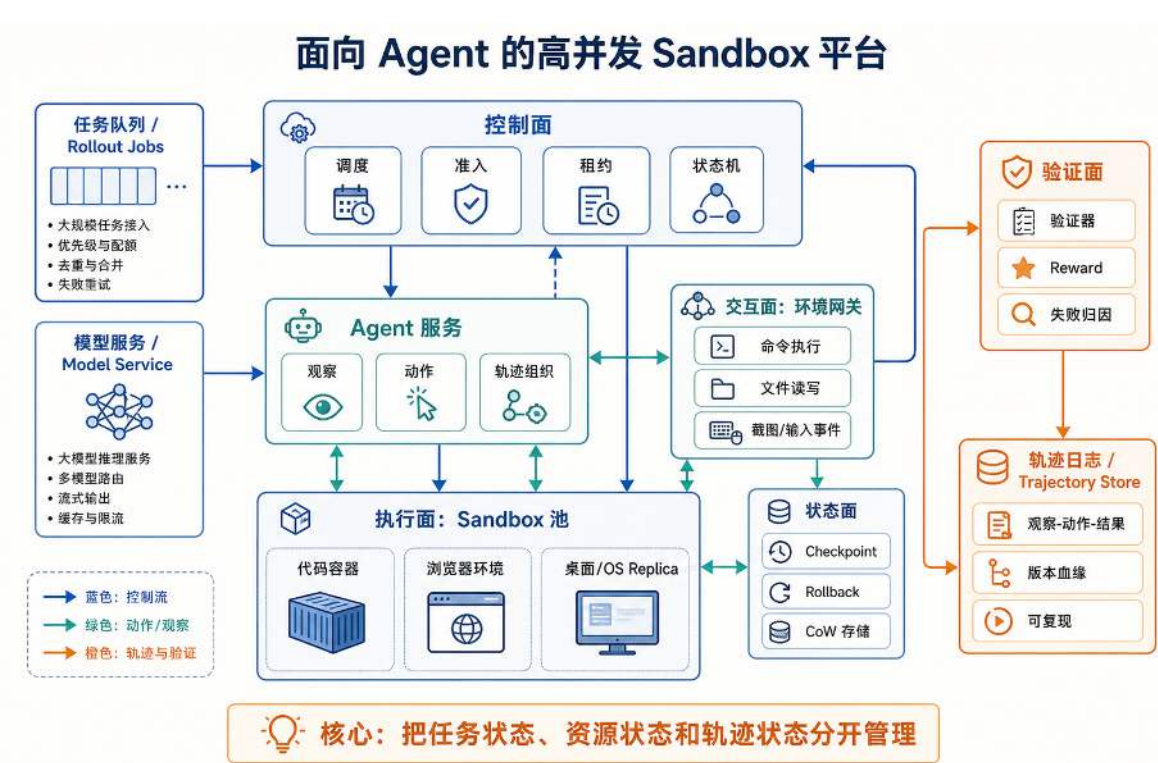


图 88: 高并发 Agent sandbox 平台将任务状态、资源状态和轨迹状态分开管理，并分别组织控制面、执行面、交互面、状态面和验证面。

这五个面向沿用前几节分析在线系统的同一套视角，这次用来观察 Agent 平台内部——控制、执行、交互、状态、验证在不同系统里名字可能不同，但要解决的问题是相通的。Agent sandbox 的特殊之处在于，它把一次请求的生命周期拉长为一条持续变化的执行轨迹：隔离环境要在多步动作之间保持可复位，网络通道要连续传递动作和观察，快照要支持回滚与分叉，日志

³⁵EvoCUA 技术报告：<https://arxiv.org/abs/2601.15876>；美团技术团队，EvoCUA 官方文章：<https://tech.meituan.com/2026/01/26/EvoCUA.html>。

³⁶DeepSeek-V4-Pro 技术报告：https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf。

要沉淀为可复查的训练和评测数据。下面先从单机顺序执行的最小闭环开始，再说明它在高并发平台中会被拆成哪些控制、执行和数据路径。

6.5.1 总体架构：先分清任务状态、资源状态和轨迹状态

在讨论十万级 sandbox 平台之前，先把一个 Agent 任务在单机上的完整生命周期拆开：一次只运行一个任务，系统为它启动一个容器，把模型接到这个容器里，让模型观察环境、执行动作、记录反馈，最后用验证器判断任务是否完成，再销毁容器。这个顺序执行的 Agent sandbox 闭环可以写成如下流程。

单机最小实现：顺序运行一次 Agent 任务

```
1 task := loadTask()
2 sandbox := startContainer(task.image)
3 trajectory := newTrajectory()
4
5 for step := 0; step < task.maxSteps; step++ {
6     obs := sandbox.observe()
7     action := model.generate(obs, trajectory)
8     result := sandbox.execute(action)
9     trajectory.append(obs, action, result)
10
11     if verifier.finished(sandbox, task) {
12         break
13     }
14 }
15
16 score := verifier.grade(sandbox, task)
17 saveTrajectory(task, trajectory, score)
18 destroyContainer(sandbox)
```

这段伪代码展示了 Agent 任务的基本闭环：观察环境，选择动作，执行动作，记录反馈，再根据验证器判断任务是否完成。作为单机实验它已经足够，但作为大规模训练和评测平台，它缺少三个系统能力。

第一个缺失是**状态一致性**。模型请求、环境创建、工具执行和验证器运行并不总是按顺序完成。模型返回时环境可能已经超时，环境启动完成时任务可能已经被取消，验证器异常退出时系统也不能简单地把结果记为模型失败。如果平台没有清晰的任务状态机，就很难判断一次失败到底来自模型、环境还是验证器。

状态不一致会进一步导致**资源泄漏**。Agent sandbox 不是普通函数调用。一次任务结束后，环境里可能留下临时文件、后台进程、浏览器缓存、打开的端口和未释放的磁盘写层。某台宿主机如果失联，控制面还要知道上面有哪些 sandbox 正在运行、哪些任务需要重试、哪些资源需要

回收。没有租约、心跳和复位机制，资源泄漏会随着任务数量持续放大。

资源泄漏之上还有**数据可追溯性**问题。一条轨迹不只是最终答案，还包括观察、动作、工具返回、终端输出、截图、文件变化、验证结果和失败原因。如果这些数据分散在不同日志里，缺少统一的任务版本、环境版本和模型版本标记，后续就很难复现这次运行，也很难判断这条轨迹能不能进入训练集。平台运行规模越大，越需要避免把难以解释的运行痕迹误当成高质量训练数据。

因此，设计大型 Agent sandbox 平台时，起点应当是三类事实来源：任务状态、资源状态和轨迹状态。容器、虚拟机或调度框架只有放到这些状态边界之后，才能明确各自负责哪一段问题。如果多个模块各自维护一份状态副本，系统在重试、故障恢复和并发调度时就无法确定哪个副本是权威来源。任务进展由谁记录、资源占用由谁追踪、轨迹数据由谁归档，必须在架构一开始就划清边界。

任务状态记录“一件事从开始到结束经历了什么”。比如一个训练批次有 10 万道任务，每道任务可能被同一个模型尝试多次，也可能被不同模型对比尝试。平台要区分清楚：这是一个批次里的任务，还是某一次具体尝试；这是第一次执行，还是从某个 checkpoint 恢复后的分支。可以把它抽象成 RolloutJob、TaskInstance、Attempt、Step 这样的对象层次。名称可以不同，但层次必须分开：批次、任务、尝试和步骤各自有独立标识，否则系统无法回答“这次失败发生在哪一次尝试的哪一步”。

任务状态还需要状态机。一个 Attempt 可能处在等待调度、环境创建、环境就绪、执行中、等待模型、等待工具返回、验证中、成功、失败、取消、超时等状态。每个状态都应该有进入条件、退出条件和超时规则。比如环境创建超过 30 秒没有成功，应该释放模型预算；等待模型返回超过限制，应该先保留环境一段时间，避免模型返回后找不到上下文；验证器异常退出，不能直接把模型判错，而应该标成验证失败。状态机是大型分布式系统稳定性的骨架：每个状态都有明确的进入条件、退出条件和超时规则。

状态机还需要被持久化，并且转移要有版本约束。否则，两个控制器实例可能同时认为自己有权推进同一个 Attempt：一个把它标记为超时回收，另一个却刚收到模型返回并准备继续执行。生产系统通常会给状态记录增加版本号或租约持有者，更新时检查“我看到的旧状态是否仍然成立”。只有检查通过，状态才能从 Running 转成 Verifying 或 TimedOut。这类 compare-and-swap 式的更新规则，保证控制面在重试、故障切换和并发调度下仍然只有一条被承认的任务时间线。

图 89 将 Attempt 的状态转移约束收束为一条生命周期：成功、任务失败、验证异常、已取消、已超时都是终态，进入终态后不再继续跳转；每次状态写回都必须通过版本号与租约持有者的 CAS 检查，避免并发控制器把验证异常误改为任务失败。

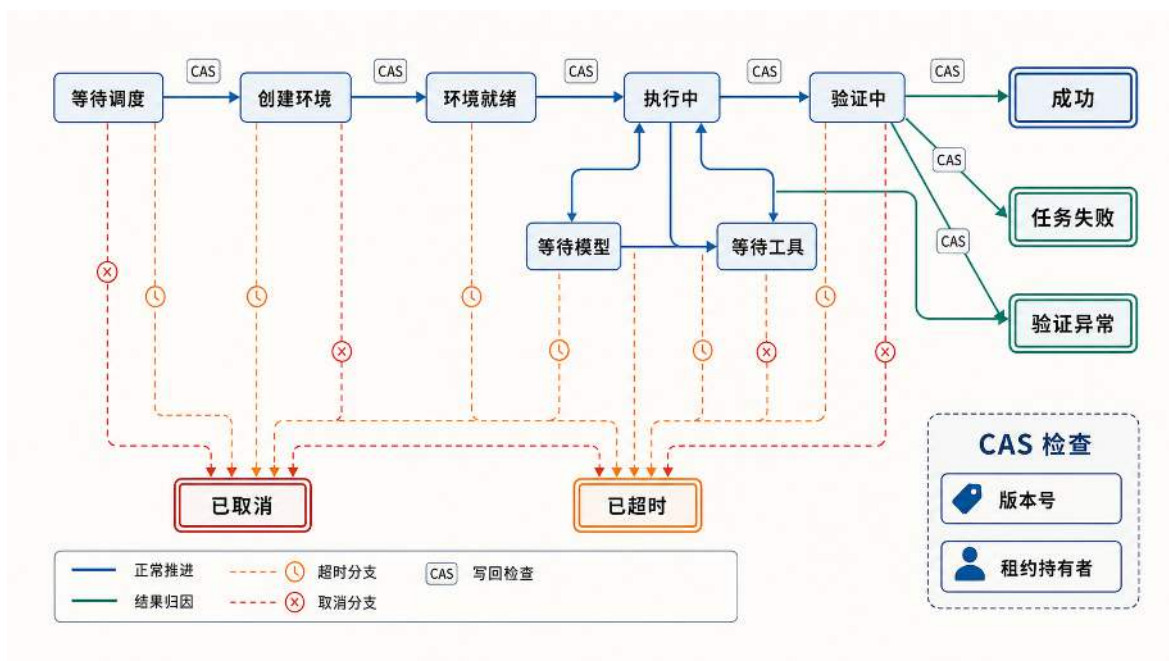


图 89: Attempt 生命周期状态机：状态转移、超时分支与 CAS 版本检查共同保证并发控制器只承认一条任务时间线。

有了这条时间线，后文的准入、租约、超时回收和验证归因就不再是分散规则，而是围绕同一个 Attempt 状态记录展开的控制面动作。

资源状态记录“谁占用了什么”。Agent sandbox 用完不清理，就会留下文件、缓存、后台服务、浏览器 profile、临时端口和进程树。平台需要知道哪些基础镜像可用，哪些 sandbox 在预热池里，哪些 sandbox 正在被某个 Attempt 使用，哪些需要复位，哪些已经损坏。SandboxLease 即租约：某次尝试在一段时间内借走某个环境。有了租约，控制面就能在任务超时、心跳丢失或节点失联时回收资源。

轨迹状态记录“训练样本是怎样产生的”。一条轨迹至少包括任务版本、环境镜像、模型版本、观察、动作、工具返回、终端输出、截图、文件变化、checkpoint 信息、验证结果、reward 和失败原因。这些信息构成一条 TrajectoryRecord，即一份可追溯的执行档案。它们的体积差别很大：状态更新可能只有几百字节，截图和日志可能很大；任务开始和结束不频繁，工具调用却可能每秒发生多次。如果将所有数据集中写入一个中心服务，I/O 压力会随任务规模线性增长，系统很快成为自身数据量的瓶颈。

这三类状态分开后，后面的设计才有落点。控制面负责让任务状态和资源状态保持一致；执行面负责提供真实、隔离、可复位的环境；交互面负责承接高频观察和动作；状态面负责把探索过程变成可回滚、可分叉的过程；验证面负责把运行痕迹变成可以训练和评测的数据。大型系统需要先把责任边界切清楚，后面每一层的扩缩容、恢复和排错才有依据。三类状态中，任务状态将在控制面中继续展开，资源状态在执行面中展开，轨迹状态则在后文的验证面中展开。下面先从控制面入手，说明大型平台如何让任务状态与资源状态保持一致。

6.5.2 控制面：维护全局秩序

Kubernetes 和 HPA 中的控制面职责是：维护期望状态，观察当前状态，再发出调度和回收动作。Agent sandbox 平台沿用同样的思路，只是被管理的对象从普通服务副本变成了大量会被模型反复修改的执行环境。

有了任务状态、资源状态和轨迹状态，下一步就是让这三类状态在运行时保持一致。这正是控制面的职责。它要解决的第一个问题，是让系统在十万级 sandbox 下仍然知道“谁在做什么”。在单机顺序流程里，步骤是线性的：启动环境，调用模型，执行动作，收集结果。工业系统里，这些步骤会同时发生在成千上万个任务上。有的任务在等模型返回，有的任务在等环境启动，有的任务正在跑测试，有的任务已经卡死，还有的任务被用户取消了。控制面如果只提供几个无状态的转发 API，就无法追踪任务进展，也无法在故障时做出正确的回收或重试决策。

控制面的第一项设计，是把模型服务、Agent 服务和环境服务分开管理。三类工作使用的资源不同，运行节奏不同，失败方式也不同。模型服务处理昂贵计算，一次推理可能占用 GPU、显存、KV cache 和推理队列。它关心模型版本、上下文长度、token 预算、批处理和限流。token 预算是对一批任务可消耗的模型计算总量的上限约束。模型服务过载时，控制面应减少新的推理请求，让任务在队列里等待；继续启动更多环境，只会让这些环境空等模型返回。

Agent 服务处理任务推进。它主要维护 **Attempt** 的状态：当前走到第几步，上一轮观察是什么，模型给出的动作能否被解析，工具返回是否超时，轨迹是否已经写入，是否应该进入验证器。Agent 服务的角色是一次尝试的流程控制器：把观察送给模型，把模型输出转化为工具动作，再把环境反馈组织成轨迹。容器创建和 GPU 调度则属于环境层与模型层的职责。

环境服务处理 sandbox 生命周期。它管理镜像、租约、启动、就绪检查、命令执行、文件读写、网络策略、残留进程清理和环境复位。环境服务不评价模型做得好不好，也不决定训练样本是否有价值；它只保证某个 **Attempt** 能拿到一个隔离、可用、可回收的执行环境。

如果把三类工作合并到一个大进程里，小规模时尚可运行，但并发量上升后各类耦合故障会迅速暴露。模型请求慢了，环境可能已经启动却一直空等；环境创建失败了，模型窗口可能已经被占住；Agent 流程重试了一次，环境层可能创建出两个 sandbox；验证器异常退出时，系统无法判定这次失败应该归因于模型、环境还是验证器。服务拆分的作用，是让每一类失败停留在自己的边界内，并且能被控制面明确记录和处理。

一个合理拆分的检验标准是：某一层重启或限流时，其他层仍然能保持自己的状态。模型服务限流，任务可以继续排队，环境不必提前创建；环境服务扩容，模型服务不必改变推理策略；Agent 服务重试，环境服务可以根据同一个 **AttemptId** 返回已有租约，避免重复创建资源。MegaFlow³⁷ 将 agent 训练基础设施拆成 Model Service、Agent Service 和 Environment Service，正是为了让模型计算、任务推进和环境生命周期可以独立扩缩容。

服务边界清楚后，控制面才能分别处理扩缩容和故障恢复。模型服务负载较高时，就限制新任务的模型请求；环境池紧张，就减少新任务准入；轨迹存储变慢，就先把结果写入本地缓冲，再异步归档。控制面维护的是全局秩序：谁在排队，谁拿到了环境，谁占用了模型预算，谁超时

³⁷MegaFlow: Large-Scale Distributed Orchestration System for the Agentic Era, arXiv:2601.07526.

了，谁需要重试，谁应该被回收。

这里的调度不能只按机器空闲程度决策。一次 **Attempt** 正式开始之前，控制面要先确认三件事：环境层能不能提供 sandbox 租约，模型层有没有推理预算，数据层能不能可靠写入轨迹日志。这三个条件对应的是同一条执行链路上的三个瓶颈。只要其中一层已经饱和，继续把新任务放进系统，反而会产生更多无法完成的执行：环境启动了，却一直等不到模型返回；模型窗口占住了，却发现环境还没准备好；执行过程还在产生截图和命令输出，后面的存储系统却无法及时写入。

因此，调度器不能只依据当前还有多少 CPU 和内存，还要把模型预算、环境租约和日志积压一起纳入准入判断。一个入口检查过程可以这样组织：控制面先查看队首任务，依次检查模型、环境和日志三条链路；只有三条链路都能继续承载新任务时，**Attempt** 才进入执行阶段。

准入与反压：一次 Attempt 进入系统前的检查

```
1 attempt := taskQueue.peek()
2
3 if modelBudget.full() {
4     keepWaiting(attempt)
5     return
6 }
7
8 if sandboxPool.noLease(attempt.envSpec) {
9     keepWaiting(attempt)
10    return
11 }
12
13 if trajectoryLog.backlogHigh() {
14     reduceNewAttempts()
15     reduceScreenshotRate()
16     return
17 }
18
19 lease := sandboxPool.acquire(attempt.envSpec)
20 taskQueue.pop()
21 startAttempt(attempt, lease)
```

这种入口减速机制称为**反压** (backpressure)。反压的含义是：当下游的处理能力不足时，上游不要继续全速生产，而要主动减速，把压力挡在入口处。在这个例子里，模型预算满了，任务就留在队列里；环境池没有租约，就不要提前占住模型窗口；轨迹日志积压过高，就降低新任务准入，必要时减少高频截图。这样做会牺牲一部分吞吐，但已经进入系统的任务仍然有环境可用、有模型可调、有日志可写。对大型平台来说，短时间变慢仍可接受；更严重的后果是各层独立推

进而互不协调，最后产生一批无法复现、无法判因、也不适合进入训练集的轨迹。

实际系统里，这个入口检查还要进一步变成“资源预留”。检查只回答“现在看起来能不能跑”，预留则保证“接下来这次尝试确实有资源可以用”。如果调度器只检查不预留，两个并发 Attempt 可能同时看到同一份空闲模型预算或同一个空闲 sandbox，随后发生超分配。更稳妥的做法，是把模型预算、sandbox 租约和轨迹写入通道一起纳入一次准入事务：任何一步失败，都要撤销已经完成的预留，让任务回到等待状态。

图 90 把这次准入排成一条泳道，能看清它其实压着两条边界。一条是服务边界：模型服务、环境服务和轨迹日志各自独立，任一处的故障不会越界扩散。另一条是事务边界：三处预留要么一起确认、一起提交，要么在任一步失败后沿已经完成的预留逐一撤销，让 Attempt 启动前的资源状态始终一致。

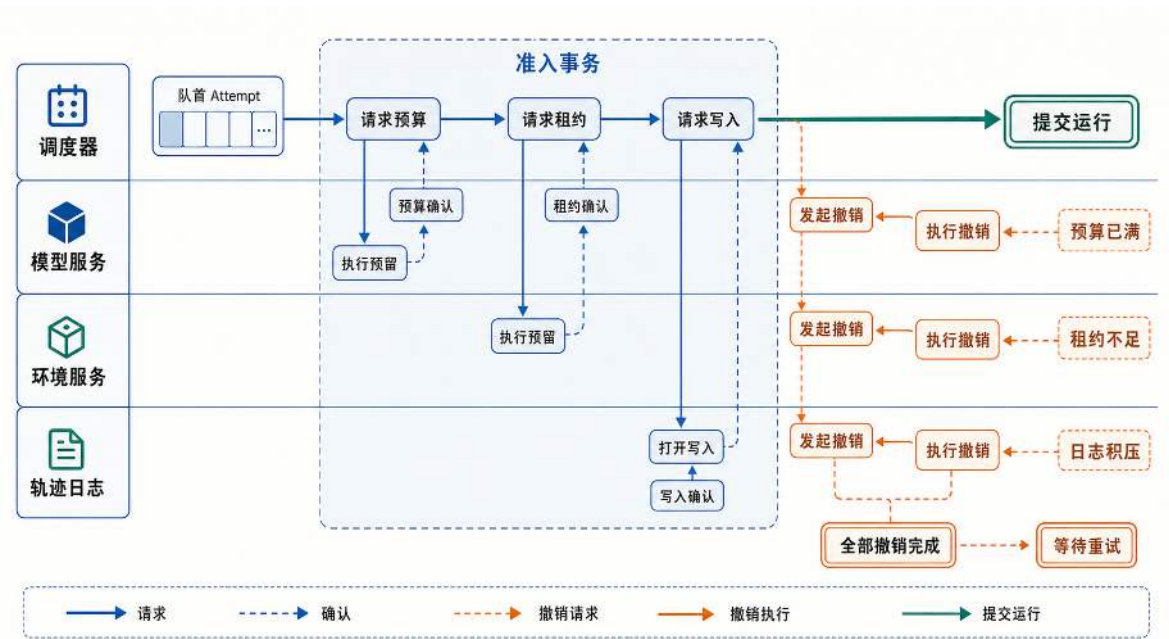


图 90: 控制面准入事务把模型预算、sandbox 租约和轨迹写入通道绑定为一次可撤销的资源承诺，任一环节失败都回滚已预留资源。

把这条事务落成代码，每完成一步预留就登记到一份可回滚的记录里，任一步失败都据此逐条撤销：

准入事务：把检查变成可撤销的资源预留

```
1 attempt := taskQueue.peek()
2
3 reservation := newReservation(attempt.id)
4
5 if !modelBudget.reserve(attempt.id, attempt.maxTokens) {
6     keepWaiting(attempt)
7     return
8 }
9 reservation.add(modelBudget)
10
11 if !sandboxPool.reserve(attempt.id, attempt.envSpec) {
12     reservation.releaseAll()
13     keepWaiting(attempt)
14     return
15 }
16 reservation.add(sandboxPool)
17
18 if !trajectoryLog.openWriter(attempt.id) {
19     reservation.releaseAll()
20     slowDownAdmission()
21     return
22 }
23 reservation.add(trajectoryLog)
24
25 taskQueue.pop()
26 markRunning(attempt, reservation)
```

这段逻辑对应云平台里的 admission controller 与 lease management。控制面在动作开始前建立可追踪的资源承诺；当任务取消、超时或节点失联时，它也能沿着这些承诺释放资源。MegaFlow 的三服务拆分解解决的是“谁负责哪类状态”，这里的准入事务解决的是“这些状态怎样在一次启动中保持一致”。

控制面还必须处理重复请求和部分失败。设想以下场景：Agent 服务准备开始某个 Attempt，于是向环境服务发送“创建 sandbox”的请求。环境服务可能已经在某台宿主机上创建好了容器，并分配了磁盘写层、端口和进程组；但就在它把 sandbox 地址返回给 Agent 服务之前，网络连接断开了。此时 Agent 服务只知道“我没有收到结果”，却不知道环境服务到底有没有把环境创建出来。

如果 Agent 服务直接重发一次创建请求，环境服务可能会再创建一个新的 sandbox。这样，同一个 Attempt 就会对应两个环境：第一个环境已经存在但没有被 Agent 服务记录下来，第二

个环境是重试时新建的。前者会继续占用 CPU、内存、磁盘和端口，却没有任务接管；后者则被 Agent 服务拿来继续执行。任务数量一大，这类“没人认领的环境”会变成严重的资源泄漏。反过来，如果 Agent 服务为避免重复创建而选择不重试，那么它手里没有 sandbox 地址，任务状态只能停在正在创建环境，后续模型调用、工具执行和验证都无法开始。

解决这个问题，需要让创建请求带上稳定的幂等标识，比如 `AttemptId`。幂等 (idempotency) 强调的是“同一逻辑请求应得到一致返回”，常见实现是去重键 + 状态查询：环境服务收到 `AttemptId=A` 时，先查该尝试是否已有历史结果；若已有历史，则返回历史状态（已创建/创建中/失败）并继续推进，而不直接发起第二次创建。如果返回结果是 `Unknown`，需要结合状态确认而不是只依赖单次返回。这样可降低“网络抖动带来的重复重试”的概率，但并不自动等价于通用的 `exactly-once` 副作用语义。

控制面还要把任务准入做成明确规则。队列里还有位置，并不代表任务就可以立刻开始；控制面还要知道这个任务需要什么样的环境。代码修复任务可能只需要一个 Linux 容器、一份仓库、编译器和测试脚本；网页任务可能需要浏览器、固定分辨率、网络访问策略和截图接口；桌面任务还可能还需要窗口系统、剪贴板、输入设备和更大的磁盘写层。它们都叫 sandbox，但资源成本和启动路径完全不同。

因此，任务进入系统之前，控制面需要先生成一张环境需求清单。这里把它记作 `EnvironmentSpec`。这张清单至少要写清楚基础镜像、CPU 和内存配额、磁盘大小、是否需要浏览器或桌面、是否允许访问外网、最多允许执行多少步、验证器从哪里启动。调度器拿到这张清单后，才能判断应该从哪个环境池里申请租约，应该把任务放到哪类宿主机上，应该预留多少模型预算和日志空间。如果没有这张清单，调度器只知道“我要一个 sandbox”，却不知道它到底是轻量代码容器、浏览器环境，还是带 GUI 的桌面环境。结果往往是两种：轻任务被分配到昂贵的桌面环境里，浪费资源；重任务被放入普通容器里，启动后因资源不足而无法正常运行。

生产级平台还会把控制面拆得更细。DeepSeek-V4-Pro 技术报告中的 DSec 使用 API gateway、per-host agent 和 cluster monitor 三类组件：入口层处理请求和鉴权，宿主机侧组件管理本机 sandbox，监控组件维护集群状态。这个设计说明，大规模 sandbox 控制面要持续维护生命周期、租约、故障和恢复关系；“创建容器 API”只是其中很小的一部分。

到这个规模，控制面管理的已经超过了“启动多少个容器”这一层。它要维护每次尝试背后的一组状态记录：环境是否已经分配，模型推理额度是否已经占用，任务是否超过时间限制，资源是否完成回收，轨迹是否已经可靠落盘。只要这些状态有明确的生命周期、超时规则、租约关系和幂等约束，十万级 sandbox 才能作为一个整体系统运行，避免退化成十万个彼此孤立的进程。

6.5.3 执行面：环境保真度与资源密度的取舍

控制面把任务分配出去之后，压力转移至执行环境本身。第 6.1 节中，容器依靠 Namespace、Cgroups 和 OverlayFS 提供隔离、限额和文件系统视图；Serverless 部分则涉及冷启动、预热和轻量沙箱。在 Agent sandbox 场景里，这些机制会同时面对两个要求：环境要足够真实，又要能被大规模创建、复位和回收。

控制面解决“谁该拿到环境”，执行面解决“这个环境到底长什么样”。这里不能简单问“用容器还是虚拟机”，而要先问任务需要多真实的环境。`EnvironmentSpec` 里写的，正是这些问题：操作系统是什么，CPU 和内存给多少，磁盘从哪个镜像启动，能不能联网，允许哪些工具，是否需要浏览器，是否需要桌面。代码任务可能只需要一个 Linux 容器、一个仓库和一套测试工具。网页任务需要浏览器、页面状态、Cookie、截图和网络条件。Computer-use 任务的资源需求更高，它需要窗口系统、输入设备、屏幕流、剪贴板、应用安装状态和后台服务。

环境越真实，轨迹数据越有价值，但成本也越高。纯容器启动快、密度高，适合命令行和代码任务；虚拟机隔离更强，系统行为更完整，但启动慢、占用大；Docker 加 KVM 是一种分层折中：外层容器接入 Kubernetes 等集群调度体系负责编排与生命周期管理，内层通过 KVM 虚拟化提供更完整的操作系统语义和更强的隔离边界。DeepSeek DSec 把 function call、container、microVM 和 fullVM 放到统一接口之后，调用方可以用同一套命令执行、文件传输和 TTY 访问接口使用不同后端。function call 适合无状态工具调用，可以放进预热容器池消除冷启动；container 适合 Docker 兼容的代码任务；microVM 通过 Firecracker 提供更强隔离；fullVM 则面向需要完整客户机操作系统的任务。这些后端体现了分级执行的设计原则：平台必须按任务风险、保真度和成本选择运行时，统一接口只能解决调用方式一致，不能替代运行时选择。

OSGym³⁸ 这类工作把基本单元做成 OS replica，也就是一份可配置、可复位、可操作、可评测的操作系统副本，正是为了保留真实桌面任务需要的窗口、用户目录、后台进程、应用缓存和 GUI 反馈。它进一步说明，完整 OS 环境会把执行面压力推向 CPU、内存、磁盘和内核参数。系统如果想在台宿主机上运行大量 replica，就要考虑硬件感知调度、预热池、资源守护和故障隔离。

OS replica 的成本压力集中在三个方面。磁盘方面，每个 replica 都需要一块可启动磁盘，如果全量复制，几百个环境就会迅速消耗数 TB 存储；OSGym 使用 KVM 虚拟化配合 copy-on-write 磁盘，让多个 VM 共享同一份基础镜像，只为各自修改过的块付费。启动链路方面，按需创建和销毁 VM 容易让训练批次卡在环境准备阶段，因此 OSGym 维护固定大小的预热 runner pool：环境在训练前先准备好，任务来了就分配，任务结束后复位再放回池中。宿主机资源方面，大量 VM 运行在容器中，会触碰文件描述符、inotify watches、AIO context、连接跟踪表等内核限制；资源守护和系统参数调优不是附加细节，而是执行面能否高密度运行的条件。

可以据此得到一个判断标准：如果任务结果主要由命令行输出和文件变化决定，容器通常就够了；如果任务结果依赖系统级行为，比如窗口、剪贴板、桌面应用、后台服务和浏览器状态，就需要更完整的 OS replica。把所有任务都放进 VM，会浪费集群资源；把桌面任务压进普通容器，又会让模型学到失真的环境规律。

从调度角度看，执行环境按保真度可以分级。命令行 sandbox 适合代码补丁、脚本运行、编译测试，它关心的是文件系统、编译器、依赖管理和进程隔离。浏览器 sandbox 适合网页操作和表单任务，它除了文件和进程，还关心浏览器 profile、Cookie、页面加载、截图和网络策略。桌面 sandbox 适合 Computer-use 任务，它需要窗口系统、输入设备、屏幕流、剪贴板、应用安装状态和后台服务。不同级别不应该用同一套成本模型。让桌面 sandbox 执行简单代码任务，是浪

³⁸OSGym: Scalable OS Infra for Computer Use Agents, arXiv:2511.11672。

费；让普通容器去模拟真实桌面，则会失真。

保真度并不是抽象概念。字体缺失会改变页面布局，分辨率不同会改变按钮位置，浏览器版本不同会改变 DOM 行为，键盘映射不同会让输入动作失败，后台服务残留会影响下一轮任务。对人来说，这些都是小问题；对模型来说，这些就是观察空间的一部分。如果训练时环境保真度不足，模型学到的动作在真实系统里会打折。如果环境过于完整，单个 sandbox 的内存、磁盘和冷启动成本又会压垮集群。

执行面的核心工作，是在保真度、隔离强度、启动速度和资源密度之间建立可量化的取舍关系。文件系统可以使用 copy-on-write。逻辑上，每个环境都有自己的完整磁盘；物理上，它们共享同一个只读基础镜像，只为自己修改过的块付费。这样既能保证隔离，又不会为每个环境全量复制几十 GB 的系统盘。镜像也要分层：系统层、语言运行时层、浏览器层、任务依赖层、用户写层。层次分清后，更新某个依赖包不需要重做整个桌面镜像。DeepSeek DSec 中使用 3FS 支撑的只读 EROFS 层和 overlaybd 写层，本质上也是用分层存储和按需加载缓解大规模镜像启动压力。

冷启动路径也要拆开看。一个环境从“没有”到“可用”，中间可能经历拉取镜像、创建磁盘、启动执行环境（容器或 VM）、启动桌面、打开浏览器、加载任务仓库、运行就绪检查。每一步都可能慢。仅知道启动一个 sandbox 需要 3 秒并不充分，平台必须知道慢在哪里。镜像拉取慢，就做镜像预分发；磁盘创建慢，就做预创建；桌面启动慢，就维护 warm pool；任务依赖安装慢，就把常用依赖烘焙进基础层。

预热池是执行面最重要的工程手段之一。训练和评测常常按 batch 运行，如果每来一个任务才从零启动 VM、挂载磁盘、加载桌面、启动应用，模型会一直等环境。预热池提前准备一批干净环境，任务来了就借走，任务结束后复位再放回。它的代价是占用常驻资源，收益是吞吐稳定、尾延迟下降。工业系统里经常要维护多个池：代码池、浏览器池、桌面池，不同池对应不同镜像和不同启动成本。

复位比启动更难。启动只要把环境拉起来，复位却要保证环境回到干净状态。代码仓库有没有残留修改，浏览器缓存有没有污染，后台服务有没有退出，临时端口有没有释放，用户目录里有没有上一次任务留下的文件，都要检查。许多 sandbox 平台表面上可以运行任务，是因为它只验证了能启动；而真正决定训练数据质量的，是任务结束后环境能否可靠地恢复到初始状态。

执行面还要负责安全边界。Agent 会运行模型生成的命令，可能写文件、启动服务、下载依赖、访问网络。平台不能默认这些动作都是安全的。常见做法包括限制出网、限制可挂载目录、限制系统调用、限制进程数量、限制 CPU 和内存、隔离用户身份、记录所有工具调用。越接近真实电脑，攻击面越大；越强调安全，环境越可能受限。因此，安全策略也应当随任务类型和风险等级分级配置。上述保真度、隔离强度、启动速度和资源密度之间的取舍如果处理不当，后续的调度、交互、checkpoint 和训练吞吐都会受到连锁影响。

6.5.4 交互面：分离控制路径与高频数据路径

执行环境准备好之后，系统还要把模型和环境之间的高频交互稳定传输。Agent sandbox 的交互面承担的正是这一职责：将模型发出的动作送达执行环境，将环境产生的观察送回模型。这

需要回到云平台中常见的控制面与数据面分工：控制面决定谁可以使用哪个环境，数据面承载具体的数据传输。容器网络和 Serverless 中已有类似分工：控制逻辑要稳定，数据路径要短、快、可扩展。Agent sandbox 的数据面还要传输截图、终端输出、文件片段、工具调用结果和鼠标键盘事件，压力比普通请求响应更复杂。

普通批处理任务启动后独立执行直到结束，Agent sandbox 却要 and 模型持续通信。模型每走一步，都可能产生观察和动作。观察可能是一张截图、一段终端输出、一个网页 DOM、一个文件片段；动作可能是鼠标点击、键盘输入、命令执行、文件写入、浏览器跳转。一次任务可能几十步、上百步；一批任务可能成千上万条轨迹同时执行。

最朴素的设计，是让所有交互都经过中心控制器。模型要执行命令，需先请求控制器；控制器转发给 sandbox；sandbox 返回输出；控制器再转给 Agent 服务。这个设计在小规模下可行，但在高并发下会迅速成为瓶颈。控制器原本只需要管理任务状态和资源租约，现在却要转发截图、终端输出、文件内容和工具调用结果。请求量增大后，控制器的带宽和 CPU 被数据转发占满，调度、回收和健康检查的响应时间随之恶化。

更合理的做法，是把控制面和数据面分开。控制面管“谁可以使用哪个环境”，数据面管“这个环境里的每一步动作和观察怎样传输”。环境创建、任务分配、状态更新、租约回收走控制面；截图传输、命令执行、文件读写、鼠标键盘事件走更贴近环境的数据通道。EvoCUA 在高并发下使用异步网关承接大量交互请求，体现了这种分工：控制面维持生命周期秩序，高频交互由数据通道处理。

分开之后还要回答一个问题：数据既然不再绕经中心控制器，控制会不会随之丢失？图 91 表明，控制并未随之丢失：低频的创建、删除、策略、心跳仍归控制面，高频的动作与观察改走 Agent 服务到 I/O 网关再到 sandbox 的短路径；而网关上的 SandboxId 路由、租约版本校验和沙箱锁，恰好把控制面的所有权约束钉在数据面的入口上。

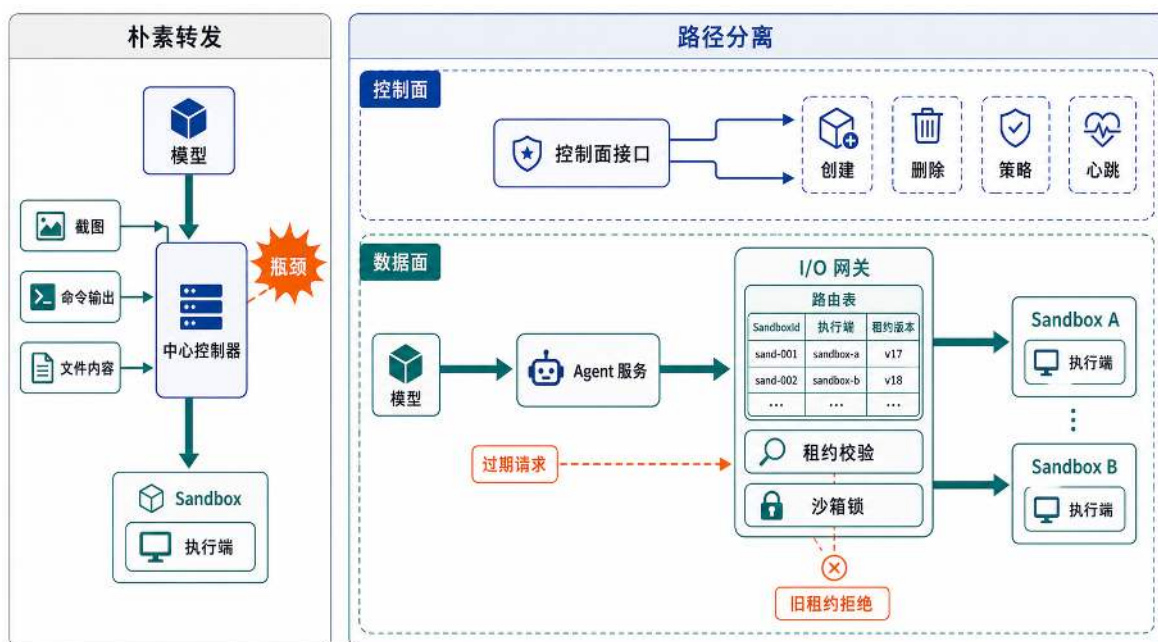


图 91: 交互面将低频生命周期控制路径与高频动作观察数据路径分离，数据面经 I/O 网关直达 sandbox 执行端，并用租约校验阻止旧请求误送。

顺着这条数据面路径，一次工具调用会这样走。模型输出一个动作，比如“运行测试”。Agent 服务先做语义检查：这个动作是否属于允许工具，参数是否合法，是否超过任务预算。然后它把动作发送给环境网关，网关根据 `SandboxId` 找到对应的执行端。执行端在 sandbox 内运行命令，收集 `stdout`、`stderr`、退出码和运行时间，再把观察返回。控制面只需要知道这一步开始了、结束了、是否超时，不应该亲自搬运所有输出内容。

这条热路径需要一张稳定的路由表：`SandboxId` 对应哪个网关、哪个宿主机、哪个执行端，以及当前租约版本是多少。租约版本很重要，因为 sandbox 可能已经被回收后重新分配给另一个 `Attempt`。如果旧请求带着过期版本号到达，网关必须拒绝它，而不能把旧动作误送进新环境。高并发交互面还会给每次工具调用分配 `ToolCallId`，用于重试去重和结果归档。它帮助识别“同一条调用的重复重试”，但在返回状态未知或执行超时，网关仍需查阅状态并做幂等确认，否则不能简单用 ID 就宣称“只执行一次”。

环境网关是 sandbox 对外暴露的唯一入口，承担动作转发和权限校验双重职责。比如同样是文件读取，读任务目录可以，读宿主机目录不行；同样是网络请求，访问任务提供的本地服务可以，访问公网可能不行；同样是命令执行，普通用户权限可以，提权操作不行。交互面如果只追求吞吐，不做权限检查，就会把 sandbox 变成一个高并发的风险入口。

交互面要承载高频 I/O，就需要异步 I/O 网关、连接复用、分片路由和限流。异步网关让大量慢请求不阻塞工作线程；连接复用减少频繁建连的开销；分片路由把不同 sandbox 的流量分散到不同网关；限流和背压保护环境、模型服务和日志系统。Orchard Env³⁹ 的环境服务把生命周期管理放在 `orchestrator` 中，而把命令执行、文件读写和健康检查直接路由到 sandbox pod 内

³⁹Orchard: An Open-Source Agentic Modeling Framework, arXiv:2605.15040.

部的轻量执行 agent，并绕开 Kubernetes API server 的热路径。这个例子说明，云平台里的控制面稳定性和数据面吞吐不能混在一条路径上解决。

Orchard Env 的做法可以抽象成冷路径和热路径分离。冷路径包括创建 Pod、删除 Pod、设置资源限制、下发 NetworkPolicy、等待 readiness probe，这些操作低频但需要一致性，适合由 orchestrator 通过 Kubernetes API 管理。热路径包括 `/exec`、文件上传下载、目录列举、执行端状态查询和命令超时处理，这些操作高频且延迟敏感，适合直接路由到 sandbox 内部的执行 agent。为了避免同一个 sandbox 内部多个命令并发冲突，执行端还需要 per-sandbox lock；为了避免长时间无人使用的环境泄漏，控制面还要通过 heartbeat 清理过期 sandbox。这里的云计算概念仍然是控制面和数据面分离，只是数据面承载的不再是普通 HTTP 请求，而是 Agent 的动作和观察。

交互面还需要处理观察数据分级。终端输出可能需要完整保存，给训练和排错使用；屏幕截图可能只需要在轨迹中保存关键帧，连续帧可以压缩或降采样；大型文件内容不应该直接写入轨迹，可以保存引用和摘要；调试日志可以短期保留，验证结果要长期保留。数据分级的目标是保证训练数据可用，同时避免高频交互请求在存储和网络带宽上形成新的瓶颈。

因此，交互面的指标不能只看平均延迟。它还要看尾延迟，也就是最慢的那一小部分请求。如果一次截图传输偶尔延迟 5 秒，模型就会在错误的时间点上做判断；如果命令输出偶尔丢失部分内容，轨迹就可能失去关键证据。Agent 任务是长链路任务，单步小抖动会沿着轨迹累积，最后表现为任务失败或数据不可用。

在高频、多并发的交互负载下，瓶颈可能首先出现在 I/O 网关、日志管道、截图传输和连接管理上。没有稳定的交互面，环境即使已经成功启动，模型也无法及时获得可靠反馈，长链路任务会在延迟、丢包和观察缺失中逐步失真。

6.5.5 状态面：把回滚和分叉放进训练循环

交互过程一旦变长，环境的中间状态就会成为一种资源。前面讨论 Serverless 冷启动时，快照主要用于加快环境恢复；在 Agent sandbox 中，快照还要支持探索分叉。模型走到某个中间状态后，平台可能需要从这里保存、回滚，再尝试另一条路径。因此，这里讨论的仍然是云计算里的快照、恢复和存储一致性问题，只是它们被放进了训练与评测循环里。

如果 Agent 只线性执行一次，环境坏了可以从头重跑，虽然慢，但还能接受。训练和评测很快会走到更复杂的形态：同一道题要跑多条候选轨迹；搜索算法要从中间状态分叉；调试代码时，模型会尝试一个 patch、跑测试、失败后回到修改前；抢占式云资源被回收时，平台希望换一台机器继续跑。

这些场景都要求 sandbox 支持 checkpoint 和 rollback。在这里，checkpoint 已经进入训练循环本身。如果第 20 步失败，每次都从第 1 步重放，会浪费模型调用、工具执行和环境等待时间。系统需要从关键分岔点恢复，再尝试另一条路径。恢复越快，同样的模型预算就能覆盖更多分支。

checkpoint 的难点不只是保存一份文件。平台要先找到一个一致性边界：上一条工具调用已经结束，文件系统写入已经落到可追踪层，后台进程状态已经可枚举，Agent 侧也记录了当前 turn 的位置。否则，系统可能保存到一半状态：磁盘上已经有新文件，进程内存还停留在旧逻辑；

或者 Agent 记录已经进入下一步，环境却还没完成上一条命令。这样的快照恢复出来后看似可用，实际会把轨迹带到一个从未真实出现过的中间状态。

问题是，Agent 的状态到底在哪里？只保存聊天记录不够。模型可能记得自己执行过 `pip install`，但环境里的依赖树并没有恢复。只保存文件系统也不够。Python 解释器堆、后台进程、打开的文件描述符、临时服务都可能影响下一步。只保存进程也不够，因为磁盘可能已经被另一个分支改过。对 Agent sandbox 来说，一个可恢复点至少要把文件系统状态、进程状态、GUI 状态、后台服务和 Agent 的 turn 位置对齐。

可以把状态分成四层。第一层是任务状态，也就是 Agent 已经走到第几轮、上下文里有哪些观察、当前预算还剩多少。第二层是文件系统状态，包括代码修改、依赖安装、临时文件和浏览器 profile。第三层是进程状态，包括正在运行的服务、打开的连接、解释器内存和文件描述符。第四层是外部状态，比如远端网站、数据库、消息队列和网络服务。前三层平台还能控制，第四层最麻烦，因为它可能已经产生不可逆副作用。所以训练 sandbox 通常要尽量把外部依赖也模拟进环境，或者把外部调用限制在可回放、可清理的范围内。

DeltaBox⁴⁰ 的设计重点，是让保存和回滚足够快。它把文件系统做成可切换的层，每次 checkpoint 冻结当前写层，再插入新的写层；回滚时切回对应层。进程状态则用增量 dump 和 template fork 加速恢复。这样做的目标很明确：checkpoint/rollback 不能是几秒钟一次的灾备动作，而要快到可以放进 Agent 的搜索循环里。如果保存一次状态本身就很慢，模型宁可从头跑，checkpoint 就失去了训练价值。

Crab⁴¹ 的设计重点，是不要每一步都存。Agent 的一轮动作可能只是读取文件，也可能修改源码、安装依赖、启动服务。OS 能看到系统调用和状态变化，却不知道这一轮对 Agent 有没有意义；Agent 框架知道工具调用，却看不到底层 OS 副作用。这就是 Agent 层和 OS 层之间的语义断层。Crab 在每一轮动作结束时观察操作系统可见的影响，再决定这一轮不存、只存文件系统、只存进程，还是做完整 checkpoint。这样可以避免把每一步都做成重型快照。

checkpoint 的位置也要有策略。任务刚开始时，环境很干净，保存一次基础点有价值；模型完成安装依赖后，保存一次可以避免重复安装；修改关键文件前，保存一次便于分支；运行大型测试前，保存一次可以从测试前恢复。相反，连续读取文件时没有必要频繁保存。好的状态面要回答四个问题：什么时候保存，保存到什么粒度，保存多久，谁来清理。

checkpoint 策略可以写成一个简单规则：在语义上可能分叉、在计算上重跑很贵、在资源上还能承受时保存。三者缺一不可。只是语义上重要但保存太贵，系统会被快照拖慢；只是保存很便宜但没有分叉意义，快照会变成垃圾数据；只是重跑很贵但状态不完整，恢复出来的环境也不可信。一种简化的粒度选择逻辑如下。

⁴⁰DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback, arXiv:2605.22781.

⁴¹Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes, arXiv:2604.28138.

语义感知 checkpoint：按 OS 可见影响选择保存粒度

```
1 effect := inspectOSEffects(lastStep)
2
3 if effect.readOnly() {
4     skipCheckpoint()
5 } else if effect.onlyFileChanged() {
6     checkpointFileSystemLayer()
7 } else if effect.onlyProcessChanged() {
8     checkpointProcessState()
9 } else if effect.fileAndProcessChanged() {
10    checkpointFileSystemAndProcess()
11 }
12
13 if isBranchPoint(lastStep) && replayCostHigh(lastStep) {
14    pinCheckpointForSearch()
15 }
```

这段逻辑说明，checkpoint 不是越多越好，也不是越完整越好。状态面真正要做的是把 Agent 层的轮次语义和 OS 层的实际副作用对齐：读文件通常不需要保存，修改源码至少要保存文件系统，安装依赖或启动后台服务可能需要同时保存文件系统和进程状态。这样的选择既保证恢复正确，又避免把宿主机 I/O 消耗在无意义的快照上。

一次失败恢复可以分成两个阶段。第 i 步执行前后，平台先根据这一轮动作的副作用决定保存哪些状态：只改文件时冻结文件写层，改变运行中进程时保存进程状态，两类副作用同时出现时则把文件层和进程状态纳入同一个 checkpoint。只有当任务轮次、文件写层和进程状态彼此对齐时，这个 checkpoint 才能成为后续恢复继续执行（resume）的起点。图 92 将这一过程展开到同一条时间线上，显示第 i 步失败后系统为什么要回到最近的一致状态点，而不是只回退其中某一层。

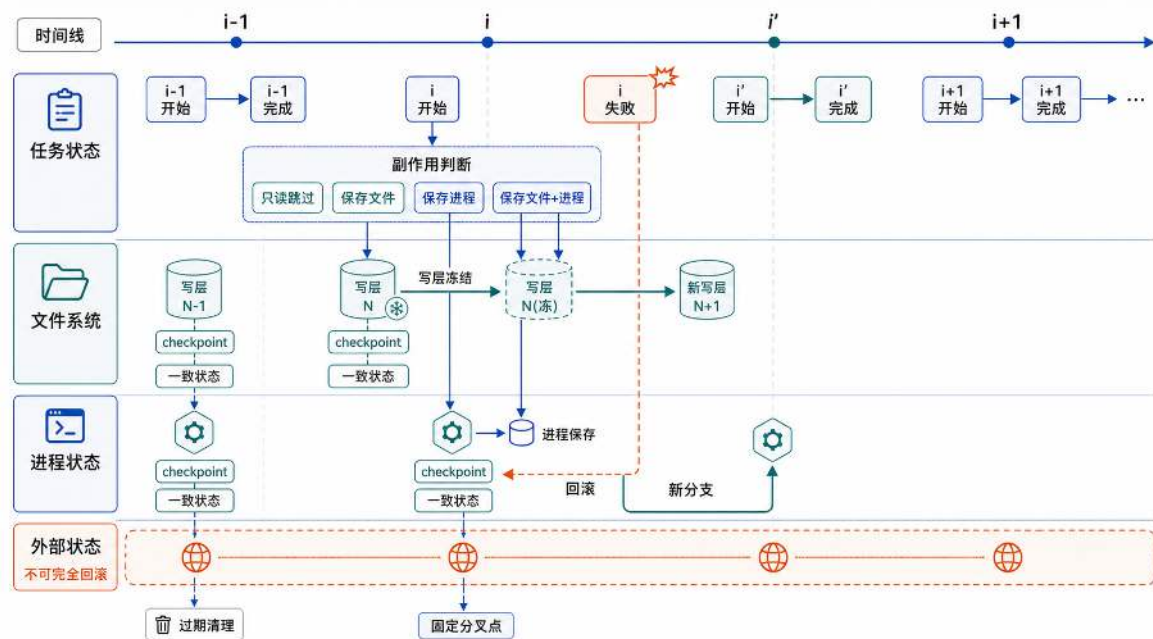


图 92: 语义感知 checkpoint 的失败恢复过程：平台在第 i 步按副作用选择保存文件层、进程状态或二者；失败后回滚到最近的一致状态点，从该状态点启动新分支 i' 。

第 i 步失败后，恢复动作本质上是在重新建立一个可继续执行的环境，而不是把某个日志指针往回拨。系统不能只把 Agent 的会话上下文退回去，也不能只把磁盘文件退回去；它必须恢复到最近一个文件系统、进程状态和任务轮次同时成立的 checkpoint，再从那里启动新分支 i' 。外部接口、时钟和随机源位于平台难以完全控制的边界外，因此它们通常不能像文件写层和进程状态那样可靠回滚，只能通过模拟、录制回放或访问限制来降低不一致风险。这样，checkpoint 才真正成为恢复继续执行的入口，而不是一份看似完整、恢复后却无法继续执行的快照。

高并发时，checkpoint 还会变成资源调度问题。几十个环境同时 dump 内存或写快照，会使宿主机 I/O 饱和。状态面需要一个主机级调度器，把 checkpoint 当作资源来排队。正在等模型返回的任务，可以把 checkpoint 工作安排在等待时间里；模型已经返回、任务正被恢复阻塞，就要提高优先级。快照还要有垃圾回收策略，过期分支、失败分支和低价值中间点不能无限保留。否则，状态面会从“提升探索效率的工具”变成新的性能瓶颈。

checkpoint 的价值不只在节省重跑时间。它也为后面的失败轨迹分析提供了可操作的入口：如果系统能把环境恢复到某个关键错误发生前，就可以从同一个中间状态重新探索。状态面保存的不只是执行现场，也是在为训练数据的再加工保留分叉点。

6.5.6 验证面：让轨迹能被复现、筛选和归因

环境能启动、能交互、能回滚之后，还必须回答最后一个云平台问题：运行过程是否可观察、可审计、可追溯。扩缩容和 Serverless 场景中，监控指标用来判断系统是否健康；在 Agent sandbox 中，日志和指标还要进一步变成训练数据的证据链。一次任务是否成功，不能只看最终分数，还要能追溯它使用了哪个镜像、哪个模型、哪个验证器，以及中间是否发生环境异常。

完成调度、交互和状态保存之后，一个大型 sandbox 平台已经具备执行能力。但训练和评测还需要最后一层保证：轨迹必须能被解释。否则，平台运行规模越大，越可能积累大量无法用于训练和评测的数据。对训练系统来说，错误归因的轨迹比没有轨迹更危险，因为它会把模型带向错误经验。

验证面承担三项职责：判断轨迹能否复现，判断轨迹能否进入训练或评测数据，以及在失败时将原因归属到模型、环境、工具或验证器。只有这三项职责得到满足，前面生产出来的轨迹才有价值。

一条可用轨迹至少要回答这些问题：这次任务从哪个任务版本开始？用了哪个环境镜像？允许哪些工具？网络是否开放？模型是哪一个版本？中间有没有从 checkpoint 恢复？验证器是哪一版？reward 是怎样计算的？失败前是否重试过？如果这些信息丢失，轨迹数量虽大，训练价值却会下降。百万条轨迹如果来自不同版本的任务和评测器，却没有记录版本对应关系，数据的统计意义就无法保证。

验证面首先要追求可复现。这里的可复现，并不要求模型每次都走出完全相同的动作序列；它要求给定任务版本、环境版本、工具权限、随机种子、模型版本和验证器版本后，工程师能重新构造出当时的评测条件。代码任务要能重新拉起仓库和依赖，桌面任务要能恢复屏幕与应用状态，网页任务要能恢复页面数据或模拟服务。没有复现能力，失败分析就缺乏可验证的基础。

这也是为什么轨迹要记录来源与派生关系。平台要知道一条轨迹从哪个任务模板开始，运行在哪个环境镜像上，由哪个模型产生动作，调用过哪些工具，最后由哪个验证器给出分数。如果一条恢复样本是从失败轨迹中分叉出来的，平台还要记录它来自哪条失败轨迹、从哪个 checkpoint 继续执行。缺少这些来源记录，轨迹就像没有实验条件的实验结果，看起来有数字，实际上很难使用。

reward 也不能只靠模型自评。代码任务要跑测试，桌面任务要检查窗口或文件状态，网页任务要验证页面结果。EvoCUA 中把任务和可执行验证器一起生成，强调的正是这一点：环境交互轨迹只有配上可执行、可复查的验证规则，才能成为训练和评测数据。更好的做法是把验证器也纳入版本管理：任务版本、环境版本、验证器版本和模型版本一起记录。这样，当某批轨迹分数异常时，工程师才能判断是模型退化、环境漂移，还是验证器改了规则。

失败归因同样重要。评测平台真正难处理的是失败原因无法明确归属：模型决策错误、浏览器响应超时、环境复位不完整、reward 脚本版本变更，这些因素可能同时存在。模型错误应该进入模型分析；环境错误应该进入基础设施告警；评测器错误应该冻结结果，等待修复后重跑。把这些混在一起，训练系统就会把基础设施噪声当成模型经验。

失败轨迹还可以进一步转化为训练资源。一次 Agent 任务最终失败，并不意味着整条轨迹上的每一步都是错误的。更常见的情况是：前面的观察、定位和部分操作都可能是合理的，真正把任务带偏的只是某一个关键步骤——在一条数十步的轨迹中，模型可能前几步的观察和操作都正确，失败仅源于某一步的错误判断或错误分支选择。

因此，验证面可以尝试从失败轨迹中定位关键错误步骤。设一条轨迹在第 n 步失败，系统通过验证器、日志和状态差异分析判断，第 i 步可能是导致失败的动作。此时，如果状态面保存了第 $i-1$ 步之前的 checkpoint，平台就可以把 sandbox 恢复到错误发生前的环境状态，再从这一

点重新引导模型选择动作。若新的分支最终通过验证，系统就得到了一条很有价值的恢复样本：它不只告诉模型“任务怎样做成”，还告诉模型“在一个复杂中间状态下，怎样从即将走错的位置恢复回来”。

这类恢复样本对 Agent 能力训练尤其重要。普通成功轨迹提供的是一条可行路径；失败后的恢复轨迹提供的是纠错经验。模型在真实环境中执行任务时，难免会遇到命令超时、测试失败、页面状态异常、文件修改走偏等情况。它需要学习的不是永远不犯错，而是在环境已经发生变化之后，仍然能够识别偏差、回到合理路径，并继续推进任务。

上述机制说明，高质量 Agent 训练会反过来要求更强的 sandbox 云基建。轨迹日志负责记录每一步发生了什么，验证器负责判断哪一条分支真正成功，checkpoint/rollback 负责把环境恢复到关键步骤之前，轨迹来源记录负责说明恢复样本来自哪条失败轨迹、从哪个 checkpoint 分叉出来。只有这些能力同时成立，失败轨迹才能从一次运行结果，变成可以复现、可以分叉、可以筛选的训练数据。

图 93 把前面这些环节串联成一条完整的派生链：验证器先在失败轨迹上定位第 i 步，状态面据此把 sandbox 回滚到第 $i-1$ 步的 checkpoint，Agent 服务从这个环境状态重新探索。这条链上有一道不能省的关卡：新分支跑完必须重新过一遍验证，只有验证通过的分支，才带着来源字段写回轨迹库。

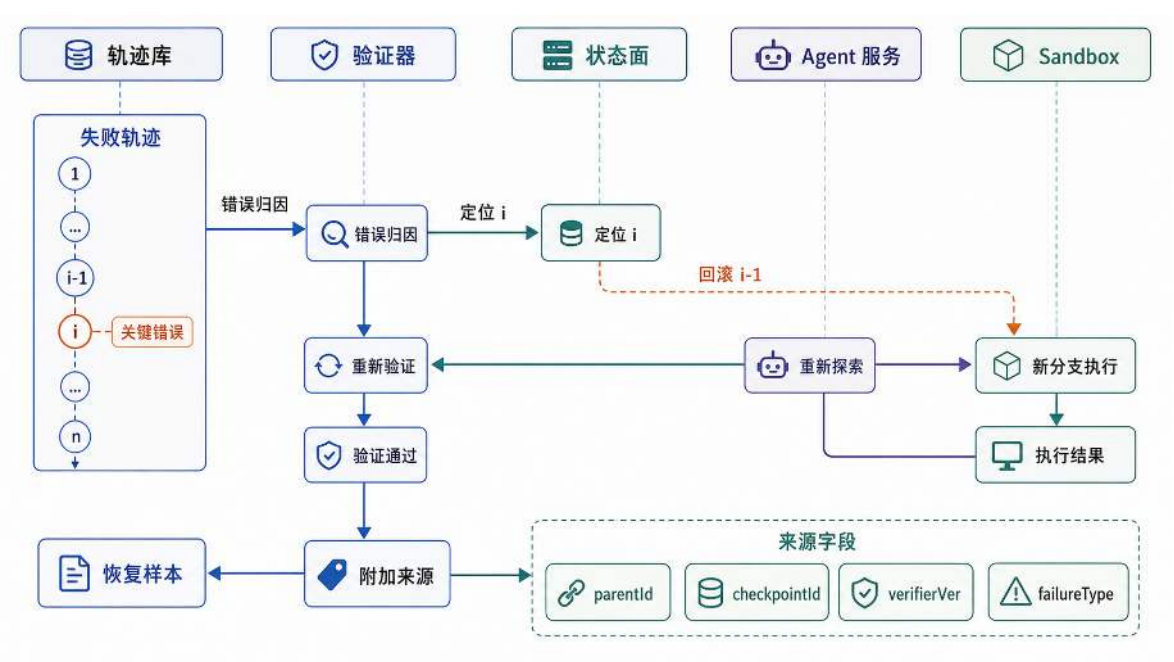


图 93: 失败轨迹可以经由错误归因、checkpoint 回滚、Agent 重新探索和验证器重新验证派生为恢复样本，并在轨迹记录中保留来源关系。

为了让上述过程产出的恢复样本能够被复现、筛选或用于训练，验证面必须在轨迹记录中写入来源字段：父轨迹、checkpoint、验证器版本和失败类型分别说明样本从哪里来、从哪个状态分叉、由哪个验证逻辑判定，以及原始失败属于哪一类。

训练数据还需要筛选。成功轨迹有价值，但失败轨迹也可能具有训练价值。一次失败可能暴露了模型不会恢复的分支，也可能只是 sandbox 网络抖动。平台要把失败分成几类：模型决策错误、工具使用错误、环境异常、验证器异常、超预算、用户任务本身有问题。只有分类清楚，失败轨迹才能用于训练模型的纠错能力，而不是把系统故障教给模型。

为了支撑这些判断，轨迹记录需要从一开始就结构化，而不是事后从日志里拼。一条轨迹的最小结构可以组织为如下记录：

TrajectoryRecord：一条可复查轨迹的最小结构

```
1 TrajectoryRecord {
2     taskVersion
3     environmentSpec
4     imageDigest
5     modelVersion
6     toolPolicy
7     steps[] {
8         observationRef
9         action
10        toolResult
11        timestamp
12        checkpointId
13    }
14    verifierVersion
15    reward
16    failureType
17 }
```

这里的 `imageDigest` 用来确认环境镜像没有漂移，`toolPolicy` 用来确认工具权限一致，`checkpointId` 把轨迹和状态面连接起来，`verifierVersion` 则保证 `reward` 的含义可追溯。这样，验证面就不再只是保存日志，而是在为后续训练、评测、审计和复现实验建立统一的轨迹来源与派生记录。

最后，验证面要把轨迹变成数据资产，而不是一次性日志。DeepSeek DSec 维护每个 sandbox 的全局有序轨迹日志，记录命令调用及其结果，用于抢占后的快速续跑、细粒度溯源和确定性重放。这个设计说明，日志在 Agent sandbox 中不只是排错材料，也是一种恢复和审计机制。轨迹可以被重放，可以被抽样审查，可以按任务类型、工具类型、失败类型、环境版本检索。这样，研究人员才能提出有意义的问题：模型是不是在安装依赖时经常走错？是不是浏览器任务里的截图延迟导致失败？是不是某个基础镜像升级后通过率下降？在这一层面上，sandbox 平台承担的职责已经超过任务执行本身，还包括为模型能力改进提供可复查的实验记录。

6.5.7 小结：Agent sandbox 的五个面向

从整个 Agent sandbox 子系统来看，存在一条清楚的设计链路：先把任务状态、资源状态和轨迹状态分清；再用控制面维持全局秩序；用执行面提供真实而可承受的环境；用交互面承接高频动作和观察；用状态面支持回滚与分叉；最后用验证面把轨迹变成可信数据。只要这条链路中任何一环不稳，问题就可能传导到训练数据的质量中。

这些设计并没有脱离云原生的基础机制。容器和 VM 提供隔离边界，OverlayFS 和 CoW 磁盘降低复制成本，HPA 和 Serverless 提供弹性调度思路，快照与恢复把冷启动和状态迁移变成可优化对象，日志和监控让失败原因有迹可循。Agent 训练与评测只是提出了一种更难的新工作负载：它不只要运行服务，还要批量运行一个个会被模型改变的环境。

因此，面向 Agent 的高并发 sandbox 是云计算问题在新工作负载下的延伸。云原生让普通应用可以被批量部署和伸缩；Agent sandbox 则让可交互环境本身也能被批量创建、隔离、复位、分叉、验证和回收。支撑这些能力的仍然是进程、文件系统、网络、调度和控制循环，只是这一次，被管理的不再只是服务副本，而是一条条正在试错的执行轨迹。

6.6 小结：从基石机制到系统工程方法

回到本章开头的问题：当系统不再只是“运行一个服务”，而是要批量创建隔离环境、接入网络、调度资源、快速启动、保存状态，并在失败后复现与验证时，云原生平台到底承担了哪些工作？学完这一章，读者应当看到，云原生不是一组产品名，而是一套围绕进程、文件系统、网络、资源、状态和日志展开的系统工程方法。

本章先拆开这些能力的基础机制。容器用 Namespace 改变进程看到的系统视图，用 Cgroups 约束资源使用，用 rootfs 构造文件系统环境，并以 OverlayFS 说明镜像层共享和容器独立写入的一种常见实现；容器网络用 veth、bridge、NAT、VXLAN 和端口映射，把隔离的网络命名空间重新接入服务系统，并在高频交互场景下面对 Overlay 封装开销。应用要接受自动伸缩，无状态服务必须做到状态外置、稳定接入和安全下线，有状态服务则必须先划分状态类型，再明确状态归属、分片方式和重平衡协议。平台内部用指标、目标、动作和校验组成反馈闭环，并让 HPA、调度器和节点扩容器按不同时间尺度分层协作；调度器先用硬约束形成可行集合，再用多目标评分选择放置位置。Serverless 把容量管理继续下沉到按需执行环境，但冷启动、预热池、快照恢复和启动路径拆分会决定它能否服务低延迟场景。

面向 Agent 的高并发 sandbox 把这些基础机制重新组合到更长的任务生命周期中。同一个反馈闭环，从弹性伸缩的副本管理延伸到 Agent 任务的尝试、保存、回滚与验证；同一套隔离与网络机制，从承载服务请求延伸到承载模型对环境的持续修改；同一种存储与日志思路，从记录系统健康延伸到记录可复现的训练数据轨迹。Agent sandbox 并没有脱离云计算主线，它只是把隔离、网络、调度、状态和可观测性推到更高强度：平台管理的不再只是服务副本，也是一条条会改变环境、会失败、会恢复、会被重新用于训练的数据轨迹。

6.6.1 未来展望：云原生的下一站

云原生技术不会停在 Kubernetes、Serverless 或某一种 sandbox 形态上。更值得关注的是，新的工作负载会不断把旧问题推到更高强度：更多实例、更短启动、更复杂状态、更长执行过程，以及更严格的复现和审计要求。沿着本章的主线看，未来的云平台至少有四个方向值得持续关注。

第一，云平台会从托管服务副本，走向托管执行轨迹 传统云平台主要管理服务副本：一个 Pod 是否健康，一个函数实例是否就绪，一个服务是否需要扩容。Agent sandbox、自动化评测、CI/CD 流水线 and 数据处理任务提出了更复杂的要求：平台不仅要知道一个环境是否在运行，还要知道一段执行过程走到了哪里，中间产生了哪些状态，能不能从某一步恢复，失败后能不能复现。也就是说，平台管理的对象正在从“运行中的实例”扩展到“可记录、可恢复、可验证的执行轨迹”。这意味着任务状态机、租约、checkpoint、日志和验证器需要成为更基础的平台能力。

第二，控制面会从阈值响应，走向半自治闭环 HPA 已经展示了控制闭环的基本形态：观察指标，计算目标，执行动作，再等待结果生效。未来的控制面仍然会遵循这个框架，但观测信息会更丰富，决策过程也会更接近半自治。平台可能结合历史流量、活动日历、实时指标和故障信号提前扩容；也可能在延迟异常时自动收集日志、隔离实例、触发重试，并把处理过程交给工程师复核。关键在于让控制面具备更好的异常归因、动作建议和结果验证能力。无论形式怎样变化，云平台仍然离不开清晰的观测、决策、执行和回滚边界。

第三，执行环境会走向分层沙箱 未来不会只有一种标准沙箱。容器启动快、密度高，适合命令行和普通服务；microVM 提供更强隔离，适合多租户和不可信代码；完整 VM 或 OS replica 保留更多系统语义，适合桌面、浏览器和复杂工具链任务；Wasm 则把隔离边界下沉到指令集与运行时层，适合短、轻、系统调用依赖少的计算单元。平台要做的不是把所有任务塞进同一种运行时，而是根据隔离强度、环境保真度、启动速度和资源成本选择合适的执行后端。第 6.1 节讲的 Namespace、Cgroups、OverlayFS，以及 Serverless 中的冷启动、预热和快照，会继续以不同组合出现在这些沙箱形态中。

第四，状态和日志会成为云平台的核心资产 过去，很多系统把状态管理留给应用，把日志主要用于排错。随着 Serverless、Agent sandbox 和自动化评测的发展，状态与日志会越来越靠近平台本身。状态需要被保存、迁移、回滚和分叉；日志需要记录任务版本、环境版本、模型版本、工具调用和验证结果。它们不再只是故障发生后的辅助材料，而是恢复执行、复现实验、筛选训练数据和审计系统行为的依据。未来的云平台如果只会启动实例，却不能解释实例里发生过什么，就很难支撑更复杂的自动化工作负载。

这些方向看起来指向不同技术，背后仍然是本章反复出现的基础问题：执行环境怎样隔离，资源怎样调度，状态怎样保存，失败怎样恢复，运行过程怎样被记录和验证。理解这些问题，比记住某个具体平台的名字更重要。

6.6.2 写给读者的寄语：不畏浮云遮望眼

作为这本教材云原生核心原理篇的收尾，我们想留下一个朴素但重要的工程信念。

今天的开源工具和云平台已经足够丰富，许多问题看起来只需要几行 YAML、几个 API 就能解决。可是，一旦系统进入真实生产环境，排队、卡顿、断连、OOM、冷启动、状态漂移和日志缺失都会重新出现。真正决定工程师能否处理这些问题的，往往是沿着系统边界向下追问的能力：进程在哪里运行，资源在哪里受限，网络路径如何转发，状态怎样保存，失败又被记录到了哪里。

这也是本章坚持拆解底层机制的原因。在计算机科学中，越是复杂的上层系统，越离不开那些看似朴素的基础构件：进程、内存、文件系统、网络 I/O、调度循环、状态恢复和日志记录。容器、Kubernetes、Serverless，乃至面向 Agent 的 sandbox 平台，都只是这些构件在不同压力下的重新组合。

希望本章能成为读者心中的一个起点。以后面对新的微服务框架、调度算法、云平台形态，甚至下一代由 AI 驱动的基础设施时，不必停在黑盒外面反复试错，而是敢于打开它，沿着隔离、网络、调度、状态和可观测性一层层看下去。只要能把复杂系统拆回这些基本问题，就已经抓住了理解它的入口。

云计算的技术形态还会继续变化，但底层问题不会消失。应用仍然要被隔离，资源仍然要被调度，网络仍然要被连通，状态仍然要被恢复，失败仍然要被记录。愿读者在之后的学习和工程实践中，始终保留这种追问底层的勇气：不止会使用工具，也能理解工具；不止能跑通系统，也能解释系统；不止在黑盒外等待答案，也能亲手打开下一个黑盒。

6.7 思考题

1. 容器 A 内部运行 `ps` 时只能看到自己的进程，但它和宿主机共享同一个 Linux 内核。请解释 PID Namespace 如何让容器看不到宿主机进程，并说明为什么共享内核使容器隔离弱于虚拟机隔离。进一步思考：如果宿主机内核存在漏洞，容器共享内核会带来什么风险？为什么虚拟机的独立内核边界更强？
2. 战斗服容器的 CPU limit 设为 2 核，某一帧的物理计算突然需要超过 2 核的计算能力。请解释 Cgroups 会如何限制 CPU 使用，玩家侧可能看到什么现象。注意这里的“2 核”通常指在一个配额周期内最多使用相当于 2 个 CPU 的累计运行时间，并不意味着进程被固定绑定到某两个物理核心（只有配合 CPU Manager、cpuset 等绑核机制时才涉及固定核心集合）。再比较 CPU throttling 与内存超限后的 OOM kill：它们分别在什么资源压力下发生，对进程生命周期有什么不同影响？
3. 容器 A (`lobby-api`) 要访问同一台宿主机上的容器 B (`redis`)。请画出数据包从 A 的 `eth0` 到 B 的 `eth0` 经过的完整路径，包括 veth、bridge 和对端 veth。然后解释为什么容器内的 `127.0.0.1` 只指向容器自己的网络命名空间，而不是宿主机或另一个容器。

4. 活动刚开始, QPS 在 30 秒内翻倍。HPA 的采样周期是 15 秒, 新 Pod 从创建到 Ready 需要 45 秒。请估算从负载上升到新容量真正生效, 系统至少会经历哪些延迟阶段。进一步思考: 最小副本数和预热池分别怎样缩短玩家排队时间, 它们又会带来什么成本?
5. 一个 Serverless 函数的冷启动耗时 1.2 秒, 其中沙箱创建约 200ms、运行时启动约 400ms、应用初始化约 600ms。请判断 MicroVM、Wasm、Snapshot 和暖池分别属于“减少工作”“复用结果”还是“预留容量”, 并说明它们主要影响哪些阶段。进一步比较: 快照按需恢复需要支付哪些补齐成本, 暖池为什么会产生持续的资源成本, Wasm 又为什么不适合强依赖完整 OS 语义的函数?
6. 某匹配系统需要扩容, 其中包含可从数据库重建的排行榜缓存、玩家 WebSocket 连接、全局匹配队列和账户余额。请分别判断它们属于哪类状态, 并说明哪些状态可以直接重建、哪些需要排空或迁移、哪些必须由协调或持久化机制保护。
7. 一个 Pod 需要 4 核 CPU、GPU 标签并要求与同类副本分散部署。请说明调度器怎样先用硬约束形成可行节点集合, 再怎样按资源均衡、故障域分散和数据本地性评分。如果当前没有可行节点, 为什么 Cluster Autoscaler 也不一定能够通过增加任意 Node 解决问题?
8. lobby-api 的平均停留时间为 200ms, 当前请求到达率为每秒 500 个。请用 Little 定律估算系统内的平均请求数。现在假设到达率突然从 500 升到每秒 1000 个, 系统仍能持续完成每秒 800 个请求, 并且不拒绝、不丢弃请求。请计算队列每秒净增加多少请求, 以及 t 秒后比负载上升前多积压多少请求; 再解释为什么系统失去稳态后, 不能继续用一个固定的平均停留时间和平均请求数描述它。进一步思考: 为什么 P99 往往会比平均值更早、更明显地恶化?
9. 一个 Agent 任务启动前需要同时获得 sandbox、GPU 配额和网络出口。平台先后完成前两项预留, 却在申请网络出口时失败。若平台直接重试整个流程, 可能出现哪些资源泄漏或重复分配问题? 请说明幂等预留、超时释放和后台对账分别怎样处理这类部分失败, 并解释为什么“检查时资源充足”不能保证后续申请一定成功。
10. 控制器故障重启后, 把同一个 Attempt 分配给了新的执行节点, 但旧节点尚未停止, 并继续提交工具调用。请说明租约版本或 fencing token 怎样阻止旧节点继续写入状态, 再说明 ToolCallId 幂等键怎样避免同一次工具调用被重复执行。为什么只使用其中一种机制仍然不够?
11. Agent 在第 i 步修改了 sandbox 内的代码文件, 同时向外部服务发送了一条消息。平台随后回滚到第 $i - 1$ 步的 checkpoint。哪些状态能够随 checkpoint 恢复, 哪些外部副作用不会自动撤销? 为了判断回滚后的新分支是否可信, 轨迹记录和验证器至少还应保留哪些版本与来源信息?

A 章节重难点知识点索引

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
2.1	分布式系统核心挑战 (视图一致性、时序不确定性、信任边界)	用于分布式数据库的 CAP 定理权衡、金融交易系统的事务顺序保障及零信任安全架构。	解决玩家屏幕间的“平行宇宙”问题，确保技能判定顺序公平，防止客户端篡改数据作弊。
2.2	网络通信与初步同步 (Socket 编程、TCP 三次握手、帧锁定/Lockstep)	实现 Web 服务器与客户端的长连接通信，或者用于传统数据库的主从同步。	建立 P2P 直连通道，通过“按步就班”的帧锁定确保两端逻辑帧序列完全一致。
2.2	网络分层与 Socket 内核缓冲	区分应用层语义、传输层连接、网络层寻址和链路层转发，理解 Read/Write 实际读写的是内核缓冲区。	解释为什么一次发送不等于一次接收，帮助读者从操作系统和网络协议层面理解联机消息的到达过程。
2.2	应用层分帧与粘包/拆包	在 TCP 字节流之上定义长度前缀、消息类型和上限检查，避免把任意一次读取误当作完整业务消息。	保证移动、攻击、心跳等指令能被稳定还原成独立消息，防止半包、粘包导致错误裁决。
2.2	WebSocket、TLS、HTTP/2、QUIC 与 HTTP/3	理解现代应用协议如何在可靠性、队头阻塞、多路复用、加密和连接复用之间取舍。	浏览器实时通信可用 WebSocket/WSS；低延迟链路需要继续判断消息语义、拥塞控制和服务器权威应由哪一层承担。
2.3	系统架构范式 (P2P 架构 vs. CS 架构、权威服务器)	在 CDN 或区块链中采用 P2P；在标准的 Web 服务 (如 RESTful API) 中采用 CS 模式。	将游戏逻辑从“多方协商”转向“独裁仲裁”，确立服务器作为游戏真相的唯一来源。
2.4.1	连接管理 (心跳检测、Session 维护、超时策略)	用于负载均衡器探测后端实例健康状态，或 Web 网关维护用户登录态 (Token)。	追踪玩家在线状态，量化网络质量，并为玩家提供断线重连 (影子状态) 的平滑体验。
2.4.2	状态仲裁 (输入校验、API 网关的权限校验与请求碰撞检测、先校验后执行)	过滤，防止 SQL 注入或非法业务逻辑请求。	对玩家的移动和攻击意图进行逻辑复核，杜绝加速、穿墙等物理层面的作弊行为。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
2.4.3	世界同步 (增量同步/Delta、快照/Snapshot、Tick Rate)	分布式缓存的数据同步，或消息队列 (Pub/Sub) 中的增量数据下发。	在有限带宽下，高频分发世界状态变更，结合客户端预测实现平滑的视觉表现。
2.4.3	客户端预测、服务器和解与实体插值	将本地即时反馈、权威状态校正和远端实体平滑绘制分开处理，是实时系统在低延迟体验与权威一致性之间的经典折中。	本地玩家先看到动作反馈，服务器快照返回后再重放未确认输入；远端玩家通过插值减少画面跳变。
2.4.3	延迟补偿与历史状态回看	在受限时间窗口内用服务器保存的权威历史状态进行判定，补偿网络传播时间，同时避免客户端任意改写历史。	射击或技能命中判定不只看“服务器当前坐标”，而是按输入序号或接收时间回看目标在权威历史中的位置。
2.5	分布式状态演进 (状态机复制/SMR、原子性校验、回滚/Rollback)	保证分布式数据库 (如 Raft 协议) 的日志一致性，以及多阶段提交 (2PC) 的事务处理。	管理对战生命周期，确保“相遇-战斗-结算”流程的原子性，在异常时恢复玩家逻辑状态。
3.1.1	并发 (Concurrency)	通过任务拆解提升系统吞吐量，是构建高并发 Web 服务、API 网关、微服务架构的基础组织方式	将玩家请求拆解为独立执行单元，实现多人同图场景下的并发处理，突破顺序执行的吞吐瓶颈
3.1.2	Goroutine (轻量级协程)	为每个 HTTP 请求、数据库连接、消息队列消费者分配独立协程，以极低的内存开销支撑百万级并发连接	为每个玩家连接或业务请求分配独立 Goroutine，实现“每连接一协程”架构，支持数万玩家同时在线
3.1.2	M:N 调度模型	将大量用户态任务复用到少量 OS 线程上，避免线程爆炸问题，提升云服务的资源利用率和弹性伸缩能力	在 I/O 密集的游戏服务器中，当 Goroutine 等待网络或数据库响应时，调度器自动切换到其他就绪任务，充分利用 CPU 资源
3.1.3	Channel (消息队列)	实现微服务间异步通信、事件驱动架构、任务队列系统，解耦服务组件并提供流量削峰能力	将网络 I/O 层与业务逻辑层解耦，通过消息传递实现玩家操作的异步处理和流量控制

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
3.1.3	Select 多路复用	实现超时控制、健康检查、优雅关闭等服务治理功能，同时监听多个事件源以提升系统响应性	实现玩家心跳检测、连接超时断开、多源消息处理，防止僵尸连接占用服务器资源
3.1.4	I/O 模型四象限（同步/异步、阻塞/非阻塞）	判断调用方是否等待结果、执行流是否被挂起，是理解高并发网络服务入口设计的基础。	区分阻塞读 socket、非阻塞轮询、异步回调和等待异步结果，避免把网络等待拖进世界主循环。
3.1.4	epoll、I/O 多路复用与 Reactor 模式	让内核集中管理大量文件描述符的就绪事件，程序按事件分发处理器，降低大量连接下的扫描和等待成本。	网关服务可以承载大量长连接，多数连接空闲时不需要为每个连接占用一条真正忙等的执行路径。
3.2.1	数据竞争（Race Condition）	云存储、分布式数据库、缓存系统中的并发写入冲突检测，保证多租户环境下的数据一致性	检测并防止多玩家同时拾取唯一物品、同时修改共享状态等并发冲突，维护游戏规则的确定性
3.2.3	互斥锁（Mutex）	保护共享资源如全局配置、计数器、状态机的并发访问，确保关键操作的原子性和顺序一致性	将”检查-修改”等关键游戏规则操作收敛为原子区，保证物品唯一性、战斗裁决正确性等业务约束
3.2.4	锁粒度设计	在分布式系统中权衡锁的范围，全局锁用于配置中心，细粒度锁用于分片数据库，平衡一致性与性能	全局锁用于服务器配置，区域锁用于地图分块，对象锁用于玩家数据，最大化游戏世界的并发操作能力
3.2.4	happens-before 与内存可见性	用锁、channel、条件变量等同步事件建立可推理的先后关系，避免并发执行流读到旧值或乱序结果。	解释为什么“代码里已经写了”不等于另一条 goroutine 一定能立刻看见，帮助审查共享状态更新是否可靠。
3.2.4	CAS 与原子操作	对计数器、状态位等小范围共享变量提供无锁更新方式，但需要理解重试、自旋成本和 ABA 风险。	适合在线人数计数、简单状态标记等局部更新；复杂规则仍应交给锁、单写者事件循环或事务边界。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
3.3.1	连接池 (Connection Pool)	复用数据库连接、缓存连接、消息队列连接, 降低建连开销, 防止端口耗尽, 是云服务高并发的基础设施	复用与数据库、Redis 的 TCP 连接, 消除频繁握手挥手的性能损耗, 提升单机服务器的查询吞吐量
3.3.1	资源池化 (Resource Pooling)	云计算五大特征之一, 将虚拟机、存储、网络等资源抽象为共享池, 按需分配, 提升资源利用率	通过连接池和对象池实现资源的借出-归还循环, 避免反复创建销毁, 体现云原生的资源管理理念
3.3.2	对象池 (sync.Pool)	缓存 HTTP 请求缓冲区、JSON 编解码器等高频临时对象, 降低 GC 压力, 提升微服务的响应稳定性	复用网络消息缓冲区、协议解析器等短生命周期对象, 减少内存分配频率, 降低游戏服务器的延迟抖动
3.3.3	Redis 缓存 (热数据)	缓存用户会话、API 响应、实时排行榜等高频访问数据, 为 Web 应用提供亚毫秒级读写性能	存储玩家在线状态、实时位置、战斗状态等高频更新数据, 保证游戏交互的低延迟响应
3.3.3	PostgreSQL (冷数据)	持久化用户账号、订单历史、审计日志等关键业务数据, 提供 ACID 事务保证和长期可靠存储	存储玩家注册信息、历史成就、装备配置等低频访问数据, 保证数据持久性和可恢复性
3.3.3	写穿 (Write Through)	缓存更新时同步写入后端数据库, 用于金融交易、订单支付等强一致性场景	金币交易、装备绑定等关键资产操作同步写入 Redis 和数据库, 防止数据不一致导致经济损失
3.3.3	写回 (Write Back)	先写缓存后异步刷盘, 用于日志聚合、指标采集等高吞吐低延迟场景, 允许短时数据丢失	玩家位置、技能冷却等临时状态先写 Redis, 定期批量刷入数据库, 牺牲部分持久性换取性能
3.3.3	失效更新 (Cache Aside)	更新数据库后删除缓存, 适合配置管理、内容发布等读多写少场景, 降低缓存维护复杂度	物品配置、技能描述等静态数据更新时删除 Redis 缓存, 下次读取时从数据库重新加载并回填

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
3.3.3	LRU/LFU 与缓存淘汰	当缓存容量有限时，用最近访问或访问频率决定哪些键先被移出，避免缓存层无限增长。	在线状态、排行榜、配置数据的访问模式不同，淘汰策略和 TTL 设计也应不同。
3.3.3	缓存穿透、击穿与雪崩	识别不存在键反复回源、热点键过期集中回源、大量键同时失效三类典型缓存故障，并通过空值缓存、互斥回源、TTL 抖动等方式缓解。	防止恶意玩家 ID、热门排行榜键或活动配置批量过期把数据库瞬间打满。
3.3.4	QPS 吞吐量	衡量云服务每秒处理请求数，用于容量规划、自动扩缩容决策、SLA 性能保证	测量游戏服务器每秒处理的移动、战斗、同步请求数，评估单机承载能力和优化效果
3.3.4	P95/P99 延迟	云服务 SLA 中的关键指标，反映用户真实体验，用于识别长尾延迟和系统稳定性问题	评估游戏操作响应延迟，即使平均延迟低，P99 延迟过高仍会导致部分玩家感知卡顿
3.3.4	性能基线与瓶颈识别	通过可观测性平台监控 CPU、内存、网络等资源占用，科学判断何时进行纵向扩展或横向扩展	通过压测确定单机性能极限，当 CPU 达 90% 时判断纵向优化收益递减，决策转向分布式架构
4.1.1	单点瓶颈与风险集中	单一数据库的硬件资源天花板和故障全局影响，是分布式架构演进的根本动机	单服承载 2000-3000 玩家时性能下降，硬件故障导致全服不可用，迫使数据层横向扩展
4.1.2	分片键选择	决定数据分配规则的关键字段，需要唯一性、不可变性和查询关联性	选择玩家 ID 作为分片键，因其唯一固定且大部分查询基于玩家 ID
4.1.2	哈希分片	通过哈希函数打散数据分布，负载均衡但扩容困难	playerID % 4 将连续 ID 打散到 4 个分片，扩展到 5 个分片时需重新分配几乎所有数据

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
4.1.2	一致性哈希	通过哈希环映射减少扩容时的数据迁移量，支持平滑扩展	新增分片时仅 20% 数据需迁移，通过虚拟节点提升负载均衡效果
4.1.2	CAP 与 PACELC	CAP 说明网络分区下需要在一致性与可用性之间取舍；PACELC 进一步提醒即使没有分区，也要在延迟与一致性之间权衡。	跨服资产交接更偏强一致；排行榜刷新可以接受短暂滞后，用可用性与低延迟换取更好的体验。
4.1.4	跨分片查询	需要聚合多个分片数据的查询，性能开销大且实现复杂	全服排行榜需向所有分片查询 Top100，应用层合并排序，建议改用异步汇总到缓存
4.1.4	两阶段提交 (2PC)	通过准备和提交两个阶段保证分布式事务原子性，但增加延迟和锁时间	跨分片更新战绩时，协调者向分片发送准备请求，全部就绪后提交，否则回滚
4.1.4	避免跨分片事务策略	通过异步化、数据模型重组或容忍最终一致性规避分布式事务复杂性	战绩更新改为消息队列异步处理，或将战绩数据独立到专用服务
4.1.5	主从复制	主库处理写操作，从库同步数据并分担读操作，提升查询吞吐和容错能力	主库写入战绩，从库通过 binlog 同步，查询请求分散到多个从库，主库故障时从库提升
4.2.2	轮询与最少连接策略	轮询按固定顺序分配，最少连接优先分配到负载最低的服务器	轮询无法感知实时负载，最少连接通过心跳收集在线人数动态分配
4.2.2	加权负载评估	综合多个指标计算负载值，比单一指标更准确反映服务器压力	$\text{负载值} = \text{玩家数} \times 0.2 + \text{CPU} \times 0.3 + \text{内存} \times 0.2 + \text{响应时间} \times 0.2 + \text{战斗数} \times 0.1$
4.2.3	硬件权重与动态调整	根据服务器配置设置权重，并根据实时表现微调权重	8 核权重 1、16 核权重 2，综合负载持续偏高时逐步降低权重减少分配
4.2.4	健康检查与故障隔离	定期探测服务器可用性，故障时自动隔离并在恢复后分阶段恢复流量	连续三次探测失败标记为不可用，暂停分配，恢复后只分配少量玩家观察稳定性

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
4.2.5	运行时动态迁移	将过载服务器上的空闲玩家迁移到空闲服务器，主动缓解负载不均	选择挂机 10 分钟以上的玩家，保存状态传输到目标服务器，通知重连
4.3.2	协调服务器架构	引入专门服务器维护全局状态和执行权威裁决，各应用服务器作为代理	擂台协调服务器维护全局报名列表和战绩，游戏服务器转发请求并执行指令
4.3.3	分页查询与筛选	避免一次返回全部数据，通过条件筛选和分页减少传输量	只返回战力 ± 5000 范围内的对手，每次 50 条记录，滚动加载更多
4.3.4	唯一真相源	所有权威裁决由单一协调点做出，避免多个服务器各自裁决导致的不一致	只有协调服务器判定胜负，应用服务器不独立裁决，防止双重历史问题
4.3.4	双重历史问题	不同服务器对同一事件有不同记录，破坏一致性和公平性	若各服务器独立裁决，服务器 A 认为甲获胜，服务器 B 认为乙获胜，产生矛盾
4.3.5	协调服务器高可用	通过主从热备或分布式共识保证协调服务器容错能力	部署一主一从，主服务器崩溃时从服务器接管，或使用 Raft 集群自动选举 Leader
4.3.5	Quorum（法定人数）与脑裂防护	共识系统用多数派交集保证同一时刻只有一个合法写入方向，避免网络分区下出现两个同时生效的 Leader。	跨服协调服务在分区时不能让两个中心同时裁决同一场比赛，否则会产生双重历史和资产重复写入。
5.1.1	环境隐性依赖与“换机就崩”	识别运行时对库版本、内核接口、权限与时区等隐性依赖，是构建可复现交付物与自动化运维的前提	高峰扩容或故障恢复时，避免“新机器拉起服务失败/行为不同”，降低值班排障成本
5.1.2	手工安装、虚拟机与容器（取舍）	在隔离强度、启动速度与资源效率之间做权衡：虚拟机偏强隔离，容器偏高密度与快速交付	游戏服更看重快速扩容与一致性交付；对高风险组件可用更强隔离作为补充

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
5.2.1	镜像/容器/数据卷：模板、实例与持久化边界	将“不可变交付物”（镜像）与“可变运行实例”（容器）分离；用数据卷承接有状态数据，支持滚更与重建	多副本大厅/网关共享镜像层；数据库等有状态组件必须挂卷，避免重建导致数据丢失
5.2.2	Namespace 与 Cgroups：视图隔离与资源上限	支撑多租户隔离与资源配额，是容器密度、按量计费与“吵闹邻居”治理的底座	让不同区域/不同对局实例互不干扰；限制异常战斗逻辑的 CPU/内存占用，减少全机抖动
5.2.3	容器网络视角与端口转接（127.0.0.1 陷阱）	在虚拟网络内以“名字或 IP:Port”寻址，并通过端口转接暴露公网入口，是云原生服务互联的最小直觉	gateway 对外只暴露一个端口；服务间通过 lobby-api:8080 等名字寻址，避免把依赖写成容器内的 127.0.0.1
5.2.4	Dockerfile 分层与多阶段构建	用可缓存的分层构建把“编译环境”与“运行环境”解耦，减小镜像体积并加速构建/分发	用一份蓝图构建 lobby/battle 等多个入口，缩短热修发布与扩容拉取时间
5.2.5	Registry 分发与 Digest 版本锁定	通过镜像仓库进行一致性交付；用摘要锁定版本避免 Tag 漂移，便于回滚与审计	多台云服务器拉取同一摘要，避免“同名版本”实际内容不同导致线上行为不一致
5.2.6	单机最小闭环（入口层 + 后端 + 依赖）	先在单机上验证“入口可达 + 依赖连通”的最小系统，再将同一拓扑推广到编排/集群，降低排障维度	用 gateway 串起大厅、战斗、缓存和数据库，快速复现“登录转圈/超时”并定位链路断点
5.2.6	Twelve-Factor App	将代码、依赖、配置、日志和进程生命周期分清边界，使同一份应用制品可以在开发、测试和生产环境中复用。	游戏服务镜像不内置环境差异；数据库地址、功能开关、日志等级等由部署环境注入。
5.3.1	Compose：把终端历史提升为“期望拓扑”	用声明式文件描述服务依赖、环境变量、端口与卷，使部署具备可复现与可回滚的属性	一键启动一套“可运行大区”用于开发/联调/回归，避免靠记忆拼命令

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
5.3.2	Compose 服务网络与“名字即接口”	内置 DNS 让调用方只依赖服务名，屏蔽容器 IP 的重启变化，降低配置传播成本	lobby-api、redis 等以服务名互联；容器重启换 IP 不影响上游寻址
5.3.3	Compose 的边界：单机故障域与非持续调谐	Compose 只负责把单机状态拉到目标附近，不具备跨机器调度、持续补位与滚更闭环	上线后需要多机容错与自动补位时，必须引入集群编排（如 Kubernetes）
5.4.1	多机本质难题：实例会变（副本/位置/IP/成员）	当实例集合不断变化时，需要平台统一解决调度、故障恢复、滚动更新与服务发现	活动高峰临时加机器、节点故障、热修发布同时发生时，系统仍应保持玩家可用
5.4.2	平台维持的三类目标：入口稳定、规模稳定、升级可控	把稳定性目标拆成可长期检查的约束，再由不同控制器分别托管，避免人为流程失控	玩家侧表现为：少量失败可自愈、升级不集中掉线、容量波动更可预期
5.4.3	Node/Pod/Deployment/Service：对象分工	Node 提供资源底座；Pod 是最小调度与替换单元；Deployment 维持副本与滚更；Service 提供稳定入口	gateway 入口稳定、lobby-api 可补位可滚更；上游不追着 Pod IP 改配置
5.4.3	CNI 与 Pod 网络	Service 解决稳定入口和成员集合，CNI 负责 Pod IP、跨节点连通、封装和路由，是 Kubernetes 网络模型的底层连接器。	大厅服、战斗服、缓存和数据库可能分布在不同节点，仍应像在同一集群网络中一样互通。
5.4.4	声明式与调谐循环 (Reconciliation)	以期望状态为中心持续纠偏，把“中途失败”转化为“偏离目标”，使运维动作可幂等收敛	热修发布一半失败时，避免出现版本状态不一致、流量半切换等不可解释状态；平台持续拉回可用集合
5.4.4	ConfigMap、Secret 与运行时配置注入	将普通配置与敏感配置从镜像中拆出来，由平台在运行时注入，配合权限、审计和轮换机制管理。	同一份 lobby-api 镜像可以用于测试服和正式服；数据库口令、证书和访问令牌不写进镜像。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
5.4.4	requests 与 limits	用 requests 告诉调度器最低资源需求，用 limits 限制运行时最高资源占用，是资源调度与 Cgroups 限额的连接点。	避免轻任务占用昂贵节点，也避免异常战斗服挤占同机其他对局的 CPU 和内存。
5.4.5	Readiness 闸门：启动成功不等于可接流量	用就绪探针把启动阶段的不确定性挡在入口后面，并配合滚动替换控制升级节奏	新副本未连上数据库/缓存时不接玩家请求，避免上线初期“偶发超时/偶发卡死”
5.4.6	调度与健康检查：放得下、放得对、坏了能识别	调度决定副本落点与资源边界；健康检查决定流量是否应进入后端集合，是弹性闭环的前置条件	即使扩了副本也可能因放置不当仍卡顿；健康检查与就绪闸门减少“挂着但不可用”的假健康
5.4.7	游戏服务特殊挑战：有状态、粘性与排水边界	平台擅长补位但不懂业务语义，需要业务提供“何时可中断/如何迁移”的退出边界	战斗服对局中途被替换会导致一批对局中断；缩容与滚更必须配合排水/会话外置等设计
5.5.1	固定容量遇上波动流量：体验与成本矛盾	业务负载潮汐导致“按峰值准备浪费/按均值准备排队”，弹性是结构性需求	周五活动开场排队、工作日资源空转；需要把容量随流量调整成可控机制
5.5.2	弹性闭环：观测-决策-执行（时延与抖动）	伸缩是控制系统问题：指标噪声、执行时延与冷却策略决定是否振荡	以匹配队列、登录超时率等信号触发扩容；需要平滑与限速避免“扩了又缩”
5.5.2	可观测性三支柱（Metrics、Logs、Traces）	指标用于发现趋势，日志用于还原事件，链路追踪用于定位跨服务调用中的延迟和错误传播。	登录、匹配、战斗和数据库链路一旦变慢，系统需要知道是哪个服务、哪个调用阶段、哪类请求出了问题。
5.5.2	SLI/SLO/SLA 与错误预算	用可测量指标定义目标水位，再用错误预算平衡可靠性与迭代速度，是弹性和运维决策的依据。	例如“99.9% 登录请求 300ms 内完成”可以转化为扩容、限流、回滚和暂停发布的明确信号。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
5.5.2	Little 定律与排队积压	稳态下系统内平均请求数等于到达率乘以平均停留时间，帮助从等待时间目标反推并行处理能力。	当登录请求到达率突然翻倍而处理能力不变时，队列和尾延迟会迅速恶化，玩家首先感受到排队。
5.5.3	HPA：Pod 级水平自动扩缩	根据指标计算目标副本数并持续纠偏，适合无状态或可快速替换的工作负载	大厅服/网关随请求量增加副本，避免活动峰值把单实例打满导致超时
5.5.4	缩容边界与排水：撤掉容量更危险	缩容会主动中断实例，必须处理连接排空、任务迁移与最小可用副本，避免抖动放大	长连接链路缩容若无排水会出现一批玩家集中断线重连；对局服务更易引发中断事故
5.5.5	Cluster Autoscaler：Node 级扩缩	当 Pod 放不下时扩物理容量（加节点），空闲时缩回去，补齐弹性闭环的底座边界	活动开场 HPA 拉副本但节点不够时自动加机器；低谷期回收节点节省成本
5.5.6	成本经济学：基线容量、暖池与峰值预热	用基线容量兜底响应，用暖池/预热缩短扩容时延，在成本与体验之间做结构化取舍	保证活动开场不排队，同时避免全天候按峰值常驻；对启动慢的战斗服尤其关键
5.5.6	为失败而设计 (Design for Failure)	将副本冗余、故障隔离、自动补位、限流和降级视为常态机制，而不是故障发生后的临时补救。	活动高峰时即使某个 Pod、节点或下游依赖出问题，也应尽量把影响限制在局部并保留核心玩法入口。
5.6.1	Serverless：从常驻服务到按需执行	以事件/请求驱动拉起执行实例，按调用计费；平台可能重试，因此业务需幂等	签到、邮件、排行榜结算等低频任务按需执行；奖励发放需“只发一次”的幂等保护
5.6.2	适用性分析：哪些链路能归零、哪些必须常驻	以连接驻留、状态可迁移与尾延迟容忍度为判断轴，决定运行形态	实时对战/网关需常驻；异步任务与事件驱动外围可归零以降低运维负担
5.6.3	冷启动：用首次延迟换常驻成本	函数实例创建与依赖加载会引入首次延迟，需要通过预热、并发上限与混合架构缓解	活动首次触发可能慢一拍；可对关键链路保留常驻实例，对外围任务享受按需弹性

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
6.1.1	Namespaces（命名空间）视图隔离	改变进程的系统视图，实现进程号、网络栈、挂载树等系统资源的逻辑切分，是多租户隔离的基础。	确保同一台物理机上运行的多个独立游戏服（如不同区域的战斗进程）互相看不见，避免进程误杀或端口冲突。
6.1.2	Cgroups（控制组）资源限额	为进程组限定 CPU 时间预算和内存上限，是云计算按量计费 and 防止“吵闹的邻居”的底层内核机制。	精确控制单局游戏或某个地图实例的最大算力消耗，防止某局战斗代码死循环拖垮整台服务器上的其他对局。
6.1.3	rootfs 与 OverlayFS（联合文件系统）	提供只读的基础镜像层和专属的可写容器层，利用写时复制（CoW）大幅节省磁盘空间与启动时间。	在频繁开服或热更时，成百上千个游戏实例共享同一套庞大的底层运行环境，仅将玩家产生的独立日志和热数据写入独立层。
6.1.4	容器安全边界与共享内核隐患	揭示容器隔离并非虚拟机级别的硬件隔离，恶意代码可能通过共享的宿主机内核漏洞实施“容器逃逸”。	警惕玩家上传的 UGC 自定义脚本或恶意 Mod 穿透容器隔离层，危及云端游戏服务器的物理节点安全。
6.2.1	veth 对（虚拟以太网对）	像虚拟网线一样连接不同的网络命名空间，是容器突破内向隔离、与宿主机进行数据包交互的出入口。	为新启动的游戏容器插入第一根“网线”，使其能将同步数据帧发出自己封闭的网络世界。
6.2.2	bridge（软件网桥）二层组网	充当内核中的虚拟交换机，通过 MAC 学习将多根 veth 网线汇聚，在宿主机内部搭出虚拟局域网。	让部署在同一物理节点上的大厅服、匹配服、聊天服通过内部局域网实现高速、低延迟的直连通信。
6.2.3	容器出网：网关与 SNAT（源地址转换）	宿主机充当路由器，利用连接跟踪将容器私网 IP 伪装成物理机 IP，解决私网不可路由的问题。	允许内部游戏容器主动向外平台（如 Steam、微信接口）发起鉴权、防沉迷验证或充值回调请求。
6.2.4	容器入网：DNAT（目的地址转换）与端口映射	拦截打向宿主机的外部网络请求，根据端口转接规则将其精准投递到特定容器的私网 IP 上。	玩家客户端始终只连接单一的公网 IP 和对外端口，由底层机制隐式转接给集群内部真实的战斗网关容器。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
6.2.5	VXLAN 与跨节点覆盖网络	在原始数据包外再封装一层外层包，让不同物理节点上的容器像处在同一虚拟二层网络中。	大厅服和战斗服即使落在不同节点，也能通过 Pod 网络互通；物理网络只需转发节点之间的外层包。
6.3.1	无状态服务的伸缩条件	状态外置、幂等性、稳定入口、服务发现、就绪检查和优雅下线共同保证实例可以被安全替换。	大厅接口扩容后由 Service 接入流量，缩容前先排空存量请求，重复投递不会造成重复发奖。
6.3.2	有状态服务的状态分类与重平衡	按丢失后果和协调范围区分可重建缓存、连接与会话状态、共享协调状态和权威持久状态；伸缩时先复制和追平数据，再切换归属。	战斗服新增实例只能承接新对局；已有对局若需迁移，必须先完成状态复制、增量追平和路由切换。
6.3.3	应用与平台的职责分工	应用定义状态语义、幂等和迁移协议，平台负责容量计算、实例生命周期、服务发现和资源放置，专用控制器执行数据复制与重平衡。	游戏服务定义对局何时可迁移，平台提供调度和生命周期接口，房间控制器按照业务协议完成交接。
6.4.2	弹性控制闭环	控制系统以指标、目标、动作和校验构成闭环，并通过容忍区间、稳定窗口和速率限制抑制执行时延与振荡。	监控匹配队列长度或玩家等待时间，过滤瞬时抖动，并在扩容后继续验证积压是否下降。
6.4.5	弹性的空间局限 (Amdahl 定律)	无法水平扩展的单点组件会限制整体吞吐量上限，控制器只能调整已经具备并行扩展条件的部分。	即使大厅查询服持续增加副本，底层单分片数据库的写锁仍可能成为整体瓶颈。
6.4.6	反馈式分层控制	HPA、调度器和节点扩容器分别处理副本数量、Pod 放置和基础设施供给，慢路径与快路径通过对象状态逐层交接。	HPA 先增加大厅服务副本；现有节点放不下时，节点扩容器再评估并申请满足约束的新机器。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
6.4.7	多目标调度	先用资源、标签和拓扑等硬约束形成可行节点集合，再按资源均衡、故障域分散和数据本地性等软目标评分。	GPU 任务先过滤出带有相应硬件的节点，再根据数据位置和副本分散要求选择最终位置。
6.4.8	Serverless 的事件驱动与冷启动	将调度粒度细化到按需执行环境，并通过 MicroVM、快照、预热池或 Wasm 缩短请求到达后的准备路径。	签到和异步结算等低频任务可以按事件启动，但需在首次延迟与常驻预热成本之间取舍。
6.4.4	Wasm、Snapshot 与冷启动路径拆解	将冷启动拆成镜像准备、文件系统、网络、沙箱、运行时和应用初始化等阶段，再用不同技术压缩不同阶段。	外围短任务可用轻量运行时和快照恢复减少首次延迟；强依赖完整 OS 语义的战斗链路仍更适合常驻服务。
6.4.4	强状态业务的 Serverless 禁区	无状态要求和网络连接的短命性，对依赖常驻对局、高频物理帧循环的强状态架构形成物理惩罚。	动作游戏/FPS 游戏的战斗服需要毫秒级的内存状态共享与极低延迟，无法承受 Serverless 必须去外部数据库读写状态的 I/O 损耗。
6.5	Agent sandbox：环境交互轨迹	将一次任务执行记录为观察、动作、工具调用、环境状态和验证结果组成的轨迹，使训练和评测数据可复现、可筛选。	这类压力场景把容器、调度、快照、日志和验证重新压到一起，帮助理解云平台如何承载大规模可执行环境。
6.5	控制面：准入、租约、幂等与生命周期	统一管理任务是否进入系统、拿到哪类环境、占用多少预算、何时超时和如何回收，避免大量半完成状态失控。	大量自动化任务同时运行时，控制面决定谁排队、谁获得 sandbox、失败后是否重试以及资源何时释放。
6.5	执行面：容器、虚拟机、镜像与运行时	根据隔离强度、启动速度、环境保真度和资源密度选择容器、microVM、完整 VM 或 Wasm 等执行后端。	代码任务可能只需 Linux 容器；浏览器或桌面任务需要更完整的系统语义和更高成本的运行环境。

续下页

表 9 – 续表

章节位置	知识点	云计算系统中的用途	游戏系统中的用途
6.5	交互面：网关、高吞吐 I/O 与反压	在模型服务、Agent 服务和环境服务之间处理高频请求、超时、限流和日志写入压力，防止下游拥塞传染到全系统。	如果模型预算、环境租约或轨迹日志任一环节拥塞，平台应降低新任务准入，而不是继续制造无法复现的失败轨迹。
6.5	状态面：checkpoint、rollback 与错误轨迹修复	将文件系统、进程状态和任务语义一起纳入可恢复边界，使失败前的关键状态能够稳定回放和分叉探索。	当一次任务在第 i 步走错时，平台可以恢复到第 $i - 1$ 步继续引导执行，生成“从错误中恢复”的高价值训练数据。
6.5	验证面：日志、可观测性与轨迹可追溯	记录任务版本、环境版本、工具调用、截图、命令输出、文件变化和验证结果，让执行过程可以审计和复现。	最终判断不只看成功或失败，还要能定位失败发生在哪一步、由什么环境状态触发、是否适合进入训练集。

B 配套实验与开源代码

本书配套设计了四组课程实验，仓库地址为 <https://gitee.com/hnu-cloudcomputing/cloud-compute-book-code>，其中包含程序骨架、实验说明和自动评分工具。四组实验都使用 Go 语言编写，围绕同一个文字游戏逐步扩大系统规模，读者不需要从零搭建项目，只需在预留的 TODO 位置填入核心逻辑，再通过自动化脚本验证得分。

每组实验大致对应书中一章或两章的内容：实验一对应第 1 章的网络通信与客户端/服务器架构；实验二对应第 2 章的并发与共享状态；实验三对应第 3 章和第 4 章的分布式协同与一致性；实验四对应第 5 章和第 6 章的容器编排与弹性伸缩。读者可以在学完对应章节后进入对应实验，把正文中的机制落到可运行的程序上，再通过观察运行结果反过来加深对概念的理解。

仓库中 Lab1 到 Lab4 四个目录分别对应四组实验，每个目录下都有三个部分：README.md 给出实验目标、任务分解和评分标准；student/ 是学生填写目录，大部分代码已经写好，只在关键位置留有 TODO；test/ 提供自动测试脚本，运行后会输出各用例的通过情况和对应得分。

四组实验的推进线索是同一个游戏系统的规模扩张。实验一先建立两个客户端与一个服务器之间的网络闭环；实验二把双人对战扩展为多人实时世界，处理同一进程内的并发访问；实验三进一步拆出多张地图和多个逻辑节点，引入跨节点路由、事务与故障恢复；实验四把这套分布式游戏部署到 Kubernetes 集群，在正确性、可用性、弹性和资源成本之间反复调优。它们共同展示了一个在线系统从能够通信，到能够并发运行、跨节点协作并在云平台上持续服务的演进过程。

B.1 实验一：双人对战的网络闭环

实验一构造一个最小但完整的客户端/服务器系统。两个玩家分别运行客户端，通过 TCP 连接同一个游戏服务器。客户端负责读取玩家的移动、攻击和使用药水等操作，并把这些操作编码成消息发送给服务器；服务器为玩家分配身份，维护坐标、生命值和回合状态，检查移动是否越界、攻击距离是否合法，再把更新后的状态发送给双方。

这个实验包含三部分内容。第一部分是 TCP 连接的建立：服务器监听端口并接受连接，客户端主动连接服务器。第二部分是应用层消息传输：程序使用 JSON 表示消息，并在一条持续存在的字节流上依次编码和解析多条消息。第三部分是服务器权威裁决：客户端提交的是“向哪个方向移动”或“发起攻击”这样的操作意图，真正的坐标变化、伤害计算和胜负判断都由服务器完成。

完成后的程序能够让两个客户端通过服务器进行一场完整对战。读者可以由此观察到，网络连通只解决了字节如何到达；在线系统还必须定义消息怎样区分、状态由谁保存以及结果由谁决定。

实验一以填空形式呈现，学生需要在 TCP 监听与连接、JSON 消息收发、移动与攻击判定三个位置填入核心代码。测试脚本会自动启动服务器和两个客户端，依次验证连接握手、边界检查、攻击距离和断线处理等用例，读者可以根据失败定位快速回溯具体机制。后续实验将在这个最小闭环上继续增加并发和分布式协作能力。

B.2 实验二：多人世界中的并发控制

实验二把双人回合制对战扩展为多人实时开放世界。玩家可以随时加入或离开，服务器为每个连接启动独立的 Goroutine，并周期性地向所有在线玩家广播世界快照。客户端同样需要并发处理：主执行流阻塞在键盘输入上，另一个 Goroutine 负责接收服务器推送，两者共同维护屏幕上的角色位置与事件记录。只有这样，玩家才能在操作自己角色的同时，实时看到其他玩家的移动、攻击、死亡与复活。

多人同时活动后，所有连接处理逻辑都会访问同一份世界状态。实验围绕这份共享状态设计并发安全接口：加入玩家、离开世界、移动和攻击会修改玩家集合，需要互斥写入；生成广播快照只读取状态，可以使用读锁允许多个读取同时进行。玩家死亡后的延迟复活还会启动新的 Goroutine，因此程序必须明确当前操作何时释放写锁，保证复活逻辑随后能够重新取得锁，避免后台任务长期等待甚至形成死锁。

这个实验呈现了“功能能够运行”和“并发执行仍然正确”的区别。程序可能在少量玩家下表现正常，却在多个连接同时读写映射时出现数据竞争、状态覆盖或死锁。通过多人交互、周期广播和 Go 内置的 race detector 竞争检测，读者可以观察 Goroutine 如何组织并行工作，也可以理解为什么共享状态必须通过明确的临界区和并发安全接口加以保护。

B.3 实验三：多地图与多节点协同

实验三把单地图、单节点的世界扩展为由三张地图和三个逻辑服务节点组成的分布式战场。每张地图独立维护玩家、NPC、宝物和地图版本，并拥有负责读写的主节点与保存副本的节点。玩家可以在地图之间切换，系统必须把操作路由到当前地图的主节点，同时维护玩家会话中的地图和节点位置。

地图放置与节点变化构成了实验的第一条主线。系统使用一致性哈希为地图选择主节点和副本节点，并在成员变化时生成迁移计划；Gossip 成员表传播节点的存活、可疑和失效状态。当某个节点失效后，系统需要提升仍然可用的副本，修正在线玩家的路由，并把新的地图归属写入 Raft 元数据日志。归属变更只有在多数节点复制后才能提交，从而避免不同节点同时把自己视为主节点。

跨节点操作构成了第二条主线。不同地图上的玩家转移战利品时，转出与转入必须同时成功或同时撤销，实验用两阶段提交组织这一过程。世界 Boss 的生命值则由所有地图共同观察和修改，用来呈现跨分片共享状态。与此同时，账号资料与在线会话分别作为冷数据和热数据保存，地图检查点会写入存储并同步到副本，为节点故障后的状态恢复提供起点。

这些机制不是彼此独立的算法练习。一次节点故障会同时影响成员状态、地图归属、请求路由和状态恢复；一次跨节点交易也会同时涉及玩家位置、参与节点和提交结果。实验还提供了一个独立的管理命令行工具，读者可以用它查看集群拓扑、手动模拟节点故障，以及在故障后观察系统如何自动完成副本提升和路由修正。实验三的重点正是把放置、通信、事务、一致性和恢复放进同一条游戏流程，观察分布式机制如何共同维持一个可继续运行的世界。

B.4 实验四：Kubernetes 上的部署与弹性运行

实验四把分布式文字游戏部署到自行配置的 Kubernetes 集群。与前三组不同，这个实验的重点不是再增加新的功能，而是把一个已经写好的分布式系统搬到云平台上，让读者在真实的容器编排环境中体会弹性伸缩、故障恢复和资源成本之间的取舍。

待部署的系统由一个对外提供 TCP 入口的 gateway、一个负责登录与全局协调的 coordinator、green、cave、ruins 三个地图服务以及保存共享状态的 Redis 组成。客户端请求从集群外进入 gateway，再经过 coordinator 到达相应地图服务；玩家资料、在线会话、地图检查点和服务租约等状态保存在 Redis 中。

实验的第一层任务是把系统跑起来。读者需要把不同程序制作成容器镜像，再用 Kubernetes 对象描述整套服务。五个业务组件分别使用 Deployment 管理副本，Redis 使用 StatefulSet 保存状态；Service 为外部入口和集群内部组件提供稳定访问地址。资源声明、健康检查和配置注入共同决定 Pod 能否被调度、何时可以接收流量，以及组件之间能否按预期连接。实验还涉及 ServiceAccount 与 RBAC，使业务组件能够在最小权限范围内读取或更新必要的集群信息。完成这一层后，整套服务能够在集群中正常运行，客户端可以通过固定地址接入游戏。

但”跑起来”只是实验的起点。实验真正的重点，是让读者在弹性、可用性和资源成本之间反复调优，观察不同配置带来的系统行为差异。gateway、coordinator 与三个地图服务都配置水平自动伸缩，根据负载增加或减少副本，而 Redis 作为状态中心不参与同样的水平伸缩。业务 Pod 在被缩容、滚动更新或手动删除时，需要先进入排空状态，通过就绪检查退出流量路径，并尽量完成状态保存和存量操作。系统还可以根据实例承载的活跃玩家数量调整 Pod 的删除代价，降低平台优先回收繁忙实例的概率。这些配置没有标准答案——CPU 目标值设得高，资源利用率好但扩容不及时；设得低，响应快但副本数容易虚高。读者需要自己权衡并验证。

配套的竞技评分从六个维度衡量部署质量：空闲时是否发生无意义扩容、低压力下是否保持稳定、高压力下能否及时增加容量、扩容和重试后状态是否一致、Pod 被删除后服务能否恢复，以及资源申请和副本数量是否合理。评分不按绝对性能排序，而是看系统在该有的时候有、不该有的时候不浪费。因此，实验四并不以”成功创建一组 Pod”为终点，而是要求读者用运行结果说明，自己的部署策略在正确性、可用性、弹性和资源成本之间做出了怎样的平衡，以及为什么这种平衡是合理的。